

GUILHERME ÁLVARO R. M. ESMERALDO

FUNDAMENTOS E PRÁTICAS EM
**ARQUITETURA E
ORGANIZAÇÃO DE
COMPUTADORES**

Estudos de Caso com o Simulador CompSim



VOLUME 1



FUNDAMENTOS E PRÁTICAS EM
ARQUITETURA E
ORGANIZAÇÃO DE
COMPUTADORES

Estudos de Caso com o Simulador CompSim

Volume 1

GUILHERME ÁLVARO R. M. ESMERALDO

**FUNDAMENTOS E PRÁTICAS EM
ARQUITETURA E
ORGANIZAÇÃO DE
COMPUTADORES**

Estudos de Caso com o Simulador CompSim

Volume 1

1a. Edição

© 2021 por Guilherme Álvaro Rodrigues Maia Esmeraldo.

Todos os direitos reservados.

Esta obra é publicada em acesso aberto. O conteúdo dos capítulos, os dados apresentados, bem como a revisão ortográfica e gramatical são de responsabilidade de seu autor, detentor de todos os Direitos Autorais, que permite o download e o compartilhamento, com a devida atribuição de crédito, mas sem que seja possível alterar a obra, de nenhuma forma, ou utilizá-la para fins comerciais.

Revisão, normalização e diagramação: Guilherme Álvaro Rodrigues Maia Esmeraldo

Conselho Editorial: *Me. Adriano Monteiro de Oliveira, Editor-chefe, Quipá Editora / Dra. Anny Kariny Feitosa, Instituto Federal do Ceará / Dr. Carlos Wagner Oliveira, Universidade Federal do Cariri / Me. Antoniele Silvana de Melo Souza, Secretaria de Educação de Pernambuco / Dra. Francione Charapa Alves, Universidade Federal do Cariri / Esp. Ricardo Damasceno de Oliveira, Universidade Regional do Cariri*

Dados Internacionais de Catalogação na Publicação (CIP)

E76f Esmeraldo, Guilherme Álvaro Rodrigues Maia
Fundamentos e práticas em arquitetura e organização de computadores /
Guilherme Álvaro Rodrigues Maia Esmeraldo. — Iguatu, CE : Quipá Editora, 2021.
418 p. : il.

ISBN 978-65-89973-63-8
DOI 10.36599/qped-ed1.106

1. Arquitetura de computador. 2. Organização de computador. I. Título.

CDD 004

Elaborada por Rosana de Vasconcelos Sousa — CRB-3/1409

Esta obra foi publicada pela Quipá Editora em novembro de 2021.

www.quipaeditora.com.br / @quipaeditora

PREFÁCIO

“Você já se perguntou como funciona aquela caixa chamada computador?”

Se sua resposta foi "Sim!" para a pergunta anterior, então convido você a descobrir essa fascinante aventura chamada "Arquitetura e Organização de Computadores" (AOC). Como todo bom aventureiro, é necessário preparação para que a sua jornada seja bem sucedida. Logo, um bom guia pode nos orientar em cada fase desse processo: seja fornecendo um bom planejamento do percurso e das ferramentas necessárias; trazendo riqueza nos detalhes de cada história e seus marcos; ou, até mesmo, mostrando os "caminhos das pedras", ou seja, os caminhos seguros que vão nos ajudar a completar nossa aventura com sucesso.

Assim, o nosso guia, Professor Guilherme Esmeraldo, preparou cada detalhe com toda atenção, cuidado e profissionalismo. O cara é tão bom que desenvolveu um simulador de computadores, chamado CompSim, que servirá como um mapa/bússola para desbravarmos nossa aventura. Assim, o objetivo desse livro/manual/guia de aventuras não é competir com a literatura erudita atual, mas oferecer algo diferente: tornar o processo de aprendizado de AOC uma experiência inesquecível.

Para isso, ele organizou seu livro/manual/guia de aventuras da seguinte forma: na introdução, ele apresenta o conceito de computador e a diferença entre os conceitos de arquitetura e de organização de computadores. Ainda nesse capítulo ele descreve o simulador CompSim (nossa bússola) e como ele pode ser integrado ao processo de ensino-aprendizagem em AOC, como ambiente de simulação para práticas laboratoriais. O capítulo seguinte é para quem gosta de história! Nele, são apresentados os artefatos de contagem mais antigos e sua evolução até os computadores mais modernos. Para no capítulo seguinte, mostrar um sumário das principais tendências tecnológicas que parecem ficção científica. Olha que algumas já são realidade!

A segunda parte do livro/manual/guia de aventuras trata da anatomia do computador; ou seja, ele disseca o Processador, Memórias RAM e Cache, Barramento e Subsistema de Entrada/Saída. Tudo isso apoiado com exemplos de utilização do CompSim para ilustrar cada uma dessas características. Ou seja, os demais capítulos ilustram com riqueza de detalhes cada processo e as estruturas necessárias para que todos esses componentes trabalhem como uma orquestra. Permitindo que aquele código em linguagem de programação, se transforme em bits zeros e uns, para serem executados em nível de hardware. Completando assim a nossa jornada no incrível mundo da "Arquitetura e Organização de Computadores".

Por fim, e não menos importante, falar do profissional e do incrível ser humano chamado Guilherme Esmeraldo, nosso guia nessa aventura, é uma tarefa simples. Nesses mais de 11 anos em que tenho o privilégio de compartilhar sua amizade, cada conversa, projeto ou conquista compartilhada têm promovido um enriquecimento em conhecimento e experiências que vou levar por toda a vida. Parabéns meu amigo Guilherme e muito obrigado por compartilhar um pouco das suas experiências nesse tão enriquecedor livro/manual/guia de aventuras. Com os votos de todo sucesso, um abraço do seu amigo:”

Professor Robson Gonçalves Fachine Feitosa

Instituto Federal do Ceará *campus* Crato

APRESENTAÇÃO

Ao longo dos anos, ministrando aulas da disciplina de Arquitetura e Organização de Computadores (AOC), no curso de Bacharelado em Sistemas de Informação do Instituto Federal do Ceará *campus* Crato, sempre busquei aperfeiçoar a abordagem metodológica de ensino-aprendizagem empregada. A cada nova turma, buscava um maior aprofundamento teórico, porém sem sucesso, devido à extensão e complexidade dos conteúdos da disciplina.

No princípio, montar um laboratório de hardware especializado para incluir práticas laboratoriais no programa da disciplina, parecia uma solução promissora. Porém, devido aos altos custos para aquisição dos materiais necessários, logo deixou de ser uma opção. O passo seguinte consistiu em buscar apoio nos simuladores computacionais da literatura científica especializada, já que, além de ser uma prática consolidada, poderiam ser instalados e utilizados em laboratórios de informática convencionais.

Com o passar dos semestres letivos, percebi que, na medida em que experimentava diferentes simuladores, os rendimentos das turmas permaneciam insatisfatórios, pois além dos requisitos teórico-práticos da disciplina, necessitava-se aprimorar a curva de aprendizado no uso do(s) simulador(es). É importante ressaltar que, nesse período, já havia uma grande diversidade de simuladores computacionais na literatura, porém cada um com suas próprias características. Os simuladores, em geral, buscam enfatizar um aspecto ou conjunto de aspectos específicos do computador. E, em algum momento da abordagem metodológica, tive a necessidade de incluir o uso de dois simuladores distintos por uma mesma turma de AOC. O resultado foi desastroso.

Após uma longa análise comparativa de simuladores do estado da arte e dos resultados das turmas que utilizaram os simuladores, no início de 2017, decidi iniciar o projeto de um novo simulador computacional: “CompSim - The Computer Simulator”. O CompSim surgiu com uma proposta semelhante à dos demais simuladores, que é dar suporte ao aprendizado prático na disciplina de AOC. No entanto, partindo das experiências anteriores, de uso de diferentes simuladores, o desenvolvimento da nova ferramenta foi conduzido para contemplar as lacunas observadas na metodologia educacional.

Com a finalização da primeira versão do CompSim, logo em seguida, formou-se uma cooperação técnico-científica entre os Laboratórios de Estudos Avançados em Eletrônica do Instituto Federal de Sergipe *campus* Aracaju (LEA/IFS) e de Sistemas Embarcados e Distribuídos do Instituto Federal do Ceará *campus* Crato (LEDS/IFCE), na época, coordenados pelo Prof. Dr. Edson Barbosa Lisboa e por mim, respectivamente. A cooperação, que vem durando proficuamente até a publicação deste livro, objetiva aprimorar o simulador e o seu suporte educacional em AOC. Foram criadas novas versões do simulador, com novos recursos, e o CompSim evoluiu rapidamente. Publicamos artigos científicos com os resultados do projeto em diferentes mídias nacionais e internacionais, nas áreas de educação em computação, educação em arquitetura de computadores e arquitetura de computadores, tais como: Congresso sobre Tecnologias na Educação (Ctrl+E), Fórum de Educação em Engenharia da Computação (FEEC), Workshop sobre Educação em Computação (WEI), Simpósio Brasileiro de Informática na Educação (SBIE), Workshop sobre Educação em Arquitetura de Computadores (WEAC), Workshop de Iniciação Científica em Arquitetura de Computadores e Computação de Alto Desempenho (WSCAD-WIC), *Technologies Applied to Electronics Teaching Conference* (TAEE), Revista Tecnologias na Educação (TecEdu) e *International Journal of Computer Architecture Education* (IJCAE). A relação completa das publicações científicas do Projeto CompSim pode ser conferida no [Apêndice F - Publicações](#).

Este livro tem como objetivo congregiar todas as *expertises* obtidas, ao longo dos anos, desde os aspectos que motivaram a criação de um novo simulador computacional até os resultados do seu uso em aulas práticas, para oferecer uma abordagem metodológica que integra teoria e prática no processo de ensino-aprendizagem em Arquitetura e Organização de Computadores.

Materiais

Este livro tem como objetivo familiarizar o leitor com os princípios de projeto de sistemas computacionais. Naturalmente, como exposto anteriormente, uma abordagem puramente conceitual torna-se inadequada.

Para aplicação dos conceitos teóricos, neste livro, o simulador CompSim será utilizado como estudo de caso, por: 1) Incluir componentes de hardware simulável com as características dos respectivos componentes em computadores reais; 2) Prover uma abordagem unificada e flexível de aprendizagem e projeto de novos sistemas computacionais;

3) Incluir um ambiente gráfico integrado e completo para projeto, programação, simulação, visualização e análise de desempenho de sistemas computacionais; e 4) Permitir a sua integração com hardware físico, de forma a prover simulações com interação com componentes eletrônicos reais, tornando os processos de ensino-aprendizagem e de projetos de novos sistemas computacionais mais espontâneos, atrativos, dinâmicos, motivantes, desafiadores e efetivos.

No website¹ (COMPSIM, 2021a) do projeto CompSim, além ser possível realizar o *download* do simulador, há materiais didáticos auxiliares, tais como slides de aula, exemplos de códigos-fonte em *Assembly* e *link* para vídeo aulas, que podem ser utilizados livremente, e um *link* para a lista de discussão do projeto CompSim no *Google Groups*² (COMPSIM, 2021b). A lista de discussão é o canal oficial, onde pode-se contactar desenvolvedores e outros usuários do simulador CompSim, para tirar dúvidas, compartilhar experiências, sugerir novas *features*, relatar *bugs*, entre outros. A lista também é utilizada para informes sobre o lançamento de novas versões e divulgação de notícias.

Errata

Como se trata da primeira edição deste livro, não foram medidos esforços na sua revisão para encontrar e corrigir erros. No entanto, tenho ciência de que erros podem ter passado despercebidos e que o livro pode ser continuamente melhorado. Se você encontrar erros remanescentes, não exite em me contactar pelo e-mail: guilhermealvaro@ifce.edu.br.

Além do mais, suas impressões, resenhas, críticas e sugestões também são muito bem-vindas. Aguardo ansioso por elas. ;-)

¹ CompSim - The Computer Simulator. Disponível em: <<http://compsim.crato.ifce.edu.br/>>. Acesso em: 01 abr. 2021.

² Grupo de Discussão do Projeto CompSim. Disponível em: <<https://groups.google.com/g/compsim-the-computer-simulator>>. Acesso em: 01 abr. 2021.

AGRADECIMENTOS

Agradeço a todos os docentes e discentes envolvidos, direta ou indiretamente, na produção deste livro.

Em especial, gostaria de agradecer:

Ao Professor Edson Barbosa Lisboa, do Instituto Federal de Sergipe (IFS) *campus* Aracaju, por, desde o princípio, acreditar no potencial e pela parceria, ao longo de todos esses anos, no projeto “CompSim - The Computer Simulator”.

Ao Professor Robson Gonçalves Fachine Feitosa, do Instituto Federal do Ceará (IFCE) *campus* Crato, pelas palavras de motivação para a concretização desta obra, bem como dedicação e empenho na revisão técnica do conteúdo e do texto.

Aos egressos Cícero Samuel Rodrigues Mendes e Lucas Fontes Cartaxo, e ao discente Eduardo Carlos Pereira da Silva Proto, do curso de Bacharelado em Sistemas de Informação, do Instituto Federal do Ceará *campus* Crato, pela parceria, dedicação e esforços nos trabalhos do “CS Team Crato”, realizados no Laboratório de Sistemas Embarcados e Distribuídos, do Instituto Federal do Ceará *campus* Crato (LEDS/IFCE).

Ao meu amigo Cícero Samuel Rodrigues Mendes, que prontamente me auxiliou na diagramação, ambientação e *design* da capa deste livro. Mais uma vez, seu talento fez com que ideias pequenas e simples se tornassem em belas artes gráficas. Thanks, Mr. Mendes!

Aos estudantes das turmas de Arquitetura e Organização de Computadores (AOC), do IFCE Crato, e do Curso Técnico em Eletrônica do IFS Aracaju, pois, sem vocês, não

haveria a motivação necessária para a criação de mais um simulador computacional (CompSim), de uma abordagem teórico-prática integrada para ensino-aprendizagem em AOC e de mais um livro sobre este tema.

Aos Institutos Federais do Ceará e de Sergipe, bem como à Fundação Cearense de Apoio ao Desenvolvimento Científico e Tecnológico (FUNCAP) e ao Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq) pelo fomento às pesquisas, através da concessão de bolsas de iniciação científica, para a participação de estudantes no projeto CompSim, e de recursos financeiros, para a aquisição de componentes eletrônicos.

E, acima de tudo, a Deus, pelo dom da vida, saúde e oportunidades, que permitiram realizar esta obra.

SOBRE O AUTOR

O autor possui graduação em Ciência da Computação pela Universidade Federal da Paraíba (UFPB), em 2005, e licenciatura em Matemática pela Universidade Regional do Cariri (URCA), em 2016, mestrado, em 2007, e doutorado, em 2012, em Ciência da Computação pela Universidade Federal de Pernambuco (UFPE). Atualmente, está realizando estágio Pós-Doutoral no Programa de Pós-Graduação em Ciência da Computação do Centro de Informática da Universidade Federal de Pernambuco (CIn/UFPE), na área de projetos de sistemas embarcados. É professor do Instituto Federal de Educação, Ciência e Tecnologia do Ceará *campus* Crato, onde ministra as disciplinas de Arquitetura e Organização de Computadores, Sistemas Operacionais e Sistemas Distribuídos, no curso de Bacharelado em Sistemas de Informação. Na pesquisa, conduz projetos nas áreas de educação e projetos de sistemas computacionais embarcados, tecnologias educacionais, estatística computacional e inteligência artificial.

*Dedico a minha esposa Cristiana, à minha filha
Maria Caroline e aos meus pais, Dilza e José.*

Guilherme Álvaro R. M. Esmeraldo

SUMÁRIO

PREFÁCIO

APRESENTAÇÃO

AGRADECIMENTOS

SOBRE O AUTOR

PARTE I - CONCEITOS INTRODUTÓRIOS **18**

CAPÍTULO 1 - INTRODUÇÃO **19**

- 1.1 Arquitetura e Organização de Computadores 21
- 1.2 A Disciplina de Arquitetura e Organização de Computadores 23
- 1.3 Os Simuladores Computacionais 25
- 1.4 O Simulador CompSim 26
- 1.5 Uma Proposta de Abordagem de Ensino-Aprendizagem por Uso do Simulador CompSim 30
- 1.6 Como o Livro está Organizado 33

CAPÍTULO 2 – HISTÓRIA DO COMPUTADOR **36**

- 2.1 Das Varas de Contagem até as Calculadoras Mecânicas 36
- 2.2 O Nascimento do Computador 38
- 2.3 A Máquina de Turing 40
- 2.4 A Lógica e sua Relação com os Computadores 41
- 2.5 Os Computadores Eletromecânicos 43
- 2.6 A Primeira Geração de Computadores Eletrônicos 47
- 2.7 A Segunda Geração de Computadores Eletrônicos 50
- 2.8 A Terceira Geração de Computadores Eletrônicos 55
- 2.9 Resumo 59

CAPÍTULO 3 – TENDÊNCIAS TECNOLÓGICAS **60**

- 3.1 Unidade de Processamento Gráfico de Propósito Geral 62
- 3.2 Unidade de Processamento Acelerado 62
- 3.3 Multiprocessador de Sinais Digitais 63
- 3.4 Matriz de Portas Programáveis 63
- 3.5 Unidade de Processamento de Visão Computacional 64
- 3.6 Acelerador de Inteligência Artificial 64
- 3.7 Matriz de Processadores Massivamente Paralelos 65

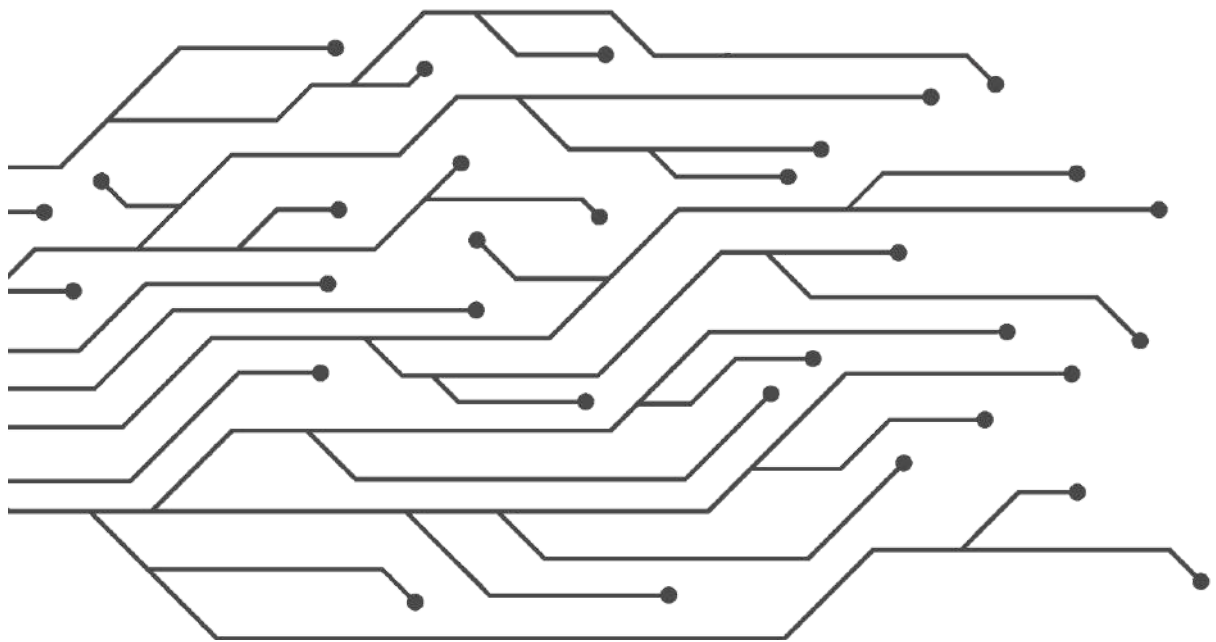
3.8 Processador de Internet das Coisas	65
3.9 Computador Óptico ou Computador Fotônico	66
3.10 Unidades de Processamento Quântico	67
3.11 Computadores Biológicos ou Computadores Genéticos ou Computadores de DNA	68
3.12 Uma Breve Análise da História do Computador	69
3.13 Resumo	70
PARTE II - CONCEITOS EM ORGANIZAÇÃO E ARQUITETURA DE COMPUTADORES	71
CAPÍTULO 4 – INTRODUÇÃO À ORGANIZAÇÃO DE COMPUTADORES	72
4.1 Processador	72
4.1.1 Processador Cariri	75
4.2 Memória Principal (RAM)	78
4.3 Memória Cache	80
4.4 Barramento	89
4.5 Subsistema de Entrada/Saída	93
4.6 Resumo	97
CAPÍTULO 5 – INTRODUÇÃO À ARQUITETURA DE COMPUTADORES	98
5.1 Ciclo de Instrução	99
5.1.1 Ciclo de Instrução do Processador Cariri	101
5.2 Instruções de Máquina	104
5.2.1 Instruções de Máquina do Processador Cariri	111
5.3 Linguagem de Montagem	113
5.4 Resumo	115
PARTE III - ASPECTOS PRÁTICOS EM ARQUITETURA DE COMPUTADORES	117
CAPÍTULO 6 – INSTALAÇÃO DO SIMULADOR COMPSIM	118
6.1. Website do Projeto CompSim	118
6.2 Download do Simulador	120
6.3 Instalação do CompSim	122
6.4 Resumo	124
CAPÍTULO 7 – SIMULAÇÃO COM COMPSIM	125
7.1 Criação de Uma Plataforma Simulável	125
7.2 Criação de Uma Aplicação	129
7.3 Mapeamento de uma Aplicação para a Plataforma de Hardware Virtual	132
7.4 Configuração de Simulação e Execução da Simulação	133
7.5 Visualização	134
7.6 Análise de Desempenho	138
7.7 Resumo	140
CAPÍTULO 8 – MANIPULAÇÃO DE DADOS EM MEMÓRIA	141
8.1 Abstração da Memória RAM	141
8.2 Modelo de Memória de um Programa	143
8.3 Suporte do Processador Cariri para Segmentação da Memória de um Programa	144

8.4 Estrutura Básica de um Programa em Assembly no CompSim	145
8.5 Criação de Variáveis e Estruturas de Dados em Memória	147
8.5.1 Definição de Variáveis e Estruturas de Dados	147
8.5.2 Declaração de Variáveis e Estruturas de Dados	152
8.5.3 Alocação de Espaços Livres de Memória	154
8.5.4 Exemplo Prático de Definição e Declaração de Variáveis e Arrays/Vetores	154
8.6 Acesso aos Dados Alocados em Memória	157
8.7 Resumo	161
CAPÍTULO 9 – PROCESSAMENTO DE DADOS	163
9.1 Operações Aritméticas	163
9.2 Operações Lógicas	168
9.2.1 Instrução NAND	168
9.2.1.1 Operação Lógica de Negação	170
9.2.1.2 Operação Lógica de Conjunção	172
9.2.1.3 Operação Lógica de Disjunção	174
9.2.2 Instrução SHIFT	176
9.3 Composição de Operações mais Complexas	179
9.3.1 Multiplicação Inteira	180
9.3.2 Serialização de Bits Paralelos	181
9.3.3 Paralelização de Bits Seriais	183
9.4 Resumo	185
CAPÍTULO 10 – CONTROLE DE FLUXO DE EXECUÇÃO DO PROGRAMA	186
10.1 Instruções de Transferência de Controle	186
10.2 Estruturas de Controle de Fluxo de Linguagens de Alto Nível	190
10.2.1 Estrutura Condicional If..Then	191
10.2.2 Estrutura Condicional If..Else	192
10.2.3 Estrutura Condicional Switch..Case	194
10.2.4 Estrutura de Repetição For...Do	196
10.2.5 Estrutura de Repetição While..Do	197
10.2.6 Estrutura de Repetição Do..While	199
10.3 Exemplos Práticos	200
10.3.1 Exemplo 1: Multiplicação Inteira	200
10.3.2 Exemplo 2: Divisão Inteira	201
10.3.3 Exemplo 3: Elemento da Sequência de Fibonacci	203
10.4 Resumo	205
CAPÍTULO 11 – MÉTODOS AVANÇADOS DE ACESSO À MEMÓRIA	207
11.1 Ponteiros	207
11.2 Operações em Estruturas de Dados e Strings	213
11.3 Pilha do Programa	219
11.4 Alocação Dinâmica de Memória	226
11.5 Relocação de Memória	229
11.6 Modos de Endereçamento do Processador Cariri	234
11.7 Resumo	236

CAPÍTULO 12 – MODULARIZAÇÃO DE PROGRAMAS	237
12.1 Definição e Chamada de Funções	237
12.2 Funções com Parâmetros de Entrada e Retorno de Resultado	242
12.2.1 Variáveis Globais	244
12.2.2 Registradores	246
12.2.3 Pilha do Programa	248
12.3 Variáveis Locais	251
12.4 Múltiplas Chamadas de Funções	256
12.5 Funções Recursivas	258
12.6 Resumo	263
CAPÍTULO 13 – ENTRADA/SAÍDA	264
13.1 Instalação de Periféricos no CompSim	264
13.2 Subsistema de Entrada/Saída	268
13.3 Tipos de Periféricos	269
13.4 Operações de Entrada/Saída	271
13.5 Entrada/Saída com Video e Keyboard	272
13.5.1 Video	272
13.5.2 Keyboard	276
13.6 Entrada/Saída com Arduino UNO	279
13.6.1 Interface de Integração entre o CompSim e uma board do Arduino UNO	281
13.6.2 Operação de Escrita com Saída Digital	284
13.6.3 Operação de Leitura com Entrada Digital	287
13.6.4 Operação de Escrita com Saída Analógica	289
13.6.5 Operação de Leitura com Entrada Analógica	292
13.7 Entrada/Saída com VArduino	294
13.8 Resumo	301
PARTE IV - TÓPICOS AVANÇADOS	302
CAPÍTULO 14 – OTIMIZAÇÃO DE DESEMPENHO	303
14.1 Paralelismo	303
14.2 Aumentando o Desempenho da Memória Cache	307
14.3 Boas Práticas de Programação	310
14.3.1 Redução de Esforço	311
14.3.2 Utilizando Registradores Eficientemente	313
14.3.3 Otimizando Estruturas de Repetição	314
14.3.4 Remoção de Recursão	317
14.3.5 Outros Tipos de Otimizações	321
14.4 Resumo	321
CAPÍTULO 15 – DESENVOLVIMENTO DE PERIFÉRICOS PARA O SIMULADOR COMPSIM	323
15.1 Padrão de Interface de Periféricos	323
15.2 Assistente de Criação de Interface de Periféricos	330
15.3 Exemplos de Criação de Periféricos	333

15.3.1 Exemplo de Periférico Virtual: Visor Numérico	334
15.3.2 Exemplo de Periférico Físico: Led Serial	338
15.4 Resumo	346
REFERÊNCIAS BIBLIOGRÁFICAS	348
APÊNDICES	356
APÊNDICE A – INSTRUÇÕES DO PROCESSADOR CARIRI	357
APÊNDICE B – INTRODUÇÃO À ARITMÉTICA COMPUTACIONAL	361
B.1 Números Inteiros	361
B.1.1 Bases Inteiras	361
B.1.2 Conversão entre Bases Inteiras	362
B.1.2.1 Método da Expansão de Base	362
B.1.2.2 Método da Divisão	364
B.1.2.3 Método da Tabela de Conversão	365
B.2 Representação de Números Inteiros pelo Computador	368
B.2.1 Notação Sinal Magnitude	369
B.2.2 Notação Complemento a 1	370
B.2.3 Notação Complemento a 2	371
B.3 Representação de Números de Ponto Flutuante pelo Computador	372
APÊNDICE C – INTRODUÇÃO À LÓGICA BOOLEANA E CIRCUITOS LÓGICOS	378
C.1 Operações Booleanas	378
C.2 Funções Booleanas	379
C.3 Propriedades da Álgebra Booleana	381
C.4 Conceitos de Circuitos Lógicos	383
APÊNDICE D - TABELA ASCII	388
APÊNDICE E – FERRAMENTAS DO SIMULADOR COMPSIM	390
E.1 Code Helper	390
E.2 CPU Datasheet	393
E.3 ASCII Table	394
E.4. Converter	395
E.5 Editor de Preferências Gráficas	396
E.6 Emulador de Sistema para Arduino UNO	399
APÊNDICE F – PUBLICAÇÕES	403
F.1 Artigos Completos Publicados em Periódicos	403
F.2 Capítulos de Livros	404
F.3 Artigos Completos Publicados em Anais de Congressos	405
F.4 Resumos Expandidos Publicados em Anais de Congressos	408
F.5 Resumos Publicados em Anais de Congressos	409
TERMOS PARA BUSCA	412

PARTE I - CONCEITOS INTRODUTÓRIOS



CAPÍTULO 1 - INTRODUÇÃO

Antes de iniciar os estudos em Arquitetura e Organização de Computadores, é importante compreender que o computador é um sistema bastante complexo. Sabe-se que é composto por vários componentes e que, dependendo do componente, pode possuir bilhões de componentes eletrônicos menores e elementares.

O computador pode ser definido e estudado de diferentes maneiras, por exemplo: 1) como uma estrutura de subsistemas, componentes, suas hierarquias e funções; e 2) uma organização estruturada em camadas.

Na primeira abordagem, Stallings (2010) cita que o computador é um sistema eletrônico que possui uma estrutura com subsistemas interrelacionados, e que cada subsistema também pode ser subdividido em novos subsistemas, formando uma estrutura hierárquica. Dessa forma, pode-se estudar e projetar cada subsistema independentemente e em momentos distintos. Em cada nível da hierarquia, deve-se lidar com dois aspectos: função e estrutura.

Do ponto de vista funcional, o computador desempenha quatro macro funções básicas:

1. Processamento de dados: envolve os requisitos e métodos para processamento de diferentes tipos de dados;
2. Armazenamento de dados: trata do armazenamento temporário de dados, em casos em que os dados são inseridos para processamento imediato, e armazenamento persistente de dados, para posterior recuperação e atualização;
3. Transferência de dados: refere-se aos processos de interação com o computador. Nesses processos estão envolvidos os dispositivos de entrada e saída de dados, também conhecidos como Periféricos, diretamente conectados ao computador; e
4. Controle: permite o gerenciamento dos recursos do computador bem como coordena o desempenho das partes funcionais do computador, em resposta à execução das instruções dos programas.

Capítulo 1 - Introdução

A estrutura trata dos componentes do computador e abrange quatro componentes principais:

1. Unidade central de processamento (*Central Processing Unit* - CPU): gerencia a operação do computador e realiza as funções de processamento de dados. Em geral, este componente é referenciado como Processador;
2. Memória principal: responsável pelo armazenamento de dados e instruções dos programas do computador;
3. Entrada e Saída: permite a transferência de dados entre o computador e o ambiente externo; e
4. Interconexões de sistema: consiste na infraestrutura de comunicação que fornece os meios de transferência de dados entre processador, memória e periféricos.

Esses componentes podem ainda ser subdivididos em componentes ou subsistemas menores, que possuem estruturas e funções próprias.

Também muito aceita na literatura, Tanenbaum e Austin (2013) apresentam uma segunda abordagem para estudo do computador, nomeada por eles como “Organização Estruturada de Computadores”. A abordagem considera que, em cada camada, há uma linguagem específica para a programação do computador. Nesse contexto, entende-se que a linguagem de uma determinada camada depende da linguagem da sua camada antecessora. Desta forma, um programador que cria um programa de computador utilizando a linguagem de uma determinada camada, não precisa se preocupar com tradução para linguagens das camadas subjacentes. Segundo os autores, os computadores contemporâneos podem conter até seis camadas, que são:

- Camada 5 - Nível de linguagem orientada a problemas: este nível inclui as linguagens de programação de alto nível, tais como C/C++ e Python, projetadas para serem utilizadas por programadores de aplicações. Programas escritos nessas linguagens geralmente são traduzidos (compilação) ou apenas interpretados pela linguagem da camada subjacente;
- Camada 4 - Nível de linguagem montagem (*Assembly*): a linguagem de montagem, ou linguagem de baixo nível, como veremos ao longo deste livro, consiste de um conjunto de símbolos para representação das instruções de um programa em linguagem das Camadas 3 ou 2;
- Camada 3 - Nível de máquina de sistema operacional: esta camada é considerada híbrida, pois a maior parte das suas instruções está na Camada 2, porém inclui, além de suas próprias instruções, organizações de memória diferentes, suporte à execução

de dois ou mais programas simultaneamente, entre outros recursos. As facilidades oferecidas na Camada 3 devem-se a um interpretador executando na Camada 2, o qual historicamente é conhecido como Sistema Operacional. É importante destacar que a partir da próxima camada, as linguagens são todas numéricas e, portanto, incompreensíveis pelos programadores;

- Camada 2 - Nível de arquitetura do conjunto de instrução: também conhecida como Conjunto de Instruções da Arquitetura (*Instruction Set Architecture* - ISA) e inclui as instruções do processador, executadas de modo interpretado pelo microprograma, contido na camada inferior;
- Camada 1 - Nível de microarquitetura: este nível envolve o caminho de dados, que consiste de um conjunto de memórias temporárias, chamadas Registradores, e uma unidade de processamento de dados, chamada de Unidade Lógica e Aritmética (ULA), que estão conectados para realização de operações de processamento de dados; e
- Camada 0 - Nível lógico digital: neste nível, os objetos de interesse são componentes analógicos, tais como transistores, que são modelados com precisão, através de tecnologias avançadas de manufatura de circuitos integrados, para compor as portas lógicas. A combinação (programação) das portas lógicas permite formar componentes mais complexos, tais como memórias e unidades de processamento de dados.

Analisando as duas abordagens apresentadas, pode-se concluir que, de uma forma geral, um computador é um sistema eletrônico complexo e programável, que realiza principalmente as funções de processamento, armazenamento e transferência de dados, e controle de execução. Para tratar essa complexidade, no projeto de um computador, sua estrutura é frequentemente subdividida em módulos, os quais possuem características e funções próprias, e que devem se comunicar para que, desta maneira, seja possível compor as funções do computador.

1.1 Arquitetura e Organização de Computadores

Arquitetura e Organização são conceitos distintos relacionados ao estudo e ao projeto de sistemas computacionais (neste livro, o termo “Sistema Computacional” trata não apenas de computadores, mas de qualquer dispositivo que realiza algum tipo de computação, tais

como calculadoras, *tablets*, *smartphones*, *smartwatches*, sistemas embarcados, servidores de grande porte, *data centers*, entre outros).

Segundo Tanenbaum e Austin (2013), na prática, a arquitetura e a organização de computadores são basicamente a mesma coisa. Stallings (2010) concorda afirmando que é difícil definir e distinguir com precisão os termos arquitetura e organização de computadores, no entanto, cita que há um consenso geral sobre as áreas tratadas em cada um desses conceitos.

Arquitetura de computadores trata dos aspectos visíveis ao programador, ou, em outras palavras, dos aspectos que tratam da execução lógica dos programas de computador. Segundo Stallings (2010), um termo frequentemente presente no estudo de arquitetura de computadores é o conjunto de instruções da arquitetura (ISA), que trata do conjunto das instruções do processador, os tamanhos e os formatos das instruções, tipos de operandos, os componentes envolvidos na execução das instruções, tais como registradores e memória, os modos de acesso aos dados do programa e os algoritmos que fazem uso das instruções.

Já a Organização de computadores trata dos componentes do computador, suas estruturas e submódulos, suas funções e as formas de comunicação entre si. Em resumo, trata dos aspectos transparentes aos programadores do computador, tais como sinais de controle, interfaces de comunicação entre o computador e os periféricos, bem como as tecnologias utilizadas em seus componentes (STALLINGS, 2010).

Atualmente, é possível observar que os fabricantes de computadores oferecem computadores com a mesma arquitetura para manter a compatibilidade de software, porém com organizações diferentes, caracterizando assim diferentes modelos de uma família de computadores, como são os casos das plataformas processadores i3, i5, i7 e i9 da Intel.

Além disso, é possível observar sensíveis variações de arquiteturas e organizações entre fabricantes distintos, como são os casos dos processadores da Intel e AMD, cujos programas podem ser compatíveis e executados em ambos. Porém, um programa compilado especificamente para uma dessas arquiteturas, pode fazer uso de instruções nativas da arquitetura Intel ou da AMD, o que faz com que se perca a compatibilidade entre ambas ao custo de obter um maior desempenho por usufruir dos recursos específicos da arquitetura alvo.

Há ainda os projetos em que as arquiteturas e organizações são totalmente distintas, de forma que não há compatibilidade entre elas, como são os casos das arquiteturas x86, que utiliza a abordagem *Complex Instruction Set Computers* (CISC), e ARM, que segue a abordagem *Reduced Instruction Set Computer* (RISC). Computadores CISC possuem

instruções mais complexas, que frequentemente realizam mais de uma operação em uma única instrução, e portanto necessitam ser subdivididas para serem executadas por microinstruções. Computadores RISC, por outro lado, possuem instruções simples e indivisíveis, não necessitando de microinstruções, e, por consequência, tornam-se mais eficientes por executarem diretamente no hardware. Dessa forma, é possível perceber que os programas criados para arquiteturas CISC possuem tamanhos menores, ao apresentarem menos instruções, e, por consequência, consomem menos recursos de memória. Por outro lado, os programas criados para arquitetura RISC são maiores, por conter mais instruções, porém executam de forma mais eficiente.

1.2 A Disciplina de Arquitetura e Organização de Computadores

Arquitetura e Organização de Computadores (AOC) é uma disciplina presente em cursos técnicos e superiores nas áreas de Ciência da Computação, Engenharia da Computação, Engenharia Elétrica, Engenharia Eletrônica, Mecatrônica, Automação Industrial, dentre outras (ACM; IEEE, 2013) (SBC, 2005). A disciplina inclui o estudo dos componentes do computador, das suas funções e dos modelos de comunicação, bem como dos aspectos visíveis ao programador (STALLINGS, 2010). Esses conhecimentos são fundamentais à operação, projeto, programação e otimização de desempenho de sistemas computacionais, além de estarem alinhados com as novas tendências tecnológicas, tais como: Internet das Coisas (IoT), Indústria 4.0, *Smart Cities*, Robótica, Computação de Alto Desempenho, Computação em Nuvem, Inteligência Artificial, entre outras.

Os conteúdos trabalhados na disciplina variam de acordo com o perfil de cada curso. Por exemplo, em cursos de graduação em Engenharia da Computação, há necessidade de um maior aprofundamento teórico-prático, o que já não ocorre em cursos de Sistemas de Informação. A ACM (*Association for Computing Machinery*) e o IEEE (*Institute of Electrical and Electronic Engineers*) (ACM; IEEE, 2013), bem como a Sociedade Brasileira de Computação (SBC) (SBC, 2005) apresentaram propostas de conteúdos curriculares para a disciplina de AOC em cursos de Computação. Dentre os tópicos, cinco deles são apontados como fundamentais para qualquer curso básico, que são:

1. Lógica e Sistemas Digitais: introdução e apresentação da história da arquitetura de computadores e introdução ao projeto de sistemas digitais;

2. Representação de dados em nível de máquina: representação de dados numéricos e não numéricos e de conjuntos de dados (registros e arrays) em nível de máquina;
3. Organização da máquina em nível de montagem: organização básica da máquina de von Neumann, organização básica do processador, ciclo de instrução e fundamentos de programação em *Assembly* (formatos de instrução, modos de endereçamento, subrotinas, entrada e saída e segmentos de código);
4. Organização e arquitetura de memória: tipos, organização e hierarquia de memórias e memória virtual;
5. Interfaceamento e comunicação: fundamentos de entrada e saída (*handshaking*, *buffering*, entrada e saída programada e orientada à interrupção) e barramentos (protocolos, mecanismos de arbitragem, acesso direto à memória e interrupções).

Além desses conteúdos, o currículo da ACM e IEEE (ACM; IEEE, 2013) inclui habilidades práticas que devem ser desenvolvidas pelos estudantes, com objetivo de promover o aprofundamento teórico e permitir que os estudantes possam explorar as características dos sistemas mais modernos (NIKOLIC et al., 2009). A ACM e IEEE definiram então seis habilidades práticas a serem desenvolvidas em cursos de AOC, que são:

1. Projeto de blocos básicos de um computador (Unidade Lógica e Aritmética - ULA, Unidade Central de Processamento - UCP, registradores e memória);
2. Uso de ferramentas CAD (*Computer-Aided Design*) para capturar, sintetizar e simular para avaliar a construção de blocos básicos;
3. Conversão de dados numéricos entre bases;
4. Escrita de programas simples ao nível de máquina (*Assembly*) para diferentes cenários de uso do computador;
5. Demonstração de como construções de linguagens de alto nível são implementadas em nível de máquina;
6. Cálculo dos tempos de acesso aos dados, considerando diferentes configurações de memória principal e cache, bem como diferentes tipos de referência a dados e instruções.

Percebe-se então que a organização de um curso de AOC torna-se uma atividade complexa, devido à necessidade de inclusão de uma quantidade significativa e diversificada de práticas laboratoriais em seu programa. Isso implicará em uma seleção mais refinada dos tópicos que serão cobertos e na alocação de horas/aula para as respectivas práticas e avaliações laboratoriais (NIKOLIC et al., 2009).

1.3 Os Simuladores Computacionais

O desenvolvimento das habilidades práticas pelos estudantes em AOC necessita de laboratórios de hardware especializados, com disponibilidade de componentes de hardware, ferramentas para manuseio de componentes eletrônicos, instrumentos de medição de diferentes grandezas eletrônicas, entre outros. Percebe-se assim que, compor e manter um laboratório desse porte vai exigir maiores recursos financeiros e não é uma tarefa simples, pois necessitará de um projeto estrutural bem elaborado, de apoio de técnico de laboratório para preparação e acompanhamento dos experimentos, de manutenção dos equipamentos e de reposição dos componentes eletrônicos.

O uso de simuladores computacionais, ou ambientes virtuais, como prática pedagógica complementar, não é uma atividade nova (BALAMURALITHARA; WOODS, 2009). Estudos, como os apresentados em (URIBE et al., 2016) (GARCIA; PACHECO; GARCIA, 2014) (BALAMURALITHARA; WOODS, 2009) (TAN et al, 2009), mostram que, ao se utilizar ambientes virtuais para fins educativos, foi possível aumentar o desempenho acadêmico em cursos de tecnologia e engenharia. Os simuladores são ferramentas importantes no processo de apropriação do conhecimento, pois possibilitam o desenvolvimento de habilidades e experiências práticas, de forma assíncrona, em cenários virtuais que se assemelham aos reais (WOLFFE et al., 2002). São também fundamentais para compor laboratórios específicos na ausência de infraestrutura (GARCIA; PACHECO; GARCIA, 2014) ou ainda quando há necessidade de se reduzir custos, realizar configurações rápidas e obter resultados instantâneos (URIBE et al., 2016). Os simuladores são particularmente úteis para representação ou abstração de cenários complexos (SRIVASTAVA; ATLURI, 2002) (BAHK et al., 2013) e frequentemente abordam os conteúdos presentes no estado da arte (WOLFFE et al., 2002).

Algumas ferramentas especializadas para projetos profissionais são adotadas ou adaptadas para compor ambientes virtuais de aprendizagem. Porém, apesar de oferecerem um ambiente completo de projeto, não é possível concluir se, nesses casos, o suporte educacional é efetivo (MAGANA; BROPHY; BODNER, 2012).

Desta maneira, considerar os aspectos didático-pedagógicos torna-se o grande desafio no projeto de qualquer ambiente de aprendizagem. Além destes, outros requisitos como, por exemplo, desempenho, facilidade de uso, interface intuitiva, portabilidade, granularidade, acessibilidade, interatividade e independência (URIBE et al., 2016) (BALAMURALITHARA; WOODS, 2009), são fatores importantes para sua aplicabilidade e

necessários para estimular e permitir que o estudante possa desenvolver seu próprio aprendizado.

Este livro apresenta uma abordagem integrada para o processo de ensino-aprendizagem em Arquitetura e Organização de Computadores, que considera os aspectos discutidos anteriormente, respaldada pelo apoio pedagógico de uma ferramenta de simulação. A ferramenta utilizada neste livro, chamada de CompSim (COMPSIM, 2021a), busca reunir as principais características dos simuladores do estado da arte, bem como o suporte necessário para o desenvolvimento das principais habilidades práticas, constantes nos currículos de referência em (ACM; IEEE, 2013) e (SBC, 2005).

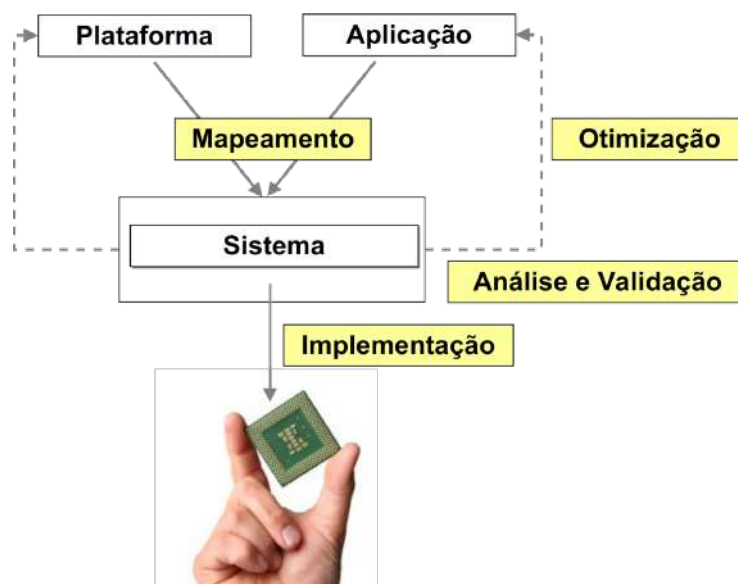
1.4 O Simulador CompSim

CompSim (COMPSIM, 2021a), que é um acrônimo para *Computer Simulator*, consiste de um ambiente virtual para apoio ao ensino-aprendizado em AOC e ao projeto de novos sistemas computacionais. A arquitetura de software do CompSim está dividida em três camadas, que são: 1) Plataforma de Hardware Virtual; 2) Interface Gráfica; e 3) Periféricos.

Na primeira camada, Plataforma de Hardware Virtual, o simulador CompSim inclui uma plataforma de hardware parametrizável e simulável, chamada “Mandacaru”. O conceito de plataforma segue a abordagem de Projeto Baseado em Plataformas (*Platform Based Design* - PBD) (DAVARE et al., 2007). Nesta abordagem, o sistema computacional a ser desenvolvido é, inicialmente, especificado através de uma descrição em alto nível, como descrições textuais e/ou diagramas. As funções do sistema, contidas nessa especificação, são selecionadas para serem implementadas em software (que será mapeado para execução em microprocessadores) ou em hardware (componentes de hardware de finalidade específica - *Application Specific Integrated Circuit* ou ASIC, ou implementados a partir de reuso de outros componentes, ou compilados a partir de ferramentas de síntese - *Field Programmable Gate Array* ou FPGA) (BARKALOV; TITARENKO; MAZURKIEWICZ, 2019). Estes componentes de hardware fazem parte de uma arquitetura predefinida, conhecida como “Plataforma”, que pode ser modificada de forma que sua estrutura possa ser adaptada às necessidades da aplicação. Essa modificação pode envolver substituição, inclusão e/ou configuração de componentes de hardware, bem como realizar ajustes na estrutura de comunicação. O diagrama na Figura 1.1 apresenta o gráfico “Y” (ou “Y-Chart”), que apresenta o fluxo básico de projetos baseados em plataformas.

De acordo com a Figura 1.1, inicialmente parte-se de uma plataforma preconcebida e uma aplicação. A aplicação é mapeada para a plataforma (Mapeamento), de forma que suas funções poderão ser implementadas em software ou em hardware, compondo assim o sistema computacional. Em seguida, o sistema é analisado de acordo com determinadas métricas consideradas como essenciais ao projeto (Análise). Após a análise, se necessário, o projetista do sistema computacional tem a liberdade para realizar ajustes (Otimização) na aplicação, na plataforma ou, até mesmo, no mapeamento entre ambas. Durante esses ajustes, o foco é explorar o espaço de opções de projeto em busca dos melhores resultados ou daqueles que atendam aos requisitos predefinidos no projeto (Validação). Após a validação, o sistema computacional segue então para sua implementação final.

Figura 1.1. Fluxo de projetos baseados em plataformas.

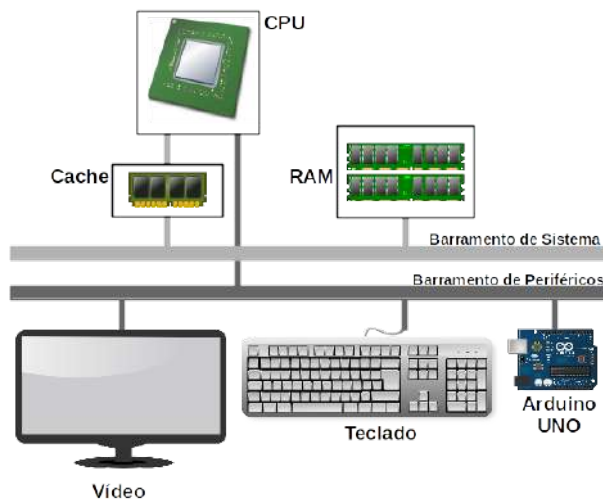


Fonte: Próprio autor (2021).

A plataforma Mandacaru possui componentes simuláveis (desenvolvidos em software) dos principais componentes do computador. É importante destacar que esses modelos possuem características dos respectivos componentes de hardware físico, de forma a oferecer cenários em que é possível simular ambientes reais. A plataforma Mandacaru possui os seguintes componentes: um modelo de Processador teórico (*Central Processing Unit* - CPU), chamado “Cariri”, criado para fins educacionais, mas que possui características dos principais processadores do estado da arte; uma Memória Principal (*Random Access Memory* – RAM); uma Memória Cache; dois Barramentos Compartilhados, sendo um deles de Sistema e o outro de Periféricos; e um Subsistema de Entrada/Saída, que permite conectar e

utilizar diferentes tipos de periféricos, sendo eles virtuais e/ou físicos. A Figura 1.2 ilustra a plataforma Mandacaru, onde visualiza-se a interligação dos componentes CPU, memórias Cache e RAM, Barramentos de Sistema e de Periféricos, e os periféricos Vídeo, Teclado e Arduino UNO.

Figura 1.2. Plataforma Mandacaru.



Fonte: Próprio autor (2021).

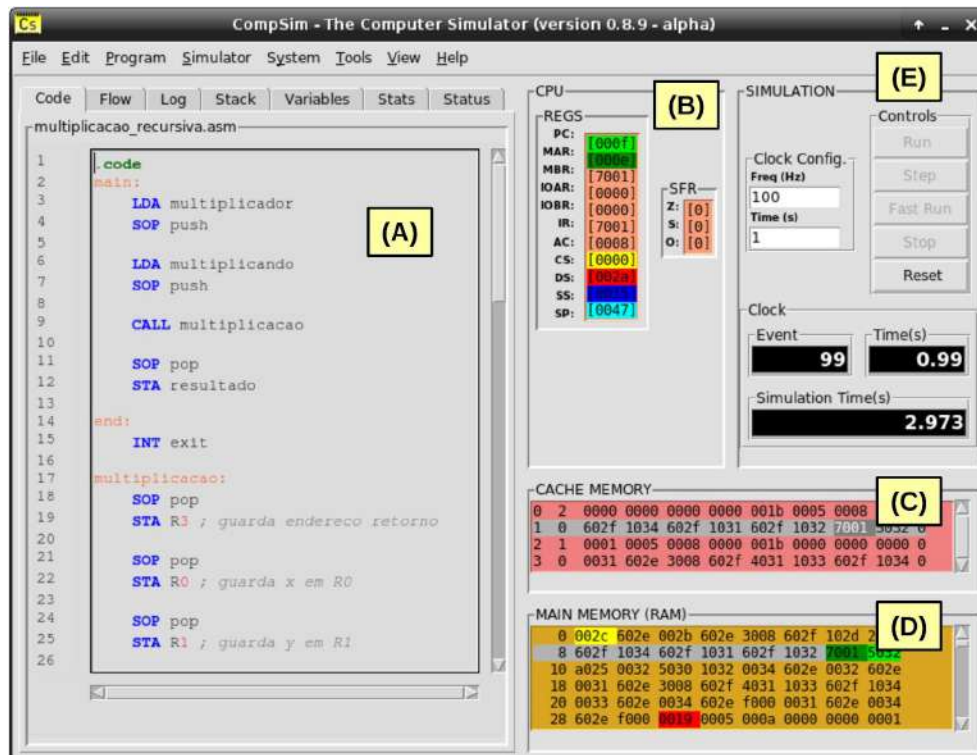
A segunda camada, Interface Gráfica, consiste de uma GUI (*Graphical User Interface*) que está integrada aos componentes da plataforma Mandacaru. Assim, com a GUI é possível: criar uma plataforma, configurá-la, programá-la em linguagem de baixo nível (linguagem de montagem *Assembly*) e simulá-la; visualizar os status dos componentes de hardware da plataforma e elementos do programa em execução, durante uma simulação; e realizar análises de desempenho do sistema computacional em desenvolvimento. A Figura 1.3 apresenta a interface gráfica do simulador CompSim.

Na Figura 1.3 é possível visualizar em:

- A. Editor de código: inclui recursos para simplificar a codificação de uma aplicação, como número de linhas, teclas de atalho para recursos de edição (copiar, recortar, colar, ir para determinada linha, etc.), um assistente de codificação (permite auxiliar a construção de instruções de código com a sintaxe correta), destaque de palavras-chave (*syntax highlight*), entre outros. Este componente está integrado a um montador (*Assembler*) que realiza análises léxica, sintática e semântica no código-fonte da aplicação, tradução deste para código de máquina (*bytecodes*), carregamento dos códigos gerados na memória RAM, além de produzir um relatório do programa, que

inclui a tabela de símbolos, endereços de memória dos segmentos e *bytecodes* gerados;

Figura 1.3. Interface gráfica do simulador CompSim.



Fonte: Próprio autor (2021).

- B. Processador: durante uma simulação, exibe os registradores do processador Cariri e respectivos valores assumidos. Os registradores de endereçamento possuem cores diferenciadas, onde as respectivas cores são utilizadas para indicar as diferentes posições referenciadas na memória RAM;
- C. Memória cache: exibe as linhas da cache e respectivos valores, destaca também as linhas e as palavras endereçadas pelo processador durante uma simulação;
- D. Memória RAM: exibe os conteúdos (instruções e dados) de todos os endereços do componente virtual memória RAM. Os componentes gráficos Memória RAM e Processador estão integrados, de forma que, durante uma simulação, as posições de memória são destacadas de acordo com as respectivas cores dos registradores de endereçamento; e
- E. Componentes de controle de configuração e execução de simulação: inclui controles que permitem configurar o tempo total de simulação e a frequência de relógio de

sistema (*clock*), iniciar, executar (em modos normal, passo a passo ou rápido), parar e reiniciar uma simulação.

A terceira camada, Periféricos, inclui os periféricos que podem ser conectados ao Subsistema de Entrada/Saída da plataforma Mandacaru. O Subsistema de Entrada/Saída permite que um novo periférico possa ser conectado automaticamente ao barramento de periféricos, tornando-o disponível para comunicação com os demais componentes da plataforma. Os periféricos suportados pelo Subsistema de Entrada/Saída podem ser do tipo virtual (implementado exclusivamente em software que emula um periférico real) e físico (possui uma parte em software, para conexão com o barramento de periféricos, e uma parte em hardware físico).

Com esses recursos, o simulador CompSim torna-se um ambiente integrado ideal para a aplicação prática dos conceitos aprendidos nas aulas teóricas de AOC, bem como permite o desenvolvimento de habilidades práticas, uma vez que evita a necessidade da disponibilidade de laboratórios de hardware especializados, e de toda a logística para sua criação e manutenção.

1.5 Uma Proposta de Abordagem de Ensino-Aprendizagem por Uso do Simulador CompSim

Na literatura, tem-se empregado diferentes abordagens metodológicas para otimizar o aprendizado dos conceitos da disciplina de AOC, como *Problem Based Learning*, Ensino baseado em Projetos, em Analogia, em Aplicação, entre outras (FERNANDES; SILVA, 2017). Em resumo, ao analisá-las, é possível concluir que, ao empregar situações-problema e atividades práticas, é possível favorecer o aprendizado em AOC. Além disso, as abordagens metodológicas devem possibilitar que o estudante entre em contato com hardware real, pois, segundo Black (2016), esse contato motiva e promove a busca por estudos avançados em AOC, que podem ser necessários no exercício da profissão.

Desta forma, apresenta-se aqui uma proposta de abordagem para uso do simulador CompSim como apoio à disciplina de AOC. Na abordagem proposta, o CompSim pode dar suporte ao estudo prático de um conjunto de conteúdos da disciplina, tais como: introdução à aritmética computacional, componentes do computador e suas funções, conjunto de instruções do processador, modos de endereçamento, programação em baixo nível, análise de desempenho e entrada e saída.

Da mesma forma, durante as aulas práticas, podem ainda ser propostos diferentes cenários para o desenvolvimento de sistemas computacionais com periféricos, em que os estudantes podem manusear componentes eletrônicos como resistores, capacitores, sensores, motores, leds, displays, chaves tácteis, botões de pressão, entre outros, para a criação de novos dispositivos integrados ao CompSim. Entre os cenários propostos estão: piscar leds (“arvore-de-natal”, semáforos e jogos), exibição de dados em displays (contadores numéricos, relógio e menu interativo), monitoramento (temperatura, umidade, luminosidade, proximidade e infravermelho), sinalizadores (alarme e código morse) e motores (hélices, carrinho e braço mecânico).

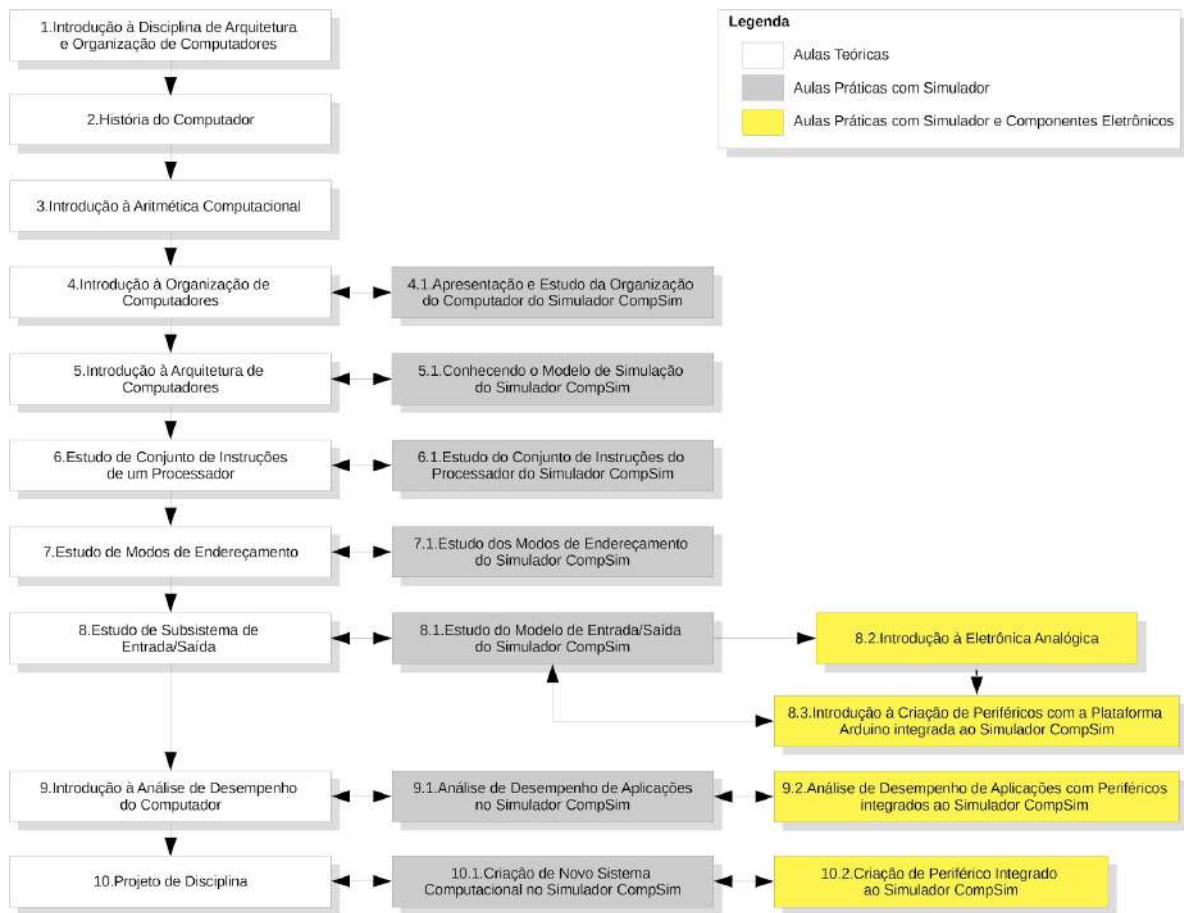
A Figura 1.4 ilustra, através de um fluxograma, as etapas da abordagem de ensino proposta neste livro. As etapas em cor branca estão relacionadas às aulas onde há exposição puramente teórica dos conteúdos da disciplina de AOC; as etapas em cor cinza estão relacionadas às aulas práticas em laboratório com suporte do simulador CompSim; e as etapas em cor amarelo abrangem as aulas práticas em laboratório com suporte do simulador CompSim e de componentes eletrônicos. Ressalta-se que, neste último grupo de atividades, deverão estar disponíveis para os estudantes, além do simulador, os periféricos virtuais e/ou físicos, sendo estes últimos compostos por kits com a plataforma Arduino e componentes eletrônicos, necessários à realização dos cenários propostos pelo professor da disciplina.

No fluxo ilustrado na Figura 1.4, o uso do simulador CompSim inicia durante os estudos de organização de computadores (Estágio 4). O estudo da organização de computadores inclui uma análise detalhada da estrutura, funções, parâmetros de configuração, interfaces de comunicação e interconexões de cada um dos componentes da plataforma de hardware virtual do CompSim.

De acordo com o fluxo proposto, as próximas etapas abrangem o estudo de arquitetura de computadores, realizado com programação em linguagem de baixo nível. Nessas etapas, o simulador CompSim é usado novamente como estudo de caso, onde o conjunto de instruções da arquitetura de seu processador, modos de endereços suportados e mecanismos de entrada/saída são estudados com mais detalhes, seguidos pela análise de desempenho de aplicativos computacionais (Estágios 6.1, 7.1, 8.1 e 9.1, respectivamente).

Observa-se que os Estágios 8.1 e 9.1 são complementados por projetos com periféricos, pois envolvem operações de entrada/saída. No Estágio 8.1, os estudantes aprendem a utilizar os periféricos virtuais e físicos do CompSim e podem aprender a criar novos periféricos. Já no Estágio 9.2, poderão aplicar os conceitos de otimização de desempenho em sistemas computacionais com periféricos.

Figura 1.4. Fluxograma da abordagem de ensino proposta.



Fonte: Próprio autor (2021).

Na última etapa do fluxo proposto (Estágio 10), os estudantes devem combinar os conhecimentos adquiridos, durante as aulas do curso, para construir novos conhecimentos. Nesse ponto, eles poderão utilizar o simulador CompSim para desenvolver um novo sistema computacional completo, que deve incluir uma solução de programação de baixo nível (Estágio 10.1) e pode incluir um periférico virtual e/ou físico (Estágio 10.2).

Considerando as diferentes ementas e objetivos, contextos locais, níveis de formação e cursos de computação/eletrônica, a disciplina de AOC pode sofrer variações quanto à extensão e verticalização de seus conteúdos. Nesse sentido, a abordagem aqui apresentada pode ser customizada para atender aos requisitos particulares de cada disciplina, de forma que: 1) novos conteúdos poderão ser adicionados e os existentes podem ser removidos; 2) a sequência dos conteúdos trabalhados pode ser alterada; 3) pode-se enfatizar determinados conteúdos em função de outros; e 4) diferentes números de aulas podem ser alocados para cada um dos estágios propostos.

1.6 Como o Livro está Organizado

Este livro traz uma abordagem teórico-prática integrada para apoiar o processo de ensino-aprendizado dos conceitos de Arquitetura e Organização de Computadores. Desta forma, este livro está organizado em quatro partes, descritas a seguir.

A primeira parte envolve os conceitos introdutórios e inclui três capítulos.

No [Capítulo 1](#), faz-se uma breve introdução ao conceito de computador e uma diferenciação entre os conceitos de arquitetura e de organização de computadores. Na sequência, apresenta-se as características da disciplina de AOC, e de suas necessidades pedagógicas, em que os simuladores computacionais surgiram com a proposta de dar suporte aos processos metodológicos, oferecendo cenários semelhantes aos reais para apoiar as práticas laboratoriais da disciplina. O capítulo traz ainda descrições do simulador CompSim e de uma proposta de abordagem metodológica integrada para ensino-aprendizagem em AOC, em que o CompSim é utilizado como estudo de caso e como ambiente de simulação para as práticas laboratoriais.

O [Capítulo 2](#) traz uma breve introdução à história do computador. São apresentados desde os artefatos de contagem mais antigos até os computadores mais modernos. Apresenta-se como se deu a evolução, através do surgimento de novas teorias e tecnologias, as quais fizeram surgir os computadores eletrônicos modernos e suas gerações.

O [Capítulo 3](#) traz uma análise da evolução do computador e um sumário das principais tendências tecnológicas, em que algumas já são realidade, e outras trazem promessas de continuidade da evolução.

A segunda parte também traz dois capítulos, os quais tratam dos conceitos em organização e arquitetura de computadores.

O [Capítulo 4](#), que trata de organização de computadores, apresenta e detalha as características dos principais componentes do computador, que são: Processador, Memórias RAM e Cache, Barramento e Subsistema de Entrada/Saída. Ainda nesse capítulo, já se inicia a abordagem metodológica integrada, onde os componentes de hardware de simulação do CompSim são utilizados para ilustrar as características dos componentes do computador.

O [Capítulo 5](#) foca nos aspectos de arquitetura de computadores, onde enfatiza-se os aspectos de projeto de um processador, como ciclo de instrução, formato das instruções, tipos de operandos, modos de endereçamento, entre outros.

Capítulo 1 - Introdução

A terceira parte inclui oito capítulos que lidam detalhadamente com os aspectos de arquitetura de computadores, sendo aplicados, com diversos exemplos práticos, diretamente no simulador CompSim.

O [Capítulo 6](#) mostra os procedimentos necessários para instalação do simulador CompSim.

O [Capítulo 7](#) traz detalhes dos principais recursos do CompSim e como realizar as primeiras simulações em seu ambiente virtual integrado.

O [Capítulo 8](#) inicia o estudo aplicado dos aspectos de arquitetura de computadores, onde foca na estrutura de um programa em memória, no suporte do processador para estruturação do programa em memória, apresenta a estrutura básica de um programa em linguagem de montagem (*Assembly*), como alocar variáveis e estruturas de dados em memória, bem como realizar acessos básicos aos dados alocados.

O [Capítulo 9](#) lida com processamento de dados. Nele são apresentadas as instruções aritméticas e lógicas do processador, como podem ser utilizadas para realização de operações simples e para compor operações mais complexas.

O [Capítulo 10](#) apresenta as instruções de transferência de controle do processador e como podem ser utilizadas para compor estruturas condicionais, tais como If..Else e Switch..Case, e de repetição, como While..Do e For..Do, presentes em linguagens de programação de alto nível. Ao fim do capítulo, as estruturas apresentadas são aplicadas para ilustrar a criação de aplicações mais complexas.

O [Capítulo 11](#) aborda novamente aspectos de acesso à memória. Porém, desta vez são tratados aspectos mais avançados, tais como a implementação de ponteiros, operações em estruturas de dados e cadeias de caracteres, utilização da pilha do programa, alocação dinâmica e relocação de memória. Ao fim do capítulo, faz-se um sumário dos modos de endereçamento suportados pelo processador do simulador CompSim.

O [Capítulo 12](#) trata de modularização para tratar da complexidade dos programas. O capítulo inicia apresentando como pode-se modularizar um programa através de funções. Em seguida, trata de funções com passagem de parâmetros e retorno de resultado, alocação de variáveis locais, múltiplas chamadas de funções e, encerra, com funções recursivas.

O [Capítulo 13](#) lida com diversos aspectos de entrada e saída de dados. Inicialmente, são apresentados os procedimentos para instalação de periféricos no simulador CompSim, que podem ser obtidos no website do projeto CompSim e podem ser dos tipos virtual e físico. O capítulo trata ainda do Subsistema de Entrada/Saída, que permite a integração dos periféricos com os demais componentes da plataforma de hardware virtual, dos recursos do

processador para a comunicação com periféricos e, na sequência do capítulo, são apresentadas diversas aplicações práticas que ilustram o uso de periféricos.

A última parte do livro inclui dois capítulos que tratam de tópicos avançados.

O [Capítulo 14](#) traz diferentes abordagens para análise e otimização de desempenho no projeto de sistemas computacionais. São tratados aspectos de paralelização ao nível de instrução e de tarefas, aumento de desempenho em sistemas com memória Cache, boas práticas de programação que favorecem o aumento do desempenho dos programas, entre outras.

Por fim, o [Capítulo 15](#) apresenta o processo de criação de novos periféricos e como eles podem ser utilizados no simulador CompSim. Apresenta-se, inicialmente, uma abordagem manual com criação de um periférico simples, como estudo de caso, e, em seguida, uma abordagem mais dinâmica, que faz uso de uma ferramenta que automatiza partes do processo de criação de novos periféricos. Ao fim do capítulo, são ilustrados os processos de criação de dois periféricos, sendo um deles do tipo virtual e o outro do tipo físico (que utiliza componentes eletrônicos reais).

CAPÍTULO 2 – HISTÓRIA DO COMPUTADOR

O contexto tecnológico atual – com dispositivos eletrônicos, brinquedos e eletrodomésticos inteligentes e interligados à Internet, computação em nuvem, robótica, carros autônomos, Indústria 4.0, entre outros – só foi possível graças ao surgimento do computador. Certamente, o computador é uma das maiores invenções da humanidade. Porém, o computador, como é conhecido hoje, nem sempre foi assim. Partindo de instrumentos arcaicos e com o surgimento de novas teorias matemáticas e avanços tecnológicos, ao longo dos anos, houve todo um processo histórico para sua evolução.

Este capítulo trata dos primeiros artefatos criados para contagem, os principais eventos que culminaram no seu aprimoramento até evoluírem ao computador em seu estágio atual.

2.1 Das Varas de Contagem até as Calculadoras Mecânicas

Os primeiros instrumentos, que se tem notícia, criados pelo homem e que foram utilizados para contagem, foram as Varas de Contagem (20.000 a.C.) e o Ábaco (2.700 a.C.). A vara de contagem era utilizada para registrar quantidades de animais em rebanhos, através de marcações (riscos) ao longo do seu corpo. Já o ábaco, permite, além da representação dos números, onde cada linha representa uma magnitude numérica e as esferas representam os números, realizar operações sobre eles: através do movimento das esferas é possível realizar somas e subtrações. A Figura 2.1 apresenta o “Osso de Ishango” (a), uma das mais antigas varas de contagem que se tem registro, e o ábaco (b).

Figura 2.1. Primeiros instrumentos de contagem.



(a) Osso de Ishango³.



(b) Ábaco⁴.

Fonte: Wikimedia (2021).

Com o surgimento e desenvolvimento de novas ciências, como a astronomia, engenharia e economia, a demanda por representações de números e cálculos mais complexos fez com que surgissem novas teorias matemáticas, como a criação do número zero, algarismos arábicos, logaritmos, automatização de cálculos (mais tarde sendo conhecido como “algoritmo”). Essas teorias e a criação de variados autômatos (dispositivos mecânicos movidos por animais, água e pelo próprio homem para automatizar tarefas), permitiram a criação e aperfeiçoamento de novos instrumentos, como as calculadoras mecânicas, entre os séculos XVII e XIX.

De uma forma geral, uma calculadora mecânica consistia de uma série de mecanismos mecânicos, como manivelas, engrenagens e cilindros, que permitiam a representação e configuração dos números e, na medida em que eram manipulados, as operações aritméticas eram realizadas.

Seguem das calculadoras mecânicas que mais se destacaram:

- Relógio Calculador (1623) : Desenvolvido por Wilhelm Schickard, o Relógio Calculador é considerado a primeira calculadora funcional do mundo;
- Pascalina (1642) : Desenvolvida por Blaise Pascal, a Pascalina era mais robusta e resolvia alguns problemas do Relógio Calculador, como o atrito gerado pelas engrenagens no processo de “vai-um” em operações aritméticas;
- Calculador de Leibniz (1694) : Criado por de Gottfried Leibniz, seu projeto aperfeiçoou e automatizou de fato as operações de multiplicação e divisão da Pascalina. Leibniz criou ainda, alguns anos antes (em 1679), o sistema de numeração

³ By Sideroff. Licença CC BY-SA 4.0, Disponível em:
<https://commons.wikimedia.org/wiki/File:Ishango_bone.png>.

⁴ By David R. Tribble. Licença CC BY-SA 3.0, Disponível em:
<https://upload.wikimedia.org/wikipedia/commons/2/28/Abacus_1.jpg>.

binário, justificando a necessidade de criação de um sistema numérico mais simples de entender e de realizar operações;

- Tear de Jacquard (1801) : Joseph Marie Charles dit Jacquard aperfeiçoou os teares automáticos, que eram mecanismos que realizavam tramas em fios para criar tecidos, com uma interface programável por cartões perfurados (pode-se dizer que esta foi a primeira interface para programação e de entrada de dados em grande quantidade).

A Figura 2.2 apresenta, em (a), a Pascalina, e, em (b), a interface programável por cartões perfurados do Tear de Jacquard.

Figura 2.2. Calculadoras mecânicas.



(a) Pascalina de Blaise Pascal⁵.



(b) Tear de Jacquard⁶.

Fonte: Wikimedia (2021).

2.2 O Nascimento do Computador

Alguns anos depois, Charles Babbage criou o projeto de duas máquinas que revolucionaram todas as calculadoras mecânicas da época. A primeira máquina, chamada de Máquina Diferencial (1821), seria capaz de calcular séries de polinômios, com ordens variadas, utilizando o método das diferenças (uma técnica que permite que se calcule uma sequência de valores assumidos, as raízes, por um determinado polinômio, utilizando apenas operações de soma). No entanto, Babbage, que havia conseguido recursos generosos do Governo Inglês, construiu apenas partes da máquina diferencial. A história conta que, ao

⁵ By David Monniaux. Licença CC BY-SA 3.0. Disponível em:

<https://upload.wikimedia.org/wikipedia/commons/8/80/Arts_et_Metiers_Pascaline_dsc03869.jpg>.

⁶ By GeorgeOnline. Licença CC BY-SA 3.0. Disponível em:

<https://upload.wikimedia.org/wikipedia/commons/c/ce/Jacquard_loom%2C_6_of_6.jpg>.

longo do percurso, ele ficou desmotivado, pois o projeto de sua segunda máquina, a Máquina Analítica, era muito mais promissor.

Ao contrário da anterior, a máquina analítica seria programável para computar qualquer função - surgia aí o conceito de “Computador de Propósito Geral” -, através de cartões perfurados (inspirado no tear de Jacquard). Em sua descrição, a nova máquina possuiria um conjunto de elementos registradores, capazes de representar um conjunto de números decimais, e uma unidade de processamento, chamada de “moinho”, capaz de realizar no mínimo as quatro operações aritméticas sobre os decimais. Para a programação, seria necessário informar, através de cartões perfurados a operação a ser realizada, em quais registradores estariam os operandos e em qual registrador seria armazenado o resultado (observa-se que esta estrutura de programação é semelhante à das linguagens das máquinas atuais - essa semelhança será vista ao longo deste livro). Essa técnica de solucionar um problema, solucionando partes dele por vez, chama-se “Análise” (daí o nome “Máquina Analítica”). Outro aspecto interessante é que, devido ao conjunto de registradores, capazes de armazenar diversos números, cunhou-se, nesta época, o termo “memória”. Assim como ocorreu com a máquina anterior, Babbage não conseguiu concluir a construção da máquina analítica, pois, após muitos anos de desenvolvimento e muitos recursos financeiros investidos, o governo britânico decidiu encerrar os investimentos nos projetos.

Curiosidade!

Ada Augusta Byron King - que ficou mais conhecida por “Ada Lovelace”, pois, pelo seu casamento com o barão Willian King, tempos depois recebeu o título de condessa de Lovelace -, a filha do Lord Byron, um conhecido poeta britânico, que é precursor da corrente literária conhecida por “Mal do Século”, era uma matemática amadora e se interessou muito cedo pelo segundo trabalho de Babbage. Ada Lovelace percebeu que a máquina analítica tinha um grande potencial e logo ela conseguiu imaginar programas diversos para a máquina analítica, além de estruturar diversas características presentes nas linguagens de programação modernas, como estruturas condicionais, laços de repetição, sub-rotinas, símbolos para representar valores (conceito de “variável”), comando de atribuição, entre outros. Dessa forma, por todas as suas contribuições, Ada Lovelace é reconhecida como a primeira programadora da história.

2.3 A Máquina de Turing

A partir das ideias de Charles Babbage, nas quais seria possível conceber um dispositivo mecânico capaz de computar qualquer função, muitos matemáticos buscaram formalizar matematicamente o que de fato poderia ser computável ou não.

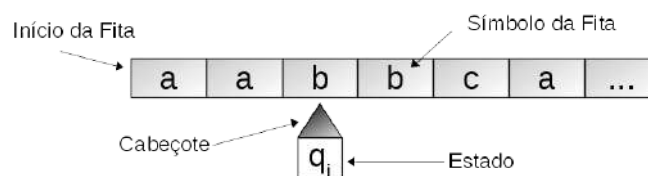
Alan Turing, um matemático inglês, publicou um artigo em 1937, no qual, finalmente, apresenta um modelo teórico que pôs um ponto final a essa questão.

Turing definiu matematicamente uma máquina teórica muito simples, que ele chamou de “Máquina A” (“A” de “Automática”), com as seguintes características:

- Possuía uma espécie de cabeçote capaz de ler e escrever símbolos;
- O conjunto de símbolos que podia ser lido e escrito deveria ser finito;
- Os símbolos deveriam ser lidos e escritos em uma fita, pelo cabeçote;
- A fita teria um início e não necessariamente um fim (considera-se que a fita teria um tamanho infinito);
- Possuía um conjunto de estados finitos, em que cada um deles caracterizava o estado da máquina em um determinado instante; e
- Possuía um conjunto de regras que permitiam coordenar o que a máquina deveria realizar a partir da leitura de um símbolo da fita e do seu estado naquele momento. As ações que a máquina teórica poderia realizar, basicamente, seriam: mover o cabeçote uma posição à frente ou uma posição para trás na fita, manter o cabeçote parado e/ou escrever um símbolo na fita.

A Figura 2.3 apresenta uma ilustração da Máquina A de Turing. Na figura, pode-se observar a fita, com símbolos escritos em cada posição, e o cabeçote de leitura da fita, com um estado q_i .

Figura 2.3. Ilustração da Máquina de Turing.



Fonte: Próprio autor (2021).

A grande notoriedade da máquina de Turing não é pela sua simplicidade, mas sim porque até os dias atuais ninguém conseguiu desenvolver alguma máquina (teórica ou real)

capaz de computar funções que ela não possa (isso vale também para os computadores mais modernos).

Turing chegou ainda a propor variações da sua máquina, tais como com uma fita infinita para os dois lados, com mais de uma fita e com mais de um cabeçote. No entanto, ele percebeu que todas as variações poderiam ser simuladas pela Máquina A. Daí, ele propôs um outro tipo de máquina, que chamou-a de “Máquina U” (“U” de “Universal”), que consistia de uma máquina de Turing capaz de simular outra máquina de Turing qualquer. Basicamente, a Máquina Universal pode realizar a simulação a partir da leitura de dois tipos de informação (símbolos) da fita: a descrição da máquina a ser simulada (em que considera-se que ela inclui um conjunto de instruções) e sua respectiva entrada de dados. Por conta disso, alguns grandes cientistas atribuem a autoria do conceito de “computador com programa armazenado” a Turing.

Por fim, baseando-se nas ideias de Turing, formulou-se o conceito de “Máquina Turing-Completa”, que consiste em caracterizar máquinas computacionais que conseguem computar qualquer tipo de função. Por exemplo, a Máquina Analítica de Babbage, seria Turing-Completa, enquanto que sua Máquina Diferencial não é.

Turing era um gênio e também deu muitas outras contribuições à ciência nos campos da Criptografia, Inteligência Artificial e Química.

2.4 A Lógica e sua Relação com os Computadores

Em 1854, durante o período de evolução dos computadores mecânicos, George Boole percebeu que era possível expressar lógica (raciocínio) de forma semelhante à aplicação da álgebra, ou seja através de formalismos e regras matemáticas, assim como a aritmética. A princípio, ele observou que era possível representar ideias com símbolos (proposições) e que era possível aplicar operações sobre eles (operações lógicas, tais como “E” e “OU”).

Ao longo dos anos, muitos outros cientistas aperfeiçoaram a Lógica de Boole (ou Álgebra de Boole). Ela se tornou tão importante que, no século seguinte, com a evolução das tecnologias, influenciou a construção de computadores eletrônicos e linguagens de programação (praticamente todas as linguagens de programação modernas possuem o tipo de dados “booleano”, utilizado para representar os valores lógicos “Verdadeiro” e “Falso”).

Já em 1879, Gottlob Frege publicou um livro formalizando a lógica de predicados. Ele introduziu os conceitos de propriedades sobre ideias (“predicados”) e os quantificadores

“universal” e “existencial”. Com esses novos conceitos, tornou-se possível representar com mais precisão elementos lógicos, eliminando ambiguidades existentes até mesmo nas linguagens naturais. Em outras palavras, criou-se um sistema formal onde é possível expressar ideias, as quais possuem propriedades, quantificá-las e relacioná-las.

Um ano após (1880), Charles Pierce descobriu, ao analisar tabelas verdade para operadores booleanos binários (com duas variáveis), que era possível representar os operadores lógicos que até então se conhecia na época (eram 16 ao todo), com apenas dois deles: o “NÃO E” e “NÃO OU”, conhecidos também por “NAND” e “NOR”, respectivamente. Anos depois, essa característica tornou-se fundamental no projeto das “Portas Lógicas” dos computadores, impactando no desempenho dos processos de construção de processadores e de memórias.

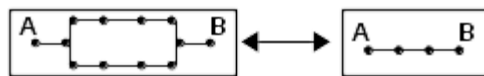
Muitos anos depois, Claude Shannon, em sua dissertação de mestrado, intitulada "A Symbolic Analysis of Relay and Switching Circuits", no Massachusetts Institute of Technology (MIT) (1940), sistematizou o uso da álgebra de Boole e da aritmética binária na eletrônica para construção de computadores. O sistema criado por Shannon permitiu a criação de qualquer circuito eletrônico através do emprego de funções lógicas, além da aplicação de regras da álgebra Booleana para simplificação de circuitos, tornando-os menores, mais econômicos e mais eficientes.

Basicamente, o sistema de Shannon considera que o contato entre dois pontos pode estar aberto (não permite passagem de eletricidade) ou fechado (permite a passagem de eletricidade). Em um circuito fechado, para Shannon, a passagem de eletricidade é representada pelo valor booleano 0 (zero), e, em um circuito aberto, a ausência de eletricidade é representada pelo valor booleano 1 (um). Assim, a partir dessas definições e com a aplicação dos operadores booleanos – considera-se aqui que o símbolo “+” consiste no operador de conjunção (AND), que representa um circuito em série, e o símbolo “.” representa o operador de disjunção (OR), que representa um circuito em paralelo – é possível, por exemplo, representar os seguintes circuitos:

- Dois circuitos abertos, em série, equivalem a um único circuito aberto ($1 + 1 = 1$);



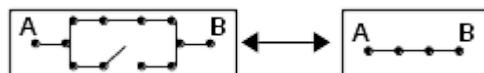
- Dois circuitos fechados, em paralelo, equivalem a um único circuito fechado ($0 . 0 = 0$);



- Um circuito aberto em série com outro fechado, equivale a um circuito aberto ($1 + 0 = 1$);



- Um circuito aberto em paralelo com outro fechado, equivale a um circuito fechado ($1 \cdot 0 = 0$).



O trabalho de Shannon é considerado o mais importante do século XX, pois até então, os circuitos eletrônicos eram construídos apenas por projetistas com muita experiência e de forma puramente intuitiva.

2.5 Os Computadores Eletromecânicos

O campo da eletrônica surgiu a alguns séculos atrás, mais precisamente durante o século XVIII, com as experiências de Benjamin Franklin. Em 1750, Franklin concebeu uma teoria em que os corpos possuem cargas positivas e negativas e, ao entrarem em contato entre si, essas cargas se moviam de um corpo para outro. Com o descobrimento do elétron, em 1897, por Joseph Thomson, verificou-se que esse movimento de cargas consistia, na realidade, no movimento de elétrons, conhecido hoje por “corrente elétrica”.

Outros avanços da eletricidade, desde então permitiram o desenvolvimento da eletrônica, alguns deles:

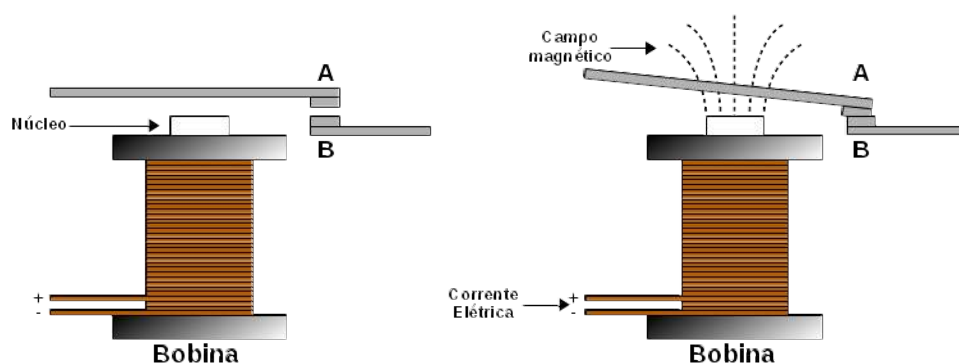
- Alessandro Giuseppe Volta observou ser possível armazenar eletricidade em uma pilha de cobre e zinco (1800);

- Michael Faraday descobriu que um campo magnético pode gerar eletricidade numa bobina, que consiste em um fio de cobre condutor enrolado várias vezes ao longo de um cilindro (1831);
- Thomas Edison desenvolveu a lâmpada elétrica (1880); e
- George Westinghouse criou o primeiro motor elétrico, a partir dos estudos de Faraday (1888).

Um cientista americano, chamado Joseph Henry, ao estudar o eletromagnetismo (1835), descobriu o fenômeno conhecido por “indução eletromagnética”: uma bobina ao receber uma corrente elétrica, gera um campo magnético e se torna capaz de atrair metais. Assim, ele percebeu que era possível construir circuitos, onde as bobinas ao receberem corrente elétrica, fecham canais de circuito e com isso permitem a passagem de corrente elétrica de um ponto a outro no circuito. De forma análoga, ao não receber corrente elétrica, a bobina não atrai o metal e com isso cria um curto circuito. Essa bobina foi chamada de “Relé Eletromecânico”.

A Figura 2.4 ilustra o princípio básico de funcionamento de um relé eletromecânico. Na Figura 2.4 (a), pode-se ver uma bobina cujo núcleo se aproxima de um terminal metálico A. Ao aplicar uma corrente elétrica nos contatos de cobre a bobina, como é mostrado na Figura 2.4 (b), gera-se um campo magnético no núcleo dela, que atrairá o terminal metálico A e fará com que ele vá de encontro ao terminal metálico B, fechando assim o circuito e, com isso, permitindo a passagem de energia.

Figura 2.4 - Relé eletromecânico.



(a) Bobina sem campo magnético.

(b) Bobina com campo magnético.

Fonte: Próprio autor (2021).

Em relação às engrenagens mecânicas, construir um circuito utilizando relés era muito mais simples. Outro ponto interessante, é que podia-se reconfigurar facilmente um circuito

com relés simplesmente utilizando conectores e cabos elétricos, permitindo a construção de circuitos diferentes. O relé eletromecânico foi uma descoberta muito importante pois com ele pôde-se construir o telégrafo, em 1838, por Samuel Morse.

Em 1937, um pesquisador da Bell Telephone Laboratories, George Robert Stibitz, na cozinha da sua casa, montou um circuito somador binário simples, utilizando dois relés telefônicos, duas lâmpadas de lanterna, um bateria, alguns fios e interruptores improvisados a partir de uma lata de alimento em conserva. Compreende-se que as lâmpadas foram utilizadas para representar o resultado de uma soma de dois números binários informados pelos interruptores. A esposa de Stibitz batizou o resultado desse experimento de “Model K” (“K” de *Kitchen*, cozinha em inglês).

Com o passar do tempo, as calculadoras mecânicas passaram a não dar conta dos cálculos matemáticos, que estavam ficando cada vez mais complicados e volumosos, principalmente quando se tratavam de operações sobre números complexos. A partir daí, a Bell Labs decidiu investir na ideia de Stibitz, e assim começaram a surgir os primeiros computadores do século XX.

Seguem os principais computadores que se basearam em relés eletromecânicos:

- Z1 (1936) : Produzido por Konrad Zuse, foi o primeiro computador eletromecânico produzido com relés. No entanto, possuía partes mecânicas também. Utilizava também um sistema binário para os cálculos;
- Z2 (1940) : Também produzido por Zuse, este modelo aperfeiçoou o modelo anterior ao apresentar a lógica de controle e aritmética implementados totalmente com relés. No entanto, ainda possuía partes mecânicas;
- Model I (1940) : Construído por George Robert Stibitz, na Bell Labs, originalmente criado para trabalhar com números complexos, era chamado de *Complex Number Calculator*, foi o primeiro computador a utilizar múltiplos terminais e a ser controlado remotamente;
- Z3 (1941) : Mais uma das invenções de Zuse, o Z3, que foi construído inteiramente com relés, foi também o primeiro computador totalmente automático e inteiramente controlado por programas. Os programas eram armazenados em fitas perfuradas;
- Harvard Mark I (1944) : Primeiro computador de propósito geral norte-americano e foi utilizado apenas para fins militares, inclusive na criação da bomba atômica. Grace Hopper, analista de sistemas da marinha, era uma das programadoras do Mark I e foi ela que cunhou os termos “bug” (inseto em inglês), ao descobrir um inseto morto

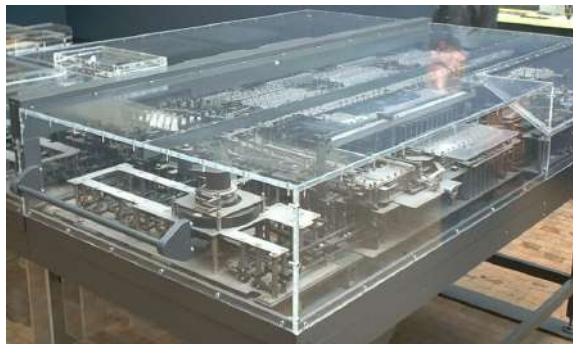
dentro de um dos relés do Mark I, e "debugging", que hoje significa remover erros de programas;

- Z4 (1945) : Desta vez, além de melhorias no projeto do Z3, Zuse introduziu instruções de decisão e impressão de resultados em máquina de escrever ou perfuradora de cartões, produzindo o primeiro computador comercial do mundo com hardware com suporte de operações de ponto flutuante. O Z4 era tão confiável que poderia ser utilizado sem a necessidade de supervisão, recurso que o ENIAC não permitia, como será visto adiante;
- Model V (1946) : Construído por George Robert Stibitz, na Bell Labs, além de novas instruções, foi o primeiro computador americano a utilizar aritmética de ponto flutuante e foi um dos precursores do multiprocessamento, pois possuía dois processadores (com capacidade de ter até seis);
- ARC (1948) : O *Automatic Relay Computer* diferiu significativamente por possuir um sistema de armazenamento de dados mais eficiente em relação aos computadores construídos até então, pois utilizava armazenamento de dados em meio magnético (uma espécie de tambor de níquel rotativo), sendo que os demais utilizavam fitas ou cartões perfurados;
- SAPO (1957) : Primeiro computador construído na antiga Tchecoslováquia e foi o primeiro tolerante a falhas: possuía três unidades lógico-aritméticas paralelas, que decidiam o resultado correto de um processamento por votação (em caso de dois resultados idênticos, este seria considerado o correto; já no caso de todos resultados fossem diferentes, a operação seria repetida).

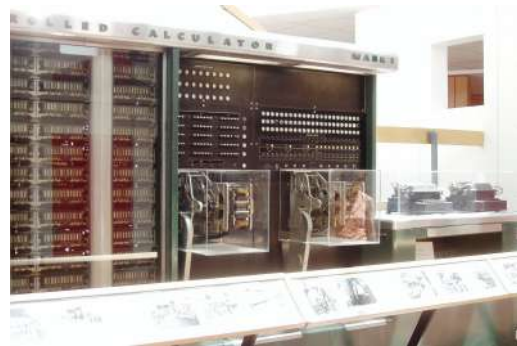
A Figura 2.5 apresenta ilustrações dos computadores eletromecânicos Zuse Z1, em (a), e Harvard Mark I, em (b).

Apesar de todas as vantagens sobre as engrenagens mecânicas, os relés eletromecânicos eram caros, ocupavam muito espaço e eram lentos. Estima-se que levavam em torno de 1 milésimo de segundo para fechar um circuito, o que, para os padrões atuais, considera-se tempo demasiado.

Figura 2.5. Computadores eletromecânicos.



(a) Zuse Z1⁷.



(b) Harvard Mark I⁸.

Fonte: Wikimedia (2021).

2.6 A Primeira Geração de Computadores Eletrônicos

Alguns anos depois, já em 1904, John Ambrose Fleming apresentou uma nova tecnologia, a Válvula Eletrônica, que se tornou a base para os primeiros computadores eletrônicos.

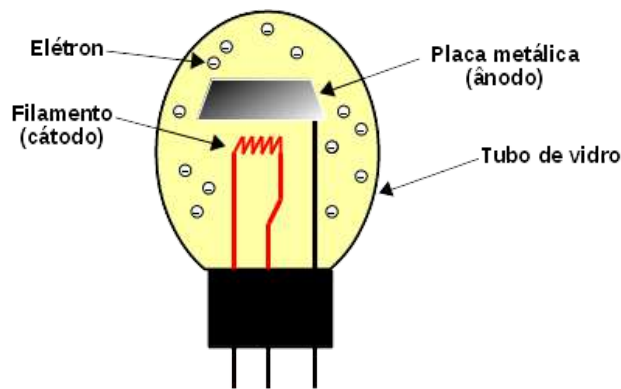
Uma válvula eletrônica, que está ilustrada na Figura 2.6, consiste basicamente de um tubo de vidro, fechado a vácuo, onde, dentro dele há um filamento e uma placa metálicos. Seu funcionamento é bastante simples e se dá da seguinte forma: o filamento (cátodo) é aquecido, de forma a gerar energia suficiente para expulsar os elétrons de seu metal, os quais ficam distribuídos dentro do tubo; ao aplicar uma carga positiva na placa metálica (ânodo), os elétrons são atraídos e movidos para lá, gerando uma corrente elétrica; do contrário, ao aplicar uma carga negativa na placa metálica, os elétrons serão repelidos e, portanto, não haverá corrente elétrica nela.

As válvulas eletrônicas foram os componentes básicos utilizados na construção das portas lógicas utilizadas nos primeiros computadores totalmente eletrônicos (lembrando que até então, as tecnologias utilizadas eram instrumentos mecânicos ou eletromecânicos).

⁷ By Stahlkocher. Licença CC BY-SA 3.0. Disponível em:
<https://upload.wikimedia.org/wikipedia/commons/6/6f/Zuse_Z1.jpg>.

⁸ By Topory. Licença CC BY-SA 3.0. Disponível em:
<https://upload.wikimedia.org/wikipedia/commons/3/35/Harvard_Mark_I_Computer_-_Right_Segment.JPG>.

Figura 2.6 - Válvula eletrônica.



Fonte: Próprio autor (2021).

A partir do emprego das válvulas, em substituição aos relés eletromecânicos, os computadores passaram a ser construídos com componentes puramente eletrônicos. Com a mudança de tecnologia, caracteriza-se que os computadores criados desde então compõem o que chama-se de Primeira Geração dos Computadores.

Nesta época, a Segunda Guerra Mundial (1945) impulsionou a necessidade de construção de máquinas computadoradas capazes de decifrar códigos - as mensagens inimigas era frequentemente interceptadas, porém necessitavam ser decifradas - e computar tabelas com distâncias de alcance e rotas para uso de artilharia pesada. Esses cálculos, a priori, eram realizados por dezenas de pessoas com apoio de calculadoras de mesa e, por sua complexidade, levavam horas e até dias para se encontrar os resultados.

Com isso, em diversos locais do mundo, com a necessidade de tomar vantagens estratégicas frente à guerra, diferentes tipos de computadores foram surgindo. Seguem os principais computadores da primeira geração:

- **Bombe (1939)** : Os britânicos descobriram que as mensagens alemãs interceptadas eram codificadas por um dispositivo muito engenhoso, chamado de *Enigma*. Com isso o governo britânico criou um laboratório de pesquisas ultrassecretas, que contou com a ajuda de Alan Turing para a construção de uma máquina eletromecânica capaz de encontrar, através de técnicas de tentativa e erro, a codificação diária utilizada pela Enigma. A princípio, a Bombe não era tão eficiente, pois haviam muitas combinações a serem testadas antes de se decodificar completamente uma mensagem. No entanto, os pesquisadores descobriram que as mensagens interceptadas tinham certos padrões formais no texto, e com isso conseguiram eliminar a tarefa de decodificação dessas partes da mensagem, reduzindo assim a necessidade de realizar tantos testes de combinações. Com as mensagens decodificadas, foi possível prever as ações das

forças inimigas alemãs com um dia de antecedência. A história conta que essa proeza fez com que a Segunda Guerra Mundial fosse vencida pelos aliados;

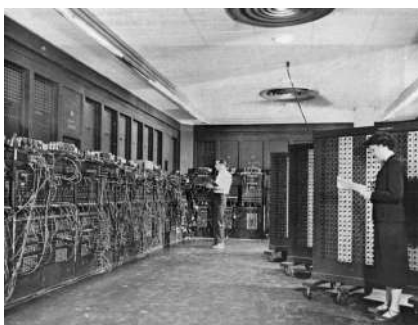
- ABC (1942) : *Atanasoff-Berry Computer* foi primeiro computador construído apenas com válvulas eletrônicas, porém não era programável e era de propósito específico, pois era apenas utilizado para resolução de sistemas de equações lineares;
- COLOSSUS (1943) : No mesmo laboratório britânico de pesquisas ultrassecretas, que teve ajuda de Alan Turing, Thomas Flowers trabalhou em um outra máquina de quebra de criptografia, chamada de Colossus. Ela era um computador digital programável de propósito específico e era utilizada para decodificar as mensagens codificadas por uma máquina diferente da Enigma (descobriu-se, na sequência, que a máquina utilizada para codificar as mensagens era a *Lorenz SZ*);
- ENIAC (1945) : O *Electronic Numerical Integrator And Computer* foi o primeiro computador de propósito geral construído com válvulas eletrônicas. Construído na Universidade da Pensilvânia, por John Eckert e John Mauchly, ele possuía muitos painéis com milhares de interruptores e necessitava de várias pessoas para programá-lo. Além de cálculos para balística, o ENIAC foi utilizado para detectar falhas no projeto da bomba de hidrogênio e para demonstrar, através de simulações, a eficácia do método estatístico Monte Carlo;
- IAS (1951) : Construído no Institute for Advanced Study (IAS), um centro de pesquisas em Princeton, o IAS foi um dos computadores mais importantes, pois ele foi o primeiro a implementar o modelo de von Neumann: computava números em formato binário, implementou o conceito de programa armazenado e possuía unidades funcionais bem diferenciadas (unidades de processamento e controle, memória de dados e programas e mecanismo de entrada/saída);
- EDVAC (1951) : A ideia de programa armazenado foi proposta inicialmente por John von Neumann para o EDVAC. Inclusive, o artigo que tornou famoso o modelo de von Neumann citava o projeto do EDVAC. No entanto, após os criadores do ENIAC decidirem deixar a Universidade da Pensilvânia para criar sua própria empresa de computadores, von Neumann decidiu deixar o projeto para construir sua própria máquina (IAS);
- UNIVAC (1951) : Foi o primeiro computador produzido em massa para fins comerciais nos Estados Unidos (acredita-se que tenha sido o primeiro computador eletrônico comercial do mundo). Ele foi projetado pelos criadores do ENIAC, porém agora na empresa Eckert-Mauchly Computer Corporation, e foram vendidas 46

unidades, em que a primeira unidade foi destinada ao escritório de censo dos Estados Unidos e a segunda ao Pentágono;

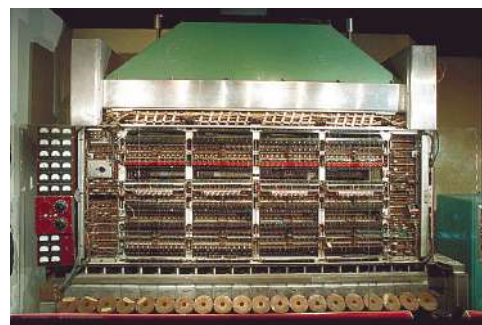
- IBM 701 (1952) : Também conhecido como *Defense Calculator*, o IBM 701 foi o primeiro computador comercial para fins científicos. Seu projeto foi baseado no IAS, e deu origem a vários outros sucessores, como o IBM 702, que era orientado ao processamento de grande bases de dados em aplicações empresariais, e o IBM 650, que era um computador de propósito geral de baixo custo;
- IBM 650 (1954) : Foi o primeiro computador a ser produzido em grande escala e estima-se que foram vendidas mais de 2000 unidades. Por conta do seu baixo custo, foi utilizado para diferentes tipos de aplicações científicas, empresariais, de engenharia, simulação, modelagem e até mesmo para o ensino de programação para estudantes do ensino médio e superior.

A Figura 2.7 apresenta os computadores ENIAC, em (a), e o IAS, em (b). Na figura, percebe-se a diferença de dimensões e de interfaces programáveis entre os computadores.

Figura 2.7. Computadores da primeira geração.



(a) ENIAC⁹.



(b) IAS¹⁰.

Fonte: Wikimedia (2021).

2.7 A Segunda Geração de Computadores Eletrônicos

Em paralelo à criação dos primeiros computadores da primeira geração, já estava em desenvolvimento a tecnologia que substituiria as válvulas eletrônicas. Em 1947, os físicos

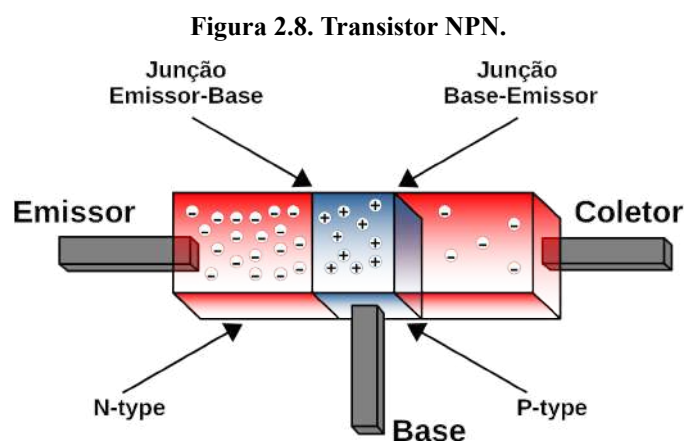
⁹ By Stw. Licença sob Domínio Público. Disponível em: <https://upload.wikimedia.org/wikipedia/commons/4/4e/Eniac.jpg>.

¹⁰ By Nixdorf. Licença sob Domínio Público. Disponível em: https://upload.wikimedia.org/wikipedia/commons/d/d6/IAS_machine_at_Smithsonian.jpg.

John Bardeen, Walter Brattain e William Shockley criaram o primeiro “Transistor”, na Bell Labs.

O transistor é um componente eletrônico composto de silício, que é um material semicondutor (não permite passagem de corrente elétrica sobre ele com facilidade) e presente em grãos de areia. Porém, se o silício for tratado com determinados tipos de materiais (impurezas), processo conhecido como *Dopping*, tais como arsênico, fósforo ou antimônio, é possível modificar suas propriedades, de forma que ele fique com mais elétrons livres, e com isso ele possa conduzir corrente elétrica. Este tipo de silício é chamado de “N-type” (tipo negativo). É possível ainda dopar o silício com boro, gálio e alumínio, fazendo com que ele tenha menos elétrons livres, portanto os elétrons de outros materiais próximos tendem a fluir para ele. Este segundo tipo de silício é chamado de “P-type” (tipo positivo).

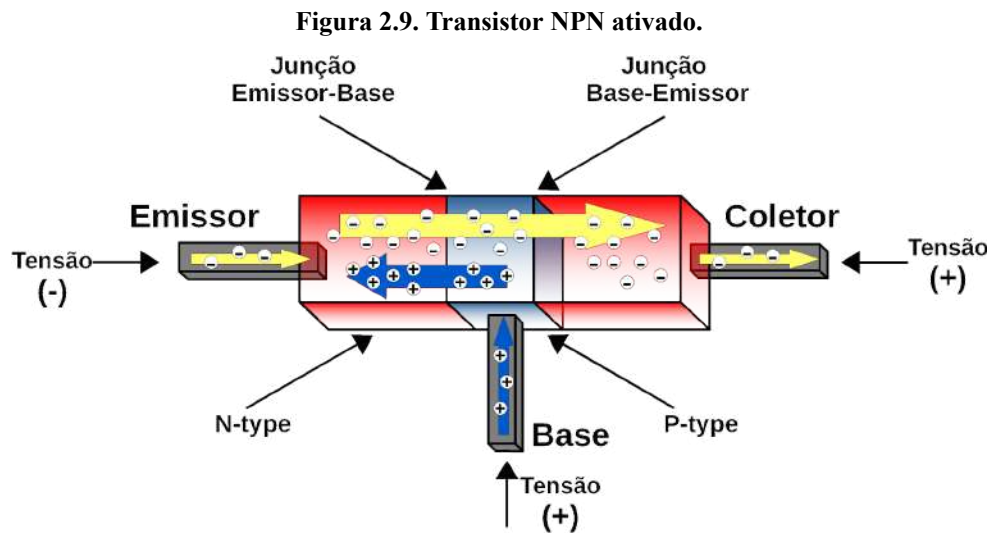
Supondo que utiliza-se, por exemplo, três camadas de silício, sendo que estão associadas na forma NPN: uma camada do tipo negativo com maior dopagem (camada com maior número de elétrons), uma camada fina do tipo positivo com menor dopagem (ausência de elétrons) e uma camada do tipo negativo com menor dopagem (poucos elétrons). Supondo agora que essas três camadas estão ligadas a contatos elétricos, chamados de Emissor, Base e Coletor, respectivamente. Entre as camadas há ainda junções de depleção, que basicamente, são regiões neutras, onde há íons positivos e negativos fixos cristalinos, ou seja, não há elétrons livres e são barreiras para passagem de elétrons de uma camada para outra. Essas junções são chamadas de “Emissor-Base” e “Base-Coletor”, por estarem justamente entre as respectivas camadas, como mostra a Figura 2.8.



Fonte: Próprio autor (2021).

Ao aplicar uma tensão positiva pequena à Base, uma tensão negativa ao Emissor e uma tensão positiva ao Coletor, o transistor é ativado. Os elétrons fluirão do Emissor ao

Coletor, passando pela camada P, gerando assim uma corrente elétrica dentro do transistor, como mostra a Figura 2.9.



Para que seja possível que os elétrons saltem entre as camadas separadas pelas junções Emissor-Base e Base-Coletor, uma parte das tensões aplicadas nos terminais do transistor é consumida pelas junções de depleção (em geral consome-se 0,7V), de forma que as suas propriedades são mudadas e passam a conduzir elétrons intensamente.

No final da década de 40, os computadores passaram a ser construídos com válvulas eletrônicas, que eram bem mais rápidas do que os relés eletromecânicos. No entanto, os computadores ainda eram muito caros, ocupavam muito espaço físico e as válvulas eletrônicas consumiam muita energia e queimavam com frequência. Praticamente, somente alguns governos, universidades e grandes empresas podiam adquirir computadores nessa época. Naturalmente, o próximo passo na fabricação de computadores seria investir em semicondutores, o transistor, para substituir as válvulas eletrônicas.

Em 1953, na Universidade de Manchester, Tom Kilburn produziu o primeiro computador transistorizado. Em 1954, a Bell Labs criou uma máquina semelhante, o TRADIC (*TRAnsistor Digital Computer*). No entanto, ambos não eram totalmente transistorizados, pois utilizavam válvulas para gerar pulsos de *clock* – os computadores necessitam de um mecanismo para regular e sincronizar o instante em que as ações são executadas, esse mecanismo é chamado de “relógio” ou *clock* –, pois, naquela época, os transistores não produziam energia suficiente para manter os computadores executando tão rapidamente.

O primeiro computador totalmente transistorizado foi desenvolvido em uma instalação de pesquisas atômicas em Harwell, no Reino Unido, entre 1953 e 1955, e chamava-se CADET, que é a sigla ao contrário de *Transistor Electronic Digital Automatic Computer*. Por conta do uso exclusivo de transistores, o *clock* do CADET ficou com frequência de operação baixa (em torno de 58 KHz), em relação à dos computadores a válvulas eletrônicas.

Até o final da década de 60, muitos computadores transistorizados foram construídos. Seguem alguns que fizeram história:

- TX-0 (1956) : Projetado no MIT, nos anos de 1955 e 1956, ele foi utilizado continuamente durante os anos 60. Com o seu grande sucesso, logo se iniciou a produção do TX-1, que era muito maior e mais complexo. No entanto, o projeto se tornou inviável devido à alta complexidade, então ele foi reprojetoado e, em 1958, foi entregue o TX-2. O TX-2 teve um papel muito importante no avanço da inteligência artificial e da interação homem-computador. Além disso, o TX-2 teve papel fundamental na criação da Internet, pois foi nele que Leonard Kleinrock simulou sua teoria matemática sobre redes de comutação de pacotes;
- IBM 608 (1957) : É considerado o primeiro computador transistorizado fabricado para o mercado comercial. Logo após o lançamento do 608, a IBM determinou que seus produtos não mais seriam fabricados com válvulas eletrônicas. A partir dele, surgiram novas linhas de computadores, tendo como sucessor direto o IBM 650, que foi sucedido pelo IBM 7070 (o primeiro computador transistorizado com programa armazenado);
- Philco Transac (1958) : A Philco foi uma das pioneiras em computadores transistorizados, justamente porque desenvolveu o primeiro modelo de transistor de alta frequência, também chamado de transistor de germânio de superfície-barreira, em 1953, que era muito mais rápido que os até então existentes. Ele era capaz de operar a até 60 MHz. O Transac (*Transistor Automatic Computer*) modelo S-1000 foi inicialmente lançado como um computador científico. Logo em seguida, foi lançado o modelo S-2000 voltado para o mercado de mainframes empresariais e científicos. Porém, como a IBM dominava o mercado de mainframes, a Philco faliu, sendo comprada, em 1961, pela Ford Motor Company. Outros modelos continuaram sendo lançados, como os modelos 210, 211 e 212;
- IBM 7070 (1958) : Foi o primeiro computador baseado em transistores que suportava programa armazenado. Apesar de ter sido o sucessor do IBM 650, eles não eram

compatíveis. Foi necessário criar um simulador para fosse possível executar instruções do 650, e assim emular a execução dos seus programas no 7070. Era um computador que lidava com números decimais de até 10 dígitos e um dígito adicional para o sinal (positivo/negativo). Ele deu origem a uma série de computadores mais rápidos, como IBM 7074 (1960) e o IBM 7072 (1961), mas foram substituídos pelo System/360, posteriormente;

- PDP-1 (1959) : Foi o primeiro computador da Digital Equipment Corporation (DEC). O PDP-1 tem muitas histórias curiosas, como por exemplo, cita-se que a cultura hacker foi criada com ele, que foi o minicomputador a ter o primeiro game da história (*Spacewar* de Steve Russell, em 1962), o primeiro editor e processador de textos (*Expensive Typewriter* de Stephen Piner, em 1962) e uns dos primeiros sistemas de compartilhamento de tempo entre tarefas (criado pela empresa Bolt, Beranek e Newman – BBN, 1962). Historicamente, o PDP-1 foi baseado nos TX-0 e TX-2, do MIT (em 1961, a DEC doou uma unidade do PDP-1 ao MIT, que foi colocado ao lado do seu antecessor TX-0). Diz-se ainda que o PDP-1 mudou radicalmente o design dos computadores, pois foi o primeiro computador a focar na interação com o usuário (antes dele, as pesquisas concentravam-se exclusivamente no aumento do desempenho e uso eficiente dos computadores). A partir do PDP-1, muitos outros modelos foram criados; e
- System/360 (1964) : É uma família de mainframes produzida pela IBM, que substituiu a família 7070, para cobrir qualquer tipo de aplicação comercial e científica. A família System/360 possuía vários modelos, com diferentes capacidades e recursos computacionais. Isso fez com que ele fosse bem sucedido no mercado, pois ao adquirir um dos modelos, caso as necessidades computacionais mudassem, seria possível adquirir um outro modelo (com maior ou menor número de recursos, dependendo da necessidade), sem perder todos os aplicativos, dados e dispositivos periféricos. Outro ponto interessante, é que haviam simuladores que permitiam emular outros computadores e outras famílias de computadores da IBM. Por exemplo, a partir do System/360 modelo 50 foi possível emular os computadores da família antecessora (7070).

Curiosidade!

Em 1960, no curso de eletrônica do Instituto Militar de Engenharia (IME), criou-se o primeiro computador do Brasil, o “Lourinha”. Ele possuía circuitos digitais e analógicos capazes de realizar cálculos complexos, em tempo real, tais como simulação de sistemas de equações diferenciais. Após sua apresentação, pois tratava-se de um projeto de conclusão de curso, o Lourinha foi desmontado e suas partes foram utilizadas para o estudo de Arquitetura de Computadores até os anos 70. Em 1961, os estudantes do Instituto Tecnológico da Aeronáutica (ITA) juntamente com a Escola Politécnica da Universidade de São Paulo (USP) e a Pontifícia Universidade Católica do Rio de Janeiro (PUC/Rio), apresentaram, também como projeto de conclusão de curso, um computador didático que mostrava como a informação se processava dentro dele. Esse computador, denominado “ITA I”, e posteriormente batizado de “Zezinho”, foi construído com transistores discretos, usando soquetes de válvulas para demonstração e uso em laboratório. A partir do Lourinha e Zezinho, dez anos depois, outros computadores foram surgindo, como o “Patinho Feio” e o “G-10”. Assim nasceu a indústria de computadores no Brasil.

2.8 A Terceira Geração de Computadores Eletrônicos

No final da década de 50 e início da década de 60, aumentaram as demandas de aquisição de computadores e os requisitos por equipamentos com mais recursos computacionais. A indústria estava com dificuldades para entregar e lançar novos produtos, pois os computadores eram produzidos com componentes discretos – transistores, capacitores, resistores, entre outros –, que eram soldados em placas e conectados por fios e mais fios. Em média, os computadores daquela época possuíam 10.000 transistores, sendo que o processo de fabricação dos transistores ainda era caro e consumia muito esforço da indústria para entregar o produto em larga escala. Além do mais, quanto maiores os computadores ficavam, mais transistores eram necessários e maiores os problemas a serem tratados (por exemplo, maior consumo de energia e custo de manutenção, e problemas físicos, tais como superaquecimento e atrasos para propagação de sinais elétricos por conta dos tamanhos dos fios para conexão dos componentes).

Um pouco antes, George Dummer (1952), que trabalhava com engenharia de confiabilidade de circuitos em um laboratório de pesquisas no Reino Unido, publicou um artigo no qual afirmava que, ao reduzir o tamanho dos transistores, os circuitos se tornavam com frequência mais confiáveis. Com isso ele concluiu ser possível construir circuitos

confiáveis integrados a partir de um bloco de cristal, como o silício. Esta foi a primeira publicação que tratava do conceito de Circuito Integrado.

No entanto, somente em 1960, na Texas Instruments, o primeiro circuito integrado foi construído, por Jack Killby. Em paralelo, na Fairchild Semicondutores, Robert Noyce trabalhava em um projeto com o mesmo conceito, porém com a diferença de ser produzido em um processo mais simples. O circuito integrado de Killby além de ser composto por germânio, era híbrido – o circuito era composto por partes conectadas –, enquanto que o de Noyce era por silício e monolítico. Ambas as empresas entraram com pedidos de patente, que, por consequência, geraram processos judiciais para reconhecimento da invenção. Por volta de 1966, após um acordo entre partes, reconheceu-se que ambas haviam inventado o circuito integrado. Killby e Noyce foram contemplados e dividiram o Prêmio Nobel de Física pela criação do circuito integrado no ano de 2000.

Logo após a criação dos circuitos integrados, as empresas passaram a imaginar possibilidades de desenvolver componentes complexos integrados em um único “chip” (este termo se refere a uma pequena lâmina de silício, utilizada na construção de transistores, díodos ou outros semicondutores miniaturizados). Por exemplo, ainda em março de 1960, a Texas Instruments introduziu sua primeira família de produtos de circuitos integrados, com o nome de *Solid Circuits*. Essa família, também chamada de Série 51, tinha os chips SN510 e SN514, que foram os primeiros circuitos integrados a orbitar a Terra, a bordo do satélite IMP, lançado pelos EUA em 27 de novembro de 1963. O SN510 e SN14 foram utilizados no satélite como contadores binários, circuitos inibidores e *flip-flops* (o elemento básico de armazenamento de dados na lógica do computador). Já do outro lado, na Fairchild, foi anunciado, em novembro de 1960, seu primeiro circuito integrado comercial, *um flip-flop*. A partir de 1962, outras empresas passaram também a produzir circuitos integrados, como a Motorola e Signetics.

Em 1968, Marcian “Ted” Hoff, após concluir seu doutorado, decidiu trabalhar na indústria, e, após uma entrevista com Robert Noyce (um dos inventores do circuito integrado), foi o décimo segundo funcionário contratado para trabalhar numa *startup* chamada Intel, recentemente criada por Noyce. Hoff percebeu que, com a tecnologia de circuitos integrados da Intel e uma arquitetura bastante simplificada, que incluía memória, funções de controle e processamento, seria possível integrar, em um único chip, os vários circuitos necessários para construir um processador. Surge aí o conceito de Microprocessador. Após a Intel desenvolver as tecnologias necessárias para implementar o conceito de Hoff, em 1971, surge o primeiro microprocessador, o Intel 4004.

A partir do Intel 4004, muitos outros microprocessadores foram lançados até os dias atuais. Seguem alguns deles que fizeram história:

- Intel 4004 (1971): Era um microprocessador de 4-bits com o mesmo poder de processamento do ENIAC, construído 25 anos antes. Possuía 2.300 transistores, tinha capacidade de processar entre 60 e 90 mil instruções por segundo. Na mesma família de circuitos, havia o Intel 4001 (memória ROM), Intel 4002 (memória RAM que podia armazenar até 80 dados de 4-bits) e o Intel 4003 (um registrador de deslocamento utilizado para entrada de dados como, por exemplo, pelo teclado);
- Intel 8008 (1972): Foi o primeiro microprocessador de 8-bits do mundo. Apesar do seu lançamento ter sido logo posteriormente ao do Intel 4004, seu projeto era totalmente diferente, sendo projetado pela Computer Terminals Corporation, a partir do projeto do chip CTC 1201. Do Intel 8008 sucedeu o Intel 8080, e as empresas concorrentes lançaram o Zilog Z80 e os Motorola 6800 e Motorola 6502. O computador Mark-8, que era vendido como um kit de baixo custo para construção de um computador do tipo DIY (“Do It Yourself” ou “faça você mesmo”), utilizava o Intel 8008 como microprocessador;
- Intel 8080 (1974): Foi o segundo processador de 8-bits da Intel, ele estendeu e melhorou o projeto do Intel 8008, porém eles não eram compatíveis (as aplicações que executavam no Intel 8008 não executavam no Intel 8080, e vice-versa). A linha de computadores Altair 8800 e posteriores, inicialmente, eram embarcados com o Intel 8080, sendo posteriormente substituído pelo seu concorrente, o Zilog Z80. O sistema operacional CP/M (*Control Program/Monitor*) foi projetado, em 1976, para computadores com o Intel 8080, por Gary Kildall da empresa Digital Research Inc. Ressalta-se que o CP/M foi primeiro sistema operacional criado para microprocessadores e o precursor do sistema operacional DOS. Outro grande fator de sucesso do Intel 8080 é que foram criados outros chips que adicionaram novos recursos, como o Intel 8257, que é um controlador de acesso direto à memória (*Direct Memory Access* - DMA), e o Intel 8259, que é um controlador de interrupções programáveis (*Programmable Interrupt Controller* - PIC) ;
- Zilog Z80 (1976): Foi produzido pela empresa Zilog, como uma extensão e aprimoramento do Intel 8080. Seu projeto foi copiado por várias fabricantes de chips, como a NEC, Toshiba, Sharp e Hitachi. Ele era compatível com o Intel 8080, de forma que, para execução no Zilog Z80, não era necessário realizar modificações nas aplicações, incluindo o próprio sistema operacional CP/M. Inicialmente, projetado

para sistemas embarcados, foi utilizado em computadores até a década de 1980, tais como Heathkit H89, o portátil Osborne 1, a série Kaypro, o Epson QX-10, o TRS-80, Sinclair, Amstrad e Commodore 128. O computador Apple II permitiu embarcar um Zilog Z80 e um Motorola 6502, contando assim com dois processadores, e rodava também o sistema operacional CP/M;

- **Motorola 6502 (1975):** Produzido a partir do Motorola 6800, o projeto 6502 (ou Motorola MC6502) era mais simples, mais barato e mais rápido. Em relação aos concorrentes, foi considerado o mais barato (estima-se que custava 1/6 do valor do Intel 8080 e do seu antecessor Motorola 6800). Assim como o Zilog Z80, o Motorola 6502 revolucionou o mercado de computadores domésticos no início dos anos 1980. Diversos consoles de videogames e computadores pessoais foram produzidos com o 6502, tais como Atari 2600, Atari Lynx, Apple I e II, Nintendo Entertainment System (NES), Commodore PET e VIC-20, BB micro entre outros. O sucessor direto do 6502, o Motorola 6510 foi utilizado no Commodore 64, listado no *Guinness World Record Book* (Livro dos Recordes) como o computador mais vendido de todos os tempos. Outros pontos curiosos sobre o 6502 é que era o processador utilizado pelo ciborgue T-800, vivido pelo ator Arnold Schwarzenegger, no filme *Exterminador do Futuro* (1984), e pelo Robô Bender, da série animada *Futurama* (1999 à 2003);
- **Intel 8086 (1978):** Foi o primeiro processador de 16-bits da Intel. Era considerado superior ao Motorola 6502 e ao Intel 8080, bem como houve uma variação dele, o Intel 8088, que tinha menor desempenho mas era compatível com os microprocessadores de 8-bits. O Intel 8086 foi a base para o primeiro computador pessoal (*Personal Computer* - PC) produzido pela IBM, em 1982, o IBM-PC. Além de projetado para bater a concorrência forte de microprocessadores de 8-bits (Zilog Z80 e Motorola 6502), e ser compatível com os Intel 8008 e 8080, através de conversão simplificada de código-fonte para código equivalente, o Intel 8086, indiretamente, definiu um novo padrão da indústria, a compatibilidade de código binário (*Binary-code Compatibility*), também conhecida por retrocompatibilidade. Em termos gerais, isso significa que todos programas criados para execução no Intel 8086 poderão ser executados em seus microprocessadores sucessores, porém de forma muito mais rápida. A partir do Intel 8086, vieram os Intel 80286, 80386, 80486, Pentium, entre outros, que ficaram conhecidos como a Família x86. Praticamente, todos os computadores pessoais mais modernos possuem arquiteturas compatíveis com o Intel 8086.

2.9 Resumo

Este capítulo apresentou brevemente o surgimento e a evolução do computador. Em uma linha de tempo, foram apresentados os primeiros artefatos utilizados para contagem, a criação de novas teorias matemáticas, tecnologias de fabricação e os principais eventos que resultaram no atual estado da arte do computador.

Destaca-se que este capítulo não buscou esgotar o tema. Para uma leitura mais detalhada e aprofundada sobre os fatos históricos que fizeram parte da criação e evolução do computador, bem como da ciência da computação, recomenda-se o livro “História da Computação” de Wazlawick (2016).

CAPÍTULO 3 – TENDÊNCIAS TECNOLÓGICAS

Antes de iniciar a discussão sobre as tendências tecnológicas, é importante esclarecer o porquê deste livro tratar apenas de três gerações de computadores eletrônicos. Alguns autores consagrados, como Tanenbaum e Austin (2013), citam quarta, quinta e mais gerações. No entanto, considerando que o que caracteriza uma geração é a tecnologia empregada na fabricação dos principais componentes do computador, admite-se então que o computador está na terceira geração, em que a tecnologia vigente ainda é o circuito integrado.

Assim, com foco na geração atual, a terceira geração, com o passar dos anos, as tecnologias de fabricação de circuitos integrados foram evoluindo. Cada vez mais, os transistores vêm diminuindo de tamanho e, com isso, está sendo possível integrar mais transistores em um único chip. Por exemplo, o Intel 8086 (1978) possuía 29.000 transistores. Já um processador mais moderno, como, por exemplo, o Intel i7-8086k (2018) possui aproximadamente 3 bilhões de transistores (ALCORN, 2018).

Gordon Moore, cofundador da Intel, ao analisar o número de transistores por processador, ao longo dos anos, conseguiu formular a famosa “Lei de Moore”, que estabelece que: o número de transistores, em um circuito integrado, dobra a cada 18 meses. Isso significa que, se hoje existe um processador com 3 bilhões de transistores, daqui a um ano e meio, é provável que sejam produzidos processadores com 6 bilhões de transistores.

Com a redução do tamanho do transistor, foi possível agregar mais recursos aos processadores, tais como aumento: da precisão numérica (e.g. o Intel 4004 lidava com dados de 4-bits, o Intel 8008 de 8-bits, o Intel 8086 de 16-bits, o Intel 386 de 32-bits e o Intel Itanium de 64-bits); da frequência de operação; do tamanho do espaço de endereçamento à memória; do tamanho e do número de memórias cache embarcadas; do número de núcleos processadores em um único chip, entre outros.

Em muitos momentos da história, publicizou-se que a Lei de Moore havia chegado ao fim, justamente pelas dificuldades físicas em se reduzir o tamanho de um transistor. Nesse

sentido, a indústria vem buscando se reinventar, conseguindo, à base de investimentos financeiros exorbitantes em pesquisas científicas, continuar minimizando o tamanho do transistor. No entanto, estudos recentes, tal como o apresentado em (MARKOFF, 2015) apontam que, em breve, a Lei de Moore chegará ao fim.

A indústria de semicondutores, já prevendo os impactos do fim da Lei de Moore, em 2016, criou uma versão do *International Technology Roadmap for Semiconductors* (ITRS), que consiste de um conjunto de documentos que trazem direcionamentos, com uma estratégia chamada *More than Moore*, para os 15 anos seguintes, em pesquisas em diferentes áreas em projetos de circuitos integrados, tais como processos de fabricação, modelagem e simulação, bem como o uso de materiais emergentes. De uma forma geral, essa nova abordagem considera que, no projeto de novos circuitos integrados, o escopo será derivado a partir das necessidades das aplicações, e não mais no dimensionamento dos transistores (WALDROP, 2016).

Com isso, a indústria passou a realizar pesquisas em diferentes áreas, tais como:

- Na busca por materiais alternativos ao silício para produção de transistores, tais como o grafeno, silício-germânio e gálio. No entanto, novos materiais possuem propriedades físicas próprias, e, com isso, todo o conhecimento de décadas de projetos com silício, não é aproveitado. Assim, desafios, como o controle das propriedades térmicas, se tornam novos nesse novo cenário;
- No desenvolvimento de novas técnicas de produção de circuitos integrados, tais como: um melhor aproveitamento volumétrico de um chip pelo uso de transistores empilhados em camadas tridimensionais, e a inclusão de mais núcleos de processamento e de canais de comunicação mais avançados intrachip (*on-chip*), de forma a prover um melhor suporte de hardware para sistemas multitarefas;
- No desenvolvimento de novas técnicas de programação, de forma a aproveitar melhor o hardware disponível. Nesse contexto, vêm surgindo novas técnicas, bibliotecas, *frameworks* e conjuntos de ferramentas integradas (*Software Development Kit* - SDK) para programação multitarefa, visando aumentar o desempenho das aplicações, a redução do consumo de energia e dissipação de calor dos sistemas computacionais; e
- Na criação de circuitos integrados especializados, os quais, como dito anteriormente, são demandados pela necessidade de novas aplicações ou de recursos computacionais específicos.

Este capítulo apresenta os principais esforços científicos e da indústria, que visam dar continuidade à evolução do computador, em que alguns já se tornaram realidade e outros em tendências tecnológicas.

3.1 Unidade de Processamento Gráfico de Propósito Geral

General Purpose Graphic Processing Unit (GPGPU) são especializações das placas gráficas tradicionais (*Graphic Processing Unit* - GPU) e que podem ser utilizadas para processamento paralelo de dados de alta densidade.

Uma GPGPU possui várias unidades de processamento que são utilizadas para realizar operações paralelas sobre *streams*, vetores ou matrizes, apresentando capacidade de processamento superior aos processadores convencionais.

Para programar GPGPUs, atualmente, há plataformas de desenvolvimento, sendo CUDA (*Compute Unified Device Architecture*) (CUDA, 2021) e OpenCL (*Open Computing Language*) (OPENCL, 2021), as mais utilizadas.

Um exemplo de uma GPGPU robusta é a NVidia GeForce RTX 3090, que apresenta 10.496 núcleos de processamento, chamados de “CUDA cores”, e um barramento de dados com largura de 384-bits, utilizado para leitura de grandes volumes de dados da sua memória RAM (TECHPOWERUP, 2021).

3.2 Unidade de Processamento Acelerado

Accelerated Processing Unit (APU) consiste de uma unidade de processamento composta por processadores (*Central Processing Unit* - CPU) e GPU. A ideia básica é explorar, em um único circuito integrado, o potencial de processamento da combinação de CPUs e unidades de processamento gráfico de propósito geral (GPGPU). Com isso, aumenta-se a eficiência de processamento, uma vez que os componentes de processamento e intercomunicação são totalmente *on-chip*. Atualmente, todos os computadores pessoais da Intel e AMD são equipados com processadores do tipo APU.

3.3 Multiprocessador de Sinais Digitais

Digital Signal Processor (DSP) consiste de um processador especializado para realizar operações sobre sinais digitais, tais como sinais de áudio, imagens digitais, radares, sonares/ultrassom, sistemas de reconhecimento de voz, telecomunicações, entre outros.

Um *Multiprocessor Digital Signal Processor* (MDSP) é basicamente um circuito integrado composto por CPUs, utilizadas para tratar a entrada/saída de dados, e por DSPs, responsáveis pelo processamento dos dados. Um exemplo de um MDSP com 8 CPUs e 16 DSPs pode ser visto no *datasheet* em (CRADLE, 2021).

3.4 Matriz de Portas Programáveis

Field Programmable Gate Arrays (FPGA) é uma tecnologia de circuitos integrados onde é possível programar seu comportamento físico através de uma linguagem de descrição de hardware (*Hardware Description Language* - HDL). Em linhas gerais, a programação de um FPGA define como os blocos lógicos do circuito integrado serão interligados para definir um determinado comportamento (processo conhecido como *Hardwired Programming*).

Assim, um FPGA pode ser programado em nível de hardware para se comportar de diversas maneiras, como para implementar portas lógicas simples, tais como AND, OR e NOT, e até mesmo compor um sistema computacional completo (*System-on-Chip* - SoC), que pode conter processadores, memórias e barramentos.

A tecnologia FPGA vem ganhando destaque por seu potencial de otimização de tarefas computacionais (em questões de desempenho, é muito mais interessante ter rotinas específicas de computação intensiva implementadas em hardware do que em software), estando presente em diversas aplicações de áudio, automotiva, bioinformática, eletrônicos de consumo, indústria, médica, computação de alto desempenho, segurança, aeroespço, comunicações, computação em nuvem, entre outros.

O campo é tão promissor que as maiores fabricantes de processadores adquiriram as maiores fabricantes de FPGA: em 2015, a Intel adquiriu a empresa Altera por mais de 16 bilhões de dólares (INTEL NEWROOM, 2015) e, mais recentemente, no final de 2020, a AMD adquiriu a empresa Xilinx por aproximadamente 35 bilhões de dólares (AMD, 2020).

3.5 Unidade de Processamento de Visão Computacional

Vision Processing Unit (VPU) é um tipo de processador especializado em aplicações de visão computacional.

Ao contrário das unidades de processamento de vídeo, que são especializadas em codificação e decodificação de streams de áudio/vídeo, as VPUs são especializadas para aplicações de inteligência artificial, tais como redes neurais convolucionais (*Convolutional Neural Network* - CNN), e para localização e descrição de características em imagens (*Scale-Invariant Feature Transform* - SIFT). A combinação dessas duas técnicas permite a extração de informações úteis a partir da captura de vídeo em tempo real.

Essa tecnologia tem sido empregada em robótica, Internet das Coisas (IoT), realidade virtual e realidade aumentada, câmeras inteligentes, veículos autônomos, entre outros. A Microsoft possui um tipo de VPU, batizada de *Holographic Processing Unit* (HPU), que foi produzida especialmente para embarcar a tecnologia *Hololens*, que é um tipo de acessório em formato de óculos inteligentes (*Mixed Reality Smartglasses*), para suportar aplicações de realidade virtual. A HPU da Microsoft conta também com 28 DSPs, que são utilizados para tarefas como mapeamento espacial, reconhecimento de voz, da fala e de gestos (BRIGHT, 2016).

3.6 Acelerador de Inteligência Artificial

Artificial Intelligence Accelerator (AI ACCELERATOR) é um tipo de hardware especializado em aplicações de inteligência artificial, tais como redes neurais artificiais, visão computacional e aprendizado de máquina.

Estudos mostram que ao utilizar GPUs e FPGAs é possível aumentar o desempenho de aplicações de inteligência artificial em relação às CPUs. No entanto, ao utilizar um circuito especializado, esse aumento de desempenho pode chegar à ordem de 10x (MIT, 2016).

A empresa Google produziu um tipo de acelerador de inteligência artificial, chamado de *Tensor Processing Unit* (TPU), que é utilizado para aprendizado de máquina em aplicações em nuvem. Basicamente, a TPU permite acelerar o tempo de computação algébrica linear, além de minimizar o esforço no treinamento de redes neurais grandes e

complexas. Segundo (GOOGLE CLOUD, 2021), o processo de treinamento de redes neurais complexas, que antes levava semanas, foi reduzido a horas com o suporte de TPUs.

3.7 Matriz de Processadores Massivamente Paralelos

Massively Parallel Processor Array (MPPA) é um tipo de processador especializado com alto grau de paralelismo, por incluir vários núcleos independentes de processamento (arquitetura do tipo *Multiple Instructions, Multiple Data* - MIMD).

Em termos gerais, os MPPA se diferenciam dos processadores *multicores* (e.g. processadores comerciais com 2, 4, 8 e 16 núcleos de processamento) por serem dedicados a aplicações puramente paralelas, uma vez que os multicores são otimizados tanto para aplicações sequenciais quanto paralelas. Os MPPA também se diferenciam das GPUs por apresentarem maior desempenho em aplicações com cargas de trabalhos irregulares, já as GPUs se sobressaem em cargas de trabalho regulares (uma GPU possui várias unidades de processamento que aplicam as mesmas operações em dados diferentes, que estão dispostos em vetores ou matrizes - arquitetura do tipo *Single Instruction, Multiple Data* - SIMD) (CARAGEA et al., 2010).

Os MPPAs têm sido amplamente utilizados em supercomputadores. Um ranking com os quinhentos supercomputadores mais rápidos do mundo pode ser conferido em (TOP500, 2021). Por exemplo, o supercomputador chinês *Sunway TaihuLight*, que em novembro de 2017 era o Top 500, é equipado com processadores do tipo Sunway SW26010, que possui 260 núcleos de processamento. Ao todo, o *Sunway TaihuLight* possui 10.649.600 núcleos de processamento (SUNWAY, 2020).

3.8 Processador de Internet das Coisas

A Internet das Coisas (*Internet of Things* - IoT) se refere a uma rede de objetos físicos conectados à Internet, e que são capazes de armazenar e transmitir dados para gerar informação útil.

Os *Internet of Things Processor* (IoT Processor) são projetados para lidar diretamente com os requisitos da Internet das Coisas, tais como: 1) Conectividade, utilizando algum padrão de comunicação sem fio, tal como Wi-Fi, Bluetooth ou telefonia celular; 2) Baixo consumo de energia, pois alguns dispositivos são alimentados por baterias; 3)

Heterogeneidade, para suportar diferentes tipos de dispositivos, aplicações e contextos; 4) Complexidade, para permitir a execução de uma grande variedade de aplicações; 5) escalabilidade, os processadores devem ter baixo custo e ser eficientes quanto ao consumo de energia e tamanho físico, para suportar o crescimento da quantidade de dispositivos e da conectividade entre eles; 6) restrições de tempo-real, pois uma classe de aplicações de IoT está relacionada a tempo real, tais como monitoramento (de pacientes, de rotas de aviões, em sistemas de segurança), diagnósticos médicos, entre outros; e 7) Compartilhamento de recursos, em algumas situações talvez seja necessário distribuir os recursos entre os dispositivos de IoT, em vez de sobrecarregar um deles (ADEGBIJA et al., 2017).

Alguns processadores de IoT são bastante complexos, tal como o Redpine RS9116N-DBT, que inclui submódulos para processamento de dados, conectividade por Wi-Fi, Bluetooth e Zigbee, inteligência artificial, sensores, segurança, entre outros (YOSHIDA, 2019).

3.9 Computador Óptico ou Computador Fotônico

Optical Computer ou *Photonic Computer* são computadores que utilizam fótons (partículas elementares que formam a luz), produzidos por laser ou diodos, para representação de dados.

Os computadores ópticos utilizam o mesmo princípio básico do transistor: a intensidade da luz recebida afeta a intensidade da luz que atravessa os cristais ópticos não lineares, que são os materiais que compõem o “transistor óptico”. Com isso, pode-se produzir portas lógicas, que, por conseguinte, podem formar blocos lógicos mais complexos até compor uma CPU.

Há basicamente dois tipos de computadores ópticos:

1. Aqueles que buscam substituir algumas partes eletrônicas de um computador tradicional por partes ópticas, resultando em um computador híbrido óptico-eletrônico. Essa abordagem é interessante porque habilita, em curto prazo, a produção, comercialização e o uso de computadores ópticos. No entanto, segundo Nolte (2001), a conversão de dados entre os formatos óptico e elétrico, e vice-versa, faz com que o computador perca até 30% de desempenho no consumo de energia e gera atrasos na transferência de dados; e

2. Aqueles que são totalmente ópticos. Os computadores ópticos possuem um grande potencial por permitirem larguras de banda maiores na transferência de dados, em relação aos sistemas eletrônicos, como pode ser constatado no comparativo entre as taxas de transferência obtidas nos meios de comunicação em fibra óptica e em metais (e.g. cabos de par trançado).

Em 2018, o *All-Russian Scientific Research Institute of Experimental Physics* criou um supercomputador óptico híbrido com capacidade de processamento de 50 petaflops (50 quatrilhões de operações de ponto flutuante por segundo), com tamanho físico bastante reduzido e com um consumo de energia de apenas 100 watts. Um computador tradicional, com esse mesmo consumo, só poderia atingir a taxa de 5 teraflops, ou seja, um desempenho 10.000 vezes menor (SPUTNIK, 2018).

3.10 Unidades de Processamento Quântico

A computação quântica se baseia em dois fenômenos físicos: 1) Superposição: é quando dois ou mais estados quânticos (e.g. estados de um elétron), podem coexistir simultaneamente antes de serem medidos; e 2) Entrelaçamento: é quando dois ou mais objetos quânticos estão ligados tão fortemente, que ao se descrever as propriedades de um dos objetos, automaticamente já se possui a descrição das propriedades do outro.

Enquanto um computador tradicional considera os bits para representação de “0” ou “1” e o processamento de dados se dá de forma sequencial, uma unidade de processamento quântico (*Quantum Processor Unit* - QPU) utiliza o “Qubit” (*Quantum Bit*) para armazenar “0” e “1” ao mesmo tempo (sobreposição), e a computação é dada pelo cálculo da probabilidade de um dos dois resultados ocorrer, obtendo-se automaticamente a probabilidade de ocorrência do outro resultado (entrelaçamento).

Os QPUs são produzidos com supercondutores (materiais que não apresentam resistência elétrica quando resfriados a temperaturas próximas do zero absoluto) ou fotônica (utilizam fótons como unidade de informação e instrumentos ópticos para manipulá-los).

Até certo tempo atrás, os computadores quânticos eram apresentados como projetos promissores, com potencial de acelerar computações de classes de problemas bem específicos, porém não estavam totalmente validados. Em 2019, a Google divulgou ter atingido a “Supremacia Quântica”: momento em que um computador quântico conseguiu resolver um problema que um computador tradicional é incapaz ou levaria anos para

solucioná-lo. Segundo artigo publicado no *Financial Times* (MURGIA; WATERS, 2019), o computador quântico “Sycamore” conseguiu realizar, em apenas 3 minutos e 20 segundos, uma tarefa que o computador mais rápido da época, o supercomputador IBM Summit, levaria 10 mil anos para completá-la.

Os computadores quânticos cada vez mais estão se tornando realidade. Como são os casos: do IBM Q System One, que foi o primeiro computador quântico comercial; da linguagem de programação Microsoft Q#, que permite criar aplicações para computadores quânticos a partir de computadores tradicionais; e os *IBM Quantum Composer* e *IBM Quantum Lab*, que formam uma plataforma em nuvem que permite acesso público a serviços de computação quântica e a diversos materiais didáticos.

3.11 Computadores Biológicos ou Computadores Genéticos ou Computadores de DNA

Biocomputers ou *Genetic computers* ou *DNA computers* são computadores que utilizam moléculas, tais como proteínas ou conteúdo genético de seres vivos (DNA), para realizar armazenamento e processamento de dados.

Em termos gerais, os materiais biológicos podem ser manipulados para se comportarem de uma determinada maneira, com base em condições de entrada (estímulos tais como mudanças em condições ambientais ou inserção de compostos químicos) para, através das reações, produzir um determinado resultado. Este processo pode ser interpretado como uma tipo de análise computacional (FREITAS, 1999).

Em 2012, cientistas do Instituto Federal de Tecnologia de Zurich e da Universidade de Basel conseguiram organizar células humanas em diferentes disposições tridimensionais para realizar determinados tipos de processamento. As células utilizam duas moléculas, a eritromicina, que é um tipo de antibiótico, e a phloretin, uma substância encontrada em macieiras, como entradas para ativar uma reação dentro das células. A reação leva à produção de uma proteína fluorescente de cor vermelha ou verde, que sinalizam o resultado binário do cálculo (AUSLÄNDER et al., 2012).

Segundo os autores desse estudo, as reações não interferem nas ações naturais das células, sendo possível estabelecer a linguagem natural binária dos computadores, sem afetá-las. No estudo, foi possível realizar, com as células, operações lógicas, tais como NOT, AND, NAND, XOR e N-IMPLY (negação da implicação), e com o uso de uma lógica

combinacional com três dessas portas lógicas, foi possível implementar circuitos somadores e subtratores de 1-bit.

Já em 2013, pesquisadores do Instituto Europeu de Bioinformática demonstraram ser capazes de armazenar um grande volume de dados, mais especificamente, 700 Terabytes ou o equivalente a quase meio milhão de DVDs, em apenas 1 grama de DNA (GOLDMAN et al., 2013).

3.12 Uma Breve Análise da História do Computador

As páginas do capítulo anterior e deste mostram que, ao longo dos anos, surgiram muitas pesquisas científicas, em diversos campos do conhecimento, que levaram aos avanços tecnológicos do computador. Naturalmente, há, atualmente, muitos outros estudos científicos sendo desenvolvidos em paralelo, como os que visam evoluir os projetos de computadores com as tecnologias atuais e os que buscam experimentar tecnologias alternativas, promovendo a quebra de paradigmas.

Iniciando com o primeiro instrumento somador, as varas de contagem (20.000 a.c); seguindo pela primeira calculadora mecânica, o Relógio Calculador de Schickard (1623); pelo somador de Stibitz, criado, na cozinha de sua casa, com relés eletromecânicos (1937); pelo ENIAC, o primeiro computador eletrônico, produzido por John Eckert e John Mauchly utilizando válvulas eletrônicas (1945); o primeiro computador transistorizado, criado na Universidade de Manchester por Tom Kilburn (1953); o primeiro processador completo totalmente embarcado um único circuito integrado, o microprocessador 4004, produzido pela Intel, a partir dos conceitos de Hoff (1971); o somador celular, criado pelo Instituto Federal de Tecnologia de Zurich e da Universidade de Basel, utilizando técnicas de computação biológica (2012); o supercomputador óptico, criado na All-Russian Scientific Research Institute of Experimental Physics (2018); e, finalmente, a Supremacia Quântica, anunciada pela Google (2019); percebe-se que, ao longo de vários anos, o computador foi, e continua, sendo aperfeiçoado e reinventado.

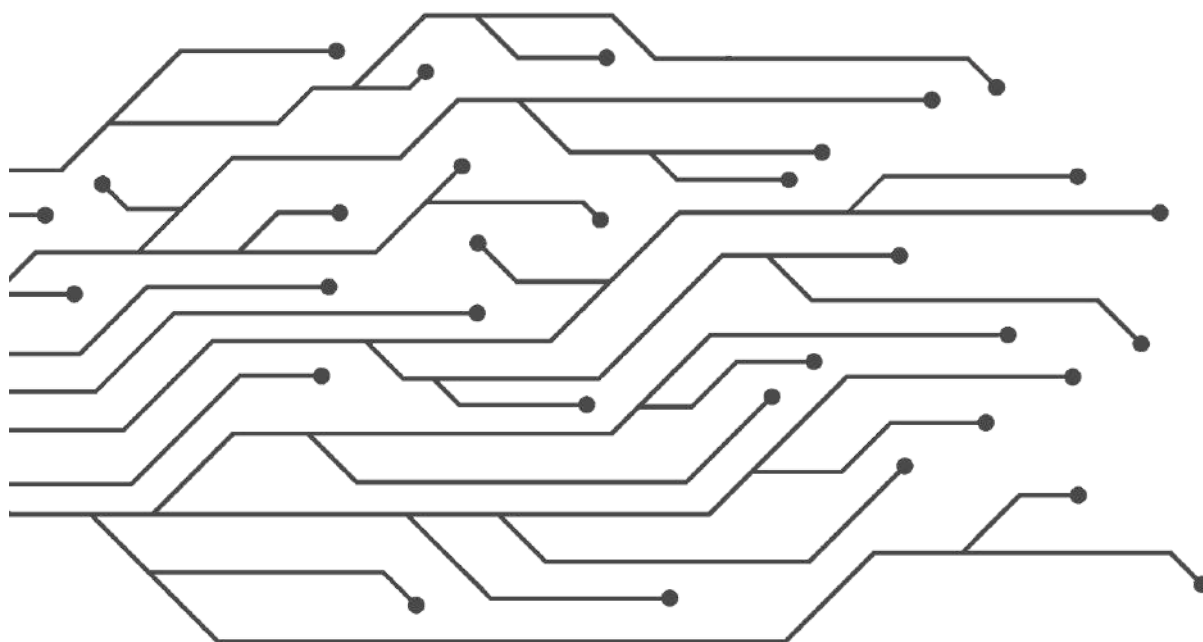
É praticamente impossível imaginar o ponto final no avanço tecnológico do computador. Entretanto, enquanto houver imaginação, pode-se definir os próximos passos e o sentido nessa direção.

3.13 Resumo

Este capítulo trouxe uma breve discussão sobre o possível fim da Lei de Moore, e, por consequência, o fim de uma era de avanços tecnológicos.

Os impactos na indústria de computadores já foram previstos, de forma que novos esforços já vêm sendo empregados em diversos campos científicos, aqui também apresentados, para continuar suprimindo as demandas de mais recursos computacionais, as novas demandas do mercado e/ou para a reinvenção do computador.

PARTE II - CONCEITOS EM ORGANIZAÇÃO E ARQUITETURA DE COMPUTADORES



CAPÍTULO 4 – INTRODUÇÃO À ORGANIZAÇÃO DE COMPUTADORES

Um computador é um dispositivo eletrônico que desempenha quatro funções principais: processamento, armazenamento e transferência de dados; e, controle de operações, que envolve a coordenação das demais funções para permitir que os computadores realizem tarefas variadas (STALLINGS, 2010).

Um computador é dividido em diferentes subsistemas, ou componentes, onde cada um deles possui uma estrutura e comportamento bem definidos. Assim, esses componentes devem se comunicar para trocar informações visando compor e realizar as funções do computador.

As subseções a seguir trazem os conceitos básicos relacionados aos principais componentes do computador, que são Processador, Memória Principal, Memória Cache, Barramento e Subsistema de Entrada/Saída; assim como, para fins de estudos de caso, fazem paralelos com os respectivos componentes da plataforma de hardware virtual do CompSim.

4.1 Processador

O processador é o elemento central de qualquer computador. Ele é responsável por buscar instruções na memória, decodificá-las para compreender as tarefas que devem ser realizadas e executá-las (TANENBAUM; AUSTIN, 2013). Além disso, ele interage com todos os componentes do computador, dentre eles a memória e os periféricos.

O processador é o componente mais complexo do computador e, portanto, possui diversos subsistemas. Dentre seus subsistemas, destacam-se o Caminho de Dados e a Unidade de Controle.

O Caminho de Dados (também conhecido por *Datapath*) contém a Unidade Lógica e Aritmética - ULA (*Arithmetic and Logic Unit* - ALU) e os Registradores (ABD-EL-BARR; EL-REWINI, 2005).

A ULA é a parte do processador que é responsável por realizar o processamento de dados, tais como as operações aritméticas (adição, subtração, divisão, multiplicação, incremento, decremento, módulo, etc.) e as operações lógicas (AND, OR, NOT, XOR, etc.) (STALLINGS, 2010). A ULA é um circuito do tipo combinacional: ao receber dados de entrada, imediatamente processa-os e devolve um resultado.

Um registrador é uma unidade de memória temporária interna do processador. Os registradores podem ser de: 1) Propósito geral, que podem ser utilizados com diferentes finalidades, como para armazenamento temporário de dados, em operações aritméticas e lógicas, transferência de dados entre processador, memória e periféricos; ou, 2) Propósito específico, que são utilizados exclusivamente para uma única finalidade, como, por exemplo, para interfaceamento com a memória e com periféricos.

Dentre os registradores de propósito específico, os mais importantes são o registrador Contador de Programa (*Program Counter* - PC), que é utilizado para indicar o endereço de memória da próxima instrução que será buscada para execução, e o Registrador de Instrução (*Instruction Register* - IR), que guarda a instrução que está em execução (TANENBAUM; AUSTIN, 2013). Além desses, há outros tipos de registradores, tais como: 1) para endereçamento de dados em memória; 2) transferência de dados entre processador e memória; 3) endereçamento de periféricos; 4) transferência de dados entre processador e periféricos; 5) apontadores de segmentos (um programa de computador, quando carregado na memória, pode ser dividido em segmentos lógicos, que guardam instruções, dados e a pilha do programa); 6) apontador de topo da pilha do programa; 7) de indexação, que são utilizados para indexar posições consecutivas em memória; e 8) de controle de condição (ou de *Flags*), que informam os status resultantes das operações do processador.

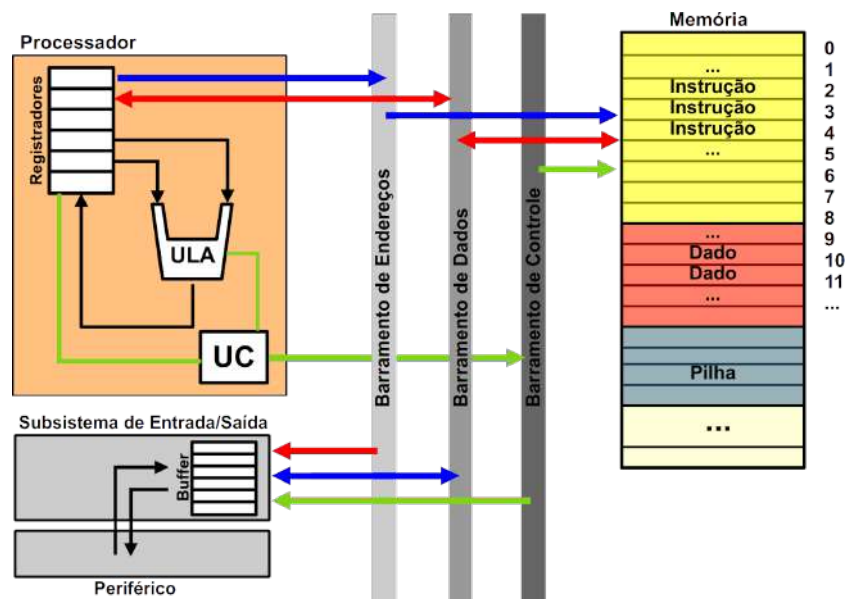
Há muitos tipos de processadores, então é natural que nem todos incluam todos esses tipos de registradores. Também não é difícil de encontrar processadores que incluem uma grande quantidade de registradores e de diferentes tipos.

A Unidade de Controle (UC) contém toda a lógica de execução do processador, sendo responsável por gerenciar as operações de processamento e transferência de dados no *Datapath*, bem como as operações de transferência de dados com memória e periféricos (PATTERSON; HENNESSY, 2017). A lógica de execução do processador é chamada de Ciclo de Instrução, que consiste em uma sequência de passos que o processador deve realizar

para desempenhar suas operações. Dentre esses passos, estão: a busca de instrução na memória, decodificação da instrução, busca de operandos, execução da instrução e gravação do resultado. O ciclo de instrução será detalhado no [Capítulo 5 - Introdução à Arquitetura de Computadores](#).

A Figura 4.1 apresenta um diagrama com os principais componentes do processador e sua interconexão com a memória e um periférico.

Figura 4.1. Diagrama de componentes do processador e suas interconexões.



Fonte: Próprio autor (2021).

Na Figura 4.1 é possível visualizar que:

- No topo à esquerda, o Processador é composto pelos Registradores, Unidade Lógica e Aritmética (ULA) e Unidade de Controle (UC);
- A UC coordena, através das linhas em cor verde, as operações da ULA, o uso dos Registradores e a comunicação entre Processador, Memória e Periférico;
- A UC também coordena as operações de endereçamento de Memória e de Periférico (ilustradas por setas em cor azul) e de transferências de dados (ilustradas por setas em cor vermelha) entre Processador-Memória e Processador-Periférico;
- A ULA recebe seus operandos a partir de dois Registradores e, após o processamento dos dados, grava o resultado em outro Registrador;
- Uma estrutura de comunicação, chamada de Barramento, que é composto pelos Barramentos de Endereços, de Dados e de Controle, é utilizado em operações de transferência de dados entre Processador, Memória e Periférico;

- A Memória está dividida logicamente em três segmentos, sendo que o primeiro inclui as instruções do programa, o segundo inclui os dados e o terceiro inclui a pilha do programa; e
- O Subsistema de Entrada/Saída faz a interface entre o Periférico com Processador e Memória, para transferências de dados através do Barramento.

4.1.1 Processador Cariri

O simulador CompSim conta com um processador conceitual, chamado de “Cariri”, que apresenta as principais características de processadores reais. Dentre suas características, pode-se destacar:

- Possui arquitetura de 16-bits baseada em acumulador, onde, na maioria das suas operações, utiliza-se implicitamente o registrador de propósito geral, chamado Acumulador. Por exemplo, em uma operação de adição, que necessita de dois operandos, a instrução deve referenciar apenas um dos operandos, pois considera-se que o outro operando já está contido no Acumulador. Arquiteturas baseadas em acumulador são mais simples de projetar e programar, pois as instruções do processador possuem tamanhos e complexidade reduzidos (NULL; LOBUR, 2009);
- Possui uma Unidade Lógica e Aritmética (ULA), responsável por realizar o processamento de dados;
- Possui uma Unidade de Controle (UC), que é responsável por coordenar todas as operações do processador;
- Possui um circuito Contador (*Counter*), que é um componente utilizado para incrementar automaticamente os endereços das instruções, contidos no registrador Contador de Programa (*Program Counter* - PC), que serão buscadas pelo processador na memória principal. Com o suporte do Contador, as instruções do programa são buscadas e executadas de forma sequencial;
- Conta com um Banco de Registradores, para suportar diferentes ações do processador, tais como: busca de instrução na memória e decodificação; operações lógicas e aritméticas; operações de entrada/saída; de acesso à memória e à pilha de programa, para acesso e armazenamento de dados, entre outras;
- Possui espaço de endereçamento diferenciado para operações de entrada/saída e de acesso à memória principal (este tipo de espaço de endereçamento é conhecido como

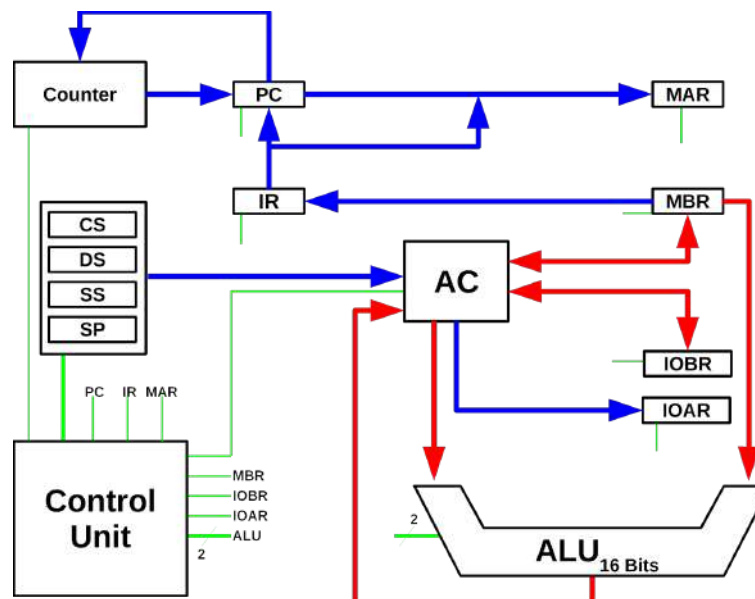
Port-Mapped). Em alguns processadores, há apenas um espaço de endereços para acesso à memória e aos periféricos (conhecido como *Memory-Mapped*), como no diagrama ilustrado na Figura 4.1. No caso do CompSim, há dois espaços de endereços, sendo um deles utilizado exclusivamente para acesso a cada uma das posições da memória principal, e outro para acessar cada um dos periféricos;

- Conta com 16 instruções de baixo nível, para diferentes operações tais como: transferência de dados, operações aritméticas e lógicas, controle de fluxo de programa e de sistema. As instruções são de 16-bits, sendo que 4-bits são reservados para o código de operação, ou seja, determinam a operação da instrução, e os demais 12-bits são reservados para o operando da instrução;
- Suporta dois tipos de operandos: inteiro de 16-bits com sinalização (*signed int*) e caractere (*char* ou *byte*);
- Suporta os modos de endereçamento de dados:
 - imediato: onde o operando está na própria instrução;
 - direto: onde o operando pode ser acessado pelo endereço de memória na instrução (há um acesso à memória para busca do operando);
 - indireto: a instrução contém um endereço de memória que aponta para um outro endereço de memória, que é o endereço do operando (há dois acessos à memória para busca do operando);
 - registrador: o operando está contido em um registrador;
 - implícito: não há necessidade de informar onde está o operando.

A Figura 4.2 mostra um diagrama de blocos do processador Cariri, onde pode-se visualizar o caminho de dados (*Datapath*) e a unidade de controle.

Na Figura 4.2 observa-se a Unidade de Controle (“Control Unit”), o Contador de Instruções (“Counter”) e as conexões entre a Unidade Lógica e Aritmética (“ALU”) e os registradores. Na figura, as linhas: em cor verde consistem dos sinais de controle da Unidade de Controle; em cor azul consistem dos sinais de endereçamento de memória e de periféricos; e em vermelho consistem dos sinais por onde os dados são transferidos.

Figura 4.2. Diagrama de blocos do Processador Cariri.



Fonte: Próprio autor (2021).

Os registradores apresentados na Figura 4.2 são:

- De propósito geral (*Acumulador - AC*): utilizado em operações de transferência e de processamento de dados;
- Contador de programa (*Program Counter - PC*): inclui o endereço de memória da próxima instrução que será executada;
- Decodificação de instruções (*Instruction Register - IR*): guarda a instrução em execução;
- Endereçamento à memória (*Memory Address Register - MAR*): em qualquer operação de transferência de dados entre processador e memória, este registrador é utilizado para endereçamento de memória;
- Transferência de dados da memória (*Memory Buffer Register - MBR*): em qualquer operação de transferência de dados entre processador e memória, este registrador é utilizado para armazenar o dado transferido;
- Endereçamento de Entrada/Saída (*Input/Output Address Register - IOAR*): em qualquer operação de transferência de dados entre processador e periféricos, este registrador é utilizado para endereçamento de um periférico;
- Transferência de dados de Entrada/Saída (*Input/Output Buffer Register - IOBR*): em qualquer operação de transferência de dados entre processador e periféricos, este registrador é utilizado para armazenar o dado transferido;

- Endereçamento de segmento de código (*Code Segment* - CS): aponta para o segmento lógico do programa na memória que contém as instruções do programa;
- Endereçamento de segmento de dados (*Data Segment* - DS): aponta para o segmento lógico do programa na memória que contém os dados do programa;
- Endereçamento de segmento de pilha (*Stack Segment* - SS): aponta para o início do segmento lógico do programa na memória que contém a pilha do programa; e
- Apontador de topo de pilha (*Stack Pointer* - SP): aponta para uma posição de memória referente ao topo da pilha do programa.

4.2 Memória Principal (RAM)

A memória principal (ou *Random Access Memory* - RAM) é o componente do computador onde as instruções e os dados de um programa em execução estão temporariamente armazenados (costuma-se denominar a RAM como “memória de trabalho”). Para a execução de um programa, o processador deve ler instruções e trocar dados com a memória RAM (STALLINGS, 2010).

A memória RAM permite armazenar palavras. O termo “palavra” (“word”) refere-se a um conjunto de bits ou bytes, que consiste da unidade de informação que pode ser armazenada, transmitida e/ou processada em um computador (STALLINGS, 2010).

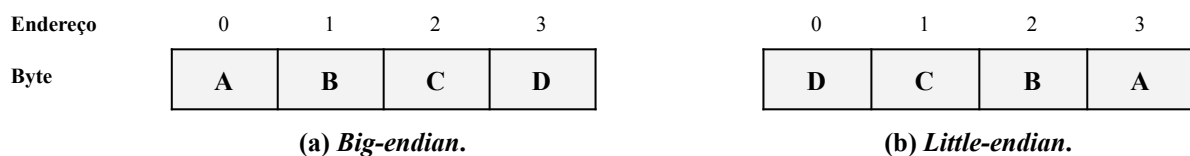
As memórias RAM frequentemente são organizadas em matrizes, que não necessariamente precisam ser quadradas, onde cada uma de suas células é utilizada para armazenamento de dados (TANENBAUM; AUSTIN, 2013). Desta forma, um endereço de memória RAM é dividido em duas partes, sendo uma para seleção da linha e a outra para seleção da coluna da matriz de dados, de forma a tornar disponível uma célula de armazenamento para leitura ou escrita de dados.

Os computadores, ao longo dos anos, utilizaram diferentes modelos de instruções para caracterizar seus conjuntos de instruções. Isso quer dizer que há computadores em que todas as instruções possuem o mesmo tamanho (tamanho fixo), que normalmente, é do tamanho de uma palavra de memória (e.g. 16-bits). Neste caso, por exemplo, há memórias produzidas para armazenar palavras de 16-bits (2 bytes). Já outros computadores incluem instruções com diferentes tamanhos (e.g. instruções de 16-bits, 32-bits e 64-bits), característica bastante comum nos processadores mais modernos. Assim, para permitir a compatibilidade das memórias RAM com processadores com instruções com tamanhos variados, as memórias

RAM passaram a armazenar e endereçar bytes, tornando-se assim, um padrão de indústria. Desta forma, os processadores poderão realizar leituras e escritas de palavras com 1, 2 ou mais bytes em memória RAM.

Um ponto a ser observado, sobre palavras formadas por múltiplos bytes endereçáveis em memória, é a ordem em que os bytes podem ser organizados em memória. Esse conceito é conhecido como *Endianess*. Basicamente, há dois modos de ordenação dos bytes: 1) *Big-endian*, em que o byte mais significativo da palavra é armazenado na menor posição endereçável da palavra; e 2) *Little-endian*, em que o byte menos significativo da palavra é armazenado na menor posição endereçável da palavra. Considerando, por exemplo, uma palavra de 32-bits formada pelos bytes “ABCD”, a Figura 4.3 mostra como ela poderia ser armazenada em memória na ordem *Big-endian*, em (a), e *Little-endian*, em (b).

Figura 4.3. Modos de ordenação de byte de uma palavra em memória.

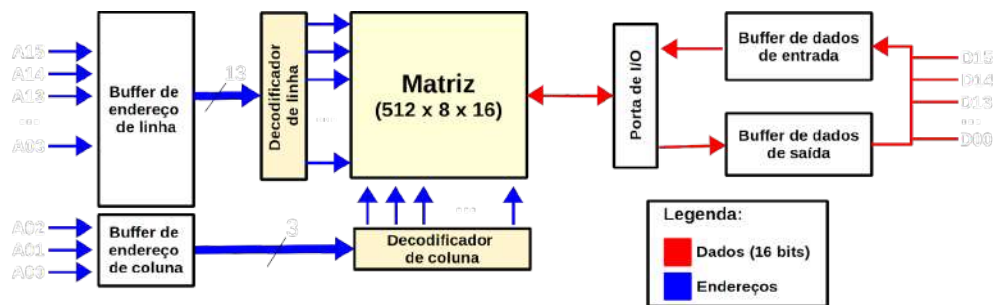


Fonte: Próprio autor (2021).

Na memória principal do CompSim, o modelo de endereçamento proposto utiliza endereços de 12-bits, sendo que os 9 bits mais significativos são utilizados para endereçamento de linha e os 3-bits menos significativos para endereçamento de coluna. Com essa configuração, é possível endereçar até 512 (2^9) linhas de memória, onde cada linha possui 8 (2^3) colunas, totalizando assim 4096 (512×8) diferentes endereços. Como a memória do CompSim armazena palavras de 16-bits, sua capacidade de armazenamento total é de 65.536 bits ($512 \times 8 \times 16$ -bits), ou simplesmente 8 KBytes. A Figura 4.4 apresenta um diagrama da organização da memória RAM do CompSim.

Na Figura 4.4, observa-se, ao centro, a Matriz de dados, organizada em 512 linhas, 8 colunas e palavras de 16-bits (notação “512 x 8 x 16”); observa-se à esquerda, os sinais de endereço, onde os bits em A03 a A11 são utilizados para endereçar uma linha e os sinais em A00 a A02 endereçam uma coluna da matriz de dados; e, observa-se à direita, o barramento de dados, exposto pelos sinais em D00 à D15, onde os dados e instruções de 16-bits são transferidos de/para a memória RAM.

Figura 4.4. Organização da memória RAM do CompSim.



Fonte: Próprio autor (2021).

Ao considerar o modo de ordenação de bits de uma palavra de 16-bits na memória principal do CompSim, é importante observar a função dos componentes detalhados nos tópicos a seguir.

4.3 Memória Cache

Na execução de um programa, o processador busca instruções e dados na memória RAM. Algumas instruções, por exemplo, necessitam realizar mais acessos à memória para busca dos operandos. E, considerando que um acesso à memória RAM leva muito tempo (consome vários ciclos de relógio de sistema), comparado à taxa de execução do processador, percebe-se que os acessos podem degradar o desempenho do processador e, por consequência, do sistema. Em resumo, o processador atua em uma taxa de execução muito mais alta do que a taxa de entrega de dados da memória RAM, caracterizando assim um modelo de comunicação frágil (muitos autores utilizam o termo “gargalo de comunicação”) e que tem grande impacto no desempenho do sistema (MONTEIRO, 2007).

Buscando otimizar o desempenho de comunicação entre Processador e Memória RAM, os projetistas de computadores criaram um tipo de memória de armazenamento temporário, localizada entre o processador e a memória RAM e que é fabricada com a mesma tecnologia do processador (transistores), chamada de Memória Cache. A memória cache tem como função acelerar a entrega de instruções e dados, minimizando assim os ciclos de espera pelo processador.

A memória cache possui uma organização diferente da memória RAM. Ela foi criada para usufruir das propriedades de Localidade Espacial e Localidade Temporal. Quando um programa está em execução, existe uma probabilidade alta de que, em um determinado bloco de instruções, as instruções sejam executadas em sequência. Ou seja, quando o processador

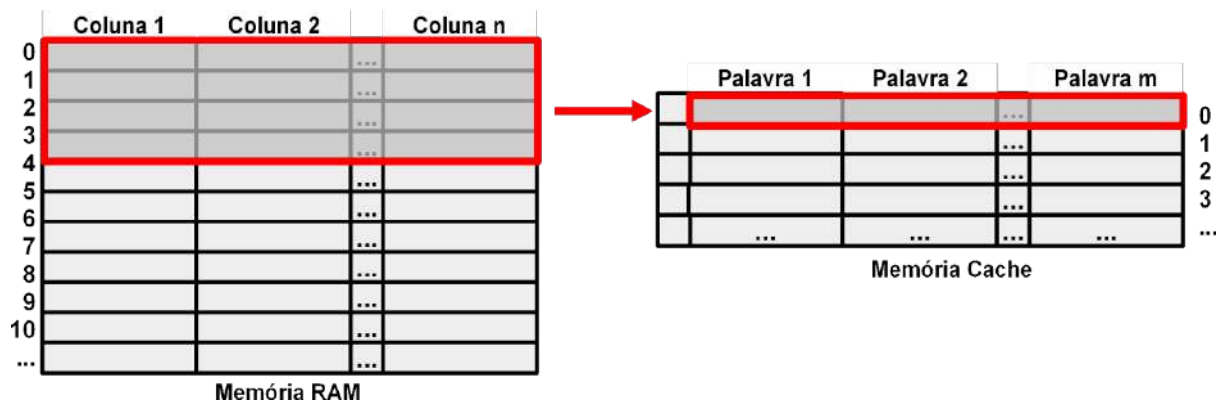
está executando uma determinada instrução, existe uma probabilidade alta de que a próxima instrução a ser executada seja a instrução subsequente à atual. Essa propriedade é conhecida por Localidade Espacial. Em um outro cenário, considerando que há um bloco de instruções e/ou dados que fazem parte de uma construção iterativa, como um bloco WHILE..DO ou FOR..DO, em linguagens de alto nível, há uma probabilidade alta de que as instruções/dados serão acessados repetidamente. Essa segunda propriedade é conhecida por Localidade Temporal.

Assim, a memória cache, que considera as propriedades de Localidade Espacial e Temporal, guarda as instruções e dados com alta probabilidade de acesso pelo processador, deixando o restante do programa, que não está sendo utilizado naquele momento, na memória RAM. É importante ressaltar que a memória cache, por ter um processo de fabricação e tecnologias de maior custo do que as da memória RAM, possui menor capacidade de armazenamento. Desta forma, de um lado, a memória cache entrega instruções e dados mais rapidamente ao processador, uma vez que é fabricada com a mesma tecnologia. Por outro lado, ela se comunica de forma mais lenta, porém mais robusta, com a memória RAM, transferindo e armazenando uma parte das instruções e dados do programa (blocos da memória RAM), quando um determinado dado é solicitado pelo processador, buscando manter assim as propriedades de Localidade Espacial e Temporal.

Quando o processador busca uma instrução ou dado, é realizado um acesso à memória cache. Se a informação solicitada estiver disponível na cache, isso é caracterizado como um Acerto (*Hit*). A informação é então transferida em alta velocidade ao processador. Caso a informação não esteja disponível na cache, isso será caracterizado como uma Falta (*Miss*) e desencadeará uma série de operações: a execução do programa é bloqueada, a memória cache fará acesso à memória RAM pra trazer e armazenar um bloco de palavras que contenha a informação solicitada pelo processador, em seguida, será caracterizado nesse momento um *Hit* e a informação solicitada será transferida em alta velocidade para o processador. Para que haja aumento de desempenho do sistema pelo uso de memória cache, o número de *Hits* deve ser superior ao número de *Misses*. Segundo Monteiro (2007), na maioria dos casos, a taxa de *Hits* varia entre 80% e 99%, o que comprova o benefício do uso de memórias cache.

A memória cache tem uma organização diferente da memória RAM. Enquanto que a memória RAM inclui uma matriz para armazenamento de dados, a memória cache armazena os blocos transferidos da memória RAM em linhas, como mostra a Figura 4.5. Na figura, um bloco de dados da memória RAM pode conter palavras distribuídas em diversas linhas e colunas. Quando transferido para a memória cache, o bloco ocupará uma única linha.

Figura 4.5. Transferência de bloco de dados da memória RAM para memória Cache.



Fonte: Próprio autor (2021).

Como a capacidade de armazenamento de blocos da memória cache é bastante inferior à da memória RAM, cada linha da cache pode ser utilizada para armazenar diferentes blocos, em momentos distintos. A abordagem de escolha de armazenamento de determinado bloco de memória RAM em uma determinada linha da memória cache se chama Mapeamento. Há diferentes técnicas de mapeamento, das quais pode-se destacar três:

1. Direto: nesta abordagem, define-se previamente que um conjunto de blocos poderão ser armazenados exclusivamente em uma determinada linha da cache. Por exemplo, os blocos 1 e 3 da memória RAM poderão ser armazenados na linha 1 e os blocos 2 e 4 poderão ser armazenados na linha 2 da memória cache. Esta técnica é a mais simples de implementar e gera pouco atraso para o cálculo do endereço de linha onde o bloco transferido da memória RAM será armazenado. No entanto, por conta do endereçamento fixo dos blocos, caso haja necessidade de uso de blocos distintos mapeados em uma mesma linha, podem ocorrer substituições sucessivas. No exemplo anterior, supondo que há necessidade de acesso a informações contidas nos blocos 1 e 3, a linha 1 da memória cache não poderá armazená-los ao mesmo tempo, fazendo com que um bloco seja substituído pelo outro sob demanda. O número de substituições sucessivas em excesso, causado por uma alta taxa de *Misses*, é um fenômeno conhecido por *Thrashing*;
2. Associativo: ao contrário da abordagem anterior, não há uma ordem predefinida para armazenamento de blocos nas linhas da memória cache. Cada linha pode armazenar qualquer bloco da memória RAM. Embora essa técnica não apresente o problema de endereçamento fixo, presente na técnica de mapeamento direto, para obter

mapeamento associativo deve-se utilizar uma lógica mais complexa, visto que é necessário acessar rapidamente todas as linhas da cache para verificar se a informação solicitada pelo processador está presente na cache; e

3. Associativo por Conjunto: esta abordagem combina práticas das técnicas anteriores, para minimizar seus problemas (endereçamento fixo e mecanismo complexo para busca de informação). Nesta técnica, as linhas da memória cache são divididas em grupos (ou conjuntos), onde cada conjunto tem comportamento associativo (pode armazenar qualquer bloco da memória RAM). Dessa forma, determinados blocos têm mapeamento fixo em um dado conjunto, porém, dentro desse conjunto, os blocos podem ser armazenados em qualquer uma de suas linhas.

Para maximizar o desempenho do sistema pelo uso de memórias cache, é importante que todas as linhas da memória cache estejam ocupadas com blocos mapeados da memória RAM. Porém, dependendo do programa em execução, haverá necessidade de retirar blocos armazenados na memória cache para ceder espaço para novos blocos que estão sendo demandados (é importante lembrar que a capacidade de armazenamento da memória cache é bastante inferior à da memória RAM). Quando se utiliza mapeamento direto, já se tem, *a priori*, a informação de qual bloco será retirado da memória cache, visto que o novo bloco somente pode ser armazenado em uma linha predeterminada. Já para os mapeamentos associativo e associativo por conjunto, pode-se utilizar diferentes algoritmos de substituição, tais como:

- Menos Recentemente Usado (*Least Recently Used* - LRU): será escolhido para substituição do bloco que foi acessado a mais tempo dentre os demais;
- Menos Frequentemente Usado (*Least Frequently Used* - LFU): será escolhido para substituição o bloco que menos acessado dentre os demais;
- Fila Circular (*Round Robin* ou *First-In First-Out* - FIFO): será escolhido para substituição o bloco que foi armazenado anteriormente aos demais, seguindo a ordem de fila, independentemente do tempo ou frequência de acesso; e
- Aleatório (*Random*): será escolhido aleatoriamente para substituição um dos blocos armazenados na cache.

Alguns sistemas computacionais, incluem mais de uma memória cache (cache de nível 1 - *Level 1* ou *L1*, cache de nível 2 - *Level 2* ou *L2* e cache de nível 3 - *Level 3* ou *L3*), atualmente, todas embutidas no próprio processador. Em uma hierarquia de memórias caches, cada nível possui uma capacidade de armazenamento diferente, sendo que o nível 1 contém a

menor e o nível 3 a maior capacidade ($L1 < L2 < L3$), e são utilizados de forma semelhante e com o mesmo objetivo do modelo de comunicação processador-cache-RAM.

Nos computadores mais modernos, o nível 1 é dividido em duas caches, sendo então uma memória cache exclusivamente para dados (*L1 Data Cache*) e uma para instruções (*L1 Instruction Cache*). Essa divisão torna o circuito mais simples, uma vez que a cache de instruções será utilizada apenas para leitura (as instruções armazenadas na cache não são modificadas). Já na cache de dados, ao modificar um determinado dado, o mesmo deve ser atualizado, nos níveis de cache seguintes (L2 e L3) e na memória RAM, de forma a manter a coerência da informação. O processo de manter os dados atualizados entre as memórias caches e RAM é conhecido por Coerência de Memória.

Curiosidade!

Há ainda o termo Coerência de Cache, que é mais comumente utilizado em sistemas com múltiplos processadores. Nesses sistemas, cada processador possui suas próprias memórias cache. Quando há comunicação entre tarefas que executam em processadores distintos, as caches são utilizadas para armazenar os dados transferidos entre tarefas. Quando um dado compartilhado entre tarefas é modificado por uma delas, é necessário que essa modificação seja reproduzida entre os respectivos dados que estão armazenados nas memórias cache de cada um dos processadores envolvidos na comunicação.

Para manter a coerência entre as memórias cache e RAM, deve-se utilizar alguma Política de Escrita (ou Política de Atualização) nas memórias cache. As políticas de escrita podem ser divididas em dois grupos (CRAGON, 1996):

1. Escrita com acerto (*Write Hit*): quando um dado é modificado pelo processador e o respectivo bloco já encontra-se mapeado na memória cache (*Hit*). Neste grupo há as seguintes políticas de escrita:
 - a. *Write-Through*: o dado modificado é escrito tanto na memória cache quanto na memória RAM. Para aumentar o desempenho da abordagem, pode-se utilizar um *buffer* para escrita do dado na memória RAM, liberando o processador para realizar outras operações enquanto a escrita está pendente;
 - b. *Write-Back*: o dado modificado é escrito apenas na memória cache. Caso o bloco onde consta o dado atualizado necessite ser substituído, o bloco será escrito na memória RAM. É possível utilizar um mecanismo, chamado de *Dirty Bit*, para aumentar o desempenho da abordagem. O *dirty bit* marca os

dados modificados em um bloco armazenado na memória cache. Com isso, caso o bloco necessite ser substituído, somente os dados marcados serão copiados para a memória RAM. Uma desvantagem desta abordagem é que a memória RAM ficará desatualizada até que o bloco ou o dado modificado seja atualizado;

- c. *Write-Once*: esta política combina as anteriores da seguinte forma: quando há mais de uma escrita em um mesmo bloco, a primeira escrita é realizada utilizando a política *Write-Through* e as demais utilizando *Write-Back*, buscando reduzir o tráfego de comunicação nas escritas consecutivas para manter a coerência dos dados.
2. Escrita com falta (*Write Miss*): não encontra-se mapeado na memória cache (*Miss*) o bloco referente ao dado modificado. Neste grupo há as seguintes políticas de escrita:
- a. *Write-Allocate*: o bloco é mapeado da memória RAM para a memória cache e, em seguida, o dado modificado é escrito na cache. A partir deste ponto, pode-se utilizar alguma política de escrita em acerto, tal como *Write-Through*, *Write-Back* ou *Write-Once*.
 - b. *Write-Around* (*No-Write-Allocate* ou *Write-Direct*): nesta política o dado é escrito diretamente na memória ou em um *buffer* de escrita na memória RAM, sem realizar o mapeamento do respectivo bloco em memória cache.

No CompSim, a plataforma Mandacaru inclui um componente de memória cache, com as seguintes características:

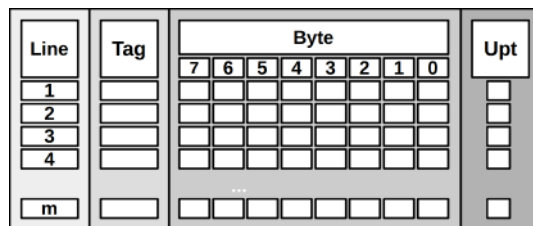
- Número de linhas parametrizável: é possível configurar o número total de linhas da memória cache.
- Bloco de 8 palavras: cada linha poderá armazenar 8 palavras. A memória RAM do CompSim armazena 8 palavras de 16-bits por linha da sua matriz de dados. Assim, uma linha da memória cache poderá armazenar os dados contidos em uma linha da memória RAM;
- Técnicas de mapeamento: são suportadas as técnicas de mapeamento Direto, Associativo e Associativo por Conjunto;
- Algoritmos de substituição: são suportadas as técnicas de substituição Menos Recentemente Usado (LRU), Fila Circular (FIFO) e Aleatório (*Random*).
- Políticas de Escrita: são suportadas as políticas de escrita com acerto *Write-Through* e *Write-Back*, e as políticas de escrita com falta *Write-Allocate* e *Write-Around*.

A Figura 4.6 ilustra as organizações da memória cache sob diferentes técnicas de mapeamento, sendo em (a) Direto, em (b) Associativo e em (c) Associativo por Conjunto. Na figura observa-se que a técnica de mapeamento utilizada impacta na complexidade da organização da memória cache.

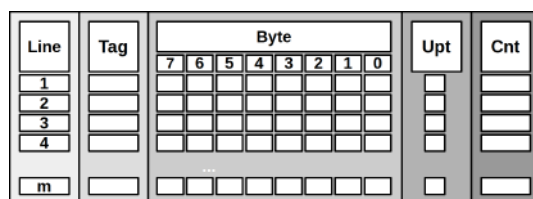
Já a organização da memória cache, por sua vez, define a forma como um endereço de acesso à memória, informado pelo processador, será subdividido para endereçar uma determinada palavra na memória cache. No CompSim, por exemplo, um endereço de memória de 12-bits, ao utilizar o mapeamento:

- Direto: será subdividido em 3 campos, que são: 1) “Line”: identifica em que linha da memória cache pode ser armazenado o bloco referente à palavra solicitada; 2) “Tag”: identifica qual bloco está mapeado em uma determinada linha da memória cache; e 3) “Byte”: identifica a palavra solicitada, caso o bloco esteja mapeado em uma determinada linha da memória cache. Na Figura 4.6 (a), há ainda o campo “Upt” (“Update”), na memória cache com mapeamento direto. Este campo informa a ocorrência de modificações em uma ou mais palavras da linha da cache, para que a linha, antes de ser substituída, seja atualizada na memória RAM (este campo é utilizado pelas políticas de escrita com acerto *Write-Back* e *Write-Once*);

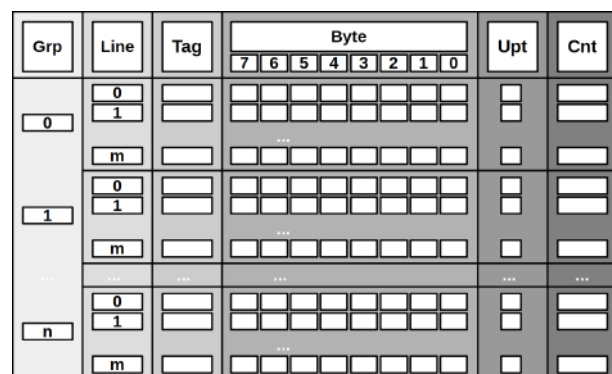
Figura 4.6. Organização de memória Cache com diferentes técnicas de mapeamento.



(a) Organização com mapeamento direto.



(b) Organização com mapeamento associativo.



(c) Organização com mapeamento associativo por conjunto.

Fonte: Próprio autor (2021).

- Associativo: será subdividido em 2 campos, que são: 1) “Tag”: uma vez que qualquer bloco da memória RAM pode ser mapeado em qualquer linha da memória cache, este campo é analisado em todas as linhas da memória cache para verificar se o bloco referente à palavra solicitada está mapeado; e 2) “Byte”: identifica a palavra solicitada, caso o bloco esteja mapeado em uma determinada linha da memória cache. Na Figura 4.6 (b), há ainda os campos “Upt” (*Update*) e “Cnt” (*Counter*), na memória cache com mapeamento associativo. Assim como no mapeamento direto, o campo “Upt” informa a ocorrência de modificações em uma ou mais palavras da linha da cache, para que a linha, antes de ser substituída, seja atualizada na memória RAM (este campo é utilizado pelas políticas de escrita com acerto *Write-Back* e *Write-Once*). O campo “Cnt” é utilizado para contabilizar a frequência ou o instante de acesso a cada linha da memória cache, sendo utilizado quando emprega-se os algoritmos de substituição de linhas LFU ou LRU, respectivamente; e
- Associativo por conjunto: será subdividido em 3 campos, que são: 1) *Group* (“Grp”): em mapeamento associativo por conjunto os blocos da memória RAM são mapeados em determinados grupos (ou conjuntos), então este campo identifica o conjunto em que pode estar mapeado o bloco referente à palavra solicitada; 2) *Tag*: uma vez que um bloco da memória RAM pode ser mapeado em qualquer linha dentro de um determinado conjunto da memória cache, este campo é analisado em todas as linhas pertencentes ao conjunto para verificar se o bloco referente à palavra solicitada está mapeado; e 3) *Byte*: identifica a palavra solicitada, caso o bloco esteja mapeado em uma determinada linha de um determinado conjunto da memória cache. Na Figura 4.6 (c), há ainda os campos “Upt” (*Update*) e “Cnt” (*Counter*), na memória cache com mapeamento associativo por conjunto. Estes campos possuem as mesmas finalidades dos respectivos campos presentes na organização da memória cache com mapeamento associativo.

Para exemplificar a divisão de um endereço de memória, informado pelo processador, em campos que serão analisados pelo circuito controlador da memória cache no CompSim para verificar se o bloco e a palavra solicitada estão mapeados, consideremos o seguinte endereço de 12-bits:

LSB						LSB					
0	1	0	1	0	1	1	1	0	1	1	0

- Ao utilizar mapeamento direto, o endereço poderá ser dividido da seguinte maneira:

Tag			Line						Byte		
0	1	0	1	0	1	1	1	0	1	1	0

, onde o campo “Tag” informa o número do bloco (bloco 2 ou 010 em binário), que pode estar mapeado especificamente na linha da memória cache descrita no campo “Line” (linha 46 ou 101110 em binário), e o campo “Byte” informa a posição da palavra solicitada pelo processador na respectiva linha (palavra na posição 6 ou 110 em binário). É importante ressaltar que nesta divisão dos campos, poderão ser mapeados 8 (2^3) blocos por linha da memória cache (“Tag”) e um total de 64 (2^6) linhas (“Line”), com 8 (2^3) palavras por linha (“Byte”).

- Ao utilizar mapeamento associativo, o endereço será dividido da seguinte maneira:

Tag									Byte		
0	1	0	1	0	1	1	1	0	1	1	0

, onde o campo “Tag” informa o número do bloco (bloco 174 ou 010101110 em binário), que pode estar mapeado em qualquer uma das linhas da memória cache, e o campo “Byte” informa a posição da palavra solicitada pelo processador na linha (palavra na posição 6 ou 110 em binário). Nesta divisão dos campos, poderão ser mapeados 512 (2^9) blocos por linha da memória cache (“Tag”), com 8 (2^3) palavras por linha (“Byte”).

- Por fim, ao utilizar mapeamento associativo por conjunto, o endereço poderá ser dividido da seguinte maneira:

Tag						Group			Byte		
0	1	0	1	0	1	1	1	0	1	1	0

, onde o campo “Tag” informa o número do bloco (bloco 21 ou 010101 em binário), que pode estar mapeado em qualquer uma das linhas pertencentes ao conjunto descrito no campo “Group” da memória cache (conjunto 6 ou 110 em binário), e o campo “Byte” informa a posição da palavra solicitada pelo processador na linha (palavra na posição 6 ou 110 em binário). Com esta divisão dos campos, poderão ser mapeados 64 (2^6) blocos por conjunto da

memória cache (“Tag”), com um total de 8 (2^3) conjuntos (“Group”) e de 8 (2^3) palavras por linha (“Byte”).

4.4 Barramento

Um barramento é um recurso utilizado para conectar os componentes do computador (NULL; LOBUR, 2009), de forma que podem realizar comunicações com transferências de dados. Em geral, os barramentos têm baixo custo e são muito versáteis pois permitem conectar, facilmente, novos dispositivos ao sistema.

Quanto à sua estrutura, os barramentos podem ser Ponto-à-Ponto (ou de Canal de Dedicado), que permite conectar diretamente dois componentes específicos; ou Compartilhados, que possuem um modelo de comunicação que permite conectar vários dispositivos. Apesar de compartilhado, este tipo de barramento somente pode ser utilizado por um dispositivo por vez, o que pode resultar, frequentemente, em atrasos nas comunicações. O desempenho de um barramento compartilhado é afetado por seu comprimento e pelo número dos dispositivos conectados que o compartilham (NULL; LOBUR, 2009).

De fato, os barramentos compartilhados são divididos em três tipos de barramentos:

1. Endereço: é utilizado para informar um endereço de algum componente para realizar algum tipo de comunicação;
2. Controle: é utilizado para informar o tipo de comunicação que será realizada com o componente endereçado no barramento de endereço; e
3. Dados: é utilizado para a transferência de dados entre os componentes comunicantes.

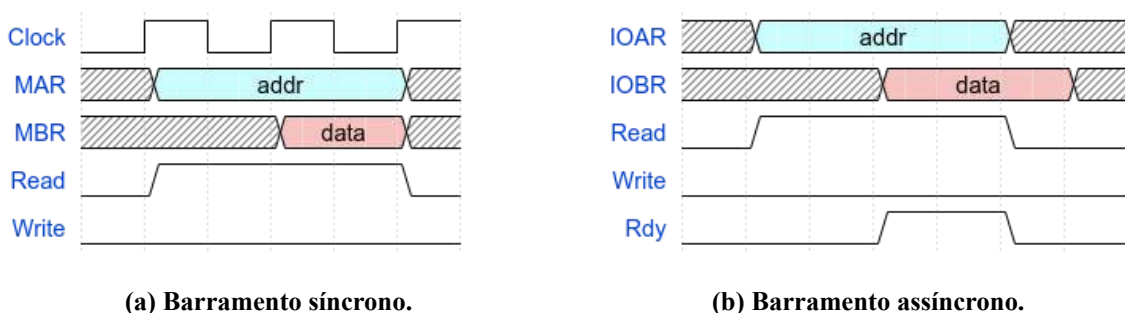
As larguras, ou os números de linhas ou de sinais, dos barramentos de dados e de endereços, são parâmetros importantes que têm grande impacto no desempenho de comunicação e na capacidade de endereçamento do sistema computacional. A largura do barramento de dados define a quantidade de bits que podem ser transferidos em paralelo em uma única comunicação. Assim, ao considerar um barramento de dados com largura de 32-bits, por exemplo, pode-se transferir dados inteiros com precisão de 2^{32} , o que no caso dos números inteiros sinalizados, o intervalo numérico estaria entre -2.147.483.648 e 2.147.483.647, e dos inteiros sem sinal o intervalo estaria entre 0 e 4.294.967.295. A largura do barramento de endereços define a quantidade de unidades endereçáveis, quer sejam

endereços de memória ou número de periféricos. Por exemplo, um barramento de endereços com largura de 32-bits permite endereçar uma memória RAM com até 4.294.967.296 (ou 2^{32}) endereços. Neste último cenário, considerando que cada célula de armazenamento da memória RAM permite armazenar um byte, a capacidade de armazenamento de dados da memória seria então de 2^{32} endereços x 1 byte, o que daria um total de 4.294.967.296 bytes ou 4 GBytes.

Os barramentos podem ser classificados em síncronos e assíncronos. Barramentos síncronos são aqueles em que a sequência de eventos de uma transmissão de dados (transação) é controlada por eventos gerados pelo relógio do sistema (*clock*). Já os barramentos assíncronos possuem linhas de controle que coordenam a transmissão de dados através de um protocolo de comunicação (*handshaking*).

Os barramentos síncronos, por utilizarem um *clock*, que geralmente possui uma frequência alta na geração de eventos para coordenação das transações, são utilizados para conectar componentes de alto desempenho de comunicação, tais como: processadores; memórias Cache e RAM; e coprocessadores, que são tipos de processadores que complementam as funções do processador principal, como por exemplo para processamento de dados em ponto flutuante, multimídia (decodificadores de áudio e vídeo), sinais digitais, gráficos, comunicações, inteligência artificial, entre outros. Já os barramentos assíncronos, por utilizarem um protocolo de *handshake* para coordenar a sequência de passos em uma transação ao invés de um *clock*, permitem que dispositivos com diferentes tempos de operação, geralmente periféricos, possam se comunicar, tornando os barramentos assíncronos mais escalares. Já As Figuras 4.7 (a) e (b) ilustram operações de leitura de dados em barramentos síncrono e assíncrono, respectivamente, através de diagramas de formas de onda (*Waveforms*).

Figura 4.7. Diagramas de formas de onda de barramentos síncrono e assíncrono.



(a) Barramento síncrono.

(b) Barramento assíncrono.

Fonte: Próprio autor (2021).

De acordo com o exemplo na Figura 4.7 (a), uma operação de leitura de dados em memória, através de um barramento síncrono, poderia ser dada da seguinte maneira:

1. Ao primeiro ciclo de *clock*, o processador informa o endereço de memória de onde o dado será lido, através da configuração do barramento de endereços (MAR);
2. Em paralelo, o processador configura o sinal *Read* para informar que será realizada uma operação de leitura de dados;
3. No segundo ciclo de *clock*, a memória disponibiliza para leitura o dado solicitado no barramento de dados (MBR);
4. Por fim, no terceiro ciclo de *clock*, após o dado disponibilizado ter sido lido, o processador desconfigura o sinal de leitura (*Read*) e a memória remove o dado do barramento de dados (MBR), encerrando assim a operação de leitura de dados.

No exemplo da Figura 4.7 (b), uma operação de leitura de dados a partir de um determinado periférico, utilizando um barramento assíncrono, poderia ser dada com os passos a seguir:

1. Inicialmente, o processador ativar um sinal *Read* para informar que será realizada uma operação de leitura de dados;
2. Em paralelo, o processador configura no barramento de endereços (IOAR) para informar o endereço do periférico do qual será lido o dado solicitado;
3. Ao receber esses sinais, o periférico disponibiliza a informação solicitada no barramento de dados (IOBR);
4. Em paralelo, o periférico informa ao processador que o dado solicitado está disponível no barramento de dados (IOBR), através da configuração do sinal *Rdy* (*Ready*);
5. Ao ler o dado solicitado, o processador desativa o sinal de configuração de operação de leitura de dados (*Read*), para informar o fim da operação de leitura, e remove o endereço inserido no barramento de endereços (IOAR);
6. Por fim, ao perceber que o processador sinalizou o fim da operação, o periférico remove o dado do barramento de dados (IOBR) e desconfigura o sinal *Rdy*, encerrando assim a operação de leitura de dados.

Além da operação de leitura de dados, os barramentos podem incluir outros tipos de operações (STALLINGS, 2010), tais como:

- Escrita: transferência de dados para memória ou periféricos;
- Leitura/Escrita em Rajada ou em Bloco (*Burst* ou *Block*): com uma única requisição realiza-se a transferência de múltiplos dados em sequência;

- **Leitura-Modificação-Escrita (*Read-Modify-Write*):** uma transferência de leitura, onde os dados lidos são modificados e escritos novamente no mesmo endereço de leitura. Esta operação é utilizada quando necessita-se alterar alguns bits ou bytes de uma palavra em memória;
- **Leitura-Após-Escrita (*Read-After-Write*):** uma transferência de escrita seguida por uma operação de leitura a partir do mesmo endereço de memória. Este tipo de operação é comumente utilizado para propósitos de checagem de consistência;
- **Divisão (*Split*):** é um tipo de transferência que pode ser interrompida por um periférico para ser realizada em um outro momento. Normalmente, é utilizado por periféricos que não conseguem atender imediatamente requisições de transferência de dados ou quando há periféricos com maior prioridade solicitando o uso do barramento.

Ao conectar muitos componentes em um barramento compartilhado, seu desempenho de comunicação pode ser degradado, pois como não é possível realizar transações simultâneas. Neste caso, dependendo do cenário, algumas transações terão que aguardar por muitos ciclos de *clock* até que possam ser realizadas. Nesses casos, utiliza-se uma Hierarquia de Barramentos, em que dois ou mais barramentos são conectados por um componente chamado Ponte (*Bridge*), que por sua vez tem a função de permitir que as transações se propaguem entre barramentos.

No CompSim, o processador Cariri possui espaços de endereçamento diferenciados para comunicação com a memória e com os periféricos. Assim, a plataforma Mandacaru inclui um barramento síncrono de sistema para conectar processador, memória cache e memória RAM, e um barramento assíncrono de periféricos para conectar o processador Cariri ao Subsistema de Entrada/Saída, para se comunicar com os periféricos. Os barramentos são independentes um do outro (não estão interligados por uma ponte).

O barramento de sistema possui larguras de barramentos de dados e de endereços de 16-bits (comunicações de 2 bytes e endereçamento de até 65.536 posições de memória RAM), suporta as operações de leitura e escrita simples para comunicação entre processador e memória cache; bem como de leitura e escrita em rajada para comunicação entre memórias cache e RAM; enquanto que o barramento de periféricos possui larguras de barramentos de dados e de endereços de 8-bits (comunicações de 1 byte e endereçamento de até 256 periféricos) e suporta apenas as operações de leitura e escrita simples.

4.5 Subsistema de Entrada/Saída

Os periféricos são componentes fundamentais em computadores, pois são eles que possibilitam a interação entre o computador e o usuário (*Human Readable*), o computador e outros sistemas computacionais (*Machine Readable*) e comunicação entre dispositivos remotos (*Communication*). Cada tipo de periférico possui características próprias, tais como tecnologia de fabricação, funcionalidades, mecanismos de operação, taxas de transferência de dados, formato de dados e tamanhos de palavra (STALLINGS, 2010).

De forma a demonstrar e potencializar essas diferenças, o Quadro 4.1 apresenta um comparativo entre diferentes periféricos de armazenamento de dados.

Quadro 4.1 - Comparativo entre periféricos de armazenamento de dados.

Periférico	Tecnologia de Armazenamento	Capacidade de Armazenamento	Taxa de Transferência
DDS (<i>Digital Data Storage</i>)	Padrões magnéticos organizados em trilhas paralelas na superfície de uma fita de material plástico.	320 GB (HP DAT 320)	12 MB/s (HP DAT 320)
HDD (<i>Hard Disk Drive</i>)	Padrões magnéticos organizados em trilhas circulares na superfície de um disco de alumínio coberto com uma liga à base de cobalto.	4 TB (Seagate Barracuda HDD)	190 MB/s (Seagate Barracuda HDD)
OD (<i>Optical Disk</i>)	Padrões de reflexão sobre uma superfície metálica, que pode ser de prata, ouro ou platina.	100 GB (Ultra HD Blue Ray three layers)	16 MB/s (Ultra HD Blue Ray three layers)
SSD (<i>Solid State Disk</i>)	Padrões de elétrons gravados em transistores em circuitos integrados, produzidos com material semicondutores à base de silício.	8 TB (Samsung 870 QVO)	560MB/s (Samsung 870 QVO)

Fonte: Próprio autor (2021).

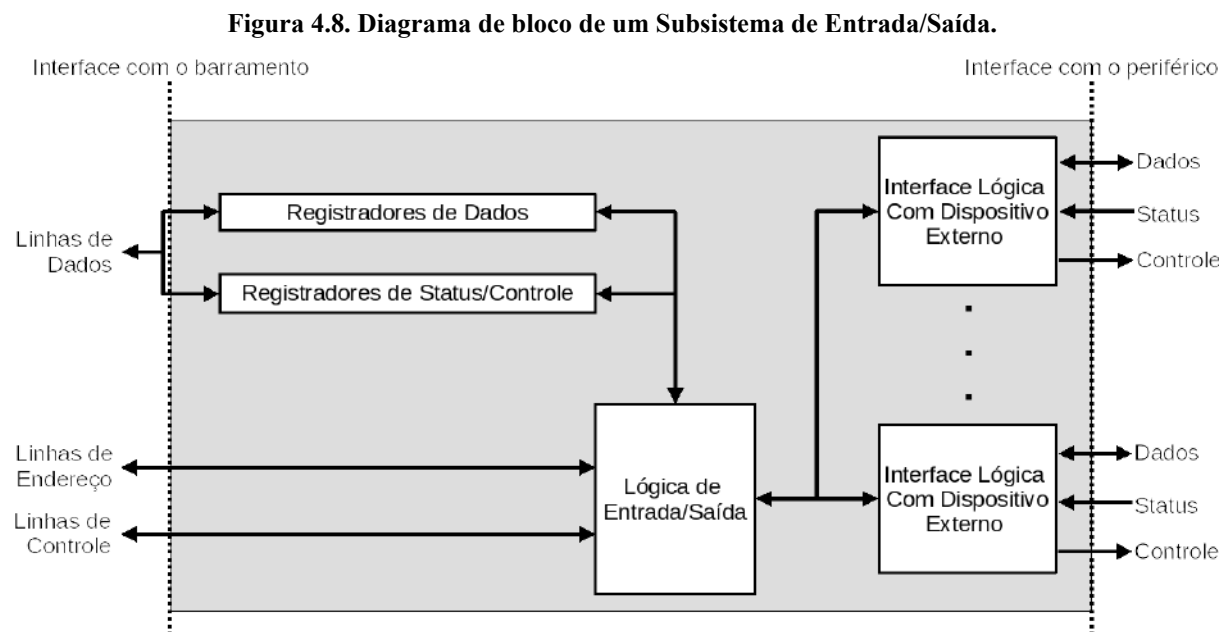
No Quadro 4.1, são comparados os periféricos de armazenamento Fita Magnética (DDS), Disco Magnético (HDD), Mídia Óptica (OD) e Disco de Estado Sólido (SSD). No quadro, pode-se perceber que as tecnologias, as capacidades de armazenamento e as taxas de transferência de dados diferem de um tipo de periférico para outro.

Ampliando o espectro para além de armazenamento de dados, incluindo outros tipos de periféricos, como, por exemplo, teclado, mouse, scanner, webcam, leitor de código de barra e joystick, que são dispositivos de entrada de dados, e monitor, placa de vídeo, placa de

som e impressora, que são dispositivos de saída de dados, as diferenças são claramente acentuadas.

O processador se comunica com os periféricos através do barramento de periféricos, o qual, por sua vez, não possui capacidade para realizar conexão com as interfaces nativas de cada um dos tipos de periféricos. Em outras palavras, é necessário que cada periférico, independentemente da sua natureza, inclua um mecanismo adaptador que permita sua conexão com uma interface padronizada presente no barramento de periféricos e traduza o modelo de comunicação entre processador e periférico. Esse mecanismo é conhecido como Subsistema de Entrada/Saída, Módulo de Entrada/Saída ou ainda Controlador do Periférico.

O Subsistema de Entrada/Saída possui duas funções principais: realizar interface do periférico com processador e memória, através da conexão com o barramento, e interface com um ou mais periféricos (STALLINGS, 2010). A Figura 4.8 apresenta um diagrama de bloco que ilustra a estrutura de um Subsistema de Entrada/Saída.



Fonte: Stallings (2010).

Na Figura 4.8, do lado esquerdo do Subsistema de Entrada/Saída, pode-se observar sua interface com o barramento, que tem como funções:

- Trocar dados entre o processador e o Subsistema de Entrada/Saída, armazenando-os nos Registradores de Dados;
- Decodificação dos comandos enviados pelo processador e que ficam armazenados nos Registradores de Controle;

- Informação do status do periférico, que pode estar pronto para responder a uma requisição (*Ready*) ou ocupado realizando uma determinada tarefa (*Busy*), ou de variadas condições de falha do periférico ou de comunicação. Essas informações são guardadas nos Registradores de Status;
- Reconhecimento de endereçamento do(s) periférico(s) controlado(s) pelo Subsistema de Entrada/Saída, via sinais de endereço do barramento. Ressalta-se aqui que um endereço de periférico possui a terminologia de “Porta”; e
- Interação entre o processador e o Subsistema de Entrada/Saída, via sinais de controle do barramento. Entre essas interações podem estar a seleção de um dos Subsistema de Entrada/Saída, para realizar uma operação de entrada/saída em um periférico, ou o Subsistema de Entrada/Saída envia sinais de status ao processador.

Ainda na Figura 4.8, no lado direito do Subsistema de Entrada/Saída pode-se visualizar sua interface com o(s) periférico(s). Neste lado, as funções principais do Subsistema de Entrada/Saída são:

- Comunicação com o periférico, cujo procedimento pode envolver envio de comandos, troca de dados e de informações de status;
- Armazenamento temporário de dados (*Data Buffering*), para os casos onde a taxa dos dados transferidos é muito maior que a taxa de operação do periférico. Assim, os dados são armazenados na memória temporária (*buffer*) e, daí, são transferidos na taxa de comunicação suportada pelo periférico; e
- Detecção de erros no periférico, que podem ser de diferentes tipos, tais como falhas elétricas ou mecânicas (e.g. impressão sem papel e disco com setores defeituosos - *bad blocks*) e de erros de transmissão de dados entre o periférico e o Subsistema de Entrada/Saída. Ressalta-se que, ao identificar um erro, é papel do Subsistema de Entrada/Saída notificá-lo ao processador.

As operações de entrada e saída podem variar de arquitetura para arquitetura (NULL; LOBUR, 2009). As instruções dos processadores podem suportar três técnicas básicas de entrada/saída, que são (TANENBAUM; AUSTIN, 2013):

- Entrada/Saída Programada com Espera Ocupada (*Polled*): neste tipo de operação, o processador fica monitorando um registrador do dispositivo de entrada/saída (*polling*), aguardando até que o dado solicitado fique disponível. O benefício do uso desta técnica é que tem-se controle total sobre a programação do comportamento do periférico. Por outro lado, o processador fica ocupado sempre que realiza alguma operação de entrada/saída. Este tipo de técnica é utilizada em sistemas de propósito

específico, tais como caixas eletrônicos e sistemas de monitoramento de eventos ambientais (NULL; LOBUR, 2009);

- Entrada/Saída por Interrupção (*Interrupts*): nesta técnica, ao contrário da anterior, ao realizar uma operação de entrada/saída, o processador não fica monitorando o periférico. Enquanto a operação de entrada/saída está em andamento, o processador passa a realizar outras tarefas. Quando a operação de entrada/saída é concluída, o periférico envia uma requisição de interrupção ao processador. Então, o processador interrompe a tarefa em execução, salva o contexto de execução da tarefa interrompida e trata a interrupção gerada pelo periférico para encerrar a operação de entrada/saída. Após o tratamento da interrupção, o processador restaura o contexto salvo da tarefa interrompida e retoma a execução do ponto onde parou. O uso de interrupções permite aumentar o desempenho do processador, uma vez que ele não fica dedicado ao monitoramento de periféricos em operações de entrada/saída. No entanto, segundo Tanenbaum e Austin (2013), tratar uma interrupção é uma tarefa custosa. Em sistemas com múltiplos periféricos, podem ser geradas muitas interrupções; há ainda periféricos que geram continuamente interrupções, como é o caso do teclado que gera uma interrupção a cada tecla pressionada; dessa forma, nesses casos, pode-se concluir que o uso da técnica de comunicação por interrupção não torna o sistema mais eficiente; e
- Entrada/Saída por Acesso Direto à Memória (*Direct Memory Access - DMA*): esta técnica combina as técnicas anteriores, buscando minimizar os problemas de espera ocupada e de múltiplas interrupções. Basicamente, é introduzido, no barramento de periféricos, um novo periférico, o Controlador de DMA, que tem também acesso à memória RAM. O processador programa, através de operações de entrada/saída, o controlador de DMA para transferir um conjunto de palavras da memória para um determinado periférico ou de um determinado periférico para a memória RAM. Ao ser programado, por exemplo, o controlador de DMA fará as leituras das palavras da memória RAM e realizará as escritas no periférico. Enquanto isso, o processador estará liberado para realizar outras tarefas. Ao final da transferência programada dos dados, o controlador de DMA gera uma interrupção para notificar o processador que concluiu sua tarefa. Esta técnica evita que o processador trate uma interrupção a cada transferência de palavra. No entanto, enquanto o controlador de DMA está atuando, o barramento fica ocupado, de forma que o processador fica bloqueado ao tentar acessar a memória RAM ou algum periférico, pois, segundo Tanenbaum e Austin (2013), o

controlador de DMA possui maior prioridade de acesso aos barramentos, uma vez que os periféricos não toleram atrasos na comunicação.

O simulador CompSim inclui um Subsistema de Entrada/Saída conectado ao barramento de periféricos da plataforma Mandacaru. Esse Subsistema de Entrada/Saída permite que novos periféricos sejam conectados de forma automática ao barramento de periféricos, bastando que o periférico implemente a interface de comunicação padrão com o barramento.

Quanto aos periféricos, é possível ter periféricos do tipo virtual, que são aqueles implementados em software e emulam o comportamento de periféricos reais; e do tipo físico, que são divididos em software, para implementar a interface padrão com o barramento, e em hardware para compor o periférico físico em si e seu comportamento.

Na atual versão do CompSim, dentre as três técnicas básicas de entrada/saída, apenas a técnica de comunicação de entrada/saída programada por espera ocupada é suportada, pois a versão atual do processador Cariri não possui suporte ao tratamento de interrupções e tampouco um periférico Controlador de DMA.

4.6 Resumo

Este capítulo apresentou os conceitos básicos de organização de computadores, ao trazer detalhes da estrutura, funções e de aspectos de comunicação dos principais componentes do computador. Também buscou-se relacionar as principais características dos componentes do computador com os respectivos componentes da plataforma de hardware virtual do simulador CompSim.

CAPÍTULO 5 – INTRODUÇÃO À ARQUITETURA DE COMPUTADORES

O processador, ou a Unidade Central de Processamento (*Central Processing Unit* - CPU), é o componente mais importante do computador. Segundo Tanenbaum e Austin (2013), ele é considerado o “cérebro” do computador, pois sua função é executar os programas armazenados na memória RAM, buscando suas instruções, interpretando-as e executando-as uma a uma, de maneira sequencial.

Independentemente da linguagem de programação utilizada para criar os programas de computador, o processador somente reconhece o seu próprio conjunto de instruções da arquitetura (*Instruction Set Architecture* - ISA). Assim, para que seja possível executar os programas criados pelos programadores de computadores, suas instruções devem ser traduzidas, por um compilador, para um novo programa que estará descrito em termos da ISA do processador. Um programa escrito na linguagem da ISA do processador terá suas instruções lidas, decodificadas e executadas pelo processador.

Alguns processadores possuem uma ISA reduzida, com instruções simples, de tamanho fixo e que requerem poucos ciclos de *clock* para serem executadas (*Reduced Instruction Set Computer* - RISC); já outros trazem um conjunto mais amplo, com instruções de tamanho variado e que requerem vários ciclos de *clock* para serem executadas (*Complex Instruction Set Computer* - CISC). A arquitetura RISC, por conter instruções mais simples, pode ser considerada mais rápida do que a arquitetura CISC, por outro lado, os programas tendem a conter mais instruções. Atualmente, dispositivos móveis, tais como smartphones e tablets, incluem processadores RISC. Já os computadores pessoais, tais como desktop e notebook, possuem processadores CISC.

Este capítulo descreve o modelo básico de comportamento da CPU, os aspectos que determinam o projeto do conjunto de instruções da arquitetura e apresenta as linguagens de montagem. Assim como no capítulo anterior, serão realizados paralelos entre os conceitos apresentados e as respectivas características presentes no simulador CompSim.

5.1 Ciclo de Instrução

O processador é um componente que está continuamente em atividade. No momento em que o computador é ligado, o processador entra em operação e só encerra suas atividades quando o computador é desligado.

Para desempenhar suas tarefas, o processador deve realizar um acesso à memória RAM para trazer para si uma cópia de uma das instruções do programa. Essa tarefa de buscar uma instrução na memória RAM é conhecida por *Fetch*. Após receber a instrução, que está codificada em um conjunto de bits, ou em bytes (depende da arquitetura), o processador precisa decodificá-la para compreender o que ela instrui para, em seguida, realizar a sua execução. Após a execução de uma instrução, o processador continua o processo de execução do programa, acessando novamente a memória RAM, buscando a próxima instrução a ser executada (*fetch*), decodificando-a e executando-a. Esse processo contínuo de busca, decodificação e execução de instruções é conhecido como Ciclo de Instrução (STALLINGS, 2010) ou Ciclo Buscar-Decodificar-Executar (TANENBAUM; AUSTIN, 2013), e está ilustrado na Figura 5.1.

Figura 5.1. Diagrama do Ciclo de Instrução.



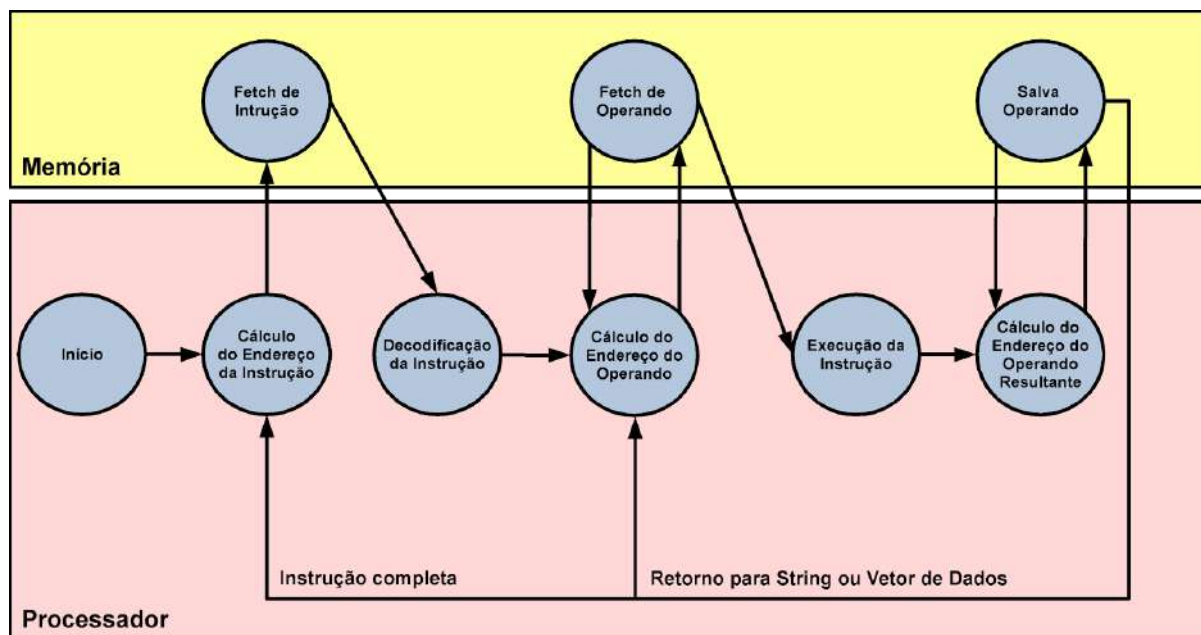
Fonte: Próprio autor (2021).

A Figura 5.2 mostra um diagrama de estados, que detalha a implementação das etapas de *fetch*, decodificação e execução, presentes no ciclo de instrução. No diagrama da figura, os estados descrevem as ações/tarefas que envolvem somente o processador (estados contidos no bloco inferior, em cor de rosa) e as que envolvem acessos à memória RAM (estados contidos no bloco superior, em cor amarela).

De acordo com o diagrama da Figura 5.2, após o início da execução do processador, o primeiro passo será calcular o endereço da instrução que será buscada na memória RAM. Após o cálculo do endereço, realiza-se o *fetch*. Em seguida, uma vez que uma instrução é copiada e levada ao processador, inicia-se a etapa de decodificação da instrução, em que o processador identifica qual é a instrução a ser executada, se possui operandos e se os

operandos estão na própria instrução, no processador, na memória RAM ou em algum periférico.

Figura 5.2. Diagrama detalhado do Ciclo de Instrução.



Fonte: Stallings (2010).

Ainda no diagrama da Figura 5.2, após a decodificação, o processador calcula os endereços de cada um dos operandos da instrução para trazê-los da memória RAM. Os passos seguintes, considerando que a instrução já foi decodificada e o(s) operando(s) já encontram-se no processador, consistem na execução da instrução, no cálculo do endereço e no armazenamento na memória RAM do operando resultante da execução da instrução. Há processadores que incluem instruções que atuam sobre vetores. Se a instrução decodificada é do tipo vetorial, o passo seguinte será buscar novos operandos na memória RAM e executar a mesma instrução para o processamento deles. Ao fim do armazenamento do(s) operando(s) resultante(s), considera-se que a execução da instrução foi concluída, de forma que o fluxo retorna ao início do ciclo de instrução, para o cálculo do endereço da próxima instrução que será executada.

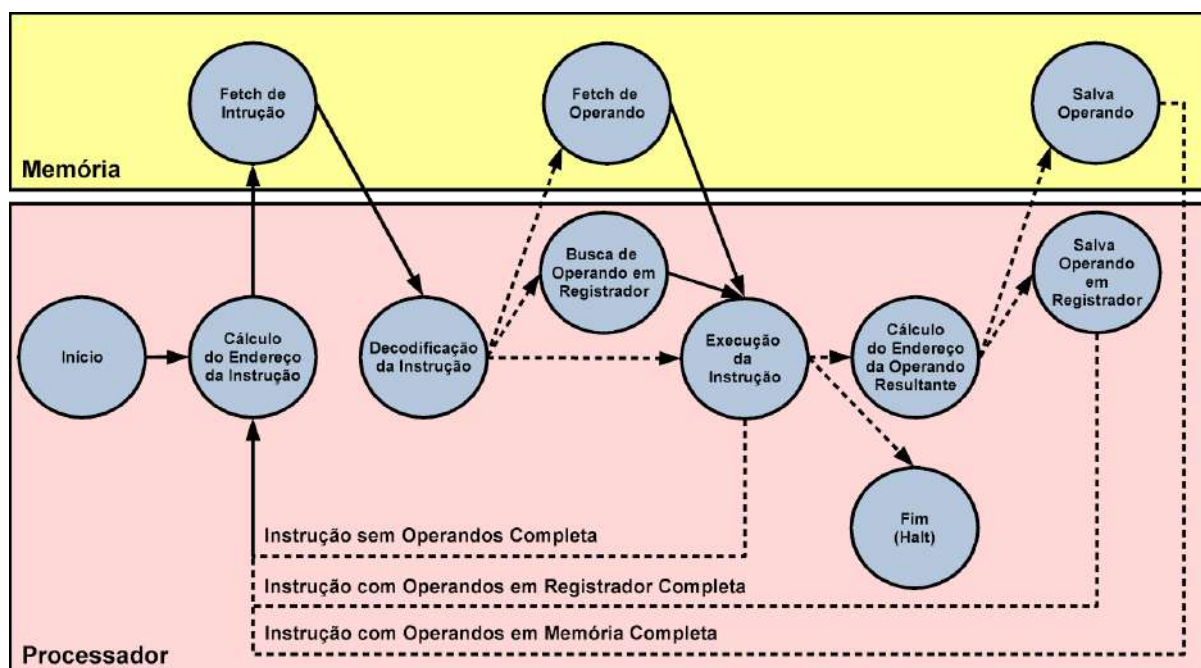
Como pôde ser visto, o processador é um componente complexo e realiza muitas tarefas no processo de execução de uma instrução. Para dar suporte ao ciclo de instrução, o processador conta com diversos subsistemas, tais como: a Unidade de Controle (UC), que possui o papel de coordenar cada uma das etapas do ciclo de instrução e os componentes envolvidos; a Unidade Lógica e Aritmética (ULA), que é responsável pela execução das

instruções de processamento de dados; e os Registradores, que são utilizados para dar suporte às tarefas do processador, tais como na decodificação de instrução, para endereçamento à memória e aos periféricos, armazenamento de operandos de entrada e resultantes, e sinalização dos status resultantes da execução de cada instrução.

5.1.1 Ciclo de Instrução do Processador Cariri

O processador Cariri, presente no simulador CompSim, também possui um ciclo de instrução. O diagrama de estados do ciclo de instrução do processador Cariri pode ser visto na Figura 5.3.

Figura 5.3. Diagrama detalhado do Ciclo de Instrução do Processador Cariri.



Fonte: Próprio autor (2021).

Na Figura 5.3, o ciclo de instrução do processador Cariri inicia com o cálculo do endereço da instrução que será buscada na memória RAM. Após o *fetch*, segue a decodificação da instrução. O processador Cariri possui instruções que podem conter um único operando, o qual pode estar localizado em memória ou em registrador, ou que não possuem operando. Com isso, após a etapa de decodificação, dependendo da instrução, o processador poderá buscar o operando na memória ou em registrador, ou saltar diretamente para a etapa de execução.

O processador Cariri possui uma instrução de controle de parada de simulação (a instrução *HALT*), e instruções que, após a execução, podem gerar ou não um operando resultante. Após a etapa de decodificação, se for identificada a instrução de parada, o processador encerra o ciclo de instrução; se for identificada uma instrução que gera operando resultante, após a sua execução, realiza-se o cálculo do endereço do operando resultante, que pode ser armazenado em memória ou em registrador; e, se a instrução identificada não gera operandos resultantes, inicia-se um novo ciclo de instrução.

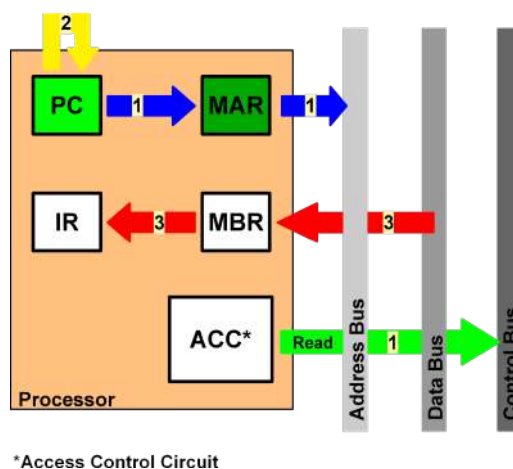
Para dar suporte ao ciclo de instrução, o processador Cariri conta com:

- Unidade de Controle (UC), para coordenar cada uma das etapas do ciclo de instrução e os componentes envolvidos;
- Unidade Lógica e Aritmética (ULA), que realiza o processamento de dados, através de operações aritméticas e lógicas;
- E os seguintes *Registradores*:
 - *Program Counter* (PC): é utilizado para calcular o endereço de memória da próxima instrução que será buscada para execução. À cada ciclo de instrução, o PC é incrementado automaticamente para endereçar a instrução seguinte;
 - *Memory Address Register* (MAR): é utilizado para endereçamento de memória, em operações de transferência de instruções e de dados. Este registrador está ligado ao barramento de endereços do barramento de sistema, o qual, por sua vez, está ligado à memória;
 - *Memory Buffer Register* (MBR): é utilizado para transferência de dados em operações de leitura/gravação de dados ou leitura de instruções da memória. Este registrador está ligado ao barramento de dados do barramento de sistema, o qual, por sua vez, está ligado à memória;
 - *Input/Output Address Register* (IOAR): é utilizado para endereçamento de um periférico em operações de transferência de dados. Este registrador está ligado ao barramento de endereços do barramento de periféricos, o qual, por sua vez, está ligado ao Subsistema de Entrada/Saída;
 - *Input/Output Buffer Register* (IOBR): é utilizado para transferência de dados em operações de leitura e gravação em periféricos. Este registrador está ligado ao barramento de dados do barramento de periféricos, o qual, por sua vez, está ligado ao Subsistema de Entrada/Saída;

- *Instruction Register (IR)*: após o *fetch*, a instrução que foi trazida da memória estará armazenada no registrador MBR. Então, ela será copiada do MBR para o IR, pois é neste último que ocorre a decodificação da instrução;
- *Accumulator (AC)*: é o único registrador de propósito geral do processador Cariri. Ele pode ser utilizado por diferentes instruções, tais como para armazenar um operando de entrada ou um operando resultante em operações aritméticas e lógicas, bem como um dado que será transferido, de ou para um outro registrador, para a memória ou para um periférico.

A Figura 5.4 ilustra a interação entre alguns desses componentes para realização do *fetch* de uma instrução no processador Cariri.

Figura 5.4. Diagrama de organização: Fetch de Instrução.



*Access Control Circuit

Fonte: Próprio autor (2021).

No diagrama da Figura 5.4, os passos para o *fetch*, são realizados na seguinte ordem:

1. Copia-se o endereço contido no registrador contador de programa (PC), que é o endereço da próxima instrução que será buscada na memória, para o registrador de endereçamento à memória (MAR). O MAR, ao receber um endereço, automaticamente já o propaga no barramento de endereços do barramento de sistema, ficando assim disponível para a memória. Em paralelo, o circuito de controle de acesso (*Access Control Unit - ACC*), que é parte da Unidade de Controle (UC), sinaliza no barramento de controle do barramento de sistema que será realizada uma operação de leitura de dados da memória;

2. Já contemplando o próximo ciclo de instrução, o registrador PC automaticamente sofre incremento do valor do endereço contido nele, para endereçar a próxima instrução que será executada;
3. Após receber um endereço, via barramento de endereços, e o comando de leitura de dados, via barramento de controle, após alguns instantes, a memória disponibiliza a instrução solicitada no barramento de dados. A instrução será então copiada do barramento de dados para o registrador de armazenamento temporário de dados da memória (MBR) e, posteriormente, do MBR para o registrador de instrução (IR). Com isso, é encerrada a etapa de *fetch*. As etapas seguintes consistem na decodificação e execução da instrução.

5.2 Instruções de Máquina

Analisando o diagrama detalhado de ciclo de instrução da Figura 5.2, pode-se induzir que uma instrução pode conter diferentes elementos. Uma instrução é um conjunto de bits, ou bytes, transferidos da memória para o processador. Na etapa de decodificação, esse conjunto de bits é dividido em campos, os quais, por sua vez, podem caracterizar a instrução e seus operandos, sendo que estes podem estar contidos na própria instrução ou armazenados em registradores no próprio processador, em memória ou em algum periférico. A Figura 5.5 ilustra um exemplo de uma instrução de máquina de uma arquitetura de 32-bits.

Figura 5.5. Exemplo de instrução de máquina.

Código de Operação	End. Operando 1	End. Operando 2	End. Operando Resultante
5-bits	9-bits	9-bits	9-bits

Fonte: Próprio autor (2021).

Na instrução de máquina da Figura 5.5, tem-se os seguintes campos:

- Código de Operação (*Operation Code - Opcode*): formado pelos 5-bits mais à esquerda (*Most Significant Bits - MSB*) da instrução e é utilizado para identificar a instrução a partir da combinação de bits. Por exemplo, a combinação “11100” poderia se referir a uma operação de soma, enquanto que a combinação “00011” poderia indicar uma subtração.

- End. Operando 1 e End. Operando 2: estes campos são formados por 9-bits e indicam referências ao primeiro e segundo operandos de entrada da instrução. Dependendo da instrução, uma referência pode ser um endereço de memória, de um registrador ou de periférico.
- End. Operando Resultante: formado pelos 9-bits menos significativos (à direita) da instrução e indica uma referência para o operando resultante da execução da instrução.

No projeto da ISA de um processador, deve-se considerar o número e os tipos de instruções, bem como o número máximo de operandos, seus tipos suportados e como podem ser referenciados em cada instrução. Esses detalhes impactam diretamente no tamanho das instruções de uma ISA (NULL; LOBUR, 2009).

Atualmente, as ISAs podem ser formatadas de duas maneiras: 1) instruções de tamanho fixo: desperdiçam espaço em bits dedicados para compor as instruções, porém resultam em instruções mais eficientes e com lógica mais simples para decodificação; e 2) instruções de tamanho variado: aproveitam melhor o espaço de bits dedicados para as instruções, porém apresentam maior complexidade na decodificação (NULL; LOBUR, 2009).

Em arquiteturas cujo número de bits reservados para os *opcodes* é fixo, é possível calcular facilmente o máximo de instruções que o processador pode conter. No exemplo, da Figura 5.5, foram reservados 5-bits, que dão um total de 2^5 ou 32 instruções. Instruções com *opcodes* de tamanho fixo também são muito simples de serem decodificadas. No entanto, para manter retrocompatibilidade com arquiteturas anteriores e flexibilidade, é possível ter ISAs com *opcodes* de tamanhos variados. Para tanto, utiliza-se uma técnica chamada *Expanding Opcodes*, que permite estabelecer um compromisso entre a necessidade de se ter uma ISA com várias instruções e que inclua também instruções curtas. Essa técnica basicamente considera que pode-se reservar poucos bits de uma instrução para os *opcodes* e, com isso, liberar uma quantidade maior de bits para os operandos. No entanto, quando houver instruções sem operandos, com poucos operandos ou referências curtas a operandos, pode-se aumentar o número de bits reservados para os *opcodes*. Com isso, pode-se compor uma ISA com instruções com *opcodes* reduzidos e mais operandos, bem como com *opcodes* maiores e menos operandos (NULL; LOBUR, 2009).

O número de bits reservados em uma instrução para os *opcodes* varia entre arquiteturas. No entanto, diferentes arquiteturas podem conter semelhanças de tipos de instruções em suas ISAs (STALLINGS, 2010). As instruções podem ser dos tipos de (NULL; LOBUR, 2009) (STALLINGS, 2010):

- Transferência de dados: as instruções de transferência de dados são as mais comuns nas arquiteturas. Geralmente, elas devem informar a origem dos dados e o destino, que podem estar na própria instrução, em registradores, memória ou periféricos; o tamanho dos dados a serem transferidos; e o modo de endereçamento de cada um dos operandos;
- Aritméticas: em geral, as arquiteturas fornecem instruções para as operações aritméticas, tais como de soma, subtração, multiplicação e divisão de números inteiros. Muitas tratam também de números de ponto flutuante e de representação decimal (*Binary-Coded Decimal* - BCD). Além das operações aritméticas fundamentais, é possível também encontrar instruções unárias, tais como para cálculo de valor absoluto, inversão de sinal de operando e incrementar ou decrementar o operando em 1 unidade;
- Lógicas: a maioria das arquiteturas inclui operações lógicas, que são operações que manipulam os bits individuais dos dados. Entre as instruções lógicas, é muito comum encontrar as de inversão (NOT), conjunção (AND), disjunção (OR), disjunção exclusiva (XOR) e de deslocamentos aritmético ou lógico de bits (SHIFT e ROTATE);
- Entrada/saída: as instruções de entrada/saída permitem a comunicação entre processador, memória e periféricos. Em geral, grande parte das arquiteturas disponibilizam poucas instruções voltadas para entrada/saída, que podem implementar diferentes técnicas, como entrada/saída programada com espera ocupada, interrupções e acesso direto à memória (*Direct Memory Access* - DMA);
- Transferência de controle: são instruções que permitem a alteração do fluxo de execução de um programa. Nessas instruções, a CPU atualiza o conteúdo do registrador contador de programa (PC), de forma que, no próximo ciclo de instrução, a CPU buscará a instrução nesse novo endereço. Exemplos de instruções de transferência de controle incluem: as de desvio de fluxo, que têm como operando o endereço da próxima instrução a ser executada e são utilizadas na implementação de construções de linguagens de alto nível, tais como desvios condicionais e iterações; e as de chamada de procedimentos, que são utilizadas para modularização de código pelo uso de procedimentos e funções;
- Propósito específico: exemplos de instruções de propósito específico incluem as de:

- Conversão de dados: incluem operações que mudam o formato dos dados. Um exemplo é a conversão de um dado em formato decimal (*Binary-Coded Decimal* - BCD) para binário.
- Controle de sistema: são instruções reservadas para uso dos sistemas operacionais e podem ser executadas quando o processador está em modo de execução privilegiado ou quando há algum programa alocado em região protegida de memória. Um exemplo deste tipo de instrução consiste em instruções para leitura ou modificação do conteúdo de registradores de controle, os quais possibilitam a mudança de comportamento da CPU para atuar, por exemplo, no gerenciamento de interrupções, controle de coprocessador e de sistema de paginação; e
- Processamento de cadeia de caracteres: incluem instruções para manipulação de cadeias de caracteres (*strings*), tais como para cópia, movimento, edição, comparação e pesquisa.

O número de operandos é um outro ponto importante no projeto de uma ISA. Naturalmente, o número de operandos pode ser condicionado ao tipo de operação que a instrução realiza. Instruções binárias, por exemplo, requerem dois operandos de entrada e pelo menos um operando resultante, como é o caso das operações aritméticas de soma e subtração. Com o número de operandos de uma instrução, pode-se estabelecer diferentes abordagens para projetar a estrutura e, por consequência, a largura da instrução. Tomando o exemplo das operações binárias, pode-se estabelecer instruções com três campos para operandos, sendo dois campos para os operandos de entrada e um campo para o operando resultante, como é o caso da instrução apresentada na Figura 5.5. Segundo Stallings (2010), o projeto de ISAs com instruções com três operandos não é comum, visto que pode resultar em instruções com grandes larguras de bits. Uma forma de reduzir o tamanho das instruções consiste em utilizar operandos implícitos. Por exemplo, em uma instrução binária com dois operandos, um dos operandos poderia referenciar tanto um operando de entrada quanto o operando resultante. Assim, após a operação, o resultado seria armazenado na referência a um dos operandos de entrada. Por fim, há projetos de ISAs que consideram instruções com um único operando ou sem operandos, sendo que, nesses casos, pode haver mais de um operando implícito, que frequentemente é um registrador. Instruções com poucos operandos resultam em instruções de menor largura e, por isso, tornam os projetos da ISA e do processador menos complexos. Porém, o número de instruções nos programas se torna maior, podendo resultar em programas mais complexos e com menor desempenho de execução. Já em

instruções com múltiplos operandos, frequentemente, o processador contém múltiplos registradores de propósito geral, o que faz com que parte das instruções operem apenas sobre os registradores, em vez da memória, tornando assim mais eficiente a execução da instrução.

Os operandos podem ser de diferentes tipos, tais como:

- Numéricos, que envolvem os seguintes tipos de números:
 - Inteiros: podem ter diferentes tamanhos/precisão (*short*: 16-bits, *int*: 32-bits e *long*: 64-bits), apresentar sinal (*signed*) ou não apresentar sinal (*unsigned*), e podem ser representados em diferentes técnicas de codificação binária, tais como Sinal-Magnitude ou Complemento a 2 (ver [Apêndice B - Introdução à Aritmética Computacional](#));
 - Números reais: são os números de ponto flutuante, que podem ser de precisão simples (*float*) ou dupla (*double*). Atualmente, todas as arquiteturas utilizam o padrão IEEE 754 para de codificação binária de números ponto flutuante (ver [Apêndice B - Introdução à Aritmética Computacional](#)); e
 - Decimais: neste tipo numérico, os dígitos decimais de 0 à 9 são codificados em sequências de 4-bits (*Binary-Coded Decimal* - *BCD*). Por exemplo, o número 0 (zero) é representado pela sequência “0000” e o número 5 é representado por “1001”. Apesar de no passado, este tipo de representação numérica ter sido bastante utilizado, pois simplificava a conversão dos dígitos decimais em sequências de bits em operações de entrada/saída de dados, não tem sido implementado nas ISAs mais recentes, por conta da complexidade para implementação das operações aritméticas.
- Não numéricos, que envolvem:
 - Caracteres: podem ser caracteres simples (*char* ou *byte*) ou cadeias de caracteres (*strings*). Um caractere é frequentemente codificado em uma sequência de bits, de forma que a quantidade de caracteres a serem manipulados pelo computador impacta no número de bits utilizados para representar todo o conjunto de caracteres. Um dos esquemas de codificação de caracteres mais utilizados é o *American Standard Code for Information Interchange* (ASCII), que utiliza 7-bits para representar 128 caracteres (TANENBAUM; AUSTIN, 2013). Desses 128 caracteres, uma parte deles é imprimível, que são os caracteres utilizados na escrita humana, por exemplo, e a outra é para controle, que são padrões de caracteres que controlam a impressão dos caracteres em uma página, como, por exemplo, tabulação

horizontal (*Horizontal Tab* - HT ou TAB), retrocesso (*Backspace* - BS) e retorno de carro (*Carriage Return* - CR ou Enter). Uma tabela com todos os caracteres e respectivos códigos ASCII pode ser vista no [Apêndice D - Tabela ASCII](#);

- Lógicos: também conhecidos por valores Booleanos, que possuem este nome com base na Álgebra de Boole, podem assumir dois possíveis valores, que são Verdadeiro (*True*) ou Falso (*False*). Para representação desses valores, normalmente utiliza-se o seguinte esquema de codificação: uma palavra com todos os bits configurados em 0 (zero) para representar o valor lógico Falso, e qualquer outra sequência de bits para representar o valor Verdadeiro;
- Ponteiros: também conhecidos como Apontadores, consistem de endereços de memória. Na prática, eles são implementados como números inteiros sem sinal e são bastante úteis em operações para manipulação de dados em memória, tais como para manipulação de estruturas de dados, alocação dinâmica de memória e alteração dos parâmetros de uma função por passagem por referência.

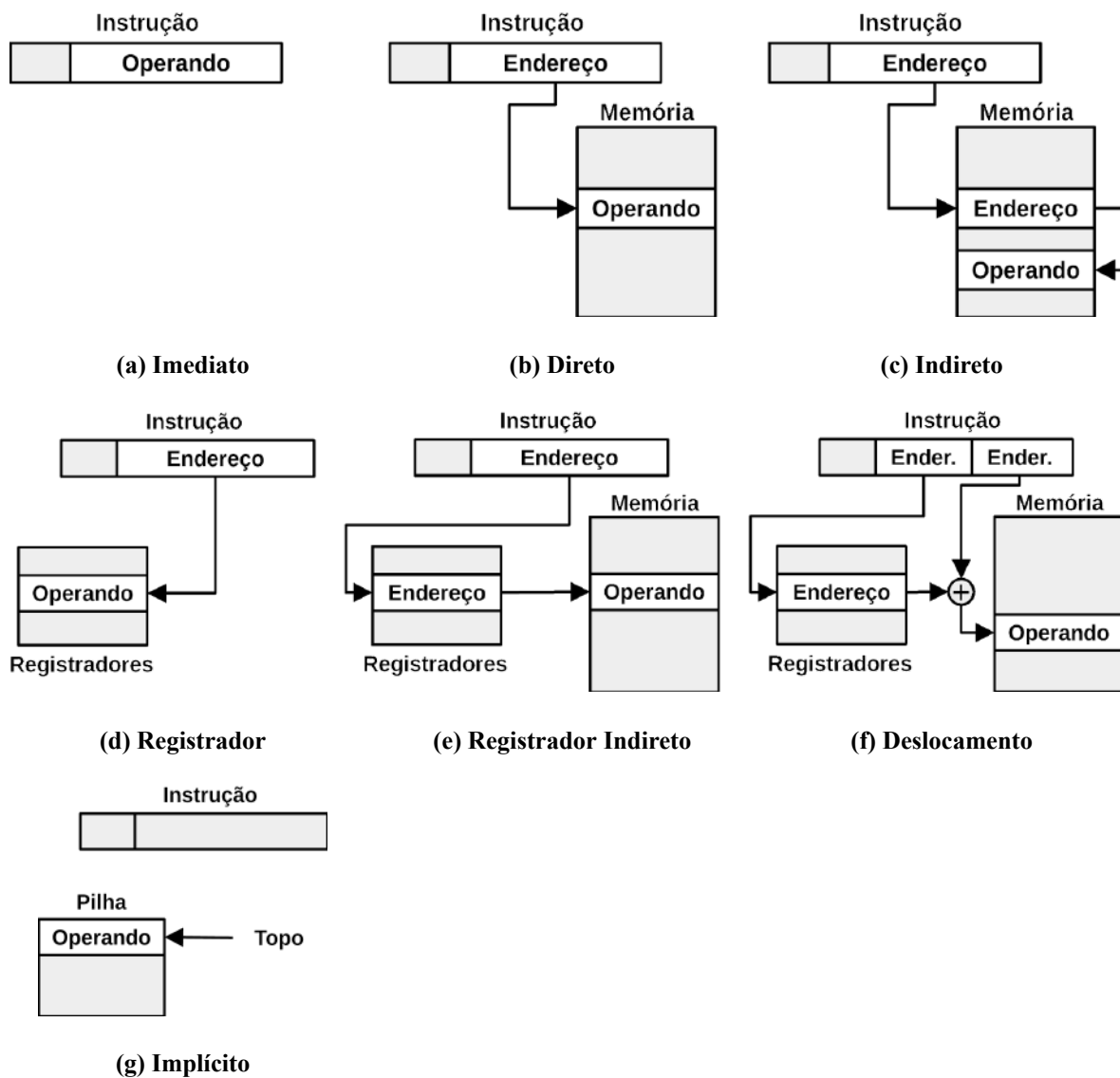
Dependendo da instrução, os campos reservados para os operandos podem ser interpretados de diferentes maneiras, que são denominadas como Modos de Endereçamento. Um modo de endereçamento permite especificar onde os operandos da instrução estão localizados, como na própria instrução, em um registrador ou em memória (NULL; LOBUR, 2009). Há diversos tipos de modos de endereçamento, porém os mais comuns são:

- Imediato: o operando está incluso na própria instrução e é utilizado para atribuir valores iniciais a variáveis ou a registradores, como está ilustrado na Figura 5.6 (a);
- Direto: o campo de operando inclui o endereço de memória no qual o operando está armazenado, como está ilustrado na Figura 5.6 (b). Este modo de endereçamento requer um acesso à memória. Como o número de bits reservados para o endereçamento do operando é reduzido na instrução, o espaço de endereçamento torna-se limitado. Por exemplo, se o campo de operando da instrução possui apenas 9-bits, como foi visto na Figura 5.5, somente é possível endereçar até 2^9 endereços, mesmo que o computador utilize memórias com 32-bits de endereçamento (2^{32} endereços);
- Indireto: o campo de operando inclui um endereço de memória no qual está contido um segundo endereço de memória que leva ao operando, como está ilustrado na Figura 5.6 (c). Neste tipo de endereçamento, são realizados dois acessos à memória,

porém permite aumentar o espaço de endereçamento da instrução. Utilizando o mesmo cenário descrito no modo de endereçamento anterior, com o endereçamento indireto, os 9-bits da instrução, reservados para endereçamento de operando, levariam a uma posição de memória que conteria um endereço de 32-bits, e, com este, se tornaria possível endereçar todos os 2^{32} endereços da memória;

- Registrador: É semelhante ao endereçamento direto, porém a diferença é que no campo do operando encontra-se um endereço de um registrador, como está ilustrado na Figura 5.6 (d). As vantagens deste modo de endereçamento é que não requer acessos à memória e, por conta do número limitado de registradores, seu espaço de endereçamento é reduzido, ocupando assim menos bits na instrução;
- Registrador Indireto: É semelhante ao modo de endereçamento indireto, porém a diferença é que no campo do operando encontra-se um endereço de um registrador, e neste, por sua vez, está contido um endereço de memória que aponta para o operando, como está ilustrado na Figura 5.6 (e). A vantagem deste modo de endereçamento é que permite ampliar o espaço de endereçamento da memória, com apenas um acesso à memória;
- Deslocamento: Requer que a instrução contenha dois operandos, sendo que o primeiro deles é um endereço de memória, que aponta para o endereço inicial (endereço base) de uma sequência de endereços consecutivos, e o segundo é um endereço de um registrador, como está ilustrado na Figura 5.6 (f). O modo de endereçamento por deslocamento atua somando o endereço base de memória ao valor contido no registrador para calcular o endereço de um deslocamento, em relação ao endereço base, para o acesso ao operando. Este modo de endereçamento pode ser utilizado, por exemplo, para otimizar acessos aos elementos contidos em estruturas de dados indexadas, tais como arrays, vetores ou matrizes;
- Implícito: Neste modo de endereçamento, não há necessidade de incluir campos para operandos ou para uma parte dos operandos, pois, implicitamente, eles já são referenciados na instrução, como está ilustrado na Figura 5.6 (b). Alguns processadores incluem instruções que suportam este modo de endereçamento, como são os casos das instruções POP e PUSH, que adicionam elementos ao topo da pilha do programa (neste caso, o endereço do topo é implícito à instrução).

Figura 5.6. Modos de endereçamento.



Fonte: Stallings (2010).

5.2.1 Instruções de Máquina do Processador Cariri

O processador Cariri trata exclusivamente de palavras de 16-bits. Isso significa que suas instruções e os tipos de dados suportados possuem larguras de 16-bits.

Visando simplificar a lógica de decodificação, o processador Cariri possui instruções de tamanho fixo, as quais são divididas em dois campos, *Opcode* e Operando, como é ilustrado na Figura 5.7.

Figura 5.7. Instrução de máquina do Processador Cariri.

Opcode	Operando
4-bits	12-bits

Fonte: Próprio autor (2021).

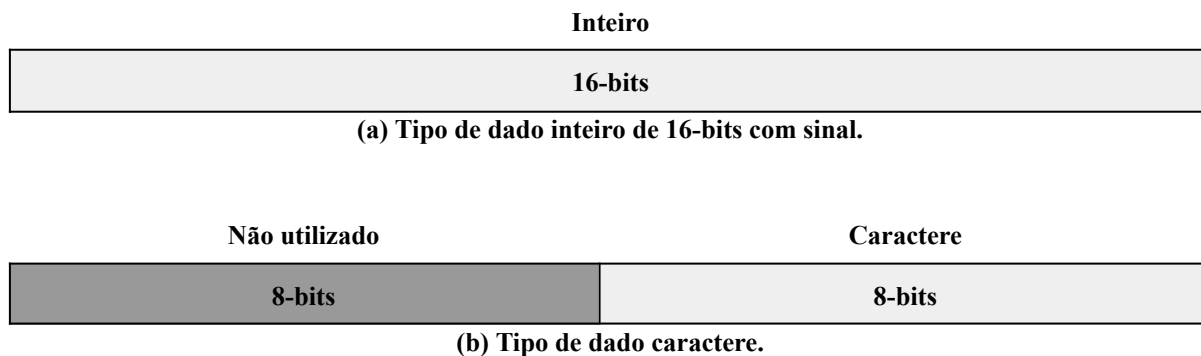
Na Figura 5.7, observa-se que, em uma instrução, foram reservados 4-bits para o campo *Opcode* e 12-bits para o campo Operando. Com *opcodes* de 4-bits, o processador Cariri suporta 16 (2^4) instruções, que podem ser dos tipos de transferência de dados, aritméticas, lógicas, entrada/saída, transferência de controle e controle de sistema. Ressalta-se que, apesar do processador Cariri oferecer uma ISA restrita (poucas instruções), suas instruções possuem versatilidade suficiente para: incluir duas ou mais operações em uma única instrução; suportar diferentes modos de endereçamento e, com isso, ampliar os recursos de manipulação de dados pelo processador; e suportar a composição de combinações de instruções para implementar operações de outras instruções e/ou operações mais complexas.

Apesar de conter apenas um campo para operando, as instruções podem ser do tipo: 1) Unárias: o campo Operando pode conter o próprio operando ou um endereço de memória para acesso ao operando; 2) Binárias: o campo Operando contém um endereço de memória para o primeiro operando e, o segundo operando, que é implícito, está contido no registrador Acumulador (AC). Após a execução da instrução, o resultado da operação é armazenado no próprio registrador AC; e 3) Sem operandos: neste tipo de instrução, o campo Operando não é considerado.

As instruções do processador Cariri podem suportar um tipo de operando numérico ou um tipo de dado não numérico. O dado numérico consiste de números inteiros de 16-bits com sinal (*signed int*), codificados em notação Complemento a 2. Já o dado não numérico consiste de caracteres (*char*), codificados no padrão ASCII. A Figura 5.8 ilustra os tipos de dados suportados pelo processador Cariri.

Na Figura 5.8, pode-se observar que, em (a), um número inteiro com sinal consome 16-bits, e, em (b), um caractere consome apenas os 8-bits menos significativos (*Least Significant Bits* - LSB) de uma palavra.

Figura 5.8. Tipos de dados suportados pelo Processador Cariri.



Fonte: Próprio autor (2021).

Por fim, para o acesso aos dados, dependendo da instrução, o processador Cariri pode suportar os modos de endereçamento imediato, direto, indireto, registrador e implícito.

5.3 Linguagem de Montagem

No ciclo de instrução, uma das tarefas do processador envolve decodificar instruções. Como visto anteriormente, instruções são sequências de bits, que representam os códigos de operação (*opcodes*) e seus respectivos operandos. O processador então, no processo de decodificação, deve identificar padrões nas sequências de bits de uma instrução.

Com base nisso, percebe-se que programar novos programas codificados diretamente na linguagem do computador, também conhecida por Linguagem de Máquina, torna-se uma tarefa extremamente difícil, exaustiva e suscetível a erros, visto que o programador deve empregar esforço para estabelecer os padrões de bits corretos, a cada instrução.

Visando simplificar a codificação de novos programas em linguagem de máquina, surgiu a linguagem de montagem, ou *Assembly Language*. Em um programa escrito em *Assembly*, cada linha consiste de uma instrução de máquina, e, as instruções, por sua vez, representam uma posição de memória.

Dependendo da arquitetura, uma instrução em linguagem *Assembly* pode conter mais de um campo, separados por espaços. A linguagem *Assembly* permite atribuir símbolos aos elementos nos campos das instruções, de forma a reduzir o esforço na criação de novos programas em linguagem de máquina.

Como cada instrução está localizada em um determinado endereço de memória, para simplificar uma referência a uma instrução, em vez de utilizar o próprio endereço de memória, a linguagem *Assembly* traz um recurso chamado de Rótulo (*Label*), que consiste de uma cadeia de caracteres que pode ser utilizada para simbolizar o endereço de memória da instrução.

Um outro recurso bastante útil na linguagem *Assembly*, é o uso de cadeias de caracteres abreviadas (aqui chamadas de Mnemônicos) para simbolizar os *opcodes* das instruções de máquina. Desta maneira, o programador pode então utilizar, por exemplo, os mnemônicos “ADD” e “SUB” para representar os *opcodes* de adição e subtração, em vez de sequências de bits, tais como “11100” e “00111”, respectivamente.

Uma terceira característica da linguagem *Assembly* é que os operandos também podem ser referenciados através de símbolos. Por exemplo, torna-se possível atribuir cadeias de caracteres para simbolizar endereços de memória (surge aí o conceito de Variável de Programa).

Outras características da linguagem *Assembly* podem ser destacadas, tais como: uso de mnemônicos para representar pseudo-instruções (instruções não estão presentes na ISA do processador, mas que realizam tarefas importantes, tal como na alocação de dados em memória) e suporte à comentários de código.

Um exemplo de trecho de programa escrito na linguagem *Assembly* do processador Cariri pode ser visto na Figura 5.9.

Figura 5.9. Exemplo de código Assembly do Processador Cariri.

Linha	Código
...	...
10	<code>ler_var1: LDA var1 ;carrega o valor de var1 em AC.</code>
...	...
32	<code>var1: DD 10 ;variavel inteira inicializada com valor 10.</code>
...	...

Fonte: Próprio autor (2021).

Na Figura 5.9, na Linha 10, a string “ler_var1:” consiste do rótulo para a instrução “LDA var1”, em que “LDA” é mnemônico para a instrução de leitura de dados da memória e “var1” é o identificador de uma variável (ele aponta para um endereço de memória). Na mesma linha, após “var1” pode observar o comentário de código “;carrega o valor de var1 em AC”, pois, na linguagem *Assembly* do simulador CompSim, um comentário de código inicia com o símbolo “;” (ponto-e-vírgula).

Ainda na Figura 5.9, a Linha 32 representa a definição de um dado em memória, onde: “var1” representa o nome de uma variável; “DD” é o mnemônico de uma pseudo-instrução utilizada para alocação em memória de um dado do tipo inteiro de 16-bits com sinal; e “10” é o valor atribuído na inicialização da variável.

Após a codificação de um programa em linguagem *Assembly*, deve-se utilizar uma ferramenta chamada Montador (*Assembler*), para traduzi-lo para a respectiva linguagem de máquina. Um montador traduz código *Assembly* para uma arquitetura específica e frequentemente pode atuar em dois passos: no primeiro passo, o montador pode realizar análises no programa (dos tipos léxica, sintática e semântica), construção de uma tabela de símbolos (esta tabela contém os nomes dos rótulos de instruções e das variáveis, e os respectivos endereços de memória) e de uma representação intermediária do código; já no segundo passo, combinando as informações contidas na tabela de símbolos com a representação intermediária do programa, o montador gera o programa de máquina final para a arquitetura alvo.

Ao analisar o trecho de código da Figura 5.9, percebe-se que, ao utilizar uma linguagem *Assembly*, a tarefa de codificação de novos programas torna-se mais intuitiva e produtiva, quando comparada à programação propriamente dita em linguagem de máquina. Além do mais, com o uso do montador, é possível ter suporte de análise e tradução, minimizando-se assim a ocorrência de erros no programa de máquina final.

5.4 Resumo

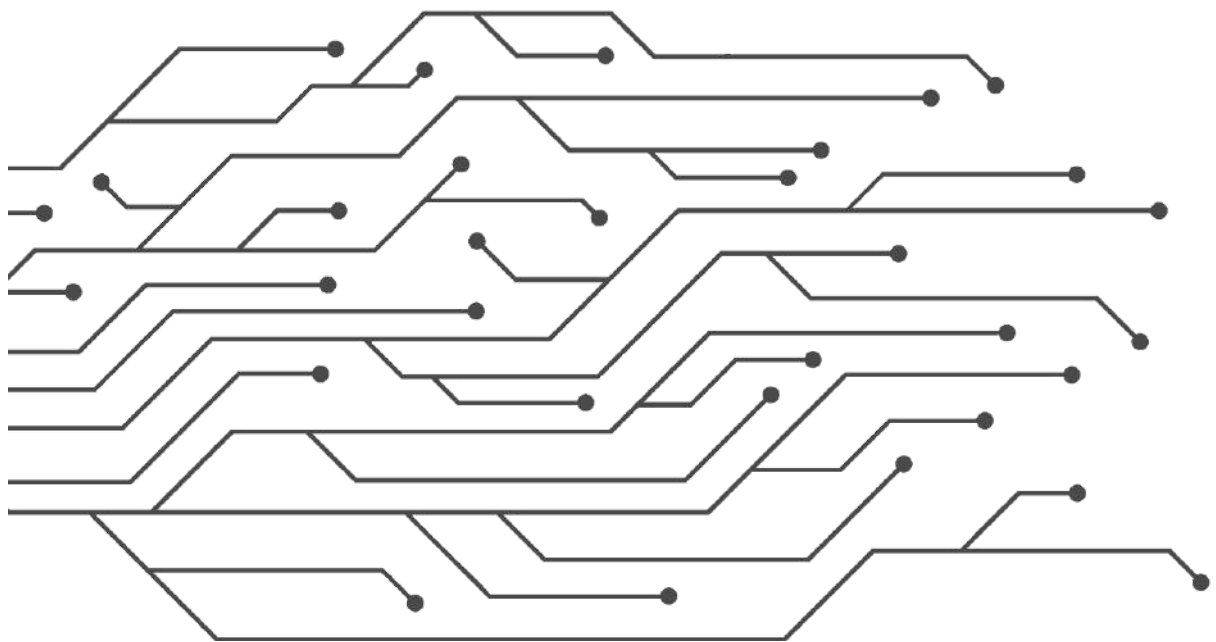
Este capítulo apresentou, inicialmente, os conceitos teóricos relacionados às tarefas desempenhadas pelo processador, esboçadas através do ciclo de instrução e de cada uma de suas etapas internas. Fez-se ainda uma discussão sobre os componentes do computador que estão envolvidos no suporte à execução de cada uma das etapas do ciclo de instrução. Em um segundo momento, foram apresentados os principais aspectos considerados no projeto do conjunto de instruções de uma arquitetura (ISA). E, por fim, discorreu-se sobre as linguagens de montagem e as facilidades que oferecem para a programação do computador, em relação às linguagens de máquina.

Em alguns pontos do capítulo, buscou-se realizar paralelos entre os conceitos apresentados e os respectivos recursos do simulador CompSim, sem explorá-los profundamente.

Capítulo 5 - Introdução à Arquitetura de Computadores

A terceira parte deste livro, [Parte III - Aspectos Práticos de Arquitetura de Computadores](#), traz capítulos que detalham o simulador CompSim e o emprego dos conceitos apresentados neste e nos capítulos anteriores, ilustrados com diversos exemplos de aplicações práticas.

PARTE III - ASPECTOS PRÁTICOS EM ARQUITETURA DE COMPUTADORES



CAPÍTULO 6 – INSTALAÇÃO DO SIMULADOR COMPSIM

O Simulador CompSim é uma ferramenta de software que é desenvolvida em cooperação técnica entre diferentes instituições públicas nacionais, envolve pesquisas científicas e o esforço de vários participantes do projeto.

Nesse sentido, elaborou-se um website visando centralizar todas as informações do projeto CompSim, tais como descrição do simulador, notícias, download do simulador e de complementos, materiais didáticos, informações de participantes, publicações científicas, endereço para a lista de discussão, entre outras.

Este capítulo apresenta o website do projeto CompSim e como realizar a instalação do simulador.

6.1. Website do Projeto CompSim

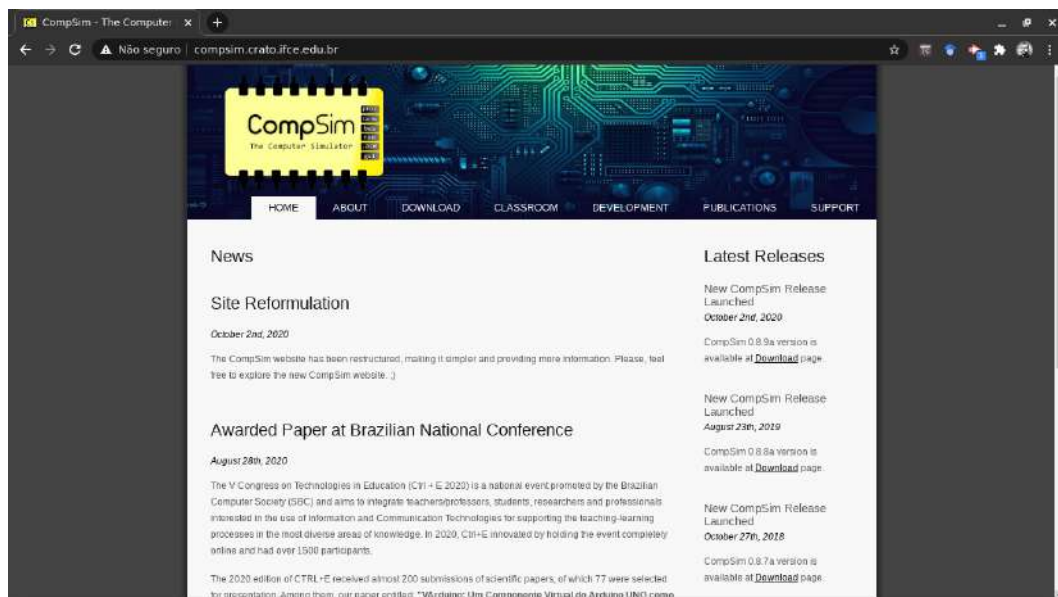
O projeto CompSim possui seu próprio website¹¹ (COMPSIM, 2021a), como pode ser visto na Figura 6.1.

O website é dividido nas seguintes seções:

- **Home:** Esta é a seção inicial do website e apresenta notícias com novidades do projeto, participações em eventos, resultados, atividades dos participantes, entre outras;
- **About:** Esta seção apresenta uma breve descrição do simulador CompSim. São apresentadas, de forma ilustrada, as suas principais características;

¹¹ CompSim - The Computer Simulator. Disponível em: <<http://compsim.crato.ifce.edu.br/>>. Acesso em: 01 abr. 2021.

Figura 6.1. Website do Projeto CompSim.



Fonte: Próprio autor (2021).

- **Download:** Seção que inclui *links* para *download* do simulador CompSim, em diferentes versões (0.8.7a, 0.8.8a e 0.8.9a), para diferentes arquiteturas (32 e 64-bits) e para diferentes plataformas operacionais (MS Windows e GNU/Linux). Além disso, permite também o *download* dos periféricos oficiais suportados pelo CompSim, para diferentes plataformas operacionais (MS Windows e GNU/Linux).

Dica!

Ao final da seção *Download*, há um link para os *ChangeLogs*, que apresentam informações detalhadas sobre o lançamento de novas versões do simulador, tais como correções de bugs e descrições de novos recursos.

- **Classroom:** Inclui materiais didáticos que podem ser utilizados por professores e estudantes, em sala de aula ou práticas laboratoriais. Inclui materiais em slides e exemplos de códigos-fonte em linguagem de baixo nível (*Assembly*). Além disso, inclui endereço para o Canal do Youtube do projeto CompSim, onde podem ser encontradas vídeo aulas que tratam diversos assuntos relacionados ao aprendizado de Arquitetura e Organização de Computadores (AOC) com apoio CompSim. No canal, há vídeo aulas introdutórias ao uso do simulador e que tratam de conceitos avançados de AOC, bem como palestras e minicursos realizados *online*.

- **Development:** O simulador CompSim é fruto do esforço de várias pessoas. Esta seção inclui dados sobre as instituições, e respectivos participantes, que promovem o desenvolvimento, uso e divulgação do simulador CompSim.
- **Publications:** O projeto CompSim busca, através de pesquisas científicas, estar sempre alinhado às novas tendências tecnológicas em projetos de sistemas computacionais e nas diferentes abordagens educacionais em AOC. Com isso, nesta seção, estão relacionados os artigos com os principais resultados das pesquisas que foram sob forma de artigos publicados em periódicos e conferências científicas, bem como , capítulos de livros.
- **Support:** O projeto CompSim, ao longo dos anos, vem recebendo continuamente *feedback* de seus usuários, quer sejam eles estudantes, professores e entusiastas. Esses *feedbacks* são muito importantes, pois com eles surgem demandas de novos recursos, reportes de bugs, dúvidas sobre uso das ferramentas do simulador e aspectos de codificação em linguagem de alto nível, entre outros. Assim, esta seção inclui o endereço para o grupo de discussão do projeto CompSim no Google Groups¹² (COMPSIM, 2021b).

6.2 Download do Simulador

Para realizar o *download* do simulador CompSim, deve-se acessar a seção *Download* no website do projeto. A Figura 6.2. mostra a página de download, onde pode-se visualizar um quadro com informações sobre cada uma das versões do CompSim.

Em seguida, antes de realizar o *download*, é necessário verificar qual é o sistema operacional (MS Windows ou GNU/Linux) e qual é a arquitetura (32 ou 64-bits) do computador onde o simulador será instalado. Se houver dúvidas sobre qual é a arquitetura do computador onde será instalado o simulador CompSim, sugere-se realizar o *download* da versão para arquitetura 32-bits, pois a mesma é compatível com computadores tanto de 32 quanto de 64-bits. Observa-se aqui que não há diferença significativa de desempenho ou de quantidade de espaço ocupado em disco entre versões de 32 e 64-bits.

¹² Grupo de Discussão do Projeto CompSim. Disponível em:
<<https://groups.google.com/g/compsim-the-computer-simulator>>. Acesso em: 01 abr. 2021.

Figura 6.2. Página de download do website do Projeto CompSim.



Fonte: Próprio autor (2021).

Em relação à versão do simulador CompSim, sugere-se realizar download da mais recente (0.8.9a), pois a mesma inclui mais recursos e menos *bugs*, em relação às versões anteriores, e consiste da versão atual de trabalho, podendo passar por novos ciclos de desenvolvimento. As versões mais antigas estão descontinuadas e são mantidas no website por questões de compatibilidade com projetos legados. No entanto, com o passar do tempo, elas serão retiradas, ficando indisponíveis para *download*.

Após verificados os requisitos (sistema operacional e arquitetura do computador) e escolhida a versão do simulador, para realizar o *download* basta clicar no link “ZIP” à direita da versão escolhida. Ao clicar, será realizado o download de um arquivo compactado em formato “.zip”, com a seguinte nomenclatura:

CompSim_v0.8.Xa-YYYZZ.zip

, onde:

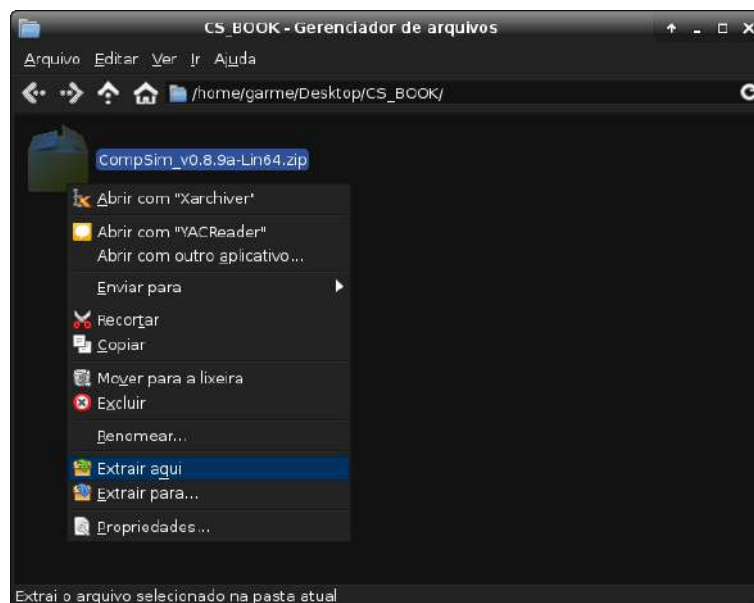
- “X” é a versão (pode ser “7”, “8” ou “9”);
- “YYY” é sistema operacional (pode ser “Lin” ou “Win”); e
- “ZZ” é a arquitetura (podendo ser “32” ou “64” bits).

6.3 Instalação do CompSim

Após o *download* do arquivo de instalação, como apresentado na seção anterior, o processo de instalação do CompSim é bastante simples: basta descompactar o arquivo .ZIP. Dependendo do sistema operacional, há várias ferramentas que podem ser utilizadas para esse fim.

A Figura 6.3 mostra um forma de descompactar o arquivo CompSim_v0.8.9a-Lin64.zip, utilizando o gerenciador de arquivos “Thunar” do sistema operacional GNU/Linux.

Figura 6.3. Instalação do simulador CompSim.



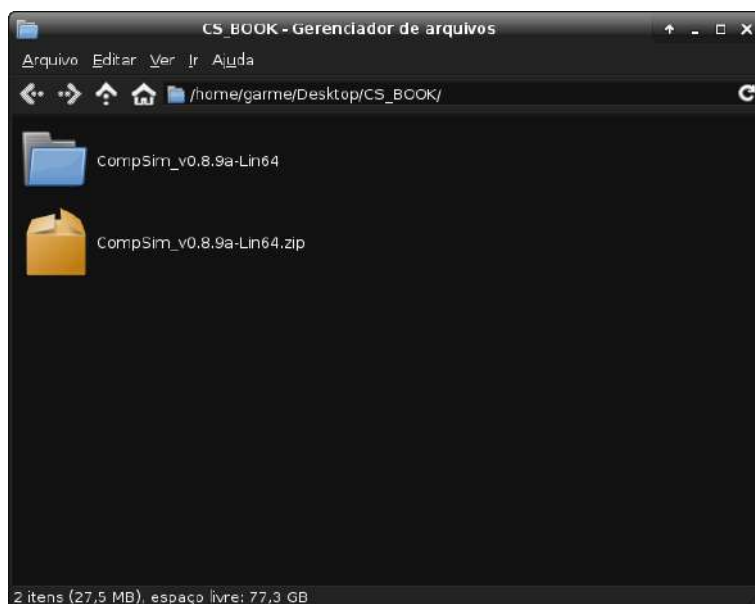
Fonte. Próprio autor (2021).

Após a extração do arquivo, será criada uma pasta com o mesmo nome do arquivo sem a extensão “.zip”, como pode ser visto na Figura 6.4.

Nesta nova pasta, haverá apenas um arquivo chamado “CompSim”. Este é o arquivo para execução do simulador. Para executá-lo, basta clicar duas vezes com o ponteiro do mouse sobre o arquivo “CompSim”. A Figura 6.5 mostra a interface gráfica do simulador CompSim em sua primeira execução.

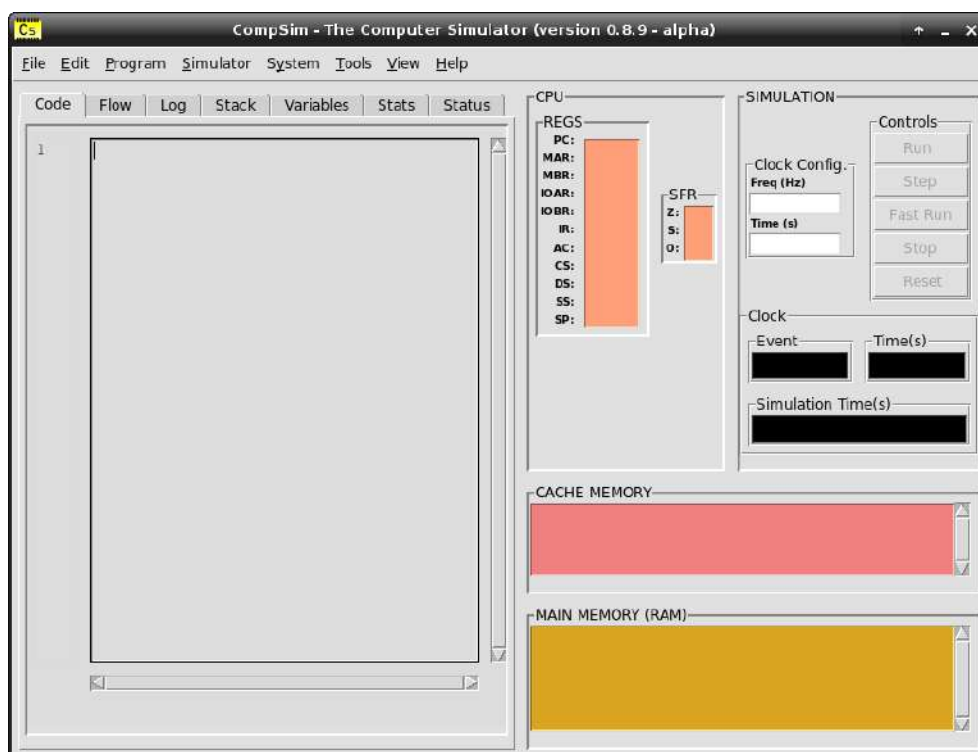
Capítulo 6 - Instalação do Simulador CompSim

Figura 6.4. Pasta do simulador CompSim.



Fonte: Próprio autor (2021).

Figura 6.5. Simulador CompSim em execução.



Fonte: Próprio autor (2021).

6.4 Resumo

Este capítulo iniciou com uma apresentação do website do projeto CompSim. Nele estão concentradas as principais informações do projeto, tais como descrições dos recursos do simulador, notícias, *downloads*, materiais didáticos, participantes do projeto, publicações, entre outras.

Na sequência do capítulo, foram ilustrados os passos necessários para realizar o *download*, instalação e execução do simulador CompSim.

CAPÍTULO 7 – SIMULAÇÃO COM COMPSIM

Uma vez que o simulador CompSim está instalado, já é possível criar sistemas computacionais e simulá-los. No entanto, como ocorre com qualquer outra ferramenta, é necessário conhecer sua dinâmica de trabalho para compreender como utilizar suas funções.

Considerando que o CompSim segue a abordagem de Projeto Baseado em Plataformas (*Platform Based Design* - PBD) (DAVARE et al., 2007), antes de realizar uma simulação de um sistema computacional, são necessários os seguintes passos: criação e configuração de uma plataforma de hardware virtual (PHV), codificação da aplicação que será executada na plataforma criada, mapeamento da aplicação para a plataforma e configuração da simulação.

No simulador CompSim, durante uma simulação, é possível visualizar os status dos componentes da PHV e de diferentes elementos da aplicação em execução; e, ao final de uma simulação, é possível ainda realizar análises de desempenho do sistema simulado.

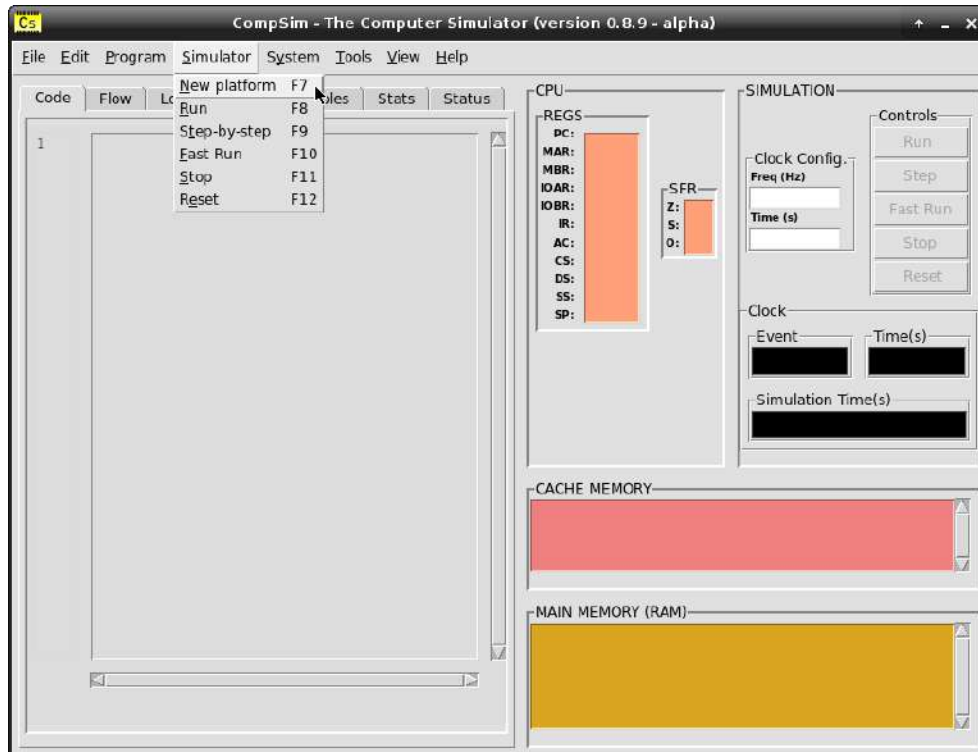
Este capítulo apresenta descrições dos recursos e procedimentos para realização de cada um desses passos, no simulador CompSim.

7.1 Criação de Uma Plataforma Simulável

No [Capítulo 4 - Introdução à Organização de Computadores](#), estudou-se a PHV do simulador CompSim, chamada de “Plataforma Mandacaru”. Essa plataforma possui os principais componentes do computador, tais como processador, memórias, barramentos e subsistema de entrada/saída, e que os mesmos apresentam características dos respectivos componentes reais.

A criação de uma PHV pode ser realizada acessando o menu “Simulator”, opção “New Platform”, ou pressionando a tecla “F7”, como pode ser visto na Figura 7.1.

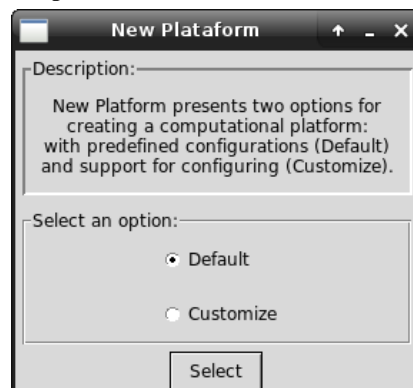
Figura 7.1. Menu para criação de Plataforma de Hardware Virtual.



Fonte: Próprio autor (2021).

Em seguida, será apresentado um painel com duas opções de criação da PHV, como pode ser visto na Figura 7.2.

Figura 7.2. Painel com opções de criação de Plataforma de Hardware Virtual.



Fonte: Próprio autor (2021).

Dentre as opções de criação de plataforma apresentadas na Figura 7.2, há as opções “Default” e “Customize”. A primeira opção (“Default”) permite criar rapidamente uma plataforma com todos os componentes da plataforma Mandacaru e configurações predefinidas para as memórias Cache e RAM, que são:

1. Memória Cache:
 - a. Técnica de mapeamento (*Mapping*): Direto (*Direct*);

- b. Política de substituição (*Replacement Policy*): *First-in, First-out* (FIFO);
- c. Política de escrita com acerto (*Write-Hit*): *Write-Through*;
- d. Política de escrita com falta (*Write-Miss*): *Write-Allocate*; e
- e. Número de linhas: 4.

2. Memória RAM:

- a. Número de linhas: 64 linhas.

Já ao selecionar a segunda opção de criação de plataforma, a opção “Customize”, será apresentado um novo painel, como pode ser visto na Figura 7.3, em que é possível configurar os seguintes parâmetros da plataforma, com as respectivas opções de configuração:

1. Memória Cache:

- a. Técnica de mapeamento (*Mapping*): Direto (*Direct*), Associativo (*Associative*) ou Associativo por Conjunto (*Set Associative*);
- b. Política de substituição (*Replacement Policy*): *First-in, First-out* (FIFO), Menos Recentemente Usado (*Least Recently Used* - LRU) ou Aleatório (*Random*);
- c. Política de escrita com acerto (*Write-Hit*): *Write-Through* ou *Write-Back*;
- d. Política de escrita com falta (*Write-Miss*): *Write-Allocate* ou *Write-Around*;
- e. Número de linhas: de 4 (2^2) à 128 (2^7).

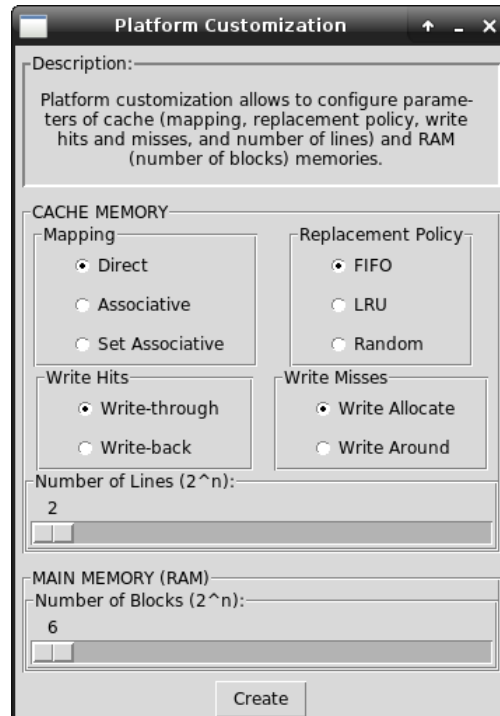
2. Memória RAM:

- a. Número de linhas: de 64 (2^6) à 1024 (2^{10}).

Na versão atual do CompSim (v0.8.9a), por padrão, um bloco de memória RAM é formado por uma linha de sua matriz de dados, a qual possui 8 palavras de 16-bits e, por consequência, uma linha da memória Cache também armazenará 8 palavras (bloco). Optou-se por fixar este parâmetro por questões de simplificação da visualização dos conteúdos das duas memórias na interface gráfica do simulador.

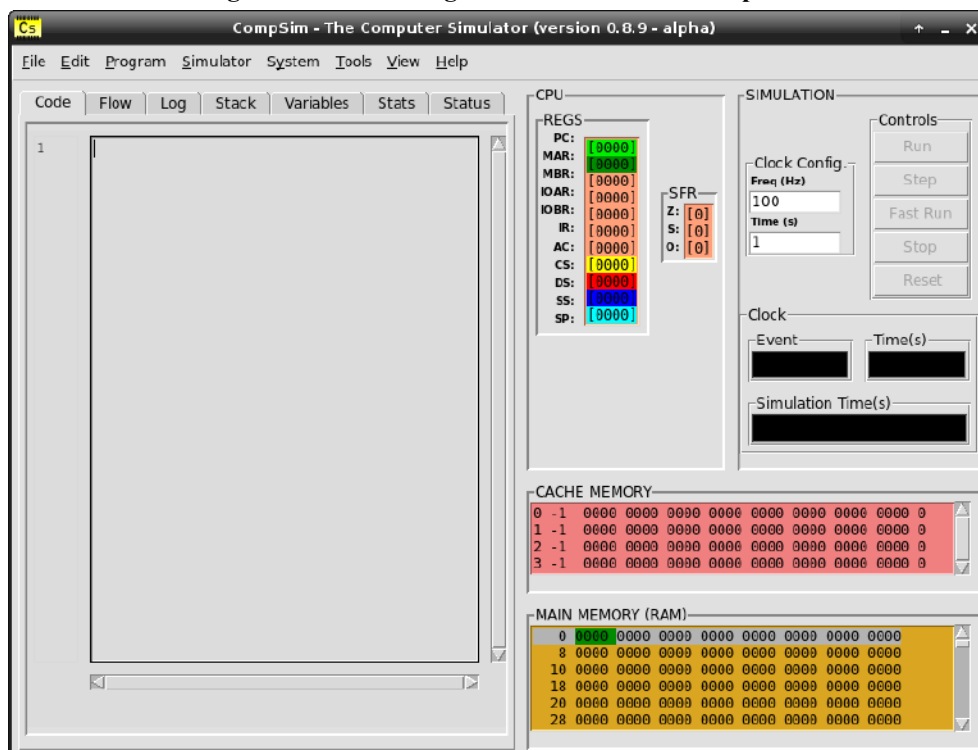
Ao criar uma plataforma, a interface gráfica passa a apresentar o conteúdo dos registradores do processador, no componente gráfico “CPU”; o conteúdo das memórias Cache e RAM, nos componentes gráficos “CACHE MEMORY” e “MAIN MEMORY (RAM)”, respectivamente; e, habilita as configurações de simulação, tais como a frequência do *clock* do sistema e de tempo de simulação, no componente gráfico “SIMULATION”, como pode ser visto na Figura 7.4.

Figura 7.3. Painel com opções de configuração de Plataforma de Hardware Virtual.



Fonte: Próprio autor (2021).

Figura 7.4. Interface gráfica do simulador CompSim



Fonte: Próprio autor (2021).

Destaca-se aqui que, para a maioria das simulações apresentadas neste livro, adota-se que será utilizada a configuração padrão (“Default”) para criação da PHV. Nos casos excepcionais, serão descritas as respectivas configurações customizadas utilizadas.

Após a criação de uma PHV, o passo seguinte consistirá na criação de uma aplicação, que será executada pela plataforma.

7.2 Criação de Uma Aplicação

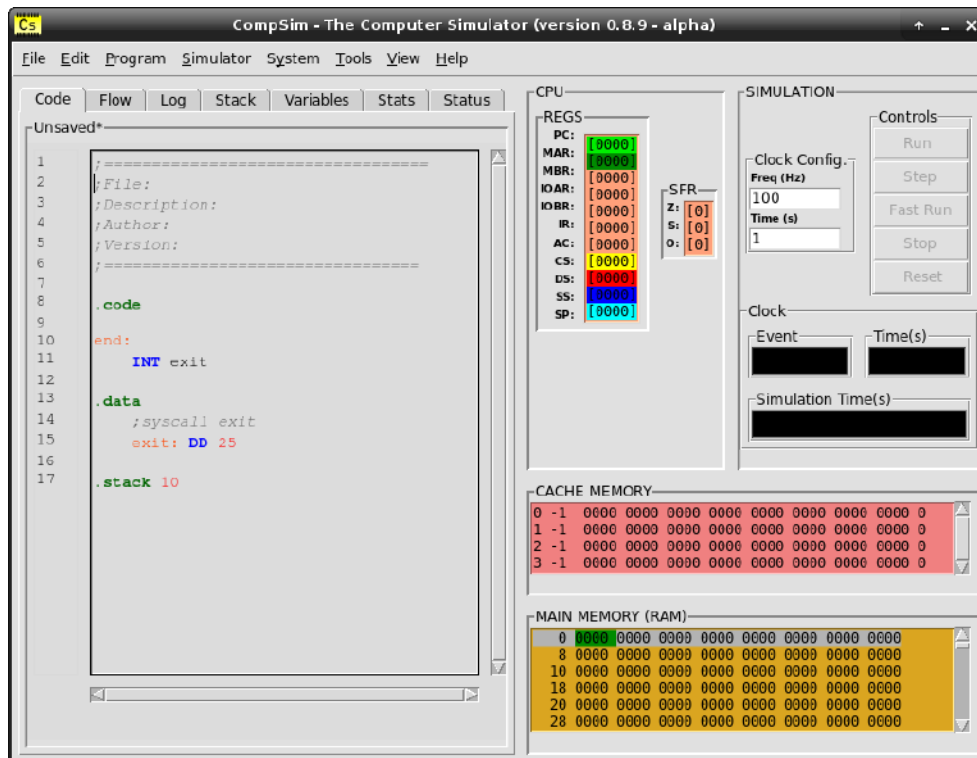
O CompSim possui um ambiente integrado de desenvolvimento (*Integrated Development Environment* - IDE) que tem como objetivo dar suporte à criação de novas aplicações em linguagem *Assembly*, as quais poderão ser executadas na plataforma de hardware do simulador. O ambiente de desenvolvimento inclui um editor de código, que possui os seguintes recursos:

- Criação, abertura e salvamento de arquivos de código-fonte em linguagem *Assembly* (arquivos com extensão “.asm”);
- Exibição de número de linhas e de nome do arquivo em edição;
- Destaque (*Syntax Highlight*) de palavras-chave da linguagem *Assembly* do processador Cariri;
- Recursos de edição de código-fonte, tais como desfazer/refazer (*Undo/Redo*), cortar/copiar/colar (*Cut/Copy/Paste*) e salto para determinada linha do código-fonte (*Go to Line*);
- Assistente (“Code Helper”) para auxiliar na implementação, com sintaxe correta, das instruções de um programa em *Assembly*. O Code Helper pode ser visto com mais detalhes no [Apêndice E - Ferramentas do Simulador CompSim](#);
- Integração com o montador (*Assembler*), o qual destaca, na cor vermelha, a linha do programa com algum tipo de erro, após análises léxica, sintática e semântica;
- Integração com um gerador de relatórios (“Asm Report”), que é um componente que utiliza o montador para realizar análises léxica, sintática e semântica no código-fonte, e, caso o código seja validado, gera um relatório com diversas informações do programa, tais como tabela de símbolos, endereços dos segmentos de memória, tamanho da pilha de programa e o respectivo código de máquina gerado.

A codificação de uma aplicação na linguagem *Assembly*, no CompSim, inicia com a criação de um arquivo de código-fonte, através do menu “File”, opção “New”, ou

pressionando as teclas de atalho “Ctrl+n”. Com isso, o editor de código (componente gráfico “Code”) cria um template de código-fonte em linguagem *Assembly*, como pode ser visto, ao centro e à esquerda da Figura 7.5.

Figura 7.5. Template de código-fonte em Assembly.



Fonte: Próprio autor (2021).

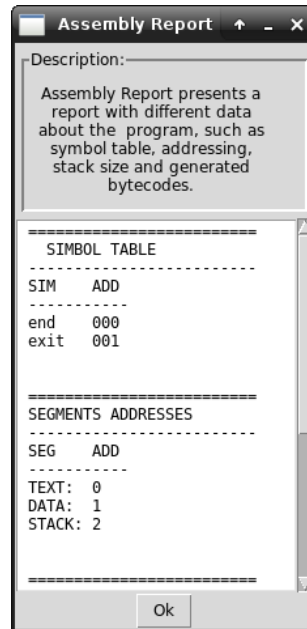
O *template* de código-fonte, apresentado na Figura 7.5, inclui a estrutura básica de um programa em linguagem *Assembly* do processador Cariri. O *template* pode ser editado para a codificação de novos programas que poderão ser executados na PHV.

Após a codificação do programa, é possível validá-lo, através da realização de análises léxica, sintática e semântica no código-fonte. Para tanto, deve-se acessar o menu “Program”, opção “Assembly report”, ou pressionar a tecla “F5”.

Caso os resultados das análises sejam bem-sucedidos, será apresentada uma nova janela contendo um relatório do programa, como o exemplo exposto na Figura 7.6.

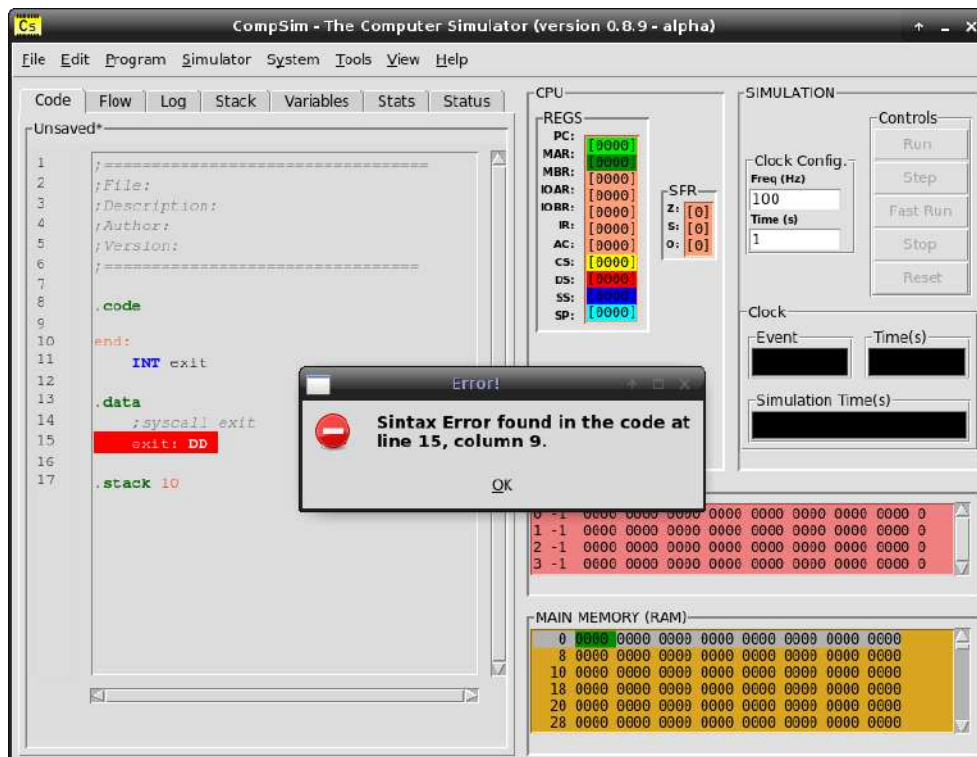
Caso os resultados da análises sejam mal-sucedidos, será destacada, em vermelho, a primeira linha no código-fonte que apresentou erro e será exibida uma mensagem indicando o tipo de erro e onde o mesmo se encontra (número de linha e de coluna) no código-fonte, como pode ser visto na Figura 7.7.

Figura 7.6. Exemplo de relatório de um programa em linguagem Assembly.



Fonte: Próprio autor (2021).

Figura 7.7. Exemplo de erro de sintaxe em um programa em linguagem Assembly.



Fonte: Próprio autor (2021).

De acordo com o exposto na Figura 7.7, o “Assembly report” identificou um erro de sintaxe na Linha 15 e Coluna 9 do código-fonte do programa. Neste tipo de situação, o programador terá que corrigir o erro e reiniciar a análise do código do programa.

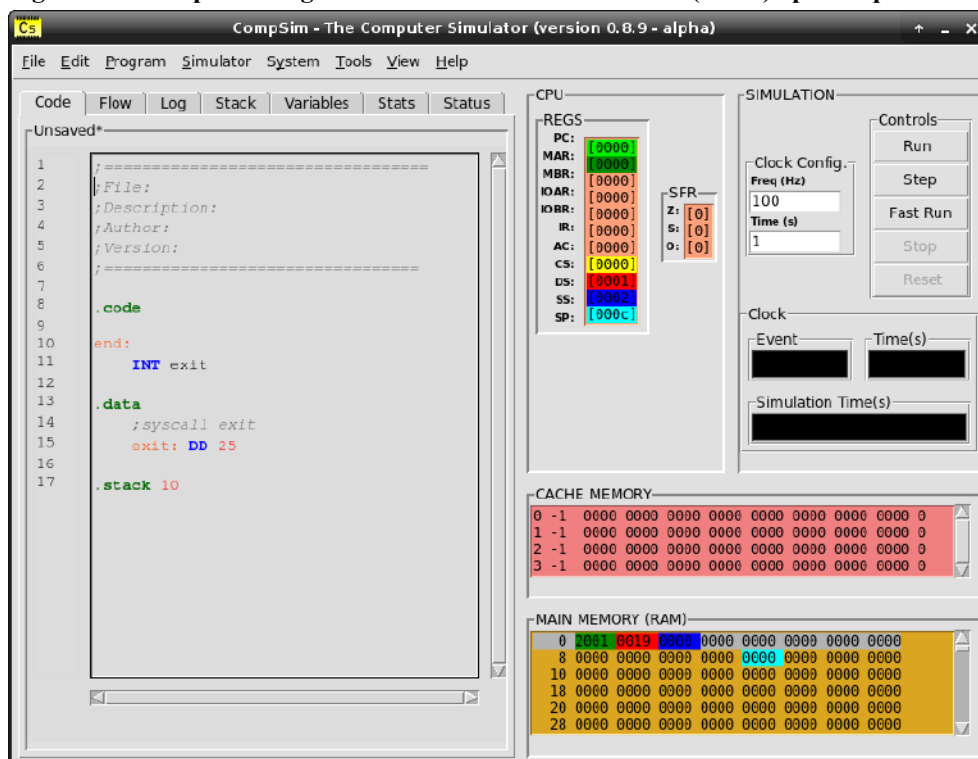
Uma vez que a codificação da aplicação é encerrada e validada, através das análises realizadas com o componente “Assembly report”, a etapa seguinte será a realização do mapeamento da aplicação para a PHV.

7.3 Mapeamento de uma Aplicação para a Plataforma de Hardware Virtual

A etapa de Mapeamento é fundamental pois ela é responsável por preparar a plataforma para execução da aplicação. Em mais detalhes, no CompSim, ela consiste em utilizar o montador (*Assembler*) para gerar código de máquina a partir do código criado em linguagem *Assembly*, carregá-lo na memória RAM e configurar os registradores do processador para apontar para diferentes posições de memória, tais como o início dos segmentos e o endereço da primeira instrução que será executada.

Para realizar o mapeamento, deve-se acessar o menu “Program”, opção “Assembly and Load”, ou pressionar a tecla “F6”. Após o mapeamento, é possível verificar que os valores contidos de alguns dos registradores da CPU e de algumas das posições da memória RAM foram modificados, como mostra a Figura 7.8.

Figura 7.8. Componentes gráficos CPU e MAIN MEMORY (RAM) após mapeamento.



Fonte: Próprio autor (2021).

Na Figura 7.8, pode-se perceber ainda que, em algumas posições da memória, há marcações em diferentes cores (e.g. verde escuro, vermelho, azul escuro e azul claro). Essas marcações indicam as posições de memória referenciadas por registradores da CPU, os quais estarão destacados com as respectivas cores.

Após o mapeamento, o sistema (aplicação e PHV) está pronto para ser executado. Os passos seguintes consistem da configuração da simulação, seguida da execução da simulação em si.

7.4 Configuração de Simulação e Execução da Simulação

A etapa de configuração de simulação consiste em definir os seguintes parâmetros: a frequência de *clock* do sistema, medida em Hertz (Hz), e o tempo total de simulação, medido em Segundos (s). O componente gráfico “SIMULATION” inclui campos onde é possível definir essas configurações, como pode ser visto na Figura 7.8.

O componente gráfico “SIMULATION” inclui ainda botões para gerenciamento de simulação. Com os botões é possível iniciar uma simulação em diferentes modos, pará-la e reiniciá-la. Mais especificamente, os botões são:

1. *Run*: este botão ativa o modo de simulação convencional. Neste modo, o *clock* do sistema gera eventos continuamente, respeitando a frequência configurada, de forma que o processador Cariri inicia e segue realizando o ciclo de instrução até o fim da simulação. Ainda neste modo, é possível visualizar os status dos componentes da PHV e do programa, durante a execução da simulação;
2. *Step*: este botão ativa o modo de simulação passo-a-passo. Neste modo, a cada clique no botão, gera-se um evento de *clock* de sistema. Neste modo, é possível avançar a simulação passo-a-passo e, com isso, acompanhar com precisão a progressão dos status dos componentes da PHV e do programa em execução;
3. *Fast Run*: este botão ativa o modo de simulação rápida. Neste modo, é possível realizar uma simulação contínua, assim como no modo convencional, porém com maior desempenho. Outra diferença deste modo de simulação para o convencional é que, no modo de simulação rápida não é possível visualizar os status dos componentes da PHV e do programa em execução. Este modo de simulação é normalmente

utilizado quando o projeto do sistema computacional está concluído e deseja-se apenas executá-lo de maneira mais eficiente ou para interação com periféricos;

4. *Stop*: ao clicar neste botão, uma simulação em andamento é encerrada. Dependendo do modo de simulação escolhido (*Run* ou *Step*), é possível visualizar os status finais dos componentes da PHV e do programa em execução; e
5. *Reset*: após encerrada uma simulação, este botão permite reconfigurar a PHV e o programa para os estados iniciais, habilitando assim o sistema para uma nova simulação.

7.5 Visualização

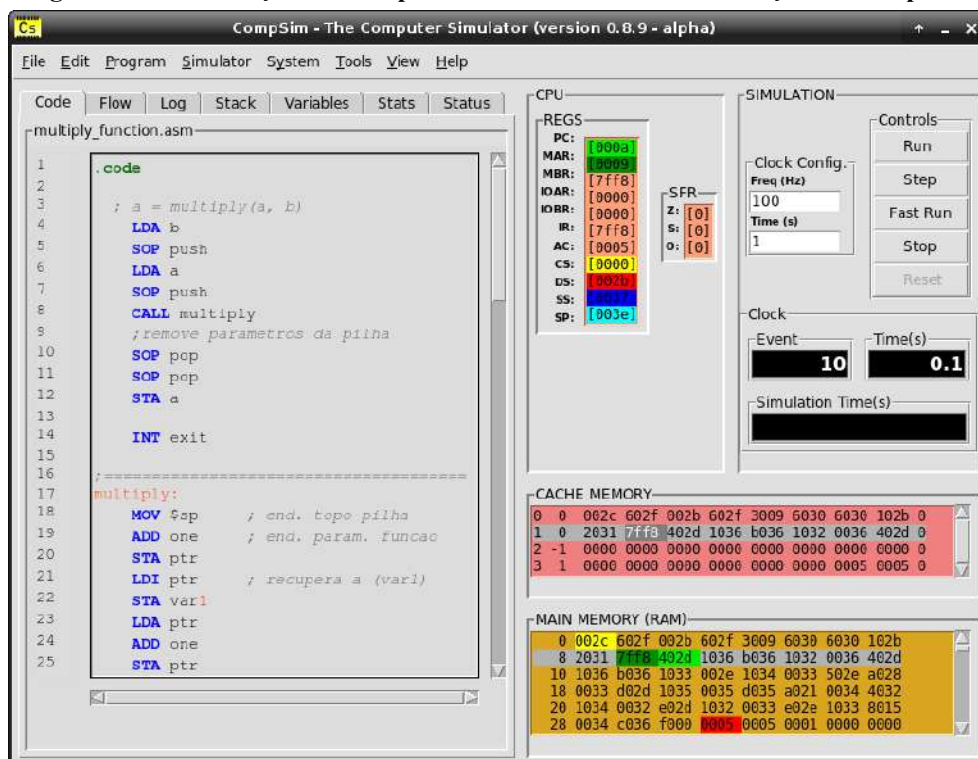
O simulador CompSim oferece recursos para visualização de diferentes aspectos dos componentes da PHV e do programa em execução, durante uma simulação nos modos convencional (*Run*) e passo-a-passo (*Step*).

Como foi visto anteriormente, o CompSim dispõe dos componentes gráficos “CPU”, “CACHE MEMORY” e “MAIN MEMORY (RAM)” que apresentam os conteúdos dos registradores, das linhas da memória Cache e de cada uma das posições da memória RAM, respectivamente. Durante uma simulação convencional ou passo-a-passo, esses componentes gráficos atualizam os status dos respectivos componentes de hardware virtual, a cada evento de *clock*. A Figura 7.9 ilustra um cenário de simulação, cujo componente gráfico:

- “SIMULATION”: mostra que o *clock* do sistema gerou, até aquele momento, 10 eventos e o tempo de simulação encontra-se em 0.1 segundos;
- “CPU”, nos registradores:
 - “PC” aponta para a próxima instrução que será executada, cujo endereço de memória é “000a”;
 - “MAR” está endereçando a posição de memória “0009”;
 - “MBR” contém a palavra “7ff8”;
 - “IR” contém a instrução “7ff8”; e
 - AC contém o valor “0005”.
- “MEMORY CACHE”: destaca a segunda linha e a palavra “7ff8”, significando que foram acessadas pela CPU. Observar que a palavra destacada é idêntica às palavras contidas nos registradores MBR e IR, o que indica que consiste de uma instrução que foi lida da memória pela CPU (etapa de *fetch*);

- “MAIN MEMORY (RAM)”’: destaca a segunda linha, o que reflete que é a linha que contém a instrução sendo endereçada pela CPU. É importante observar que, para que a CPU tenha acesso a uma palavra de memória, um bloco de memória RAM, que no CompSim consiste de uma linha da memória RAM, deve estar mapeado em uma das linhas da memória Cache. Na figura, pode-se observar que o conteúdo da linha destacada na memória Cache é o mesmo da linha destacada na memória RAM, o que indica que realizou-se o mapeamento com sucesso. Ainda na memória RAM, é possível verificar que, na linha destacada, algumas palavras encontram-se nas cores verde claro e verde escuro. A palavra destacada em verde escuro refere-se à palavra endereçada pelo registrador MAR da CPU. Essa palavra foi transferida para a CPU, pois o mesmo valor está contido no registrador MBR, e consiste de uma instrução, pois também está contido no registrador IR. Já a palavra destacada em verde claro consiste da próxima instrução que será executada, pois ela está sendo endereçada pelo registrador PC, destacado também na mesma cor.

Figura 7.9. Visualização dos componentes de hardware em simulação no CompSim.

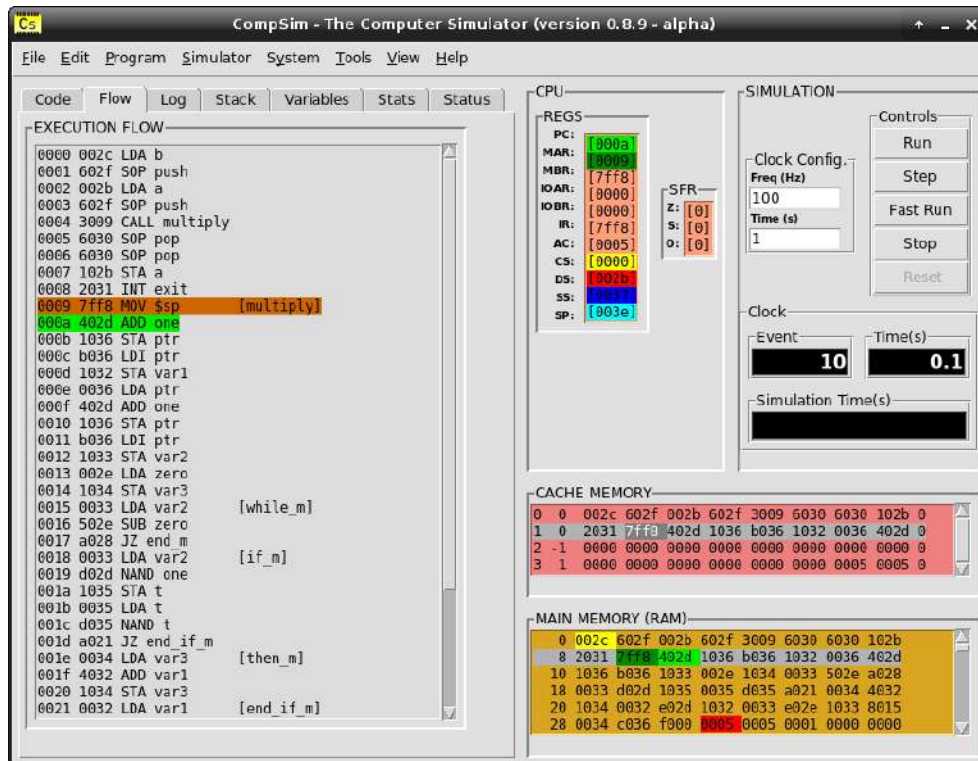


Fonte: Próprio autor (2021).

Além dos componentes gráficos de visualização dos componentes da PHV, o simulador CompSim traz, nas abas vizinhas à aba “Code”, os seguintes componentes gráficos:

- Aba “Flow”, componente “EXECUTION FLOW”: permite o acompanhamento do fluxo de execução do programa, durante uma simulação. Ele apresenta os endereços de memória em sequência, com as respectivas instruções do programa em execução em linguagem de máquina e em linguagem *Assembly*; e destaca na cor marrom a instrução em execução e, em verde claro, a próxima instrução a ser executada (mesma cor que é utilizada em destaque do registrador PC), como pode ser visto na Figura 7.10.

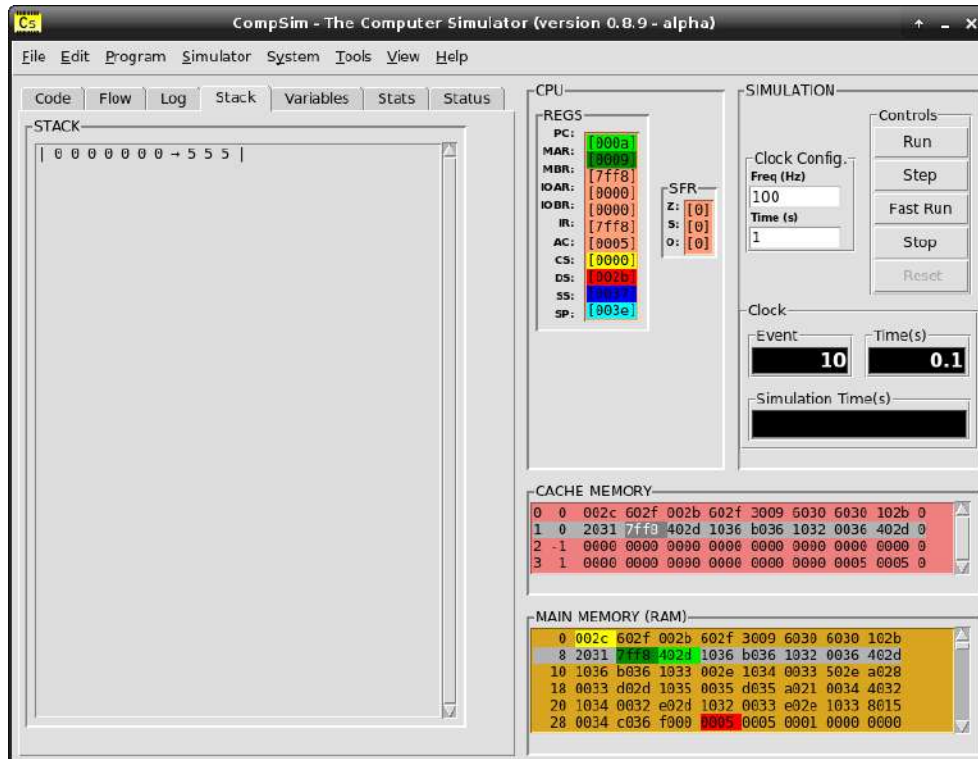
Figura 7.10. Visualização de fluxo de execução do programa em simulação no CompSim.



Fonte: Próprio autor (2021).

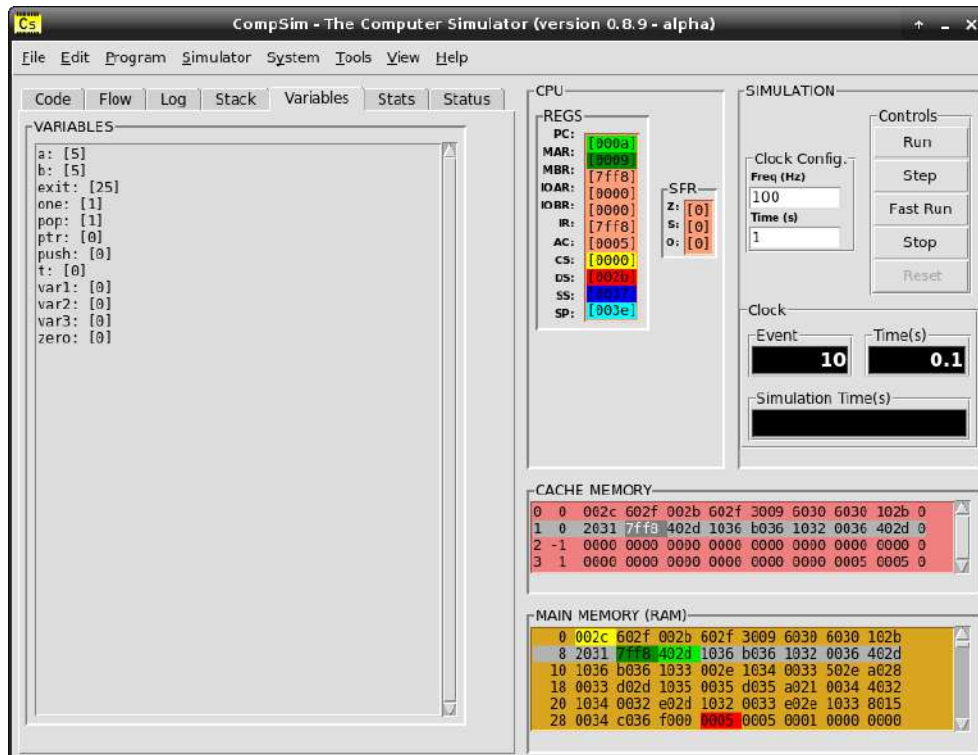
- Aba “Stack”, componente “STACK”: permite o acompanhamento dos status da pilha do programa. Durante uma simulação, são apresentados os dados que são adicionados na pilha do programa e um indicador do topo da pilha. A Figura 7.11 ilustra a visualização da pilha do programa, onde observa-se que foram adicionados os números “5”, “5” e “5”, bem como o indicador do topo da pilha (símbolo “→”), o qual aponta para o número “5” mais à esquerda (topo da pilha); e
- Aba “Variables”, componente “VARIABLES”: permite visualizar, durante a simulação, a alteração dos valores assumidos pelas variáveis e estruturas de dados do programa. Na Figura 7.12, é possível visualizar, por exemplo, os valores assumidos, em base decimal, pelas variáveis “a”, “b”, “exit”, “one”, “pop”, “ptr”, entre outras.

Figura 7.11. Visualização de pilha do programa em simulação no CompSim.



Fonte: Próprio autor (2021).

Figura 7.12. Visualização das variáveis do programa em simulação no CompSim.



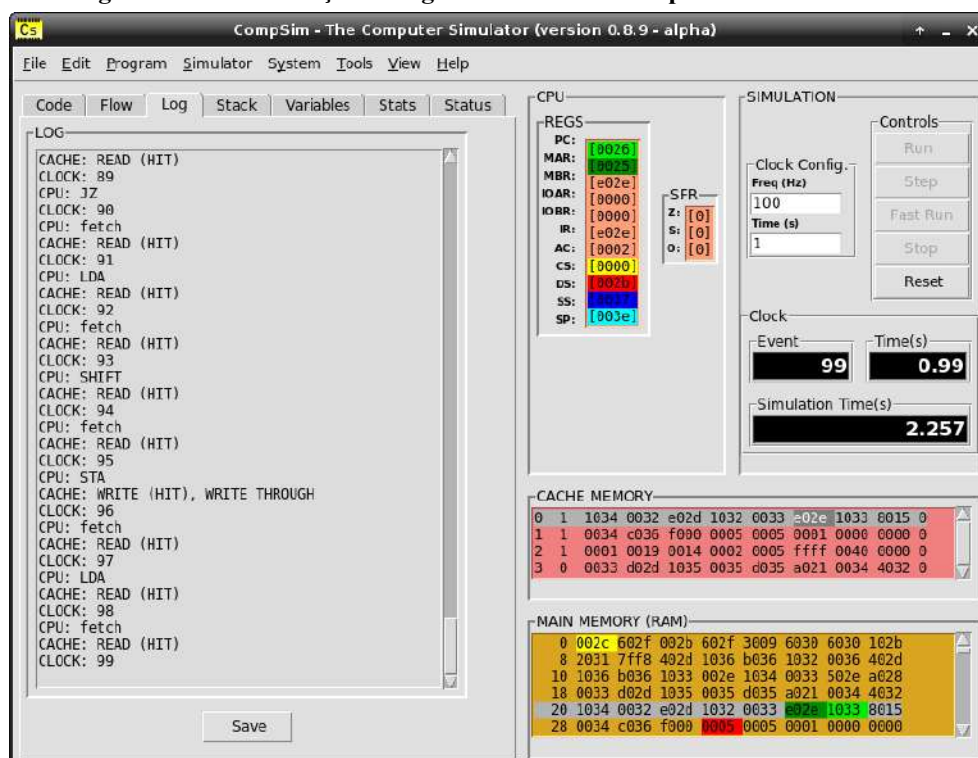
Fonte: Próprio autor (2021).

7.6 Análise de Desempenho

O simulador CompSim traz ainda ferramentas para análise de desempenho do sistema computacional, após uma simulação. Podem ser analisados os componentes de hardware virtual, tais como *clock* do sistema, processador, memórias Cache e RAM. Dentre as ferramentas de análise de desempenho, há:

- Aba “Log”, componente “LOG”: apresenta descrições de todos os eventos ocorridos com os componentes de hardware simulável, durante uma simulação. Dentre os eventos apresentados, tem-se: eventos de *clock*, etapas do ciclo de instrução do processador, instruções decodificadas pelo processador, acertos (*Hits*) e faltas (*Misses*) em memória cache, entre outros. O componente “LOG” permite ainda salvar sua lista de eventos (log) em arquivo no disco, através do botão “Save”, como pode ser visto na Figura 7.13. Com o suporte de “LOG”, é possível analisar detalhadamente o comportamento de cada componente de hardware simulável e identificar gargalos no desempenho, como, por exemplo, excessos de *misses* na memória Cache, o que, por consequência, gerará mais acessos e transferências de blocos entre memórias Cache e RAM no barramento;

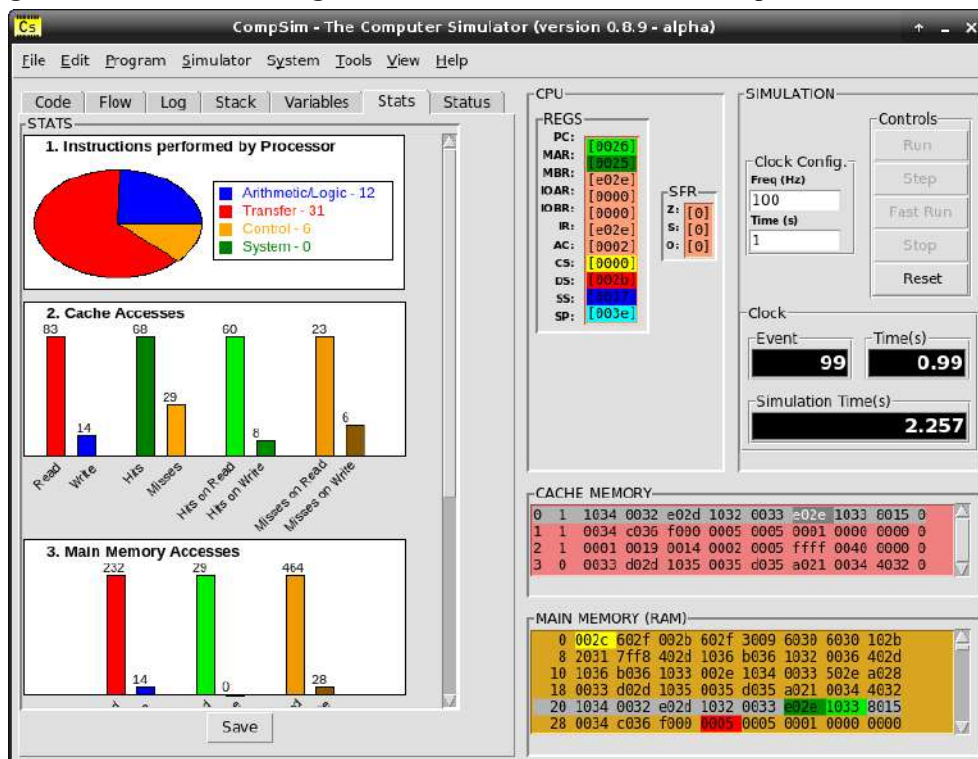
Figura 7.13. Visualização de logs de rventos dos componentes de hardware.



Fonte: Próprio autor (2021).

- Aba “Stats”, componente “STATS”: após encerrar uma simulação, o simulador CompSim sumariza todos os eventos gerados pelos componentes processador, memórias Cache e RAM, e barramentos, e cria gráficos estatísticos, como podem ser vistos na Figura 7.14. Dentre os gráficos estatísticos, estão: Gráfico de Setores (ou Pizza) com comparativo entre os quantitativos dos tipos de instruções executadas pelo processador; e Gráficos de Barras com totais de leituras e escritas, bem como *hits* e *misses* em memória Cache, totais de blocos transferidos em leituras e escritas em memória RAM, totais de operações de leitura e escrita em periféricos, entre outros; e

Figura 7.14. Visualização de gráficos estatísticos de eventos dos componentes de hardware.



Fonte: Próprio autor (2021).

- “Simulation Time(s)”, no componente gráfico “SIMULATION”: apresenta-se o tempo real de simulação e pode ser utilizado na avaliação do desempenho de simulações. Neste ponto, é importante compreender que, no ambiente de simulação do CompSim, há dois tipos de tempos: o tempo simulável, que é o intervalo de tempo fictício para que o sistema computacional virtual execute suas tarefas (este tempo é configurado através do componente gráfico “Time (s)”; e o tempo de simulação, que é o intervalo de tempo real despendido entre o início e o fim de uma simulação.

7.7 Resumo

Este capítulo apresentou a dinâmica para realização de simulações no simulador CompSim. As subseções detalharam os passos para: criação e configuração de plataformas de hardware simulável, criação e mapeamento de novas aplicações para execução nas plataformas criadas, e configuração e execução de simulações.

Além disso, foram apresentados recursos para visualização dos componentes de hardware simulável e diferentes elementos do programa em execução, durante uma simulação; bem como, recursos para análise de desempenho dos sistemas simulados, após as simulações.

CAPÍTULO 8 – MANIPULAÇÃO DE DADOS EM MEMÓRIA

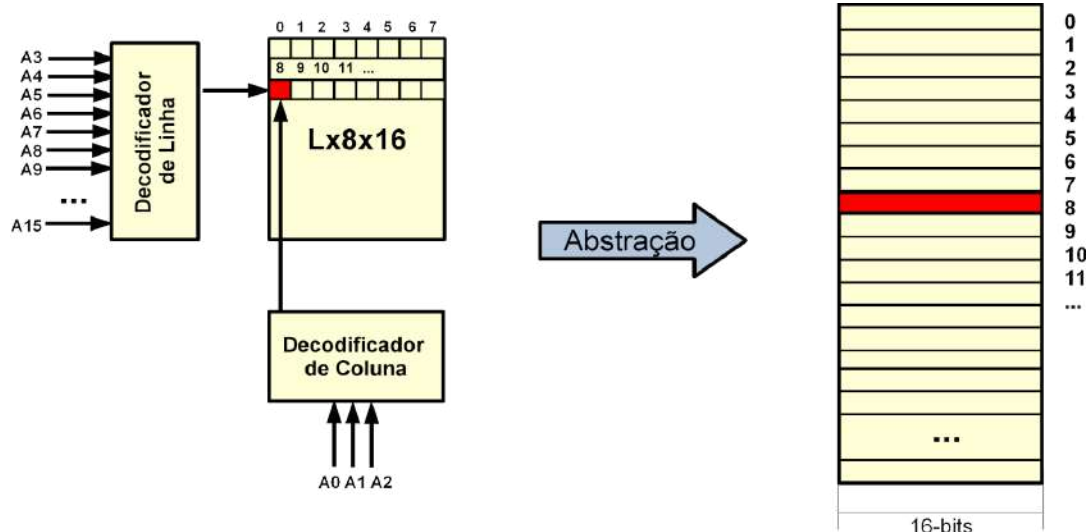
O computador foi criado para processamento de dados. Viu-se que os dados manipulados pelo computador, podem ser de diferentes tipos e necessitam obrigatoriamente estar armazenados na memória RAM, juntamente com as instruções dos programas em execução. No entanto, a memória RAM armazena sequências de bits ou de bytes, sendo portanto, do ponto de vista da memória, impossível diferenciar o que são dados ou instruções, ou os diferentes tipos de dados.

Este capítulo apresenta como um programa de computador pode ser estruturado e alocado na memória RAM e quais os mecanismos do processador do simulador Cariri são utilizados para dar suporte a essas tarefas. Na sequência, será visto como alocar diferentes estruturas de dados em memória, com diferentes tipos de dados, e como acessá-los, em operações de leitura e escrita em memória, no CompSim.

8.1 Abstração da Memória RAM

Como visto no [Capítulo 4 - Introdução à Organização de Computadores](#), a memória RAM consiste de uma matriz de dados, os quais podem ser acessados por meio dos endereços contidos no barramento de endereços do barramento de sistema, em operações de leitura e escrita, como está ilustrado à esquerda no diagrama da Figura 8.1. Na figura, a matriz de dados da memória RAM ilustrada possui “L” linhas, em que, em cada linha, há 8 colunas e em cada célula da matriz pode ser armazenado um dado de 16-bits (notação “Lx8x16”).

Figura 8.1. Abstração linear da matriz de dados da memória RAM.



Fonte: Próprio autor (2021).

Um endereço de memória inserido no barramento de endereços, direciona o acesso a uma determinada célula da matriz de dados. Dessa forma, o endereço é dividido em duas partes: algumas linhas (ou sinais) do barramento de endereços são utilizadas para seleção de uma das linhas da matriz de dados, e as demais linhas do barramento são utilizadas para seleção de uma das colunas, na linha selecionada. Na Figura 8.1, a memória ilustrada utiliza as linhas de endereços de “A3” à “A15” para endereçamento de linha, e as linhas de endereços de “A0” à “A2” para endereçamento de coluna.

De forma a simplificar o estudo de como a memória RAM está estruturada para suportar a execução de programas de computador, neste livro será adotada uma abstração da matriz de dados da memória RAM. A matriz passará a ser vista como uma estrutura de dados linear, na qual os endereços de memória e as respectivas células de dados serão vistos em uma perspectiva sequencial, como pode ser visto à direita da Figura 8.1. Com essa abstração, as células da matriz estarão dispostas em sequência em uma única coluna, cujos endereços e células de dados estarão em disposição *top-down* e com endereços em ordem crescente. Na Figura 8.1, por exemplo, na matriz de dados, vista à esquerda, a célula de armazenamento com endereço 8 está presente na primeira coluna da segunda linha. Já na estrutura abstrata linear, à direita na Figura 8.1, essa mesma célula passa a ocupar o elemento da coluna com endereço 8.

8.2 Modelo de Memória de um Programa

Um programa de computador, para ser executado, deve estar alocado em memória RAM, pois é lá que o processador busca as instruções e os dados necessários para realizar suas tarefas no ciclo de instrução.

Em cada célula de dados da memória RAM, é possível armazenar uma palavra, que consiste de uma sequência de bits ou de bytes. As células podem ser utilizadas para armazenamento de diferentes tipos de dados e de instruções, desde que estejam codificados em bits.

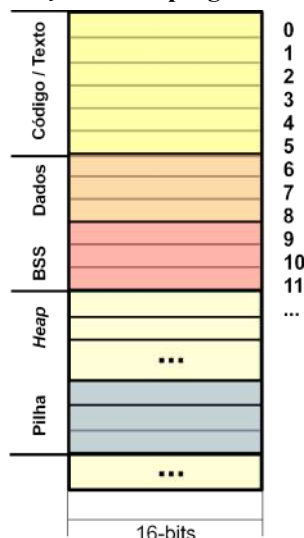
Para diferenciar dados e instruções, bem como reservar outros recursos na memória RAM, normalmente, os programas de computador, quando alocados em memória, estarão divididos em segmentos, como:

1. Segmento de Código ou de Texto (*Code/Text Segment*): inclui todas as instruções do programa;
2. Segmento de Dados (*Data Segment*): inclui todas as variáveis e estruturas de dados globais ou estáticas, que foram definidas (inicializadas com valores na sua criação);
3. Segmento BSS (*Block Started by Symbol Segment*): inclui todas as variáveis e estruturas de dados globais ou estáticas, que foram apenas declaradas (não foram inicializadas);
4. *Heap*: é um segmento que contém espaços de memória não utilizados, os quais podem ser demandados em operações de alocação dinâmica de memória, tais como pelo uso das instruções “malloc” e “calloc” da linguagem de programação C;
5. Segmento de Pilha (*Stack Segment*): este segmento é utilizado para implementar a pilha do programa (*program stack*), que basicamente é uma estrutura de dados do tipo LIFO (*Last-In, First-Out*) e pode ser utilizada para diversos fins, tais como: para armazenamento temporário de dados, passagem de parâmetros de entrada e de retorno em funções, implementação de variáveis locais, entre outros.

A Figura 8.2 ilustra como um programa de computador, após ser segmentado, é alocado em memória RAM. Na figura, observa-se que o primeiro segmento é o de código/texto, seguido pelos segmentos de dados, BSS, *Heap* e de pilha. O *Heap* pode ser dinamicamente redimensionado, aumentando em tamanho no sentido do segmento de pilha, para dispor de mais memória livre e alocável ao programa. No segmento de pilha, os elementos adicionados na pilha do programa são alocados nos endereços finais do segmento, de forma que a pilha cresce no sentido em direção ao *Heap* (o apontador de topo da pilha será

decrementado a cada adição e incrementado a cada remoção de elementos na pilha do programa).

Figura 8.2. Segmentação de um programa em memória RAM.



Fonte: Próprio autor (2021).

No simulador CompSim, um programa, alocado em memória, pode ser dividido em segmentos de código/texto, dados e pilha. É possível ainda reservar espaços livres entre os segmentos, para, por exemplo, implementar um *Heap* entre os segmentos de dados e de pilha, como será visto adiante.

8.3 Suporte do Processador Cariri para Segmentação da Memória de um Programa

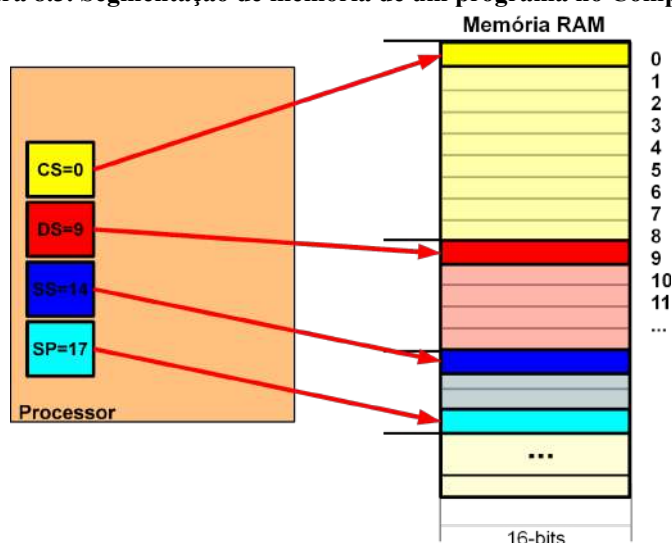
O processador Cariri conta com 4 (quatro) registradores para dar suporte à segmentação da memória de um programa. São eles:

1. Registrador de Segmento de Código (*Code Segment - CS*): armazena o endereço da primeira posição de memória alocada para o segmento de código;
2. Registrador de Segmento de Dados (*Data Segment - DS*): armazena o endereço da primeira posição de memória alocada para o segmento de dados;
3. Registrador de Segmento de Pilha (*Stack Segment - SS*): armazena o endereço da primeira posição de memória alocada para o segmento de pilha; e

4. Registrador Apontador de Topo de Pilha (*Stack Pointer* - SP): aponta para o topo da pilha do programa.

Com suporte dos registradores de segmentos, o processador Cariri tem ciência dos limites de cada um dos segmentos. A Figura 8.3 ilustra o uso dos registradores CS, DS, SS e SP para segmentação da memória de um determinado programa. Na figura, os registradores CS, DS, SS e SP apontam para os endereços 0, 9, 14 e 17, respectivamente. Com essa disposição, percebe-se que alocou-se 9 espaços de memória para comportar as instruções do programa, 5 espaços para os dados e 3 espaços para a pilha do programa (a última posição alocada para a pilha do programa somente é utilizada para sinalizar que a pilha está vazia, portanto não sendo utilizada).

Figura 8.3. Segmentação de memória de um programa no CompSim.



Fonte: Próprio autor (2021).

8.4 Estrutura Básica de um Programa em Assembly no CompSim

No capítulo passado, viu-se que ao criar um novo arquivo de código-fonte no CompSim, automaticamente, é apresentado um *template* de código em linguagem *Assembly* do processador Cariri, que pode ser editado para a criação de novas aplicações. O *template* pode ser visto na Figura 8.4.

Figura 8.4. Template de código Assembly do Processador Cariri.

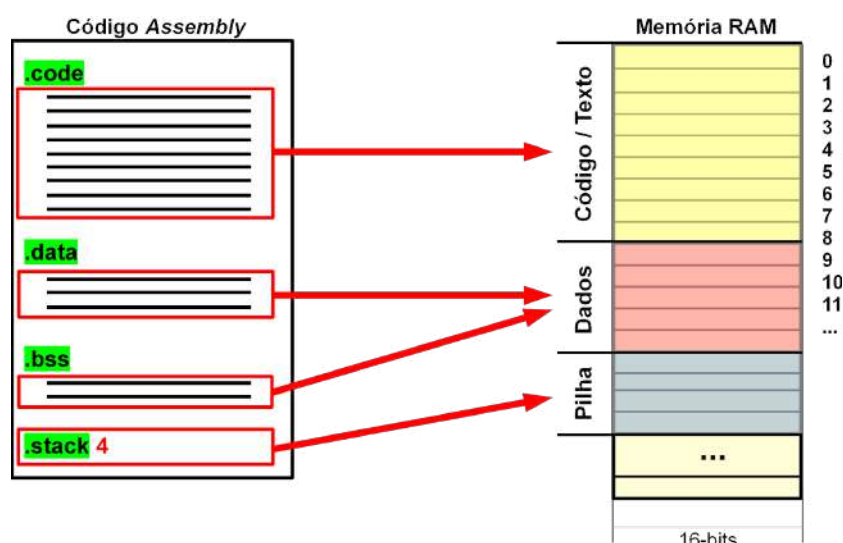
Linha	Código
1	<code>;</code> =====
2	<code>;</code> File:
3	<code>;</code> Description:
4	<code>;</code> Author:
5	<code>;</code> Version:
6	<code>;</code> =====
7	
8	<code>.code</code>
9	
10	
11	<code>end:</code>
12	<code>INT exit</code>
13	<code>.data</code>
14	<code>;</code> syscall exit
15	<code>exit: DD 25</code>
16	
17	<code>.stack 10</code>

Fonte: Próprio autor (2021).

Na Figura 8.4, as Linhas 8, 13 e 17 incluem as palavras-chave “.code”, “.data” e “.stack”, que são delimitadores de seções em um código *Assembly* e indicam: o início da seção que conterá todas as instruções do programa; o início da seção que conterá a definição (declaração com inicialização) de variáveis e estruturas de dados globais e estáticas do programa; e o tamanho da pilha, respectivamente. A palavra-chave “.stack” deve ser sucedida por um número inteiro positivo, o qual informa a quantidade de posições de memória que serão alocadas para a pilha do programa. Entre as seções “.data” e “.stack”, é possível ainda acrescentar a palavra-chave “.bss”, que indicará o início da seção que conterá a declaração (sem inicialização) de variáveis e estruturas de dados globais e estáticas do programa.

Ao analisar a estrutura de um código-fonte em *Assembly*, é possível verificar que existe um mapeamento direto entre as seções do código e os segmentos de memória do programa. Por exemplo, as instruções contidas na seção “.code” serão mapeadas no segmento de código/texto do programa em memória. Da mesma forma, as variáveis e estruturas de dados definidas e/ou declaradas nas seções “.data” e “.bss”, respectivamente, serão mapeadas no segmento de dados do programa em memória. Por fim, a seção “.stack” trará uma indicação do número de posições de memória que serão alocadas para a pilha do programa. A Figura 8.5, ilustra o mapeamento direto entre as seções de código *Assembly* e os respectivos segmentos de memória.

Figura 8.5. Mapeamento entre seções de código Assembly em segmentos de programa em Memória.



Fonte: Próprio autor (2021).

8.5 Criação de Variáveis e Estruturas de Dados em Memória

No CompSim, a criação de variáveis e estruturas de dados é dada pelo uso de pseudo-instruções do montador (*Assembler*). Uma pseudo-instrução é um tipo de instrução que existe na linguagem *Assembly*, porém é somente reconhecida pelo montador (não faz parte do conjunto de instruções do processador).

Após o processo de montagem do código-fonte, durante o carregamento do programa em memória, o montador fica responsável por alocar memória suficiente para comportar as estruturas de dados criadas com as respectivas pseudo-instruções no código-fonte.

O processador Cariri suporta dois tipos de dados: números inteiros de 16-bits com sinal, codificados com a notação Complemento a 2 (ver [Apêndice B - Introdução à Aritmética Computacional](#)) e caractere (*char*), codificados com o padrão ASCII (ver [Apêndice D - Tabela ASCII](#)). As subseções a seguir apresentam diferentes maneiras de criação de variáveis e estruturas de dados na linguagem *Assembly* do processador Cariri.

8.5.1 Definição de Variáveis e Estruturas de Dados

A definição de uma variável, ou de uma estrutura de dados, consiste na sua declaração e atribuição de um valor inicial. As definições devem ser realizadas exclusivamente na seção “.data” do código *Assembly* do processador Cariri.

Uma variável do tipo inteiro, no CompSim, contém um número inteiro de 16-bits com sinal. A definição de variáveis do tipo inteiro pode ser realizada através da pseudo-instrução “DD”, da linguagem *Assembly* do processador Cariri. O quadro a seguir traz um resumo com: uma descrição da pseudo-instrução DD; em qual seção do código pode ser utilizada; sua sintaxe de uso; e, exemplos de como utilizá-la. É importante ressaltar que é possível informar, em bases decimal, hexadecimal e binária, os valores que serão utilizados para inicialização de uma variável inteira. É possível ainda definir variáveis inteiras inicializadas com números negativos (caso utilize as bases hexadecimal e binária para representação do número, deve-se observar como números inteiros negativos são representados em notação Complemento a 2, como mostra o [Apêndice B - Introdução à Aritmética Computacional](#)).

Pseudo-Instrução: DD	
Descrição: Utilizada para definição de uma variável do tipo inteiro.	
Seção: .data	
Sintaxe: <nome>. DD <inteiro>	
Exemplos: var1: DD 10 ;variavel inteira inicializada com o valor 10 em decimal. var1: DD 0xa ;variavel inteira inicializada com o valor 10 em hexadecimal. var1: DD 1010b ;variavel inteira inicializada com o valor 10 em binário. var2: DD -1 ;variavel inteira inicializada com o valor -1 em decimal. var2: DD 0xffff ;variavel inteira inicializada com o valor -1 em hexadecimal. var2: DD 1111111111111111b ;variavel inteira inicializada com o valor -1 em decimal.	

Já uma variável do tipo caractere (*char*), no CompSim, armazena o respectivo código ASCII do caractere. Na definição de uma variável do tipo caractere, é possível também inicializá-la com uma cadeia de caracteres (*string*), de forma que a variável criada apontará para o primeiro caractere da cadeia.

A definição de variáveis do tipo caractere pode ser realizada através da pseudo-instrução “DB”, da linguagem *Assembly* do processador Cariri. O quadro a seguir traz um resumo com: uma descrição da pseudo-instrução DB; em qual seção do código pode ser utilizada; sua sintaxe de uso; e, exemplos de como utilizá-la na criação de variáveis com caracteres e cadeias de caracteres.

Pseudo-Instrução: DB
Descrição: Utilizada para definição de uma variável dos tipos <i>char</i> ou <i>string</i> .
Seção: .data
Sintaxe: <nome>: DB '<char>' <nome>: DB "<string>"
Exemplos: var1: DB 'A' ;variavel do tipo char inicializada com 'A'. var2: DB "Hello, World!" ;variavel do tipo string inicializada com "Hello, World!".

O Compsim suporta a definição de estruturas de dados do tipo matriz, implementadas através de arrays (matrizes multidimensionais) e vetores (matrizes com uma única dimensão). Neste livro, será utilizado o termo “array/vetor” para fazer referência à estrutura de dados matriz, quer seja uni ou multidimensional, visto que, do ponto de vista de alocação de memória, as linhas de uma matriz multidimensional são armazenadas sequencialmente em memória, assumindo assim uma disposição linear (vetorial). Quando for estritamente necessário diferenciar o tipo de estrutura de dados, os termos “array”, “vetor” e “matriz” serão utilizados de forma dissociada.

A definição de arrays/vetores do tipo inteiro de 16-bits com sinal pode ser realizada através da pseudo-instrução “INITD”, da linguagem *Assembly* do processador Cariri. O quadro a seguir traz um resumo com: uma descrição da pseudo-instrução INITD; em qual seção do código pode ser utilizada; sua sintaxe de uso; e, exemplos de como utilizá-la.

Pseudo-Instrução: INITD
Descrição: Utilizada para definição de um array/vetor do tipo inteiro.
Seção: .data
Sintaxe: <nome>: INITD <inteiro1>, <inteiro2>, ...
Exemplo: array1: INITD 1, 2, 3, 4, 5 ;array/vetor do tipo inteiro inicializado com 1,2,3,4,5. array2: INITD -1, -2, -3, -4, -5 ;array/vetor do tipo inteiro inicializado com -1,-2,-3,-4,-5.

Um array/vetor de caracteres consiste, na realidade, em uma cadeia de caracteres (*string*). Já uma cadeia de caracteres, por sua vez, em termos de alocação em memória, apresenta-se como um array/vetor de números inteiros, visto que os caracteres da cadeia são codificados no padrão ASCII.

A definição de arrays/vetores do tipo caractere pode ser realizada através da pseudo-instrução “INITB”, da linguagem *Assembly* do processador Cariri. O quadro a seguir traz um resumo com: uma descrição da pseudo-instrução INITB; em qual seção do código pode ser utilizada; sua sintaxe de uso; e, exemplos de como utilizá-la na criação de arrays/vetores de caracteres (*strings*) e de cadeias de caracteres. Como pode ser visto no quadro a seguir, a pseudo-instrução INITB permite criar array/vetores de caracteres (*char*), de cadeias de caracteres (*string*) e de números inteiros, bem como combinar esses elementos em uma única estrutura de dados.

Pseudo-Instrução: INITB	
Descrição: Utilizada para definição de um array/vetor dos tipos <i>char</i> , <i>string</i> e/ou inteiro.	
Seção: .data	
Sintaxe: <nome>: INITB ‘<char1>’, ‘<char2>’, ... <nome>: INITB “<string1>”, “<string2>”, ... <nome>: INITB ‘<char>’, “<string>”, <inteiro>, ...	
Exemplos: array2: INITB ‘A’, ‘B’, ‘C’, ‘D’, ‘E’ ;array/vetor do tipo char inicializado com “ABCDE”. array3: INITB “Hello”, ” World!” ;array/vetor do tipo char inicializado com “Hello, World!”. array4: INITB “Hello”, ” World!”, 0 ;array/vetor do tipo char inicializado com “Hello, World!\0”. array5: INITB “Hello”, “World”, ‘!’, 0 ;array/vetor do tipo char inicializado com “Hello, World!\0”.	

A definição de estruturas de dados mais complexas, tais como matrizes e registros, também pode ser realizada com apoio das instruções INITD e INITB.

Uma matriz, independente das suas dimensões, do ponto de vista de memória, consiste de um conjunto de posições de memória alocadas em endereços consecutivos. No exemplo da Figura 8.6, as definições em código C das matrizes “mA”, “mB” e “mC”, com dimensões 1x6, 2x3 e 3x2, respectivamente, apesar de sugerirem disposições diferenciadas para estruturação dos seus 6 elementos, não fazem distinções na forma como serão alocadas em memória e como os seus elementos serão armazenados (nos três casos, os elementos serão guardados em posições consecutivas na memória). Naturalmente, em código C, haverá diferenças em como os elementos serão referenciados em cada uma das matrizes. Por exemplo, para referenciar o inteiro 4 na matriz “mA” utiliza-se o índice “mA[3]”, na matriz “mB” utiliza-se “mB[1][0]” e na matriz “mC” utiliza-se “mC[1][1]”.

Figura 8.6. Exemplo de código C para definição de matrizes.

Linha	Código
1	<code>int mA[6] = {1, 2, 3, 4, 5, 6};</code>
2	<code>int mB[2][3] = {{1, 2, 3}, {4, 5, 6}};</code>
3	<code>int mC[3][2] = {{1, 2}, {3, 4}, {5, 6}};</code>

Fonte: Próprio autor (2021).

Já um registro é uma estrutura de dados mais complexa, pois, diferente das matrizes, pode agregar elementos de tipos diferentes em uma mesma estrutura de dados, que são referenciados através de rótulos individuais não numéricos (não utiliza-se índices, como nas matrizes). No exemplo de código C da Figura 8.7, cria-se, nas Linhas 1 à 5, um tipo de registro, chamado “reg”, que contém campos para guardar uma *string* com até 5 caracteres (“nome”), uma *string* com até 10 caracteres (“endereço”) e um número inteiro (“numero”). Na Linha 7, cria-se um registro chamado “registro1”, do tipo “reg”, que tem seus campos “nome”, “endereço” e “numero” inicializados com “José”, “Rua Silva” e 151, respectivamente. Da mesma forma como ocorre com as matrizes, os campos de um registro são alocados na mesma ordem em que são declarados em posições com endereços consecutivos de memória. Na linguagem C, para referenciar o campo “nome” do registro “registro1” utiliza-se a notação “registro1.nome”, para o campo “endereço” utiliza-se “registro1.endereço” e para o campo “numero” utiliza-se “registro1.numero”.

Figura 8.7. Exemplo de código C para definição de registro.

Linha	Código
1	<code>struct reg{</code>
2	<code> char nome[5];</code>
3	<code> char endereco[10];</code>
4	<code> int numero;</code>
5	<code>};</code>
6	
7	<code>struct reg registro1 = {"José", "Rua Silva", 151};</code>

Fonte: Próprio autor (2021).

Curiosidade!

Quando um programa em linguagem C contém estruturas de dados complexas, tais como matrizes ou registros, é compilado, os índices ou os rótulos são traduzidos em endereços de memória calculados a partir de deslocamentos relativos ao endereço de memória inicial (endereço base) alocado para a estrutura de dados. Por exemplo, considerando uma matriz “mB”, com dimensões 2x3, para referenciar um de seus elementos, calcula-se o endereço de memória relativo ao endereço base da matriz, a partir da seguinte equação:

$$\&\text{mB}[x][y] = \text{mB}_{\text{end_base}} + x \times \text{elemento}_{\text{tamanho}} \times N + y \times \text{elemento}_{\text{tamanho}}$$

, onde:

- x é o índice de linha;
- y é o índice de coluna;
- $\text{mB}_{\text{end_base}}$ é o endereço de memória inicial (endereço base) alocado para a matriz;
- $\text{elemento}_{\text{tamanho}}$ é o tamanho ocupado em memória por um elemento da matriz; e
- N é o número de elementos em uma linha da matriz.

Assim, para obter o endereço de memória do elemento cujo índice é “mB[1][1]”, considerando que a matriz está alocada em memória a partir do endereço 0xfff0 (endereço base) e que cada elemento da matriz ocupa 1 posição de memória, basta ajustar a equação anterior para:

$$\begin{aligned}\&\text{mB}[1][1] &= 0xfff0 + 1 \times 1 \times 3 + 1 \times 1 \\ \&\text{mB}[1][1] &= 0xfff4\end{aligned}$$

, onde 0xfff4 consiste no endereço de memória do elemento da matriz “mB” referenciado pelo índice “mB[1][1]”.

8.5.2 Declaração de Variáveis e Estruturas de Dados

Uma variável, ou uma estrutura de dados, pode ser declarada e não possuir um valor de inicialização. As declarações devem ser realizadas exclusivamente na seção “.bss” do código *Assembly* do processador Cariri. Desta forma, o montador reconhece, na declaração, o tipo e a quantidade de elementos que a estrutura de dados necessita e, com isso, aloca espaços de memória suficientes para comportá-la.

A declaração de variáveis ou de arrays/vetores do tipo inteiro pode ser realizada através da pseudo-instrução “RESB”, da linguagem *Assembly* do processador Cariri. O quadro a seguir traz um resumo com: uma descrição da pseudo-instrução RESB; em qual

seção do código pode ser utilizada; sua sintaxe de uso; e, exemplos de como utilizá-la na criação de variáveis e arrays/vetores de inteiros.

Pseudo-Instrução: RESB	
Descrição:	Utilizada para declaração de uma variável ou array/vetor do tipo inteiro.
Seção:	.bss
Sintaxe:	<nome>: RESB <inteiro>
Exemplos:	<pre> var1: RESB 1 ;variavel do tipo inteiro - será reservada apenas 1 posição de memória. array1: RESB 10 ;array/vetor do tipo inteiro - serão reservadas 10 posições de memória. array1: RESB 0xa ;array/vetor do tipo inteiro - serão reservadas 10 posições de memória. array1: RESB 1010b ;array/vetor do tipo inteiro - serão reservadas 10 posições de memória. </pre>

Já a declaração de variáveis ou de arrays/vetores do tipo caractere (*char*) pode ser realizada através da pseudo-instrução “RESB”, da linguagem *Assembly* do processador Cariri. O quadro a seguir traz um resumo com: uma descrição da pseudo-instrução RESB; em qual seção do código pode ser utilizada; sua sintaxe de uso; e, exemplos de como utilizá-la na criação de variáveis e arrays/vetores de caracteres.

Pseudo-Instrução: RESB	
Descrição:	Utilizada para declaração de uma variável ou array/vetor do tipo <i>char</i> .
Seção:	.bss
Sintaxe:	<nome>: RESB <inteiro>
Exemplos:	<pre> var2: RESB 1 ;variavel do tipo char - será reservada apenas 1 posição de memória. array2: RESB 10 ;array/vetor do tipo char - serão reservadas 10 posições de memória. array2: RESB 0xa ;array/vetor do tipo char - serão reservadas 10 posições de memória. array2: RESB 1010b ;array/vetor do tipo char - serão reservadas 10 posições de memória. </pre>

A declaração de estruturas de dados mais complexas, tais como matrizes e registros, também pode ser realizada com apoio das instruções RESB e RESB.

8.5.3 Alocação de Espaços Livres de Memória

Dependendo da aplicação, pode haver necessidade de alocar espaços livres em memória. Um exemplo é a implementação do segmento *Heap*, que pode ser utilizado para alocação dinâmica de memória para estruturas de dados.

A reserva de espaços livres em memória pode ser realizada através da pseudo-instrução “ORG”, da linguagem *Assembly* do processador Cariri. O quadro a seguir traz um resumo com: uma descrição da pseudo-instrução ORG; em quais seções do código pode ser utilizada; sua sintaxe de uso; e, um exemplo de como utilizá-la.

Pseudo-Instrução: ORG
Descrição: Utilizada para reservar espaços livres de memória entre instruções, estruturas de dados e/ou segmentos de memória.
Seções: .code, .data e .bss
Sintaxe: ORG <inteiro>
Exemplo: ORG 10 ;reserva 10 espaços livres de memória.

8.5.4 Exemplo Prático de Definição e Declaração de Variáveis e Arrays/Vetores

Esta subseção tem como objetivo ilustrar a estrutura de um programa em *Assembly* do processador Cariri, com definições e declarações de diferentes variáveis e arrays/vetores, bem como demonstrar como esses elementos são alocados em memória e como reserva-se espaços livres de memória entre os segmentos de dados e pilha. A Figura 8.8 inclui o código-fonte de estudo desta subseção.

Figura 8.8. Exemplo de código Assembly para definição e declaração de estruturas de dados.

Linha	Código
1	.code
2	
3	end:
4	INT exit
5	
6	.data
7	;syscall exit
8	exit: DD 25
9	var1: DD 10
10	var2: DB 'A'
11	array1: INITD 10,11,12
12	array2: INITB 'A',"BC",0

13	
14	.bss
15	var3: RESD 1
16	array3: RESB 3
17	
18	ORG 3
19	
20	.stack 2

Fonte: Próprio autor (2021).

O código-fonte em *Assembly* do processador Cariri apresentado na Figura 8.8 inclui definições e declarações de variáveis e arrays/vetores dos tipos inteiro e caractere (*char*).

Na Figura 8.8, na seção “.data”, que ocupa as Linhas 6 à 12, são definidos:

Na Figura 8.6, na seção “.data”, que ocupa as Linhas 6 à 12, são definidos:

- Na Linha 8, a variável “exit”, inicializada com o inteiro 25;
- Na Linha 9, a variável “var1”, inicializada com o inteiro 10;
- Na Linha 10, a variável “var2”, inicializada com o caractere ‘A’;
- Na Linha 11, o array/vetor “array1”, inicializado com os inteiros 10,11,12; e
- Na Linha 12, o array/vetor “array2”, inicializado com o caractere ‘A’, a cadeia de caracteres “BC” e o inteiro 0, que equivalem à string “ABC\0”.

Ainda na Figura 8.8, na seção “.bss”, que ocupa as Linhas 14 à 16, são declarados:

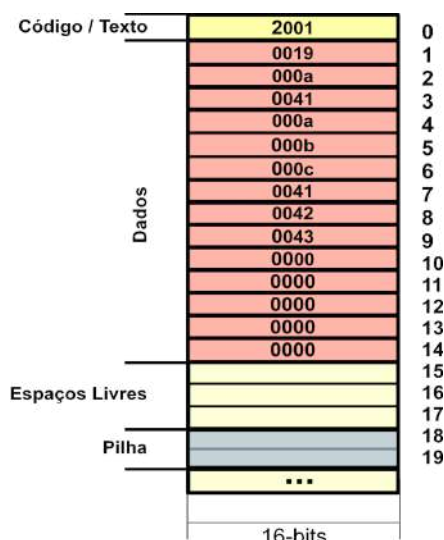
- Na Linha 15, a variável “var3”, do tipo inteiro, e será reservado 1 espaço de memória para ela; e
- Na Linha 16, o array/vetor “array3”, do tipo char, e serão reservados 3 espaços de memória para ele.

Ainda na Figura 8.8, a Linha 18 inclui a pseudo-instrução “ORG”, a qual instrui o montador a alocar 3 espaços livres de memória entre os segmentos de dados e pilha. A Figura 8.9 ilustra a memória alocada para o programa apresentado na Figura 8.8.

Na Figura 8.9, pode ser visto que o segmento de código/texto contém apenas uma posição de memória, referente à instrução na Linha 4, do código-fonte da Figura 8.8. O segmento de dados contém no endereço:

- 1, o valor hexadecimal 0019, referente à variável “exit” com valor 25, em decimal;
- 2, o valor hexadecimal 000a, referente à variável “var1” com valor 10, em decimal;
- 3, o valor hexadecimal 0041, referente à variável “var2” com valor 65, que consiste no código ASCII do caractere ‘A’, em decimal;
- 4 a 6, os valores hexadecimais 000a, 000b e 000c, referentes ao array/vetor “array1” com valores 10, 11 e 12, em decimal;

Figura 8.9. Memória do Programa Apresentado na Figura 8.8.



Fonte: Próprio autor (2021).

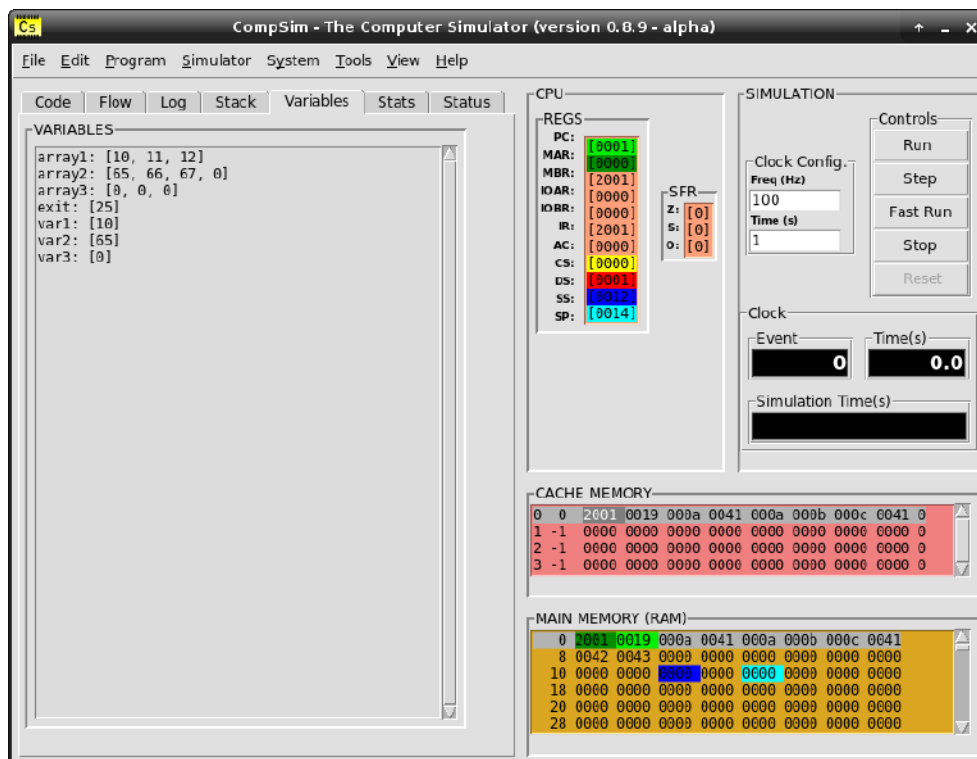
- 8 a 10, os valores hexadecimais 0041, 0042, 0043 e 0000, referentes ao array/vetor “array2”, com valores decimais 65, 66, 67 e 0, em que os três primeiros são os códigos ASCII dos caracteres ‘A’, ‘B’ e ‘C’;
- 11, o valor hexadecimal 0000, referente à variável “var3” com valor 0, em decimal, que é o valor padrão de uma variável não inicializada; e
- 12 a 14, os valores hexadecimais 0000, 0000 e 0000, referentes ao array/vetor “array3”, com valores 0, 0 e 0, em decimal, que são os valores padrão das posições de um array/vetor não inicializado.

Por fim, na Figura 8.9, os endereços de 15 a 17 foram reservados como livres, e os endereços 18 e 19 consistem do segmento de pilha, configurado pela instrução contida na Linha 18, do código-fonte da Figura 8.8.

A Figura 8.10 mostra a interface gráfica do CompSim, onde, ao iniciar uma simulação, é possível ver, na aba “Variables”, o conteúdo das variáveis e arrays/vetores do programa apresentado na Figura 8.8.

Ainda na Figura 8.10, é possível visualizar, no componente gráfico “CPU”, os endereços de início dos segmentos do programa, através dos registradores CS, DS e SS, com os respectivos endereços “0000”, “0001” e “0012” (ou 0, 2, e 18, em base decimal), e SP com endereço “0014” (20 em base decimal), que é uma posição posterior aos espaços de memória alocados para a pilha.

Figura 8.10. Visualização de alocação de memória no CompSim.



Fonte: Próprio autor (2021).

Após compreender os conceitos e dominar os mecanismos para alocação de dados em memória no simulador CompSim, o passo seguinte consiste em aprender a acessá-los, como será visto na próxima seção.

8.6 Acesso aos Dados Alocados em Memória

O processador Cariri possui instruções para realizar operações de leitura e de escrita de dados em memória. Nessas instruções, o acesso aos dados alocados em memória pode ser realizado a partir de um nome de uma variável, um nome de um array/vetor ou por um endereço de memória.

As instruções para leitura e gravação de um dado em memória são “LDA” e “STA”, respectivamente. Os quadros a seguir trazem resumos com as descrições das instruções LDA e STA, em qual seção do código podem ser utilizadas, suas sintaxes de uso e exemplos de como utilizá-las.

Instrução: LDA (Loa D from A ddress)	
Descrição: Leitura de um dado a partir de uma determinada posição de memória. Após a leitura, o dado será armazenado no registrador acumulador (AC).	
Seção: .code	
Sintaxe: [<rotulo>:] LDA @<endereço> [<rotulo>:] LDA <variável>	
Exemplos: LDA @100 ;realiza a leitura de um dado armazenado no endereço 100 da memória. LDA var1 ;realiza a leitura de um dado armazenado na variavel var1. Le_End: LDA @100 ;instrucao com rotulo para leitura de dado no endereço 100 da memória. Le_Var: LDA var1 ;instrucao, com rotulo em linha separada, para leitura de dado na variavel var1.	

Instrução: STA (Save T o A ddress)	
Descrição: Gravação de um dado em uma determinada posição de memória. Será gravado em memória o dado armazenado no registrador acumulador (AC).	
Seção: .code	
Sintaxe: [<rotulo>:] STA @<endereço> [<rotulo>:] STA <variável>	
Exemplos: STA @100 ;realiza a gravação de um dado em AC no endereço 100 da memória. STA var1 ;realiza a gravação de um dado em AC na variavel var1. Salva_End: STA @100 ;instrucao com rotulo para gravação um dado em AC no endereço 100 da memória. Salva_Var: STA var1 ;instrucao, com rotulo em linha separada, para gravação de dado em AC na variavel var1.	

Nos quadros anteriores, é possível observar que:

- As instruções podem ser precedidas por rótulos. Um rótulo é um identificador de instrução. Um rótulo e a respectiva instrução podem estar localizados na mesma linha ou em linhas distintas, desde que o rótulo preceda a instrução. Os rótulos são importantes para permitir alterações no fluxo de execução do programa, como será visto no [Capítulo 10 - Controle de Fluxo de Execução do Programa](#), e para a modularização do programa, através da criação de procedimentos e funções, como será visto no [Capítulo 12 - Modularização de Programas](#); e
- Caso o operando da instrução seja um endereço de memória, o valor do endereço deve ser precedido pelo símbolo “@” (que faz uma analogia a *address* ou endereço).

Capítulo 8 - Manipulação de Dados em Memória

Para ilustrar o uso das instruções LDA e STA, será estudado um programa em *Assembly* que realiza a seguinte sequência de instruções de um programa escrito em linguagem C:

Figura 8.11. Exemplo de código C para acesso à dados em memória.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2;</code>
3	<code>var2 = var1;</code>

Fonte: Próprio autor (2021).

No programa da Figura 8.11, a Linha 1 define uma variável “var1” com valor inicial 10. Na Linha 2, tem-se a declaração de uma nova variável, chamada “var2”. Por fim, na Linha 3, o valor da variável “var1” é atribuído à variável “var2”. É importante ressaltar que, na última instrução, são realizados dois acessos à memória: o primeiro consiste na leitura do valor da variável “var1”; e o segundo consiste na gravação do valor lido de “var1” na variável “var2”.

Um exemplo de programa em *Assembly* do processador Cariri, que implementa o mesmo comportamento do código em linguagem C no quadro anterior, pode ser visto na Figura 8.12.

Figura 8.12. Exemplo de código Assembly para acesso à dados em memória.

Linha	Código
1	<code>.code</code>
2	<code> LDA var1 ;as duas instrucoes implementam var2 = var1.</code>
3	<code> STA var2</code>
4	
5	<code>end:</code>
6	<code> INT exit</code>
7	
8	<code>.data</code>
9	<code> ;syscall exit</code>
10	<code> exit: DD 25</code>
11	<code> var1: DD 10 ;int var1 = 10.</code>
12	
13	<code>.bss</code>
14	<code> var2: RESD 1 ;int var2.</code>
15	
16	<code>.stack 1</code>

Fonte: Próprio autor (2021).

No código *Assembly* da Figura 8.12, pode-se ver, na Linha 11, a definição da variável “var1”, sendo inicializada com o número inteiro 10, e, na Linha 14, a declaração da variável “var2”. Já na seção “.code”, as Linhas 2 e 3 incluem as instruções LDA e STA que

implementam, respectivamente, a operação de leitura do valor 10, contido na variável “var1” em memória, para o registrador AC, e a gravação do dado lido na variável “var2” em memória.

Ainda no código da Figura 8.12, as Linhas 5 e 6 apresentam o rótulo “end:” e a respectiva instrução “INT exit”. A instrução INT, que recebe, como operando, uma variável ou um endereço de memória, possui múltiplas funções, dependendo do valor lido da memória, como será visto ao longo deste livro. Na Linha 6, a instrução INT realiza a leitura do valor da variável “exit”, a qual contém o valor inteiro 25 (a variável “exit” foi definida na Linha 10 do código na Figura 8.12). Uma instrução INT com parâmetro inteiro no valor de 25, quando decodificada pelo processador, será compreendida como a instrução de controle de parada de simulação (a instrução *HALT*). Neste caso, na fase de execução da instrução, o processador encerrará seu ciclo de instrução. O quadro a seguir sumariza as informações do uso da instrução INT para encerramento do ciclo de instrução do processador.

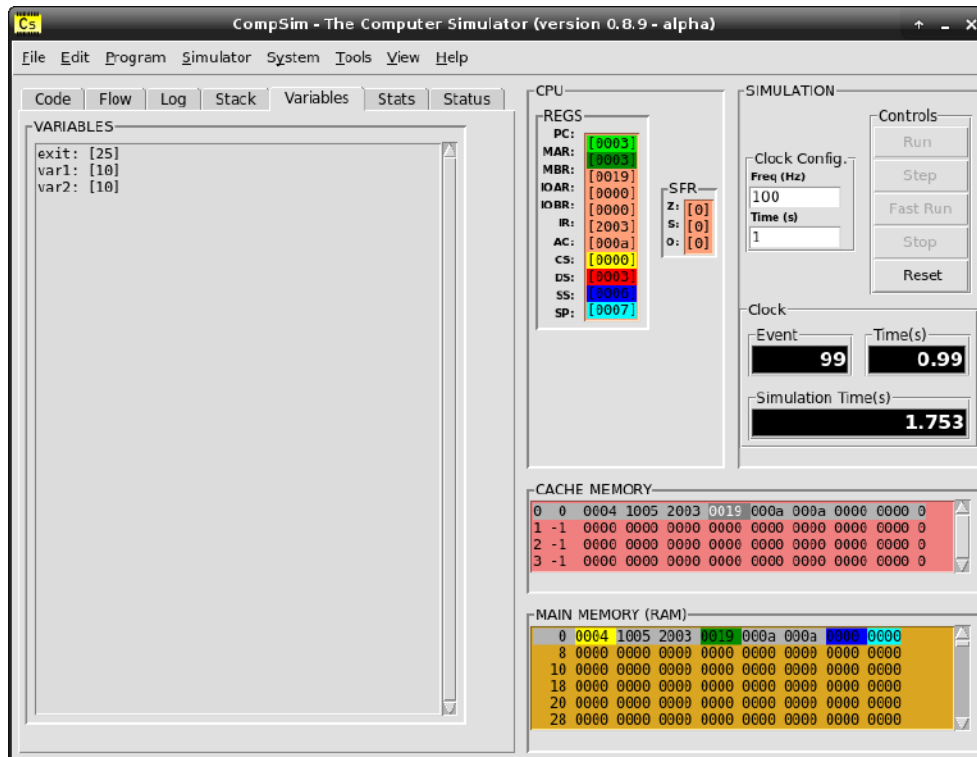
Instrução: INT 25 (HALT)
Descrição: Encerra o ciclo de instrução do processador.
Seção: .code
Sintaxe: [<rotulo>:] INT @<endereço> [<rotulo>:] INT <variável>
Exemplo: <pre>.code end: INT exit .data ;syscall exit exit: DD 25</pre>

A Figura 8.13 apresenta a interface gráfica do CompSim, onde, após a execução do programa da Figura 8.12, é possível visualizar que:

- A aba “Variables” mostra que o valor 10, contido na variável “var1”, foi copiado para “var2”;
- No componente gráfico “CPU”, o registrador AC contém o valor “000a” (valor 10 em base hexadecimal), que consiste do valor lido de “var1” e escrito em “var2”; e
- No componente gráfico “MAIN MEMORY (RAM)”, as localizações de memória com endereços 4 e 5, referentes às variáveis “var1” e “var2”, respectivamente, possuem

valores “000a”, o que mostra novamente, porém agora do ponto de vista da memória, que houve a cópia de dados de uma localização para outra.

Figura 8.13. Visualização de operações de leitura e escrita em memória no CompSim.



Fonte: Próprio autor (2021).

8.7 Resumo

Este capítulo iniciou com uma proposta de abstração da organização de uma memória RAM, de forma a simplificar o aprendizado dos conceitos de arquitetura de computadores com apoio do simulador CompSim. A abstração proposta transforma a matriz de dados da memória RAM em uma estrutura de dados linear, sendo, portanto, mais simples de visualizar os dados armazenados.

Na sequência, discutiu-se sobre como um programa de computador é estruturado, visando separar instruções, dados, pilha do programa e demais elementos, bem como ele pode ser alocado e segmentado em memória. Viu-se ainda os recursos do processador Cariri que dão suporte à segmentação dos programas em memória.

Por fim, estudou-se como alocar dados em memória, através da definição e declaração de variáveis e estruturas de dados em programas codificados em linguagem de *Assembly*, e

Capítulo 8 - Manipulação de Dados em Memória

como realizar acessos aos dados alocados, utilizando as instruções LDA e STA do processador Cariri.

CAPÍTULO 9 – PROCESSAMENTO DE DADOS

No computador, o processamento de dados é realizado no processador ou, mais especificamente, na Unidade Lógica e Aritmética (ULA) do processador. A ULA é um circuito do tipo combinacional, que dependendo da sua entrada, ele logo apresenta uma saída. A ULA deve dar suporte aos tipos de processamento de dados demandados pelas instruções presentes na ISA (*Instruction Set Architecture*) do processador. Dependendo da instrução de processamento de dados, a Unidade de Controle decodifica a instrução e configura a ULA para realizar tal operação.

Nos processadores, é possível encontrar diferentes tipos de instruções para processamento de dados, sendo as mais comuns as operações aritméticas, tais como adição, subtração, multiplicação, divisão, bem como as operações lógicas, tais como os operadores lógicos AND, OR, NOT e XOR e operações de deslocamento de bits.

Neste capítulo apresentam-se as operações aritméticas e lógicas do CompSim, com diversos exemplos práticos ilustrativos.

9.1 Operações Aritméticas

O Processador Cariri do CompSim suporta apenas duas operações aritméticas: adição (“ADD”) e subtração (“SUB”). Considera-se que, com a combinação dessas duas operações com as operações lógicas do CompSim, que serão vistas mais adiante neste capítulo, é possível compor operações mais complexas.

Os quadros a seguir trazem resumos com as descrições das instruções ADD e SUB, em qual seção do código podem ser utilizadas, suas sintaxes de uso e exemplos de como utilizá-las.

Instrução: ADD (ADD ition)	
Descrição: Soma o valor contido no registrador acumulador (AC) com o valor contido em um endereço de memória passado como operando da instrução. O resultado da adição é armazenado no registrador AC.	
Seção: .code	
Sintaxe: [<rotulo>:] ADD @<endereço> [<rotulo>:] ADD <variável>	
Flags afetadas do SFR (Status Flags Register): Z (Zero), S (Signal) e O (Overflow)	
Exemplos: ADD @100 ; soma o valor contido no endereço 100 ao valor em AC. ADD var1 ; soma o valor contido em var1 ao valor em AC. Soma_100: ADD @100 ; instrução com rotulo para somar o valor contido no endereço 100 com AC. Soma_Var1: ADD var1 ; instrução, com rotulo em linha separada, para soma do valor em var1 com AC.	

Instrução: SUB (SUB traction)	
Descrição: Subtrai do valor contido no registrador acumulador (AC) o valor contido em um endereço de memória passado como operando da instrução. O resultado da subtração é armazenado no registrador AC.	
Seção: .code	
Sintaxe: [<rotulo>:] SUB @<endereço> [<rotulo>:] SUB <variável>	
Flags afetadas do SFR (Status Flags Register): Z (Zero), S (Signal) e O (Overflow)	
Exemplos: SUB @100 ; subtrai o valor contido no endereço 100 do valor em AC. SUB var1 ; subtrai o valor contido em var1 do valor em AC. Subtrai_100: SUB @100 ; instrução com rotulo para subtrair o valor contido no endereço 100 de AC. Subtrai_Var1: SUB var1 ; instrução, com rotulo em linha separada, para subtrair o valor em var1 de AC.	

Nos quadros anteriores, viu-se que as instruções ADD e SUB possuem sintaxes semelhantes às instruções LDA e STA, vistas no capítulo anterior. Elas podem ter rótulos e recebem como operando um endereço de memória ou o nome de uma variável.

Ao utilizar as instruções ADD e SUB, dependendo do resultado da operação, podem ocorrer situações inesperadas ou que necessitam apresentar um *feedback*. Nesses casos, entra em ação o Registrador de Flags de Status (*Status Flag Register* - SFR), que possui apenas 3-bits, sendo eles:

1. Z (Zero): ao ser configurado em 1, informa que uma determinada operação resultou em valor zero no registrador acumulador (AC);

- 2. S (*Signal*): ao ser configurado em 1, informa que uma determinada operação resultou em um número negativo em AC; e
- 3. O (*Overflow*): ao ser configurado em 1, informa que uma determinada operação resultou em um número maior do que 32767 ou menor do que -32768, que são, respectivamente, os valores máximo e mínimo possíveis para inteiros de 16-bits com sinal. Com isso, o número será truncado para que seja possível armazená-lo em AC.

O código na Figura 9.1 apresenta um exemplo básico de uso da instrução ADD para incrementar o valor de uma variável chamada “var1”.

Figura 9.1. Exemplo de código Assembly para operação de adição.

Linha	Código
1	<code>.code</code>
2	<code> LDA var1 ;este bloco realiza: var1 = var1 + 1.</code>
3	<code> ADD one</code>
4	<code> STA var1</code>
5	
6	<code>end:</code>
7	<code> INT exit</code>
8	
9	<code>.data</code>
10	<code> ;syscall exit</code>
11	<code> exit: DD 25</code>
12	<code> var1: DD 10 ;int var1 = 10.</code>
13	<code> one: DD 1 ;int one = 1.</code>
14	
15	<code>.stack 1</code>

Fonte: Próprio autor (2021).

No código da Figura 9.1, na Linha 2, o valor 10, contido na variável “var1” é carregado para o acumulador (AC). Na Linha 3, é lido o valor da variável “one” da memória, somando ao valor contido em AC e o resultado é gravado em AC. Por fim, na Linha 4, o valor em AC é gravado na variável “var1”, em memória.

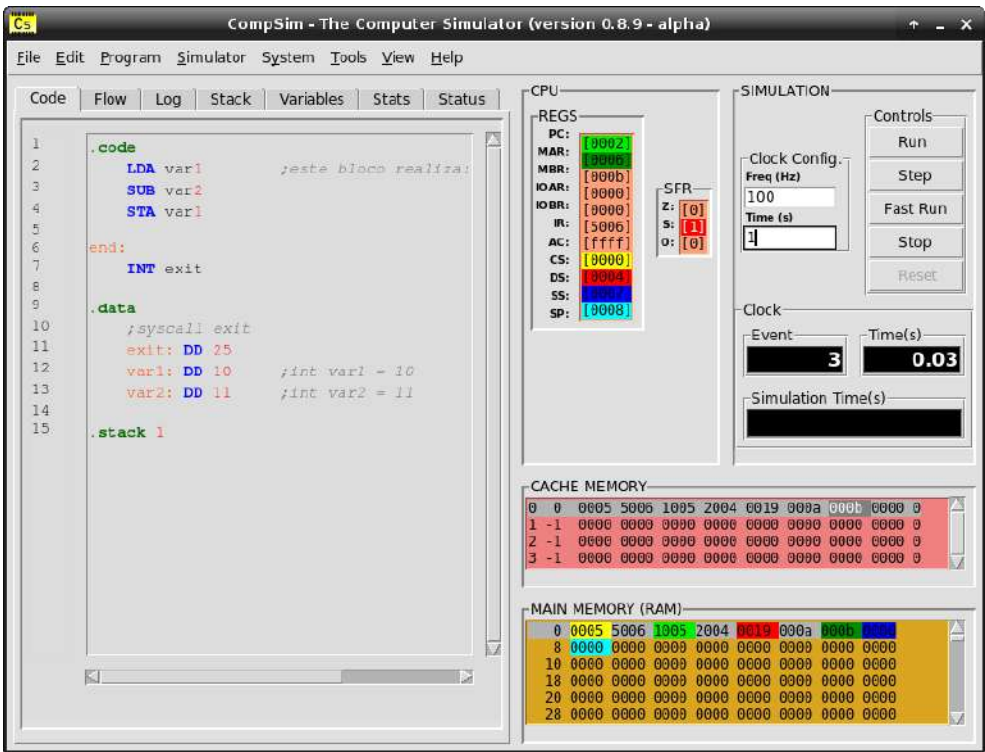
A Figura 9.2 ilustra o uso da instrução de subtração. Na Figura 9.2, a Linha 2 carrega em AC o valor 10, contido na variável “var1”. Na Linha 3, o valor 11, contido na variável “var2”, é trazido da memória e subtraído do valor contido em AC, sendo o resultado armazenado em AC. Percebe-se que, no exemplo, está sendo subtraído o valor 11 de 10, o que resultará no número inteiro negativo -1. Ao fim da operação de subtração, será disparada a flag “S” (*Signal*) do registrador SFR, acusando que a operação resultou em um número negativo. No CompSim, os bits de SFR, ao serem configurados em 1, ficam destacados em vermelho, como pode ser visto o bit S, no componente gráfico “CPU”, na Figura 9.3.

Figura 9.2. Exemplo de código Assembly para operação de subtração.

Linha	Código
1	.code
2	LDA var1 ;este bloco realiza: var1 - var2.
3	SUB var2
4	
5	end:
6	INT exit
7	
8	.data
9	;syscall exit
10	exit: DD 25
11	var1: DD 10 ;int var1 = 10.
12	var2: DD 11 ;int var2 = 11.
13	
14	.stack 1

Fonte: Próprio autor (2021).

Figura 9.3. Visualização de operação de subtração com resultado negativo.



Fonte: Próprio autor (2021).

A Figura 9.4 ilustra novamente o uso da instrução de adição, porém, desta vez, será provocado um *overflow* negativo. Na Figura 9.4, a Linha 11 apresenta a definição da variável “var1”, que recebe o valor inteiro negativo -32768, e, na Linha 12, é definida a variável “var2” com valor “0xffff”, que na representação Complemento a 2, equivale ao valor inteiro negativo -1 (vide [Apêndice B - Introdução à Aritmética Computacional](#) para mais informações sobre essa e outras conversões). A Linha 2 carrega o valor -32728, contido na

variável “var1” em AC. Já, na Linha 2, o valor -1, contido na variável “var2”, é trazido da memória e somado do valor contido em AC, sendo o resultado armazenado em AC.

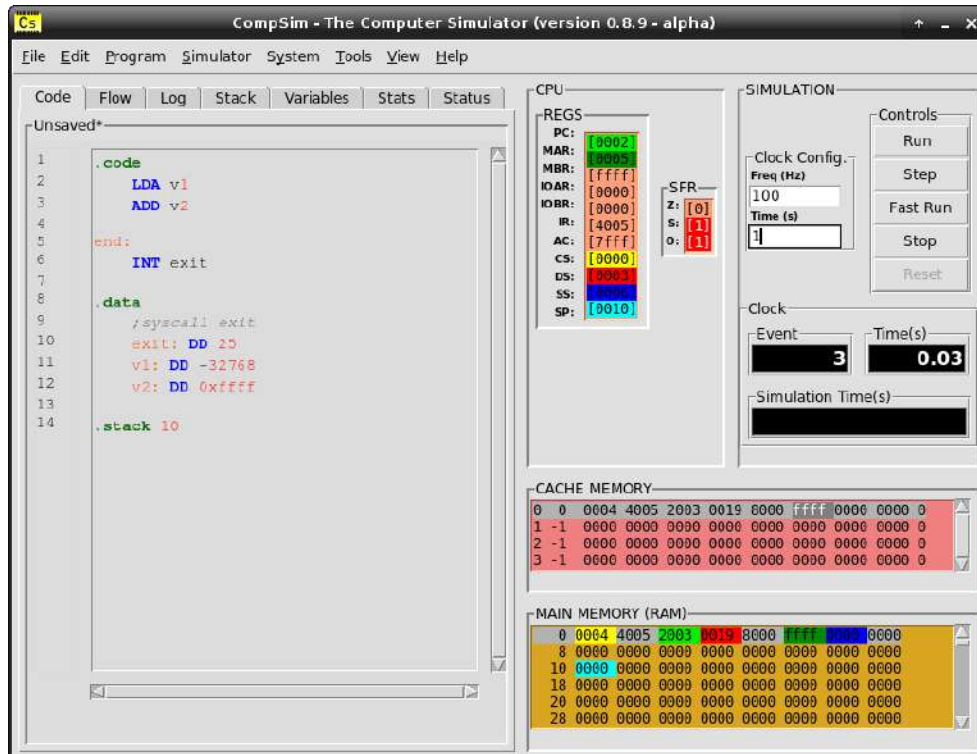
Figura 9.4. Exemplo de código Assembly para operação de adição com overflow negativo.

Linha	Código
1	<code>.code</code>
2	<code> LDA var1 ;este bloco realiza: var1 + var2.</code>
3	<code> ADD var2</code>
4	
5	<code>end:</code>
6	<code> INT exit</code>
7	
8	<code>.data</code>
9	<code> ;syscall exit</code>
10	<code> exit: DD 25</code>
11	<code> var1: DD -32768 ;int var1 = -32768.</code>
12	<code> var2: DD 0xffff ;int var2 = -1.</code>
13	
14	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Percebe-se que, no exemplo, será somado o valor -32768, que é o valor mínimo de um inteiro de 16-bits com sinal, a -1, gerando assim um *overflow* negativo. Ao fim da operação de adição, serão disparadas as flags S (*Signal*), pois a soma de dois números negativos resulta em um número negativo, e O (*Overflow*), pois ultrapassa o limite de representação de número inteiro de 16-bits, como pode ser visto no componente gráfico “CPU”, na Figura 9.5. Na figura, pode-se observar ainda que AC contém o valor “7fff”, que é um valor inconsistente, pois foi gerado pelo truncamento do resultado da operação de adição com *overflow* negativo.

Figura 9.5. Visualização de operação de adição com resultado negativo e overflow.



Fonte: Próprio autor (2021).

9.2 Operações Lógicas

Assim como ocorre com as operações aritméticas, o CompSim suporta apenas duas instruções lógicas: negação da conjunção (“NAND”) e deslocamento de bits (“SHIFT”).

As subseções a seguir detalham cada uma dessas operações lógicas e trazem exemplos práticos de sua aplicação.

9.2.1 Instrução NAND

A operação lógica NAND, ou NOT AND, consiste em realizar uma operação de conjunção (AND) entre duas variáveis lógicas e, com o resultado, aplicar a operação lógica de negação (NOT).

O quadro a seguir traz um resumo com uma descrição da instrução NAND, em qual seção do código pode ser utilizada, sua sintaxe de uso e exemplos de como utilizá-la.

Instrução: NAND (Not AND)
Descrição: Realiza a operação lógica de negação da conjunção entre os bits do valor contido no registrador acumulador (AC) e do valor contido em um endereço de memória passado como operando da instrução. O resultado da operação lógica NAND é armazenado no registrador AC.
Seção: .code
Sintaxe: [<rotulo>:] NAND @<endereço> [<rotulo>:] NAND <variável>
Flags afetadas do SFR (<i>Status Flags Register</i>): Z (Zero) e S (<i>Signal</i>)
Exemplos: NAND @100 ;realiza NAND entre os bits contidos no endereço 100 e em AC. NAND var1 ;realiza NAND entre os bits contidos em var1 e em AC. NAND_100: NAND @100 ;instrucao com rotulo para NAND entre os bits no endereço 100 e em AC. NAND_Var1: NAND var1 ;instrucao, com rotulo em linha separada, para NAND entre os bits em var1 e em AC.

A Figura 9.6 traz um exemplo básico de uso da instrução NAND. Nas Linhas 11 e 12, são definidas as variáveis “var1”, cujos 16-bits foram configurados em 1, e “var2”, com os 16-bits configurados em 0. Na Linha 2, os bits de “var1” são carregados em AC e, na Linha 3, realiza-se a operação lógica NAND entre os bits em AC e os contidos na variável “var2”, lidos da memória, sendo o resultado armazenado em AC.

Figura 9.6. Exemplo de código Assembly para ilustrar o uso da instrução NAND.

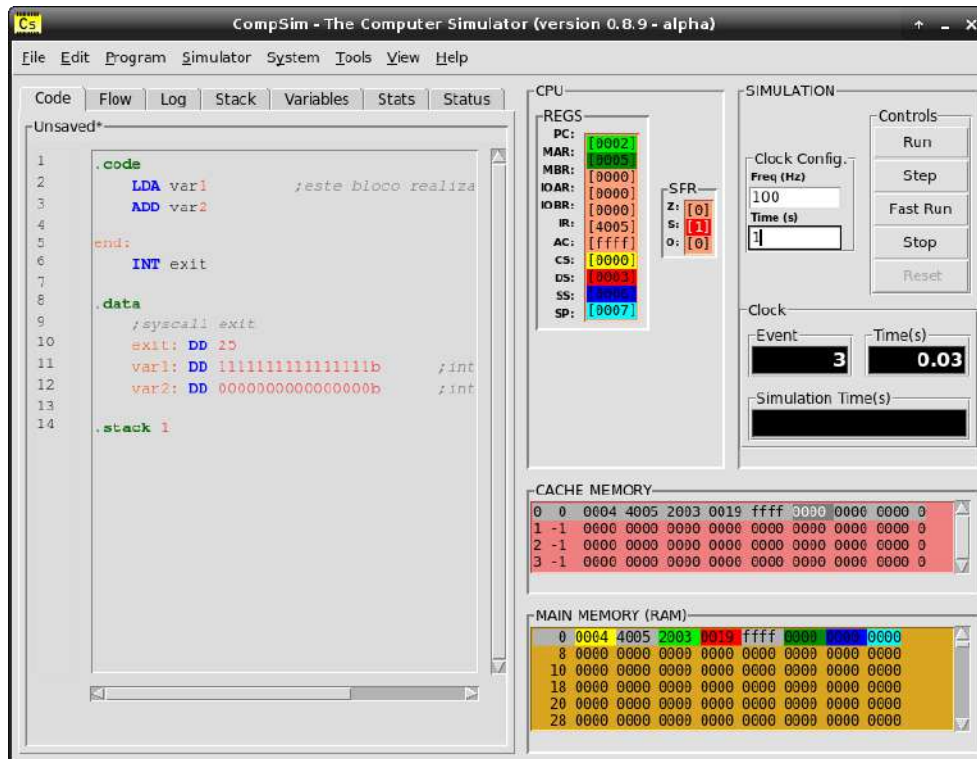
Linha	Código
1	.code
2	LDA var1 ;este bloco realiza: NAND(var1, var2).
3	NAND var2
4	
5	end:
6	INT exit
7	
8	.data
9	;syscall exit
10	exit: DD 25
11	var1: DD 111111111111111b ;int var1 = 0xffff.
12	var2: DD 000000000000000b ;int var2 = 0.
13	
14	.stack 1

Fonte: Próprio autor (2021).

O resultado da operação NAND do código anterior pode ser visto na Figura 9.7. Na figura, observa-se que, após a execução da instrução NAND, o registrador AC apresenta o valor hexadecimal “ffff”, o que significa que todos os bits do registrador foram setados em 1, pois NAND(0,1) é igual a 1. Observa-se ainda que a sequência de bits resultante equivale ao

número -1 (notação Complemento a 2), fazendo com que a flag S (*Signal*) do registrador SFR seja configurada em 1.

Figura 9.7. Visualização de operação NAND.



Fonte: Próprio autor (2021).

A operação lógica NAND é bastante versátil, pois com ela é possível compor outras operações e funções lógicas, utilizando algumas propriedades da álgebra booleana. Por conta disso, a operação lógica NAND é conhecida como “Porta Lógica Universal” (ver [Apêndice C - Introdução à Lógica Booleana e Circuitos Lógicos](#)).

As subseções a seguir ilustram como utilizar a instrução NAND do processador Cariri para compor as operações lógicas de negação (NOT), conjunção (AND) e disjunção (OR).

9.2.1.1 Operação Lógica de Negação

A composição da operação lógica de negação ou inversão (NOT), utilizando a negação da conjunção (NAND), pode ser vista na equação a seguir:

$\neg A = \neg (A.A)$	(1)
-----------------------	-----

Onde:

- “A” é uma variável lógica;
- “¬” representa o operador lógico de negação; e
- “.” representa o operador lógico de conjunção.

Observa-se, na equação booleana acima, que a negação de uma variável lógica “A” pode ser dada pela negação da conjunção (NAND) da variável “A” com ela mesma.

O código na figura a seguir ilustra como a instrução NAND pode ser utilizada para compor a operação lógica de negação (NOT).

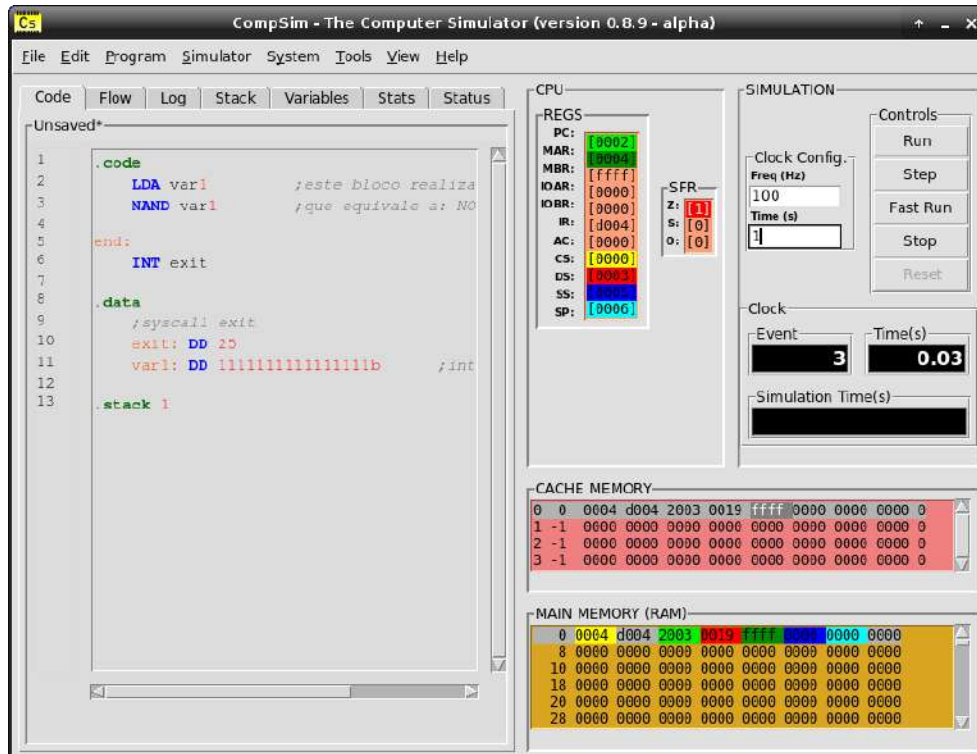
Figura 9.8. Exemplo de código Assembly para ilustrar o uso da instrução NAND para compor a operação lógica NOT.

Linha	Código
1	<code>.code</code>
2	<code> LDA var1 ;este bloco realiza: NAND(var1, var1).</code>
3	<code> NAND var1 ;que equivale a: NOT(var1).</code>
4	
5	<code>end:</code>
6	<code> INT exit</code>
7	
8	<code>.data</code>
9	<code> ;syscall exit</code>
10	<code>exit: DD 25</code>
11	<code>var1: DD 111111111111111b ;int var1 = 0xffff.</code>
12	
13	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Na Figura 9.8, na Linha 11 é definida a variável “var1” com todos os seus bits configurados em 1. Na Linha 2, os bits da variável “var1” são carregados em AC e, na Linha 3, realiza-se a operação lógica NAND entre os bits em AC e os da própria variável “var1”, em memória, sendo o resultado armazenado em AC. Observa-se na Figura 9.9, que, após a execução da instrução NAND, AC apresentou resultado “0000”, o que mostra a negação dos bits anteriormente armazenados em AC. É possível ver também que, devido ao resultado da instrução NAND, a flag Z (Zero) do registrador SFR foi configurada em 1.

Figura 9.9. Visualização de operação lógica NOT implementada com a instrução NAND.



Fonte: Próprio autor (2021).

9.2.1.2 Operação Lógica de Conjunção

A implementação da operação lógica de conjunção (AND) utilizando a negação da conjunção (NAND) pode ser vista na equação a seguir:

$A.B = \neg(\neg(A.B))$	(2)
-------------------------	-----

Onde:

- “A e “B” são variáveis lógicas;
- “¬” representa o operador lógico de negação; e
- “.” representa o operador lógico de conjunção.

Observa-se que, na equação booleana acima, a conjunção de uma variável lógica “A” com uma variável lógica “B” pode ser dada pela negação de uma operação NAND entre as variáveis “A” e “B”.

Tomando a equação (1) como base, podemos expor a negação mais externa da equação (2) da seguinte maneira:

$\neg (\neg (A.B)) = \neg ((\neg (A.B)) . (\neg (A.B)))$	(3)
--	-----

Conclui-se então que uma operação lógica de conjunção (AND) pode ser composta: 1) pela negação da conjunção (NAND) das variáveis lógicas “A” e “B”; e 2) com o resultado, aplicar uma nova negação da conjunção (NAND) do resultado anterior com ele mesmo.

O código na Figura 9.10 ilustra como a instrução NAND pode ser utilizada para compor a operação lógica de conjunção AND. Na Linha 14, é definida a variável “var1” com todos os seus 16-bits configurados em 1. Na Linha 15, define-se a variável “var2”, com os 8-bits mais significativos configurados em 1 e os 8-bits menos significativos em 0. Nas Linhas 2 à 4, realiza-se a operação lógica NAND entre os bits das variáveis “var1” e “var2”, e o resultado é armazenado na variável “tmp”. O próximo passo será realizar uma nova negação dos bits em “tmp”, aplicando a operação lógica NAND dos bits de “tmp” com eles mesmos. Assim, a Linha 6 realiza a operação lógica NAND entre os bits em AC e na variável “tmp” (neste caso, os bits em AC estão idênticos aos de “tmp”).

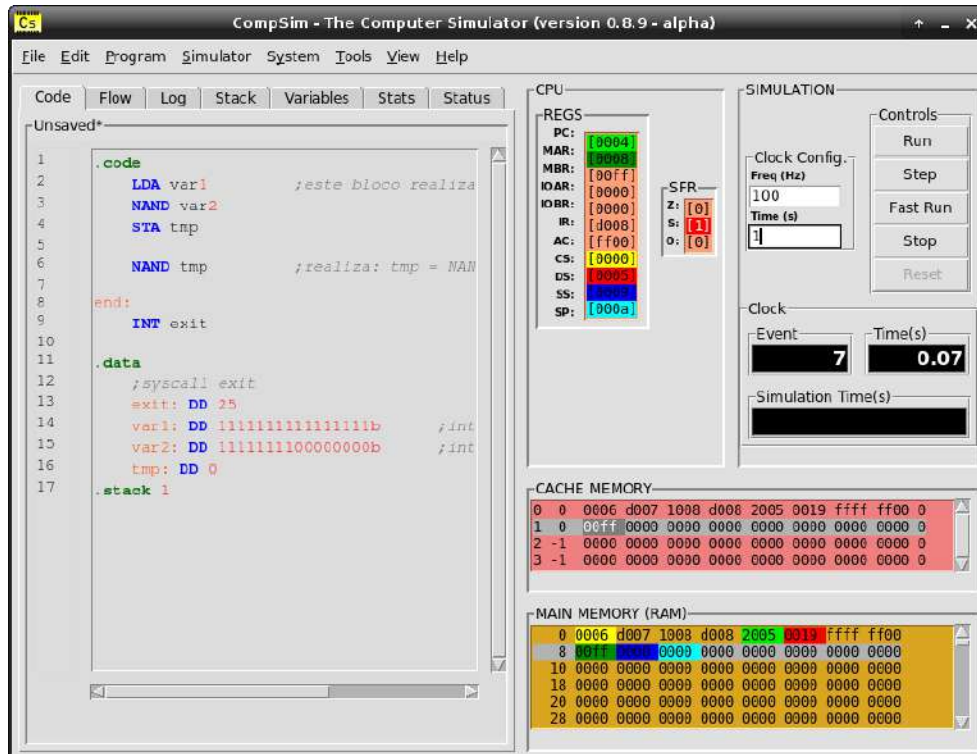
Figura 9.10. Exemplo de código Assembly para ilustrar o uso da instrução NAND para compor a operação lógica AND.

Linha	Código
1	<code>.code</code>
2	<code> LDA var1 ;este bloco realiza: tmp = NAND(var1, var2).</code>
3	<code> NAND var2</code>
4	<code> STA tmp</code>
5	
6	<code> NAND tmp ;realiza: tmp = NAND(AC, tmp).</code>
7	
8	<code>end:</code>
9	<code> INT exit</code>
10	
11	<code>.data</code>
12	<code> ;syscall exit</code>
13	<code> exit: DD 25</code>
14	<code> var1: DD 11111111111111b ;int var1 = 0xffff.</code>
15	<code> var2: DD 1111111100000000b ;int var1 = 0xff00.</code>
16	<code> tmp: DD 0</code>
17	<code>.stack 1</code>

Fonte: Próprio autor (2021).

A Figura 9.11 ilustra o resultado da execução do código anterior. Na figura, pode-se verificar que o registrador AC apresenta o valor hexadecimal “ff00” que é o resultado da operação lógica AND entre os bits contidos nas variáveis “var1” e “var2”. Observa-se ainda que o valor resultante equivale à -256, em representação Complemento a 2, e, por conta disso, a flag S (*Signal*) do registrador SFR foi configurada em 1.

Figura 9.11. Visualização de operação lógica AND implementada com a instrução NAND.



Fonte: Próprio autor (2021).

9.2.1.3 Operação Lógica de Disjunção

A implementação da operação lógica de disjunção (OR) utilizando a negação da conjunção (NAND) pode ser vista na equação a seguir:

$A+B = \neg (\neg (A) \cdot \neg (B))$	(4)
---	-----

Onde:

- “A” e “B” são variáveis lógicas;
- “+” representa o operador lógico de disjunção;
- “¬” representa o operador lógico de negação; e
- “.” representa o operador lógico de conjunção.

Tomando a equação (1) como base, podemos expor a negação das variáveis lógicas “A” e “B” da equação (4) da seguinte maneira:

$\neg (\neg (A) \cdot \neg (B)) = \neg (\neg (A.A) \cdot \neg (B.B))$	(5)
---	-----

Conclui-se então que uma operação lógica de disjunção (OR) pode ser composta: 1) pela negação da conjunção (NAND) da variável lógica “A” com ela mesma; 2) pela negação da conjunção (NAND) da variável lógica “B” com ela mesma; e 3) com os resultados, aplicar uma nova negação da conjunção (NAND).

O código na Figura 9.12 ilustra como a instrução NAND pode ser utilizada para compor a operação lógica de disjunção OR. Na Linha 17, é definida a variável “var1” com os 8-bits menos significativos configurados em 1 e os 8-bits mais significativos configurados em 0. Na Linha 18, define-se a variável “var2”, com os 8-bits mais significativos configurados em 1 e os 8-bits menos significativos configurados em 0. Nas Linhas 2 à 4, utiliza-se a instrução NAND para realizar a operação lógica de negação (NOT) da variável “var1” e o resultado é armazenado na variável “tmp”. Nas Linhas 6 e 7, realiza-se também a negação dos bits da variável “var2” e o resultado fica armazenado no registrador AC. Por fim, na Linha 9, realiza-se a operação lógica NAND entre os bits em AC e os da variável “tmp”, em memória.

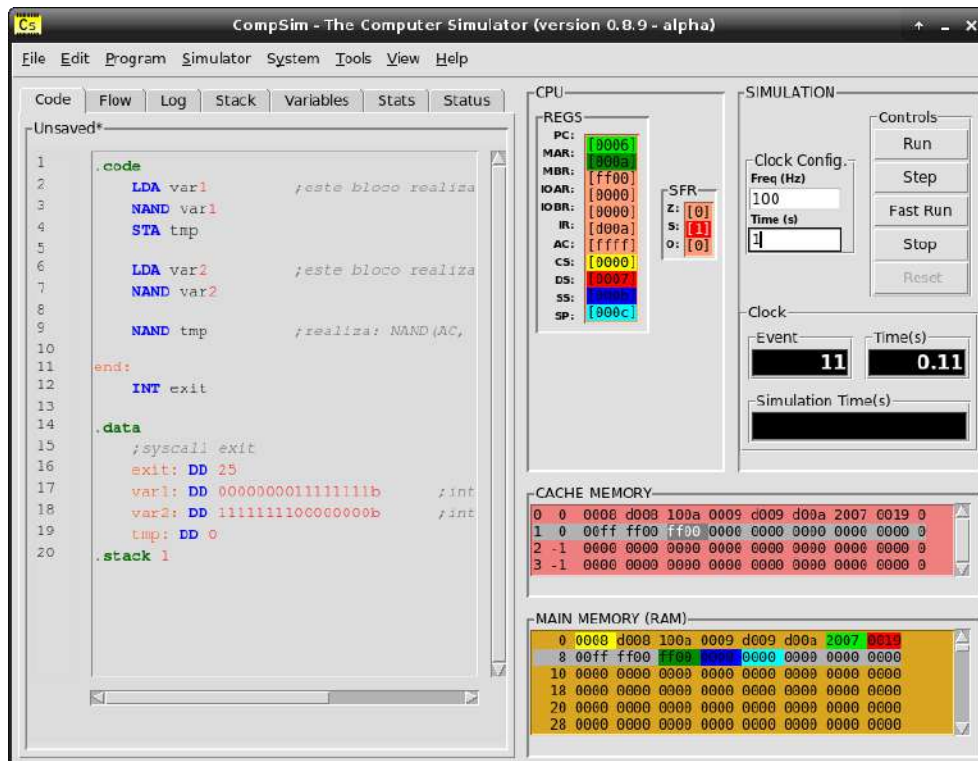
Figura 9.12. Exemplo de código Assembly para ilustrar o uso da instrução NAND para compor a operação lógica OR.

Linha	Código
1	<code>.code</code>
2	<code>LD A, var1 ;este bloco realiza: tmp = NAND(var1, var1).</code>
3	<code>NAND A, A</code>
4	<code>ST tmp</code>
5	
6	<code>LD A, var2 ;este bloco realiza: AC = NAND(var2, var2).</code>
7	<code>NAND A, A</code>
8	
9	<code>NAND A, tmp ;realiza: NAND(AC, tmp).</code>
10	
11	<code>end:</code>
12	<code>INT exit</code>
13	
14	<code>.data</code>
15	<code>;syscall exit</code>
16	<code>exit: DD 25</code>
17	<code>var1: DD 000000001111111b ;int var1 = 0xffff.</code>
18	<code>var2: DD 111111110000000b ;int var1 = 0xff00.</code>
19	<code>tmp: DD 0</code>
20	<code>.stack 1</code>

Fonte: Próprio autor (2021).

A Figura 9.13 ilustra o resultado da execução do código anterior. Na figura, pode-se verificar que o registrador AC apresenta o valor hexadecimal “ffff”, que é o resultado da operação lógica OR entre os bits contidos nas variáveis “var1” e “var2”. Observa-se ainda que o valor resultante equivale a -1, em representação Complemento a 2, e, por conta disso, a flag S (Signal) do registrador SFR foi configurada em 1.

Figura 9.13. Visualização de operação lógica OR implementada com a instrução NAND.



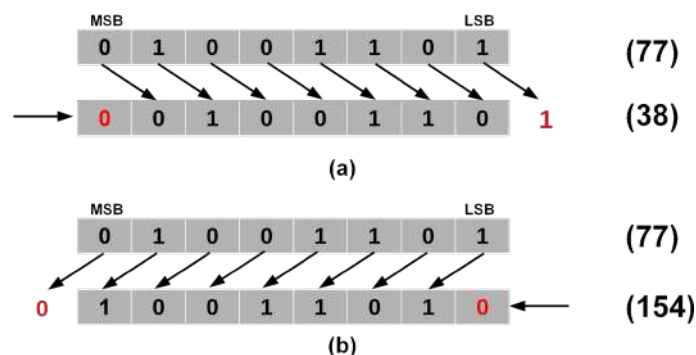
Fonte: Próprio autor (2021).

9.2.2 Instrução SHIFT

A operação lógica de deslocamento de bits (SHIFT) consiste em mover bits de uma determinada palavra nos sentidos à direita ou à esquerda. Basicamente, há dois tipos de deslocamento de bits: o lógico e o aritmético.

A Figura 9.14 apresenta exemplos de deslocamento lógico de bits. Na Figura 9.14 (a) o deslocamento está sendo realizado no sentido à direita. Já na Figura 9.14 (b), o sentido está realizado no sentido à esquerda.

Figura 9.14. Deslocamento Lógico de bits.

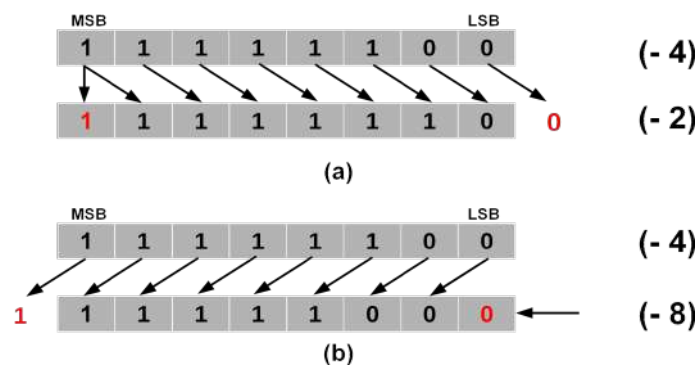


Fonte: Próprio autor (2021).

Analisando a Figura 9.14 (a), percebe-se que ao realizar um deslocamento de bits à direita, o valor 77, após o deslocamento, sofre uma divisão inteira por 2, resultando assim no valor 38. Já no deslocamento de bits à esquerda (b), o valor 77, após o deslocamento, sofre uma multiplicação inteira por 2, resultando assim no valor 154.

A Figura 9.15 apresenta exemplos de deslocamento aritmético de bits. Para estes exemplos, utiliza-se a representação Complemento a 2 para codificação dos números inteiros com sinal em bits. Na Figura 9.15 (a) o deslocamento está sendo realizado no sentido à direita. Já na Figura 9.15 (b), o sentido está realizado no sentido à esquerda.

Figura 9.15. Deslocamento Aritmético de bits.



Fonte: Próprio autor (2021).

Analisando a Figura 9.15 (a), percebe-se que ao realizar um deslocamento aritmético de bits à direita, o valor -4, após o deslocamento, sofre uma divisão inteira por 2, resultando assim no valor -2. Já no deslocamento de bits à esquerda (b), o valor -4, após o deslocamento, sofre uma multiplicação inteira por 2, resultando assim no valor -8.

O deslocamento aritmético de bits à esquerda é idêntico ao deslocamento lógico à esquerda. A diferença surge no deslocamento aritmético à direita, no qual é preservado o bit mais significativo (*Most Significant Bit* - MSB), como mostra a Figura 9.15 (a). Ao preservar o MSB, o deslocamento aritmético mantém o sinal do número representado pela sequência de bits. No exemplo da Figura 9.15 (a), se o MSB não tivesse sido preservado, o deslocamento de bits à direita resultaria no valor 126 ("01111110" em binário).

Já no CompSim, a instrução SHIFT do processador Cariri realiza deslocamento aritmético de 1 bit em palavras de 16-bits, nos sentidos à direita ou à esquerda.

O quadro a seguir traz um resumo com uma descrição da instrução SHIFT, em qual seção do código pode ser utilizada, sua sintaxe de uso e exemplos de como utilizá-la.

Instrução: SHIFT
Descrição: Realiza a operação lógica de deslocamento aritmético de bits à direita ou à esquerda dos bits contidos no registrador acumulador (AC). Para tanto, o operando da instrução contém um endereço de memória, cujo valor armazenado na respectiva posição de memória indicará o sentido do deslocamento dos bits: um valor 0 indica deslocamento à direita e um valor 1 indica deslocamento à esquerda. O resultado da operação lógica SHIFT é armazenado no registrador AC.
Seção: .code
Sintaxe: [<rotulo>:] SHIFT @<endereço> [<rotulo>:] SHIFT <variável>
Flags afetadas do SFR (Status Flags Register): Z (Zero), S (Signal) e O (Overflow).
Exemplos: SHIFT @100 ;realiza SHIFT dos bits em AC. O sentido é indicado no endereço 100. SHIFT var1 ;realiza SHIFT dos bits em AC. O sentido é indicado no valor em var1. SHIFT_100: SHIFT @100 ;instrucao com rotulo para SHIFT dos bits em AC. SHIFT_Var1: SHIFT var1 ;instrucao, com rotulo em linha separada, para SHIFT dos bits em AC.

A Figura 9.16 traz um exemplo básico de uso da instrução SHIFT. Nas Linhas 11 e 12, são definidas as variáveis “var1”, inicializada com o valor -2, em binário, e “direita”, com valor 0. Na Linha 2, os bits de “var1” são carregados em AC e, na Linha 3, realiza-se o deslocamento aritmético de 1 bit à direita, sendo o resultado armazenado em AC.

Figura 9.16. Exemplo de código Assembly para ilustrar o uso da instrução SHIFT.

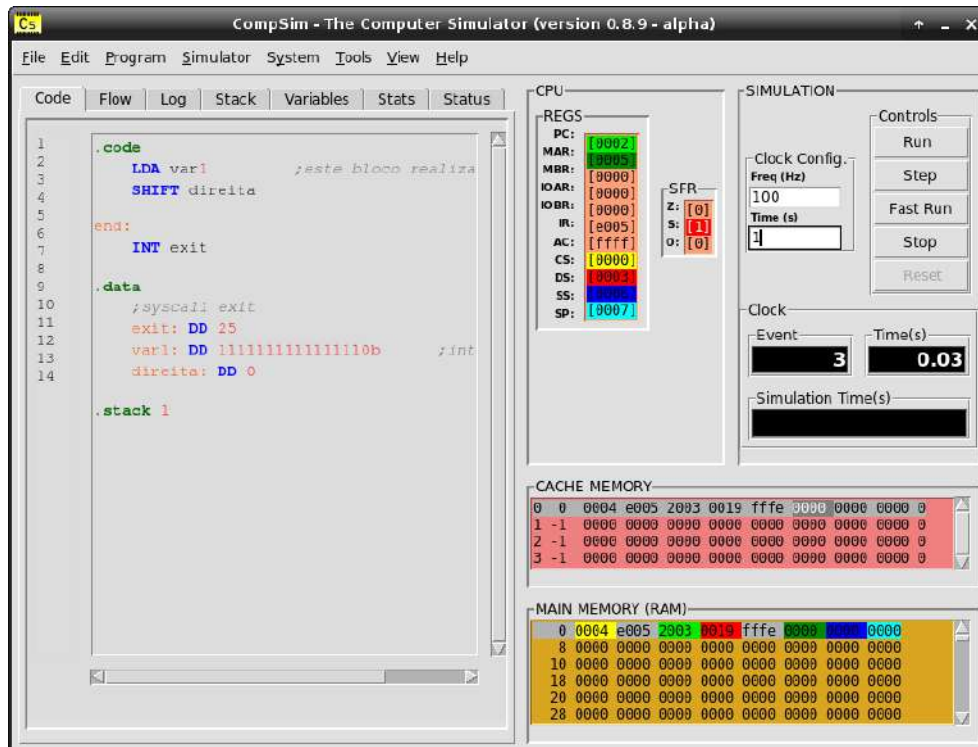
Linha	Código
1	.code
2	LDA var1 ;este bloco realiza: SHIFT(var1, direita).
3	SHIFT direita
4	
5	end:
6	INT exit
7	
8	.data
9	;syscall exit
10	exit: DD 25
11	var1: DD 111111111111110b ;int var1 = -2.
12	direita: DD 0
13	
	.stack 1

Fonte: Próprio autor (2021).

A Figura 9.17 ilustra o resultado da execução do código anterior. Na figura, pode-se verificar que o registrador AC apresenta o valor hexadecimal “ffff” (equivalente a -1 em notação Complemento a 2), que é o resultado da operação lógica de deslocamento aritmético de 1-bit (SHIFT) à direita dos bits da variável “var1”. O valor inteiro -2, anteriormente

contido em AC, após o deslocamento de bit, passou a ser -1 e, por conta disso, a flag S (*Signal*) do registrador SFR foi configurada em 1.

Figura 9.17. Visualização de operação lógica de deslocamento aritmético de bit à direita.



Fonte: Próprio autor (2021).

Como foi visto anteriormente, a operação lógica de deslocamento aritmético de bits pode ser utilizada para compor as operações de multiplicação e de divisão por 2. O deslocamento de bits também é bastante utilizado para compor operações de multiplicação e divisão inteiras por números diferentes de 2, bem como para serialização e paralelização de bits, tarefas frequentes em processos de comunicação.

A subseção a seguir ilustra três aplicações, que combinam as instruções ADD, SUB, NAND e SHIFT para compor: multiplicação de um número inteiro por 5, serialização e paralelização de um conjunto de bits.

9.3 Composição de Operações mais Complexas

Esta seção apresenta exemplos práticos de composição de operações complexas pela combinação das operações aritméticas e lógicas do simulador CompSim.

9.3.1 Multiplicação Inteira

O primeiro exemplo consiste na implementação de operação de multiplicação do número inteiro 10 por 5. Basicamente, há duas formas de realizar essa tarefa: 1) Adições sucessivas: o número 10 é somando a ele mesmo 4 vezes (e.g. $10 + 10 + 10 + 10 + 10 = 50$); ou 2) Deslocamentos de bits seguidos por soma: o número 10 sofre dois deslocamentos aritméticos à esquerda, o que caracteriza a multiplicação inteira por 4, seguido por uma soma de 10, que resulta em 50. Os códigos nas Figuras 9.18 e 9.19 ilustram como compor a operação de multiplicação utilizando adições sucessivas e utilizando deslocamentos de bit com adição, respectivamente.

Figura 9.18. Exemplo de código Assembly para multiplicação de 10 por 5, utilizando adições sucessivas.

Linha	Código
1	<code>.code</code>
2	<code> LDA var1 ;este bloco realiza: var1+var1+var1+var1+var1.</code>
3	<code> ADD var1</code>
4	<code> ADD var1</code>
5	<code> ADD var1</code>
6	<code> ADD var1</code>
7	
8	<code>end:</code>
9	<code> INT exit</code>
10	
11	<code>.data</code>
12	<code> ;syscall exit</code>
13	<code> exit: DD 25</code>
14	<code> var1: DD 10</code>
15	
16	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Figura 9.19. Exemplo de código Assembly para multiplicação de 10 por 5, utilizando deslocamentos de bit e adição.

Linha	Código
1	<code>.code</code>
2	<code> LDA var1 ;este bloco realiza: (var1*2)*2 + var1.</code>
3	<code> SHIFT esquerda</code>
4	<code> SHIFT esquerda</code>
5	<code> ADD var1</code>
6	
7	<code>end:</code>
8	<code> INT exit</code>
9	
10	<code>.data</code>
11	<code> ;syscall exit</code>
12	<code> exit: DD 25</code>
13	<code> var1: DD 10</code>
14	<code> esquerda: DD 1</code>
15	
16	<code>.stack 1</code>

Fonte: Próprio autor (2021).

9.3.2 Serialização de Bits Paralelos

O segundo exemplo consiste em realizar a serialização de bits paralelos, como é ilustrado no código da Figura 9.20.

Figura 9.20. Exemplo de código Assembly para serialização de bits paralelos.

Linha	Código
1	<code>.code</code>
2	<code>LDA paralelo ;este bloco realiza: s1 = AND(paralelo, mascara).</code>
3	<code>NAND mascara</code>
4	<code>STA tmp</code>
5	<code>NAND tmp</code>
6	<code>STA s1</code>
7	
8	<code>LDA paralelo ;este bloco realiza: paralelo=SHIFT(paralelo,direita).</code>
9	<code>SHIFT direita</code>
10	<code>STA paralelo</code>
11	
12	<code>NAND mascara ;este bloco realiza: s2 = AND(paralelo, mascara).</code>
13	<code>STA tmp</code>
14	<code>NAND tmp</code>
15	<code>STA s2</code>
16	
17	<code>LDA paralelo ;este bloco realiza: paralelo=SHIFT(paralelo,direita).</code>
18	<code>SHIFT direita</code>
19	<code>STA paralelo</code>
20	
21	<code>NAND mascara ;este bloco realiza: s3 = AND(paralelo, mascara).</code>
22	<code>STA tmp</code>
23	<code>NAND tmp</code>
24	<code>STA s3</code>
25	
26	<code>LDA paralelo ;este bloco realiza: paralelo=SHIFT(paralelo,direita).</code>
27	<code>SHIFT direita</code>
28	<code>STA paralelo</code>
29	
30	<code>NAND mascara ;este bloco realiza: s4 = AND(paralelo, mascara).</code>
31	<code>STA tmp</code>
32	<code>NAND tmp</code>
33	<code>STA s4</code>
34	
35	<code>end:</code>
36	<code>INT exit</code>
37	
38	<code>.data</code>
39	<code>;syscall exit</code>
40	<code>exit: DD 25</code>
41	<code>paralelo: DD 0000000000001010b</code>
42	<code>mascara: DD 0000000000000001b</code>
43	<code>direita: DD 0</code>
44	<code>s1: DD 0</code>
45	<code>s2: DD 0</code>
46	<code>s3: DD 0</code>
47	<code>s4: DD 0</code>
48	<code>tmp: DD 0</code>
49	
50	<code>.stack 1</code>

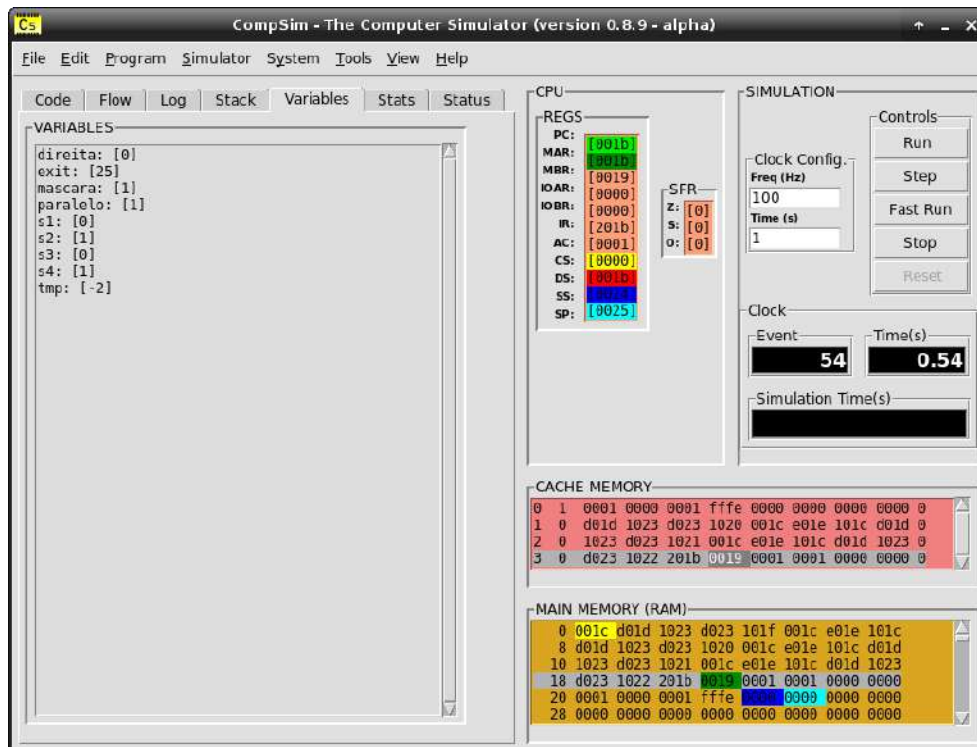
Fonte: Próprio autor (2021).

Mais especificamente, os bits paralelos estarão contidos em uma variável chamada “paralelo” e o objetivo aqui será extrair, passo-a-passo, seu bit menos significativo (*Least Significant Bit* - LSB), atualizando a variável “paralelo”, e salvando-o nas variáveis “s1”, “s2”, “s3” e “s4”. Para a serialização, será utilizada a seguinte sequência de passos:

1. Carregar os bits paralelos contidos na variável “paralelo” em AC, isolar o bit menos significativo, através de uma operação de conjunção (AND) entre os bits em AC e a máscara “0000000000000001b”, e salvar o resultado na variável “s1”;
2. Carregar os bits paralelos contidos na variável “paralelo” em AC, realizar o deslocamento aritmético de bits à direita dos bits em AC e salvar o resultado na própria variável “paralelo”, atualizando a sua sequência de bits;
3. Isolar o bit menos significativo de AC, através de uma operação de conjunção (AND) entre os bits em AC e a máscara “0000000000000001b”, e salvar o resultado em “s2”;
- e
4. Repetir mais duas vezes a sequência dos passos 2 e 3 acima, salvando os resultados em “s3” e “s4”.

A Figura 9.21 ilustra o resultado da execução do código na Figura 9.20. Na figura, pode-se verificar, no componente gráfico “Variables”, que as variáveis “s1”, “s2”, “s3” e “s4” apresentam os valores “0”, “1”, “0”, “1”, respectivamente. Esses valores foram extraídos a partir dos 4 bits menos significativos da variável “paralelo”.

Figura 9.21. Visualização de operação de serialização de bits.



Fonte: Próprio autor (2021).

9.3.3 Paralelização de Bits Seriais

O último exemplo consiste em realizar a paralelização de um conjunto de bits seriais, como é ilustrado no código da Figura 9.22. Mais especificamente será lido, passo-a-passo, o bit menos significativo (LSB) das variáveis “s1”, “s2”, “s3” e “s4”, onde, em cada passo, o bit lido será adicionado ao bit menos significativo (LSB) da variável “paralelo”, a qual, em seguida, terá seus bits deslocados para a esquerda. Ao final, os 4 bits lidos estarão armazenados de forma paralela na variável “paralelo”. Para a paralelização, será utilizada a seguinte sequência de passos:

1. Leitura do valor contido na variável “s1” para o acumulador (AC). Após a leitura, o valor em AC sofre deslocamento aritmético de bits (SHIFT) para esquerda e o resultado é armazenado na variável “paralelo”;
2. Adição do valor contido na variável “s2” ao acumulador (AC). Após a adição, o resultado em AC sofre deslocamento aritmético de bits (SHIFT) para esquerda e o resultado é armazenado na variável “paralelo”;
3. O passo anterior é repetido para a variável “s3”; e
4. Adição do valor contido na variável “s4” ao acumulador (AC). Após a adição, o resultado em AC é armazenado na variável “paralelo”.

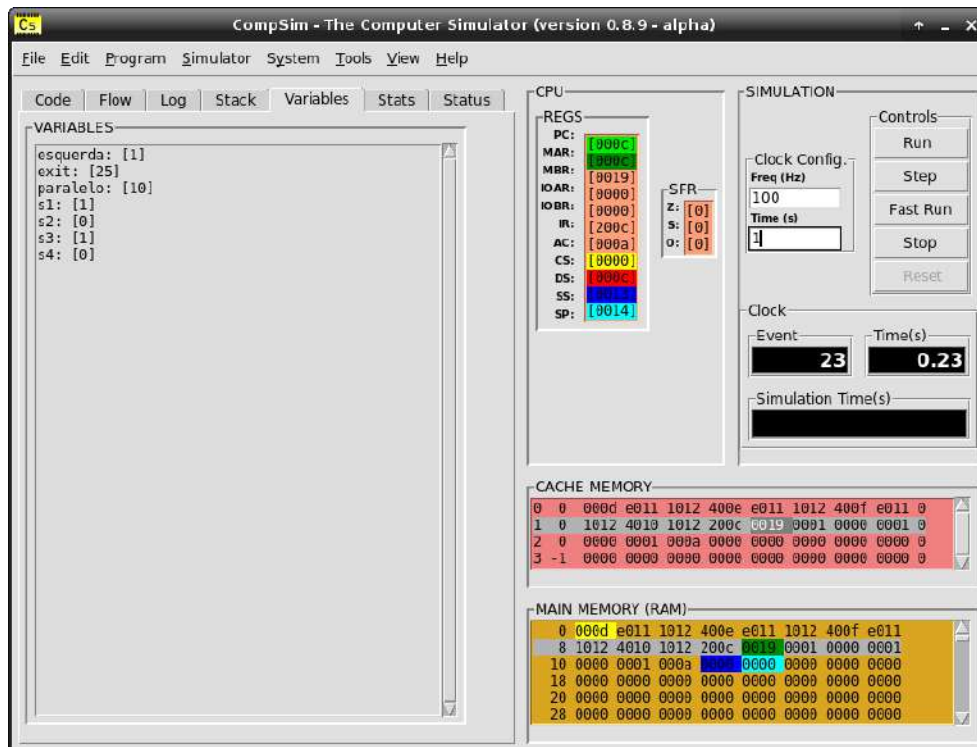
Figura 9.22. Exemplo de código Assembly para paralelização de bits seriais.

Linha	Código
1	<code>.code</code>
2	<code>LDA s1 ;este bloco realiza: paralelo = s1 << 1.</code>
3	<code>SHIFT esquerda</code>
4	<code>STA paralelo</code>
5	
6	<code>ADD s2 ;este bloco realiza: paralelo = (paralelo+s2) << 1.</code>
7	<code>SHIFT esquerda</code>
8	<code>STA paralelo</code>
9	
10	<code>ADD s3 ;este bloco realiza: paralelo = (paralelo+s3) << 1.</code>
11	<code>SHIFT esquerda</code>
12	<code>STA paralelo</code>
13	
14	<code>ADD s4 ;este bloco realiza: paralelo = paralelo + s4.</code>
15	<code>STA paralelo</code>
16	
17	<code>end:</code>
18	<code>INT exit</code>
19	
20	<code>.data</code>
21	<code>;syscall exit</code>
22	<code>exit: DD 25</code>
23	<code>s1: DD 1</code>
24	<code>s2: DD 0</code>
25	<code>s3: DD 1</code>
26	<code>s4: DD 0</code>
27	<code>esquerda: DD 1</code>
28	<code>paralelo: DD 0</code>
29	
30	<code>.stack 1</code>

Fonte: Próprio autor (2021).

A Figura 9.23 ilustra o resultado da execução do código da Figura 9.22. Na figura, pode-se verificar, no componente gráfico “Variables”, que a variável “paralelo” apresenta o valor 10, que em binário consiste da sequência de bits “1010”. Essa sequência foi obtida a partir da leitura e paralelização dos valores das variáveis “s1”, “s2”, “s3” e “s4”.

Figura 9.23. Visualização de operação de paralelização de bits.



Fonte: Próprio autor (2021).

9.4 Resumo

Este capítulo apresentou detalhes de como se dá o processamento de dados em um processador. O capítulo foi dividido em duas partes: na primeira, foram abordadas as instruções do processador que lidam com operações aritméticas; e, na segunda, as instruções que operam sobre bits, através de operações lógicas.

Ao longo do capítulo, buscou-se trazer exemplos práticos de como pode-se utilizar as instruções aritméticas e lógicas do processador Cariri na elaboração de diversas aplicações computacionais, que variaram desde simples aplicações para adição/subtração de números inteiros até composições de operações mais complexas, pela combinação de instruções do processador, tais como multiplicação, serialização e paralelização de bits.

CAPÍTULO 10 – CONTROLE DE FLUXO DE EXECUÇÃO DO PROGRAMA

Um programa de computador consiste em uma sequência de instruções, as quais são lidas e executadas respeitando uma ordem de execução. Um controle de fluxo de execução, em termos gerais, consiste em, dada uma determinada condição, alterar a ordem em que as instruções de um programa são executadas. Controle de fluxo pode ser importante em várias situações, tais como: dada uma condição, executar uma de duas possíveis ações; repetir uma sequência de ações até que uma condição não mais seja atendida; repetir determinadas ações por uma quantidade predefinida de vezes; entre outras.

Até este ponto, os programas em linguagem *Assembly* desenvolvidos neste livro não lidaram com controle de fluxo de execução. Este capítulo apresenta as instruções de controle de fluxo de execução do processador Cariri e como elas podem ser utilizadas para compor, em programas *Assembly*, as construções de controle de fluxo de linguagens de alto nível.

10.1 Instruções de Transferência de Controle

O processador Cariri possui três instruções de transferência de controle de fluxo, sendo uma delas de salto incondicional (“JMP”) e duas de salto condicional (“JZ” e “JN”).

A instrução de salto incondicional JMP (*Jump*), ao ser executada, transfere seu operando, que consiste em um endereço de memória, para o registrador contador de programa (PC), de forma que, no próximo ciclo de instrução, o programa passará a executar a instrução apontada no novo endereço em PC. Já as instruções de salto condicional, antes de atualizarem o endereço em PC, avaliam se determinados bits do registrador de flags de status (*Status Flags Register* - SFR) estão configurados em 1. No caso, a instrução JZ (*Jump if Zero*) avalia o bit Z do registrador SFR, enquanto que a instrução JN (*Jump if Negative*) avalia o bit S de

SFR. Por exemplo, se estiver sendo executada uma instrução JZ e se o bit Z de SFR estiver configurado em 1, a instrução fará com que o endereço de memória contido em seu operando seja copiado para PC. Dessa forma, no próximo ciclo de instrução, será executada a instrução apontada pelo novo endereço em PC. Após um salto condicional, as instruções JZ e JN configuram em 0 os bits Z (*Zero*) e S (*Signal*) de SFR, respectivamente.

Os quadros a seguir trazem descrições de cada uma dessas instruções, apresentando a seção do código onde podem ser utilizadas, sua sintaxe e exemplos de uso.

Instrução: JMP (JuMP)	
Descrição: Desvia o fluxo de execução do programa para uma determinada instrução. Recebe como operando um endereço de memória, o qual será copiado para o registrador PC. Com isso, no próximo ciclo de instrução será executada a instrução no novo endereço apontado em PC.	
Seção: .code	
Sintaxe: [<rotulo>:] JMP @<endereço> [<rotulo>:] JMP <rotulo>	
Exemplos: JMP @100 ;salta para a instrução no endereço 100. JMP inst1 ;salta para a instrução com rotulo inst1. Salta_100: JMP @100 ;instrucao com rotulo para salto para instrução no endereço 100. Salta_inst1: JMP inst1 ;instrucao com rotulo em linha separada, para salto para instrução com rotulo inst1.	

Instrução: JZ (Jump if Zero)	
Descrição: Caso o resultado da execução da instrução anterior tenha disparado a flag Z (<i>Zero</i>) do registrador SFR, desvia o fluxo de execução do programa para uma determinada instrução. Recebe como operando um endereço de memória, o qual será copiado para o registrador PC. Com isso, no próximo ciclo de instrução será executada a instrução no novo endereço apontado em PC.	
Seção: .code	
Sintaxe: [<rotulo>:] JZ @<endereço> [<rotulo>:] JZ <rotulo>	
Flags afetadas do SFR (<i>Status Flags Register</i>): Z (<i>Zero</i>)	
Exemplos: JZ @100 ;se SFR(Z) =1, salta para a instrução no endereço 100. JZ inst1 ;se SFR(Z) =1, salta para a instrução com rotulo inst1. Salta 100: JZ @100 ;instrucao com rotulo para salto para instrução no endereço 100, se SFR(Z) =1. Salta inst1: JZ inst1 ;instrucao com rotulo em linha separada, para salto para instrução com rotulo inst1, ; se SFR(Z) =1.	

Instrução: JN (Jump if N egative)
Descrição: Caso o resultado da execução da instrução anterior tenha disparado a flag S (<i>Signal</i>) do registrador SFR, desvia o fluxo de execução do programa para uma determinada instrução. Recebe como operando um endereço de memória, o qual será copiado para o registrador PC. Com isso, no próximo ciclo de instrução será executada a instrução no novo endereço apontado em PC.
Seção: .code
Sintaxe: [<rotulo>:] JN @<endereço> [<rotulo>:] JN <rotulo>
Flags afetadas do SFR (<i>Status Flags Register</i>): S (<i>Signal</i>)
Exemplos: JN @100 ;se SFR(S) =1, salta para a instrução no endereço 100. JN inst1 ;se SFR(S) =1, salta para a instrução com rotulo inst1. Salta_100: JN @100 ;instrucao com rotulo para salto para instrução no endereço 100, se SFR(S) =1. Salta_inst1: JN inst1 ;instrucao com rotulo em linha separada, para salto para instrução com rotulo inst1, ; se SFR(S) =1.

A Figura 10.1 apresenta um exemplo básico de uso da instrução JMP. Na figura, a Linha 1 inclui uma instrução de salto incondicional JMP para a instrução com rótulo “end”. As Linhas 4 à 6 realizam a soma dos valores das variáveis “var1” e “var2”, sendo o resultado armazenado na variável “var1”. Por fim, nas Linhas 8 e 9 há o rótulo “end:” da instrução INT, respectivamente. A instrução INT, neste exemplo, está sendo utilizada para encerrar o ciclo de instrução do processador Cariri.

Figura 10.1. Exemplo de código Assembly para ilustrar o uso da instrução JMP.

Linha	Código
1	.code
2	JMP end ;salta para end:.
3	
4	LDA var1 ;este bloco realiza: var1 = var1 + var2.
5	ADD var2
6	STA var1
7	
8	end:
9	INT exit
10	
11	.data
12	;syscall exit
13	exit: DD 25
14	var1: DD 10
15	var2: DD 20
16	
17	.stack 1

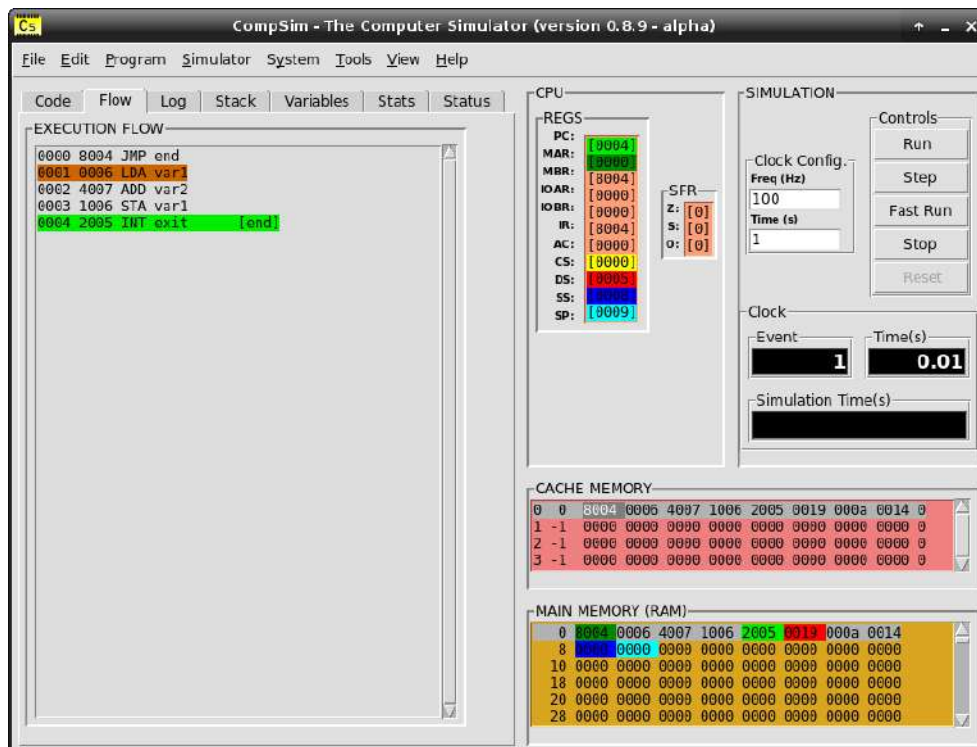
Fonte: Próprio autor (2021).

Capítulo 10 - Controle de Fluxo de Execução do Programa

Como observa-se na sequência de instruções do programa da Figura 10.1, a primeira instrução (JMP) realiza um desvio de fluxo de execução do programa para a última instrução (INT), encerrando, desta forma, a execução do programa, sem que as instruções das Linhas 4 à 6 sejam executadas.

A Figura 10.2 apresenta a visualização da execução do programa da Figura 10.1. Na figura, o componente gráfico “EXECUTION FLOW”, que apresenta a sequência de execução das instruções do programa, mostra o resultado da execução da instrução JMP. A instrução destacada em cor marrom qual seria a próxima instrução do fluxo de execução a ser executada (o registrador PC havia sido incrementado, no ciclo de instrução anterior, para apontar para ela). Porém, com a execução da instrução JMP, o endereço de memória em PC foi alterado para apontar para a instrução com rótulo “end”, que é a instrução INT e que está destacada em verde. Observa-se então, que houve um salto de fluxo de execução da instrução no endereço “0000” (JMP) para a instrução no endereço “0004” (INT).

Figura 10.2. Visualização da execução da instrução JMP.



Fonte: Próprio autor (2021).

A seção a seguir ilustra como utilizar as instruções JMP, JZ e JN para implementar as estruturas (ou construções) de desvio de fluxo de linguagens de alto nível, tais como If..Else, While..Do e For..Do.

10.2 Estruturas de Controle de Fluxo de Linguagens de Alto Nível

As linguagens de programação de alto nível geralmente incluem diferentes estruturas de controle de fluxo de execução do programa. Entre as estruturas, as mais comuns são:

- Estruturas condicionais (ou de decisão): são aquelas que permitem a execução de ações, caso uma determinada condição seja atendida. Podem ser (SCHILDT, 1997):
 - Simples: permite executar determinadas ações, caso uma condição seja satisfeita. Esta estrutura condicional é conhecida como “If..Then” (“Se..Então”);
 - Composta: permite executar determinadas ações, caso a condição seja satisfeita, porém, em caso contrário, executa um outro conjunto de ações. Esta estrutura condicional é conhecida como “If..Else” (“Se..Senão”); e
 - De Múltiplos Casos: permite executar ações, as quais estão condicionadas a um de múltiplos casos. Esta estrutura condicional é conhecida como “Switch..Case” (“Escolha..Caso”);
- Estruturas de repetição: são aquelas que permitem a execução com repetição de ações. Pode ser com:
 - Variável de controle: as ações são repetidas em uma quantidade predefinida de vezes, que geralmente é controlada por uma variável do programa. Esta estrutura de repetição é conhecida como “For..Do” (“Para..Faça”);
 - Teste no início: utilizada quando não há uma quantidade predefinida de repetições e inclui uma condição inicial que, caso seja satisfeita, permite a repetição da execução das ações. Esta estrutura de repetição é conhecida como “While..Do” (“Enquanto..Faça”); e
 - Teste no fim: utilizada quando não há uma quantidade predefinida de repetições, mas sabe-se que as ações devem ser executadas pelo menos uma vez. A condição de repetição é incluída após as ações a serem executadas. Esta estrutura de repetição é conhecida como “Do..While” (“Faça..Enquanto”).

As subseções a seguir ilustram como pode-se compor cada uma dessas estruturas em linguagem *Assembly*, utilizando as instruções JMP, JZ e JN do processador Cariri.

10.2.1 Estrutura Condicional If..Then

A Figura 10.3 apresenta um exemplo de programa escrito em linguagem C que ilustra o uso da estrutura condicional If..Then. Na figura, a condicional If..Then, presente na Linha 4, verifica se o valor da variável “var1” é maior do que o valor da variável “var2”. Se essa condição for atendida, a instrução de atribuição da Linha 5 será executada.

Figura 10.3 Exemplo de código C com uso de condicional If..Then.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2 = 20;</code>
3	
4	<code>if (var1 > var2)</code>
5	<code>var2 = var1;</code>

Fonte: Próprio autor (2021).

A Figura 10.4 ilustra um programa *Assembly* que implementa o programa C com a estrutura condicional If..Then, da figura anterior. Na Figura 10.4, o bloco nas Linhas 2 à 6 implementa a condicional If. Como não há instruções de comparação de dados no processador Cariri, utilizou-se um artifício aritmético para comparar as variáveis “var1” e “var2”. A Linha 3 faz com que o valor da variável “var1” seja carregado no acumulador e, na Linha 4, desse valor é subtraído o valor da variável “var2”. Dessa subtração, há três possíveis resultados:

1. Se o valor da variável “var1” for maior do que o de “var2”, o resultado será um inteiro positivo;
2. Se o valor da variável “var1” for igual ao de “var2”, o resultado será zero, disparando assim a flag Z (*Zero*) do registrador SFR (*Status Flag Register*); ou
3. Se o valor da variável “var1” for menor do que o de “var2”, o resultado será um inteiro negativo, disparando assim a flag S (*Signal*) do registrador SFR.

Observa-se então que a operação aritmética de subtração (SUB) e os bits S e Z do registrador SFR são suficientes para estabelecer comparações numéricas em um programa *Assembly*.

No exemplo da Figura 10.3, a condicional verifica se o valor de “var1” é maior do que o valor de “var2” e, em caso afirmativo, será executada a instrução de atribuição da Linha 5. Em *Assembly*, na Figura 10.4, essa comparação é implementada da seguinte forma:

1. Subtrai-se o valor da variável “var2” da variável “var1” (Linhas 3 e 4);

2. Verifica-se se o resultado da subtração é igual a zero (JZ) e, em caso afirmativo, compreende-se que as variáveis possuem o mesmo valor, realizando-se um salto para o fim do programa (Linha 5);
3. Verifica-se se o resultado da subtração é igual a um inteiro negativo (JN) e, em caso afirmativo, compreende-se que o valor em “var1” é menor que o valor em “var2”, realizando-se um salto para o fim do programa;
4. Caso a subtração não resulte em zero ou em um inteiro negativo, significa que o valor de “var1” é maior do que “var2”, atendendo a condicional “var1 > var2” e, portanto não atendeu às condições de salto das instruções JZ (Linha 5) e JN (Linha 6).

Com isso, no programa *Assembly*, da Figura 10.4, o fluxo de execução segue sem desvio de fluxo, sendo portanto executadas as instruções do bloco “then:” (Linhas 7 à 9), que implementam a atribuição “var2 = var1”.

Figura 10.4. Exemplo de código Assembly para ilustrar a construção de condicional If.

Linha	Código
1	<code>.code</code>
2	<code>if:</code>
3	<code> LDA var1 ;este bloco realiza: if (var1 > var2).</code>
4	<code> SUB var2</code>
5	<code> JZ end</code>
6	<code> JN end</code>
7	<code>then:</code>
8	<code> LDA var1 ;var2 = var1.</code>
9	<code> STA var2</code>
10	
11	<code>end:</code>
12	<code> INT exit</code>
13	
14	<code>.data</code>
15	<code> ;syscall exit</code>
16	<code>exit: DD 25</code>
17	<code>var1: DD 10</code>
18	<code>var2: DD 20</code>
19	
20	<code>.stack 1</code>

Fonte: Próprio autor (2021).

10.2.2 Estrutura Condicional If..Else

A Figura 10.5 apresenta um exemplo de programa escrito em linguagem C que ilustra o uso da estrutura condicional If..Else. Na figura, a condicional If, presente na Linha 4, verifica se o valor da variável “var1” é maior do que o valor da variável “var2”. Se essa condição for atendida, a instrução de atribuição da Linha 5 será executada. Caso contrário, será executada a instrução da Linha 7, no bloco “else”.

Figura 10.5 Exemplo de código C com uso de condicional If..Else.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2 = 20;</code>
3	
4	<code>if (var1 > var2)</code>
5	<code>var2 = var1;</code>
6	<code>else</code>
7	<code>var1 = var2;</code>

Fonte: Próprio autor (2021).

A Figura 10.6 ilustra um programa *Assembly* que implementa o programa C com a estrutura condicional If..Else, da figura anterior. Assim como no código *Assembly* da Figura 10.4, utilizou-se novamente a instrução de subtração (SUB) e as flags Z e S do registrador SFR. A diferença entre os programas *Assembly* das Figura 10.4 e 10.6, é que, no primeiro, o salto é realizado para o fim do programa; já no segundo, o salto segue para as instruções do bloco “else:”, implementando assim a condicional If..Else.

Na Figura 10.6, as instruções: nas Linhas 3 à 6, implementam a condicional If; nas Linhas 8 e 9, implementam a atribuição “var2 = var1”, caso a condicional seja satisfeita; e, nas Linhas 11 e 12, implementam a atribuição “var1 = var2”, caso a condicional não seja satisfeita (bloco “else:”).

Figura 10.6. Exemplo de código Assembly para ilustrar a construção de condicional If..Else.

Linha	Código
1	<code>.code</code>
2	<code>if:</code>
3	<code>LD A, var1 ;este bloco realiza: if (var1 > var2).</code>
4	<code>SUB A, var2</code>
5	<code>JN else</code>
6	<code>JZ else</code>
7	<code>then:</code>
8	<code>LD A, var1 ;var2 = var1.</code>
9	<code>STA var2</code>
10	<code>else:</code>
11	<code>LD A, var2 ;var1 = var2.</code>
12	<code>STA var1</code>
13	
14	<code>end:</code>
15	<code>INT exit</code>
16	
17	<code>.data</code>
18	<code>;syscall exit</code>
19	<code>exit: DD 25</code>
20	<code>var1: DD 10</code>
21	<code>var2: DD 20</code>
22	
23	<code>.stack 1</code>

Fonte: Próprio autor (2021).

10.2.3 Estrutura Condicional Switch..Case

A estrutura condicional Switch..Case é utilizada quando deseja-se realizar determinadas ações, que estão vinculadas à diferentes alternativas de uma condicional. Na Figura 10.7, nas Linhas 3 à 14, é possível visualizar uma estrutura condicional Switch..Case. Na Linha 3, avalia-se o conteúdo da variável “valor”; caso o conteúdo seja igual ao inteiro 1, como é verificado na Linha 4, serão executadas as instruções das Linhas 5 e 6, que atribui o valor 10 à variável “valor” e encerra a estrutura condicional, respectivamente; caso o conteúdo seja igual ao inteiro 2, como é verificado na Linha 8, serão executadas as instruções das Linhas 9 e 10, que atribui o valor 20 à variável “valor” e encerra a estrutura condicional, respectivamente; se os casos anteriores não forem verificados, será executado o bloco “default”, com execução da instrução da Linha 13, que atribui o valor 30 à variável “valor”.

Figura 10.7 Exemplo de código C com uso de condicional Switch..Case.

Linha	Código
1	<code>int valor = 0;</code>
2	
3	<code>switch (valor){</code>
4	<code> case 1:</code>
5	<code> valor = 10;</code>
6	<code> break;</code>
7	
8	<code> case 2:</code>
9	<code> valor = 20;</code>
10	<code> break;</code>
11	
12	<code> default :</code>
13	<code> valor = 30;</code>
14	<code>}</code>

Fonte: Próprio autor (2021).

A Figura 10.8 ilustra um programa *Assembly* que implementa o programa C com a estrutura condicional Switch..Case, da figura anterior. Na Figura 10.8, as instruções: das Linhas 3 e 4, verificam se o conteúdo da variável “valor” é igual a 1, e, em caso afirmativo, realiza-se o desvio de fluxo de execução para o bloco “case_1”, via instrução JZ na Linha 5; das Linhas 6 e 7, verificam se o conteúdo da variável valor é igual a 2, e, em caso afirmativo, realiza-se o desvio de fluxo de execução para o bloco “case_2”, via instrução JZ na Linha 8; e, da Linha 9, é realizado o desvio de fluxo de execução para o bloco “default”.

Figura 10.8. Exemplo de código Assembly para ilustrar a construção de condicional Switch..Case.

Linha	Código
1	<code>.code</code>
2	<code>switch:</code>
3	<code>MOV 1</code>
4	<code>SUB valor ;se valor = 1,</code>
5	<code>JZ case_1 ; salta para case_1.</code>
6	<code>MOV 2</code>
7	<code>SUB valor ;se valor = 2,</code>
8	<code>JZ case_2 ; salta para case_2.</code>
9	<code>JMP default ;caso contrario, salta para default.</code>
10	
11	<code>case_1: ;case 1.</code>
12	<code>MOV 10</code>
13	<code>STA valor</code>
14	<code>JMP end</code>
15	
16	<code>case_2: ;case 2.</code>
17	<code>MOV 20</code>
18	<code>STA valor</code>
19	<code>JMP end</code>
20	
21	<code>default: ;default.</code>
22	<code>MOV 30</code>
23	<code>STA valor</code>
24	<code>JMP end</code>
25	
26	<code>end:</code>
27	<code>INT exit</code>
28	
29	<code>.data</code>
30	<code>;syscall exit</code>
31	<code>exit: DD 25</code>
32	<code>valor: DD 0</code>
33	
34	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Observa-se ainda, na Figura 10.8, que, para a comparação da variável “valor” com o número 1, nas Linhas 3 e 4, utilizou-se uma nova instrução, a instrução “MOV”. No exemplo, na Linha 3, a instrução MOV é utilizada para atribuição do valor 1 ao registrador acumulador AC. Com isso, pode-se subtrair o conteúdo a variável “valor” do conteúdo de AC, conforme consta na Linha 4. A instrução MOV também é utilizada na Linha 6, porém para atribuição do valor 2 ao AC.

Além de atribuições de literais (inteiros positivos) ao registrador acumulador AC, a instrução MOV, pode também:

- Copiar o conteúdo de um registrador para AC;
- Copiar o conteúdo de AC para um registrador;
- Copiar o endereço de memória que aponta para uma variável para AC; e
- Copiar um determinado endereço de memória para AC.

O quadro a seguir traz um resumo com a descrição da instrução MOV, em qual seção do código pode ser utilizada, sua sintaxe de uso e exemplos de como utilizá-la.

Instrução: MOV	
Descrição: Atribui um número inteiro positivo ao registrador acumulador AC. Recebe como operando um número inteiro de 12-bits sem sinal (<i>unsigned</i>).	
Seção: .code	
Sintaxe: [<rotulo>:] MOV <inteiro> [<rotulo>:] MOV \$<registrador> [<rotulo>:] MOV -\$<registrador> [<rotulo>:] MOV <variável> [<rotulo>:] MOV @<endereço>	
Exemplos: MOV 10 ;atribui 10 ao registrar acumulador (AC). MOV \$CS ; copia o conteúdo do registrador CS para o registrador acumulador (AC). MOV -\$DS ; copia o conteúdo do registrador acumulador AC para o registrador DS. MOV var1 ; copia o endereço de memória da variável var1 para o registrador acumulador AC. MOV @100 ; copia o endereço de memória 100 para o registrador acumulador AC.	

10.2.4 Estrutura de Repetição For...Do

A estrutura de repetição For..Do é utilizada quando um conjunto de ações deve ser repetido uma por uma certa quantidade de vezes. A quantidade de repetições geralmente é controlada por uma variável do programa, como pode ser visto no Figura 10.9.

Na Figura 10.9, a variável “contador” está sendo utilizada para determinar o número de repetições da estrutura de repetição For..Do, contida nas Linhas 4 à 6. Na linha 4, a variável “contador” é inicializada com valor 0; enquanto seu valor for menor do que 5, a instrução da Linha 5 será executada repetidamente; e, a cada repetição, o conteúdo da variável “contador” será incrementado em 1 unidade. Observa-se então que, no momento em que a variável “contador” se tornar igual a 5, a estrutura de repetição será encerrada.

Figura 10.9 Exemplo de código C com uso de repetição For..Do.

Linha	Código
1	<code>int contador;</code>
2	<code>int var = 0;</code>
3	
4	<code>for(contador=0; contador<5; contador++){</code>
5	<code> var = var + 10;</code>
6	<code>}</code>

Fonte: Próprio autor (2021).

A Figura 10.10 ilustra um programa *Assembly* que implementa o programa C com a estrutura de repetição *For..Do*, da figura anterior. Na Figura 10.10, compara-se o conteúdo da variável “contador” com o valor 5, utilizando as instruções *MOV* e *SUB* das Linhas 3 e 4; e, caso os valores sejam iguais, na Linha 5 realiza-se o desvio de fluxo para o fim do programa (caracteriza-se aqui o encerramento da estrutura de repetição *For..Do*). As instruções nas Linhas 6 à 8, realizam o incremento do conteúdo da variável “contador”; e, nas Linhas 10 à 12, executa-se as ações que deverão ser executadas a cada repetição (no caso, “var = var + 10”). A linha 14 é responsável por desviar o fluxo de execução do programa para a instrução da linha 3, caracterizando assim o início de uma nova repetição.

Figura 10.10. Exemplo de código *Assembly* para ilustrar a construção de repetição *For..Do*.

Linha	Código
1	<code>.code</code>
2	<code>for:</code>
3	<code>MOV 5 ;este bloco: for(contador=0; contador<5; contador++).</code>
4	<code>SUB contador</code>
5	<code>JZ end</code>
6	<code>MOV 1</code>
7	<code>ADD contador</code>
8	<code>STA contador</code>
9	<code>do:</code>
10	<code>MOV 10 ;este bloco: var = var + 10.</code>
11	<code>ADD var</code>
12	<code>STA var</code>
13	
14	<code>JMP for ;segue para o inicio da proxima repeticao.</code>
15	
16	<code>end:</code>
17	<code>INT exit</code>
18	
19	<code>.data</code>
20	<code>;syscall exit</code>
21	<code>exit: DD 25</code>
22	<code>contador: DD 0</code>
23	<code>var: DD 0</code>
24	
25	<code>.stack 1</code>

Fonte: Próprio autor (2021).

10.2.5 Estrutura de Repetição *While..Do*

A estrutura de repetição *While..Do* é utilizada quando um conjunto de ações deve ser repetido, porém não se sabe *a priori* a quantidade de repetições. A estrutura de repetição *While..Do* inclui uma condição no início da estrutura que, caso seja satisfeita, permite a repetição da execução das ações, como pode ser visto no Figura 10.11. Na figura, na Linha 3,

a condição de repetição é satisfeita enquanto o valor da variável “contador” for menor do que 5.

Figura 10.11 Exemplo de código C com uso de repetição While..Do.

Linha	Código
1	<code>int contador = 0;</code>
2	
3	<code>while (contador < 5){</code>
4	<code> contador++;</code>
5	<code>}</code>

Fonte: Próprio autor (2021).

A Figura 10.12 ilustra um programa *Assembly* que implementa o programa C com a estrutura de repetição While..Do, da figura anterior. Na Figura 10.12, compara-se o conteúdo da variável “contador” com o valor 5, utilizando as instruções MOV e SUB das Linhas 3 e 4; e, caso os valores sejam iguais, na Linha 5 realiza-se o desvio de fluxo para o fim do programa (caracteriza-se aqui o encerramento da estrutura de repetição While..Do). As instruções nas Linhas 7 à 9, realizam as ações que deverão ser executadas a cada repetição (no caso, consiste no incremento do conteúdo da variável “contador”). A linha 11 é responsável por desviar o fluxo de execução do programa para a instrução da linha 3, caracterizando assim o início de uma nova repetição.

Figura 10.12. Exemplo de código Assembly para ilustrar a construção de repetição While..Do.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code> MOV 5 ;este bloco: while (contador < 5).</code>
4	<code> SUB contador</code>
5	<code> JZ end</code>
6	<code>do:</code>
7	<code> MOV 1 ;este bloco: contador++.</code>
8	<code> ADD contador</code>
9	<code> STA contador</code>
10	
11	<code> JMP while ;segue para o inicio da proxima repeticao.</code>
12	
13	<code>end:</code>
14	<code> INT exit</code>
15	
16	<code>.data</code>
17	<code> ;syscall exit</code>
18	<code>exit: DD 25</code>
19	<code>contador: DD 0</code>
20	
21	<code>.stack 1</code>

Fonte: Próprio autor (2021).

10.2.6 Estrutura de Repetição Do..While

A estrutura de repetição Do..While é utilizada quando um conjunto de ações deve ser repetido, porém sabe-se *a priori* que o conjunto de ações deve ser executado pelo menos uma vez. A estrutura de repetição Do..While inclui ao final da estrutura uma condição que, caso seja satisfeita, permite a repetição da execução das ações, como pode ser visto no Figura 10.13. Na figura, na Linha 5, a condição de repetição é satisfeita enquanto o valor da variável “contador” for menor do que 5.

Figura 10.13 Exemplo de código C com uso de repetição Do..While.

Linha	Código
1	<code>int contador = 0;</code>
2	
3	<code>do {</code>
4	<code> contador++;</code>
5	<code>} while (contador < 5);</code>

Fonte: Próprio autor (2021).

A Figura 10.14 ilustra um programa *Assembly* que implementa o programa C com a estrutura de repetição Do..While, da figura anterior.

Figura 10.14. Exemplo de código Assembly para ilustrar a construção de repetição Do..While.

Linha	Código
1	<code>.code</code>
2	<code>do:</code>
3	<code> MOV 1 ;este bloco: contador++.</code>
4	<code> ADD contador</code>
5	<code> STA contador</code>
6	<code>while:</code>
7	<code> MOV 5 ;este bloco: while (contador < 5).</code>
8	<code> SUB contador</code>
9	<code> JZ end</code>
10	
11	<code> JMP do ;segue para o inicio da proxima repeticao.</code>
12	
13	<code>end:</code>
14	<code> INT exit</code>
15	
16	<code>.data</code>
17	<code> ;syscall exit</code>
18	<code>exit: DD 25</code>
19	<code>contador: DD 0</code>
20	
21	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Na Figura 10.14, as instruções nas Linhas 3 à 5, realizam as ações que deverão ser executadas a cada repetição (no caso, consiste no incremento do conteúdo da variável

“contador”). Nas Linhas 7 à 9, compara-se o conteúdo da variável “contador” com o valor 5, utilizando as instruções MOV e SUB; e, caso os valores sejam iguais, na Linha 9 realiza-se o desvio de fluxo para o fim do programa (caracteriza-se aqui o encerramento da estrutura de repetição Do..While). A linha 11 é responsável por desviar o fluxo de execução do programa para a instrução da linha 3, caracterizando assim o início de uma nova repetição.

10.3 Exemplos Práticos

Esta seção apresenta três exemplos de aplicação de estruturas de controle de fluxo de execução para implementar as operações aritméticas de multiplicação e divisão inteiras, e computar um determinado número na sequência de Fibonacci.

10.3.1 Exemplo 1: Multiplicação Inteira

O código em linguagem C na Figura 10.15 ilustra como computar a multiplicação inteira utilizando a técnica de adições sucessivas. Na figura, deseja-se multiplicar 5 (variável “multiplicando”) por 3 (variável “multiplicador”). A variável “contador” gerencia o número de repetições da estrutura de repetição While..Do, em que, em cada repetição, adiciona-se o 5 à variável “resultado”. No exemplo, a multiplicação de 5 por 3, envolverá três 3 repetições da adição do valor da variável “multiplicando” à variável “resultado” (0+5+5+5).

Figura 10.15. Exemplo de código C para multiplicação por adições sucessivas.

Linha	Código
1	<code>int multiplicando = 5;</code>
2	<code>int multiplicador = 3;</code>
3	<code>int contador = 0;</code>
4	<code>int resultado = 0;</code>
5	
6	<code>while(contador < multiplicador){</code>
7	<code> resultado = resultado + multiplicando;</code>
8	<code> contador++;</code>
9	<code>}</code>

Fonte: Próprio autor (2021).

O código na Figura 10.16 apresenta uma implementação em *Assembly* do código C de multiplicação por adições sucessivas da Figura 10.15. Na Figura 10.16: as Linhas 3 à 5 consistem da condição da estrutura de repetição While..Do; as Linhas 7 à 9 realizam uma adição para contabilizar o cálculo da multiplicação; as Linhas 11 à 13 fazem o incremento da

variável “contador”; e, a Linha 15 desvia o fluxo de execução do programa para o início da estrutura de repetição While..Do, caracterizando uma nova repetição.

Figura 10.16. Exemplo de código Assembly para multiplicação por adições sucessivas.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code> LDA multiplicador ;este bloco: while (contador < multiplicador).</code>
4	<code> SUB contador</code>
5	<code> JZ end</code>
6	<code>do:</code>
7	<code> LDA resultado ;este bloco: resultado=multiplicando+resultado.</code>
8	<code> ADD multiplicando</code>
9	<code> STA resultado</code>
10	
11	<code> MOV 1 ;este bloco: contador++.</code>
12	<code> ADD contador</code>
13	<code> STA contador</code>
14	
15	<code> JMP while ;segue para o inicio da proxima repeticao.</code>
16	
17	<code>end:</code>
18	<code> INT exit</code>
19	
20	<code>.data</code>
21	<code> ;syscall exit</code>
22	<code> exit: DD 25</code>
23	<code> multiplicando: DD 5</code>
24	<code> multiplicador: DD 3</code>
25	<code> contador: DD 0</code>
26	<code> resultado: DD 0</code>
27	
28	<code>.stack 1</code>

Fonte: Próprio autor (2021).

10.3.2 Exemplo 2: Divisão Inteira

O código em linguagem C na Figura 10.17 ilustra como computar a divisão inteira utilizando a técnica de subtrações sucessivas. Na figura, deseja-se dividir 12 (variável “dividendo”) por 3 (variável “divisor”). Na estrutura de repetição While..Do, as repetições ocorrerão enquanto o valor da variável “dividendo” for maior ou igual à variável “divisor”. À cada repetição, é subtraído o valor na variável “divisor” da variável “dividendo”, e a variável “quociente” computa o número de repetições. Ao fim das repetições, a variável “quociente”, que computou o número de subtrações, contera o resultado da divisão inteira. No exemplo, a divisão de 12 por 3, envolverá 4 repetições da subtração do valor da variável “divisor” da variável “dividendo” (12 - 3 - 3 - 3 - 3).

Figura 10.17. Exemplo de código C para divisão por subtrações sucessivas.

Linha	Código
1	<code>int dividendo = 12;</code>
2	<code>int divisor = 3;</code>
3	<code>int quociente = 0;</code>
4	
5	<code>while(dividendo >= divisor){</code>
6	<code> dividendo = dividendo - divisor;</code>
7	<code> quociente++;</code>
8	<code>}</code>

Fonte: Próprio autor (2021).

O código na Figura 10.18 apresenta uma implementação em *Assembly* do código C de divisão por subtrações sucessivas da Figura 10.17. Na Figura 10.18: as Linhas 3 à 5 consistem da condição da estrutura de repetição While..Do; as Linhas 7 à 9 realizam uma subtração do divisor do dividendo; as Linhas 11 à 13 fazem a contabilização do número de repetições, que é o resultado da divisão inteira, na variável “quociente”; e, a Linha 15 desvia o fluxo de execução do programa para o início da estrutura de repetição While..Do, caracterizando uma nova repetição.

Figura 10.18. Exemplo de código Assembly para divisão por subtrações sucessivas.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code> LDA dividendo ;este bloco: while (dividendo >= divisor).</code>
4	<code> SUB divisor</code>
5	<code> JN end</code>
6	<code>do:</code>
7	<code> LDA dividendo ;este bloco: dividendo = dividendo - divisor.</code>
8	<code> SUB divisor</code>
9	<code> STA dividendo</code>
10	
11	<code> MOV 1 ;este bloco: quociente++.</code>
12	<code> ADD quociente</code>
13	<code> STA quociente</code>
14	
15	<code> JMP while ;segue para o inicio da proxima repecticao.</code>
16	
17	<code>end:</code>
18	<code> INT exit</code>
19	
20	<code>.data</code>
21	<code> ;syscall exit</code>
22	<code>exit: DD 25</code>
23	<code>dividendo: DD 12</code>
24	<code>divisor: DD 3</code>
25	<code>quociente: DD 0</code>
26	
27	<code>.stack 1</code>

Fonte: Próprio autor (2021).

10.3.3 Exemplo 3: Elemento da Sequência de Fibonacci

A sequência de Fibonacci é uma sequência numérica, em que os dois primeiros elementos da sequência são 1 e 1, bem como o cálculo de um elemento F_n da sequência é dado pela soma dos elementos F_{n-1} e F_{n-2} , ou, em termos formais: $F_n = F_{n-1} + F_{n-2}$. Assim, a sequência de Fibonacci inicia da seguinte maneira:

1o elemento: 1
 2o elemento: 1
 3o elemento: $1 + 1 = 2$
 4o elemento: $2 + 1 = 3$
 5o elemento: $3 + 2 = 5$
 6o elemento: $5 + 3 = 8$
 7o elemento: $8 + 5 = 13$
 8o elemento: $13 + 8 = 21$
 9o. elemento: $21 + 13 = 34$
 ...

A Figura 10.19 apresenta um programa em linguagem C para o cálculo de um determinado elemento da sequência de Fibonacci.

Figura 10.19. Exemplo de código C para calcular um elemento da sequência de Fibonacci.

Linha	Código
1	<code>int resultado;</code>
2	<code>int n = 6;</code>
3	<code>int ultimo = 1;</code>
4	<code>int penultimo = 1;</code>
5	<code>int atual;</code>
6	<code>int i;</code>
7	
8	<code>if (n == 1)</code>
9	<code> resultado = penultimo;</code>
10	<code>else</code>
11	<code> if (n == 2)</code>
12	<code> resultado = ultimo;</code>
13	<code> else{</code>
14	<code> atual = 0;</code>
15	<code> for(i = 2; i < n; i++){</code>
16	<code> atual = ultimo + penultimo;</code>
17	<code> penultimo = ultimo;</code>
18	<code> ultimo = atual;</code>
19	<code> }</code>
20	<code> resultado = atual;</code>
21	<code> }</code>

Fonte: Próprio autor (2021).

Na Figura 10.19, a variável “n” informa qual é a ordem do elemento a ser computado na sequência de Fibonacci. Na figura, as Linhas 8 à 12 verificam se a variável “n” contém os valores 1 ou 2, referindo-se ao 1o ou 2o elementos da sequência de Fibonacci, e, em caso afirmativo, é atribuído o valor 1 à variável “resultado”, encerrando o programa. Caso contrário, a estrutura de repetição For..Do, nas linhas 15 à 19, à cada repetição, calcula o próximo elemento da sequência até atingir o elemento do número de ordem informado na variável “n”. Ao final das repetições, a variável “ultimo” conterá o elemento da ordem informada na variável “n”, o qual será atribuído à variável “resultado”, na Linha 20.

O código na Figura 10.20 apresenta uma implementação em *Assembly* do código C para o cálculo de um determinado elemento da sequência de Fibonacci da Figura 10.19. Na Figura 10.20: as Linhas 3 à 5 e 8 à 10 verificam se o número de ordem informado consiste em 1 ou 2, respectivamente, referindo-se ao 1o ou 2o elementos da sequência de Fibonacci, e, em caso afirmativo, o fluxo do programa é desviado para o bloco “resultado_1” (Linhas 38 e 39), onde atribui-se o valor 1 à variável “resultado”, encerrando o programa logo em seguida, na instrução da Linha 42. A estrutura de repetição For..Do, presente nas Linhas 13 à 30, a cada repetição, calcula o próximo elemento da sequência de Fibonacci, até atingir o número de ordem informado na variável “n”. Ao final das repetições, o fluxo de execução do programa é desviado para o bloco “fim_for”, nas Linhas 33 à 35, onde atribui-se o valor da variável “ultimo” à variável “resultado”. Em seguida, o fluxo de execução do programa é desviado para o bloco “end”, para encerramento do programa.

Figura 10.20. Exemplo de código Assembly para calcular um elemento da sequência de Fibonacci.

Linha	Código
1	<code>.code</code>
2	<code>if:</code>
3	<code> MOV 1 ;este bloco: if (n == 1).</code>
4	<code> SUB n</code>
5	<code> JZ resultado_1</code>
6	
7	<code>else_if:</code>
8	<code> MOV 2 ;este bloco: else if (n == 2).</code>
9	<code> SUB n</code>
10	<code> JZ resultado_1</code>
11	
12	<code>else_for:</code>
13	<code> LDA n ;este bloco: for(i = 2; i < n; i++).</code>
14	<code> SUB i</code>
15	<code> JZ fim_for</code>
16	<code> MOV 1</code>
17	<code> ADD i</code>
18	<code> STA i</code>
19	<code>do:</code>
20	<code> LDA ultimo ;este bloco: atual = ultimo + penultimo.</code>

21	ADD penultimo
22	STA atual
23	
24	LDA ultimo <i>;este bloco: penultimo = ultimo.</i>
25	STA penultimo
26	
27	LDA atual <i>;este bloco: ultimo = atual.</i>
28	STA ultimo
29	
30	JMP else_for
31	
32	fim_for:
33	LDA atual
34	STA resultado
35	JMP end
36	
37	resultado_1:
38	MOV 1
39	STA resultado
40	
41	end:
42	INT exit
43	
44	.data
45	<i>;syscall exit</i>
46	exit: DD 25
47	resultado: DD 0
48	n: DD 6
49	ultimo: DD 1
50	penultimo: DD 1
51	atual: DD 0
52	i: DD 2
53	
54	.stack 1

Fonte: Próprio autor (2021).

10.4 Resumo

Este capítulo iniciou com o conceito de controle de fluxo de execução, sendo descritos os tipos de estruturas de controle de fluxo de execução implementados na maioria das linguagens de programação. As estruturas de controle são divididas entre estruturas condicionais e de repetição. As estruturas condicionais podem ser dos tipos: Simples (If..Then), Composta (If..Else) e De Múltiplos Casos (Switch..Case). Já as estruturas de repetição podem ser dos tipos de: Variável de Controle (For..Do), Teste no Início (While..Do) e Teste no Fim (Do..While).

O capítulo seguiu com exemplos de programas em C para ilustrar o uso de cada um desses tipos de estrutura de controle de fluxo de execução, e como essas estruturas podem ser implementadas em linguagem *Assembly*, utilizando as instruções JMP, JZ e JN do processador Cariri.

Capítulo 10 - Controle de Fluxo de Execução do Programa

Ao final, foram apresentados três exemplos práticos de uso das estruturas de controle de fluxo de execução para compor as operações aritméticas de multiplicação e divisão inteira, bem como para o cálculo de determinado elemento da sequência de Fibonacci.

CAPÍTULO 11 – MÉTODOS AVANÇADOS DE ACESSO À MEMÓRIA

O computador foi criado para realizar processamento de dados. Viu-se, em capítulos anteriores, que o processador possui recursos que permitem realizar o acesso e o processamento de dados armazenados em memória.

Dependendo do conjunto de dados e do modelo de processamento da aplicação, pode ser necessário utilizar mecanismos adicionais para ampliar as formas de acesso ao conjunto de dados e aos seus elementos individualmente, para, desta forma, reduzir a complexidade e otimizar a aplicação.

Este capítulo apresenta um estudo mais aprofundado sobre o acesso a dados em memória. No estudo, será visto: o conceito de ponteiro; como ele pode ser utilizado para criar rotinas para manipulação de estruturas de dados; utilizar a pilha do programa como estrutura de dados auxiliar; alocação dinâmica de memória; e relocação. Ao final do capítulo, será apresentado um sumário das instruções do processador Cariri e quais os modos de endereçamento suportados por cada uma delas.

11.1 Ponteiros

Um “Ponteiro”, ou “Apontador”, pode ser definido, basicamente, como uma variável que armazena um endereço de memória (SCHILDT, 1997). Um ponteiro armazena números inteiros sem sinal com precisão suficiente para acesso a todos os endereços de memória do espaço de endereçamento do processador (SANTOS, P. R.; LANGLOIS, 2018). Os ponteiros podem ser utilizados em diversas situações, tais como: referenciar variáveis, provendo um caminho alternativo para acesso aos dados das variáveis; acesso aos elementos de estruturas de dados com armazenamento contínuo em memória, como vetores e arrays; acesso aos

elementos de estruturas de dados não alocadas em endereços contíguos de memória, tais como árvores e grafos; passagem e retorno de parâmetros por referência em funções; acesso a blocos de memória alocados dinamicamente; e referenciar funções, de forma que elas podem ser chamadas de maneira genérica para execução, como para implementação de *callbacks* (funções que são passadas como parâmetros em funções).

O código em linguagem C da Figura 11.1 ilustra, na Linha 3, como declarar um ponteiro, variável “ptr”, que recebe o endereço de memória da variável “var1” (o endereço é obtido pelo uso do operador de endereço “&”); e, na Linha 5, com o uso do ponteiro e do operador de desreferência “*”, realiza-se a atribuição do valor em “var1” à variável “var2”.

Figura 11.1 Exemplo de código C para ilustrar o uso de ponteiro para leitura de dados.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2 = 0;</code>
3	<code>int *ptr = &var;</code>
4	
5	<code>var2 = *ptr;</code>

Fonte: Próprio autor (2021).

Na Figura 11.1, ao analisar o comportamento do ponteiro, pode-se perceber que, para acesso a um dado apontado por um endereço de memória, o ponteiro deve realizar dois acessos à memória: sendo que o primeiro consiste em recuperar o endereço de memória que aponta para o dado; e, o segundo, a partir do endereço lido, realizar o acesso ao dado. Com isso, é possível afirmar que os ponteiros são implementados com o modo de endereçamento indireto (ver [Capítulo 5 - Introdução à Arquitetura de Computadores](#)).

O processador Cariri do simulador CompSim inclui duas instruções que implementam o modo de endereçamento indireto em memória, sendo uma para leitura de dados (“LDI”) e outra para escrita de dados (“STI”). Os quadros a seguir trazem descrições de cada uma dessas instruções, apresentando a seção do código onde podem ser utilizadas, sua sintaxe e exemplos de uso.

Instrução: LDI (Load Indirect)	
Descrição: Leitura de um endereço de memória a partir de uma determinada posição de memória, para, seguida, realizar a leitura de um dado. Após a segunda leitura, o dado será armazenado no registrador acumulador (AC).	
Seção: .code	
Sintaxe: [<rotulo>:] LDI @<endereço> [<rotulo>:] LDI <variável>	
Exemplos: LDI @100 ;realiza leitura de um endereço no endereço 100 para leitura do dado da memória. LDI var1 ;realiza leitura de um endereço na variável var1 para leitura do dado da memória. Le_End: LDI @100 ;instrução com rotulo para leitura de um endereço no endereço 100 para leitura do ; dado da memória. Le_Var: LDI var1 ;instrução, com rotulo em linha separada, para leitura de um endereço na variável var1 ; para leitura do dado da memória.	

Instrução: STI (Save To Indirect)	
Descrição: Leitura de um endereço de memória a partir de uma determinada posição de memória, para gravação de um dado armazenado no registrador acumulador (AC), no novo endereço.	
Seção: .code	
Sintaxe: [<rotulo>:] STI @<endereço> [<rotulo>:] STI <variável>	
Exemplos: STI @100 ;realiza leitura de um endereço no endereço 100 para gravação do dado em AC. STI var1 ;realiza leitura de um endereço na variável var1 para gravação do dado em AC. Salva_End: STI @100 ;instrução com rotulo para leitura de um endereço no endereço 100 onde será ; gravação o dado em AC. Salva_Var: STI var1 ;instrução, com rotulo em linha separada, para leitura de um endereço na variável var1 ; onde será gravado o dado em AC.	

A Figura 11.2 ilustra um programa *Assembly* que implementa o programa C da Figura 11.1, o qual utiliza um ponteiro para fazer a leitura de um dado contido na variável “var1”. Na Figura 11.2, nas Linhas 11 e 12, são declaradas as variáveis “var1” e “var2”. Na Linha 13, observa-se que a variável “ptr” recebe a variável “var1” como parâmetro de inicialização. O montador, durante a tradução do código *Assembly* para código em linguagem de máquina, fará com que a variável “ptr” receba o endereço de memória referente à variável “var1”. Para leitura do dado contido na variável “var1”, utilizando o ponteiro “ptr”, deve-se utilizar a instrução LDI, como pode ser visto na Linha 2. A instrução LDI realiza, no primeiro momento, a leitura do endereço contido em “ptr” e, em seguida, com o novo endereço realiza

uma nova leitura na memória para recuperar o operando, que, por sua vez, será armazenado no registrador acumulador (AC). Por fim, na Linha 3, o dado contido em AC será gravado na variável “var2”.

Figura 11.2. Exemplo de código Assembly para ilustrar o uso de ponteiro para leitura de dados.

Linha	Código
1	<code>.code</code>
2	<code>LDI ptr ;este bloco realiza: var2 = *ptr.</code>
3	<code>STA var2</code>
4	
5	<code>end:</code>
6	<code>INT exit</code>
7	
8	<code>.data</code>
9	<code>;syscall exit</code>
10	<code>exit: DD 25</code>
11	<code>var1: DD 10</code>
12	<code>var2: DD 0</code>
13	<code>ptr: DD var1</code>
14	
15	<code>.stack 1</code>

Fonte: Próprio autor (2021).

O código em linguagem C na Figura 11.3, ilustra como declarar um ponteiro (variável “ptr”) apontando para a variável “var” (Linha 2) e, com o uso do ponteiro, atribuir o valor 20 à variável “var” (Linha 4).

Figura 11.3. Exemplo de código C para ilustrar o uso de ponteiro para gravação de dados.

Linha	Código
1	<code>int var = 10;</code>
2	<code>int *ptr = &var;</code>
3	
4	<code>*ptr = 20;</code>

Fonte: Próprio autor (2021).

A Figura 11.4 ilustra um programa *Assembly* que implementa o programa C da Figura 11.3, o qual utiliza um ponteiro para atribuir um dado à variável “var”. Na Figura 11.4, na Linha 12, a declaração do ponteiro “ptr” faz com que ele já aponte para a variável “var”. Na Linha 2, atribui-se o valor 20 ao registrador acumulador (AC) e, na Linha 3, com o uso da instrução STI, o valor em AC é gravado no endereço de memória apontado pelo ponteiro “ptr”, que, neste caso, o dado será gravado no endereço de memória referente à “var”.

Figura 11.4. Exemplo de código Assembly para ilustrar o uso de ponteiro para gravação de dados.

Linha	Código
1	<code>.code</code>
2	<code>MOV 20 ;este bloco realiza: *ptr = 20.</code>
3	<code>STI ptr</code>
4	
5	<code>end:</code>
6	<code>INT exit</code>
7	
8	<code>.data</code>
9	<code>;syscall exit</code>
10	<code>exit: DD 25</code>
11	<code>var: DD 10</code>
12	<code>ptr: DD var</code>
13	
14	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Durante a execução de um programa é possível fazer com que um ponteiro passe a apontar para uma outra variável. O código em linguagem C na Figura 11.5, ilustra como declarar um ponteiro (variável “ptr”) apontando para a variável “var1” (Linha 3); como alterar a referência do ponteiro “ptr” para apontar para a variável “var2” (Linha 5) e, por fim, com o uso do ponteiro, atribuir o valor 20 à variável “var2” (Linha 6).

Figura 11.5. Exemplo de código C para ilustrar a alteração de referência de um ponteiro.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2 = 10;</code>
3	<code>int *ptr = &var1;</code>
4	
5	<code>ptr = &var2;</code>
6	<code>*ptr = 20;</code>

Fonte: Próprio autor (2021).

A Figura 11.6 ilustra um programa *Assembly* que implementa o programa C da Figura 11.5, o qual atualiza a referência de um ponteiro, que antes apontava para a variável “var1”, para atribuir um dado à variável “var2”. Na Figura 11.6, na Linha 16, a declaração do ponteiro “ptr” faz com que ele já aponte para a variável “var1”. Na Linha 2, a instrução MOV atribui o endereço de memória referente à variável “var2” ao registrador acumulador (AC) e, na Linha 3, com o uso da instrução STA, o endereço contido em AC é atribuído ao ponteiro “ptr”. Com isso, o ponteiro “ptr” passa a apontar para a variável “var2”; e, nas Linhas 5 e 6, atribui-se o valor 20 à variável “var2”, por meio de “ptr”.

Figura 11.6. Exemplo de código Assembly para ilustrar a alteração de referência de um ponteiro.

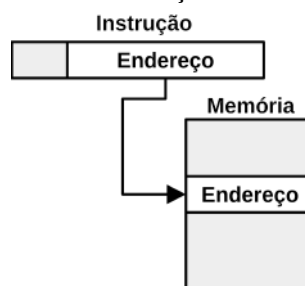
Linha	Código
1	<code>.code</code>
2	<code>MOV var2 ;este bloco realiza: ptr = &var2.</code>
3	<code>STA ptr</code>
4	
5	<code>MOV 20 ;este bloco realiza: *ptr = 20.</code>
6	<code>STI ptr</code>
7	
8	<code>end:</code>
9	<code>INT exit</code>
10	
11	<code>.data</code>
12	<code>;syscall exit</code>
13	<code>exit: DD 25</code>
14	<code>var1: DD 10</code>
15	<code>var2: DD 10</code>
16	<code>ptr: DD var1</code>
17	
18	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Nos exemplos anteriores, o ponteiro “ptr” foi utilizado com as instruções de escrita em memória que implementa o modo endereçamento direto (STA) e instruções de leitura/escrita com modo de endereçamento indireto (LDI e STI).

Ao utilizar instruções com modo de endereçamento direto, tais como LDA e STA, com operando do tipo ponteiro, faz-se acesso direto ao conteúdo do ponteiro, que consiste de um endereço de memória, como mostra a Figura 11.7. Por exemplo, ao utilizar a instrução LDA com um ponteiro, será trazido o endereço apontado pelo ponteiro para o registrador acumulador (AC). Da mesma forma, ao utilizar a instrução STA com ponteiro, será gravado o conteúdo de AC como sendo o novo endereço contido no ponteiro.

Figura 11.7. Acesso com endereçamento direto em ponteiros.

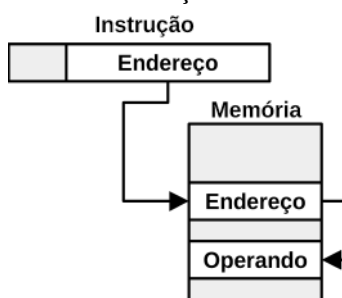


Fonte: Próprio autor (2021).

Já as instruções LDI e STI devem ser utilizadas exclusivamente com operando do tipo ponteiro, para realizar acesso indireto de leitura e escrita, respectivamente, ao operando apontado pelo endereço contido no ponteiro, como mostra a Figura 11.8. Ao utilizar, por exemplo, a instrução LDI com operando do tipo ponteiro, serão realizados dois acessos à

memória: no primeiro, será buscado o endereço de memória apontado pelo ponteiro; e, no segundo, com o endereço lido no passo anterior, faz-se um novo acesso à memória para leitura do operando. Da mesma forma, ao utilizar a instrução STI com operando do tipo ponteiro, inicialmente, será buscado o endereço de memória apontado pelo ponteiro, e, em seguida, será gravado o conteúdo de AC no endereço de memória lido anteriormente.

Figura 11.8. Acesso com endereçamento indireto em ponteiros.



Fonte: Próprio autor (2021).

Em resumo, as instruções que implementam endereçamento direto (LDA e STA), manipulam o endereço de memória contido no ponteiro, enquanto que as instruções LDI e STI, que implementam endereçamento indireto, manipulam o operando apontado pelo ponteiro.

11.2 Operações em Estruturas de Dados e Strings

Ponteiros podem ser utilizados também para auxiliar no acesso aos elementos de estruturas de dados com armazenamento contínuo em memória, tais como vetores/arrays, e cadeia de caracteres (*strings*). A seguir serão apresentados exemplos de aplicações que fazem uso de ponteiros para manipulação de estruturas de dados e de *strings*.

O código em linguagem C na Figura 11.9, traz uma aplicação que realiza a soma dos números inteiros contidos em um array. No código, para o acesso aos elementos do array, utiliza-se o ponteiro “ptr”, declarado na Linha 2 e inicializado com o endereço do primeiro elemento do array “a”. Nas Linhas 6 à 9, há uma estrutura de repetição For..Do, na qual, a cada repetição: realiza-se a adição de um dos elementos do array à variável “soma”, cujo elemento é acessado pelo uso do ponteiro “ptr” e do operador de desreferenciamento; e, após a adição, o ponteiro é incrementado para apontar para o próximo elemento do array, como pode ser visto na Linha 8.

Figura 11.9. Exemplo de código C para ilustrar o acesso aos elementos de um array com uso de ponteiro.

Linha	Código
1	<code>int a[5] = {1,2,3,4,5};</code>
2	<code>int *ptr = a; //equivalente: int *ptr = &a[0].</code>
3	<code>int soma = 0;</code>
4	<code>int i;</code>
5	
6	<code>for (i=0; i<5; i++){</code>
7	<code> soma += *ptr;</code>
8	<code> ptr++; //ptr apontara para o proximo elemento do array.</code>
9	<code>}</code>

Fonte: Próprio autor (2021).

A Figura 11.10 ilustra um programa *Assembly* que implementa o programa C da Figura 11.9, o qual realiza a soma dos elementos de um array de números inteiros, utilizando um ponteiro para referenciar cada elemento.

Figura 11.10. Exemplo de código Assembly para ilustrar o acesso aos elementos de um array com uso de ponteiro.

Linha	Código
1	<code>.code</code>
2	<code>for:</code>
3	<code> MOV 5 ;este bloco realiza: for (i=0; i<5; i++).</code>
4	<code> SUB i</code>
5	<code> JZ end</code>
6	<code> MOV 1</code>
7	<code> ADD i</code>
8	<code> STA i</code>
9	<code>do:</code>
10	<code> LDI ptr ;este bloco realiza: soma += *ptr.</code>
11	<code> ADD soma</code>
12	<code> STA soma</code>
13	
14	<code> MOV 1 ;este bloco realiza: ptr++.</code>
15	<code> ADD ptr</code>
16	<code> STA ptr</code>
17	
18	<code> JMP for</code>
19	
20	<code>end:</code>
21	<code> INT exit</code>
22	
23	<code>.data</code>
24	<code> ;syscall exit</code>
25	<code>exit: DD 25</code>
26	<code>a: INITD 1, 2, 3, 4, 5</code>
27	<code>ptr: DD a</code>
28	<code>soma: DD 0</code>
29	<code>i: DD 0</code>
30	
	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Na Figura 11.10, as Linhas 3 à 8 incluem a configuração, condição de repetição e incremento da variável de controle “i”, da estrutura de repetição For..Do. As Linhas 10, 11 e 12, fazem a leitura de um elemento do array, pelo uso da instrução LDI e do ponteiro “ptr”, a soma do elemento lido com o conteúdo da variável “soma” e a gravação do resultado na variável “soma”, respectivamente. Nas linhas 14 à 16, realiza-se o incremento do endereço de memória na variável “ptr” para que aponte para o próximo elemento do array. Na Linha 18, a instrução JMP faz com que o fluxo de execução seja desviado para iniciar uma nova repetição.

O código em linguagem C na Figura 11.11, traz uma aplicação que realiza a cópia de uma cadeia de caracteres (*string*). No código, nas Linhas 1 e 2, são definidos dois arrays do tipo *char* (caractere), sendo que o primeiro array (“str1”) é inicializado com a *string* “hello”. O objetivo aqui será copiar a *string* contida no array “str1” para o array “str2”. Para tanto, cria-se, nas Linhas 3 e 4, os ponteiros “ptr1”, que aponta para o início do array “str1”, e “ptr2”, que aponta para o início do array “str2”.

Figura 11.11. Exemplo de código C para ilustrar cópia de cadeia de caracteres.

Linha	Código
1	<code>char str1[6] = "hello";</code>
2	<code>char str2[6];</code>
3	<code>char *ptr1 = str1;</code>
4	<code>char *ptr2 = str2;</code>
5	
6	<code>while (*ptr1 != '\0'){</code>
7	<code> *ptr2 = *ptr1;</code>
8	<code> ptr1++; //ptr1 apontara para o proximo elemento do array.</code>
9	<code> ptr2++; //ptr2 apontara para o proximo elemento do array.</code>
10	<code>}</code>
11	<code>*ptr2 = '\0'; //adicionar o caractere de fim de string em str2.</code>

Fonte: Próprio autor (2021).

Para compreensão do restante do código C na Figura 11.11, é importante lembrar que a linguagem C, na definição de *strings*, acrescenta ao fim da *string* o caractere nulo ‘\0’ (também conhecido por caractere “NUL” ou “null”). Esse caractere é geralmente utilizado para informar o fim de uma *string* e, com isso, simplifica a criação de funções de manipulação de *strings*, tais como para cópia, pesquisa e comparação. Observando a definição dos arrays “str1” e “str2”, na Figura 11.11, percebe-se que, por mais que a *string* “hello” contenha apenas 5 caracteres, os arrays foram definidos com 6 posições, já prevendo o uso do caractere nulo, que é adicionado, implicitamente, pelo compilador, ao fim da *string*.

Nas Linhas 6 à 10, do código C na Figura 11.11, observa-se que: na Linha 6, a condição de repetição da estrutura de repetição `While..Do` é satisfeita enquanto o caractere apontado pelo ponteiro “ptr1” não for igual ao caractere nulo ‘\0’, ou, em outras palavras, enquanto “ptr1” não apontar para o fim da *string* em “str1”; na Linha 7, o conteúdo da posição do array “str1”, apontado pelo ponteiro “ptr1”, é copiado para a respectiva posição do array “str2”, apontada por “ptr2”; e, as Linhas 8 e 9, realizam o incremento dos endereços de memória contidos nos ponteiros “ptr1” e “ptr2”, respectivamente, fazendo-os apontarem para as próximas posições dos respectivos arrays. Ao fim da execução da estrutura de repetição `While..Do`, que é dado quando o ponteiro “ptr1” aponta para o caractere nulo ao fim da *string* “hello”, a Linha 11 adiciona o caractere nulo ao fim da *string* copiada no array “str2”, de forma a garantir que ela também conterá o caractere de fim de *string*.

O código na Figura 11.12 ilustra um programa *Assembly* que implementa o programa C da Figura 11.11, o qual realiza a cópia de uma *string*. Na Figura 11.12, a Linha 30 apresenta a definição do array “str1”, inicializado com a *string* “hello” e o caractere de fim de *string* ‘0’. O array “str2” é declarado na seção “.bss”, na Linha 36. Os ponteiros “ptr1” e “ptr2” são definidos na Linhas 31 e 32 para apontarem para os arrays “str1” e “str2”, respectivamente.

Ainda na Figura 11.12, observa-se que: as Linhas 3 à 5 implementam a condição de repetição da estrutura de repetição `While..Do`, que é atendida enquanto o conteúdo apontado por “ptr1” não for igual ao conteúdo da variável “fim_str” (que contém o inteiro 0), ou, em outras palavras, enquanto “ptr1” não apontar para o fim da *string* em “str1”; as Linhas 7 e 8 realizam, através do uso das instruções `LDI` e `STI`, a cópia do operando apontado pelo endereço em “ptr1” para o local de memória apontado pelo endereço em “ptr2”; as Linhas 10 à 12 e Linhas 14 à 16 realizam o incremento dos endereços de memória contidos nos ponteiros “ptr1” e “ptr2”, respectivamente; na Linha 18, a instrução `JMP` faz com que o fluxo de execução seja desviado para iniciar uma nova repetição; por fim, caso a condição de repetição, nas Linhas 3 à 5, não mais seja atendida, ou seja, “ptr1” passou a apontar para o caractere de fim de *string* em “str1”, haverá um desvio de fluxo de execução para o bloco em “pos_while”, o qual atribui o caractere de fim de *string* à última posição do array “str2”, utilizando o ponteiro “ptr2” (Linhas 21 e 22).

Figura 11.12. Exemplo de código Assembly para ilustrar cópia de cadeia de caracteres.

Linha	Código
1	<code>.code</code>
2	<code>c</code>
3	<code>LDI ptr1 ;este bloco realiza: while (*ptr1 != '\0').</code>
4	<code>SUB fim_str</code>
5	<code>JZ pos_while</code>
6	<code>do:</code>
7	<code>LDI ptr1 ;este bloco realiza: *ptr2 = *ptr1.</code>
8	<code>STI ptr2</code>
9	
10	<code>MOV 1 ;este bloco realiza: ptr1++.</code>
11	<code>ADD ptr1</code>
12	<code>STA ptr1</code>
13	
14	<code>MOV 1 ;este bloco realiza: ptr2++.</code>
15	<code>ADD ptr2</code>
16	<code>STA ptr2</code>
17	
18	<code>JMP while</code>
19	
20	<code>pos_while:</code>
21	<code>LDA fim_str</code>
22	<code>STI ptr2</code>
23	
24	<code>end:</code>
25	<code>INT exit</code>
26	
27	<code>.data</code>
28	<code>;syscall exit</code>
29	<code>exit: DD 25</code>
30	<code>str1: INITB "hello",0</code>
31	<code>ptr1: DD str1</code>
32	<code>ptr2: DD str2</code>
33	<code>fim_str: DD 0</code>
34	
35	<code>.bss</code>
36	<code>str2: RESB 6</code>
37	
38	<code>.stack 1</code>

O último exemplo desta subseção apresenta um programa C para uma aplicação de pesquisa de dados. A aplicação consiste na busca de um determinado caractere em uma determinada *string*, com auxílio de ponteiro. Na Figura 11.13, na Linha 1, é definido o array “str” que contém a *string* “Hello, World!”, na qual será pesquisado um caractere; na Linha 2, é criado o ponteiro “ptr” que aponta para a primeira posição do array “str”; na linha 3, é definida a variável “caractere”, a qual contém o caractere “W”, que será pesquisado na *string*; na Linha 4, a variável “contador” será utilizada para informar a posição da *string* que contém o caractere pesquisado, caso ele seja encontrado; as Linhas 6 à 11 incluem uma estrutura de repetição While..Do, na qual, nas Linhas 7 e 8, há uma estrutura condicional If..Then, que verifica se o caractere da *string* atualmente apontado por “ptr” é igual ao caractere pesquisado; em que, em caso afirmativo, a estrutura de repetição será encerrada e a variável

“contador” informará a posição na *string* onde o caractere foi encontrado; e, caso contrário, segue a execução das linhas 9 e 10, as quais realizam o incremento do valor da variável “contador” e do endereço de memória em “ptr”, respectivamente, para seguir a pesquisa no próximo caractere da *string*.

Figura 11.13. Exemplo de código C para ilustrar a pesquisa de um caractere em uma string.

Linha	Código
1	<code>char str[14] = "Hello, World!";</code>
2	<code>char *ptr = str;</code>
3	<code>char caractere = 'W';</code>
4	<code>int contador = 0;</code>
5	
6	<code>while (*ptr != '\0'){</code>
7	<code> if (*ptr == caractere)</code>
8	<code> break;</code>
9	<code> contador++;</code>
10	<code> ptr++;</code>
11	<code>}</code>

Fonte: Próprio autor (2021).

O código na Figura 11.14 ilustra um programa *Assembly* que implementa o programa C da Figura 11.13, o qual realiza a pesquisa de um determinado caractere em uma dada *string*. Na Figura 11.14, a Linha 27 apresenta a definição do array “str”, inicializado com a *string* “Hello, World!” e o caractere de fim de string ‘0’. O ponteiro “ptr” é definido na Linha 28, apontando para o array “str”. Nas Linhas 29, 30 e 31 são definidas as variáveis “caractere”, que contém o caractere “W”, que será pesquisado na *string*, “contador”, que informa a posição do caractere na *string*, caso seja encontrado na pesquisa, e “fim_str”, que contém o valor de fim de string que é utilizada pela estrutura de repetição While..Do, nas Linhas 3 à 5, como condição de parada. As Linhas 7 à 9 implementam a estrutura condicional If..Then, que verifica se o caractere da *string*, atualmente apontado por “ptr”, é igual ao caractere pesquisado, contido na variável “caractere”, e, em caso afirmativo, será desviado o fluxo de execução para o bloco “end”, encerrando a execução do programa. Caso contrário, as Linhas 11 à 13 e Linhas 15 à 17 incrementam o valor contido em “contador” e o endereço de memória em “ptr”, seguindo a pesquisa para o próximo caractere da *string*, em uma nova repetição, através da instrução JMP na Linha 19.

Figura 11.14. Exemplo de código Assembly para ilustrar a pesquisa de um caractere em uma string.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code>LDI ptr ;este bloco realiza: while (*ptr != '\0').</code>
4	<code>SUB fim_str</code>
5	<code>JZ end</code>
6	<code>do_if:</code>
7	<code>LDI ptr ;este bloco realiza: if (*ptr == caractere) break.</code>
8	<code>SUB caractere</code>
9	<code>JZ end</code>
10	<code>then:</code>
11	<code>MOV 1 ;este bloco realiza: contador++.</code>
12	<code>ADD contador</code>
13	<code>STA contador</code>
14	
15	<code>MOV 1 ;este bloco realiza: ptr++.</code>
16	<code>ADD ptr</code>
17	<code>STA ptr</code>
18	
19	<code>JMP while</code>
20	
21	<code>end:</code>
22	<code>INT exit</code>
23	
24	<code>.data</code>
25	<code>;syscall exit</code>
26	<code>exit: DD 25</code>
27	<code>str: INITB "Hello, World!",0</code>
28	<code>ptr: DD str</code>
29	<code>caractere: DB 'W'</code>
30	<code>contador: DD 0</code>
31	<code>fim_str: DD 0</code>
32	
33	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Nesta subseção, viu-se que os ponteiros são bastante úteis para auxiliar o acesso aos elementos de estruturas de dados e de *strings*. Com o suporte de ponteiros, é possível implementar outros tipos de estruturas de dados, tais como listas, filas, registros, árvores e grafos, e diferentes operações sobre estruturas de dados e *strings*, tais como pesquisa, ordenação, adição e concatenação.

11.3 Pilha do Programa

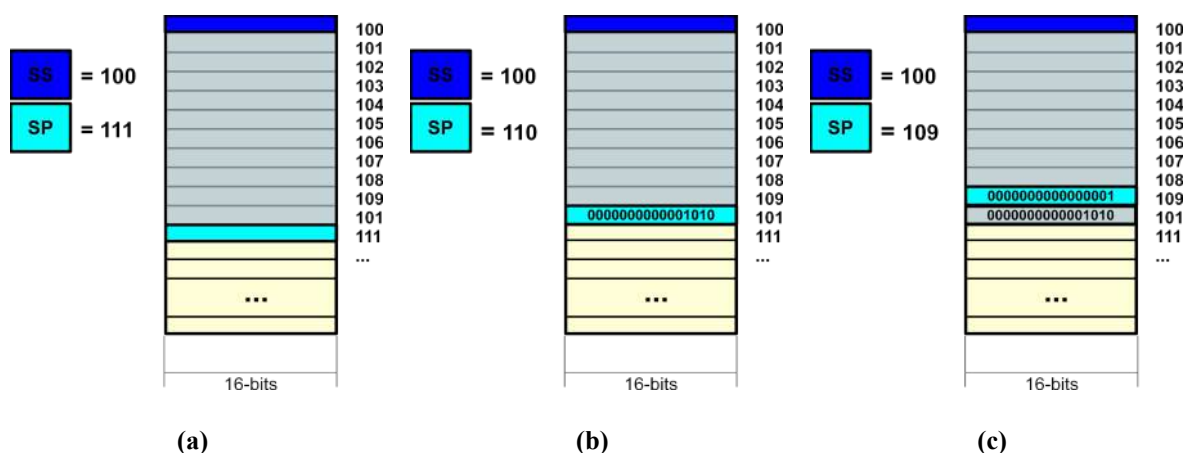
A pilha do programa é uma estrutura de dados, do tipo LIFO (*Last-In, First-Out*, ou Último a Entrar, Primeiro a Sair) (STALLINGS, 2010), que pode ser utilizada para armazenamento de dados e/ou suporte a diversas operações na execução dos programas, tais como para gerenciar a sequência de chamadas de funções, passagem de parâmetros de entrada e de retorno em funções e implementação de variáveis locais. Muitas arquiteturas

atuais, incluem registradores de propósito específico utilizados para endereçar os limites de uma região de memória alocada exclusivamente para uso da pilha do programa. Essa região de memória é comumente chamada de Segmento de Pilha, como visto no [Capítulo 8 - Manipulação de Dados em Memória](#).

Foi visto que o processador Cariri do simulador CompSim, inclui dois registradores de endereçamento ao segmento de pilha, que são: 1) Registrador de Segmento de Pilha (*Stack Segment - SS*), que armazena o endereço da primeira posição de memória alocada para o segmento de pilha; e 2) Registrador Apontador de Topo de Pilha (*Stack Pointer - SP*), que aponta, inicialmente, para uma posição de memória posterior ao final do bloco de memória reservado para a pilha.

Na Figura 11.15 (a), pode-se observar uma ilustração de um segmento de pilha, alocado com 10 posições de memória. O segmento inicia no endereço de memória 100 e encerra no endereço 110, visto que os registradores SS e SP possuem valores 100 e 111, respectivamente. Ao adicionar um elemento na pilha, como mostra a Figura 11.15 (b), o registrador de topo de pilha SP é decrementado e passa a apontar para o endereço 110, que é o endereço onde o novo dado é inserido. A Figura 11.15 (c) ilustra a adição de um novo dado na pilha, em que novamente o endereço no registrador SP é decrementado, para, em seguida, adição de um novo dado no topo da pilha do programa.

Figura 11.15. Ilustração do segmento de pilha de um programa em memória no simulador CompSim.



Fonte: Próprio autor (2021).

Viu-se então que, ao adicionar um elemento na pilha do programa, o endereço contido no registrador SP é decrementado. Portanto, ao remover um elemento, o processo é contrário: o registrador SP é incrementado.

É importante ressaltar ainda que o endereço no registrador SS aponta para o limite máximo de elementos que podem ser adicionados na pilha. Então, quando a pilha do programa estiver cheia, os registradores SS e SP conterão o mesmo endereço. Já quando a pilha do programa estiver vazia, SP conterá o endereço da posição de memória posterior à última posição alocada para a pilha.

Em algumas arquiteturas, há instruções para manipulação direta de dados na pilha do programa, como é o caso da arquitetura x86 da Intel, que inclui as instruções “PUSH”, para adicionar elemento na pilha, e “POP”, para remover elemento da pilha. Já em outros processadores, não há instruções para manipulação da pilha do programa. Neles, deve-se manipular diretamente os registradores de endereçamento de pilha, como é o caso da arquitetura MIPS (PATTERSON; HENNESSY, 2017).

No processador Cariri há uma única instrução para adição e remoção de elementos na pilha do programa, que é a instrução “SOP” (*Stack Operations*). O quadro a seguir traz um resumo com uma descrição da instrução SOP, em qual seção do código pode ser utilizada, sua sintaxe de uso e exemplos de como utilizá-la.

Instrução: SOP (Stack Operations)	
Descrição: Realiza as operações PUSH (adição) ou POP (remoção) de elementos para/da pilha do programa. Para tanto, o operando da instrução contém um endereço de memória, cujo respectivo valor armazenado indicará a operação a ser realizada: 1) um valor 0 indica a operação PUSH: decrementa o endereço no registrador SP e copia o dado contido no registrador acumulador (AC) para o novo endereço apontado por SP; 2) um valor 1 indica a operação POP: copia o elemento apontado por SP para o registrador AC, e, em seguida, incrementa o endereço contido em SP.	
Seção: .code	
Sintaxe: [<rotulo>:] SOP @<endereço> [<rotulo>:] SOP <variável>	
Flags afetadas do SFR (<i>Status Flags Register</i>): Z (<i>Zero</i>), S (<i>Signal</i>) e O (<i>Overflow</i>).	
Exemplos: SOP @100 ;realiza uma operação na pilha, dependendo do operando no endereço 100. SOP var1 ;realiza uma operação na pilha, dependendo do operando na variável var1. SHIFT_100: SOP @100 ;instrucao com rotulo para operação em pilha, dependendo do operando no endereço 100. SHIFT_Var1: SOP var1 ;instrucao, com rotulo em linha separada, para operação na pilha, dependendo do operando na variável var1.	

A Figura 11.16 traz um exemplo básico de uso da instrução SOP. Nas Linhas 16 e 17, são definidas as variáveis “push”, inicializada com o valor 0, e “pop”, com valor 1. Essas

duas variáveis serão utilizadas como parâmetros da instrução SOP, para adicionar e remover elementos da pilha do programa, respectivamente. Nas Linhas 2 e 3, adiciona-se 10 ao registrador acumulador AC e, em seguida, esse valor é adicionado no topo da pilha com a instrução SOP, cujo operando é a variável “push” (que armazena o valor 0 em memória). Nas Linhas 5 e 6, adiciona-se também o valor 20 ao topo da pilha do programa. Por fim, na linha 8, remove-se o valor do topo da pilha, que é copiado para AC, através da instrução SOP, que tem como operando a variável “pop” (que armazena o valor 1 em memória).

Figura 11.16. Exemplo de código Assembly para ilustrar o uso da instrução SOP.

Linha	Código
1	<code>.code</code>
2	<code>MOV 10 ;este bloco: adiciona 10 ao topo da pilha.</code>
3	<code>SOP push</code>
4	
5	<code>MOV 20 ;este bloco: adiciona 20 ao topo da pilha.</code>
6	<code>SOP push</code>
7	
8	<code>SOP pop ;remove o valor do top da pilha para AC.</code>
9	
10	<code>end:</code>
11	<code>INT exit</code>
12	
13	<code>.data</code>
14	<code>;syscall exit</code>
15	<code>exit: DD 25</code>
16	<code>push: DD 0</code>
17	<code>pop: DD 1</code>
18	
19	<code>.stack 10</code>

Fonte: Próprio autor (2021).

Na Figura 11.17, é possível visualizar uma abstração da pilha do programa, através do componente gráfico “STACK”, na aba “Stack”, do simulador CompSim.

Na Figura 11.17, vemos a seguinte representação:

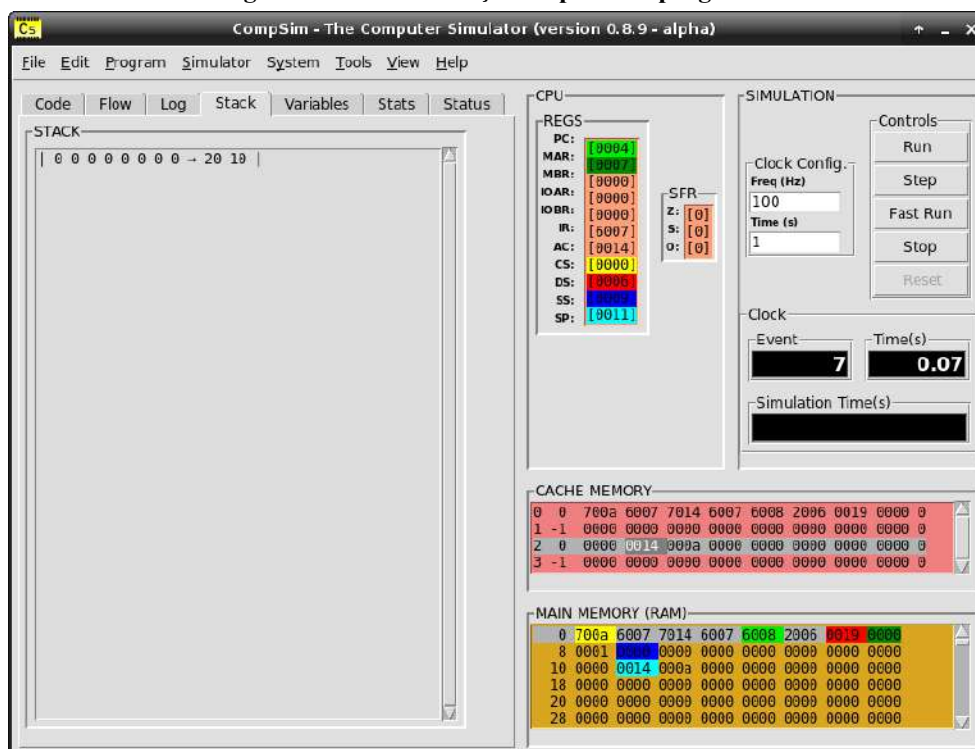
0 0 0 0 0 0 0 0 → 20 10

Onde:

- “|” representa os limites da pilha, sendo que o da esquerda representa o limite máximo para adição de elementos e, o da direita, representa a base da pilha;
- “→” representa o apontador do topo da pilha;
- os números “0” representam os espaços de memória da pilha que ainda não receberam elementos;
- “10” é o primeiro elemento adicionado à pilha do programa; e

- “20” é o último elemento adicionado à pilha do programa.

Figura 11.17. Visualização da pilha do programa.



Fonte: Próprio autor (2021).

Nessa representação, entende-se que o número 10 foi o primeiro elemento adicionado à pilha do programa e, o número 20, foi o último elemento adicionado, sendo apontado então pelo apontador de topo de pilha.

Ainda na Figura 11.17, no componente gráfico “CPU”, observa-se que o registrador SS está apontando para o endereço de memória 9 (valor hexadecimal “0009”, destacado em cor azul escuro) e que o registrador SP está apontando para o endereço 17 (valor hexadecimal “0011”, destacado em cor azul claro). Já no componente gráfico “MAIN MEMORY (RAM)”, o endereço de memória destacado pelo registrador SP contém o valor 20 (“0014” em hexadecimal) e, ao lado direito, há o valor 10 (“000a” em hexadecimal). Percebe-se então que o registrador SP, a cada novo elemento adicionado à pilha do programa, apontará para um endereço de memória menor (o valor no registrador será decrementado).

O exemplo a seguir ilustra como utilizar a pilha do programa para inverter a ordem dos caracteres de uma *string*.

Figura 11.18. Exemplo de código Assembly para inversão de string.

Linha	Código
1	.code
2	while_1:
3	LDI ptr ;este bloco: while(*ptr != '/0').
4	SUB fim_str
5	JZ end_while_1
6	do_1:
7	LDI ptr ;PUSH(*ptr).
8	SOP push
9	
10	MOV 1 ;este bloco: ptr++.
11	ADD ptr
12	STA ptr
13	
14	JMP while_1
15	
16	end_while_1:
17	MOV str ;este bloco: ptr = str.
18	STA ptr
19	
20	while_2:
21	SOP pop ;este bloco: *ptr = POP().
22	STI ptr
23	do_2:
24	SUB inicio_str ;este bloco: while(*ptr != 'H').
25	JZ end
26	
27	MOV 1 ;este bloco: ptr++.
28	ADD ptr
29	STA ptr
30	
31	JMP while_2
32	
33	end:
34	INT exit
35	
36	.data
37	;syscall exit
38	exit: DD 25
39	str: INITB "Hello",0
40	fim_str: DD 0
41	inicio_str: DB 'H'
42	ptr: DD str
43	push: DD 0
44	pop: DD 1
45	
46	.stack 10

Fonte: Próprio autor (2021).

Na Figura 11.18, nas Linhas 39 à 45, são definidas respectivamente as seguintes variáveis:

- “str”: consiste na *string* “Hello”, a qual será invertida;
- “fim_str”: consiste no símbolo de fim de *string* e será utilizado para encerrar o processo de adição dos caracteres da *string* na pilha do programa;

- “inicio_str”: consiste no símbolo de início *string* e será utilizado para encerrar o processo de remoção dos caracteres da *string* que foram adicionados na pilha do programa;
- “ptr”: ponteiro utilizado para acessar cada caractere da *string*;
- “push”: variável com o operando da instrução SOP para adição de elementos na pilha do programa; e
- “pop”: variável com o operando da instrução SOP para remoção de elementos na pilha do programa.

A Figura 11.18, as Linhas 3 à 14 incluem uma estrutura de repetição While..Do, em que, em cada repetição, o caractere da *string* apontado por “ptr” é adicionado à pilha do programa (Linhas 7 e 8). A condição de parada da estrutura While..Do é dada quando o ponteiro atinge o símbolo de fim de *string* “\0”.

Após adicionar todos os caracteres da *string* na pilha do programa, o ponteiro “ptr” é reiniciado para apontar para o início da *string* em “str”, como mostram as Linhas 17 e 18.

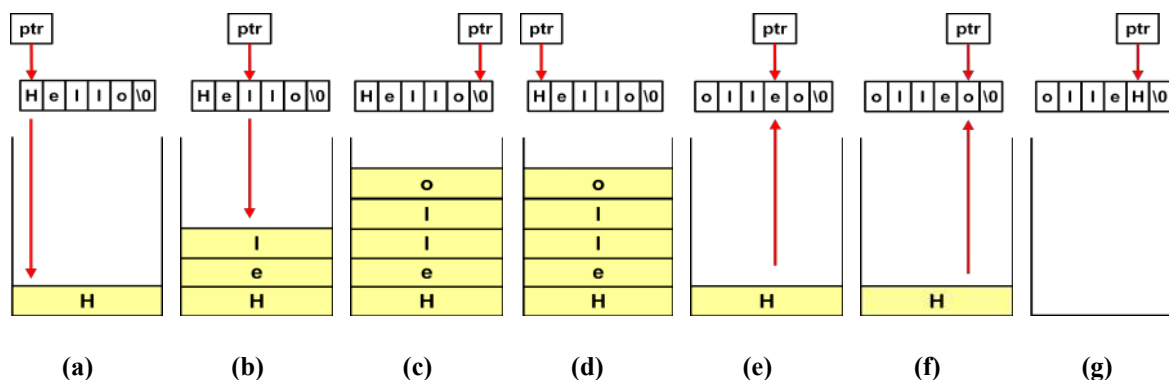
Nas Linhas 21 à 31, há uma segunda estrutura de repetição While..Do, em que, a cada repetição, é retirado um caractere da pilha do programa e adicionado na posição de memória apontada por “ptr” (Linhas 21 e 22). Nesta segunda estrutura de repetição While..Do, a condição de parada consiste em verificar se o último caractere retirado da pilha é igual à “H”, que é símbolo do início da *string* (Linhas 24 e 25).

A Figura 11.19 ilustra, de forma gráfica, a execução do programa anterior. Nela, pode-se observar que em:

- (a): o caractere “H”, que está sendo apontado pelo ponteiro “ptr”, é o primeiro caractere adicionado à pilha do programa;
- (b) o caractere “l”, que está sendo apontado pelo ponteiro “ptr”, é o terceiro caractere adicionado à pilha do programa;
- (c) o ponteiro “ptr”, ao encontrar o caractere de fim de string ‘\0’, faz com que o processo de adição dos caracteres da *string* na pilha seja encerrado;
- (d) o ponteiro “ptr” é reconfigurado para apontar para o início da *string* novamente;
- (e) o caractere “e” foi removido da pilha e atribuído à quarta posição da *string*, a qual está sendo apontada pelo ponteiro “ptr”;
- (f) o caractere “H” será removido da pilha e será atribuído à quinta posição da *string*, a qual está sendo apontada pelo ponteiro “ptr”; e
- (e) após a atribuição do caractere “H” à quinta posição da *string*, o ponteiro “ptr” ao apontar para um caractere “H”, faz com que o processo de remoção dos caracteres da

string na pilha seja encerrado e, com isso, encerre a inversão da *string*. Observa-se ainda que, ao final da inversão da *string*, o caractere de fim de *string* é preservado.

Figura 11.19. Visualização da pilha do programa.



Fonte: Próprio autor (2021).

11.4 Alocação Dinâmica de Memória

Estudou-se, no [Capítulo 8 - Manipulação de Dados em Memória](#), que a memória de um programa é dividida em segmentos de código/texto, dados e pilha. E que, para gerenciar de forma coordenada a leitura e execução das instruções do programa, a definição, declaração e acesso de variáveis e de estruturas de dados, bem como manipulação da pilha do programa, os processadores mantêm um conjunto de registradores para endereçamento dos limites de cada segmento.

Viu-se que, no simulador CompSim, há os registradores de Segmento de Código (*Code Segmento* - CS), Segmento de Dados (*Data Segment* - DS), Segmento de Pilha (*Stack Segmento* - SS) e Apontador de Top de Pilha (*Stack Pointer* - SP). Os registradores CS, DS e SS apontam para o início dos segmentos que contêm as instruções do programa, para as variáveis e estrutura de dados definidas e/ou declaradas e para o limite máximo de elementos que podem ser adicionados à pilha do programa, respectivamente. Já o registrador SP aponta inicialmente para o final do segmento de pilha, tendo sua referência modificada a cada inserção ou remoção de elementos da pilha, de forma que sempre aponte para o último elemento adicionado.

Além dessas seções, dependendo do sistema computacional e/ou do sistema operacional, pode haver ainda um segmento chamado *Heap*, que inclui posições de memória não utilizadas. Essas posições podem ser demandadas quando há necessidade de alocação dinâmica de memória, como acontece com as funções “malloc” e “calloc”, da linguagem de

programação C. Em C, há ainda funções, tal como “free”, que podem ser utilizadas para liberar o bloco de memória alocado dinamicamente (PATTERSON; HENNESSY, 2017).

No CompSim, por padrão, ao carregar um programa em memória, não há definição de um segmento para *Heap*. Um programa em *Assembly*, quando alocado em memória, terá seus elementos distribuídos entre os segmentos de código/texto e dados, bem como indicará o tamanho da memória que será alocada para a pilha do programa.

No entanto, é possível realizar a criação de um segmento *Heap* com poucos passos, como mostra o exemplo a seguir, na Figura 11.20. No exemplo, utiliza-se a pseudo-instrução *ORG*, na Linha 15, para alocar 20 posições livres em memória. Observa-se ainda que a instrução *ORG* está posicionada entre os segmentos de dados (“.data”) e de pilha do programa (“.stack”), de forma que o bloco de memória livre reservado estende o segmento de dados em 20 posições.

Figura 11.20. Exemplo de código Assembly para criação de Heap do programa.

Linha	Código
1	.code
2	set_heap:
3	MOV 1 ;este bloco: faz o ponteiro heap apontar para o
4	ADD heap ; início do bloco de memoria livre.
5	STA heap
6	
7	end:
8	INT exit
9	
10	.data
11	;syscall exit
12	exit: DD 25
13	heap: DD heap ;heap é um ponteiro que aponta para si.
14	
15	ORG 20
16	
17	.stack 10

Fonte: Próprio autor (2021).

Ainda no exemplo anterior, na Figura 11.20, é definida uma variável do tipo ponteiro, chamada “heap”, que é inicializada para apontar para si mesma. Nas Linhas 3 à 5, o ponteiro “heap” sofre o incremento do endereço apontado por ele, para que passe a apontar para a primeira posição da memória livre alocada com a pseudo-instrução *ORG*. Com isso, o ponteiro “heap” já pode ser utilizado para dispor acesso à memória livre. No entanto, é importante destacar que, no exemplo, o artifício utilizado para criação do segmento *Heap* somente é possível porque o ponteiro “heap” está sendo declarado ao final do segmento de dados e, em posição que antecede o bloco de memória livre.

O exemplo a seguir ilustra como pode ser alocado dinamicamente um vetor/array com 10 posições, utilizando o *Heap*. Na Figura 11.21, na Linha 25, é definida uma variável chamada “array”, que é do tipo inteiro e será utilizada como ponteiro para a memória que será alocada dinamicamente para o vetor/array.

Figura 11.21. Exemplo de código Assembly para alocação dinâmica de array.

Linha	Código
1	<code>.code</code>
2	<code>config_heap:</code>
3	<code>MOV 1 ;este bloco: faz o ponteiro heap apontar para o</code>
4	<code>ADD heap ; inicio do bloco de memoria livre.</code>
5	<code>STA heap</code>
6	
7	<code>aloca_array:</code>
8	<code>LDA heap ;este bloco: faz o ponteiro array apontar para heap.</code>
9	<code>STA array</code>
10	
11	<code>MOV 10 ;este bloco: atualiza o ponteiro heap em +10 posicoes.</code>
12	<code>ADD heap</code>
13	<code>STA heap</code>
14	
15	<code>atribui_100_array:</code>
16	<code>MOV 100 ;este bloco: atribui valor 100 na primeira posicao</code>
17	<code>STI array ; de memoria alocada para array.</code>
18	
19	<code>end:</code>
20	<code>INT exit</code>
21	
22	<code>.data</code>
23	<code>;syscall exit</code>
24	<code>exit: DD 25</code>
25	<code>array: DD 0</code>
26	<code>heap: DD heap ;heap é um ponteiro que aponta para si.</code>
27	
28	<code>ORG 20</code>
29	
30	<code>.stack 10</code>

Fonte: Próprio autor (2021).

Assim como no exemplo da Figura 11.20, as Linhas 3 à 5, da Figura 11.21, configuram o ponteiro “heap” para apontar para o início do bloco de memória livre, alocado com a pseudo-instrução ORG, na Linha 28. As Linhas 8 e 9 fazem com que o ponteiro “array” aponte para o mesmo endereço apontado por “heap”. Nas Linhas 10 à 13, o ponteiro “heap” será atualizado para apontar para 10 posições adiante do endereço de memória anterior. Desta maneira, haverá 10 posições de memória entre os endereços apontados por “array” e “heap”. Essas 10 posições de memória configuram então a memória dinamicamente alocada para o array/vetor apontado pelo ponteiro “array”.

11.5 Relocação de Memória

Na subseção anterior, ilustrou-se, de forma bastante simplificada, como é possível criar dinamicamente um segmento para *Heap* e como alocar memória dinamicamente para estruturas de dados.

Estas são duas tarefas corriqueiras na grande maioria dos sistemas operacionais. Além delas, os sistemas operacionais também são responsáveis, na execução de um programa, por: alocar memória para o programa, segmentá-la, copiar instruções e dados para os respectivos segmentos, configurar registradores de endereçamento de segmentos para apontarem para os respectivos segmentos e atualizar registrador de contador de programa (PC) para o início do segmento de código/texto, de forma que o programa inicie sua execução (STALLINGS, 2010).

Os sistemas operacionais modernos geralmente necessitam “Relocar” os programas, por diferentes motivos, tais como, por exemplo, para permitir que: mais de um programa seja carregado em memória; e um melhor gerenciamento de memória, através da técnica de “swapping”, que consiste na troca de blocos de palavras entre memória RAM e disco rígido, de forma a evitar desperdício de memória e, como consequência, não poder alocar novos programas ou degradar o desempenho de execução do sistema (STALLINGS, 2010).

Relocação é uma atividade que consiste em definir os endereços de memória que serão utilizados para hospedar um programa, e pode ser dos tipos: 1) estático, os endereços de memória que hospedarão o programa são calculados durante a compilação do programa ou durante o carregamento do programa em memória; e 2) dinâmico, os endereços são calculados durante a execução do programa, pelo uso de um Registrador de Relocação (sempre que houver uma referência a um endereço do programa, quer seja para instrução ou dado, o endereço será adicionado a um endereço base contido no registrador de relocação e, com isso, será redirecionado à outra região da memória) (SILBERSCHATZ; GALVIN; GAGNE, 2013).

No exemplo a seguir, será vista uma aplicação que realiza a relocação dinâmica de um programa no CompSim. O programa que será relocado apresenta-se nas Linhas 105 à 118, do código *Assembly* na Figura 11.22, e realiza a soma dos valores das variáveis “var1” e “var2”, sendo o resultado atribuído à variável “soma”.

Para a relocação, definiu-se as seguintes variáveis auxiliares:

- “tam_prg”: guarda a quantidade de endereços de memória ocupados com o programa que será relocado;

- “tam_cod”: guarda a quantidade de endereços de memória do segmento de código do programa que será relocado;
- “cnt”: variável auxiliar utilizada em estruturas de repetição While..Do;
- “push”: variável utilizada como parâmetro da instrução SOP, que são utilizadas para adição de elementos na pilha do programa;
- “pop”: variável utilizada como parâmetro da instrução SOP, que são utilizadas para remoção de elementos na pilha do programa;
- “ptr”: ponteiro utilizado para apontar para os endereços de memória do programa que será relocado;
- “ptr_n”: ponteiro utilizado para apontar para os endereços de memória de destino na relocação do programa;
- “RDR”: variável que atua como o registrador de relocação e inclui o intervalo de relocação do programa. No exemplo da Figura 11.22, o programa será movido adiante em 200 endereços de memória ($RDR = 200$); e
- “jmp_opc”: contém o opcode da instrução JMP. Após a relocação do programa, será utilizada para compor a instrução que desviará o fluxo de execução do programa para execução do programa relocado em novo endereço.

No código *Assembly* da Figura 11.22, as linhas a seguir incluem os seguintes blocos de instruções, com as respectivas descrições:

- Linha 2: esta linha realiza um desvio de fluxo para a instrução na linha 7. Após a relocação do programa de soma, esta instrução será sobrescrita para desviar o fluxo de execução para execução do programa relocado;
- Linhas 7 e 8: atribuem o endereço inicial do programa que será relocado ao ponteiro “ptr”;
- Linhas 14 à 30: incluem uma estrutura de repetição While..Do que atualiza os endereços referenciados pelas instruções do programa que será relocado. Por exemplo, a instrução na linha 106 referencia a variável “var1”, que está alocada no endereço 116. Com a relocação, a variável ocupará o endereço 316 ($116 + 200$). Com isso, este bloco atualiza as referências às variáveis nas instruções que serão relocadas;
- Linhas 36 à 55: calculam e salvam na pilha do programa os endereços de segmentos do programa após relocação, apontados pelos registradores de segmento SP, SS, DS e CS;
- Linhas 60 à 79: copiam as instruções e dados do programa que será relocado para os novos endereços de memória;

- Linhas 85 à 96: recuperam da pilha do programa os novos endereços dos segmentos do programa relocado, atribuindo-os aos registradores CS, DS, SS e SP; e
- Linhas 98 à 101: com o novo endereço do início do programa, contido no registrador CS, utiliza-se o opcode da instrução JMP, na variável “jmp_opc”, para compor a instrução de desvio de fluxo de execução para o programa relocado (Linhas 98 e 99). Em seguida, na Linha 100, a instrução composta será escrita no primeiro endereço de memória (endereço 0x0000) e, em seguida, desvia-se o fluxo do programa para ela (Linha 101), a qual, por sua vez, quando executada, realizará o desvio de fluxo do programa para execução do programa relocado.

Figura 11.22. Exemplo de código Assembly para relocação de programa.

Linha	Código
1	<code>.code</code>
2	<code>JMP init_atz_refs_insts</code>
3	<code>; =====</code>
4	<code>; Seta em ptr o endereco do programa que sera relocado.</code>
5	<code>; =====</code>
6	<code>init_atz_refs_insts:</code>
7	<code>MOV main</code>
8	<code>STA ptr</code>
9	<code>; =====</code>
10	<code>; Atualiza os enderecos dos operandos das instrucoes do programa que sera</code>
11	<code>; relocado.</code>
12	<code>; =====</code>
13	<code>atz_refs_insts:</code>
14	<code>LDA tam_cod ;while (cnt < tam_cod).</code>
15	<code>SUB cnt</code>
16	<code>JZ init_mov_prog</code>
17	
18	<code>LDI ptr ;utiliza RDR para atualizar enderecos dos operandos.</code>
19	<code>ADD RDR</code>
20	<code>STI ptr</code>
21	
22	<code>MOV 1 ;aponta para a proxima instrucao.</code>
23	<code>ADD ptr</code>
24	<code>STA ptr</code>
25	
26	<code>MOV 1 ;cnt++.</code>
27	<code>ADD cnt</code>
28	<code>STA cnt</code>
29	
30	<code>JMP atz_refs_insts</code>
31	<code>; =====</code>
32	<code>; Salva na pilha do programa os novos enderecos dos registradores de</code>
33	<code>; segmento: SP, SS, DS e CS.</code>
34	<code>; =====</code>
35	<code>init_mov_prog:</code>
36	<code>MOV \$SP ;salva na pilha o novo endereco de SP.</code>
37	<code>ADD RDR</code>
38	<code>SOP push</code>
39	
40	<code>MOV \$SS ;salva na pilha o novo endereco de SS.</code>
41	<code>ADD RDR</code>
42	<code>SOP push</code>
43	

```

44      MOV $DS          ;salva na pilha o novo endereço de DS.
45      ADD RDR
46      SOP push
47
48      MOV main
49      STA ptr
50      ADD RDR
51      STA ptr_n        ;ptr_n contem o novo endereço de CS.
52      SOP push         ;salva na pilha o novo endereço de CS.
53
54      MOV 0            ;reseta contador.
55      STA cnt
56      ; =====
57      ; Copia programa para novo endereço de memoria (relocacao).
58      ; =====
59      mov_prog:
60          LDA tam_prg    ;while (cnt < tam_prg).
61          SUB cnt
62          JZ init_prog
63
64          LDI ptr         ;carrega instrucao no endereço antigo.
65          STI ptr_n       ;grava instrucao no novo endereço (relocacao).
66
67          MOV 1           ;aponta para a proxima instrucao do programa.
68          ADD ptr
69          STA ptr
70
71          MOV 1           ;aponta para proximo endereço para copia da
72          ADD ptr_n       ; proxima instrucao do programa.
73          STA ptr_n
74
75          MOV 1           ;cnt++.
76          ADD cnt
77          STA cnt
78
79          JMP mov_prog
80      ; =====
81      ; Configura os registradores de segmento para apontarem para o programa
82      ;  relocado.
83      ; =====
84      init_prog:
85          SOP pop
86          STA ptr_n
87          MOV -$CS        ;remove elemento da pilha e atualiza CS.
88
89          SOP pop
90          MOV -$DS        ;remove elemento da pilha e atualiza DS.
91
92          SOP pop
93          MOV -$SS        ;remove elemento da pilha e atualiza SS.
94
95          SOP pop
96          MOV -$SP        ;remove elemento da pilha e atualiza SP.
97
98          MOV $CS         ;Seta no endereço de memoria 0x0 a instrucao de
99          ADD jmp_opc      ; salto para o programa relocado.
100         STA @0x0
101         JMP @0x0
102     ; =====
103     ; Programa que sera relocado em 200 enderecos: soma = v1 + v2.
104     ; =====
105     main:
106         LDA v1
107         ADD v2
108         STA soma
109
110     end:

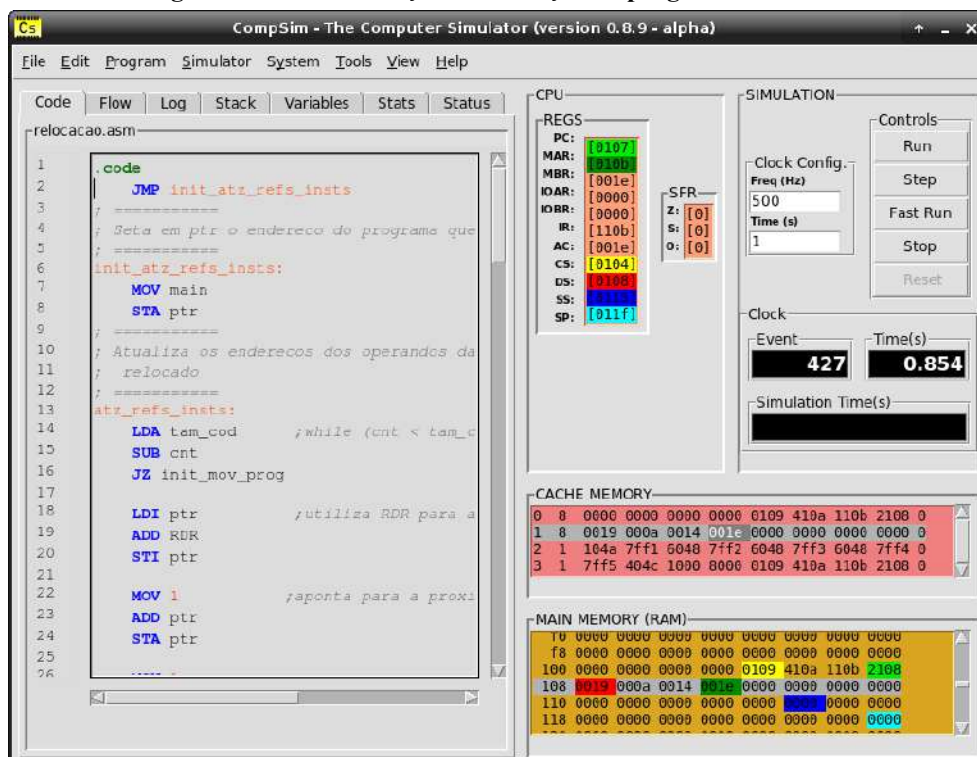
```

111	INT exit
112	
113	.data
114	;syscall exit
115	exit: DD 25
116	v1: DD 10
117	v2: DD 20
118	soma: DD 0
119	; =====
120	; Fim do Programa
121	; =====
122	tam_prg: DD 8 ;tamanho do programa (4 inst + 4 dados).
123	tam_cod: DD 4 ;tamanho do segmento de codigo.
124	cnt: DD 0
125	push: DD 0
126	pop: DD 1
127	ptr: DD 0 ;ponteiro utilizado o programa.
128	ptr_n: DD 0 ;ponteiro para mover o programa.
129	RDR: DD 200 ;atua como Registrador de Relocação.
130	jmp_opc: DD 0x8000 ;opcode da instrucao JMP.
131	
132	.stack 10

Fonte: Próprio autor (2021).

A Figura 11.23 mostra o instante em que o programa relocado grava o resultado da soma das variáveis “v1” e “v2” na variável “soma”.

Figura 11.23. Visualização de execução de programa relocado.



Fonte: Próprio autor (2021).

Na Figura 11.23, pode-se observar no componente gráfico “CPU” que o registrador CS aponta para o endereço 260 (valor “0104” em hexadecimal), DS aponta para 264 (“0108”

em hexadecimal), SS aponta para 277 ("0115" em hexadecimal) e SP aponta para 287 ("011f" em hexadecimal).

Ainda na Figura 11.23, no componente gráfico "MAIN MEMORY (RAM)", observa-se nos endereços "109", "10a" e "10b" em hexadecimal, os valores 10 ("000a" em hexadecimal), 20 ("0014" em hexadecimal) e 30 ("001e" em hexadecimal), referentes às variáveis "v1", "v2" e "soma", após a realização da operação de soma e gravação do resultado, no programa relocado.

É importante destacar que os componentes gráficos para visualização do programa "EXECUTION FLOW", na aba "Flow", "STACK" na aba "Stack", e "VARIABLES", na aba "Variables", são alimentados, inicialmente, por informações estáticas geradas pelo Montador (*Assembler*), durante a montagem e o carregamento do programa em linguagem de máquina na memória. Isso significa que o programa de soma poderá ser visualizado no endereço original, porém, após a sua relocação dinâmica para um novo endereço, esses componentes gráficos de visualização não mais refletirão os status do programa de soma.

Desta maneira, o processo de visualização do programa relocado deverá ser realizado com suporte dos componentes gráficos "CPU" e "MAIN MEMORY (RAM)". Também pode ser necessário utilizar o componente gráfico "CACHE MEMORY", visto que, dependendo das configurações da memória Cache, alguns dados modificados nela poderão não estar disponíveis na memória RAM (o bloco modificado em memória Cache ainda não foi copiado para a memória RAM, como pode acontecer na política de escrita *Write-Back* da memória Cache) (para mais detalhes sobre políticas de escrita em Cache, ver [Capítulo 4 - Introdução à Organização de Computadores](#)).

11.6 Modos de Endereçamento do Processador Cariri

Até este ponto do livro, foram vistas diversas instruções do processador Cariri, as quais realizam diferentes tipos de operações, tais como para acesso a dados em memória e registradores, processamento de dados e de desvio de fluxo de execução do programa. Dependendo da instrução, são implementados diferentes modos de acesso aos operandos (ver Modos de Endereçamento no [Capítulo 5 - Introdução à Arquitetura de Computadores](#)). O Quadro 11.1, a seguir, sumariza as instruções do processador Cariri e os respectivos modos de endereçamento suportados (para mais detalhes sobre cada uma das instruções, ver o [Apêndice A - Instruções do Processador Cariri](#)).

Quadro 11.1. Instruções do processador Cariri e respectivos modos de endereçamento.

Opcode		Instrução	Modo de Endereçamento
Bin	Hexa		
0000	0	LDA <variavel> LDA @endereco	Direto/Implícito Direto/Implícito
0001	1	STA <variavel> STA @endereco	Direto/Implícito Direto/Implícito
0010	2	INT <variavel> INT @endereco	Direto[/Implícito] Direto[/Implícito]
0011	3	CALL <rotulo> CALL @endereco	Direto Direto
0100	4	ADD <variavel> ADD @endereco	Direto/Implícito Direto/Implícito
0101	5	SUB <variavel> SUB @endereco	Direto/Implícito Direto/Implícito
0110	6	SOP <variavel> SOP @endereco	Direto/Implícito Direto/Implícito
0111	7	MOV <inteiro> MOV \$<registrador> MOV -\$<registrador> MOV <variavel> MOV @endereco	Imediato/Implícito Registrador/Implícito Registrador/Implícito Imediato/Implícito Imediato/Implícito
1000	8	JMP <rotulo> JMP @endereco	Imediato Imediato
1001	9	JN <rotulo> JN @endereco	Imediato Imediato
1010	a	JZ <rotulo> JN @endereco	Imediato Imediato
1011	b	LDI <variavel> LDI @endereco	Indireto/Implícito Indireto/Implícito
1100	c	STI <variavel> STI @endereco	Indireto/Implícito Indireto/Implícito
1101	d	NAND <variavel> NAND @endereco	Direto/Implícito Direto/Implícito
1110	e	SHIFT <variavel> SHIFT @endereco	Direto/Implícito Direto/Implícito
1111	f	RET	-

Fonte: Próprio autor (2021).

Observa-se no Quadro 11.1 que:

- as instruções LDA, STA, INT, ADD, SUB, SOP, NAND e SHIFT suportam o modo de endereçamento direto;
- as instruções LDI e STI são as únicas que suportam o modo de endereçamento indireto;
- as instruções de desvio de fluxo do programa JMP, JZ, JN e CALL são as únicas que implementam apenas um modo de endereçamento, o imediato (o endereço de memória de destino no desvio de fluxo de execução está presente na própria instrução);
- a instrução de desvio de fluxo RET é a única que não possui operando (as instruções de desvio de fluxo de execução CALL e RET serão vistas com mais detalhes no [Capítulo 12 - Modularização do Programa](#));
- as instruções LDA, STA, INT, ADD, SUB, SOP, MOV, LDI, STI, NAND e SHIFT também suportam o modo de endereçamento implícito, por fazerem referência implícita ao registrador acumulador (AC) e, no caso da instrução SOP, faz-se referência ao topo da pilha do programa;
- a instrução MOV, dentre todas as instruções do processador Cariri, é a mais versátil, pois, dependendo de como é utilizada, suporta os modos de endereçamento imediato/implícito e registrador/implícito;
- a instrução INT suporta o modo de endereçamento direto e, dependendo do valor do operando, pode suportar também o modo de endereçamento implícito (a instrução INT será explorada no [Capítulo 13 - Entrada/Saída](#)).

11.7 Resumo

Este capítulo tratou de aspectos avançados para acesso a dados em memória. Viu-se o conceito de ponteiro, o qual foi utilizado durante todo o capítulo para ilustrar, através de exemplos práticos, a criação de rotinas de manipulação de estruturas de dados, alocação dinâmica de memória e relocação de um programa em memória.

Fez-se ainda um estudo sobre a pilha do programa, e, através de exemplos práticos, viu-se como utilizá-la para armazenamento temporário de dados e como pode ser aplicada para dar suporte à manipulação de estruturas de dados.

Por fim, sumarizou-se as instruções do processador Cariri com uma discussão breve sobre os respectivos modos de endereçamento suportados.

CAPÍTULO 12 – MODULARIZAÇÃO DE PROGRAMAS

No capítulo anterior, viu-se que, dependendo dos requisitos do problema, a complexidade da solução pode aumentar. Com isso, os programas de computador podem se tornar maiores, por incluir mais instruções, e, por consequência, se tornarem muito difíceis de ler e compreender, de realizar depuração para correções, manutenção e evolução.

Ao longo dos anos, viu-se que a melhor forma de desenvolver e manter um programa maior é modularizá-lo, através da sua divisão em partes menores, também chamadas de módulos, que são mais simples de tratar em relação ao programa como um todo. Há vários motivos para modularizar um programa, entre eles estão: 1) Dividir um problema maior em problemas menores, que são mais simples de tratar, e agrupar suas soluções para compor a solução final do problema maior. Esta técnica é conhecida como “Dividir para Conquistar” (*Divide-and-Conquer*) (CORMEN et al., 2009); 2) Evitar repetição de código, onde ao se criar funções, que são blocos de instruções que realizam tarefas específicas, elas podem ser chamadas para execução sob demanda, em diferentes pontos do programa; e, 3) Abstrair o código, pois ao chamar uma função para execução, não é necessário compreender como a mesma está implementada, apenas utiliza-se a sua assinatura, como são os casos das funções “printf”, “scanf”, “malloc” e “free”, da linguagem C (DEITEL; DEITEL, 2009).

Este capítulo trata da modularização de código *Assembly*, onde serão vistas a criação e a chamada de: 1) funções simples; 2) funções com passagem de parâmetros e retorno de resultado; 3) funções com variáveis locais; 4) múltiplas chamadas de funções; e 5) funções recursivas.

12.1 Definição e Chamada de Funções

Para ilustrar a criação de funções em *Assembly*, utilizaremos como base o código em linguagem de programação C da Figura 12.1.

Figura 12.1. Exemplo de código C para ilustrar a definição e chamada de função.

Linha	Código
1	<code>int var = 0;</code>
2	<code>int resultado = 0;</code>
3	
4	<code>void incrementa_var (void) {</code>
5	<code> var++;</code>
6	<code>}</code>
7	
8	<code>void main (void) {</code>
9	<code> incrementa_var();</code>
10	<code> resultado = var;</code>
11	<code>}</code>

Fonte: Próprio autor (2021).

Na Figura 12.1, nas Linhas 1 e 2 são definidas as variáveis globais “var” e “resultado”, que são inicializadas com valor 0. Nas Linhas 4 à 6, define-se a função “incrementa_var”, que adiciona 1 ao o valor contido na variável “var” e o resultado é gravado na própria variável. Nas Linhas 8 à 11, define-se o corpo da função “main”, que é o *starting point* de qualquer programa em linguagem C (quando um programa entra em execução, suas instruções são as primeiras a serem executadas). Por fim, na Linha 9, a função “incrementa_var” é chamada para execução, onde realizará o incremento do valor contido na variável “var”, cujo resultado é atribuído à variável “resultado”, na Linha 10.

Analisando o programa da Figura 12.1, observa-se que ao chamar a função “incrementa_var” para execução, na Linha 9, o fluxo de execução do programa é desviado para a função e, com isso, será executada a instrução da Linha 5, a qual incrementa o valor da variável “var”. Após a execução dessa instrução, o fluxo de execução do programa é desviado de volta à função “main”, na Linha 10, que é a instrução posterior à utilizada para chamar a função “incrementa_var”.

Analisando a dinâmica de chamada e de retorno da função “incrementa_var”, percebe-se que há necessidade de realização dos seguintes passos: 1) armazenamento do endereço da instrução posterior a da chamada da função, que é o endereço de retorno ao concluir a execução da função; 2) desvio do fluxo de execução para a primeira instrução da função; e, após a execução das instruções da função, 3) desvio do fluxo de execução para o endereço de retorno, anteriormente armazenado.

O processador Cariri do simulador CompSim possui duas instruções que implementam os passos para chamada de uma função: a instrução “CALL”, que implementa os passos 1 e 2, tratados anteriormente; e a instrução “RET”, que implementa o passo 3. Os

quadros a seguir trazem resumos com as descrições das instruções CALL e RET, em qual seção do código podem ser utilizadas, suas sintaxes de uso e exemplos de como utilizá-las.

Instrução: CALL
Descrição: Armazena na pilha do programa o endereço de retorno da função, que é o endereço da instrução posterior à instrução CALL; atribui o operando da instrução, que é o endereço de memória da primeira instrução da função, no registrador contador de programa PC para, no ciclo de instrução seguinte, desviar o fluxo de execução para a função.
Seção: .code
Sintaxe: [<rotulo>:] CALL @<endereço> [<rotulo>:] CALL <rotulo>
Exemplos: CALL @100 ;chama a funcao que inicia no endereco 100. CALL func ;chama a funcao com rotulo func. Chama_100: CALL @100 ;instrucao com rotulo para chamar funcao que inicia no endereco 100. Chama_func: CALL func ;instrucao, com rotulo em linha separada, para chamar funcao com rotulo func.

Instrução: RET (RETurn)
Descrição: Retira da pilha do programa o endereço de retorno de uma função, que é o endereço da instrução posterior à instrução CALL; atribui o endereço lido no registrador contador de programa PC para, no ciclo de instrução seguinte, desviar o fluxo de execução de volta para a instrução posterior à que chamou a função.
Seção: .code
Sintaxe: [<rotulo>:] RET
Exemplos: RET ;desvia o fluxo de execução do programa para retorno de chamada de funcao. Ret_func: RET ;instrucao com rotulo para retorno de chamada de funcao. Ret_func: RET ;instrucao, com rotulo em linha separada, para retorno de chamada de funcao.

O código na Figura 12.2 traz um exemplo de implementação em *Assembly* do programa C da Figura 12.1, que ilustra a definição e chamada da função “incrementa_var”.

Figura 12.2. Exemplo de código Assembly para ilustrar a definição e chamada de função.

Linha	Código
1	<code>.code</code>
2	<code>main:</code>
3	<code>CALL incrementa_var ;chama a funcao incrementa_var.</code>
4	<code>LDA var</code>
5	<code>STA resultado</code>
6	
7	<code>end:</code>
8	<code>INT exit</code>
9	
10	<code>incrementa_var:</code>
11	<code>MOV 1 ;este bloco implementa a funcao incrementa_var.</code>
12	<code>ADD var</code>
13	<code>STA var</code>
14	<code>RET</code>
15	
16	<code>.data</code>
17	<code>;syscall exit</code>
18	<code>exit: DD 25</code>
19	<code>var: DD 0</code>
20	<code>resultado: DD 0</code>
21	
22	<code>.stack 10</code>

Fonte: Próprio autor (2021).

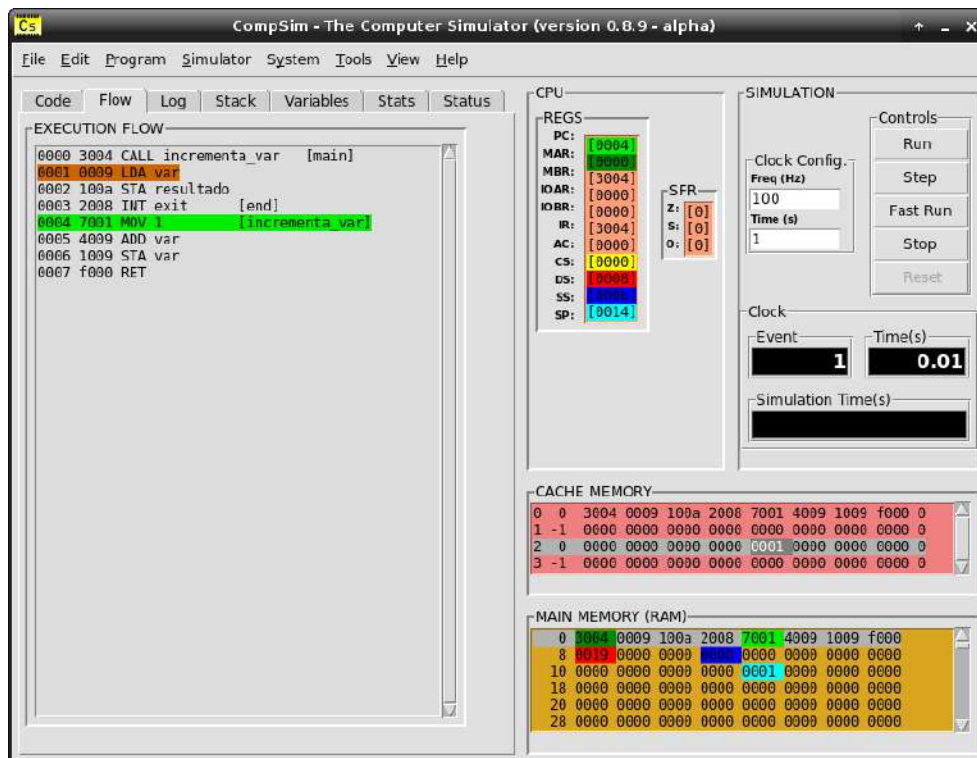
No código da Figura 12.2, a função “main” está definida nas Linhas 2 à 5 e a função “incrementa_var” está definida entre as Linhas 10 à 14. Observa-se que a função “incrementa_var” consiste de um bloco de instruções *Assembly*, em que a primeira instrução possui o rótulo “incrementa_var” e a última consiste na instrução de retorno de função RET. Já na função “main”, a instrução CALL, na Linha 3, é responsável por desviar o fluxo de execução do programa para a função “incrementa_var”. A função CALL atua da seguinte maneira: inicialmente, adiciona na pilha do programa o endereço da instrução posterior à instrução CALL, de forma que, ao fim da execução da função, o fluxo de execução possa retornar à instrução posterior à da chamada da função; e atualiza o endereço contido no registrador contador de programa PC com o endereço contido no campo operando da instrução CALL, que é o endereço da primeira instrução da função “incrementa_var”, para que, no próximo ciclo de instrução, o fluxo de execução do programa seja desviado para que ela seja executada.

Ao final da execução da função “incrementa_var”, a instrução RET faz com que o fluxo de execução seja desviado de volta para a função “main”, na instrução da Linha 4, que é a instrução posterior à de chamada da função CALL. A instrução RET atua da seguinte maneira: no primeiro momento, retira da pilha do programa o endereço da instrução de retorno da função; e, com ele, atualiza o endereço contido no registrador contador de programa PC. Assim, no próximo ciclo de instrução, o fluxo de execução do programa será

desviado para a instrução da Linha 4, que é a instrução posterior à CALL. Após o desvio de fluxo de execução para a instrução da Linha 4, via instrução RET na Linha 14, o fluxo de execução do programa segue executando as demais instruções da função “main” até encerrar o programa com a instrução INT, na Linha 8.

Durante a execução do programa da Figura 12.2, no CompSim é possível acompanhar os desvios de fluxo, através do componente gráfico “EXECUTION FLOW”, na aba “Flow”. A Figura 12.3 mostra o desvio de fluxo para execução da função “incrementa_var”. Na figura, no componente gráfico “EXECUTION FLOW”, é possível ver que, seguindo o fluxo normal de execução do programa, a instrução destacada em cor marrom seria a próxima a ser executada, porém, o fluxo será desviado para a instrução destacada em cor verde claro, que é a primeira instrução da função “incrementa_var”.

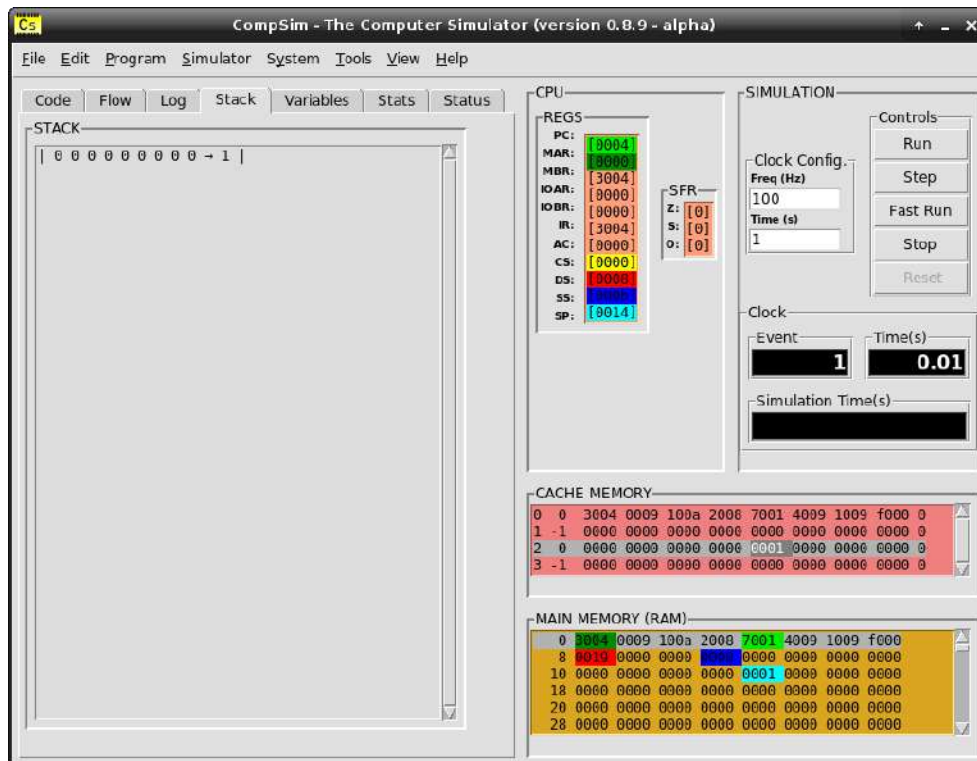
Figura 12.3. Visualização de desvio de fluxo de execução em chamada de função.



Fonte: Próprio autor (2021).

As instruções CALL e RET manipulam a pilha do programa para guardar e retirar, respectivamente, o endereço de retorno da chamada de função. Durante uma simulação, o uso da pilha do programa pode ser acompanhado pelo componente gráfico “STACK”, na aba “Stack”, como mostra a Figura 12.4. Na figura, a instrução CALL adicionou, na pilha do programa, o endereço de retorno “1” da chamada da função “incrementa_var”. Esse endereço será retirado da pilha do programa, quando for executada a instrução RET.

Figura 12.4. Visualização de desvio de fluxo de execução em retorno de função.



Fonte: Próprio autor (2021).

12.2 Funções com Parâmetros de Entrada e Retorno de Resultado

Funções com parâmetros de entrada e retorno de resultado são aquelas que recebem dados como argumentos na chamada da função, os quais serão processados internamente à função e o resultado é retornado à rotina que chamou a função. Uma função sem parâmetros de entrada e retorno de resultado, conhecida também como Procedimento (ou *Procedure*), em algumas linguagens de programação não necessita implementar algum protocolo de chamada e de retorno, visto que o processador assume essa responsabilidade, ao incluir as instruções CALL, que é responsável por chamar a função, e RET, que tem como função devolver o fluxo de controle à rotina que chamou a função (SANTOS; LANGLOIS, 2018).

No entanto, quando as funções necessitam de parâmetros de entrada, alguns aspectos devem ser considerados, que, segundo Santos e Langlois (2018), são:

- **Passagem de valores:** determina a forma como a rotina chamada modifica os valores que são passados como parâmetros, que podem ser por valor ou por referência. Se a rotina implementa a passagem por valor, os dados, que são copiados da variável original, podem ser modificados internamente à função, porém não modificam o valor

da variável original. Já na passagem por referência, será passado como parâmetro da função o endereço da variável original, de maneira que é possível manipular diretamente seu valor;

- Convenções de chamada: determina a ordem em que os valores são passados como parâmetros, bem como eles são tratados internamente à função. Em funções com mais de um parâmetro e dependendo da linguagem de programação, pode variar a ordem em que os parâmetros são avaliados na chamada de uma função. Por exemplo, em uma função que recebe dois parâmetros, pode ser avaliado o segundo parâmetro antes do primeiro; como é o caso do comportamento de grande parte dos compiladores da linguagem de programação C, por questões de facilidade e eficiência na geração do código de máquina, quando os parâmetros são passados via pilha do programa. Além disso, após a chamada da função, a remoção dos parâmetros passados pode seguir duas convenções distintas: na primeira, a rotina chamada fica responsável por tratar a retirada dos parâmetros da estrutura utilizada na passagem; já na segunda, a rotina que chama a função é que fica responsável por retirar os parâmetros da estrutura após o retorno da função; e
- Colocação dos parâmetros: determina em que recursos do processador ou da memória os parâmetros serão inseridos e a sua disposição antes da chamada da função. Há três abordagens possíveis, que são: Parâmetros em Variáveis Globais, Parâmetros em Registradores e Parâmetros em Pilha de Programa.

Já para o retorno de resultado, também devem ser considerados alguns aspectos, que são:

- Número de valores de retorno: a maioria das linguagens de programação só permite o retorno de um valor. Quando há necessidade de retornar mais de um valor, é frequente o uso de referências à estruturas de dados, ou, ainda, a rotina que chama a função ou a função chamada alocam memória adicional para inserção dos valores de retorno; e
- Colocação do retorno: assim como na passagem de parâmetros, podem ser utilizados diferentes recursos do processador ou da memória para passagem do valor de retorno. Podem ser também utilizadas variáveis globais, registradores e pilha do programa.

As subseções a seguir trazem exemplos práticos ilustrativos das abordagens utilizadas para passagem de parâmetros e retorno de resultados.

12.2.1 Variáveis Globais

Esta abordagem é uma das mais antigas e consiste em reservar espaços de memória para variáveis globais que serão utilizadas para guardar os parâmetros de entrada e valores de retorno na chamada de funções.

Para ilustrar essa abordagem, utilizaremos o exemplo da Figura 12.5, cujo programa C implementa a adição de dois valores. Nas Linhas 1 à 3, define-se as variáveis globais “var1” e “var2”, que conterão os valores a serem adicionados, e “resultado” que receberá o resultado da adição. Nas Linhas 4 à 6, são definidas variáveis globais “soma_a”, “soma_b” e “soma_c”, que serão utilizadas como variáveis auxiliares, pela função “soma”, para realizar a adição dos valores contidos nas variáveis “var1” e “var2”. A função “soma” está definida nas Linhas 8 à 10, e a função “main” está definida nas Linhas 12 à 17. Observa-se, na função “main” que, para chamar a função “soma” é necessário atribuir os valores de “var1” e “var2” às variáveis “soma_a” e “soma_b”, respectivamente. Da mesma forma, após a execução da função “soma”, o resultado na variável “soma_c”, será atribuído à variável “resultado”.

Figura 12.5. Exemplo de código C para ilustrar chamada de função com suporte de variáveis globais.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2 = 20;</code>
3	<code>int resultado = 0;</code>
4	<code>int soma_a = 0;</code>
5	<code>int soma_b = 0;</code>
6	<code>int soma_c = 0;</code>
7	
8	<code>void soma (void) {</code>
9	<code> soma_c = soma_a + soma_b;</code>
10	<code>}</code>
11	
12	<code>void main (void) {</code>
13	<code> soma_a = var1;</code>
14	<code> soma_b = var2;</code>
15	<code> soma();</code>
16	<code> resultado = soma_c;</code>
17	<code>}</code>

Fonte: Próprio autor (2021).

O código *Assembly* na Figura 12.6 traz uma implementação do programa em linguagem C da Figura 12.5. Na Figura 12.6, as variáveis globais são definidas nas Linhas 26 à 31, a função “soma” nas Linhas 17 à 21 e a função “main” nas Linhas 2 à 15. Observa-se na função “main” que, antes da chamada da função “soma”, via instrução CALL na Linha 9, faz-se a atribuição dos valores das variáveis “var1” e “var2” às variáveis “soma_a” e

“soma_b”, respectivamente. E, após a chamada da função, atribui-se o valor da variável “soma_c” à “resultado”. A função “soma”, por sua vez, faz uso das variáveis “soma_a”, soma_b” e soma_c” para realizar a adição e armazenamento de resultado.

Figura 12.6. Exemplo de código Assembly para ilustrar chamada de função com suporte de variáveis globais.

Linha	Código
1	<code>.code</code>
2	<code>main:</code>
3	<code> LDA var1 ; soma_a = var1.</code>
4	<code> STA soma_a</code>
5	
6	<code> LDA var2 ; soma_b = var2.</code>
7	<code> STA soma_b</code>
8	
9	<code> CALL soma ; chama a funcao soma.</code>
10	
11	<code> LDA soma_c</code>
12	<code> STA resultado ; resultado = soma_c.</code>
13	
14	<code>end:</code>
15	<code> INT exit</code>
16	
17	<code>soma:</code>
18	<code> LDA soma_a</code>
19	<code> ADD soma_b</code>
20	<code> STA soma_c</code>
21	<code> RET</code>
22	
23	<code>.data</code>
24	<code> ; syscall exit</code>
25	<code>exit: DD 25</code>
26	<code>var1: DD 10</code>
27	<code>var2: DD 20</code>
28	<code>resultado: DD 0</code>
29	<code>soma_a: DD 0</code>
30	<code>soma_b: DD 0</code>
31	<code>soma_c: DD 0</code>
32	
33	<code>.stack 10</code>

Fonte: Próprio autor (2021).

Apesar da simplicidade, a abordagem de passagem de parâmetros e retorno de resultado via variáveis globais possui limitações. Além da sobrecarga da memória pela alocação de variáveis globais em excesso, por conta dos parâmetros serem passados em variáveis específicas, somente é possível ter uma instância da função em execução, para não haver sobreposição dos valores dos parâmetros nas variáveis globais, o que, desta maneira, inviabiliza a implementação de recursos mais avançados, tais como a recursividade.

12.2.2 Registradores

Com o surgimento de processadores com um maior número de registradores, como é o caso do processador RISC-V que possui 32 registradores de propósito geral (PATTERSON; HENNESSY, 2017), surgiu a possibilidade de passagem de parâmetros e de retorno de resultado via registradores.

A Figura 12.7 ilustra um programa para adição descrito em linguagem C, onde a função “soma” recebe os parâmetros inteiros “a” e “b” e retorna o resultado da adição, como pode ser visto nas Linhas 5 à 7. Já a função “main”, definida nas Linhas 9 à 11, utiliza as variáveis globais “var1”, “var2”, como parâmetros de entrada, e “resultado”, para armazenar o resultado, na chamada da função “soma”.

Figura 12.7. Exemplo de código C para ilustrar chamada de função com suporte de registradores.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2 = 20;</code>
3	<code>int resultado = 0;</code>
4	
5	<code>int soma (int a, int b){</code>
6	<code> return a + b;</code>
7	<code>}</code>
8	
9	<code>void main (void) {</code>
10	<code> resultado = soma(var1, var2);</code>
11	<code>}</code>

Fonte: Próprio autor (2021).

O código *Assembly* da Figura 12.8 traz uma implementação da aplicação descrita em linguagem C, da Figura 12.7. No código da Figura 12.8, ilustra-se o uso da abordagem de passagem de parâmetros de entrada e retorno de resultado com suporte de registradores.

No CompSim, o processador Cariri possui apenas um registrador de propósito geral, que é o registrador acumulador AC. Assim, visando simular uma arquitetura com maior número de registradores, definiu-se, nas Linhas 29 e 30, as variáveis globais “R0” e “R1” que simularão, em memória, dois registradores de propósito geral. O objetivo aqui é mostrar como se dá o comportamento da abordagem que faz uso de registradores para implementação da passagem de parâmetros e retorno de resultado em chamadas de funções.

Na Figura 12.8, analisando a função “main”, definida nas Linhas 2 à 15, observa-se que a passagem de parâmetros, na chamada da função “soma”, consiste em copiar o valores das variáveis “var1” e “var2” para os pseudo-registradores “R0” e “R1”, respectivamente

(Linhas 3 à 7). A função “soma”, definida nas Linhas 16 à 20, por sua vez, realiza a adição dos valores contidos nos pseudo-registradores “R0” e “R1”, e armazena o resultado em “R0”. Após o retorno da função “soma”, na função “main” o resultado da adição é copiado do pseudo-registrador “R0” para a variável “resultado”, como mostram as Linhas 11 e 12.

Figura 12.8. Exemplo de código Assembly para ilustrar chamada de função com suporte de registradores.

Linha	Código
1	<code>.code</code>
2	<code>main:</code>
3	<code> LDA var1 ;R0 = var1.</code>
4	<code> STA R0</code>
5	
6	<code> LDA var2 ;R1 = var2.</code>
7	<code> STA R1</code>
8	
9	<code> CALL soma ;chama a funcao soma.</code>
10	
11	<code> LDA R0</code>
12	<code> STA resultado ;resultado = R0.</code>
13	
14	<code>end:</code>
15	<code> INT exit</code>
16	
17	<code>soma:</code>
18	<code> LDA R0 ;R0 = R0 + R1.</code>
19	<code> ADD R1</code>
20	<code> STA R0</code>
21	<code> RET</code>
22	
23	<code>.data</code>
24	<code> ;syscall exit</code>
25	<code>exit: DD 25</code>
26	<code>var1: DD 10</code>
27	<code>var2: DD 20</code>
28	<code>resultado: DD 0</code>
29	<code>R0: DD 0 ;simula o registrador R0.</code>
30	<code>R1: DD 0 ;simula o registrador R1.</code>
31	
32	<code>.stack 10</code>

Fonte: Próprio autor (2021).

Analisando o exemplo da Figura 12.8, observa-se que nesta segunda abordagem, dependendo da forma como as funções são implementadas, mesmo com uma grande disponibilidade de registradores, é possível que sejam esgotados na passagem de parâmetros e retorno de resultado e, com isso, sejam sobrescritos entre chamadas de funções, tendo assim, a mesma limitação da abordagem anterior que utiliza variáveis globais.

Visando evitar a sobreposição de valores em registradores, uma função pode fazer uso da pilha do programa para manter o registro dos parâmetros de entrada e de retorno de resultado (registro de contexto da função), de forma que a sobreposição não prejudique a chamada de outras funções. Destaca-se que, após o encerramento da função chamada, os

valores anteriormente armazenados na pilha do programa podem ser recuperados e copiados para os respectivos registradores, restaurando assim o contexto de execução da função chamadora.

A subseção a seguir ilustra a abordagem de passagem de parâmetros e retorno de resultado na chamada de funções que faz uso da pilha do programa.

12.2.3 Pilha do Programa

Entre as abordagens que implementam a passagem de parâmetros e de retorno de resultado em chamadas de funções, a que faz uso da pilha do programa é a mais utilizada, por ser mais flexível e solucionar os problemas apresentados nas abordagens anteriores (desperdício de memória na alocação de variáveis globais adicionais, limitação do número de registradores disponíveis e sobrescrita de valores). Além disso, a pilha do programa pode ser utilizada para agregar mais recursos às funções, como, por exemplo, na definição de variáveis locais, como será visto na seção a seguir.

Por outro lado, a abordagem com uso da pilha do programa faz com que sejam realizados acessos adicionais à memória, visto que a pilha do programa é implementada no segmento de pilha na memória. Desta forma, recomenda-se o uso de uma abordagem híbrida, que faz uso da pilha do programa e de registradores para armazenamento de dados entre chamadas e execução de funções, visando a redução de acessos à memória e, conseqüentemente, a otimização de desempenho na execução de funções (SANTOS; LANGLOIS, 2018). É importante recapitular que, na subseção anterior, a qual trata da abordagem que utiliza registradores para passagem de parâmetros e retorno de resultado, discutiu-se a importância do uso da pilha do programa para o registro do contexto de execução de cada função chamada, para sua recuperação em situações em que há múltiplas chamadas de funções. Nesse sentido, percebe-se que a abordagem híbrida pode oferecer várias vantagens sobre as demais, desde que seja utilizada de forma adequada.

O programa em linguagem *Assembly* da Figura 12.9 traz uma nova implementação do código em linguagem C da Figura 12.7, sendo que, desta vez, utiliza-se a abordagem de passagem de parâmetros de entrada e retorno de resultado com suporte da pilha do programa. Ainda no código da Figura 12.9, define-se a variável global “R0” na Linha 40, a qual será utilizada como um registrador de propósito geral auxiliar o registrador acumulador AC. O

objetivo é ilustrar como a pilha do programa e os registradores podem ser devidamente empregados na abordagem híbrida.

Figura 12.9. Exemplo de código Assembly para ilustrar chamada de função com suporte de pilha do programa.

Linha	Código
1	<code>.code</code>
2	<code>main:</code>
3	<code> LDA var2 ;adiciona var2 na pilha do programa.</code>
4	<code> SOP push</code>
5	
6	<code> LDA var1 ;adiciona var1 na pilha do programa.</code>
7	<code> SOP push</code>
8	
9	<code> CALL soma ;chama a funcao soma.</code>
10	
11	<code> SOP pop ;retira 1o parametro da pilha do programa.</code>
12	<code> SOP pop ;retira resultado da adicao da pilha do programa.</code>
13	<code> STA resultado</code>
14	
15	<code>end:</code>
16	<code> INT exit</code>
17	
18	<code>soma:</code>
19	<code> MOV \$SP ;AC recebe o endereço do endereço de retorno da funcao.</code>
20	<code> ADD one</code>
21	<code> STA ptr ;prt aponta para o 1o parametro.</code>
22	<code> LDI ptr</code>
23	<code> STA R0 ;R0 recebe o 1o parametro.</code>
24	<code> LDA ptr</code>
25	<code> ADD one</code>
26	<code> STA ptr ;prt agora aponta para o 2o parametro.</code>
27	<code> LDI ptr ;AC recebe o 2o parametro.</code>
28	<code> ADD R0</code>
29	<code> STI ptr ;2o parametro = AC + R0.</code>
30	<code> RET</code>
31	
32	<code>.data</code>
33	<code> ;syscall exit</code>
34	<code>exit: DD 25</code>
35	<code>var1: DD 10</code>
36	<code>var2: DD 20</code>
37	<code>resultado: DD 0</code>
38	<code>push: DD 0</code>
39	<code>pop: DD 1</code>
40	<code>R0: DD 0 ;simula o registrador R0.</code>
41	<code>ptr: DD 0 ;ponteiro auxiliar.</code>
42	<code>one: DD 1 ;variavel auxiliar com valor 1.</code>
43	
44	<code>.stack 10</code>

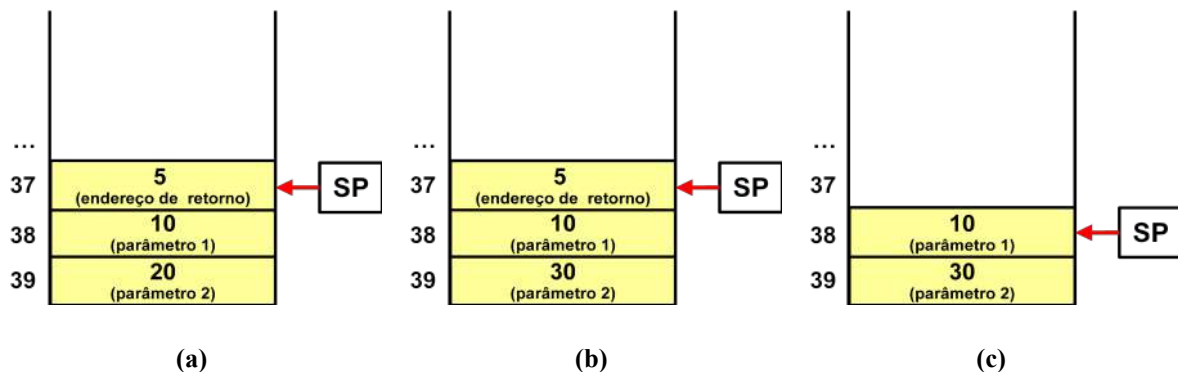
Fonte: Próprio autor (2021).

Na função “main”, que está definida nas Linhas 2 à 16, do código na Figura 12.19, antecedendo a chamada da função “soma” (Linha 9), carrega-se os valores das variáveis “var1” e “var2” na pilha do programa, através da instrução SOP, nas Linhas 4 e 7. Observa-se que primeiro adiciona-se na pilha do programa o valor da variável “var2”, que é o segundo parâmetro da função “soma”, para só então adicionar o valor da variável “var1”, que é

primeiro parâmetro. Essa inversão visa simplificar o tratamento dos parâmetros no corpo da função “soma”, definida nas Linhas 18 à 30.

Ao chamar a função “soma” para execução, na Linha 9, a instrução CALL adiciona o endereço de retorno na pilha e desvia o fluxo e execução para a função. O estado da pilha do programa, após a execução da instrução CALL, está ilustrado na Figura 12.10 (a).

Figura 12.10. Estrutura da pilha do programa após execução da instrução CALL.



Fonte: Próprio autor (2021).

Na Figura 12.10 (a), observa-se que, no topo da pilha (endereço de memória “37”), está o endereço de retorno da função (endereço “5”) e, logo abaixo, nos endereços de memória “38” e “39”, estão os valores do primeiro e segundo parâmetros da função “soma”, valores “10” e “20”, respectivamente.

A função “soma” inicia recuperando o endereço de topo da pilha, contido no registrador apontador de topo da pilha SP (Linha 19). Em seguida, a função incrementa esse endereço e atribui ao ponteiro “ptr” (Linhas 20 e 21), que passa a apontar para o endereço “38”, que consiste no primeiro parâmetro da função. Nas Linhas 22 e 23, o parâmetro é recuperado e copiado ao pseudo-registrador “R0”. Nas Linhas 24 à 26, “ptr” é incrementado novamente para apontar para o segundo parâmetro da função “soma” (endereço “39”). Nas Linhas 27 à 29, o valor do segundo parâmetro é recuperado, adiciona-se a ele o valor contido em “R0” e o resultado é gravado no endereço de memória “39”, referente ao segundo parâmetro, contido na pilha do programa, como está ilustrado na Figura 12.10 (b). Ao fim da função "soma", a instrução RET, na Linha 30, remove o endereço de retorno do topo da pilha (endereço “5”) e faz com que o fluxo de execução do programa seja desviado de volta à rotina principal. O estado da pilha do programa após a execução da instrução RET, está ilustrado na Figura 12.10 (c).

Para recuperar o resultado da adição, nas Linhas 11 e 12, são retirados dois elementos da pilha do programa, sendo que o segundo elemento sobrescreve o primeiro no registrador acumulador AC. O resultado da adição é então armazenado na variável “resultado”, na Linha 13.

O código *Assembly* da Figura 12.9 utilizou uma abordagem híbrida para implementação da aplicação de adição de números inteiros em linguagem C da Figura 12.7. Na solução proposta, utilizou-se a pilha do programa para passagem de parâmetros e retorno de resultado e registradores para apoio à operação de adição, na função “soma”.

12.3 Variáveis Locais

Sabe-se que a pilha do programa é utilizada implicitamente pelas instruções CALL e RET, para guardar e retirar, respectivamente, o endereço de retorno de uma função. Viu-se também na subseção anterior, que a pilha do programa também é utilizada na passagem de parâmetros e retorno de resultado na chamada de uma função.

Além dessas funções, a pilha do programa também pode ser utilizada para compor as variáveis locais de uma função. Uma variável local é definida/declarada em blocos de código, tais como em funções, estruturas condicionais ou de repetição, e possuem tempo de vida que corresponde ao seu tempo de execução (SANTOS; LANGLOIS, 2018).

Como as variáveis definidas e declaradas nos segmentos “.data” e “.bss”, respectivamente, possuem escopo global, ou seja, possuem visibilidade em todos os trechos do código e possuem vida útil até o encerramento da execução do programa, faz-se necessário utilizar outro artifício para a implementação das variáveis locais.

Viu-se, na subseção anterior, que a pilha do programa é utilizada frequentemente para estabelecer um contexto para execução de uma função (escopo local), entre múltiplas chamadas de funções. Nesse sentido, a pilha do programa torna-se então a candidata ideal para implementação de variáveis locais.

De forma prática, para a implementação de variáveis locais utilizando a pilha do programa, a abordagem mais frequente consiste em, após a adição dos parâmetros e do endereço de retorno da função na pilha do programa, via chamada para execução da função, alocar, também na pilha do programa, memória adicional suficiente para suprir as necessidades de armazenamento das variáveis locais.

Por exemplo, no código em linguagem C da Figura 12.11, a função soma, nas Linhas 5 à 9, inclui a definição das variáveis locais “op1” e “op2” (Linhas 6 e 7). Essas variáveis recebem os valores passados como parâmetros da função "soma", para em seguida, realizar a adição e retorno do resultado.

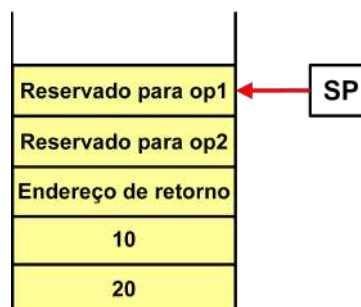
Figura 12.11. Exemplo de código C para ilustrar a criação de variáveis locais em função.

Linha	Código
1	<code>int var1 = 10;</code>
2	<code>int var2 = 20;</code>
3	<code>int resultado = 0;</code>
4	
5	<code>int soma (int a, int b){</code>
6	<code> int op1 = a;</code>
7	<code> int op2 = b;</code>
8	<code> return op1 + op2;</code>
9	<code>}</code>
10	
11	<code>void main (void) {</code>
12	<code> resultado = soma(var1, var2);</code>
13	<code>}</code>

Fonte: Próprio autor (2021).

A Figura 12.12 ilustra um possível estado da pilha do programa, após a chamada da função “soma” para execução, do programa C na Figura 12.11. Na Figura 12.12, pode-se observar, numa perspectiva *bottom-up*: os valores 20 e 10, inseridos na pilha na passagem dos parâmetros “b” e “a”; o endereço de retorno da função, inserido pela instrução CALL; e, os espaços de memória reservados para comportar as variáveis locais “op2” e “op1”.

Figura 12.12. Pilha do programa no contexto da chamada da função soma da Figura 12.11.



Fonte: Próprio autor (2021).

A tarefa de alocação de espaço de memória para as variáveis locais “op1” e “op2” deve ser realizada no início da execução da função “soma”. Além disso, o apontador de topo de pilha, registrador SP, após a criação do contexto de execução da função, que envolve a

alocação de memória para comportar as variáveis locais, apontará para o último elemento inserido na pilha, o qual, neste caso, consistirá em memória alocada para uma variável local.

É importante ressaltar que, durante a execução da função, novas variáveis locais podem ser criadas, tais como em estruturas condicionais ou de repetição, e, novamente, deve-se reservar mais espaço de memória na pilha do programa. Outro ponto importante, é que, como as variáveis locais possuem um tempo de vida limitado, ao encerrar o contexto em que estão inseridas, os espaços reservados para as respectivas variáveis locais devem ser desalocados. No caso de uma função, é necessário que os espaços reservados para as variáveis locais sejam desalocados da pilha do programa para que o endereço de retorno da função esteja disponível no topo da pilha para uso pela instrução RET.

Segundo Santos e Langlois (2018), há três formas de tratar os espaços reservados para variáveis locais na pilha do programa, que são:

1. Deslocamentos variáveis: esta técnica utiliza deslocamentos, em relação ao apontador de topo de pilha do programa SP, para fazer acesso às variáveis locais e aos valores passados como parâmetros da função. Por exemplo, pode-se utilizar um ponteiro para receber o endereço no registrador apontador de topo de pilha SP, e, com deslocamentos do ponteiro, realizar acessos às variáveis locais e valores passados como parâmetros da função;
2. Registrador Apontador de Quadro (*Frame Pointer*): utiliza-se um registrador adicional, chamado de *Frame Pointer*, ou FP, o qual aponta para o endereço de retorno na pilha do programa. Com FP, é possível realizar deslocamentos e obter acesso tanto às variáveis locais quanto aos valores passados como parâmetros da função, em vez de utilizar o registrador de apontador de topo de pilha do programa SP; e
3. Espaços temporários: determina-se o número máximo de variáveis temporárias que irão ser necessárias durante a execução da função e reserva-se o espaço máximo necessário. Com isso, cada variável terá uma posição predeterminada na pilha e, por consequência, os deslocamentos para cada uma delas será fixo.

O código *Assembly* na Figura 12.13 ilustra como podem ser implementadas as variáveis locais da função soma do programa C da Figura 12.11, utilizando a técnica de deslocamentos variáveis. O código utiliza também a pilha do programa para implementar a passagem de parâmetros e retorno de resultado da função “soma”.

Da mesma forma que na solução apresentada no código da Figura 12.9, a rotina inicial “main” do programa *Assembly* na Figura 12.13, adiciona, em ordem invertida, os valores das

variáveis “var1” e “var2” na pilha do programa para compor os parâmetros da função “soma” (Linhas 3 à 7). Após a execução da função “soma”, retira-se dois elementos da pilha do programa (Linhas 11 e 12), em que o segundo deles possui o resultado da adição, sendo então atribuído à variável “resultado”, na Linha 13.

A implementação da função “soma”, nas Linhas 18 à 61, da Figura 12.13, inicia deslocando o apontador de topo da pilha SP em duas posições de memória para reservar espaço para as variáveis locais (Linhas 19 à 22). O ponteiro “ptr_lvar” recebe o valor final do deslocamento e será utilizado para referenciar a memória alocada para as variáveis locais. Nas Linhas 25 à 29, o ponteiro “ptr” é configurado para apontar para o primeiro parâmetro da função na pilha do programa. Nas Linhas 31 e 32, o valor do primeiro parâmetro é copiado para a posição de memória referente à primeira variável local, utilizando os ponteiros “ptr_lvar” e “ptr”. Nas Linhas 34 à 40, os ponteiros “ptr_lva” e “ptr” são incrementados para apontarem para a posição de memória referente à segunda variável local e para o segundo parâmetro da função, respectivamente; e, nas Linhas 42 e 43, o valor referente ao segundo parâmetro é copiado para endereço de memória referente à segunda variável local. Nas Linhas 45 à 54: o valor da segunda variável local, apontado por “ptr_lvar”, é copiado para o pseudo-registrador “R0”; “ptr_lvar” é incrementado logo em seguida para apontar para a primeira variável local; o valor da primeira variável local é carregado do acumulador e somado ao valor em “R0”, sendo o resultado gravado na região de memória referente ao segundo parâmetro da função, que é apontado por “ptr”. Por fim, nas linhas 56 à 59, desalocam-se os espaços de memória anteriormente alocados para comportar as variáveis locais, atualizando o endereço contido no registrador apontador de topo de pilha SP (SP aponta agora para o endereço de retorno da função).

Figura 12.13. Exemplo de código Assembly para ilustrar a criação de variáveis locais em função.

Linha	Código
1	<code>.code</code>
2	<code>main:</code>
3	<code> LDA var2</code> <i>;adiciona var2 na pilha do programa.</i>
4	<code> SOP push</code>
5	
6	<code> LDA var1</code> <i>;adiciona var1 na pilha do programa.</i>
7	<code> SOP push</code>
8	
9	<code> CALL soma</code> <i>;chama a funcao soma.</i>
10	
11	<code> SOP pop</code> <i>;retira 1o parametro da pilha do programa.</i>
12	<code> SOP pop</code> <i>;retira resultado da adicao da pilha do programa.</i>
13	<code> STA resultado</code>
14	
15	<code>end:</code>

Capítulo 12 - Modularização de Programas

16	INT exit
17	
18	soma:
19	MOV \$SP ;decrementa SP em duas posicoes para alocar memoria
20	SUB one ; para as variaveis locais op1 e op2.
21	SUB one
22	MOV -\$SP ;SP e ptr_lvar apontam para a 1a posição de memoria
23	STA ptr_lvar ; reservada para as variaveis locais.
24	
25	MOV \$SP
26	ADD one ;AC contém o endereco da variavel local op2.
27	ADD one ;AC contém o endereco de retorno da funcao.
28	ADD one ;AC contém o endereco do 1o. parametro da funcao.
29	STA ptr ;prt aponta para o 1o parametro.
30	
31	LDI ptr ;atribui o valor do 1o parametro a 1a variavel local.
32	STI ptr_lvar
33	
34	LDA ptr ;ptr aponta agora para o 2a parametro.
35	ADD one
36	STA ptr
37	
38	LDA ptr_lvar
39	ADD one
40	STA ptr_lvar ;ptr_lvr aponta agora para a 2a variavel local.
41	
42	LDI ptr
43	STI ptr_lvar ;atribui o valor do 2o parametro a 2a variavel local.
44	
45	LDI ptr_lvar
46	STA R0 ;R0 recebe o valor da 2a variavel local.
47	
48	LDA ptr_lvar
49	SUB one
50	STA ptr_lvar ;prt_lvar volta a apontar para a 1a variavel local.
51	
52	LDI ptr_lvar
53	ADD R0
54	STI ptr ;2o parametro = 1a variavel local + R0.
55	
56	MOV \$SP
57	ADD one
58	ADD one ;incrementa SP em duas posicoes para desalocar memoria
59	MOV -\$SP ; das variaveis locais op1 e op2.
60	
61	RET
62	
63	.data
64	;syscall exit
65	exit: DD 25
66	var1: DD 10
67	var2: DD 20
68	resultado: DD 0
69	push: DD 0
70	pop: DD 1
71	R0: DD 0 ;simula o registrador R0.
72	ptr_lvar: DD 0 ;ponteio auxiliar para variaveis locais.
73	ptr: DD 0 ;ponteiro auxiliar.
74	one: DD 1 ;variavel auxiliar com valor 1.
75	
76	.stack 10

Fonte: Próprio autor (2021).

Viu-se no programa *Assembly* anterior que, ao executar uma função que contém a definição de variáveis locais, é necessário reservar espaços de memória na pilha do programa

para comportar as variáveis locais, ao início da execução da função; e, antes de encerrar a execução da função, os espaços alocados anteriormente para as variáveis locais devem ser desalocados da pilha do programa.

Esse processo é custoso, pois realiza acessos adicionais à memória, visto que a pilha do programa é implementada em memória, e, por consequência, aumenta o tamanho e a complexidade do programa. Porém, por outro lado, garante o registro do contexto de execução de uma função.

12.4 Múltiplas Chamadas de Funções

Viu-se, em seções anteriores, que, na função “main” da linguagem de programação C, é possível chamar funções para execução. Além da função “main”, as demais funções também podem realizar múltiplas chamadas de funções para execução. Na linguagem C, como uma função somente pode ser definida globalmente (definidas fora da função “main”), elas possuem visibilidade em todo o código e tempo de vida enquanto o programa estiver em execução.

Curiosidade!
É importante destacar que, ainda nesse contexto de escopo de definição de funções, algumas linguagens de programação, tais como a linguagem Python, permitem a definição de funções dentro dos blocos de definições de funções (funções aninhadas). Porém, neste caso, as funções definidas internamente a outras funções terão visibilidade e tempo de vida local à função onde estão definidas.

Para ilustrar como realizar múltiplas chamadas de funções, o código em linguagem C, da Figura 12.14, apresenta uma aplicação que realiza o incremento do valor de uma variável. No exemplo, a função “main” chama a função “inc_dez”, a qual, por sua vez, realiza 10 chamadas da função “inc”, onde, em cada chamada, o valor da variável “var” é incrementado. Para simplificar a compreensão do processo de múltiplas chamadas, optou-se, no exemplo, por não utilizar passagem de parâmetros, retorno de resultado e variáveis locais nas funções trabalhadas.

Figura 12.14. Exemplo de código C para ilustrar chamadas múltiplas de funções.

Linha	Código
1	<code>int var = 10;</code>
2	<code>int i;</code>
3	
4	<code>void inc (void){</code>
5	<code>var++;</code>
6	<code>}</code>
7	
8	<code>void inc_dez (void){</code>
9	<code>for(i=0; i<10; i++)</code>
10	<code>inc();</code>
11	<code>}</code>
12	
13	<code>void main (void) {</code>
14	<code>inc_dez();</code>
15	<code>}</code>

Fonte: Próprio autor (2021).

O código em linguagem *Assembly* da Figura 12.15 traz uma implementação da aplicação de incremento de variável da Figura 12.14.

Na Figura 12.15, a função “inc_dez”, que está definida nas Linhas 14 à 30, é chamada para execução na Linha 3. A função “inc_dez”, inicia com a atribuição do valor 0 à variável “i”, nas Linhas 16 e 17. Esse passo é importante, pois sempre que a função for chamada a variável “i” será reinicializada, permitindo que a estrutura de repetição, que vem a seguir, seja executada corretamente. As linhas seguintes da função “inc_dez” incluem uma estrutura de repetição For..Do, que realiza a chamando da função “inc”, a cada repetição, em um total de 10 repetições. A função “inc”, que está definida entre as Linhas 8 e 12, simplesmente incrementa o valor da variável “var” e devolve o fluxo de execução do programa à função chamadora “inc_dez”.

Figura 12.15. Exemplo de código Assembly para ilustrar chamadas múltiplas de funções.

Linha	Código
1	<code>.code</code>
2	<code>main:</code>
3	<code>CALL inc_dez ;inc_dez().</code>
4	
5	<code>end:</code>
6	<code>INT exit</code>
7	
8	<code>inc:</code>
9	<code>MOV 1 ;var++.</code>
10	<code>ADD var</code>
11	<code>STA var</code>
12	<code>RET</code>
13	
14	<code>inc_dez:</code>
15	

16	<code>MOV 0</code>	<code>;este bloco: for (i=0; i<10; i++).</code>
17	<code>STA i</code>	
18	<code>for: MOV 10</code>	
19	<code>SUB i</code>	
20	<code>JZ end_for</code>	
21	<code>MOV 1</code>	
22	<code>ADD i</code>	
23	<code>STA i</code>	
24	<code>do:</code>	
25	<code>CALL inc</code>	<code>;inc() .</code>
26		
27	<code>JMP for</code>	
28		
29	<code>end_for:</code>	
30	<code>RET</code>	
31		
32	<code>.data</code>	
33	<code>;syscall exit</code>	
34	<code>exit: DD 25</code>	
35	<code>var: DD 10</code>	
36	<code>i: DD 10</code>	
37		
38	<code>.stack 10</code>	

Fonte: Próprio autor (2021).

O exemplo anterior ilustra, de forma bastante simplificada e sem pretensão de esgotar o assunto, como as chamadas múltiplas de funções podem ser implementadas em *Assembly*. Naturalmente, em aplicações mais complexas, as funções terão estruturas mais elaboradas, com possibilidade de incluírem passagens de parâmetros, retornos de resultados e definição de variáveis locais.

12.5 Funções Recursivas

Nas seções passadas, as aplicações ilustrativas trabalhadas foram estruturadas como funções, as quais faziam chamadas entre si, de maneira sistemática e hierárquica.

Em alguns tipos de problemas, é importante ter funções que se chamem para execução (DEITEL; DEITEL, 2011). Este tipo de função é conhecida como Função Recursiva. Em termos práticos, a definição de uma função recursiva deve conter, pelo menos, uma instrução que a chama.

A Figura 12.16 traz um exemplo de programa em linguagem C que utiliza uma função recursiva para implementar a multiplicação de números inteiros por somas sucessivas. Na figura, a função recursiva de multiplicação está definida nas Linhas 5 à 11, onde os parâmetros de entrada “x” e “y” consistem do multiplicando e multiplicador, respectivamente. Caso o multiplicador “y” seja igual a 1, a função retorna o valor do multiplicando “x”, como mostram as Linhas 6 e 7. Caso o multiplicador seja diferente de 1, a

função retorna a adição do multiplicador “x” com a chamada da função recursiva de multiplicação com parâmetros multiplicador “x” e multiplicando “y - 1”, como pode ser visto nas Linhas 8 e 9.

Figura 12.16. Exemplo de código C para ilustrar a multiplicação recursiva.

Linha	Código
1	<code>int multiplicando = 5;</code>
2	<code>int multiplicador = 10;</code>
3	<code>int resultado;</code>
4	
5	<code>int multiplicacao (int x, int y){</code>
6	<code> if (y==1)</code>
7	<code> return x;</code>
8	<code> else{</code>
9	<code> return x + multiplicacao (x, y-1);</code>
10	<code> }</code>
11	<code>}</code>
12	
13	<code>void main (void) {</code>
14	<code> resultado = multiplicacao (multiplicando, multiplicador);</code>
15	<code>}</code>

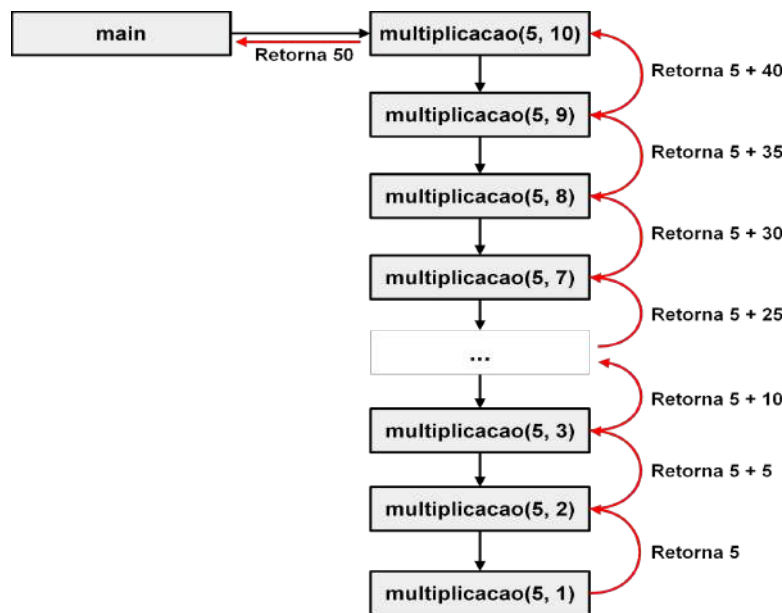
Fonte: Próprio autor (2021).

No exemplo da Figura 12.16, a função recursiva “multiplicacao” é utilizada para multiplicar o valor 5 por 10, armazenados nas variáveis globais “multiplicando” e “multiplicador”, definidas nas Linhas 1 e 2, respectivamente. A Figura 12.17 ilustra a dinâmica das chamadas recursivas da função de multiplicação.

Inicialmente, a função “main” chama a função “multiplicacao” passando os valores 5 e 10, como parâmetros. Ao verificar que o valor do parâmetro “y” não é igual a 1, a função “multiplicacao” chama a si mesma passando os valores 5 e 9 como parâmetros. Da mesma forma, a função “multiplicacao” recém-chamada verifica que o valor do parâmetro “y” não é igual a 1, então, em seguida, chama a si mesma passando os valores 5 e 8, como parâmetros.

Essa dinâmica de chamadas recursivas da função “multiplicacao” segue até que o parâmetro “y” seja igual a 1, fazendo com que a função retorne o valor 5. Com esse retorno, a função “multiplicacao” chamadora, que recebeu os parâmetros 5 e 2, soma o resultado recebido (valor 5) com 5, e retorna um novo resultado para a sua função “multiplicacao” chamadora. Essa dinâmica de retorno de novo resultado segue até a primeira chamada da função “multiplicacao”, que retornará, à função “main”, o resultado 50, sendo atribuído à variável “resultado”, como mostra a Linha 14 da Figura 12.16.

Figura 12.17. Dinâmica de chamadas da função recursiva para multiplicação de 5 por 10.



Fonte: Próprio autor (2021).

O código em linguagem *Assembly* da Figura 12.18 traz uma implementação da aplicação recursiva de multiplicação por soma sucessivas da Figura 12.16. O código utiliza a pilha do programa para implementar a passagem de parâmetros e retorno de resultado, bem como para guardar o contexto de execução de cada chamada recursiva da função “multiplicacao”.

Na figura 12.18, a função “multiplicacao” está definida nas Linhas 17 à 71, que inicia retirando da pilha do programa o endereço de retorno, o primeiro e segundo parâmetros, que são armazenados nos pseudo-registradores “R3”, “R0” e “R1”, respectivamente, como mostram as Linhas 18 à 25. Em seguida, nas Linhas 27 à 30, verifica-se se o parâmetro “y”, contido em “R1”, é igual a 1, e:

- Em caso positivo, o fluxo do programa é desviado para o bloco “return_if”, nas Linhas 64 à 71, que adiciona o parâmetro “x”, contido em “R1”, e o endereço de retorno, contido em “R3”, na pilha do programa, para, em seguida, desviar o fluxo de execução do programa de volta à função chamadora.
- Em caso negativo, o fluxo do programa segue executando o bloco “else”, nas Linhas 32 à 53, que: decrementa o valor do parâmetro “y”, contido em “R1” (Linhas 33 à 35); guarda o contexto atual da função antes de uma nova chamada recursiva da função “multiplicacao”, adicionando na pilha do programa o endereço de retorno, contido em “R3”, o parâmetro “x”, contido em “R0”, e o novo valor do parâmetro “y”, como mostram as Linhas 37 à 44; e chama recursivamente a função

“multiplicacao” (Linha 46). Quando a função “multiplicacao” encerra, retira-se da pilha do programa o retorno do resultado, adiciona-o ao parâmetro “x”, contido em “R0”, e o resultado desta adição é gravado no pseudo-registrador “R2” (Linhas 48 à 50). Em seguida, retira-se da pilha do programa o endereço de retorno, recuperando assim o contexto de execução da função. O bloco “else”, da função “multiplicacao”, encerra com a execução do bloco “return_else”, nas Linhas 64 e 71, o qual adiciona na pilha do programa o resultado da adição em “R0” e o endereço de retorno da função, bem como, através da instrução RET, na Linha 71, devia o fluxo de execução de volta à função chamadora.

Figura 12.18. Exemplo de código Assembly para ilustrar a multiplicação recursiva.

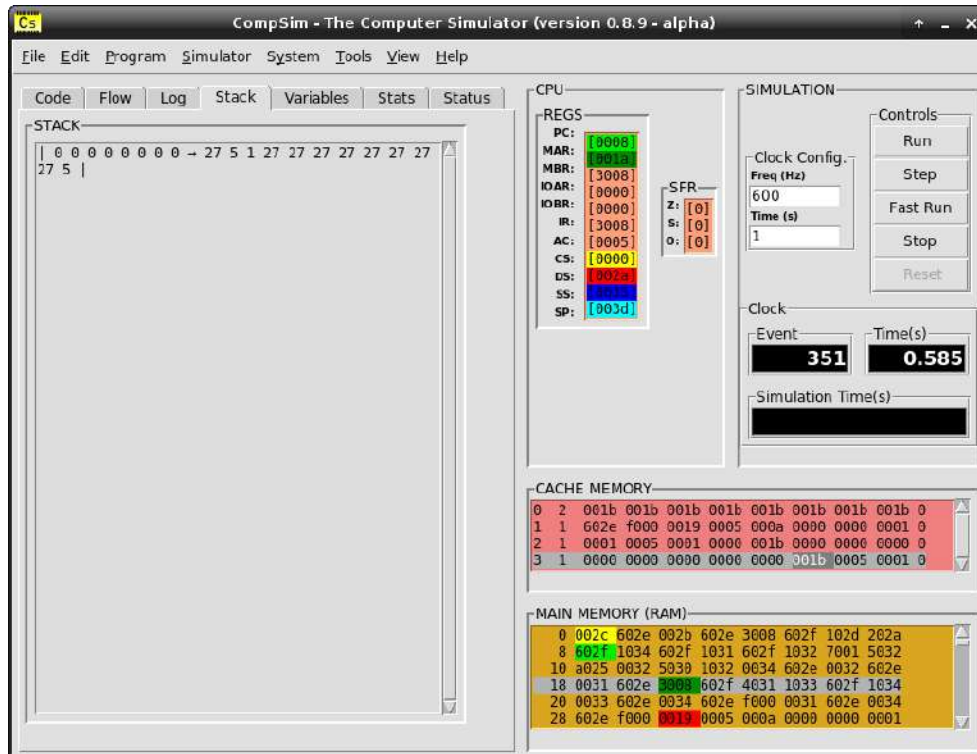
Linha	Código
1	.code
2	main:
3	LDA multiplicador
4	SOP push
5	
6	LDA multiplicando
7	SOP push
8	
9	CALL multiplicacao ;AC=multiplicacao(multiplicando,multiplicador).
10	SOP pop
11	
12	STA resultado ;resultado=AC.
13	
14	end:
15	INT exit
16	
17	multiplicacao:
18	SOP pop
19	STA R3 ;guarda endereco retorno.
20	
21	SOP pop
22	STA R0 ;guarda x em R0.
23	
24	SOP pop
25	STA R1 ;guarda y em R1.
26	
27	if:
28	MOV 1 ;if (y == 1).
29	SUB R1
30	JZ return_if
31	
32	else:
33	LDA R1
34	SUB one
35	STA R1 ;y = y - 1.
36	
37	LDA R3 ;guarda endereco de retorno na pilha.
38	SOP push
39	
40	LDA R1 ;guarda y na pilha.
41	SOP push
42	
43	LDA R0 ;guarda x na pilha.

44	SOP push	
45		
46	CALL multiplicacao	<i>;mult(x, y-1).</i>
47		
48	SOP pop	
49	ADD R0	
50	STA R2	<i>;x + multiplicacao (x, y-1).</i>
51		
52	SOP pop	
53	STA R3	<i>;recupera endereco de retorno.</i>
54		
55	return_else:	
56	LDA R2	<i>;adiciona x + multiplicacao (x, y-1) na pilha.</i>
57	SOP push	
58		
59	LDA R3	<i>;adiciona endereço de retorno na pilha.</i>
60	SOP push	
61		
62	RET	
63		
64	return_if:	
65	LDA R0	<i>;adiciona x na pilha.</i>
66	SOP push	
67		
68	LDA R3	<i>;adiciona endereço de retorno na pilha.</i>
69	SOP push	
70		
71	RET	
72		
73	.data	
74		<i>;syscall exit</i>
75	exit: DD 25	
76	multiplicando: DD 5	
77	multiplicador: DD 10	
78	resultado: DD 0	
79	push: DD 0	
80	pop: DD 1	
81	one: DD 1	
82		
83	.bss	
84	R0: RESD 1	<i>;simulam registradores R0, R1, R2 e R3.</i>
85	R1: RESD 1	
86	R2: RESD 1	
87	R3: RESD 1	
88		
89	.stack 20	

Fonte: Próprio autor (2021).

A Figura 12.19 apresenta, no componente “STACK” da aba “Stack”, o *status* da pilha do programa durante a execução da última chamada recursiva da função “multiplicacao”. Na pilha do programa, vê-se, da esquerda para direita, o endereço de retorno 27, que está no topo da pilha, o valor 5, referente ao parâmetro “x”, o valor 1, referente ao parâmetro “y”, o valor 27, seguidas vezes, que consistem dos endereços de retorno das chamadas recursivas anteriores da função “multiplicacao” (contextos de execução salvos), e o valor 5, referente ao parâmetro “x”, da primeira chamada da função “multiplicacao”.

Figura 12.19. Visualização da pilha do programa da aplicação de multiplicação recursiva.



Fonte: Próprio autor (2021).

12.6 Resumo

Este capítulo tratou de modularização de programas *Assembly*, pela definição e uso de funções. Viu-se, ao longo das seções, que há diversos aspectos a serem considerados na definição de funções, que vão desde a estrutura da função (se necessita de passagem de parâmetros de entrada, retorno de resultado e definição de variáveis locais) até os aspectos computacionais utilizados para suportar a definição da função, tais como uso de pilha do programa e registradores, bem como alocação de memória.

Ao longo do capítulo, foram abordados diversos exemplos práticos simples relacionados à definição e chamada de funções, tais como passagem de parâmetros, retorno de resultado, definição de variáveis locais, múltiplas chamadas de funções e funções recursivas, utilizando o simulador CompSim.

CAPÍTULO 13 – ENTRADA/SAÍDA

Como foi visto no [Capítulo 4 - Introdução à Organização de Computadores](#), a interação com o computador se dá através dos periféricos. Viu-se que há diferentes tipos de periféricos, onde cada tipo possui características, funcionalidades e tecnologias diferentes, e que a comunicação entre o processador e os periféricos se dá pelo Barramento de Periféricos e pelo Subsistema de Entrada/Saída.

O simulador CompSim inclui tanto um Barramento de Periféricos quanto um Subsistema de Entrada/Saída, na plataforma Mandacaru. O Subsistema de Entrada/Saída do CompSim permite a conexão automatizada de novos periféricos ao barramento de periféricos, bastando que o periférico implemente a interface de comunicação padrão com o barramento.

Quanto aos periféricos, no CompSim, é possível ter periféricos de dois tipos: 1) virtual, que são aqueles implementados em software e emulam o comportamento de periféricos reais; e 2) físico, que são divididos em software, para implementar a interface padrão com o barramento de periféricos, e em hardware para compor o periférico físico em si e seu comportamento.

Este capítulo apresenta toda a dinâmica de operações de entrada/saída no CompSim. Serão detalhados: o processo de instalação de periféricos; o funcionamento do Subsistema de Entrada/Saída; e, a formação de interação com diferentes tipos de periféricos virtuais e físicos, através de várias aplicações práticas ilustrativas.

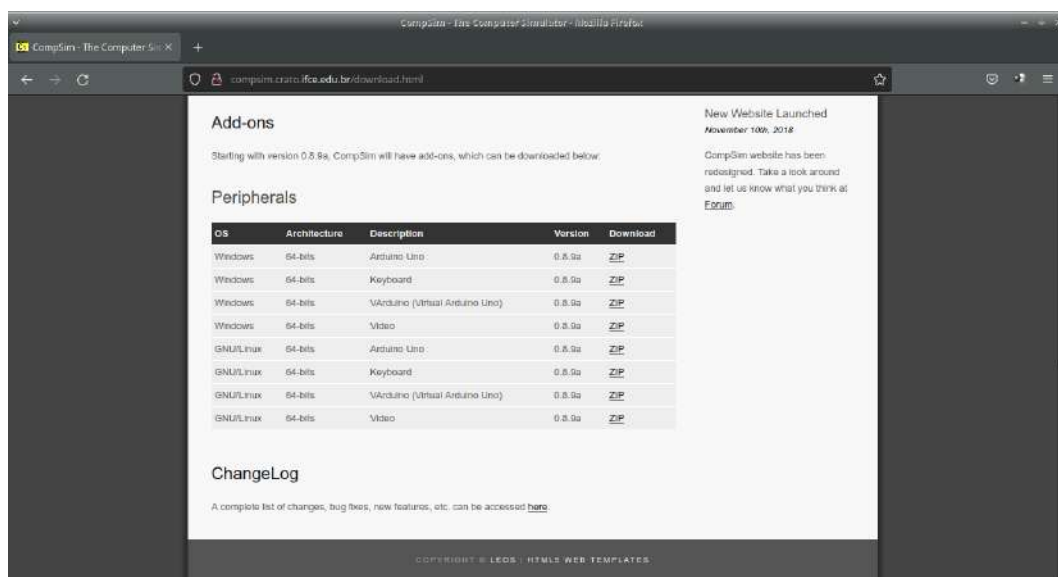
13.1 Instalação de Periféricos no CompSim

O simulador CompSim, ao ser instalado (ver [Capítulo 6 - Instalação do Simulador CompSim](#)), apresenta apenas os componentes principais do computador, que são o processador, memórias RAM e Cache, barramentos e subsistema de entrada/saída, não apresentando, por padrão, periféricos.

Assim, para que seja possível interagir com a plataforma de hardware virtual do CompSim, deve-se instalar os periféricos ao sistema computacional em desenvolvimento para simulação.

Assim como na instalação do simulador CompSim, a instalação de periféricos é bastante simples. O primeiro passo consiste em acessar o website do projeto CompSim¹³ (COMPSIM, 2021a). Em seguida, no website, acessar a seção “Download”, a qual ao fim da página, contém uma tabela com título “Peripherals”. Esta tabela inclui informações sobre os periféricos oficialmente suportados e disponíveis para instalação no simulador CompSim. Entre as informações estão: sistema operacional e arquitetura suportados, uma descrição do periférico, versão da implementação e *link* para *download*, como pode ser visto na Figura 13.1. Dentre os periféricos atualmente disponíveis estão: Video, Teclado (*Keyboard*), Arduino UNO (KURNIAWAN, 2017) e VArduino (*Virtual Arduino UNO*). Observa-se aqui que, os periféricos estão disponíveis apenas para a versão 0.8.9a do CompSim.

Figura 13.1. Página de download do website do Projeto CompSim.



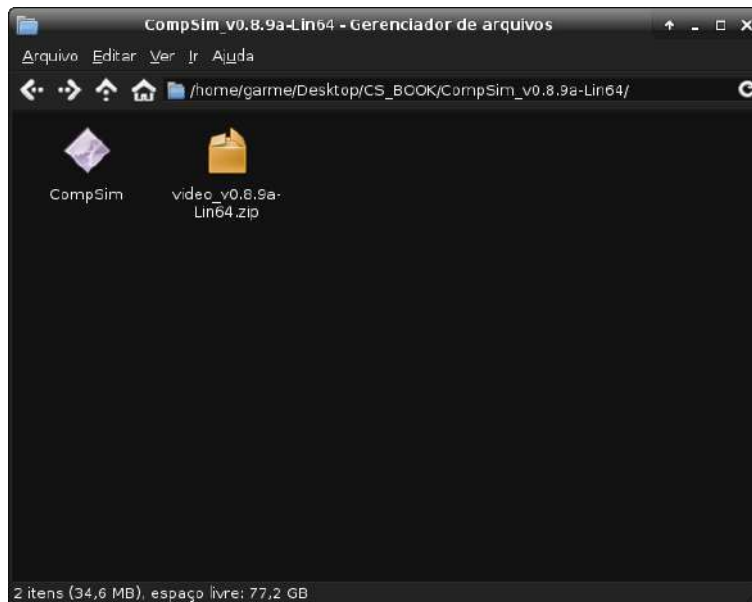
Fonte: Próprio autor (2021).

Será ilustrada aqui a instalação do periférico Video, para sistema operacional GNU/Linux e arquitetura 64-bits. Contudo, o processo é semelhante para instalação em outros sistemas operacionais e arquiteturas. Uma vez escolhido o periférico, o próximo passo consiste em realizar o download de seu arquivo de instalação, clicando no link “ZIP”, na respectiva linha do periférico escolhido, na coluna “Download” (coluna mais à direita na

¹³ CompSim - The Computer Simulator. Disponível em: <<http://compsim.crato.ifce.edu.br/>>. Acesso em: 01 abr. 2021.

tabela). O arquivo de instalação do periférico deve ser salvo no mesmo diretório onde está localizado o arquivo executável do simulador CompSim. A Figura 13.2 mostra que fez-se o *download* do arquivo “video_v0.8.9a-Lin64.zip”, referente ao periférico Vídeo para GNU/Linux 64-bits, no diretório em que encontra-se o arquivo executável do simulador CompSim.

Figura 13.2. Download de periférico do simulador CompSim.

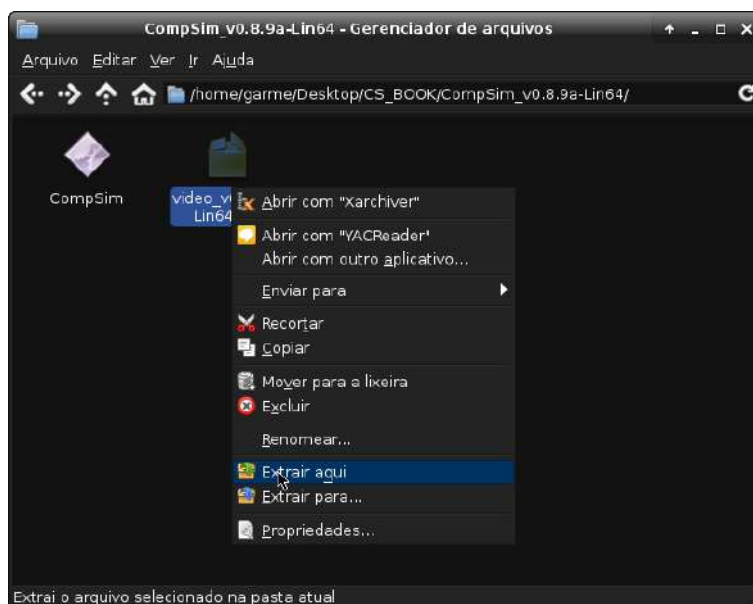


Fonte: Próprio autor (2021).

Após o *download*, o próximo passo consiste em descompactar o arquivo “video_v0.8.9a-Lin64.zip”, no mesmo diretório onde encontra-se o arquivo de execução do simulador CompSim, como mostra a Figura 13.3.

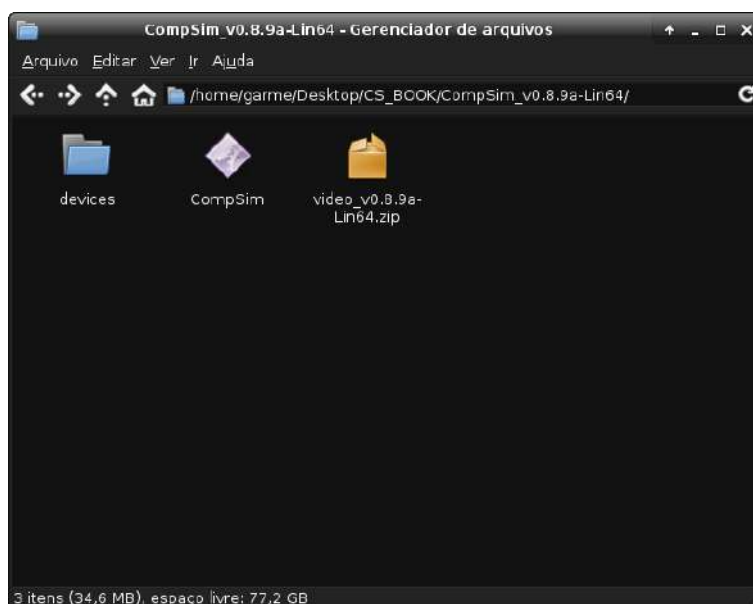
Ao descompactar o arquivo “video_v0.8.9a-Lin64.zip”, será criado o diretório “devices”, como mostra a Figura 13.4, o qual conterà todos os periféricos do CompSim, na medida em que forem sendo instalados. No exemplo de instalação do periférico Vídeo, o diretório “devices” conterà o diretório “video”, que inclui os arquivos necessários para criar e conectar o periférico ao barramento de periféricos da plataforma de hardware virtual do CompSim.

Figura 13.3. Instalação de periférico do simulador CompSim.



Fonte: Próprio autor (2021).

Figura 13.4. Diretório de periféricos do simulador CompSim.

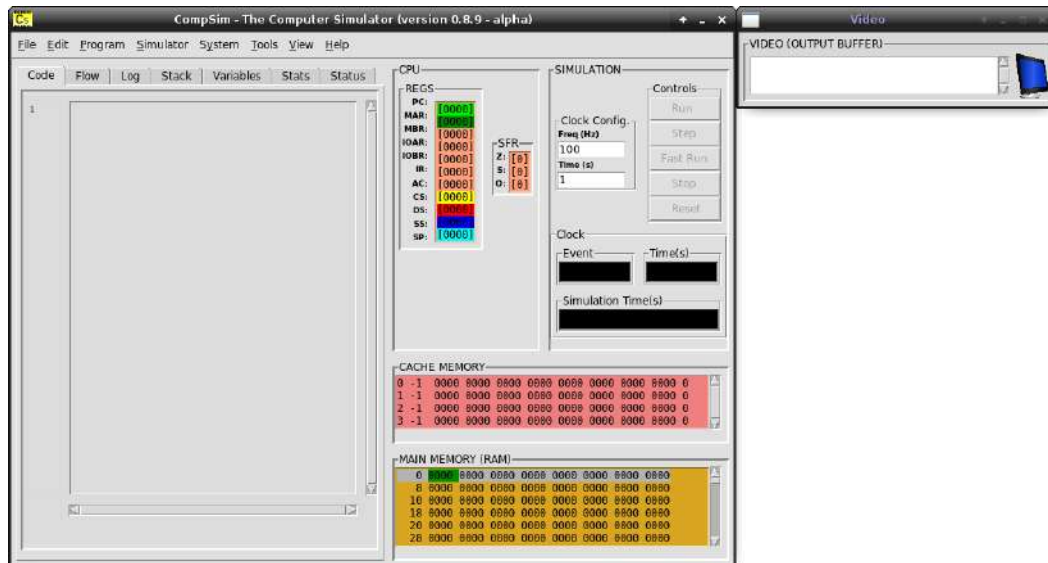


Fonte: Próprio autor (2021).

Após esses passos, o periférico Video já estará instalado. Para verificar se a instalação foi concluída com sucesso, basta executar o simulador CompSim e criar uma nova plataforma de hardware virtual, acessando o menu “Simulator”, opção “New platform” (ou pressionando a tecla “F7”) e escolher entre a opção de criação “Default” ou “Customize”. Ao criar a plataforma de hardware virtual, o periférico Video também será criado e conectado automaticamente ao barramento de periféricos.

A Figura 13.5 mostra a interface gráfica do simulador CompSim com uma janela adicional referente ao periférico Video, após criação de uma plataforma virtual de hardware.

Figura 13.5. Simulador CompSim com periférico Video.



Fonte: Próprio autor (2021).

13.2 Subsistema de Entrada/Saída

O Subsistema de Entrada/Saída, presente na plataforma de hardware virtual do CompSim, permite que novos periféricos sejam conectados automaticamente ao barramento de periféricos. Para tanto, o projeto de cada periférico deve incluir pelo menos dois arquivos:

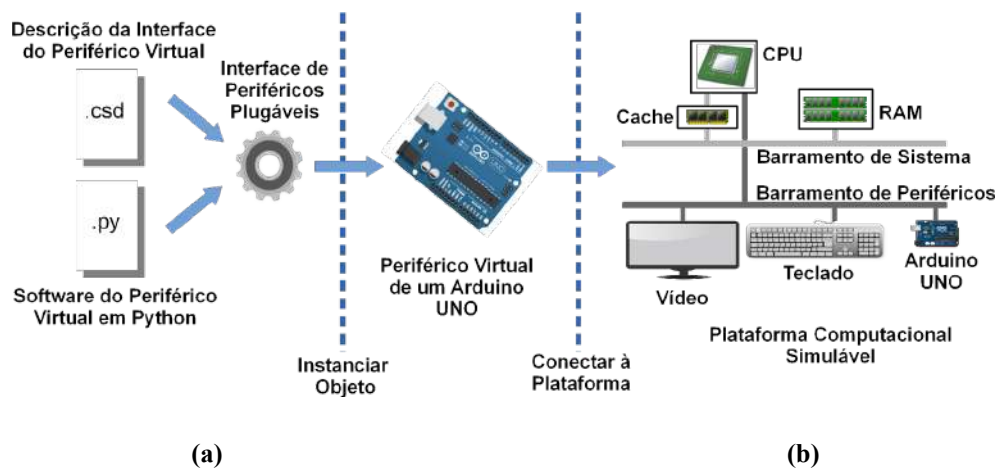
1. O primeiro consiste de arquivo com extensão “.csd” (que significa *CompSim Device*) que contém uma especificação da interface do periférico, a qual inclui as seguintes definições: número(s) da(s) porta(s) de entrada/saída; dados para instanciar o componente de software (nome do pacote e da classe de software Python); e uma curta descrição textual do respectivo periférico; e
2. O segundo arquivo inclui o programa que será utilizado para implementar o comportamento do periférico. O programa pode ser um código-fonte com extensão “.py”, ou o respectivo código binário compilado “.pyc”, o qual deve ser descrito na linguagem de programação Python v3.x, para manter compatibilidade com o CompSim.

Desta forma, ao se criar uma plataforma computacional no simulador CompSim, seu Subsistema de Entrada/Saída buscará, no diretório “devices”, pelos arquivos de descrição de

interface de componente (arquivos “.csd”) e, de forma automatizada, irá instanciar os respectivos componentes de software (contidos nos arquivos “.py” ou “.pyc”), que serão conectados ao barramento virtual de periféricos (essas atividades estão ilustradas na Figura 13.6 (a) e (b), respectivamente). Com isso, os novos periféricos já estarão prontos para interação com os demais componentes de hardware do sistema computacional simulado.

No exemplo da Figura 13.6, a partir dos arquivos “.csd” e “.py”, instancia-se o periférico virtual Arduino UNO (a), o qual, em seguida, é conectado à plataforma de hardware virtual do CompSim (b).

Figura 13.6. Criação de um periférico plugável e conexão à Plataforma de Hardware Virtual do CompSim.



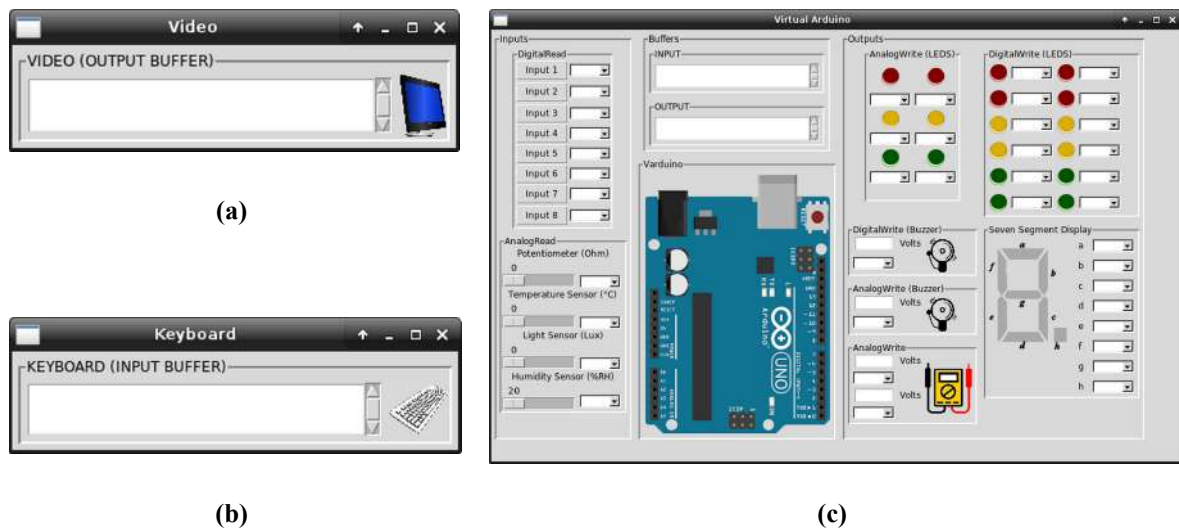
Fonte: Próprio autor (2021).

13.3 Tipos de Periféricos

Atualmente, o CompSim conta oficialmente com duas categorias de dispositivos de entrada/saída: 1) virtuais, que são aqueles desenvolvidos apenas em software e emulam o comportamento de periféricos reais, e 2) físicos, que incluem uma interface de software, para integração com CompSim, e componentes físicos de hardware, utilizados para confeccionar um periférico real.

Entre os dispositivos virtuais de entrada/saída oficiais, há Teclado (*Keyboard*), Video e Arduino UNO, como podem ser vistos nas Figura 13.7 (a), (b) e (c), respectivamente.

Figura 13.7. Periféricos virtuais do CompSim.

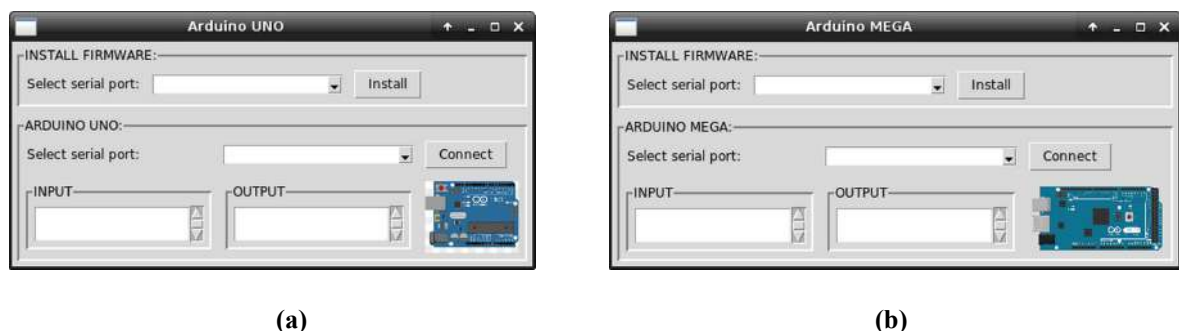


Fonte: Próprio autor (2021).

O periférico Video, ilustrado na Figura 13.7 (a), é utilizado para impressão de caracteres (saída de dados) pela aplicação, enquanto que o periférico Teclado, ilustrado na Figura 13.7 (b), é utilizado para entrada de caracteres, durante uma simulação. O Arduino UNO virtual, também chamado de "VArduino", apresentado na Figura 13.7 (c), é composto por um Arduino UNO virtual e por diferentes componentes gráficos que emulam dispositivos eletrônicos reais. Nesses componentes, a interface gráfica inclui campos de texto para exibir os estados dos *buffers* de entrada (*input*) ou saída (*output*), que contêm os dados lidos ou escritos durante as operações de comunicação.

Entre os dispositivos físicos de entrada/saída oficiais, há um Arduino UNO e Arduino MEGA (KURNIAWAN, 2017), como podem ser vistos nas Figura 13.8 (a) e (b), respectivamente. No entanto, até a escrita deste livro, o componente Arduino MEGA ainda não encontra-se disponível para *download* no website do projeto CompSim.

Figura 13.8. Periféricos físicos do CompSim.



Fonte: Próprio autor (2021).

Os dispositivos físicos de entrada/saída, como citado anteriormente, são divididos em dois submódulos, sendo que: o primeiro implementa, em software, o Módulo de Entrada/Saída e permite integração do periférico ao barramento de periféricos da plataforma virtual; e o segundo submódulo é o periférico propriamente dito. Os periféricos físicos consistem das respectivas placas de circuitos físicos (*boards*) das plataformas abertas de prototipação Arduino UNO e Arduino MEGA e, dependendo do periférico, outros componentes eletrônicos, tais como leds, botões, motores, displays, entre outros.

13.4 Operações de Entrada/Saída

Na atual versão do CompSim (v0.8.9a), dentre as técnicas básicas de entrada/saída, apenas a técnica de comunicação de entrada/saída programada por espera ocupada é suportada, pois a versão atual do processador Cariri não possui suporte ao tratamento de interrupções e ainda não há disponibilidade de um periférico controlador de DMA (*Direct Memory Access*) (o [Capítulo 5 - Introdução à Arquitetura de Computadores](#) descreve cada uma dessas técnicas de entrada/saída).

O processador Cariri, presente na plataforma virtual de hardware do CompSim, possui, em seu conjunto de instruções da arquitetura, a instrução “INT”, a qual, até este ponto deste livro, somente havia sido utilizada com parâmetro 25 para encerrar a execução do programa (HALT).

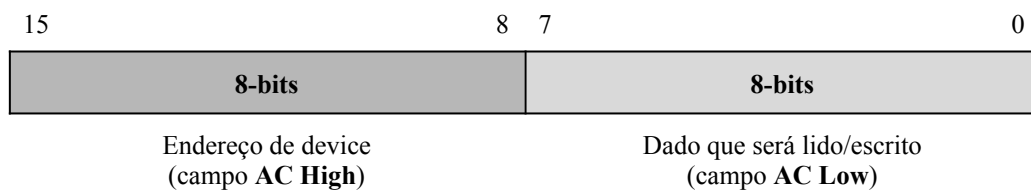
Entretanto, a instrução INT também pode ser utilizada em operações de entrada/saída em periféricos da plataforma virtual de hardware do CompSim. Para tanto, a instrução necessita, como parâmetros, dos códigos de operação 20 e 21 para leitura e escrita de dados em um determinado periférico, respectivamente.

As operações de entrada/saída, com a instrução INT, devem atender às seguintes condições:

1. Em operações de leitura/escrita, os 8-bits mais significativos do registrador acumulador AC serão utilizados para endereçamento de periféricos. Com isso, é possível ter até 256 endereços de periféricos (portas) diferentes ($8\text{-bits} \rightarrow 2^8 = 256$);
2. Em operações de escrita, os 8-bits menos significativos do registrador acumulador AC serão utilizados para guardar o dado (byte) que será escrito em algum periférico; e

3. Em operações de entrada, após a leitura de um dado de periférico, o dado lido estará disponível no registrador acumulador AC, nos 8-bits menos significativos, caso o dado lido seja um byte, ou, nos 16-bits de AC, caso seja um número inteiro. Desta forma, nas operações de entrada/saída, o registrador acumulador AC do processador Cariri passa a incluir os campos “AC High”, que inclui os bits 8 à 15 e é utilizado para endereçamento de porta de periférico, e “AC Low”, que inclui os bits 0 à 7 e é utilizado para transferência de dados, como podem ser vistos na Figura 13.9.

Figura 13.9. Campos do registrador acumulador AC em operações de entrada/saída.



Fonte: Próprio autor (2021).

13.5 Entrada/Saída com Video e Keyboard

Uma operação de entrada/saída entre a plataforma de hardware virtual do CompSim e os periféricos virtuais Video e Teclado, envolve a escrita e a leitura de um *byte*, respectivamente.

As subseções a seguir detalham o comportamento dos periféricos Video e Teclado, bem como traz exemplos práticos ilustrativos de programação e uso.

13.5.1 Video

O periférico de saída Video atua recebendo bytes, que por sua vez são convertidos nos respectivos caracteres ASCII, os quais, em seguida, são impressos no campo “VIDEO (OUTPUT BUFFER)”, da sua interface gráfica.

O código *Assembly* da Figura 13.10 ilustra a escrita do caractere “A” no periférico Video. Na figura, nas Linhas 12, 13 e 14, são definidas as variáveis: “char”, que contém o caractere “A”, que será impresso no Video; “output”, que contém o valor “21”, referente ao código de operação de escrita da instrução INT; e “video_port”, que contém o valor de porta “0x000” do periférico de Vídeo. É importante ressaltar que o valor “0x000” definido para a

porta de vídeo, é especificado no arquivo de interface do periférico vídeo (arquivo “video.csd”). A porta pode ser alterada, bastando alterar o valor do campo “ports” no arquivo “video.csd”.

Na Figura 13.10, na Linha 2, o caractere “A” é carregado no registrador acumulador AC (ocupando os 8-bits menos significativos, campo “AC Low”); em seguida, na Linha 3, adiciona-se ao AC a porta do periférico Video (ficando o endereço da porta no campo “AC High” e o caractere “A” em “AC Low”); e, por fim, na Linha 4, realiza-se a operação de escrita de dados, na qual será impresso o caractere no periférico de Video.

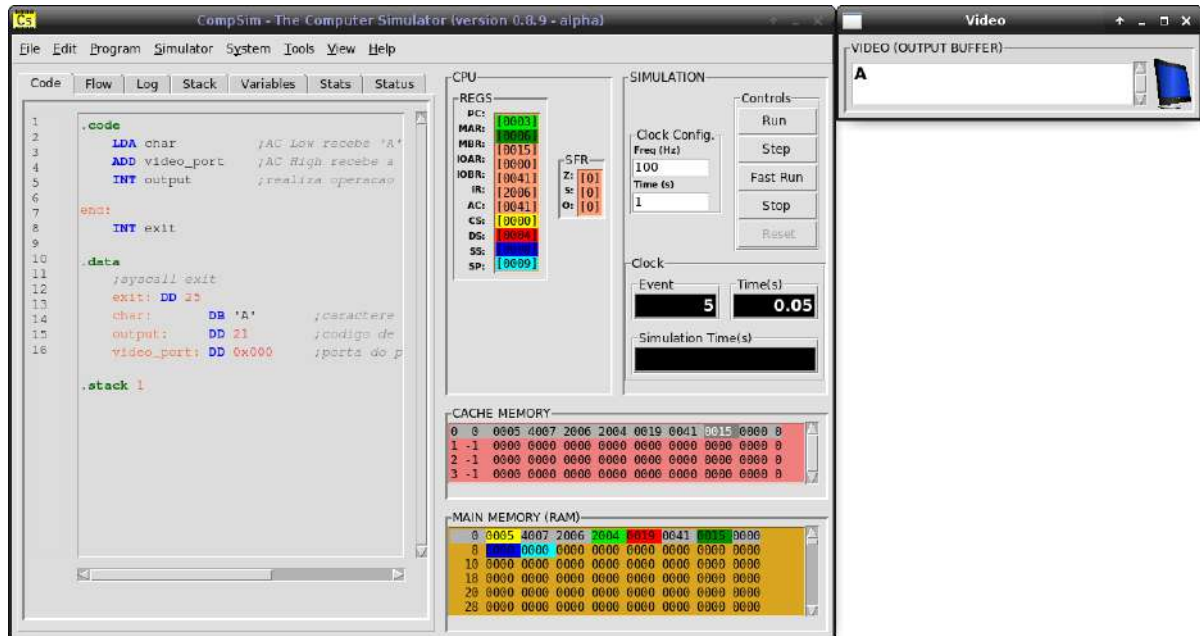
Figura 13.10. Exemplo de código Assembly para escrita de caractere no periférico Video.

Linha	Código
1	<code>.code</code>
2	<code> LDA char ;AC Low recebe 'A'.</code>
3	<code> ADD video_port ;AC High recebe a porta de Video.</code>
4	<code> INT output ;realiza operacao de output.</code>
5	
6	<code>end:</code>
7	<code> INT exit</code>
8	
9	<code>.data</code>
10	<code> ;syscall exit</code>
11	<code> exit: DD 25</code>
12	<code> char: DB 'A' ;caractere que sera impresso em Video.</code>
13	<code> output: DD 21 ;codigo de output da instrucao INT.</code>
14	<code> video_port: DD 0x000 ;porta do periferico Video.</code>
15	
16	<code>.stack 1</code>

Fonte: Próprio autor (2021).

A Figura 13.11, apresenta a visualização do periférico Video, após a operação de escrita, onde observa-se na sua interface gráfica o caractere “A” impresso.

Figura 13.11. Visualização de escrita de caractere no periférico Video.



Fonte: Próprio autor (2021).

A aplicação *Assembly* da Figura 13.12 ilustra como pode ser impressa uma cadeia de caracteres (*string*) no periférico Video. No código da Figura 13.12, a variável “msg”, definida na Linha 23, inclui a *string* “Hello, World!\0”, que será impressa no periférico Video. Para tanto, utiliza-se o ponteiro “ptr”, definido na Linha 25, como estrutura auxiliar para apontar para cada um dos caracteres da *string* em “msg” e, com isso, tem-se acesso a eles para impressão. Nas Linhas 2 à 15, há uma estrutura de repetição While..Do, em que:

- Nas Linhas 3 à 5, verifica-se se o ponteiro “ptr” aponta para o final da *string*. Essa comparação é realizada entre o caractere da *string* apontado por “ptr” e a variável “last_char”, que contém o símbolo de fim de string;
- Nas Linhas 7 à 9, é lido um caractere da *string* para “AC Low”, adiciona-se a porta do periférico Video à “AC High” e realiza-se a operação de escrita, onde o caractere lido da *string* será impresso no periférico Video; e
- Nas Linhas 11 à 13, o endereço contido em “ptr” é incrementado, de forma que “ptr” passará a apontar para o próximo caractere da *string* em “msg”.

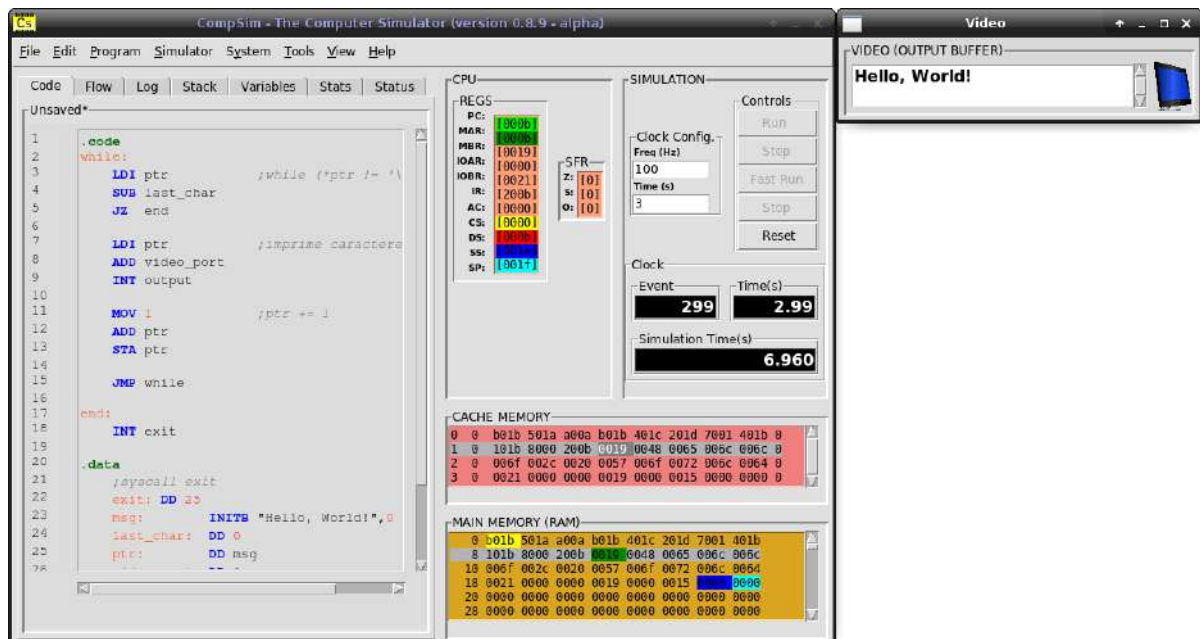
A Figura 13.13 apresenta a visualização do periférico Video, após a execução do programa *Assembly* da Figura 13.12. Na Figura 13.13, vê-se que foi impressa, na interface gráfica no periférico Video, a *string* “Hello, World!”.

Figura 13.12. Exemplo de código Assembly para escrita de cadeia de caracteres no periférico Video.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code>LDI ptr ;while (*ptr != '\0').</code>
4	<code>SUB last_char</code>
5	<code>JZ end</code>
6	<code>do:</code>
7	<code>LDI ptr ;imprime caractere no Video.</code>
8	<code>ADD video_port</code>
9	<code>INT output</code>
10	
11	<code>MOV 1 ;ptr += 1.</code>
12	<code>ADD ptr</code>
13	<code>STA ptr</code>
14	
15	<code>JMP while</code>
16	
17	<code>end:</code>
18	<code>INT exit</code>
19	
20	<code>.data</code>
21	<code>;syscall exit</code>
22	<code>exit: DD 25</code>
23	<code>msg: INITB "Hello, World!",0</code>
24	<code>last_char: DD 0</code>
25	<code>ptr: DD msg</code>
26	<code>video_port: DD 0x000</code>
27	<code>output: DD 21</code>
28	
29	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Figura 13.13. Visualização de escrita de cadeia de caracteres no periférico Video.



Fonte: Próprio autor (2021).

13.5.2 Keyboard

O periférico de entrada Teclado (*Keyboard*) atua de forma contrária ao periférico Video: ao digitar um caractere no campo “KEYBOARD (INPUT BUFFER)”, na interface gráfica do periférico Teclado, o caractere é convertido no respectivo código ASCII, que fica disponível em um *buffer* para leitura pelo processador.

O código da Figura 13.14 traz um código *Assembly* que ilustra a leitura de um caractere do periférico Teclado. Na figura, nas Linhas 12, 13 e 16 são definidas as variáveis: “keyboard_port”, que contém da porta “0x100” do periférico de Teclado (destaca-se aqui que, como o Teclado está endereçado na porta 1 e que a porta deve ser inserida no campo “AC High”, a notação “0x100” informa que será adicionado 0x1 nos 8-bits mais significativos de AC, campo “AC High”, e 0x00 nos 8-bits menos significativos, campo “AC Low”); “input”, que contém o valor “20”, referente ao código de operação de leitura da instrução INT; e “caractere”, que consiste na variável onde será gravado o código ASCII do caractere lido. Na Linha 2, adiciona-se, ao campo “AC High” do registrador acumulador AC, a porta do periférico Teclado; Na Linha 3, realiza-se a operação de leitura de dados do periférico Teclado; e, por fim, na Linha 5, o código ASCII do caractere lido é gravado na variável “caractere”.

Figura 13.14. Exemplo de código Assembly para leitura de caractere no periférico Teclado.

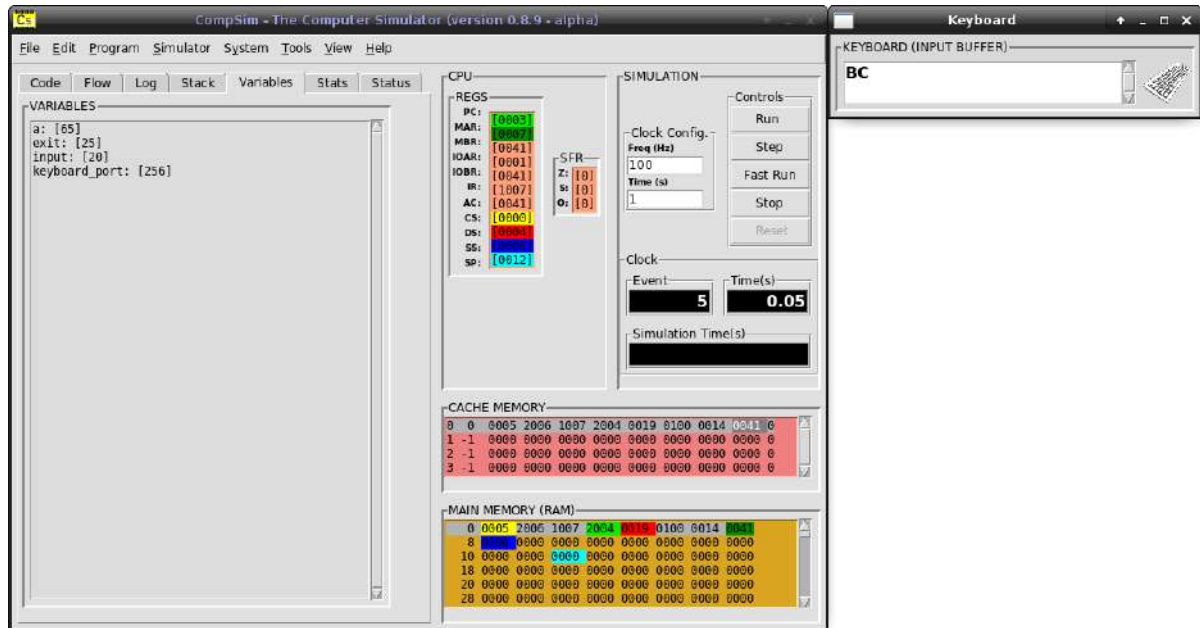
Linha	Código
1	<code>.code</code>
2	<code>LDA keyboard_port ;carrega porta de teclado em AC High.</code>
3	<code>INT output ;realiza operacao de input.</code>
4	<code>STA caractere ;armazena codigo do caractere lido do teclado.</code>
5	
6	<code>end:</code>
7	<code>INT exit</code>
8	
9	<code>.data</code>
10	<code>;syscall exit</code>
11	<code>exit: DD 25</code>
12	<code>keyboard_port: DD 0x100 ;porta do periferico teclado.</code>
13	<code>input: DD 20 ;codigo de input da instrucao INT.</code>
14	
15	<code>.bss</code>
16	<code>caractere: RESD 1 ;variavel que recebera caractere lido.</code>
17	
18	<code>.stack 1</code>
19	

Fonte: Próprio autor (2021).

A Figura 13.15 apresenta a visualização da aplicação de leitura de caractere do Teclado, onde observa-se: no componente gráfico “VARIABLES”, na aba “Variables”, que a

variável “a” recebeu o valor 65 (código ASCII do caractere “A”); e que, na interface gráfica do periférico Teclado, ainda há os caracteres “B” e “C”, que estão disponíveis para novas operações de leitura de dados.

Figura 13.15. Visualização de leitura de caractere no periférico Teclado.



Fonte: Próprio autor (2021).

O código da Figura 13.16 traz um exemplo mais elaborado de uso dos periféricos Teclado e Video: consiste de uma aplicação iterativa em que, a cada iteração, realiza a leitura de um caractere do periférico Teclado e, em seguida, escreve-o no periférico Video. No código, há uma estrutura de repetição While..Do, em que: Nas Linhas 3 à 5, realiza-se a leitura de um caractere do periférico Teclado; nas Linhas 7 à 10, verifica-se se o código de caractere lido é válido (uma leitura que retorna o código “0” significa que o *buffer* estava vazio) e, caso o *buffer* esteja vazio, o fluxo do programa é desviado de volta ao início da estrutura While..Do, não imprimindo caracteres no periférico Video; caso o *buffer* contenha caractere(s) válido(s), nas Linhas 11 à 14, o código do caractere lido anteriormente é enviado ao periférico Video para impressão; por fim, a estrutura While..Do, inicia uma nova repetição, com a instrução JMP da Linha 16. Ressalta-se que no programa da Figura 13.16, a estrutura de repetição While..Do não possui condição de parada, de maneira que as operações de leitura e escrita de caracteres seguem indefinidamente até que o tempo configurado para a simulação encerre ou a simulação seja encerrada com os controles de simulação (botão “Stop”).

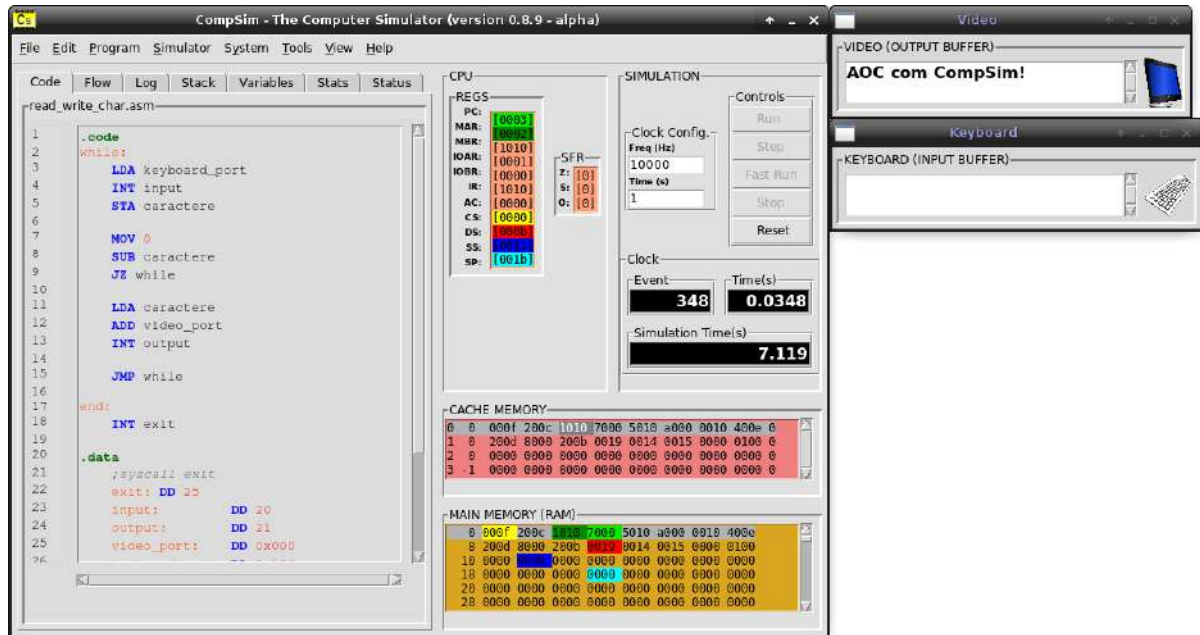
Figura 13.16. Exemplo de código Assembly para leitura e escrita de caracteres com periféricos.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code> LDA keyboard_port ;carrega porta de teclado em AC High.</code>
4	<code> INT input ;realiza operacao de input.</code>
5	<code> STA caractere ;armazena codigo do caractere lido do teclado.</code>
6	
7	<code>do_if:</code>
8	<code> MOV 0 ;if (caracatere == 0), volta para while.</code>
9	<code> SUB caractere</code>
10	<code> JZ while</code>
11	<code>else:</code>
12	<code> LDA caractere ;else, imprime caractere em Video.</code>
13	<code> ADD video_port</code>
14	<code> INT output</code>
15	
16	<code> JMP while</code>
17	
18	<code>end:</code>
19	<code> INT exit</code>
20	
21	<code>.data</code>
22	<code> ;syscall exit</code>
23	<code>exit: DD 25</code>
24	<code>input: DD 20 ;codigo de input da instrucao INT.</code>
25	<code>output: DD 21 ;codigo de output da instrucao INT.</code>
26	<code>keyboard_port: DD 0x100 ;porta do periferico teclado.</code>
27	<code>video_port: DD 0x000 ;porta do periferico video.</code>
28	
29	<code>.bss</code>
30	<code>caractere: RESD 1</code>
31	
32	<code>.stack 1</code>

Fonte: Próprio autor (2021).

A Figura 13.17 apresenta a visualização de uma execução do programa de leitura e escrita de caracteres da Figura 13.16. Na Figura 13.17, observa-se que a interface gráfica do periférico Video inclui a *string* “AOC com CompSim!”. Essa *string*, antes de ser impressa, foi digitada no campo “KEYBOARD (INPUT BUFFER)”, da interface gráfica do periférico Teclado, no qual cada caractere foi lido em sequência para, posteriormente, ser impresso.

Figura 13.17. Visualização de leitura e escrita de caracteres.



Fonte: Próprio autor (2021).

13.6 Entrada/Saída com Arduino UNO

Arduino é uma plataforma eletrônica baseada em microcontrolador, com especificação aberta e tem sido largamente utilizada em projetos de sistemas eletrônicos por estudantes, hobistas e profissionais (ARDUINO, 2018). Seu grande sucesso vem sendo atribuído a sua simplicidade, facilidade de uso, o baixo custo e à diversidade de produtos e complementos criados pela indústria (KUSHNER, 2011).

Os microcontroladores Atmega8, Atmega168 e Atmega328p, presentes nas diferentes versões do Arduino, incluem três portas. São elas:

- porta “B”: inclui pinos digitais numerados de 8 a 13;
- porta “C”: inclui pinos de entrada analógica numerados de A0 a A5; e
- porta “D”: inclui pinos digitais numerados de 0 a 7.

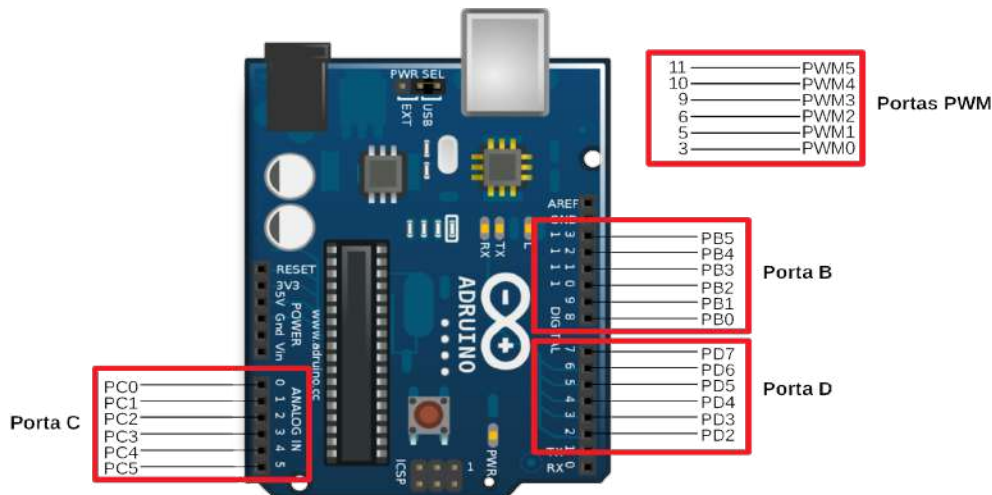
Cada uma dessas portas é controlada por 3 registradores:

- “DDRx”: determina se os pinos da porta “x” serão utilizados para leitura ou escrita de dados;
- “PORTx”: controla a tensão dos pinos da porta x, onde poderão estar com voltagem alta (“HIGH”, nível lógico “1”) ou baixa (“LOW”, nível lógico “0”); e
- “PINx”: realiza a leitura do estado dos pinos da porta “x”, configurados como entrada.

Os registradores “DDR” e “PORT” podem ser escritos ou lidos, enquanto que “PIN” somente pode ser lido.

Cada bit desses registradores corresponde a um único pino em uma *board* do Arduino. Por exemplo, o bit 0 dos registradores DDRB, PORTB e PINB, refere-se ao pino “PB0” (“Port B pin 0”), que corresponde ao pino de entrada/saída digital 8 de uma *board* do Arduino UNO, como pode ser vista na Figura 13.18.

Figura 13.18. Interface de entrada/saída do Arduino UNO.



Fonte: Próprio autor (2021).

Os pinos 3, 5, 6 (bits 4, 6 e 7 da porta D) e 9, 10, 11 (bits 1, 2 e 3 da Porta B) podem ainda ser utilizados para saída analógica via PWM (*Pulse With Modulation*) (AMARIEI, 2015), que é uma técnica que utiliza diferentes amplitudes e larguras de sinais digitais para representar um sinal analógico.

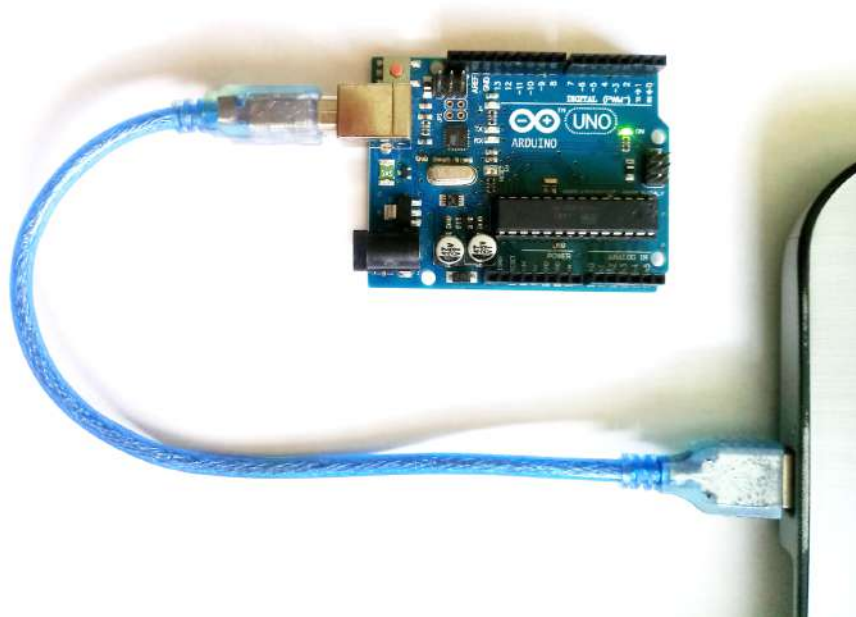
Ainda na Figura 13.18, no Arduino UNO, observa-se que, em cada uma das portas, não são utilizados todos os 8-bits. Mais especificamente a porta:

- “B” inclui os pinos PB0 a PB5 (pinos de 8 a 13 da *board* do Arduino UNO), então não utiliza os 2-bits mais significativos;
- “C” inclui os pinos PC0 a PC5 (pinos A0 a A5 da *board* do Arduino UNO), então não utiliza os 2-bits mais significativos; e
- “D” inclui os pinos PD0 a PD7 (pinos 0 a 7 da *board* do Arduino UNO), porém os pinos PD0 e PD1 são utilizados para comunicação serial (por exemplo, na comunicação entre um computador e a *board* do Arduino UNO), então não utiliza os 2-bits menos significativos.

13.6.1 Interface de Integração entre o CompSim e uma board do Arduino UNO

O primeiro passo para integrar uma *board* do Arduino UNO é conectá-la, via porta serial, ao computador onde está instalado o simulador CompSim. Para tanto, utiliza-se um cabo USB, cuja uma das extremidades está conectada a *board* do Arduino UNO e a outra extremidade a uma das interfaces USB do computador, como é ilustrado na Figura 13.19.

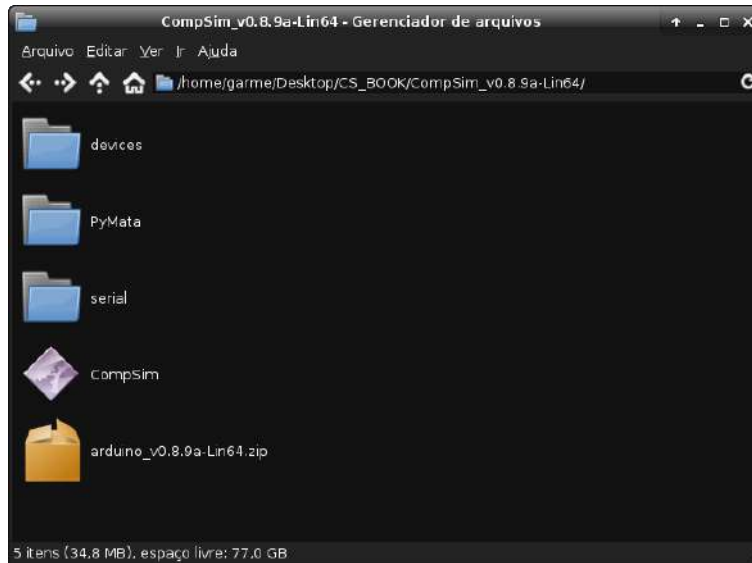
Figura 13.19. Conexão física de uma board do Arduino UNO ao Computador.



Fonte: Próprio autor (2021).

Após a conexão física da *board* do Arduino UNO ao computador, os passos seguintes são realizar o *download*, a partir do website do CompSim, e instalação do periférico Arduino UNO (ver procedimento de instalação na Seção 13.1 deste capítulo). É importante ressaltar que, ao instalar o periférico Arduino UNO, além da criação do diretório “arduino”, no diretório “devices”, serão criados os diretórios “PyMata” e “serial” no mesmo diretório onde encontra-se o arquivo executável do simulador CompSim, como mostra a Figura 13.20. Os diretórios “PyMata” e “serial” são as bibliotecas utilizadas pelo periférico Arduino UNO para estabelecer conexão e realizar transferências de dados entre o simulador CompSim e a *board* do Arduino UNO.

Figura 13.20. Instalação do periférico Arduino UNO.



Fonte: Próprio autor (2021).

Uma vez que o periférico está instalado, ao criar uma plataforma de hardware virtual no CompSim, será exibida uma interface gráfica, que traz recursos para integração entre o simulador CompSim e a *board* do Arduino conectada ao computador, como mostra a Figura 13.21. Essa interface gráfica permite: 1) instalar um *firmware*¹⁴ no Arduino UNO (controles em “INSTALL FIRMWARE”), de forma que seja possível haver comunicação entre o periférico Arduino UNO, que é a interface de software que está conectada ao barramento de endereços da plataforma de hardware virtual, e uma *board* do Arduino UNO (é possível selecionar a porta serial de comunicação, que consiste na interface USB entre o computador e a *board* (e.g. “COM4”, no sistema operacional MS Windows, e “/dev/ttyACM0”, no GNU/Linux); após a instalação do *firmware*, 2) conectar logicamente, via protocolo serial, o periférico Arduino UNO, conectado ao barramento de periféricos, com a *board* do Arduino UNO (caixa de combinação em “Select serial port” e botão “Connect”); e 3) exibir os dados transferidos em operações de leitura e escrita entre o processador e o Arduino UNO, através dos campos “INPUT” e “OUTPUT” (o periférico Arduino UNO pode operar tanto para entrada quanto para saída de dados digitais e analógicos).

¹⁴ Firmware consiste em um conjunto de instruções embarcadas diretamente no hardware para programação de dispositivos eletrônicos.

Figura 13.21. Interface gráfica do periférico Arduino UNO.



Fonte: Próprio autor (2021).

O Quadro 13.1 descreve os recursos disponíveis no periférico Arduino UNO, com respectivas portas e modos de operação.

Quadro 13.1. Recursos de comunicação disponíveis no periférico Arduino UNO.

Recurso	Modo de Operação	Endereço (AC High)		Dado de Saída (AC Low)	Retorno (AC)
		Bin	Hex		
Porta B (Pinos 8 a 13)	Entrada/Saída Digital	00000010	0x2	Byte*	Byte
Porta D (Pinos 2 a 7)	Entrada/Saída Digital	00000011	0x3	Byte**	Byte
Porta C (Pinos A0 a A5)	Entrada Analógica	00000100	0x4	Byte***	Inteiro ⁺
Porta PWM (Pino 3)	Saída Analógica	00000101	0x5	Byte	-
Porta PWM (Pino 5)	Saída Analógica	00000110	0x6	Byte	-
Porta PWM (Pino 6)	Saída Analógica	00000111	0x7	Byte	-
Porta PWM (Pino 9)	Saída Analógica	00001000	0x8	Byte	-
Porta PWM (Pino 10)	Saída Analógica	00001001	0x9	Byte	-
Porta PWM (Pino 11)	Saída Analógica	00001010	0xa	Byte	-

* Os 2-bits mais significativos são descartados (utiliza a máscara 00111111 para filtrar a entrada).

** Os 2-bits menos significativos são descartados (utiliza a máscara 11111100 para filtrar a entrada).

*** Utilizado para selecionar um dos pinos no intervalo A0 a A5. Como utiliza apenas 3-bits para representar o intervalo de pinos, os 5-bits mais significativos são descartados (utiliza a máscara 00000111 para filtrar a entrada).

⁺ Utiliza os 16-bits de AC para retornar um inteiro entre 0 e 1024.

Fonte: Próprio autor (2021).

Observa-se, no Quadro 13.1, que cada recurso do Arduino UNO é endereçado em uma porta diferente, de forma que o periférico Arduino UNO consumirá várias portas. No quadro, as portas B e D, que possuem interface com os pinos da *board* do Arduino UNO, rotulados de 8 a 13 e de 2 a 7, respectivamente, podem ser utilizadas para operações de

entrada ou saída digitais, tais como acender um *led* e leitura de estados de botões de pressão (*push buttons*). A porta C, com pinos rotulados de A0 a A5, pode ser utilizada para leitura de dados analógicos, como para leituras de dados aferidos por sensores. Já as portas PWM, com pinos 3, 5, 6, 9, 10 e 11, podem ser utilizadas para saída analógica, como para controle de luminosidade de um led ou acionar motores.

As subseções a seguir trazem exemplos básicos de como realizar operações de leitura e escrita digitais e analógicas com componentes eletrônicos simples conectados a uma *board* Arduino UNO. Essas operações podem ser combinadas e utilizadas com outros componentes eletrônicos conectados a uma *board* Arduino UNO para compor periféricos físicos mais complexos.

Atenção!

É importante frisar que, no escopo deste livro, não buscou-se oferecer conhecimentos para criação de projetos de circuitos eletrônicos para uso com o simulador CompSim. Para tanto, deve-se estudar: os fundamentos de eletrônica analógica; o comportamento e os requisitos para uso de cada componente eletrônico utilizado nos experimentos; como montar e analisar os circuitos eletrônicos desejados; bem como utilizar o próprio Arduino UNO. Recomenda-se fortemente que, antes da montagem dos cenários de circuitos eletrônicos propostos neste livro, o Arduino UNO esteja desconectado fisicamente do computador. Desta forma, não haverá corrente elétrica fluindo entre componentes eletrônicos, podendo-se manuseá-los livremente sem riscos à saúde ou riscos de danificá-los.

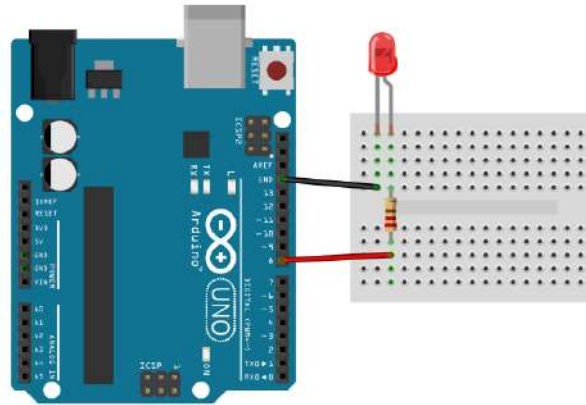
13.6.2 Operação de Escrita com Saída Digital

Esta subseção traz uma aplicação para operação de escrita com saída digital, utilizando o periférico Arduino UNO, para acender e apagar um led. Para tanto, será utilizado o cenário ilustrado no esquemático da Figura 13.22, que, além de uma *board* do Arduino UNO, utiliza-se uma *protoboard* e alguns fios (*jumpers*) para conectar um led, um resistor de 220 Ohm e o Arduino UNO. No esquemático, o resistor está ligado ao pino 8 do Arduino UNO e ao pólo positivo do led. Já o pólo negativo do led está ligado ao pino GND (“Ground” ou “Terra” de aterramento) do Arduino UNO, que é o pino de destino de uma corrente elétrica no circuito.

O circuito funciona da seguinte maneira: ao escrever um bit 1 no pino 8, será emitida uma corrente elétrica que fará com que o led acenda; ao escrever um bit 0, o led será

apagado. O resistor é utilizado para reduzir a corrente elétrica de saída do pino 8 de forma a não danificar o led (a corrente que sai do pino 8 tem intensidade média de 40mA e a maioria dos leds suportam uma corrente média de até 20mA, sendo que 10 à 15mA são suficientes para acendê-los) (ELECTRONICS CLUB, 2021). No circuito, a corrente elétrica segue do pino 8 até o GND.

Figura 13.22. Diagrama esquemático de configuração dos componentes eletrônicos para escrita digital.



Fonte: Próprio autor (2021).

Uma vez configurada a parte física do periférico, o passo seguinte consiste no desenvolvimento da aplicação que realizará as operações de escritas na Porta B do Arduino UNO, pois é nela que encontra-se o pino 8, utilizado para acender e apagar o led. A Figura 13.23 apresenta um código *Assembly* do CompSim para acender e apagar continuamente o led conectado à *board* do Arduino UNO.

Figura 13.23. Exemplo de código Assembly para acender e desligar led com periférico Arduino UNO.

Linha	Código
1	<code>.code</code>
2	<code>blink:</code>
3	<code>LDA high ;realiza a escrita digital para acender o led.</code>
4	<code>ADD arduino_portB</code>
5	<code>INT output</code>
6	
7	<code>CALL delay ;chama funcao delay para inserir um atraso.</code>
8	
9	<code>no_blink:</code>
10	<code>LDA down ;realiza a escrita digital para apagar o led.</code>
11	<code>ADD arduino_portB</code>
12	<code>INT output</code>
13	
14	<code>CALL delay ;chama funcao delay para inserir um atraso.</code>
15	
16	<code>JMP blink</code>
17	
18	<code>delay:</code>
19	<code>MOV 0 ;reseta contador.</code>

20	STA contador	
21	loop:	
22	MOV 1	<i>;while (contador < 2).</i>
23	SUB contador	
24	JZ ret_delay	
25	MOV 1	<i>;incrementa o contador.</i>
26	ADD contador	
27	STA contador	
28	JMP loop	<i>;desvio o fluxo para uma nova repetição.</i>
29	ret_delay:	
30	RET	
31		
32	end:	
33	INT exit	
34		
35	.data	
36	<i>;syscall exit</i>	
37	exit: DD 25	
38	high: DD 1	<i>;palavra que na escrita acende o led.</i>
39	down: DD 0	<i>;palavra que na escrita apaga o led.</i>
40	arduino_portB: DD 0x200	<i>;porta da PortB do periferico Arduino UNO.</i>
41	output: DD 21	<i>;codigo de input da instrucao INT.</i>
42	contador: DD 0	
43		
44	.stack 1	

Fonte: Próprio autor (2021).

Nas Linhas 41 à 45 do código na Figura 13.23, são definidas as variáveis: “high” e “low”, utilizadas na escrita digital para acender e apagar o led, respectivamente; “arduino_porB”, que contém a porta para endereçamento do registrador PortB do Arduino UNO; “output”, que inclui o código 21 para operação de escrita de dados em periféricos com a instrução INT; e “contador”, utilizada para compor um tempo de atraso (*delay*) em que o led permanece aceso ou apagado.

Ainda na Figura 13.23, nas:

- Linhas 2 à 5, realiza-se uma operação de escrita do valor contido em “high” na porta “arduino_portB”, para acender o led;
- Linhas 7 e 14, chama-se a função “delay”, que é utilizada para gerar um atraso no execução do código de forma que o led fique aceso e apagado, respectivamente, por um determinado intervalo tempo;
- Linhas 9 à 12, realiza-se uma operação de escrita do valor contido em “low” na porta “arduino_portB”, para apagar o led; e
- Linhas 18 à 30, define-se a função “delay” que utiliza uma estrutura de repetição While..Do, com repetições até que a variável “contador” atinja o valor 2. Enquanto isso, dependendo do ponto do programa em que a função “delay” foi chamada (Linha 7 ou 14), o led poderá permanecer aceso ou apagado; e

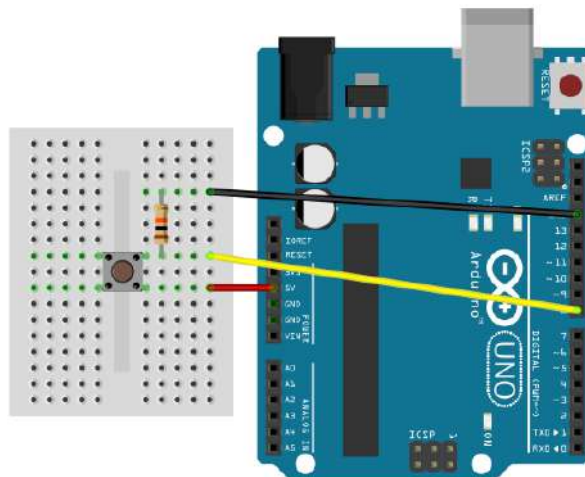
- Linha 16, desvia o fluxo de execução para o início do programa, de forma que o led será aceso e apagado até que o tempo predefinido para a execução da simulação encerre ou a simulação seja encerrada com os controles de simulação (botão “Stop”).

13.6.3 Operação de Leitura com Entrada Digital

Nesta subseção, a aplicação exemplo realiza a leitura do estado de um botão de pressão (*push button*). Se o botão estiver pressionado, será lido o valor 1 e, se não estiver pressionado, será lido o valor 0.

Para tanto, será utilizado o cenário ilustrado no esquemático da Figura 13.24, que, além de uma *board* do Arduino UNO, utiliza uma *protoboard* e alguns fios (*jumper*s) para conectar um botão de pressão, um resistor de 10K Ohm e o Arduino UNO. No esquemático, o resistor está ligado ao pino GND do Arduino UNO e a um dos pinos do botão de pressão. Esse mesmo pino do botão de pressão está ligado ao pino 8 da board do Arduino UNO, pois é nele que será lido o estado do botão (pressionado ou não). O pino vizinho ao anterior, no botão de pressão, é ligado ao pino 5V do Arduino UNO.

Figura 13.24. Diagrama Esquemático de configuração dos componentes eletrônicos para leitura digital.



Fonte: Próprio autor (2021).

Esse circuito funciona da seguinte maneira: caso o botão não esteja pressionado, ao realizar uma leitura do pino 8, verifica-se que não há corrente elétrica passante, retornando assim o valor 0. Ao pressionar o botão, será estabelecido um circuito em que a corrente fluirá do pino conectado em 5V para o pino conectado ao resistor e ao pino 8 do Arduino UNO, no qual, quando houver uma operação de leitura, será detectado que há uma corrente fluindo ali,

retornando assim o valor 1. Quando o botão é pressionado, a corrente elétrica, que vem do pino 5V do Arduino UNO, é dividida para fluir em dois sentidos: em direção ao GND e ao pino 8 do Arduino UNO. O resistor de 10K Ohm faz com que a corrente que flui em direção ao GND seja menor do que a que flui em direção ao pino 8, sendo possível portanto ler o estado 1.

O código *Assembly* da Figura 13.25 apresenta uma aplicação do CompSim que realiza continuamente operações de leitura na Porta B do Arduino UNO para verificar o estado de um botão de pressão. O programa inicia com a leitura do estado do botão de pressão, a partir da leitura dos bits da Porta B do Arduino UNO, como mostram as Linhas 3 e 4. Após a leitura, realiza-se um filtro nos bits lidos, de forma a isolar o bit menos significativo (LSB), que é o bit que informará o estado do botão de pressão. Para filtrar os bits, realiza-se uma operação lógica AND entre os bits lidos e os bits na variável “mask” (Linhas 6 à 8), sendo o resultado armazenado na variável “estado” (Linha 9).

Figura 13.25. Exemplo de código Assembly para leitura de estado de botão de pressão com periférico Arduino UNO.

Linha	Código
1	<code>.code</code>
2	<code>read_button:</code>
3	<code> LDA arduino_portB ;realiza a leitura digital do estado do botao.</code>
4	<code> INT input</code>
5	
6	<code> NAND mask ;realiza a operacao AND entre o valor lido do</code>
7	<code> STA tmp ; botao com a mascara para filtrar o valor lido</code>
8	<code> NAND tmp ; de forma que fique 0 ou 1.</code>
9	<code> STA estado ;grava o valor filtrado em estado.</code>
10	
11	<code> JMP read_button ;desvia o fluxo do programa para nova leitura.</code>
12	
13	<code>end:</code>
14	<code> INT exit</code>
15	
16	<code>.data</code>
17	<code> ;syscall exit</code>
18	<code> exit: DD 25</code>
19	<code> estado: DD 0 ;variavel que recebera o estado do botao.</code>
20	<code> mask: DD 1 ;mascara para filtrar os bits na leitura.</code>
21	<code> tmp: DD 0 ;variavel temporaria para uso no filtro.</code>
22	<code> arduino_portB: DD 0x200 ;porta da PortB do periferico Arduino UNO.</code>
23	<code> input: DD 20 ;codigo de input da instrucao INT.</code>
24	
25	<code>.stack 1</code>

Fonte: Próprio autor (2021).

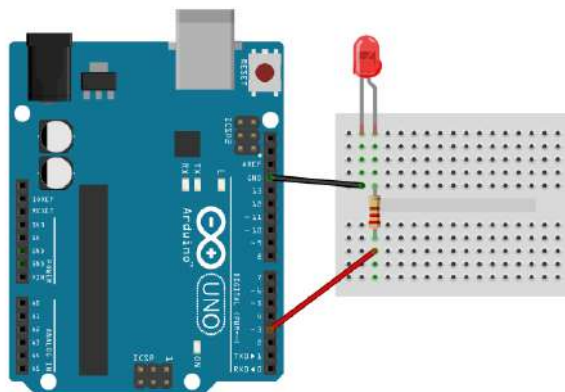
O programa da Figura 13.25 não possui condição de parada, desta forma as operações de leitura digital seguem indefinidamente até que o tempo predefinido para execução da

simulação encerre ou a simulação seja encerrada com os controles de simulação (botão “Stop”).

13.6.4 Operação de Escrita com Saída Analógica

Para ilustrar a operação de escrita com saída analógica, utilizando o periférico Arduino UNO, esta subseção traz uma aplicação para controle de luminosidade de um led. Para tanto, será utilizado um cenário semelhante ao do esquemático da Figura 13.21, porém, ao invés de conectar o resistor e led ao pino 8, eles deverão ser conectados a um dos pinos PWM (3, 5, 6, 9, 10 e 11). No experimento desta subseção, cujo esquemático é apresentado na Figura 13.26, o resistor e led estão conectados ao pino PWM 3.

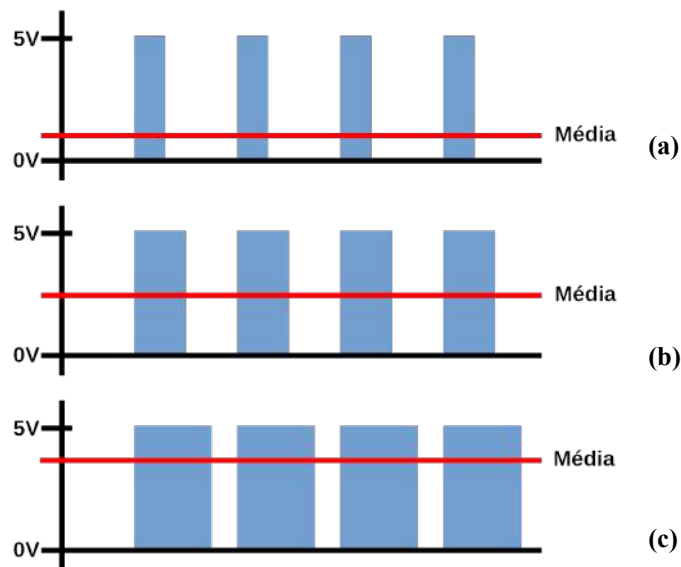
Figura 13.26. Diagrama esquemático de configuração dos componentes eletrônicos para escrita analógica.



Fonte: Próprio autor (2021).

A técnica de PWM (*Pulse Width Modulation*) emite um sinal digital com variações na sua amplitude (oscila entre HIGH e LOW) e na largura dos pulsos, em altas frequências, de forma que, ao se calcular a média da amplitude do sinal, obtém-se um sinal analógico, como está ilustrado na Figura 13.27.

Figura 13.27. Exemplos de diferentes sinais analógicos gerados por diferentes configurações de PWM.



Fonte: Próprio autor (2021).

Na Figura 13.27, em:

- (a), observa-se que a largura dos pulsos do sinal é menor em borda alta (HIGH) e maior em borda baixa (LOW), fazendo com que se obtenha uma média baixa da amplitude do sinal;
- (b), observa-se que a largura dos pulsos do sinal é a mesma tanto em borda alta (HIGH) e em borda baixa (LOW), fazendo com que se obtenha uma média intermediária da amplitude do sinal; e
- (c), observa-se que a largura dos pulsos do sinal é maior em borda alta (HIGH) e menor em borda baixa (LOW), fazendo com que se obtenha uma média alta da amplitude do sinal.

No Arduino UNO, a maioria dos pinos PWM utilizam uma frequência de 490 Hz, com exceção dos pinos 5 e 6, que utilizam frequência de aproximadamente 980 Hz (ARDUINO, 2021). Ao realizar uma escrita analógica no Arduino UNO, utilizando a técnica de PWM, deve-se informar um valor, na faixa de 0 a 255, que corresponde à fração de tempo em que o sinal permanece em borda alta (largura em borda alta ou *Duty Cycle*).

O código *Assembly* da Figura 13.28 apresenta uma aplicação do CompSim que utiliza a técnica de PWM para controlar a luminosidade de um led conectado ao pino PWM 3, cujo esquemático pode ser visto na Figura 13.26. No código, a variável “contador” será utilizada para informar o *Duty Cycle* na operação de escrita analógica por PWM no Arduino UNO.

Figura 13.28. Exemplo de código Assembly para controle de luminosidade de um led com periférico Arduino UNO.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code> LDA contador</code> <i>;escreve na porta PWM 3 o valor de contador.</i>
4	<code> ADD arduino_PWM3</code>
5	<code> INT output</code>
6	
7	<code> MOV 255</code> <i>;if (contador < 255).</i>
8	<code> SUB contador</code>
9	<code> JZ reseta_contador</code>
10	
11	<code> MOV 5</code>
12	<code> ADD contador</code>
13	<code> STA contador</code>
14	
15	<code> JMP while</code>
16	
17	<code>reseta_contador:</code>
18	<code> MOV 0</code>
19	<code> STA contador</code>
20	<code> JMP while</code>
21	
22	<code>end:</code>
23	<code> INT exit</code>
24	
25	<code>.data</code>
26	<code> ;syscall exit</code>
27	<code>exit: DD 25</code>
28	<code>contador: DD 0</code> <i>;utilizada no controle de luminosidade.</i>
29	<code>arduino_PWM3: DD 0x500</code> <i>;porta da PWM 3 do periférico Arduino UNO.</i>
30	<code>output: DD 21</code> <i>;codigo de input da instrucao INT.</i>
31	
32	<code>.stack 1</code>

Fonte: Próprio autor (2021).

O programa na Figura 13.28 inclui uma estrutura de repetição While..Do, que inicia com a leitura do valor da variável “contador”, que, em seguida, será escrito na Porta PWM 3, como mostram as Linhas 3 à 5. Em seguida, nas Linhas 7 à 9, verifica-se se o valor da variável “contador” é menor do que 255. Em caso negativo, o código segue incrementando o valor da variável “contador” em 5 unidades e, logo após, desvia o fluxo de execução para o início da estrutura de repetição, como pode ser visto nas Linhas 11 à 15. Caso o valor em “contador” seja igual a 255, o fluxo do programa será desviado para o bloco “reseta_contador”, em que atribui-se 0 à variável “contador” e desvia o fluxo de execução do programa para o início da estrutura de repetição. Desta maneira, o programa segue indefinidamente aumentando o brilho do led, até atingir o valor 255 em “contador”, o que faz com que valor de “contador” seja configurado em 0 e, com isso, o led seja apagado e volte a aumentar progressivamente sua luminosidade. O programa somente encerrará quando o

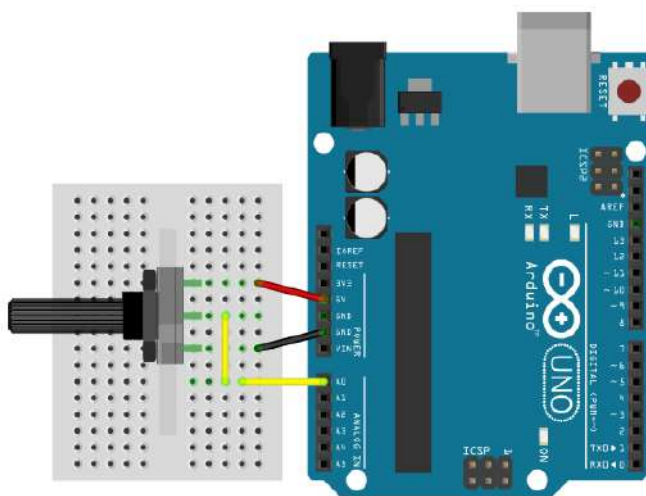
tempo predefinido para a execução da simulação chegar ao fim ou a simulação seja encerrada com os controles de simulação (botão “Stop”).

13.6.5 Operação de Leitura com Entrada Analógica

Para ilustrar a operação de leitura com entrada analógica, utilizando o periférico Arduino UNO, esta subseção traz uma aplicação para leitura de tensão de um sinal, que pode variar de acordo com a resistência configurada em um potenciômetro.

Um potenciômetro é um tipo de resistor, que pode ter sua resistência configurada a partir do giro de um eixo móvel. Além do eixo móvel, um potenciômetro possui três terminais: o primeiro é utilizado para conectar a uma fonte de tensão; o segundo é a saída da tensão modificada, dada a resistência configurada no potenciômetro; e o terceiro, é ligado ao GND. Na Figura 13.29, utilizou-se um potenciômetro de 10K Ohm, cujos terminais estão conectados, com apoio de uma protoboard e *jumpers*, aos pinos 5V (fio vermelho), A0 (fio amarelo) e GND (fio preto), do Arduino UNO.

Figura 13.29. Diagrama esquemático de configuração dos componentes eletrônicos para leitura analógica.



Fonte: Próprio autor (2021).

O Arduino UNO possui um conversor analógico-digital (*Analog-to-Digital Converter* - ADC) de 10-bits. Isso significa que, ao realizar a leitura de um sinal analógico, ele mapeia as tensões lidas entre 0 a 5V em um intervalo numérico inteiro de 0 a 1023. Se dividirmos 5V, que é a tensão máxima, por 1024, que é a quantidade total de números no intervalo de conversão, chegamos ao resultado de 4,9mV por unidade. Isso quer dizer que, se

aumentarmos 4,9mV no valor de tensão de entrada, o número convertido terá um acréscimo de 1 unidade.

O código *Assembly* da Figura 13.30 apresenta uma aplicação do CompSim que realiza a leitura analógica da tensão resultante da configuração de um potenciômetro, cujo esquemático do circuito é visto na Figura 13.29. No código, há uma estrutura de repetição While..Do, em que, nas Linhas 3 à 5, realiza-se uma leitura analógica da Porta C, no pino A0, e o valor convertido é gravado na variável “resistencia”.

Essa aplicação segue um fluxo de repetição sem condição de parada e somente encerrará quando o tempo predefinido para a execução da simulação chegar ao fim ou a simulação seja encerrada com os controles de simulação (botão “Stop”).

Durante uma simulação, é possível girar o eixo do potenciômetro nos dois sentidos de forma a obter diferentes configurações de resistência e, com isso, observar as variações das tensões lidas, expressas em números inteiros na faixa de 0 a 1023, que estão gravados na variável “resistencia”.

Figura 13.30. Exemplo de código Assembly para leitura de resistência de um potenciômetro com periférico Arduino UNO.

Linha	Código
1	.code
2	while:
3	LDA pinA0 ;realiza leitura na porta C pino A0.
4	ADD arduino_PortC
5	INT input
6	
7	STA resistencia ;grava o valor lido.
8	
9	JMP while
10	
11	end:
12	INT exit
13	
14	.data
15	;syscall exit
16	exit: DD 25
17	resistencia: DD 0 ;utilizada no controle de luminosidade.
18	pinA0: DD 0 ;utilizada no controle de luminosidade.
19	arduino_PortC: DD 0x400 ;porta da PortC do periferico Arduino UNO.
20	input: DD 20 ;codigo de input da instrucao INT.
21	
22	.stack 1

Fonte: Próprio autor (2021).

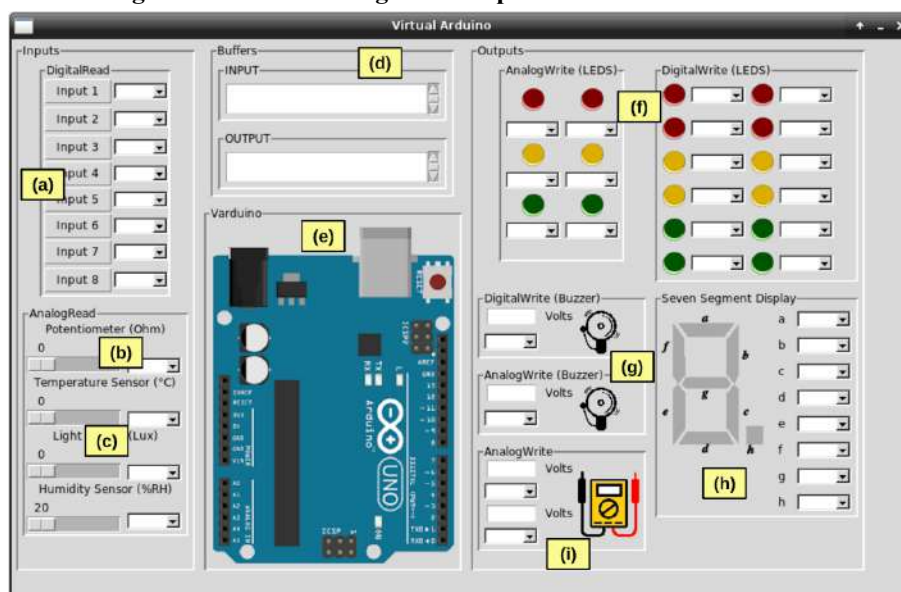
13.7 Entrada/Saída com VArduino

Na seção anterior, fez-se uma breve discussão sobre a necessidade de conhecimentos prévios de eletrônica analógica para a realização dos experimentos propostos neste livro que fazem uso do Arduino UNO e de componentes eletrônicos analógicos.

Dependendo do cenário de projeto, o processo de montagem de circuitos eletrônicos com uso de Arduino UNO e componentes eletrônicos possui ainda outras desvantagens, como apontam os trabalhos em (GONÇALVES; DURAES, 2019) e (KATAYAMA et al., 2019), tais como: alta demanda de tempo para configuração e montagem dos experimentos; dependência de determinados componentes eletrônicos complementares, o que pode elevar o custo do projeto; com o tempo, há o desgaste dos componentes eletrônicos; e estudantes/projetistas, que não possuem o nível adequado de conhecimentos e habilidades técnicas em eletrônica, podem projetar circuitos sem os devidos cuidados técnicos, de forma que podem se colocar em situações de risco à saúde ou danificar os componentes eletrônicos utilizados.

Nesse contexto surgiu a ideia de criação de um periférico virtual do Arduino UNO, o VArduino (*Virtual Arduino UNO*). Além de um Arduino UNO virtual, o periférico VArduino contém diferentes componentes eletrônicos virtuais, como será visto a seguir, visando, no seu uso, abstrair a necessidade de conhecimentos de eletrônica e do manuseio de componentes eletrônicos físicos. A Figura 13.31 apresenta a interface gráfica do VArduino.

Figura 13.31. Interface gráfica do periférico virtual VArduino.



Fonte: Próprio autor (2021).

De acordo com a Figura 13.31, a interface gráfica do VArduino está dividida em três macroblocos de componentes gráficos, onde

- Do lado esquerdo, estão os componentes de entrada de dados, que emulam: (a) Botões de pressão (*push buttons*), que retornam dois possíveis estados, HIGH, quando pressionados, e LOW, quando não pressionados; (b) Potenciômetro, cuja configuração varia de 0 a 10K Ohm; e (c) 3 sensores analógicos, sendo eles: de temperatura, que varia de 0 a 50 °C (graus Celsius); de luminosidade, que varia de 0 a 10k lux (“lux” é um termo em latim para “luz” e mede o fluxo de luz em uma unidade de área); e de umidade relativa, que varia de 20% a 90% RH (*Relative Humidity*).
- Ao centro, estão contidos os componentes gráficos que apresentam: (d) os *buffers* de entrada (“INPUT”) e de saída (“OUTPUT”), usados na comunicação entre o simulador CompSim e o VArduino, e, logo abaixo, (e) há um diagrama do Arduino UNO, utilizado para consulta de numeração de pinos de comunicação; e
- À direita, estão os componentes de saída de dados, que emulam: (f) 18 leds, sendo que 6 são utilizados em escritas analógicas e 12 em escritas digitais. Os leds analógicos possuem 6 níveis de intensidade de luminosidade, enquanto que os digitais apenas 2 (aceso ou desligado); (g) 2 campainhas sonoras (*buzzers*), sendo um deles para escrita digital, que apresenta os estados HIGH ou LOW (ativado ou desativado), e outro de escrita analógica, que apresenta a frequência do som emitido; (h) 1 *display* de 7-segmentos, onde cada segmento do *display* é identificado unicamente por uma letra de “a” a “g”, de forma que permite conectar cada um dos segmentos a um dos pinos das portas digitais do Arduino UNO virtual; e (i) 2 dispositivos genéricos de escrita analógica (ao lado há um diagrama de multímetro, que é um instrumento de medidas elétricas), que podem simular, por exemplo, o acionamento de *coolers*/exaustores de computadores, acionar e controlar o movimento de motores, gerar sinais modulados para controle de um LED infravermelho (e.g. para uso como controle remoto), gerar efeitos sonoros, entre outros.

Para acionar cada um desses componentes eletrônicos virtuais, assim como em um Arduino UNO físico, é necessário conectá-los a algum dos pinos do Arduino UNO virtual. Para tanto, cada componente eletrônico virtual possui uma caixa de combinação (*combo box*), a qual permite escolher em qual pino do Arduino UNO virtual o respectivo componente eletrônico virtual será conectado.

Como o Arduino UNO virtual também simula as operações de escrita e leitura digital e analógica, os componentes eletrônicos somente poderão ser conectados a determinados pinos. São eles:

- Componentes de leitura ou escrita digital: podem ser conectados aos pinos de 2 a 13, referentes às portas B e D do Arduino UNO.
- Componentes de leitura analógica: podem ser conectados aos pinos de A0 a A5, referentes à Porta C do Arduino UNO;
- Componentes de escrita analógica: podem ser conectados aos pinos 3, 5, 6, 9, 10 e 11, referentes aos pinos que implementam escrita analógica pela técnica de PWM, presente no Arduino UNO;

Desta maneira, para utilizar o Arduino UNO Virtual, os procedimentos são bastante semelhantes aos descritos para o periférico Arduino UNO:

1. *Download* no website do projeto CompSim e instalação do periférico VArduino (ver Seção 13.1);
2. Criação de uma plataforma de hardware virtual no CompSim, para criação do periférico VArduino e conexão do mesmo ao barramento de periféricos;
3. Configuração do cenário de projeto eletrônico do periférico na janela do VArduino, através da ligação dos componentes eletrônicos virtuais aos pinos do Arduino UNO Virtual, utilizando as caixas de combinação;
4. Desenvolvimento de código *Assembly* para programar o processador Cariri para realizar operações de entrada/saída para leituras e/ou escritas digitais e/ou analógicas, no periférico VArduino.

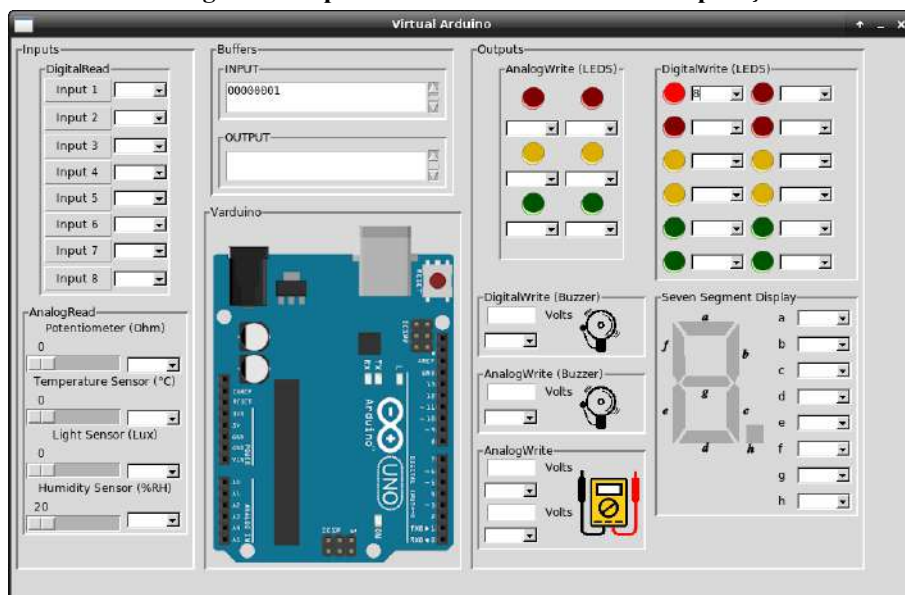
É importante ressaltar que as aplicações desenvolvidas em código *Assembly* do simulador CompSim para operações de entrada e saída, com escrita e leitura digital e analógica, no periférico físico do Arduino UNO, são completamente compatíveis com o periférico VArduino. Assim, para os cenários de projeto apresentados nas Subseções 13.6.2, 13.6.3, 13.6.4 e 13.6.5, os respectivos códigos *Assembly* podem ser reaproveitados para a composição dos mesmos cenários de projeto utilizando o periférico VArduino.

As Figuras 13.32, 13.33, 13.34 e 13.35 apresentam a interface gráfica do periférico VArduino, configurada na ordem dos cenários de projeto das Subseções 13.6.2, 13.6.3, 13.6.4 e 13.6.5.

Na Figura 13.32, que trata do cenário de operação de escrita com saída digital para acender e apagar um led, utilizou-se a caixa de combinação do primeiro led digital (led na posição superior e à esquerda no conjunto com 12 leds, em “DigitalWrite (LEDS)”) para

conectar o led ao pino 8 do Arduino UNO virtual (pino 8 refere-se ao primeiro bit da Porta B). Ao executar a simulação com o programa *Assembly* da Figura 13.23, o led configurado no VArduino será acendido e apagado continuamente.

Figura 13.32. Interface gráfica do periférico virtual VArduino em operação de escrita digital.

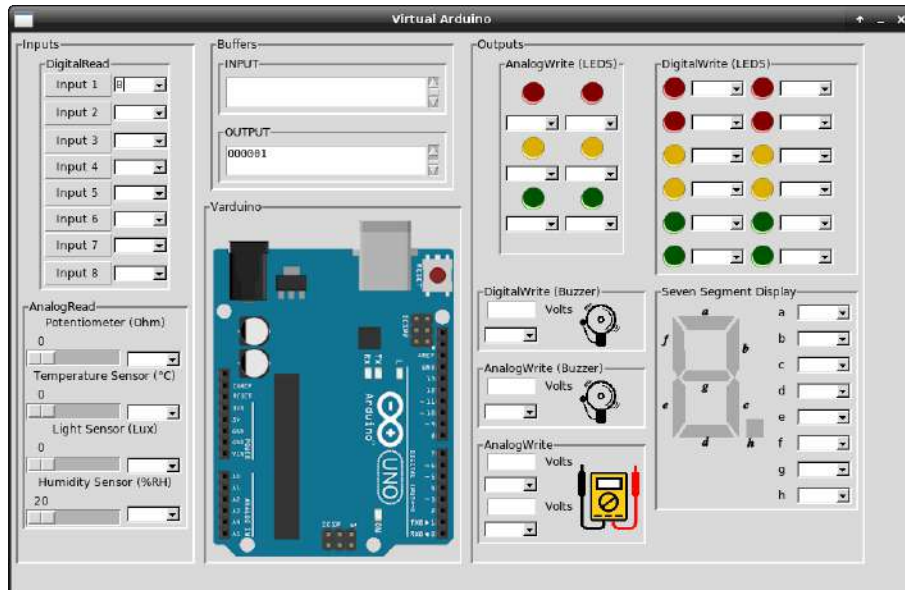


Fonte: Próprio autor (2021).

A Figura 13.32 mostra, no *buffer* “INPUT”, a sequência de bits “00000001”, os quais foram escritos na Porta B do Arduino UNO virtual e fizeram com que o led configurado seja acendido, durante a simulação.

Na Figura 13.33, que trata do cenário de operação de leitura com entrada digital para leitura do estado de um botão de pressão (*push button*), utilizou-se a caixa de combinação do botão “Input 1”, para conectar o botão ao pino 8 do Arduino UNO virtual. Ao executar a simulação com o programa *Assembly* da Figura 13.25, ao pressionar o botão “Input 1”, será gravado o valor 1 na variável “estado”. Caso o botão não tenha sido pressionado, será gravado o valor 0 na variável “estado”.

Figura 13.33. Interface gráfica do periférico virtual VArduino em operação de leitura digital.



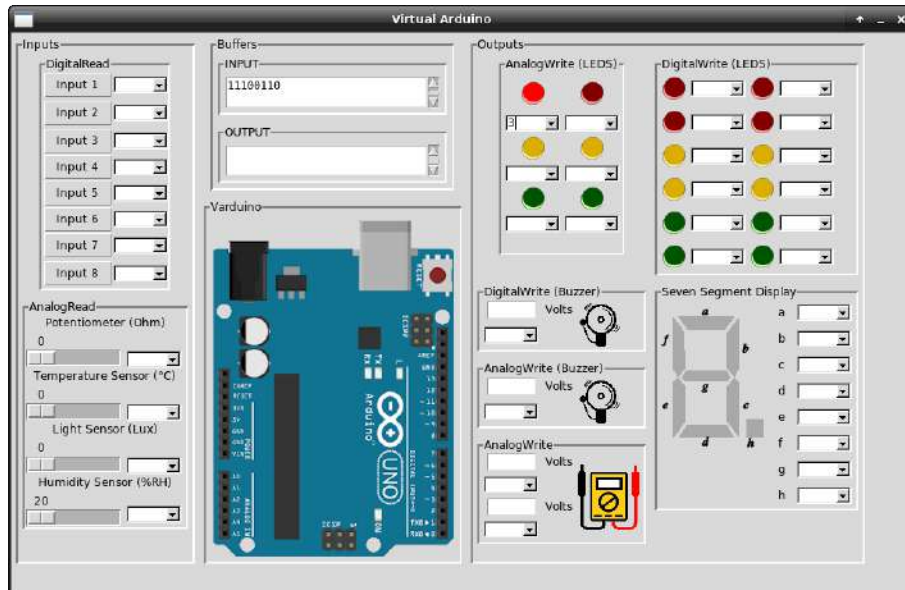
Fonte: Próprio autor (2021).

A Figura 13.33 mostra, no *buffer* “OUTPUT”, os bits “000001”, lidos da Porta B do Arduino UNO virtual, durante uma simulação. Na sequência de bits lidos, observa-se que o bit mais à direita, que é o bit menos significativo, está configurado em 1, o que informa que o botão foi pressionado.

Na Figura 13.34, que trata do cenário de operação de escrita com saída analógica para controle de luminosidade de um led, utilizou-se a caixa de combinação do primeiro led analógico (led na posição superior e à esquerda no conjunto com 6 leds, em “AnalogWrite (LEDS)”) para conectar o led ao pino 3 do Arduino UNO virtual (pino 3 refere-se a um dos pinos que implementam escrita analógica pela técnica de PWM). Ao executar a simulação com o programa *Assembly* da Figura 13.28, o led configurado no VArduino será acendido gradualmente até atingir o brilho máximo, sendo apagado em seguida. Esse processo será repetido continuamente, enquanto durar a simulação.

A Figura 13.34 mostra, no *buffer* “INPUT”, os bits “11100110” (230, em decimal) escritos na Porta PWM pino 3 do Arduino UNO virtual, que fizeram com que o led configurado tenha sido acendido com brilho alto (o brilho máximo é atingido com o valor de 255), durante a simulação.

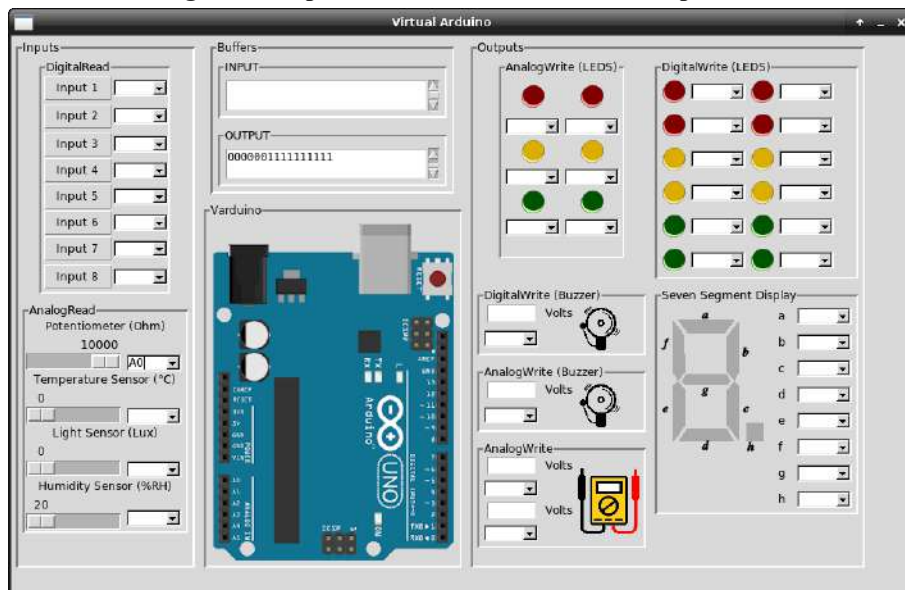
Figura 13.34. Interface gráfica do periférico virtual VArduino em operação de escrita analógica.



Fonte: Próprio autor (2021).

Na Figura 13.35, que trata do cenário de operação para leitura de uma determinada tensão, a partir da configuração de um potenciômetro, utilizou-se a caixa de combinação do componente virtual “Potentiometer (Ohm)”, para conectar o potenciômetro ao pino A0 do Arduino UNO virtual. Ao executar a simulação com o programa *Assembly* da Figura 13.30, ao movimentar a barra de rolagem do potenciômetro, o valor da resistência será modificado, alterando assim o valor da tensão lida.

Figura 13.35. Interface gráfica do periférico virtual VArduino em operação de leitura analógica.



Fonte: Próprio autor (2021).

A Figura 13.35 mostra, no *buffer* “OUTPUT”, os bits “0000001111111111” (1023 em decimal), lidos do pino A0 do Arduino UNO virtual, durante uma simulação. Na sequência de bits lidos, observa-se que os 10-bits mais à direita, que são os bits menos significativos, estão todos configurados em 1, o que mostra que o potenciômetro foi configurado para apresentar resistência mínima e, desta forma, retornar a tensão máxima.

Os demais componentes eletrônicos virtuais do VArduino também podem ser utilizados, através das configurações de conexão de pinos, pelas caixas de combinação, e pelas operações de leitura ou escrita:

- Os componentes eletrônicos virtuais leds digitais (conjunto com 12 leds agrupados), campainha sonora digital (*buzzer*) e os segmentos do *display* de 7-segmentos podem ser acionados a partir de operações de escrita com saída digital;
- Os 8 botões de pressão (*push buttons*) podem ser utilizados com operações de leitura com entrada digital;
- Os componentes leds analógicos (conjunto com 6 leds agrupados), campainha sonora analógica (*buzzer*) e os 2 dispositivos genéricos de escrita analógica podem ser acionados a partir de operações de escrita com saída analógica; e
- Os sensores de temperatura, luminosidade e de umidade, assim como no potenciômetro, podem ser utilizados com operações de leitura com entrada analógica.

É possível combinar as operações de leitura e escrita digital e analógica com os componentes eletrônicos para compor periféricos mais complexos, para dar suporte a aplicações mais elaboradas. A nível de ilustração (naturalmente, sem esgotar todas as possibilidades de combinações), pode-se, por exemplo:

1. Ler o estado de um botão de pressão (*push button*), através de uma operação de leitura digital, e, caso o valor lido seja 1, acender um led digital, através de uma operação de escrita digital;
2. Realizar a leitura da tensão configurada por um potenciômetro, através de uma operação de leitura analógica, e, dependendo do valor lido, configurar, através de uma operação de escrita analógica, a intensidade do brilho de um led analógico;
3. Realizar a leitura do nível de luminosidade (lux) do sensor de luz (“Light Sensor (Lux)”), através de uma operação de leitura analógica, e, dependendo do valor lido, acender um determinado número de leds digitais, a partir de operações de escrita digital; e

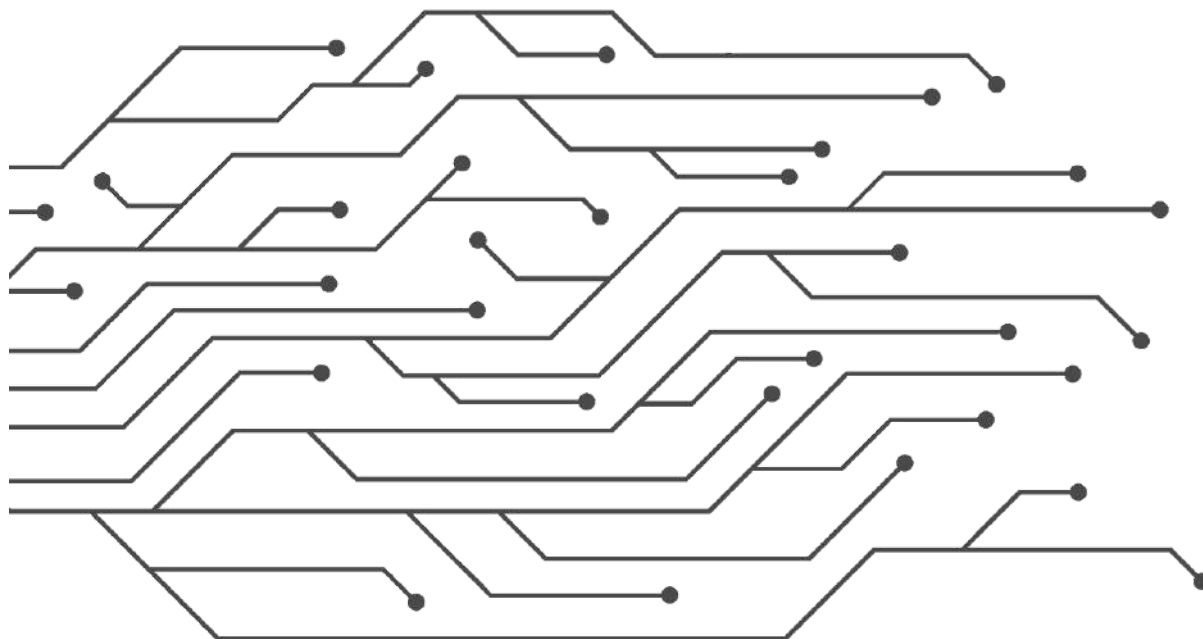
4. Ler o estado de botões de pressão (*push button*), através de operações de leitura digital, e, dependendo dos valores lidos, configurar, através de uma operação de escrita analógica, a intensidade do brilho de um led analógico.

13.8 Resumo

Este capítulo tratou dos fundamentos básicos de operações de entrada e saída com o simulador CompSim. O capítulo apresentou inicialmente os procedimentos para instalação de periféricos no CompSim. Em seguida, detalhou-se seu Subsistema de Entrada/Saída, o qual permite conectar de forma automatizada novos periféricos ao barramento de periféricos da plataforma de hardware virtual, e os tipos de periféricos suportados. Em seguida, apresentou-se a interface de comunicação entre o processador Cariri e os periféricos, bem como diversos exemplos aplicados para comunicação entre o processador Cariri e os periféricos Video, Teclado, Arduino UNO e Arduino UNO virtual (VArduino).

Ao longo do capítulo, utilizando o simulador CompSim, foram abordados diversos cenários práticos relacionados aos diferentes tipos de periféricos e à programação de aplicações para controle de periféricos em *Assembly*, que incluíram operações de leitura e escrita, com a transmissão de dados do tipos digitais e analógicos.

PARTE IV - TÓPICOS AVANÇADOS



CAPÍTULO 14 – OTIMIZAÇÃO DE DESEMPENHO

No [Capítulo 2 - História do Computador](#) fez-se uma discussão sobre as tendências tecnológicas que visam dar continuidade à evolução do computador. É importante destacar que esse processo de evolução busca tornar os computadores mais rápidos, de forma a atenderem às novas demandas computacionais.

No [Capítulo 3 - Tendências Tecnológicas](#), discutiu-se sobre as tendências para os próximos anos no projeto de circuitos integrados, que envolvem o uso de novos materiais em substituição ao silício, novas técnicas de projeto de circuitos integrados, novas técnicas de programação para um melhor aproveitamento do hardware e a criação de circuitos integrados especializados, tais como placas gráficas de propósito geral, processadores com múltiplos núcleos para processamento paralelo e processadores com propósito específico.

Percebe-se então que otimização de desempenho é uma atividade muito importante, tanto no projeto quanto na programação dos computadores. Este capítulo trata de aspectos de otimização no projeto de computadores, tais como paralelismo em diferentes níveis e aumento de desempenho em sistemas com memória Cache, bem como, boas práticas de programação para tornar os programas mais eficientes.

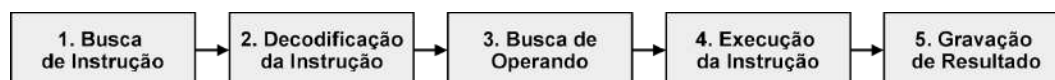
14.1 Paralelismo

Nos livros mais utilizados como literatura base em cursos de Arquitetura e Organização de Computadores, tais como (STALLINGS, 2010), (TANENBAUM; AUSTIN, 2013) e (PATTERSON; RENNESSY, 2017), o tópico de otimização de desempenho é tratado basicamente através da implementação de Paralelismo, em diferentes níveis do computador.

Há o paralelismo em nível de instrução (*Instruction-Level Parallelism* - ILP), em que o ciclo de execução de uma instrução é subdividido em estágios e, enquanto um estágio de uma instrução está em execução, os outros estágios, que são tratados por partes dedicadas de

hardware, são aproveitados para execução de outras instruções, em paralelo, caracterizando o que se chama de *Pipeline* (TANENBAUM; AUSTIN, 2013). A Figura 14.1 ilustra um cenário em que a execução de uma instrução pode ser dividida em cinco estágios, que são: 1) Busca da instrução; 2) Decodificação da instrução; 3) Busca do operando; 4) Execução da instrução; e 5) Gravação do resultado.

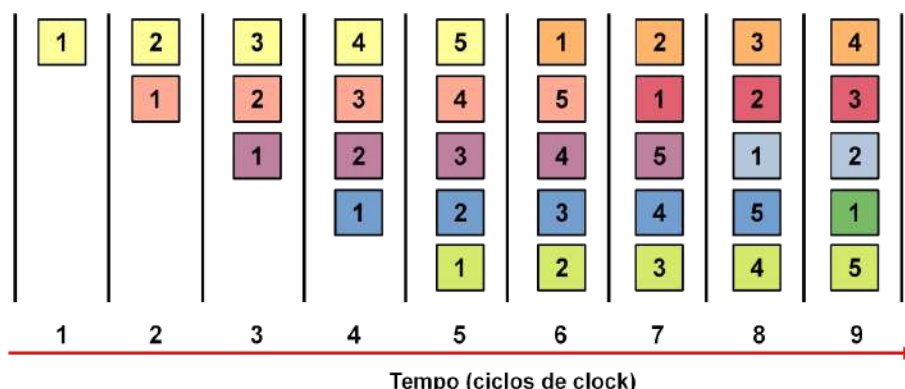
Figura 14.1. Execução de instrução em cinco estágios.



Fonte: Próprio autor (2021).

Já na Figura 14.2, ilustra-se um *pipeline* para execução de mais de uma instrução em paralelo (cada instrução é caracterizada por uma cor diferente), considerando os cinco estágios ilustrados na Figura 14.1.

Figura 14.2. Execução de instruções em pipeline com cinco estágios.

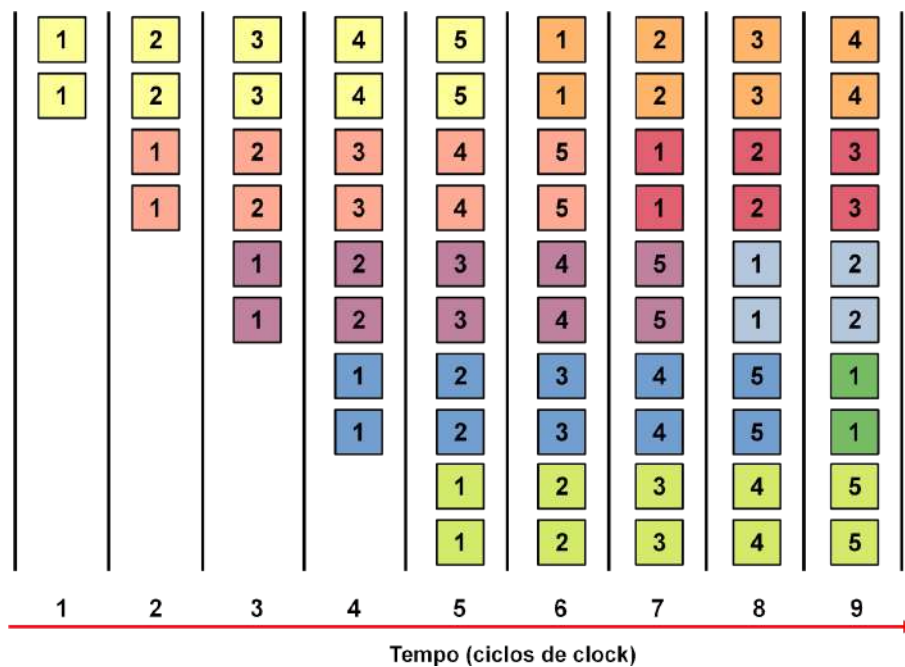


Fonte: Próprio autor (2021).

Observa-se na Figura 14.2 que, no primeiro ciclo de *clock*, inicia-se o *pipeline* com a execução do estágio 1 da primeira instrução. No segundo ciclo de *clock*, já pode-se observar que a primeira instrução passou para o estágio 2, enquanto que uma segunda instrução entra em execução, iniciando o estágio 1. A partir do quinto ciclo de *clock*, o *pipeline* atinge o estágio máximo de paralelismo, com cinco instruções sendo executadas em paralelo.

Uma outra abordagem de ILP explora o paralelismo de instrução através de múltiplos *pipelines*. Os computadores que possuem arquiteturas que implementam múltiplos *pipelines* são conhecidos como Superescalares. A Figura 14.3 ilustra o processo de execução de instruções em um computador superescalar com dois *pipelines* paralelos.

Figura 14.3. Exemplo de arquitetura superescalar.



Fonte: Próprio autor (2021).

Na Figura 14.3, observa-se que no primeiro ciclo de clock, são buscadas na memória, em paralelo, duas instruções. Já no segundo ciclo, as instruções buscadas no ciclo anterior entram no estágio 2 de execução, e, em paralelo, mais duas instruções são buscadas na memória. Assim como acontece com um pipeline simples de cinco estágios, o cenário de arquitetura superescalar também atinge o estágio máximo de paralelismo no quinto ciclo de *clock*, em que são executadas 10 instruções simultaneamente.

Segundo Tanenbaum e Austin (2013), todos os computadores modernos com suporte de *pipeline* possuem um problema: quando há uma referência a uma instrução e a instrução não encontra-se nas memórias Cache de nível 1 e 2, por exemplo, o processador deve aguardar por um longo tempo, até que a instrução venha da memória RAM e esteja disponível nas memórias Cache, fazendo com que o *pipeline* pare. De acordo com os autores, uma forma de contornar este problema é permitir que o processador execute múltiplas *threads* (fluxos de execução distintos em um mesmo processo). Em resumo, enquanto uma *thread* estiver bloqueada aguardando a instrução requisitada ser copiada da memória RAM para as memórias Cache, o processador pode executar as instruções de uma outra *thread*, mantendo o *pipeline* sempre cheio.

Outro nível de paralelismo tratado na maioria dos livros referência de AOC é o paralelismo ao nível de tarefa (*Task-Level Parallelism* - TLP) ou ao nível de processo (*Process-Level Parallelism* - PLP). Essa abordagem considera que o computador possui

múltiplos processadores que executam programas independentes simultaneamente (PATTERSON; HENNESSY, 2017).

Existem vários estudos que buscam categorizar os tipos de computadores com processadores paralelos, porém a Taxonomia de Flynn (FLYNN, 1972) é, ainda hoje, a que prevalece. A taxonomia de Flynn categoriza os sistemas computacionais em:

- *Single Instruction, Single Data* (SISD): consiste de computadores que executam uma instrução para operar um dado em memória (modelo clássico de computador de Von Neumann);
- *Single Instruction, Multiple Data* (SIMD): são computadores que possuem uma única unidade de controle e múltiplas unidades lógicas e aritméticas, de forma que uma instrução pode operar em conjuntos de dados. Nesta categoria estão os computadores vetoriais e processadores de arrays, e alguns processadores modernos que incluem instruções para processamento vetorial, como é o caso das instruções *Streaming SIMD Extensions* (SSE), *Multimedia Extension* (MMX) e *Advanced Vector Extensions* (AVX), das arquiteturas da Intel;
- *Multiple Instruction, Single Data* (MISD): este tipo de computador contém múltiplas instruções operando sobre o mesmo dado. Não existe um consenso sobre a existência desse tipo de computador, porém, em (PATTERSON; HENNESSY, 2017), cita-se um exemplo de uma possível aplicação: um sistema computacional que recebe dados via rede e os processa em passos sequenciais (como em um *pipeline*), tais como descriptação, descompressão, busca de dados por determinado critério, etc.; e
- *Multiple Instruction, Multiple Data* (MIMD): consiste de computadores com múltiplos processadores que atuam de forma independente. Nesta categoria podem ser incluídos os processadores *multicores*, *manycors* e os multicomputadores (agrupamentos de computadores, ou *clusters*, que consistem em um conjunto de computadores interconectados por uma rede de transmissão de dados que atuam como um único computador).

Com a disponibilidade dos computadores paralelos, o desafio é programá-los de forma a usufruir todo o potencial do seu hardware e obter uma execução eficiente das aplicações. Dependendo do hardware disponível, podem ser empregadas diferentes técnicas de programação paralela e de sincronização de tarefas.

Atualmente, para simplificar a exploração do hardware em aplicações de alto desempenho, o estado da arte em programação paralela inclui, principalmente:

1. MPI (*Message Passing Interface*) (GROPP; LUSK; SKJELLUM, 1999), é um padrão *de fato* que inclui uma biblioteca com um conjunto de rotinas para programação de aplicações paralelas para execução em *clusters*. A aplicação paralela, criada com suporte de MPI, distribui de forma transparente os dados e funções de uma aplicação para processamento paralelo distribuído entre os nós computacionais do *cluster*;
2. OpenMP (*Open Multi-Processing*) (CHAPMAN; JOST; VAN DER PAS, 2007), é um conjunto de diretivas que podem ser utilizadas para adicionar anotações em determinados trechos de código em linguagem C/C++, principalmente em estruturas de repetição do tipo For..Do. Quando o código é compilado, serão gerados códigos binários para execução paralela nos trechos anotados;
3. CUDA (*Compute Unified Device Architecture*) (NVIDIA, 2021), consiste de uma biblioteca e de um conjunto de ferramentas de compilação para programação paralela de GPGPUs (*General Purpose Graphics Processing Units*) projetadas pela fabricante NVIDIA. CUDA permite criar aplicações que distribuem um conjunto de dados e uma função de processamento paralelo, chamada de *computer kernel* ou somente *kernel*, entre as unidades de processamento paralelo da GPGPU, chamadas de *CUDA cores* ; e
4. 4) OpenCL (*Open Computing Language*) (KHRONOS, 2019), é uma linguagem de programação para criação de aplicações paralelas para execução em diferentes tipos de plataformas de hardware paralelo, tais como *multicores*, *manycores*, GPGPUs e MDSPs (*Multiprocessor Digital Signal Processors*). Semelhante à CUDA, além de distribuir dados e funções para processamento paralelo, é possível ainda selecionar a plataforma de hardware, caso seja suportada mais de uma no mesmo computador (e.g. em um computador *multicore* com GPGPU, pode-se optar por realizar o processamento nos *multicores* ou na GPGPU).

14.2 Aumentando o Desempenho da Memória Cache

Além do paralelismo, a otimização de desempenho também é tratada no projeto da hierarquia de memória, mais especificamente com foco na memória Cache. Como se viu no [Capítulo 4 - Introdução à Organização de Computadores](#), a memória Cache foi introduzida para aumentar a eficiência da transferência de dados entre a memória RAM e o processador, usufruindo das propriedades de Localidade Espacial e Temporal.

Quando um processador solicita um determinado dado, a memória Cache pode atuar de duas maneiras: 1) caso o dado solicitado esteja armazenado na Cache (caracteriza-se um acerto ou *Hit*), a memória Cache entrega o dado ao processador; e 2) caso o dado não esteja armazenado na Cache (caracteriza-se uma falta ou *Miss*), a memória Cache deverá transferir para ela um bloco de dados da memória RAM que contenha o dado solicitado, para só então disponibilizá-lo ao processador. Percebe-se então que a redução da taxa de faltas da memória Cache (*Cache Miss Rate*) torna-se um objeto de estudo em otimização de desempenho, visto que uma falta pode implicar em acréscimo significativo de tempo nas transferências de dados entre memória e processador.

Há, basicamente, duas técnicas para redução da taxa de faltas de memória cache (PATTERSON; RENNESSY, 2017): 1) Adicionar mais níveis de memória Cache, que é uma prática já estabelecida nos processadores mais modernos, que incluem atualmente até três níveis (*L1*, *L2* e *L3*); e 2) Reduzir a probabilidade de que dois blocos de memória distintos disputem pela mesma localização na memória Cache.

Uma das formas de reduzir essa probabilidade é balancear as configurações da memória Cache. Por exemplo, pode-se utilizar uma técnica de mapeamento mais flexível ou que se ajuste melhor a determinada aplicação ou classes de aplicações. Pode-se também aumentar o número de linhas da memória Cache para armazenamento de um maior número de blocos, e, com isso, pode-se reduzir a taxa de faltas.

Para ilustrar o impacto das configurações da memória Cache sobre o desempenho de execução de uma aplicação, criou-se o programa em linguagem *Assembly* da Figura 14.4. Nesse programa, é criado um array/vetor com 200 posições e o objetivo do programa é preencher cada uma dessas posições com o número 1. Observa-se que a aplicação inclui uma estrutura de repetição *While..Do*, utilizada para realizar escritas no array/vetor (Linhas 7 e 8) e atualizações das variáveis de apoio “ptr” (Linhas 10 à 12), que é um ponteiro utilizado para endereçar todas as posições do array, e “cnt” (Linhas 14 à 16), que é utilizada como condição de parada da estrutura de repetição *While..Do* (Linhas 3 à 5).

Figura 14.4. Exemplo de código *Assembly* para escrita de dados em array.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code> MOV 200 ;while (cnt < 200).</code>
4	<code> SUB cnt</code>
5	<code> JZ end</code>
6	<code>do:</code>
7	<code> MOV 1 ;*ptr = 1.</code>
8	<code> STI ptr</code>
9	
10	<code> MOV 1 ;ptr++.</code>
11	<code> ADD ptr</code>
12	<code> STA ptr</code>
13	
14	<code> MOV 1 ;cnt++.</code>
15	<code> ADD cnt</code>
16	<code> STA cnt</code>
17	
18	<code> JMP while</code>
19	
20	<code>end:</code>
21	<code> INT exit</code>
22	
23	<code>.data</code>
24	<code> ;syscall exit</code>
25	<code>exit: DD 25</code>
26	<code>ptr: DD array ;ponteiro para 1o. posicao do array.</code>
27	<code>cnt: DD 0 ;variavel auxiliar.</code>
28	
29	<code>.bss</code>
30	<code>array: RESD 200</code>
31	
32	<code>.stack 1</code>

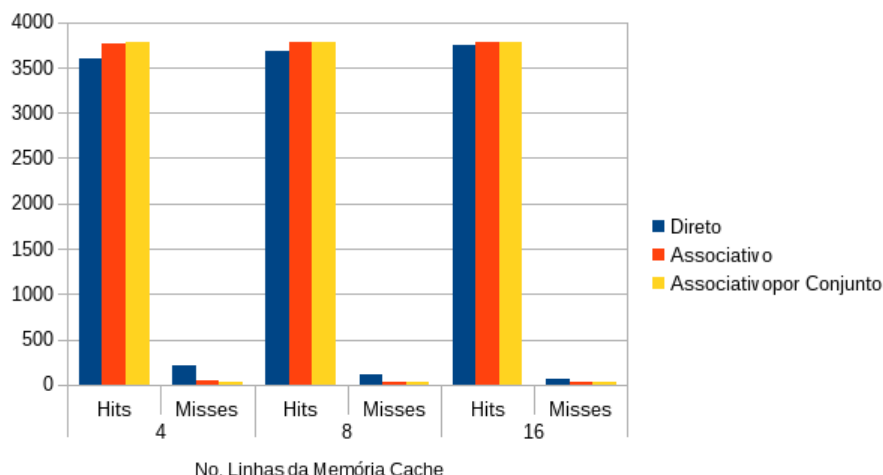
Fonte: Próprio autor (2021).

Para o experimento utilizou-se as seguintes configurações da memória Cache:

- Técnicas de mapeamento: Direto, Associativo e Associativo por Conjunto;
- Linhas de memória Cache: 4, 8 e 16;
- Política de escrita com acerto: *Write-through*; e
- Política de escrita com falta: *Write-Allocate*.

A Figura 14.5 mostra um gráfico em barras com os resultados da execução do programa da Figura 14.4, sob as diferentes configurações da memória Cache. As barras caracterizam o total de acertos (*Hits*) e faltas (*Misses*) na execução do programa, sob as diferentes configurações de técnicas de mapeamento e diferentes números de linhas da memória Cache.

Figura 14.5. Comparativo de taxas de acertos e faltas para diferentes configurações da memória Cache.



Fonte: Próprio autor (2021).

Na Figura 14.5, observa-se que a política de mapeamento Associativo por Conjunto obteve os melhores resultados (maior número de acertos) em todos os cenários, com uma sensível diferença com os resultados de mapeamento Associativo. Além disso, percebe-se que, na medida em que o número de linhas da memória Cache aumenta, os desempenhos das políticas de mapeamento também aumentam, com possível convergência.

Analisando esses resultados, percebe-se que, para o problema da Figura 14.4, fica fácil discernir qual a melhor estratégia de configuração da memória Cache. Porém, é importante ressaltar que, apesar dos bons resultados, a técnica de mapeamento Associativo por Conjunto é, dentre as demais, a mais complexa de se implementar em hardware. Além disso, um maior número de linhas da memória Cache também aumenta o consumo de área em silício no projeto do processador, uma vez que atualmente todas as memórias Cache são embarcadas. Por fim, considerando cenários onde há mais de um nível de memória Cache e Caches especializadas para dados e instruções, torna-se mais difícil a tarefa de estabelecer um compromisso entre as melhores configurações das Caches e os respectivos impactos no projeto do computador.

14.3 Boas Práticas de Programação

É possível também otimizar o desempenho de uma aplicação utilizando algumas técnicas básicas de programação. Seguem alguma delas.

14.3.1 Redução de Esforço

Viu-se que no processador Cariri não possui instruções para as operações aritméticas de multiplicação e divisão.

No entanto, apresentou-se no [Capítulo 10 - Controle de Fluxo de Execução do Programa](#), nas Figuras 10.16 e 10.18, como essas operações podem ser implementadas utilizando estruturas de repetição.

Dependendo do caso, torna-se mais eficiente utilizar apenas instruções de adição ou subtração sucessivas, ao invés de utilizar uma estrutura de repetição. Da mesma forma, pode-se aumentar o desempenho dessas operações fazendo uso de deslocamentos de bits sucessivos, seguidos por novas adições ou subtrações.

Por exemplo, a Figura 14.6, ilustra como implementar a operação de multiplicação inteira do número inteiro 2 por 10, utilizando três abordagens diferentes.

Figura 14.6. Abordagens para implementação de operação de multiplicação inteira.

Linha	Código	Código	Código
1	<code>.code</code>	<code>.code</code>	<code>.code</code>
2	<code>while:</code>	<code>LDA var ;AC=2.</code>	<code>LDA var ;AC=2.</code>
3	<code>LDA mult2 while(cnt<mult2).</code>	<code>ADD var ;AC=2+2=4.</code>	<code>SHIFT left ;AC=2*2=4.</code>
4	<code>SUB cnt</code>	<code>ADD var ;AC=4+2=6.</code>	<code>SHIFT left ;AC=4*2=8.</code>
5	<code>JZ end</code>	<code>ADD var ;AC=6+2=8.</code>	<code>SHIFT left ;AC=8*2=16.</code>
6		<code>ADD var ;AC=8+2=10.</code>	<code>ADD var ;AC=16+2=18.</code>
7	<code>LDA res ;res=res+mult1.</code>	<code>ADD var ;AC=10+2=12.</code>	<code>ADD var ;AC=18+2=20.</code>
8	<code>ADD mult1</code>	<code>ADD var ;AC=12+2=14.</code>	<code>STA res</code>
9	<code>STA res</code>	<code>ADD var ;AC=14+2=16.</code>	
10		<code>ADD var ;AC=16+2=18.</code>	<code>end:</code>
11	<code>MOV 1 ;cnt++.</code>	<code>ADD var ;AC=18+2=20.</code>	<code>INT exit</code>
12	<code>ADD cnt</code>	<code>STA res</code>	
13	<code>STA cnt</code>		<code>.data</code>
14		<code>end:</code>	<code>exit: DD 25</code>
15	<code>JMP while</code>	<code>INT exit</code>	<code>var: DD 2</code>
16			<code>res: DD 0</code>
17	<code>end:</code>	<code>.data</code>	
18	<code>INT exit</code>	<code>exit: DD 25</code>	<code>.stack 1</code>
19		<code>var: DD 2</code>	
20	<code>.data</code>	<code>res: DD 0</code>	
21	<code>exit: DD 25</code>	<code>.stack 1</code>	
22	<code>mult1: DD 2</code>		
23	<code>mult2: DD 10</code>		
24	<code>cnt: DD 0</code>		
25	<code>res: DD 0</code>		
26			
27	<code>.stack 1</code>		

(a) Multiplicação com estrutura de repetição.

(b) Multiplicação com somas sucessivas.

(c) Multiplicação com deslocamento de bits e somas sucessivas.

Fonte: Próprio autor (2021).

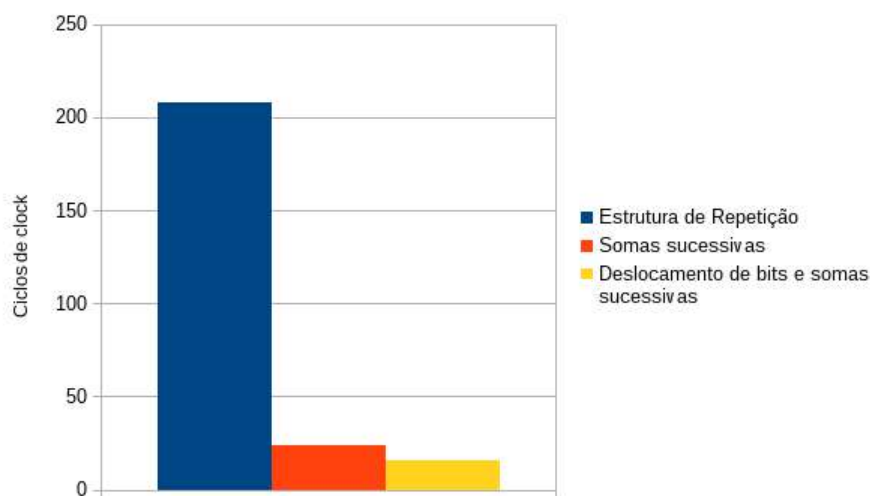
Na Figura 14.6 (a), utiliza-se a abordagem de estrutura de repetição While..Do para implementação de multiplicação por somas sucessivas; na Figura 14.6 (b), a segunda abordagem faz uso de sucessivas instruções de adição; e na Figura 14.6 (c), a terceira

abordagem utiliza deslocamentos de bits à esquerda, para implementar multiplicações por 2, seguidas por adições.

Analisando os códigos-fonte em *Assembly* das três abordagens de implementação da operação de multiplicação de 2 por 10, percebem-se diferenças, entre abordagens, do número de instruções e de variáveis que serão alocadas em memória.

A Figura 14.7 apresenta um gráfico de barras com o desempenho de execução dos programas contidos na Figura 14.6. Analisando a Figura 14.7, observa-se que apesar de conter mais instruções do que o programa em (a), o programa em (b) obteve um desempenho muito superior: para realizar a multiplicação, a abordagem em (b) consumiu 24 ciclos de clock, enquanto que em (a) consumiu-se 208 ciclos. A abordagem em (c), dentre todas, apresentou menor número de instruções e melhor desempenho, consumindo apenas 16 ciclos de clock.

Figura 14.7. Comparativo de desempenho para diferentes abordagens de implementação de multiplicação inteira.



Fonte: Próprio autor (2021).

Outro aspecto relacionado à redução do esforço, condiz com a eliminação de sub-expressões. Sub-expressões basicamente são partes de equações. A eliminação de sub-expressões consiste em analisar equações no código do programa e buscar reduzir ou remover partes dela, para minimizar o esforço na sua solução. Por exemplo, a equação:

$a = (a * 8)/2$	(1)
-----------------	-----

pode ser substituída por:

$a = a/4$	(2)
-----------	-----

Um outro exemplo, envolve criar uma variável temporária para receber o resultado de uma sub-expressão duplicada, de forma que o mesmo resultado não necessite ser computado mais de uma vez. Por exemplo, a equação:

$a = (b*c)^2 + (b*c)$	(3)
-----------------------	-----

pode ser minimizada da seguinte maneira:

$t = (b*c)$ $a = t^2 + t$	(4)
------------------------------	-----

A eliminação de sub-expressões é uma tarefa realizada com frequência por compiladores e interpretadores de linguagens de alto nível, visando otimizar o código gerado e a execução da aplicação.

14.3.2 Utilizando Registradores Eficientemente

Os registradores estão envolvidos em praticamente todas as atividades de um processador. Utilizar registradores eficientemente significa buscar minimizar acessos adicionais e/ou desnecessários à memória (a aplicação torna-se muito mais eficiente ao realizar operações com dados em registradores do que em memória).

Por exemplo, ao longo deste livro foram utilizadas duas abordagens diferentes para incrementar o valor de uma variável. Essas abordagens estão ilustradas nos dois programas da Figuras 14.8. O objetivo dos programas é incrementar o inteiro 10 contido na variável “var”.

A abordagem na Figura 14.8 (a) utiliza a variável auxiliar “one”, que é inicializada com o valor 1 e é utilizada com a instrução de adição ADD para incrementar o valor da variável “var”. Já na abordagem na Figura 14.8 (b), a instrução MOV inicializa o registrador acumulador AC com valor 1, que é adicionado à variável “var” com a instrução de adição ADD.

Figura 14.8. Abordagens para incremento de variável.

Linha	Código	Código
1	<code>.code</code>	<code>.code</code>
2	<code>LDA var ;var = var + one.</code>	<code>MOV 1 ;var1 = 1 + var1.</code>
3	<code>ADD one</code>	<code>ADD var</code>
4	<code>STA var</code>	<code>STA var</code>
5		
6	<code>end:</code>	<code>end:</code>
7	<code>INT exit</code>	<code>INT exit</code>
8		
9	<code>.data</code>	<code>.data</code>
10	<code>exit: DD 25</code>	<code>exit: DD 25</code>
11	<code>var: DD 10</code>	<code>var: DD 10</code>
12	<code>one: DD 1</code>	
13		<code>.stack 1</code>
14	<code>.stack 1</code>	

(a) Incremento com uso de variável.

(b) Incremento com uso de registrador.

Fonte: Próprio autor (2021).

Apesar dos dois programas possuírem o mesmo número de instruções, o programa da Figura 14.8 (b) é mais eficiente, pelos seguintes motivos: 1) Na instrução MOV já está incluso o operando 1, que é atribuído diretamente ao registrador acumulador AC, o que permite evitar um acesso adicional à memória para a busca do operando; 2) É possível que um acesso à memória gere uma falta (*Miss*) de memória Cache, o que faria com que se consumisse vários ciclos de *clock* até que o dado fosse lido da memória RAM e estivesse disponível no processador; e 3) Por não necessitar da variável auxiliar “one”, aloca-se menos espaço em memória, otimizando assim o consumo de memória do programa.

14.3.3 Otimizando Estruturas de Repetição

As estruturas de repetição While..Do ou For..Do, em suas implementações em *Assembly*, possuem dois pontos de desvio de fluxo de execução: no início, para o caso da condição de repetição não ser atendida; e no final, para retornar ao início da estrutura, para uma nova repetição.

Geralmente, é possível converter as estruturas de repetição While..Do ou For..Do para uma estrutura de repetição do tipo Do..While. Com isso, reduz-se a necessidade de incluir dois pontos de desvio de fluxo, pode-se reduzir o número de instruções e, por consequência, há o aumento do desempenho da estrutura de repetição.

A Figura 14.9 ilustra duas implementações diferentes para um programa que incrementa em 5 vezes o valor da variável “cnt”. Na Figura 14.9 (a) utiliza-se uma estrutura

de repetição While..Do, e, na Figura 14.9 (b), utiliza-se uma estrutura de repetição Do..While, para incrementar a variável.

Figura 14.9. Conversão entre estruturas de repetição para otimização de desempenho.

Linha	Código	Código
1	<code>.code</code>	<code>.code</code>
2	<code>while:</code>	<code>do:</code>
3	<code>MOV 5 ;while (cnt < 5).</code>	<code>MOV 1 ;cnt++.</code>
4	<code>SUB cnt</code>	<code>ADD cnt</code>
5	<code>JZ end</code>	<code>STA cnt</code>
6	<code>do:</code>	<code>while:</code>
7	<code>MOV 1 ;cnt++.</code>	<code>LDA cnt ;while(contador < 5).</code>
8	<code>ADD cnt</code>	<code>SUB max</code>
9	<code>STA cnt</code>	<code>JN do</code>
10		
11	<code>JMP while</code>	<code>end:</code>
12		<code>INT exit</code>
13	<code>end:</code>	
14	<code>INT exit</code>	<code>.data</code>
15		<code>exit: DD 25</code>
16	<code>.data</code>	<code>cnt: DD 0</code>
17	<code>exit: DD 25</code>	<code>max: DD 5</code>
18	<code>cnt: DD 0</code>	
19		<code>.stack 1</code>
20	<code>.stack 1</code>	

(a) Incremento com uso de variável.

(b) Incremento com uso de registrador.

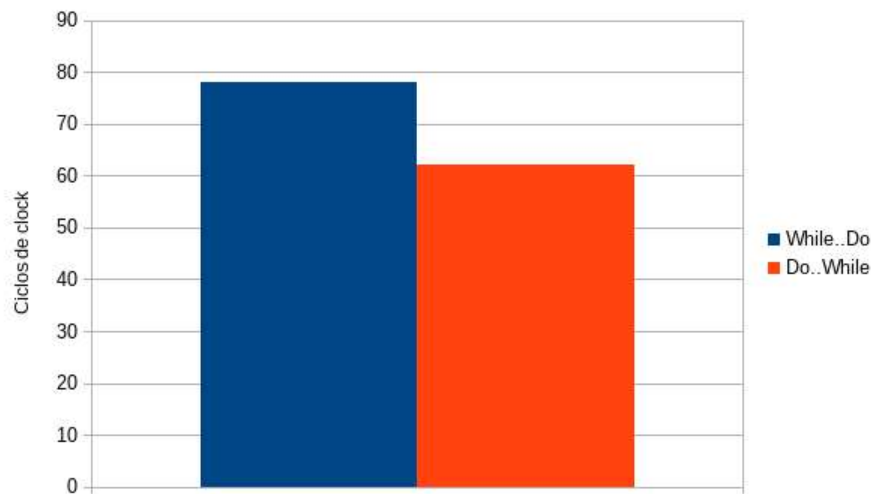
Fonte: Próprio autor (2021).

O código da Figura 14.9 (a), que implementa a estrutura de repetição While..Do, possui uma instrução a mais do que o código que implementa a estrutura de repetição Do..While, na Figura 14.9 (b). Por outro lado, este último possui uma variável a mais (variável “max”), alocada em memória.

Apesar das diferenças, ao analisar o desempenho de execução dos programas com estruturas de repetição diferentes, presentes na Figura 14.9, pode-se observar que aquele que implementa a estrutura Do..While é mais eficiente, como mostra a Figura 14.20.

Além de reduzir o número de instruções a serem executadas, a redução de pontos de desvios de fluxo de execução também pode contribuir para o aumento de desempenho da aplicação.

Figura 14.20. Comparativo de Desempenho para Diferentes Abordagens de Implementação de Estruturas de Repetição.



Fonte: Próprio autor (2021).

Em processadores que possuem suporte de *pipeline*, um desvio de fluxo de execução pode gerar uma quebra na sequência de execução das instruções no *pipeline*. Por exemplo, se uma das instruções no *pipeline* for de desvio de fluxo, quando ela for executada, as demais instruções no *pipeline*, que seriam executadas posteriormente, se tornam inválidas e terão que ser descartadas. Os compiladores e interpretadores de linguagens de alto nível realizam análises no código para reordenar a sequência de execução das instruções buscando minimizar os efeitos negativos no *pipeline* gerados por instruções de desvio de fluxo.

Outro ponto tratado pelos compiladores, é tentar desenrolar (*unroll*) estruturas de repetição. O compilador analisa a estrutura de repetição e verifica se é possível substituí-la por instruções que implementam sequencialmente (sem repetições) seu comportamento. Um exemplo de *unroll* pode ser visto nas Figuras 14.6 (a) e 14.6 (b). Nas figuras, toda a estrutura de repetição em (a) foi substituída, em (b), por instruções que fazem o trabalho que ela deveria realizar. Com isso, evita-se a necessidade de incluir instruções adicionais para controle de repetição, como para checagem de condição de repetição e desvio de fluxo de execução, e, em processadores com suporte de *pipeline*, pode-se aproveitar melhor os benefícios do *pipeline*, preenchendo-o com mais instruções.

Em (SEYFARTH, 2013) e (SANTOS; LANGLOIS, 2018), são apresentadas outras técnicas de otimização de estruturas de repetição, tratadas por compiladores, das quais podemos destacar:

- Deslocamento de códigos invariantes para fora das estruturas de repetição, evitando-se múltiplas avaliações desnecessárias nas repetições e não trarão resultados

distintos. Quando um código invariante é identificado em uma estrutura de repetição, ele pode ser deslocado para posição anterior à estrutura, atuando como um cabeçalho executado antes da estrutura de repetição;

- Fusão de duas estruturas de repetição, que atuam sobre uma mesma sequência de valores e que não possuem dependência entre elas, em apenas uma. Em geral, essa prática pode aumentar o corpo da estrutura de repetição gerada, o que faz com que haja mais instruções e, com isso, pode-se aproveitar melhor os benefícios do *pipeline*, caso o processador suporte; e
- Divisão de estrutura de repetição que atua sobre dois ou mais conjuntos de dados, caso a estrutura de repetição original exceda a capacidade de armazenamento da memória cache. De acordo com Seyfarth (2013), é preciso estabelecer um compromisso (*trade-off*) entre o uso da memória Cache e ter mais instruções no *pipeline*, de forma que, dependendo da situação, pode ser melhor dividir uma estrutura de repetição ou transformar duas ou mais estruturas em apenas uma.

14.3.4 Remoção de Recursão

Apesar do que foi dito no [Capítulo 12 - Modularização de Programas](#), que as funções recursivas são importantes para a solução de determinados tipos de problemas, sua execução tem um alto custo computacional.

Dependendo do problema, podem ser necessárias muitas chamadas recursivas da função, o que pode ocasionar:

1. Consumo elevado de memória, pois em cada chamada recursiva da função, deve-se salvar o endereço de retorno e o contexto do local da função em execução, tais como os parâmetros de entrada, as variáveis locais e os valores temporários contidos em registradores (geralmente utiliza-se a pilha do programa para o armazenamento desses dados);
2. Excessos de chamadas das instruções de chamada de função CALL e de retorno de função RET, que frequentemente são as instruções com menor desempenho, pois, manipulam a pilha do programa em memória, atualizam o valor do registrador apontador de topo de pilha (*Stack Pointer* - SP) e realizam o desvio de fluxo de execução; e
3. Desvios de fluxo de execução excessivos, que podem prejudicar o preenchimento do *pipeline*, caso seja suportado pelo processador, reduzindo assim sua eficiência.

Há diferentes técnicas para remoção de recursão em funções, como, por exemplo, tornar o comportamento da função iterativo ou, ao invés de chamar recursivamente a função, pode-se desviar o fluxo de execução do programa para o início da função.

Nesta subseção, será ilustrada a primeira técnica, em que definiu-se uma versão iterativa, que utiliza uma estrutura de repetição (ver Figura 14.21), da função recursiva de multiplicação por soma sucessivas, apresentada na Figura 12.18, para multiplicação do número inteiro 2 por 10.

O código da Figura 14.21, basicamente, fez uso do código presente na Figura 14.6 (a), que utiliza uma estrutura de repetição para implementar a operação de multiplicação por somas sucessivas, e o transformou em uma função, apresentada nas Linhas 17 à 53, da Figura 14.21. Na mesma figura, nas Linhas 3 à 7, os parâmetros da função são adicionados na pilha do programa; na Linha 9, a função iterativa de multiplicação é chamada; e, por fim, nas Linhas 11 e 12, o resultado da multiplicação é removido da pilha do programa e atribuído à variável “resultado”.

Figura 14.21. Exemplo de código Assembly para função de multiplicação inteira iterativa por somas sucessivas.

Linha	Código
1	<code>.code</code>
2	<code>main:</code>
3	<code> LDA multiplicador</code>
4	<code> SOP push</code>
5	
6	<code> LDA multiplicando</code>
7	<code> SOP push</code>
8	
9	<code> CALL multiplicacao</code>
10	
11	<code> SOP pop</code>
12	<code> STA resultado</code>
13	
14	<code>end:</code>
15	<code> INT exit</code>
16	
17	<code>multiplicacao:</code>
18	<code> SOP pop</code>
19	<code> STA R3 ;guarda endereco retorno em R3.</code>
20	
21	<code> SOP pop</code>
22	<code> STA R0 ;guarda o parametro x em R0.</code>
23	
24	<code> SOP pop</code>
25	<code> STA R1 ;guarda o paramentro y em R1.</code>
26	
27	<code> MOV 0</code>
28	<code> STA R2</code>
29	<code> STA cnt</code>
30	
31	<code>while:</code>
32	<code> LDA R1 ;while (cnt < multiplicador).</code>

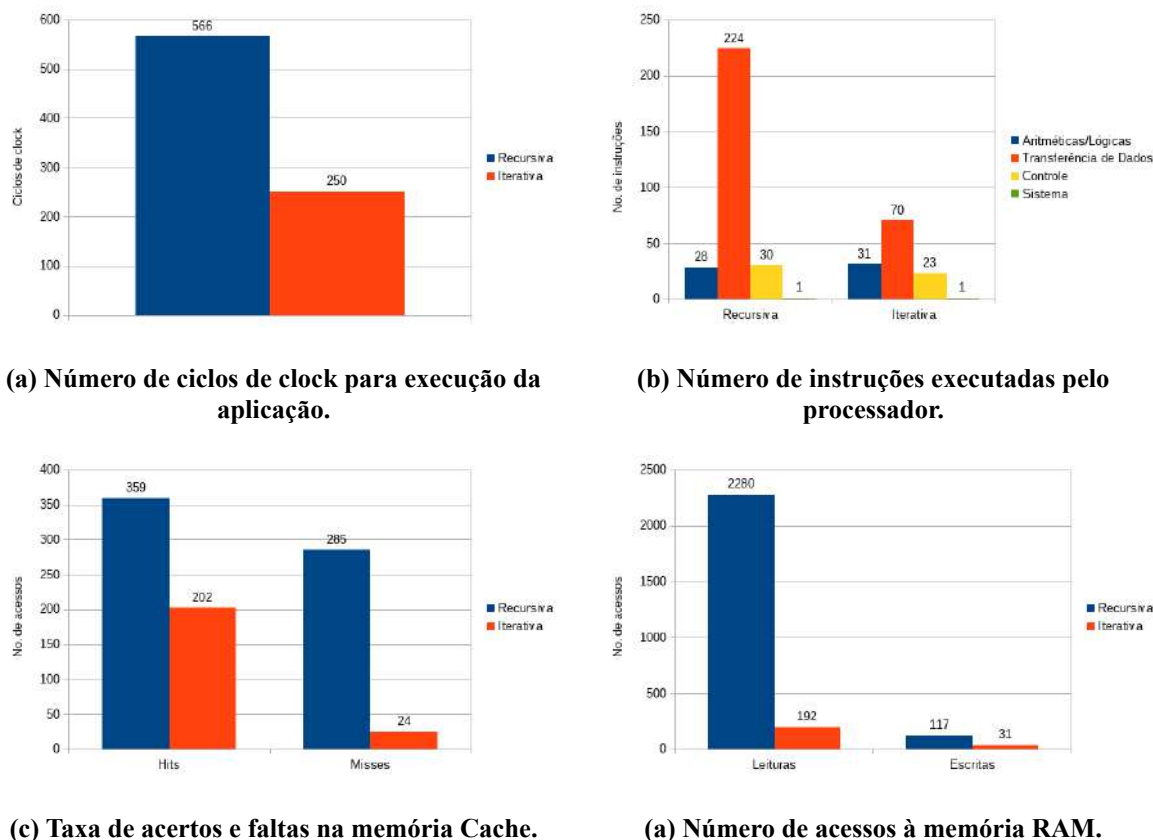
33	SUB cnt	
34	JZ return	
35	do:	
36	LDA R2	<i>;resultado = multiplicando + resultado.</i>
37	ADD R0	
38	STA R2	
39		
40	MOV 1	<i>;cnt++.</i>
41	ADD cnt	
42	STA cnt	
43		
44	JMP while	
45		
46	return:	
47	LDA R2	<i>;adiciona na pilha o resultado da multiplicacao.</i>
48	SOP push	
49		
50	LDA R3	
51	SOP push	<i>;adiciona na pilha o endereco de retorno da funcao.</i>
52		
53	RET	
54		
55	.data	
56		<i>;syscall exit</i>
57	exit: DD 25	
58	multiplicando: DD 2	
59	multiplicador: DD 10	
60	resultado: DD 0	
61	push: DD 0	
62	pop: DD 1	
63		
64	.bss	
65	R0: RESB 1	<i>;simulam registradores R0, R1, R2 e R3.</i>
66	R1: RESB 1	
67	R2: RESB 1	
68	R3: RESB 1	
69	cnt: RESB 1	<i>;variavel auxiliar para condicional do while.</i>
70		
71	.stack 3	
72		

Fonte: Próprio autor (2021).

Comparando os códigos-fontes das aplicações multiplicação recursiva (Figura 12.18) e de multiplicação iterativa (Figura 14.21), observa-se, inicialmente, que a primeira inclui mais instruções (89 contra 71) e necessita de mais memória para a pilha do programa (12 contra 3).

A Figura 14.22 traz gráficos comparativos entre os desempenhos de execução das aplicações para multiplicação recursiva e iterativa do inteiro 2 por 10.

Figura 14.22. Análise comparativa de desempenho entre aplicações de multiplicação inteira recursiva e iterativa.



Fonte: Próprio autor (2021).

Na Figura 14.22 (a), observa-se que a aplicação recursiva consumiu mais do que o dobro de ciclos de *clock* (566 contra 250) para realizar a tarefa de multiplicação.

Na Figura 14.22 (b), observa-se que, na aplicação recursiva, o processador executou 224 instruções de transferência de dados, enquanto que, na aplicação iterativa, foram necessárias apenas 70. Um número elevado de instruções de transferência de dados faz com que haja mais requisições de dados na memória Cache pelo processador, como pode ser constatado Figura 14.22 (c). Além de um número mais elevado de acessos à memória Cache, observa-se também que, as configurações utilizadas para a memória Cache não favoreceram a execução da aplicação recursiva, visto um número bastante elevado do total acertos (*Hits*) e de faltas (*Misses*).

Por fim, na Figura 14.22 (d), observa-se uma disparidade muito acentuada entre o número de acessos à memória entre as aplicações recursiva e iterativa. Isso pode ser constatado por dois motivos: número alto de instruções de transferência de dados executadas pelo processador e alta taxa de faltas (*Misses*) na memória Cache, que geraram substituições excessivas de blocos da memória RAM.

14.3.5 Outros Tipos de Otimizações

Em geral, segundo Santos e Langlois (2018), a otimização de desempenho envolve três dimensões:

1. Otimização pelo usuário: o programador deve buscar utilizar técnicas de otimização dos programas, tais como utilização de bibliotecas de rotinas com código otimizado, substituição de algoritmos e estruturas de dados por outros mais eficientes, etc.
2. Otimização independente de máquina: os compiladores e interpretadores, em geral, ao realizarem análises em programas de alto nível, buscam identificar diversas falhas e pontos de otimização, tais como: propagação de constantes (substituições no código pelo respectivo valor da constante); eliminação de código inacessível (instruções que nunca serão executadas); substituição de chamadas de funções *inline* pelo próprio código da função; fusão ou desdobramento de estruturas de repetição (ver Subseção 14.3.1.3); eliminação de sub-expressões (ver Subseção 14.3.1.1); entre outros; e
3. Otimização dependente de máquina: são otimizações realizadas com base nas seguintes técnicas: eliminação de código redundante; simplificações algébricas; redução de esforço (ver Subseção 14.3.1.1); redução e previsão de desvios de fluxo de execução (ver Subseção 14.3.1.3); otimização em termos de espaço, em que busca-se reduzir o tamanho do programa final e é voltado principalmente para microcontroladores, que possuem restrições de espaço de armazenamento de dados e instruções; entre outras.

Segundo Seyfarth (2013), é importante estudar as técnicas de otimização de desempenho utilizadas pelo compilador a fim de aprender a criar programas *Assembly* mais eficientes.

14.4 Resumo

Este capítulo tratou de três aspectos de otimização de desempenho na criação de sistemas computacionais.

O primeiro deles envolve os aspectos de paralelismo, inerentes aos projetos do computadores. Entre eles, estão: o paralelismo ao nível de instrução, como o suporte de *pipeline* e de múltiplos *pipelines* (processadores superescalares); e ao nível de tarefas ou de processos, que envolvem os processadores paralelos e multicomputadores.

O segundo está diretamente relacionado à hierarquia de memória, em que o acesso às instruções e aos dados dos programas torna-se crítico, desde que a memória RAM tem desempenho muito inferior ao do processador, e, por conta disso, utiliza-se a memória Cache como memória auxiliar. No entanto, muitos pontos devem ser tratados no projeto de computadores que utilizam memórias Caches, tais como técnicas de mapeamento, número de linhas para armazenamento de dados e múltiplos níveis de Cache.

Por fim, tratou-se de diferentes técnicas de programação em *Assembly*, que visam aumentar o desempenho das aplicações. Muitas dessas técnicas são empregadas pelos compiladores e interpretadores de linguagem de alto nível para otimizar, gerar e executar código alvo de alto desempenho.

CAPÍTULO 15 – DESENVOLVIMENTO DE PERIFÉRICOS PARA O SIMULADOR COMPSIM

No [Capítulo 13 - Entrada/Saída](#), viu-se que o simulador CompSim possui um Subsistema de Entrada/Saída que permite conectar, de forma automatizada, periféricos ao barramento de periféricos de sua plataforma de hardware virtual. Uma vez conectado ao barramento de periféricos, os componentes da plataforma já estão aptos a se comunicarem com o periférico recém-conectado. Viu-se ainda que, dentre os periféricos suportados, estão os dos tipos: virtual (Video, Teclado e VArduino) e físico (Arduino UNO e Arduino MEGA).

No entanto, o Subsistema de Entrada/Saída do CompSim permite que, além desses, outros periféricos possam ser conectados automaticamente à plataforma de hardware virtual, desde que implementem um determinado padrão de interface.

Este capítulo apresenta o padrão de interface de periféricos do Subsistema de Entrada/Saída do simulador CompSim; o uso de uma ferramenta de apoio à criação da interface; e, ilustra o processo de desenvolvimento de um periférico virtual e de um físico.

Recomenda-se que, para compreensão dos conceitos apresentados neste capítulo e implementação de novos periféricos, o leitor esteja familiarizado com a linguagem de programação Python v3.x, com o paradigma de programação orientada a objetos e com noções básicas de eletrônica analógica.

15.1 Padrão de Interface de Periféricos

O CompSim, assim como o seu Subsistema de Entrada/Saída, é codificado com a linguagem de programação Python v3.x. Assim, é preciso compreender que os periféricos devem ser desenvolvidos também na linguagem Python ou, pelo menos, implementar o padrão de interface de periféricos em Python do CompSim.

Por exemplo, os periféricos virtuais Video, Teclado e VArduino são desenvolvidos totalmente em Python. Já os periféricos físicos Arduino UNO e Arduino MEGA

implementam apenas o padrão de interface, pois o periférico em si é o próprio dispositivo eletrônico físico, uma *board* com microcontrolador, que segue a especificação aberta do Arduino, e outros componentes eletrônicos.

É possível ainda ter periféricos implementados em outras linguagens de programação. Porém, necessariamente deve haver um componente Python (*wrapper*) que implemente o padrão de interface e a comunicação com o periférico desenvolvido na outra linguagem de programação.

O padrão de interface de periféricos deve incluir pelo menos quatro arquivos. O primeiro deles consiste em um arquivo com extensão “.csd” (sigla para *CompSim Device*) e deve conter uma especificação da interface do periférico. O arquivo inclui os seguintes campos:

- “ports”: o(s) número(s) da(s) porta(s) de entrada/saída, dado que um periférico pode ser endereçado por uma ou mais portas;
- “folder”: nome do diretório, contido no diretório “devices”, onde encontram-se os arquivos referentes à interface (arquivos “.csd”) e de implementação do periférico;
- “module”: um módulo Python pode ser um arquivo de código-fonte com extensão “.py” ou um arquivo compilado pelo interpretador Python com extensão “.pyc”. Quando o interpretador Python executa um programa a partir do código-fonte em um arquivo com extensão “.py”, o interpretador gera uma representação intermediária otimizada do código, que é gravada em um arquivo com extensão “.pyc”, visando aumentar a eficiência de execução do programa;
- “controller”: é o nome da classe Python (programação orientada a objetos), que implementa o controlador do periférico. Um controlador implementa a interface plugável, de forma a permitir a comunicação da plataforma com o respectivo periférico, e o comportamento do periférico, caso seja virtual, ou a comunicação com os componentes eletrônicos, caso seja um periférico físico; e
- “identification”: consiste de uma curta descrição textual do respectivo periférico.

A Figura 15.1 ilustra um exemplo de arquivo “.csd” com a especificação da interface de um periférico simples. Na figura, o periférico será endereçado na porta 1, no diretório “devices” haverá um diretório chamado “periferico” (campo “folder”), o qual conterá um arquivo chamado “periferico.py” (ou “periferico.pyc”) (campo “module”), que consiste do módulo Python e devem incluir a definição da classe chamada “PerifericoSimples”, que implementa o controlador do periférico (campo “controller”).

Figura 15.1. Exemplo de arquivo “.csd” com especificação de interface de periférico.

Linha	Descrição
1	[Device]
2	ports = 1
3	folder = periferico
4	module = periferico
5	controller = PerifericoSimples
6	identification = Periferico simples

Fonte: Próprio autor (2021).

O segundo arquivo do padrão de interface de periféricos consiste no arquivo que implementa o controlador do periférico. Na descrição da interface da Figura 15.1, o arquivo que implementa o controlador do periférico está descrito no campo “module” e pode ser um arquivo “perifico.py” (ou “periferico.pyc”), no qual deve ser definida a classe “PerifericoSimples”, que implementa o controlador ou o comportamento do periférico.

A implementação do controlador deve seguir algumas regras básicas, definidas em um arquivo chamado “device.py”, que pode ser visto na Figura 15.2.

Figura 15.2. Arquivo “device.py” que inclui as classes base para implementação de novos periféricos.

Linha	Código
1	<code>from queue import Queue</code>
2	
3	<code>class DEVICE:</code>
4	<code> pass</code>
5	
6	<code>class INPUT_DEVICE(DEVICE):</code>
7	<code> def __init__(self):</code>
8	
9	<code> self.gui_log = Queue()</code>
10	<code> pass</code>
11	
12	<code> def read(self, data):</code>
13	<code> self.gui_log.put("INPUT DEVICE: has read something")</code>
14	<code> pass</code>
15	
16	<code>class OUTPUT_DEVICE(DEVICE):</code>
17	<code> def __init__(self):</code>
18	
19	<code> self.gui_log = Queue()</code>
20	<code> pass</code>
21	
22	<code> def write(self, data):</code>
23	<code> self.gui_log.put("INPUT DEVICE: has written something")</code>
24	<code> pass</code>

Fonte: Próprio autor (2021).

O arquivo apresentado na Figura 15.2 inclui as classes de software que o controlador do periférico deve implementar (conceito de herança em programação orientada a objetos). No caso, a classe “PerifericoSimples” deve implementar alguma das classes em “device.py”. Nas linhas 3 e 4, há a definição da classe “DEVICE”, que é a classe base de todos os tipos de periféricos. As Linhas 6 à 14 incluem a definição da classe “INPUT_DEVICE”, que consiste da classe base para implementação dos periféricos de entrada de dados. Nas Linhas 16 à 24, está definida a classe base “OUTPUT_DEVICE”, que deve ser utilizada para implementação de periféricos de saída de dados.

Ainda na Figura 15.2, analisando a classe “INPUT_DEVICE”, observa-se que, além do método construtor “__init__” (Linhas 7 à 10), ela possui o método “read” (definido nas Linhas 12 à 14), que deve ser utilizado em operações de leitura de dados. Uma operação de leitura de dados de um periférico, no CompSim, consiste na transferência de um dado disponibilizado pelo periférico para armazenamento no registrador acumulador AC do processador Cariri.

Da mesma forma, a classe “OUTPUT_DEVICE”, além do método construtor, possui o método “write” (definido nas Linhas 22 à 24), que deve ser utilizado nas operações de escrita de dados. Uma operação de escrita de dados em periférico, no CompSim, consiste na transferência do conteúdo do registrador acumulador AC para o periférico.

Observa-se ainda que, as classes “INPUT_DEVICE” e “OUTPUT_DEVICE” definem o atributo de classe “self.gui_log” (nas Linhas 9 e 19 da Figura 15.2, respectivamente), a partir da instância de um objeto da classe “Queue”. “Queue” é uma estrutura de dados do tipo fila, que pode ser utilizada para enviar mensagens sobre os status dos periféricos, durante as operações de leitura ou escrita de dados, em simulações, para que sejam exibidas no componente gráfico “Log”, na aba “LOG”, do CompSim. Dessa forma, através do log, pode-se ter um registro completo sobre os status de todas as operações de entrada e saída nos periféricos.

Um exemplo de implementação do controlador do periférico, cuja interface está descrita na Figura 15.1, pode ser visto na Figura 15.3. O periférico descrito no código da Figura 15.3 consiste de um dispositivo de saída de dados que, ao receber uma transferência de dado, ele simplesmente cria um log com o dado recebido.

Figura 15.3. Arquivo “periferico.py” que implementa um exemplo de controlador de periférico.

Linha	Código
1	<code>from device import OUTPUT_DEVICE</code>
2	<code>from queue import Queue</code>
3	
4	<code>class PerifericoSimples(OUTPUT_DEVICE):</code>
5	<code>def __init__(self):</code>
6	<code>self.gui_log = Queue()</code>
7	
8	<code>def write(self, data):</code>
9	<code>dat = format(data, "08b")</code>
10	<code>self.gui_log.put("PERIFERICO: bits recebidos: " + dat)</code>
11	
12	<code>if __name__ == "__main__":</code>
13	<code>c = PerifericoSimples()</code>

Fonte: Próprio autor (2021).

A Figura 15.3 apresenta uma descrição em linguagem Python do comportamento do controlador de um periférico de saída de dados. Nas Linhas 1 e 2, importa-se para o *workspace* a definição das classes “OUTPUT_DEVICE” e “Queue”. Nas Linhas 4 à 10, define-se a classe “PerifericoSimples”, que herda os métodos e atributos da classe “OUTPUT_DEVICE”, como pode ser visto na Linha 4.

Nas Linhas 5 e 6, está definido o construtor da classe “PerifericoSimples”, onde cria-se o atributo “self.gui_log” recebendo uma instância da classe “Queue”, que será utilizada para submissão de logs do periférico para exibição na interface gráfica do CompSim.

Como “PerifericoSimples” consiste de um periférico de escrita de dados, é necessário reescrever o método “write” para definir o comportamento do periférico. As Linhas 8 à 10 apresentam a nova implementação do método “write”, em que: na Linha 9, o dado inteiro recebido do registrador acumulador AC, presente em “data”, é convertido em uma *string*, que contém uma representação do valor recebido em notação binária de 8-bits, e é atribuída à variável “dat”; e, na Linha 10, gera-se um log da operação de escrita de dado no periférico.

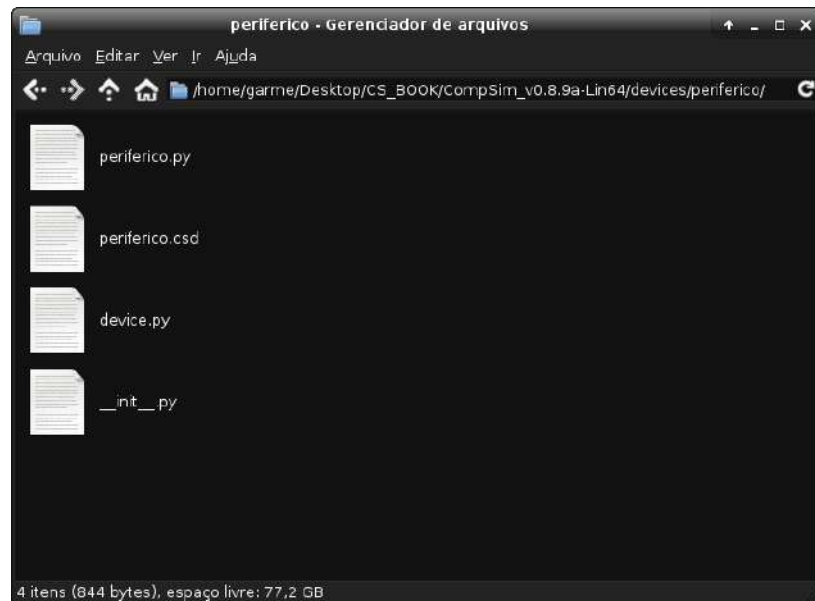
As Linhas 12 e 13, que equivalem à função “main” de programas escritos em linguagem C, são utilizadas para instanciação do periférico (Linha 13). Para uso dos periféricos no CompSim, essas linhas são desnecessárias (não são executadas pelo simulador), porém, podem ser utilizadas para realizar testes no periférico durante seu desenvolvimento.

Por fim, no diretório “periferico”, contido no diretório “devices”, além dos arquivos “periferico.csd”, “device.py” e “periferico.py”, é necessário incluir um arquivo vazio

chamado “__init__.py”. O arquivo “__init__.py”, mesmo vazio, faz com que o diretório, que contém os arquivos referentes à implementação da interface e comportamento do periférico, seja reconhecido como um módulo Python. Isto é necessário para que o CompSim possa instanciar o periférico e, posteriormente, conectá-lo ao barramento de periféricos.

A Figura 15.4 sumariza a relação de arquivos necessários para compor um periférico do simulador CompSim.

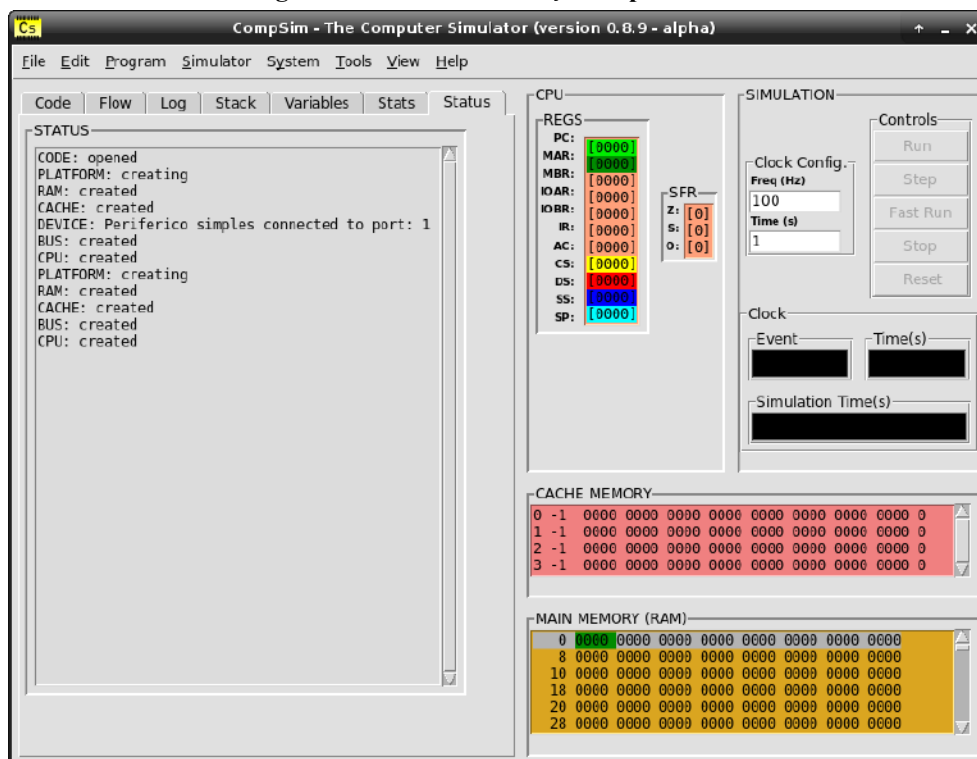
Figura 15.4. Relação de arquivos necessários para implementação de um periférico no CompSim.



Fonte: Próprio autor (2021).

Para avaliar se o periférico implementou corretamente a interface de periféricos, basta criar uma nova plataforma de hardware simulável no CompSim. Se a interface do periférico estiver correta, após a criação da plataforma, o CompSim terá instanciado o periférico, conectando-o ao barramento de periféricos. O componente gráfico “STATUS”, da aba “Status”, imprimirá a mensagem “DEVICE: Periferico simples connected to port: 1”, como pode ser vista na Figura 15.5.

Figura 15.5. Status de criação de periférico.



Fonte: Próprio autor (2021).

Por fim, para avaliar se o periférico se comporta de acordo com a sua implementação, pode-se criar uma aplicação em *Assembly* para escrita de dados no periférico, uma vez que o mesmo foi definido como do tipo de saída de dados (“OUTPUT_DEVICE”). A Figura 15.6 inclui um programa em *Assembly* que realiza uma única operação de escrita de dados, onde é transferido o inteiro 10 para o periférico.

Figura 15.6. Exemplo de código Assembly para transferência de dados para periférico.

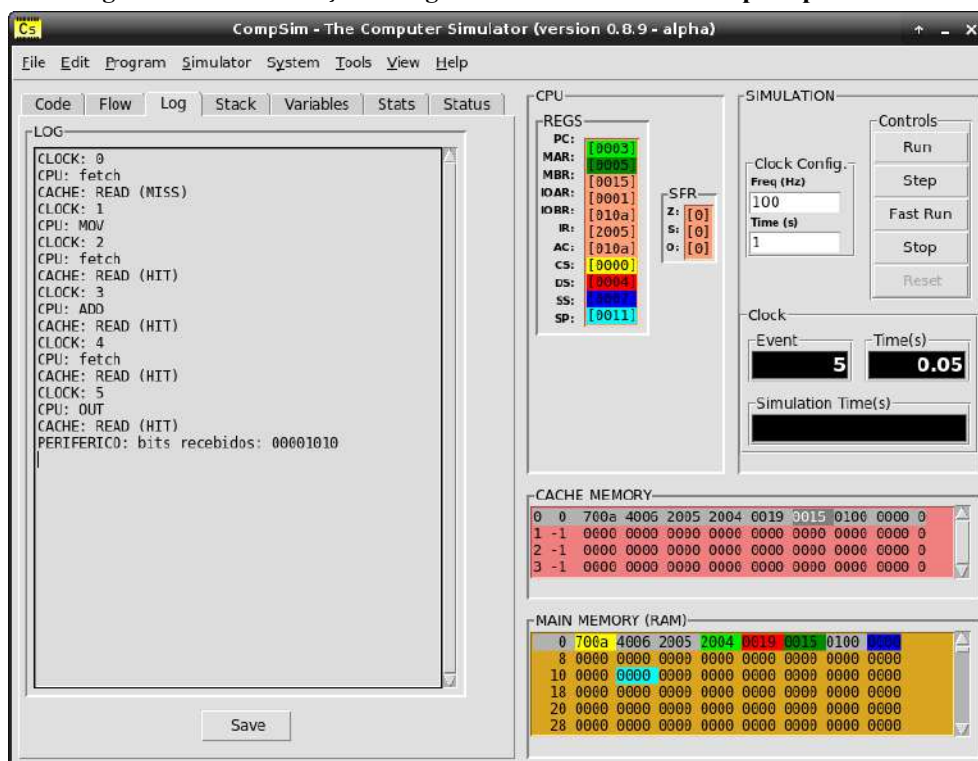
Linha	Código
1	<code>.code</code>
2	<code>MOV 10 ;AC Low recebe 10 que sera escrito no periferico.</code>
3	<code>ADD port ;Adiciona-se o endereco do periferico ao AC High.</code>
4	<code>INT output ;Realiza operacao de output.</code>
5	
6	<code>end:</code>
7	<code>INT exit</code>
8	
9	<code>.data</code>
10	<code>;syscall exit</code>
11	<code>exit: DD 25</code>
12	<code>port: DD 0x100 ;porta do periferico.</code>
13	<code>output: DD 21 ;codigo de output da instrucao INT.</code>
14	
15	<code>.stack 1</code>

Fonte: Próprio autor (2021).

Na Figura 15.6, nas Linhas 12 e 13, são definidas as variáveis que contêm a porta de endereçamento do periférico (“port” recebe o valor 0x100) e o código de operação de escrita de dados (“output” recebe o valor 21). Nas Linhas 2 à 4, carrega-se no registrador acumulador AC o valor 10, adiciona-se ao registrador AC a porta de endereçamento do periférico e realiza-se a operação de escrita de dados, respectivamente.

Para verificar se o periférico recebeu de fato o valor 10, pode-se observar, nas mensagens de log, no componente gráfico “Log”, se na aba “LOG” consta alguma mensagem do periférico. A Figura 15.7 mostra os logs de execução da aplicação da Figura 15.6.

Figura 15.7. Visualização de log de transferência de dados para periférico.



Fonte: Próprio autor (2021).

Na Figura 15.7, pode-se observar, no componente gráfico “LOG”, que, no sexto ciclo de *clock* (“CLOCK: 5”), o processador realizou uma operação de escrita de dados em periférico (“CPU: OUT”) e o periférico informou, em formato binário, qual foi o dado recebido (“PERIFERICO: bits recebidos: 00001010”).

15.2 Assistente de Criação de Interface de Periféricos

Na seção anterior, foi visto que, no processo de criação de um periférico, deve-se criar um diretório, localizado no diretório “devices”, e quatro arquivos que contêm: a descrição da

interface do periférico (arquivo com extensão “.csd”), as classes base do periférico (arquivo “device.py”); a implementação do comportamento do periférico (código-fonte do periférico em linguagem Python) e a modularização do periférico (arquivo “__init__.py”).

Visando dar suporte à automatização na criação desses elementos, o simulador CompSim inclui um assistente para criação da interface de periféricos, chamado de “Device Interface Creator” (ou simplesmente “DICreator”), disponível no menu “Tools”.

O DICreator inclui um formulário simples, que deve ser preenchido com os dados de descrição da interface e de criação do respectivo componente de software do novo periférico, como pode ser visto na Figura 15.8. Após o preenchimento do formulário, o assistente cria: o diretório do periférico, dentro do diretório “devices”; os arquivos “device.py” e “__init__.py”, bem como um *template* em linguagem Python para implementação do comportamento do periférico. Com a interface do novo periférico concluída, e já em conformidade com o padrão de interface de periféricos do CompSim, os passos seguintes concentram-se somente na implementação do comportamento do periférico.

Figura 15.8. Assistente de criação de interfaces de periféricos.

Device Interface Creator

Description

The DiCreator tool can be used to support the creation of the interface of new pluggable peripherals. You must fill the fields below with information about the interface of the new peripheral and for creating the software of its controller. After that, clicking on the "Create" button will be created a folder inside the "devices" folder for the new peripheral.

Device Interface Description:

Interface of Peripheral:

Select one or more ports: 1 1

Select main operation type: OUTPUT

Controller Software:

Module/Folder name (e.g. arduino): periferico

Controller/Class name (e.g. ArduinoUno): PerifericoSimples

ID string (e.g. Arduino Uno Board): Periferico simples

☐ Graphical User Interface (GUI)

Create Close

Fonte: Próprio autor (2021).

No formulário da Figura 15.8, deve-se informar nos respectivos campos:

- “Select one or more ports”: a(s) porta(s) de endereçamento do periférico. O intervalo de portas vai de 0 a 255, que é o número máximo de endereços de portas suportados pelo processador Cariri do CompSim. É importante destacar que um periférico pode ser endereçado por uma ou mais portas (é comum periféricos utilizarem portas

exclusivas para o recebimento de comandos de controle de operação, para transferência de dados e para informação de status);

- “Select main operation type”: se o periférico é do tipo de entrada (“INPUT”), saída (“OUTPUT”) ou de entrada e saída de dados (“INPUT/OUTPUT”);
- “Module/Folder name”: o nome do diretório que conterá os arquivos referentes ao periférico;
- “Controller/Class name”: o nome da classe de software em linguagem Python, a qual incluirá a implementação do comportamento do periférico;
- “ID String”: um breve descrição textual do periférico. Essa descrição será utilizado para referenciar o periférico nos componentes da interface gráfica do CompSim;
- “Graphical User Interface (GUI)”: se o periférico terá uma interface gráfica ou não. Alguns periféricos podem ter uma interface gráfica própria, separada da interface gráfica do simulador CompSim, como são os casos dos periféricos Video, Teclado, VArduino e Arduino UNO, vistos no [Capítulo 13 - Entrada/Saída](#). Outros periféricos podem não necessitar de uma interface gráfica própria, como é o caso periférico ilustrado na seção anterior.

Após o preenchimento do formulário, considerando os dados de preenchimento da Figura 15.8, o assistente DICreator criará: o diretório “periferico”, dentro do diretório “devices”; o arquivo “periferico.csd”, com a descrição de interface do periférico (o conteúdo do arquivo será idêntico ao da Figura 15.1); o arquivo “device.py”, com conteúdo idêntico ao da Figura 15.2; o arquivo vazio “__init__.py”, para tornar o diretório do novo periférico em um módulo da linguagem Python; e, o arquivo “periferico.py”, que é um *template* para implementação do comportamento do periférico, como pode ser visto na Figura 15.9.

O template da Figura 15.9, inclui:

- nas Linha 1 e 2, a importação da definição de classe base para periférico de saída de dados (“OUTPUT_DEVICE”) e a importação da definição da estrutura de dados do tipo fila “Queue”;
- Nas Linhas 4 à 14, a definição da classe “PerifericoSimples”;
- Nas Linhas 5 à 9, o método construtor “__init__”, da classe “PerifericoSimples”, onde é criada a fila envio de logs do periférico para impressão na interface gráfica do CompSim;
- Nas Linhas 11 à 14, o método de escrita de dados “write”, da classe “PerifericoSimples”, em que, a instrução da Linha 12 pode ser utilizada para registro

de logs, e a da Linha 14 converte o dado recebido, na operação de escrita, para um número inteiro de 8-bits; e

- Nas Linhas 17 e 18, a instruções para instanciar o objeto de software do periférico.

Figura 15.9. Arquivo “periferico.py”: template para implementação de controlador de periférico.

Linha	Código
1	<code>from device import OUTPUT_DEVICE</code>
2	<code>from queue import Queue</code>
3	
4	<code>class PerifericoSimples(OUTPUT_DEVICE):</code>
5	<code> def __init__(self):</code>
6	
7	<code> self.gui_log = Queue()</code>
8	
9	<code> print("Device PerifericoSimples created!")</code>
10	
11	<code> def write(self, data):</code>
12	<code> self.gui_log.put("INPUT DEVICE: has written something")</code>
13	
14	<code> dat = int(format(data, "016b") [8:16], 2)</code>
15	
16	
17	<code>if __name__ == "__main__":</code>
18	<code> c = PerifericoSimples()</code>

Fonte: Próprio autor (2021).

Nesta seção, como o processo de criação considerou um dispositivo de saída dados, analisando o template da Figura 15.9, percebe-se que o comportamento do periférico criado pode ser definido no método “write”.

Caso seja definido um periférico de entrada de dados, ao invés do método “write”, haverá um método “read”. Por fim, é possível ainda ter periféricos em que pode-se realizar operações de entrada e de saída de dados (“INPUT/OUTPUT”), e neles constarão tanto o método “read” quando o “write”.

15.3 Exemplos de Criação de Periféricos

Esta seção traz dois exemplos de implementações de periféricos, sendo um deles do tipo virtual e o outro do tipo físico. Destaca-se que, ao contrário dos exemplos anteriores deste capítulo, nesta seção, os periféricos conterão interfaces gráficas simples.

15.3.1 Exemplo de Periférico Virtual: Visor Numérico

O periférico virtual “Visor Numérico” tem como objetivo imprimir um número inteiro nos formatos decimal, binário e hexadecimal. O número impresso será enviado a ele, através de uma operação de escrita de dados.

Para sua implementação, inicialmente, deve-se criar a interface de conexão do novo periférico, com o suporte do DICEator, como mostra a Figura 15.10.

Figura 15.10. Descrição da interface do periférico “Visor Numérico”.

Fonte: Próprio autor (2021).

No preenchimento do formulário da Figura 15.10, definiu-se para o periférico: 1) endereçamento na porta 0; 2) Tipo de saída de dados (“OUTPUT”); 3) diretório “visornumerico”, que será criado no diretório “devices”; 4) classe de software Python chamada “VisorNumerico”; 5) *String* de descrição “Visor Numerico”; e 6) marcou-se a caixa de seleção “Graphical User Interface (GUI)”, um vez que o periférico terá uma interface gráfica.

Após pressionar o botão “Create” no formulário da Figura 15.10, será criado o diretório “visornumerico”, no diretório “devices”, e nele serão criados os arquivos “device.py”, “__init__.py”, “visornumerico.csd” e “visornumerico.py”.

Uma implementação do comportamento do periférico “Visor Numérico” está incluída no arquivo “visornumerico.py”, que pode ser visto na Figura 15.11.

Figura 15.11. Arquivo “visornumerico.py”: implementação de controlador do periférico “Visor Numérico”.

Linha	Código
1	<code>from device import OUTPUT_DEVICE</code>
2	<code>from tkinter import *</code>
3	<code>from queue import Queue</code>
4	
5	<code>class VirsorNumerico(OUTPUT_DEVICE, Toplevel):</code>
6	<code> def __init__(self, master=None):</code>
7	<code> Toplevel.__init__(self, master)</code>
8	<code> self.geometry("260x70")</code>
9	<code> self.wm_title("VISOR NUMERICO")</code>
10	<code> self.protocol("WM_DELETE_WINDOW", self.disable_event)</code>
11	
12	<code> self.gui_log = Queue()</code>
13	
14	<code> self.main_frame = LabelFrame(self, bd=2, relief=SUNKEN, text="")</code>
15	<code> self.main_frame.grid(row=0, column=0, padx=5, pady=5)</code>
16	
17	<code> self.some_label = Label(self.main_frame, text="000 00000000b 0x00", \</code>
18	<code> fg='green2', bg='black')</code>
19	<code> self.some_label.config(font=("Courier", 16, "bold"))</code>
20	<code> self.some_label.grid(row=0, column=0, padx=5, pady=5)</code>
21	<code> def disable_event(self):</code>
22	<code> pass</code>
23	
24	<code> def write(self, data):</code>
25	<code> self.gui_log.put("VISORNUMERICO: received %d" % data)</code>
26	<code> binario = format(data, "08b")</code>
27	<code> decimal = format(data, "03d")</code>
28	<code> hexadecimal = format(data, "02x")</code>
29	<code> texto = decimal + " " + binario + "b 0x" + hexadecimal</code>
30	<code> self.some_label.config(text=texto)</code>
31	
32	<code>if __name__ == "__main__":</code>
33	<code> root = Tk()</code>
34	<code> c = VirsorNumerico()</code>
35	<code> root.mainloop()</code>

Fonte: Próprio autor (2021).

Na Figura 15.11, nas Linhas:

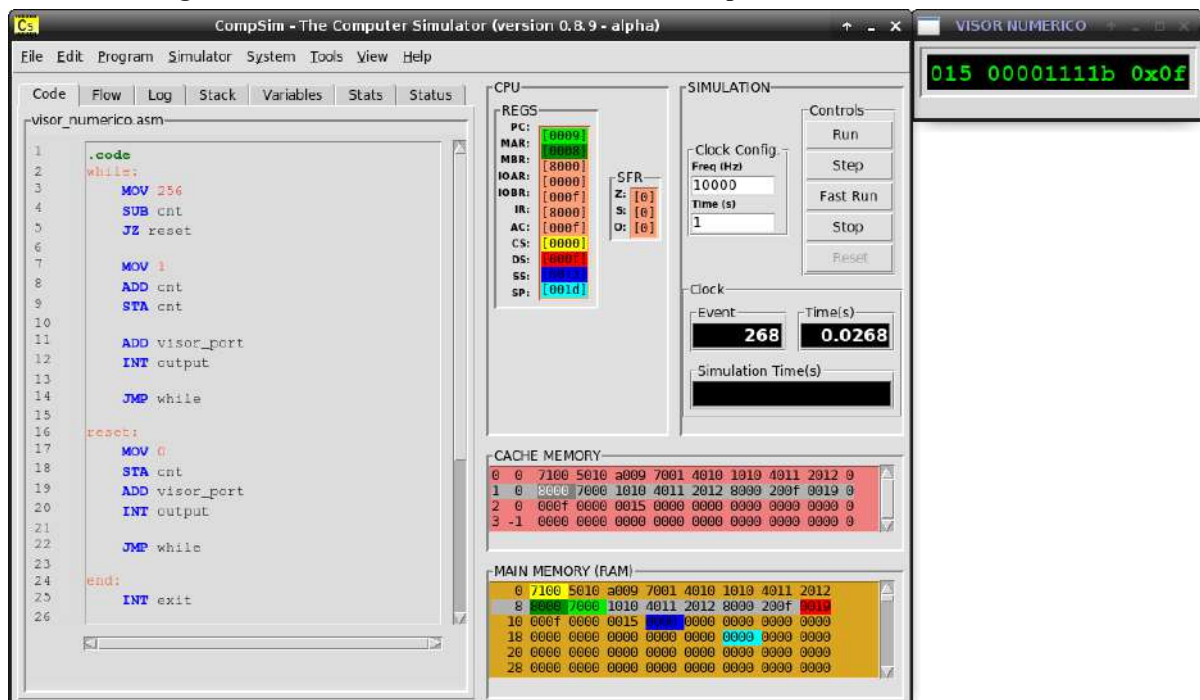
- 1 à 3, importa-se para o programa a definição do tipo de periférico (“OUTPUT_DEVICE”), do framework gráfico “tkinter”, utilizado para compor a interface gráfica do periférico, e da fila de comunicação de logs “Queue”;
- 5 à 30, define-se a implementação do controlador do periférico, através da classe “VisorNumerico”;
- 6 à 19, define-se o método construtor da classe “VisorNumerico”, em que nas Linhas:
 - 7 à 10, cria-se a janela gráfica com suas dimensões, texto da barra de título e desabilita-se o fechamento da janela ao clicar no botão com “x”, no canto superior direito;
 - 12, cria-se a fila para envio de mensagens de log;

Capítulo 15 - Desenvolvimento de Periféricos para o Simulador CompSim

- 14 e 15, cria-se um container (“LabelFrame”) que conterà os componentes gráficos do periférico;
- 17 à 19, define-se o componente gráfico “Label”, que mostrará os números recebidos, em operações de escrita de dados. O componente é configurado para mostrar uma saída inicial (*string* “000 00000000b 0x00”), para mostrar os números em cor verde com cor de fundo preta, bem como em fonte Courier, em negrito e tamanho 16;
- 20 e 21, define-se o método que desabilita a fechamento da janela com o click no botão com “x”, no canto superior direito;
- 24 à 30, define-se o método “write”, em que nas Linhas:
 - 25, gera-se um log do número inteiro recebido;
 - 26 à 28, converte-se o inteiro recebido para strings com representação decimal, binária e hexadecimal; e
 - 29 e 30, agrupa-se as representações em bases distintas do inteiro recebido em uma única *string*, que, em seguida, será impressa na interface gráfica do periférico.

A Figura 15.12 mostra a interface gráfica do periférico criado, com a impressão do número inteiro 15 em bases decimal (“015”), binária (“00001111b”) e hexadecimal (“0x0f”)

Figura 15.12. Visualização de escrita de inteiro no periférico “Visor Numérico”.



Fonte: Próprio autor (2021).

Para impressão de inteiros no periférico “Visor Numérico”, criou-se o programa em *Assembly* da Figura 15.13, o qual possui uma estrutura de repetição que realiza a impressão dos números no intervalo numérico de 0 a 255, de forma crescente e com repetições da impressão da sequência.

Figura 15.13. Exemplo de código Assembly para escrita de números inteiros no periférico “Visor Numérico”.

Linha	Código
1	<code>.code</code>
2	<code>while:</code>
3	<code> MOV 256 ;while (cnt < 256).</code>
4	<code> SUB cnt</code>
5	<code> JZ reset</code>
6	<code>do:</code>
7	<code> ADD visor_port ;imprime inteiro no Visor Numerico.</code>
8	<code> INT output</code>
9	
10	<code> MOV 1 ;cnt++.</code>
11	<code> ADD cnt</code>
12	<code> STA cnt</code>
13	
14	<code> JMP while</code>
15	
16	<code>reset:</code>
17	<code> MOV 0 ;cnt = 0.</code>
18	<code> STA cnt</code>
19	
20	<code> JMP while</code>
21	
22	<code>end:</code>
23	<code> INT exit</code>
24	
25	<code>.data</code>
26	<code> ;syscall exit</code>
27	<code>exit: DD 25</code>
28	<code>cnt: DD 0</code>
29	<code>visor_port: DD 0x000 ;porta do periferico Visor Numerico.</code>
30	<code>output: DD 21 ;codigo de output da instrucao INT.</code>
31	
32	<code>.stack 1</code>

Fonte: Próprio autor (2021).

No código *Assembly* da Figura 15.13, nas Linhas 30 à 32, são definidas as variáveis “cnt”, utilizada para conter a sequência numérico de 0 a 255, “visor_port”, que consiste da porta de endereçamento do periférico, e “output”, que é o código de operação de escrita de dados da instrução INT.

As Linhas 2 à 14 incluem a estrutura de repetição, em que: nas Linhas 3 à 5, está contida a condicional de repetição (enquanto o valor da variável “cnt” for menor do que 256); nas Linhas 7 e 8, realiza-se operação de escrita do valor em “cnt” no periférico “Visor Numérico”; e, nas Linhas de 10 à 12, a variável “cnt” é incrementada. Caso o valor da variável “cnt” ultrapasse o valor de 255, o fluxo de execução da estrutura de repetição é

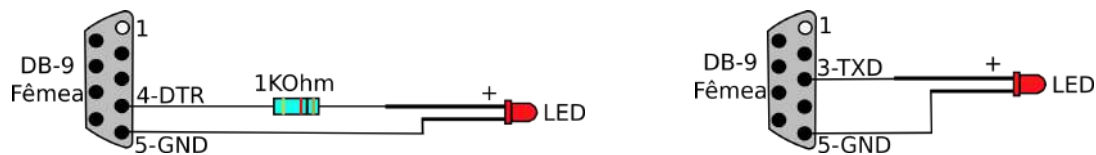
desviado para as Linhas 16 à 20, que reiniciam o valor de “cnt” e retornam o fluxo de execução para a estrutura de repetição, reiniciando a impressão da sequência numérica.

15.3.2 Exemplo de Periférico Físico: Led Serial

O periférico físico “Led Serial” consiste de um periférico de entrada e saída, que contém, em sua interface gráfica, um botão, que ao ser pressionado, acende um led conectado à porta serial do computador, em que o simulador CompSim está executando. Para tanto, serão realizadas operações de leitura de dados no periférico, para verificar o status de pressionamento do botão, na interface gráfica, e caso ele tenha sido pressionado, será realizada uma operação de escrita no periférico físico para acender o led por alguns instantes.

O periférico “Led Serial” é dividido em duas partes. A primeira parte consiste de um pequeno circuito eletrônico que inclui um conector serial do tipo DB9 fêmea, para conexão na porta serial do computador, um resistor de 10K Ohm e um led. Será utilizado o protocolo serial RS232 (RS232, 2021) para comunicação entre o CompSim e o periférico físico. A composição do circuito pode ser vista na Figura 15.14.

Figura 15.14. Esquemático do circuito eletrônico do periférico físico “Led Serial”.



(a) Periférico Físico Serial para Desktop.

(b) Periférico Físico Serial para Notebook.

Fonte: Próprio autor (2021).

Na Figura 15.14, há duas composições do circuito eletrônico “Led Serial”, sendo que o circuito em (a) é para uso na porta serial nativa de computadores *desktop* e, em (b), o circuito deve ser utilizado com adaptador USB Serial RS232 (ver Figura 15.15) para conexão em portas USB (ideal para uso em *notebooks*, que não possuem interface serial, por exemplo).

Figura 15.15. Adaptador USB Serial RS232.



Fonte: Próprio autor (2021).

Na Figura 15.14 (a), no circuito, o resistor de 10k Ohm conecta o pólo positivo do led ao pino 4 (*Data Terminal Ready* - DTR) do conector serial DB9 fêmea. O pólo negativo do led é conectado ao pino 5 (*Ground* - GND) do conector. Ao realizar uma operação de transferência de dados na porta serial, o pino DTR é ativado e somente é desativado quando a transferência é encerrada (DTR é utilizado para sinalizar que o transmissor está pronto para enviar uma nova sequência de bits). Dessa forma, aproveita-se a corrente elétrica em DTR para, ao realizar uma transferência de dados na interface serial, acender o led. O resistor de 10K Ohm impede que o led queime com as tensões -3 a -15V (para representação do nível lógico “OFF”) e +3 a +15V (para o nível lógico “ON”).

Na Figura 15.14 (b), o circuito não necessita do resistor de 10K Ohm, pois o adaptador USB Serial RS232 já realiza os ajustes de tensão. Além disso, o pólo positivo do led será conectado ao pino 3 (*Transmitted Data* - TxD), que, no protocolo serial, é o utilizado para transferência de bits do transmissor para o receptor. Nesta seção, para a composição do periférico “Led Serial”, utilizou-se esta segunda configuração, como mostra a Figura 15.16.

Figura 15.16. Circuito eletrônico do periférico “Led Serial”.



Fonte: Próprio autor (2021).

A segunda parte do periférico “Led Serial” inclui a descrição da sua interface de software de periférico (arquivo “.csd”) e da implementação em Python do comportamento do periférico.

Para gerar a descrição da interface de periférico, pode-se utilizar a ferramenta DICreator, como mostra a Figura 15.17. Na figura, o periférico “Led Serial”: 1) será endereçado nas portas 1 e 2; 2) será um periférico de entrada e saída (“INPUT/OUTPUT”); 3) estará contido no diretório “ledserial”, o qual estará contido no diretório “devices”; 4) terá o nome “LedSerial” para a classe de software em Python que implementa o comportamento do controlador; 5) contém a *string* de descrição “Led Serial”; e 6) marcou-se a caixa de seleção “Graphical User Interface (GUI)”.

Figura 15.17. Descrição da interface do periférico “Led Serial”.



Fonte: Próprio autor (2021).

Após pressionar o botão “Create”, no formulário da Figura 15.17, será criado o diretório “ledserial”, no diretório “devices”, e nele serão criados os arquivos “device.py”, “__init__.py”, “ledserial.csd” e “ledserial.py”. Uma implementação do comportamento do periférico “Led Serial” pode ser vista no arquivo “ledserial.py”, apresentado na Figura 15.18.

Figura 15.18. Arquivo “ledserial.py”: implementação de controlador do periférico “Led Serial”.

Linha	Código
1	<code>from device import INPUT_DEVICE, OUTPUT_DEVICE</code>
2	<code>from tkinter import *</code>
3	<code>from queue import Queue</code>
4	
5	<code>import serial</code>
6	
7	<code>class LedSerial(INPUT_DEVICE, OUTPUT_DEVICE, Toplevel):</code>
8	<code>def __init__(self, master=None):</code>
9	<code>Toplevel.__init__(self, master)</code>
10	<code>self.wm_title("LS")</code>
11	<code>self.geometry("150x70")</code>
12	<code>self.protocol("WM_DELETE_WINDOW", self.disable_event)</code>
13	
14	<code>self.gui_log = Queue()</code>
15	
16	<code>self.main_frame = LabelFrame(self, bd=2, relief=SUNKEN, \</code>
17	<code>text="Led Serial")</code>
18	<code>self.main_frame.pack()</code>
19	<code>self.b = Button(self.main_frame, text=' Liga ', \</code>
20	<code>command=self.set_liga_led)</code>
21	<code>self.b.grid(row=0, column=0, padx=5, pady=5)</code>
22	
23	<code>self.registrador = 0</code>
24	<code>self.led = serial.Serial("/dev/ttyUSB0", 9600)</code>
25	<code>for i in range(500):</code>
26	<code>self.led.write(b"a")</code>
27	
28	<code>def disable_event(self):</code>
29	<code>pass</code>
30	
31	<code>def set_liga_led(self):</code>
32	<code>self.registrador = 1</code>
33	
34	<code>def read(self, data):</code>
35	<code>dat = int(format(data, "016b")[8:16], 2)</code>
36	<code>add = int(format(data, "016b")[0:8], 2)</code>
37	
38	<code>if add == 1:</code>
39	<code>self.gui_log.put("LED SERIAL: leitura: %d" % (self.registrador))</code>
40	<code>return self.registrador</code>
41	
42	<code>def write(self, data):</code>
43	<code>dat = int(format(data, "016b")[8:16], 2)</code>
44	<code>add = int(format(data, "016b")[0:8], 2)</code>
45	
46	<code>self.gui_log.put("LED SERIAL: escrita: %d" % (dat))</code>
47	
48	<code>if add == 2:</code>
49	<code>if dat == 1:</code>
50	<code>for i in range(300):</code>
51	<code>self.led.write(b"a")</code>

52	<code>self.registador = 0</code>
53	
54	<code>if __name__ == "__main__":</code>
55	<code> root = Tk()</code>
56	<code> c = LedSerial()</code>
57	<code> root.mainloop()</code>

Fonte: Próprio autor (2021).

Na Figura 15.18, nas Linhas:

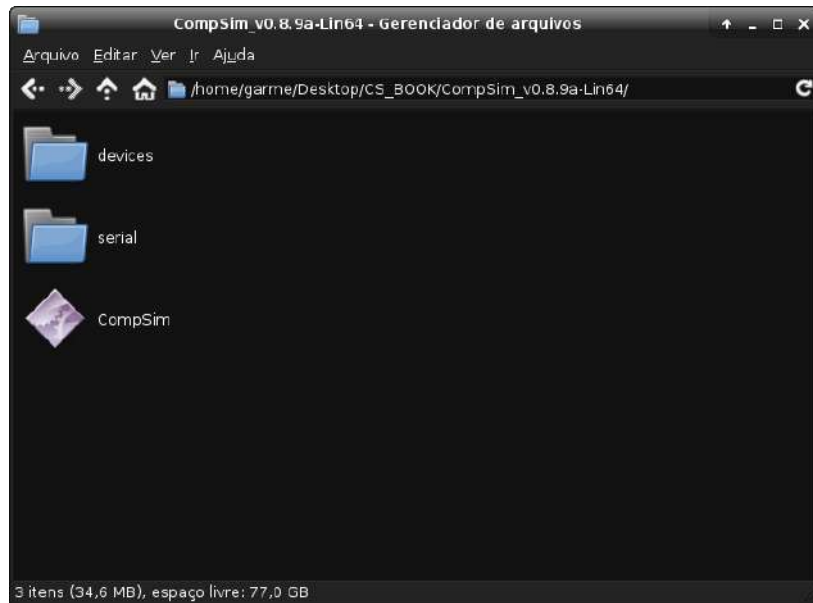
- 1 à 3, importa-se para o programa a definição dos tipos de periférico (“INPUT_DEVICE” e “OUTPUT_DEVICE”), do framework gráfico “tkinter”, utilizado para compor a interface gráfica do periférico, e da fila de comunicação de logs “Queue”;
- 5, importa-se a biblioteca “serial”, necessária para comunicação com o periférico físico conectado na interface de comunicação serial;
- 7 à 53, define-se a estrutura e o comportamento do controlador do periférico, através da classe “VisorNumerico”;
- 8 à 26, define-se o método construtor da classe “VisorNumerico”, em que nas Linhas:
 - 9 à 12, cria-se a janela gráfica com suas dimensões, texto da barra de título e desabilita-se o fechamento da janela ao clicar no botão com “x”, no canto superior direito;
 - 14, cria-se a fila para envio de logs;
 - 16 e 17, cria-se um container (“LabelFrame”) que conterá os componentes gráficos do periférico;
 - 19 à 20, define-se um botão (“Button”), que, ao ser pressionado, acenderá, por alguns instantes, o led conectado na interface de comunicação serial;
 - 22, define a variável “registador”, que informa o status de pressionamento do botão;
 - 24, cria-se o objeto de software que se comunica com a interface de comunicação serial. Ao conectar o adaptador serial à porta USB, no sistema operacional GNU/Linux, normalmente, é gerado o arquivo “/dev/ttyUSB0” (no MS Windows, geralmente é identificado como “COM4”), que será utilizado para a transferência de dados, a uma taxa de 9600 bps (bits por segundo); e

- 25 e 26, realiza-se a transferência de 500 bytes para a porta serial onde está conectado o circuito do periférico “Led Serial”. Com isso, o led será aceso por alguns instantes, quando o software do controlador do periférico é criado;
- 28 e 29, define-se o método que desabilita a fechamento da janela com click no botão com “x”, no canto superior direito da janela gráfica do periférico;
- 31 e 32, define-se o método que atribui o valor 1 à variável “registrador”, para registrar o status de que botão foi pressionado. Este método é chamada sempre que o botão for pressionado;
- 34 à 40, define-se o método “read”, em que nas Linhas:
 - 35 e 36, recupera-se o conteúdo do registrador acumulador AC passado para o periférico na operação de leitura. As variáveis “dat” e “add” recebem, respectivamente, os conteúdos de “AC Low” e “AC High”;
 - 38 à 40, verifica-se se foi endereçada a porta 1 e, em caso positivo, será gerado um log com o valor da variável “registrador”, que guarda o status de pressionamento do botão, o qual é transferido, em seguida, para o processador na operação de leitura;
- 42 à 52, define-se o método “write”, em que nas Linhas:
 - 43 e 44, recupera-se o conteúdo do registrador acumulador AC passado para o periférico na operação de leitura. As variáveis “dat” e “add” recebem, respectivamente, os conteúdos de “AC Low” e “AC High”;
 - 46, gera-se um log do inteiro recebido;
 - 48 à 51, verifica-se se foi endereçada a porta 2 e se foi recebido o valor 1, na operação de escrita de dados; em caso positivo, nas Linhas 50 e 51, realiza-se a transferência de 300 bytes para a porta serial onde está conectado o circuito do periférico “Led Serial”, para que o led seja aceso por alguns instantes;
 - 52, a variável “registrador” recebe o valor 0, modificando o status do botão para “não pressionado”.

É importante ressaltar que o código do controlador do periférico “Led Serial”, exposto na Figura 15.18, utiliza a biblioteca “serial”, que não é nativa do interpretador Python. Nesse caso, para que o periférico possa utilizá-la, a biblioteca deve ser instalada no mesmo diretório onde encontra-se o arquivo executável do simulador CompSim, como mostra a Figura 15.19. Na figura, vemos o diretório “devices”, que contém o diretório com os arquivos do periférico “Led Serial”, o diretório “serial”, que deverá conter os arquivos que implementam a

biblioteca de comunicação com a interface serial, e o arquivo “CompSim”, que é o arquivo executável do simulador.

Figura 15.19. Instalação da biblioteca “serial”.



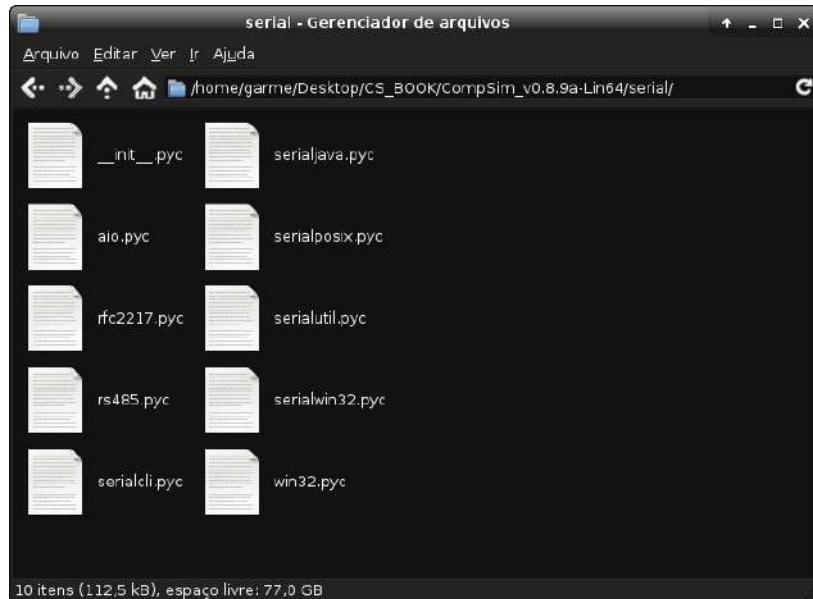
Fonte: Próprio autor (2021).

No sistema operacional GNU/Linux de 64-bits, as bibliotecas não nativas do Python, quando instaladas, podem ser encontradas no diretório “/usr/lib64/python3.8/site-packages/” (no caso, utiliza-se o interpretador Python versão 3.8). Já no sistema operacionais MS Windows, geralmente estão localizadas no diretório “C:\python38\Lib\site-packages\”.

Nesse diretório, estão contidos outros diretórios, sendo um para cada uma das bibliotecas não nativas instaladas. No caso do diretório “serial”, referente à biblioteca “serial”, nele estarão contidos vários arquivos e outros diretórios. Os arquivos com extensão “.pyc”, que são os arquivos compilados da biblioteca “serial”, deverão ser copiados para a pasta “serial” no diretório do simulador CompSim, como mostra a Figura 15.20.

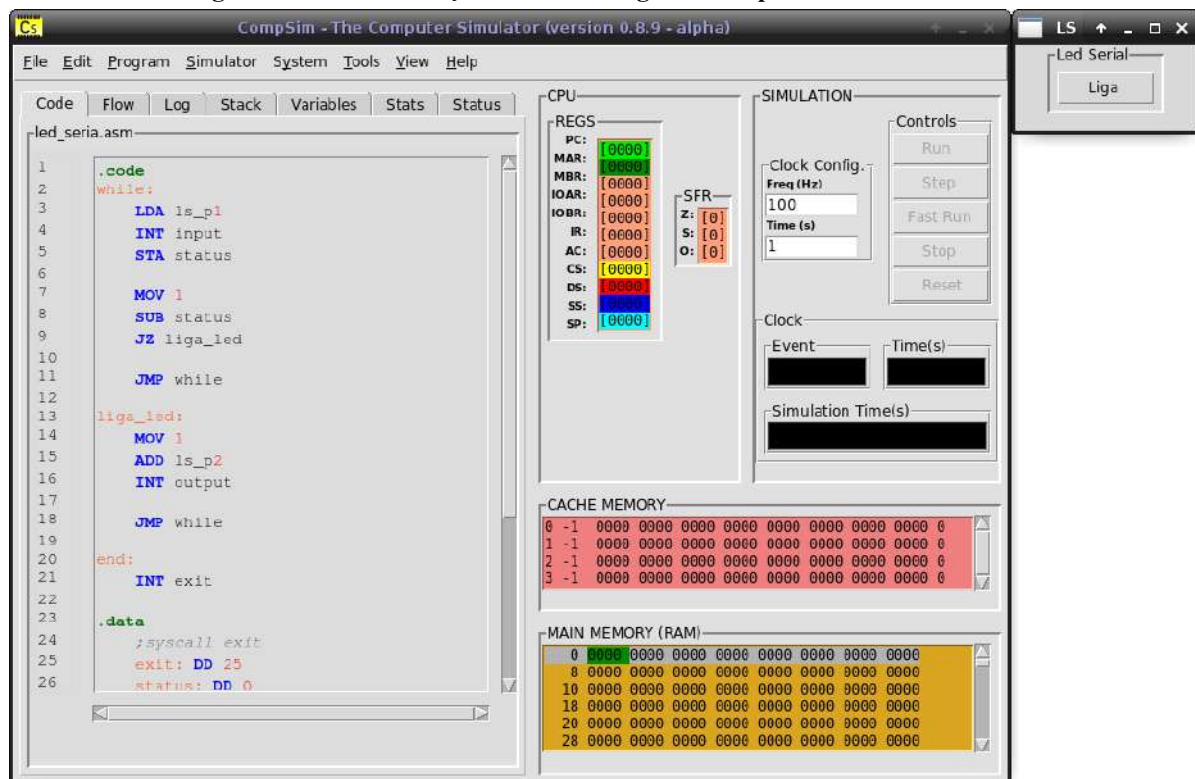
Após a instalação da biblioteca “serial” no diretório do simulador CompSim, já é possível criar uma plataforma de hardware virtual com o novo periférico conectado. A Figura 15.21 mostra, à direita, a interface gráfica do periférico “Led Serial”, com o botão para acender o led conectado à interface de comunicação serial.

Figura 15.20. Arquivos da biblioteca “serial”.



Fonte: Próprio autor (2021).

Figura 15.21. Visualização da interface gráfica do periférico “Led Serial”.



Fonte: Próprio autor (2021).

O código em *Assembly* da Figura 15.22 ilustra como acender o led conectado à interface de comunicação serial, através do periférico “Led Serial”. No código, que possui uma estrutura de repetição *While..Do*, nas Linhas 3 à 5, realiza-se uma operação de leitura de dados na porta 1 (variável “ls_p1” ou “Led Serial Porta 1”), para ler o status do botão e

gravá-lo na variável “status”. Nas Linhas 7 à 9, verifica-se o status do botão se seu valor é igual a 1 (se o botão foi pressionado) e, em caso afirmativo, o fluxo de execução do programa é desviado para o bloco nas Linhas 13 à 18, que realizam uma operação de escrita na porta 2 (variável “ls_p2” ou “Led Serial Porta 2”), para acender o led por alguns instantes, e, em seguida, desvia o fluxo de execução do programa de volta ao início do While..Do, para iniciar uma nova leitura do status do botão.

Figura 15.22. Exemplo de código Assembly para acender led conectado à porta serial.

Linha	Código
1	.code
2	while:
3	LDA ls_p1 ;Realiza leitura do status do botao.
4	INT input
5	STA status ;Salva status.
6	
7	MOV 1 ;if (status == 1).
8	SUB status
9	JZ liga_led
10	
11	JMP while
12	
13	liga_led:
14	MOV 1 ;Acende o led.
15	ADD ls_p2
16	INT output
17	
18	JMP while
19	
20	end:
21	INT exit
22	
23	.data
24	;syscall exit
25	exit: DD 25
26	status: DD 0
27	ls_p1: DD 0x100 ;porta de leitura do periferico Led Serial.
28	ls_p2: DD 0x200 ;porta de escrita do periferico Led Serial.
29	input: DD 20 ;codigo de input da instrucao INT.
30	output: DD 21 ;codigo de output da instrucao INT.
31	
32	.stack 1

Fonte: Próprio autor (2021).

15.4 Resumo

Este capítulo apresentou em detalhes o padrão de interface de periféricos do Subsistema de Entrada/Saída do simulador CompSim, ilustrando, inicialmente, como pode-se criar manualmente um periférico simples para conexão, programação e uso com a plataforma de hardware virtual do CompSim.

Na sequência, apresentou-se a ferramenta DICreator, um assistente que visa abstrair e automatizar a criação da interface de novos periféricos. Com o suporte do DICreator, é possível abstrair a criação da interface de software do periférico, e, desta forma, concentrar esforços na implementação do comportamento do periférico, durante o seu desenvolvimento.

O capítulo encerra com a demonstração dos processos de criação, programação e utilização de dois periféricos, com características diferentes. O primeiro deles, o “Visor Numérico”, que consistiu de um periférico do tipo virtual e de saída de dados, foi criado para impressão de números inteiros nas bases decimal, binária e hexadecimal. Já o segundo, o “Led Serial”, que consistiu de um periférico do tipo físico e de entrada e saída de dados, foi criado com um circuito eletrônico físico, conectado à interface de comunicação serial do computador, para acender um led, ao se clicar em um botão contido na interface gráfica do periférico.

Com os conceitos apresentados neste capítulo, acredita-se que o leitor esteja apto a explorar sua imaginação na criação e programação de novos periféricos, quer sejam virtuais ou físicos, e, com isso, compor sistemas computacionais mais complexos.

REFERÊNCIAS BIBLIOGRÁFICAS

ABD-EL-BARR, M.; EL-REWINI, H. Fundamentals of Computer Organization and Architecture. John Wiley & Sons, 2005.

ACM, Association for Computing Machinery; IEEE Computer Society, Curriculum Guidelines for Undergraduate Degree Programs in Computer Science. The Joint Task Force on Computing Curricula Association for Computing Machinery (ACM) IEEE Computer Society. December, 2013.

ADEGBIJA, T.; ROGACS, A.; PATEL, C.; GORDON-ROSS, A. Microprocessor optimizations for the internet of things: A survey. IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, v. 37, n. 1, p. 7-20, 2017.

ALCORN, P. The Core i7-8086K Review: 40 Years Of x86, 2018. Disponível em: <<https://www.tomshardware.com/reviews/intel-core-i7-8086k-cpu-8086-anniversary,5658-2.html>>. Acesso em: 05 abr. 2021.

AMARIEI, C. Arduino Development Cookbook. Packt Publishing, 2015.

AMD. AMD to Acquire Xilinx, Creating the Industry's High Performance Computing Leader, 2020. Disponível em: <<https://www.amd.com/en/press-releases/2020-10-27-amd-to-acquire-xilinx-creating-the-industry-s-high-performance-computing>>. Acesso em: 06 abr. 2021.

ARDUINO. What is Arduino? 2018. Disponível em: <<https://www.arduino.cc/en/Guide/Introduction>>. Acesso em: 23 ago. 2021.

Referências Bibliográficas

_____. analogWrite() - Documentação de Referência do Arduino. Disponível em: <<https://www.arduino.cc/reference/pt/language/functions/analog-io/analogwrite/>>. Acesso em: 07 mai. 2021.

_____. Software | Arduino. Downloads. Disponível em: <<https://www.arduino.cc/en/software>>. Acesso em: 25 mai. 2021.

AUSLÄNDER, S.; AUSLÄNDER, D.; MÜLLER, M.; WIELAND, M.; FUSSENEGGER, M. Programmable single-cell mammalian biocomputers. *Nature* 487, 2012. pp. 123–127.

BAHK, J. H.; YOUNGS, M.; YAZAWA, K.; SHAKOURI, A.; PANTCHENKO, O. An online simulator for thermoelectric cooling and power generation. *Frontiers in education Conference, IEEE*, 2013.

BALAMURALITHARA, B.; WOODS, P. C. Virtual laboratories in engineering education: The simulation lab and remote lab. *Computer Applications in Engineering Education*, Vol. 17(1), 2009. pp. 108–118.

BARKALOV, A.; TITARENKO, L.; MAZURKIEWICZ, M. Design of Embedded Systems, In: *Foundations of Embedded Systems. Studies in Systems, Decision and Control*, 195. Springer, Cham, 2019.

BLACK, M. Export to arduino: a tool to teach processor design on real hardware. *Journal of Computing Sciences in Colleges*, 31(6), 2016. pp. 21-26.

BRIGHT, P. Microsoft sheds some light on its mysterious holographic processing unit. *Ars Technica*, 2016. Disponível em: <<https://arstechnica.com/information-technology/2016/08/microsoft-sheds-some-light-on-its-mysterious-holographic-processing-unit/>>. Acesso em: 06 abr. 2021.

CARAGEA, G. C.; KECALI, F.; TZANNES, A.; VISHKIN, U. General-purpose vs. GPU: Comparison of many-cores on irregular workloads. *Proceedings of the II Usenix Workshop on Hot Topics in Parallelism*, 2010.

Referências Bibliográficas

CHAPMAN, B.; JOST, G.; VAN DER PAS, R. Using OpenMP: Portable Shared Memory Parallel Programming, MIT Press, Cambridge, MA, USA, 2007.

COMPSIM. CompSim - The Computer Simulator Website. Disponível em: <<http://compsim.crato.ifce.edu.br/>>. Acesso em: 01 abr. 2021.

_____. CompSim - The Computer Simulator - Google Groups. Disponível em: <<https://groups.google.com/g/compsim-the-computer-simulator>>. Acesso em: 01 abr. 2021.

CORMEN, T. H.; LEISERSON, C. E.; RIVEST, R. L.; STEIN, C. Introduction to Algorithms. 3rd Ed. 2009.

CRADLE. CT3600 Family Multiprocessor DSPs Advanced Datasheet. Disponível em: <<https://web.archive.org/web/20060306044333/http://www.cradle.com/downloads/CT3600-DS-A.pdf>>. Acesso em: 06 abr. 2021.

CRAGON, H. Memory Systems and Pipelined Processors. 1st. Ed. Jones and Bartlett Learning, 1996.

CUDA. CUDA Zone. NVIDIA Developer, 2021. Disponível em: <<https://developer.nvidia.com/cuda-zone>>. Acesso em: 06 abr. 2021.

DAVARE, A.; DENSMORE, D.; MEYEROWITZ, T.; PINTO, A.; SANGIOVANNI-VINCENTELLI, A.; YANG, G.; ZENG, H.; ZHU, Q. A next-generation design framework for platform-based design. In: Conference on using hardware design and verification languages (DVCon) (Vol. 152), 2007.

DEITEL, P.; DEITEL, H. C: Como Programar. 6a. Ed. Pearson Universidades, 2011.

ELECTRONICS CLUB. LEDs (Light Emitting Diodes). Disponível em: <<https://electronicsclub.info/leds.htm>>. Acesso em: 09 out. 2021

Referências Bibliográficas

FERNANDES, S. R.; SILVA, I. S. (2017) “Relato de Experiência Interdisciplinar Usando MIPS”. In: International Journal of Computer Architecture Education (IJCAE), V. 6, n. 1. pp. 52-61.

FLYNN, M. Some Computer Organizations and Their Effectiveness. IEEE Transactions on Computers, September 1972.

FREITAS, R. A. Nanomedicine Volume I: Basic Capabilities. Landes Bioscience. 1999.

GARCIA, I. A.; PACHECO, C. L.; GARCIA, J. N. Enhancing education in electronic sciences using virtual laboratories developed with effective practices. Computer Applications in Engineering Education, Vol. 22(2), 2014. pp. 283–296.

GOLDMAN, N.; BERTONE, P.; CHEN, S.; DESSIMOZ, C.; LEPROUST, E. M.; SIPOS, B.; BIRNEY, E. Towards practical, high-capacity, low-maintenance information storage in synthesized DNA. Nature, v. 494, n. 7435, p. 77-80, 2013.

GOOGLE CLOUD. Cloud Tensor Processing Units (Cloud TPUs). Disponível em: <<https://cloud.google.com/tpu/docs/tpus?hl=pt-br>>. Acesso em: 06 abr. 2021.

GROPP, W.; LUSK, E.; SKJELLUM, A. Using MPI: Portable Parallel Programming with the Message-Passing Interface, 2nd edn, MIT Press, Cambridge, MA, USA, 1999.

IEEE. ANSI/IEEE 754-1985. American National Standard — IEEE Standard for Binary Floating-Point Arithmetic. American National Standards Institute, Inc., New York, 1985.

IEEE. IEEE 754-2008. IEEE 754–2008 Standard for Floating-Point Arithmetic. August 2008.

INTEL NEWSROOM. Intel Acquisition of Altera, 2015. Disponível em: <<https://newsroom.intel.com/press-kits/intel-acquisition-of-altera/#gs.xjpbs4>>. Acesso em: 06 abr. 2021.

KHRONOS. Khronos OpenCL Working Group, The OpenCL specification 2.2-11, 2019, Disponível em:

Referências Bibliográficas

<https://www.khronos.org/registry/OpenCL/specs/2.2/html/OpenCL_API.html>. Acesso em: 14 mai. 2021.

KURNIAWAN, A. Introduction to Arduino Boards and Development. Arduino Programming with .NET and Sketch. Apress, 2017. pp. 1-19.

KUSHNER, D. The making of arduino. IEEE Spectrum, 26i, 2011.

MAGANA, A. J.; BROPHY, S. P.; BODNER, G. M. An exploratory study of engineering and science students' perceptions of nanohub.org simulation tools. In: International Journal of Engineering Education, Vol. 28(5), 2012. Pp.1019–1032.

MARKOFF, J. Smaller, faster, cheaper, over: the future of computer chips. New York Times, 2015.

MIT. Chip could bring deep learning to mobile devices: Advance could enable mobile devices to implement 'neural networks' modeled on the human brain. Massachusetts Institute of Technology, 2016. Disponível em: <www.sciencedaily.com/releases/2016/02/160203134840.htm>. Acesso em: 06 abr. 2021.

MONTEIRO, M. A. Introdução à Organização de Computadores. 4a. Ed. LTC, 2007.

MURGIA, Madhumita; WATERS, Richard. Google claims to have reached quantum supremacy. Financial Times, v. 20, 2019.

NIKOLIC, B.; RADIVOJEVIC, Z.; DJORDJEVIC, J.; MILUTINOVIC, V. A. Survey and Evaluation of Simulators Suitable for Teaching Courses in Computer Architecture and Organization. IEEE Transactions on Education, Vol. 52, No. 4, 2009.

NOLTE, D.D. Mind at Light Speed: A New Kind of Intelligence. Simon and Schuster, 2001.

NULL, L.; LOBUR, J. Princípios Básicos de Arquitetura e Organização de Computadores. Ed. Bookman, 2009.

Referências Bibliográficas

NVIDIA. CUDA Toolkit Documentation v11.3.0, 2021. Disponível em: <<https://docs.nvidia.com/cuda/>>. Acesso em: 14 mai. 2021.

OPENCL. OpenCL Overview - The Khronos Group Inc. Disponível em: <<https://www.khronos.org/opencl/>>. Acesso em: 06 abr. 2021.

PATRICK, D. R.; FARDO, S. W.; CHANDRA, V. Electronic Digital System Fundamentals. 1st Ed. River Publishers, 2020.

PATTERSON, D. A.; HENNESSY, J. L. Computer Organization and Design. The Hardware/Software Interface: RISC-V Edition. 1a. Ed. Morgan Kaufmann, 2017.

_____. Computer Organization and Design. The Hardware/Software Interface. 5th Ed. Morgan Kaufmann, 2014.

PIMENTA, T. C. Circuitos Digitais: Análise e Síntese Lógica: Aplicações em FPGA. Elsevier, 2017.

RS232. RS-232. Disponível em: <<https://en.wikipedia.org/wiki/RS-232>>. Acesso em: 12. out. 2021.

SANTOS, P. R.; LANGLOIS, T. Compiladores: da Teoria à Prática. LTC, 2018.

SBC. Sociedade Brasileira de Computação, “Currículo de Referência da SBC para Cursos de Graduação em Bacharelado em Ciência da Computação e Engenharia de Computação, 2005”, 2005. Disponível em: <<http://www.sbc.org.br/documentos-da-sbc/send/131-curriculos-de-referencia/760-curriculo-de-referencia-cc-ec-versao2005>>. Acesso em: 08 jul. 2021.

SCHILDT, H. C: Completo e Total. 3a. Ed. Pearson Universidades, 1997.

SEYFARTH, R. Introduction to 64 bit Assembly Programming for Linux and OS X. CreateSpace Independent Publishing Platform, 2013.

Referências Bibliográficas

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. Operating System Concepts. 9th Ed. Wiley, 2013.

SPUTNIK. Future of Computing: Russian Scientists Create Laser-Based Supercomputer. Sputnik International. 2018. Disponível em: <<https://sputniknews.com/science/201807031065996427-russian-optical-computer/>>. Acesso em: 06 abr. 2021.

SRIVASTAVA, D.; ATLURI, S. M. Computational nanotechnology: A current perspective. Computer Modeling in Engineering and Sciences. Vol. 3(5), 2002. pp. 531–538.

STALLINGS, W. Computer Organization and Architecture. Designing for Performance. 8th Ed. Prentice Hall, 2010.

SUNWAY. Sunway TaihuLight - Sunway MPP, Sunway SW26010 260C 1.45GHz, Sunway, 2020. Disponível em: <<https://top500.org/system/178764/>>. Acesso em: 06 abr. 2020.

TAN, H. S.; TAN, K. C.; FANG, L; WEE, M. L.; KOH, C. Using Simulations to Enhance Learning and Motivation in Machining Technology. In: Proceedings of The 17th International Conference on Computers in Education (ICCE), 2009.

TANENBAUM, A. S.; AUSTIN, T. Organização estruturada de computadores. 6a. Ed. Pearson, 2013.

TECHPOWERUP. NVIDIA GeForce RTX 3090 Specs. TechPowerUp GPU Database. Disponível em: <<https://www.techpowerup.com/gpu-specs/geforce-rtx-3090.c3622>>. Acesso em: 06 abr. 2021.

TOP500. Top500 The List. Disponível em: <<https://top500.org/>>. Acesso em: 06 abr. 2021.

URIBE, M. D. R.; MAGANA, A. J.; BAHK, J.-H.; SHAKOURI, A. Computational Simulations as Virtual Laboratories for Online Engineering Education: A Case Study in the Field of Thermoelectricity. Computer Applications in Engineering Education, Vol. 24(3), 2016. pp. 428–442.

Referências Bibliográficas

VAHID, F. Sistemas Digitais: Projeto, Otimização e HDLs. Bookman, 2008.

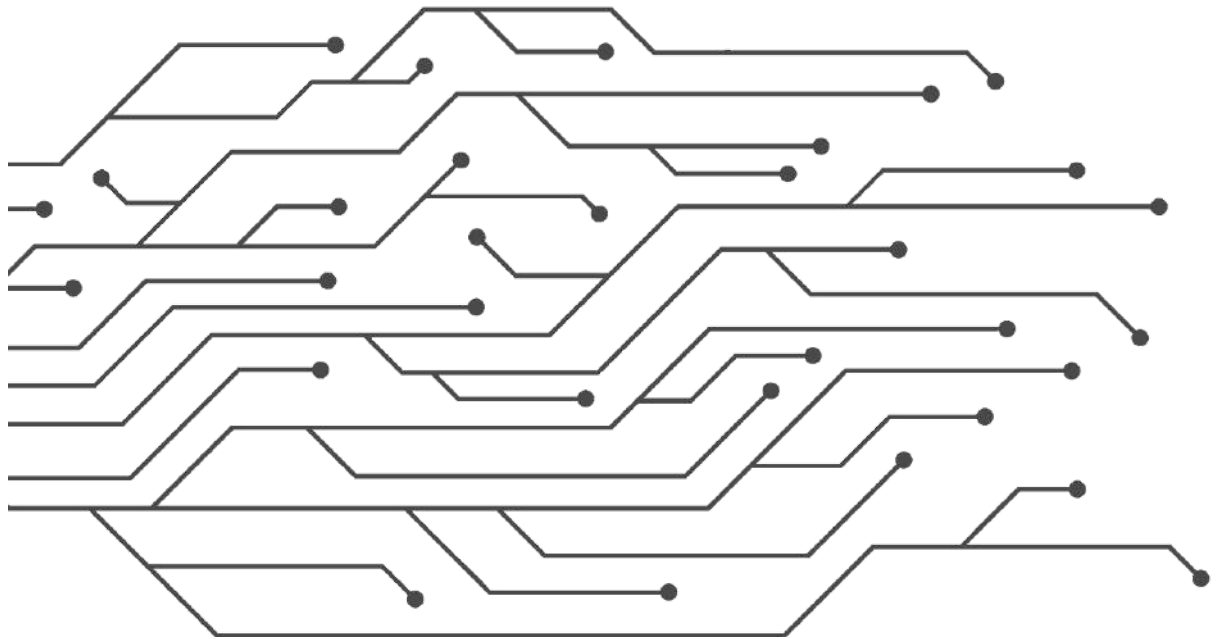
WALDROP, M. Mitchell. The chips are down for Moore's law. Nature News, v. 530, n. 7589, p. 144, 2016.

WAZLAWICK, R. S. História da Computação. 1a. Ed. GEN LTC, 2016.

WOLFFE, G. S.; YURCIK, W.; OSBORNE, H.; HOLLIDAY, M. A. Teaching computer organization/architecture with limited resources using simulators. In: Proceedings of the 33rd SIGCSE technical symposium on Computer science education, ACM SIGCSE Bulletin. Vol. 34, No. 1, 2002.

YOSHIDA, J. IoT device integrates CPU, wireless, sensors, AI engine. Embedded, 2019. Disponível em: <https://www.embedded.com/iot-device-integrates-cpu-wireless-sensors-ai-engine/>. Acesso em: 06 abr. 2021.

APÊNDICES



APÊNDICE A – INSTRUÇÕES DO PROCESSADOR CARIRI

O Quadro A.1 apresenta um resumo das instruções do processador Cariri, com respectivos *opcodes*, descrições e comportamentos (descritos em notação *Register Transfer Notation* - RTN).

Quadro A.1. Instruções do Processador Cariri.

Opcode		Instrução	Descrição	RTN
Bin	Hex			
0000	0	LDA variable LDA @address	Carrega dados da posição de memória apontada por variable ou @address em AC.	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] AC $\leftarrow$ MBR
0001	1	STA variable STA @address	Grava dados do AC na posição de memória apontada por variable ou @address	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ AC M[MAR] $\leftarrow$ MBR
0010	2	INT variable INT @address	Executa uma determinada operação, de acordo com o dado na posição de memória apontada por variable ou @address. Se o dado for igual a 20, realiza leitura de dados a partir da porta em AC High e copia em AC Low. Se o dado for igual a 21, realiza a escrita de um byte em AC Low na porta em AC High. Se o dado for igual a 25, encerra a execução do processador.	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] If MBR = 20 then IOAR $\leftarrow$ 1 IOBR $\leftarrow$ IO[IOAR] AC $\leftarrow$ IOBR If MBR = 21 then IOBR $\leftarrow$ AC IOAR $\leftarrow$ 0 IO[IOAR] $\leftarrow$ IOBR If MBR = 25 then Halt
0011	3	CALL label CALL @address	Desvia o fluxo de execução do programa para execução de uma função apontada por label ou @address. Ao encontrar uma instrução RET, retorna o fluxo de execução para a instrução seguinte a da chamada CALL.	SP $\leftarrow$ SP - 1 MAR $\leftarrow$ SP MBR $\leftarrow$ PC+1 M[MAR] $\leftarrow$ MBR PC $\leftarrow$ IR[11-0]
0100	4	ADD variable ADD @address	Adiciona o dado em AC ao dado na posição de memória apontada por variable ou @address e grava o resultado em AC.	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] AC $\leftarrow$ AC + MBR

Apêndice A - Instruções do Processador Cariri

0101	5	SUB variable SUB @address	Subtrai do dado em AC o dado na posição de memória apontada por variable ou @address e grava o resultado em AC.	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] AC $\leftarrow$ AC - MBR
0110	6	SOP variable SOP @address	Executa as operações de PUSH e POP na pilha do programa, de acordo com o dado na posição de memória apontada por variable ou @address. Se o dado for igual a 0, copia o dado em AC para o topo da pilha do programa (PUSH). Se o dado for igual a 1, remove o dado do topo da pilha do programa e copia para AC (POP).	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] If MBR = 0 then SP $\leftarrow$ SP - 1 MAR $\leftarrow$ SP MBR $\leftarrow$ ACC M[MAR] $\leftarrow$ MBR Else MAR $\leftarrow$ SP MBR $\leftarrow$ M[MAR] AC $\leftarrow$ MBR SP $\leftarrow$ SP + 1
0111	7	MOV literal MOV \$register MOV -\$register MOV variable MOV @address	Copia literal numérica para acumulador; Copia conteúdo de registrador para acumulador; Copia conteúdo de acumulador para registrador; Copia endereço de variável para o acumulador; Copia endereço para o acumulador.	AC $\leftarrow$ [0..4080] AC $\leftarrow$ CS AC $\leftarrow$ DS AC $\leftarrow$ SS AC $\leftarrow$ SP CS $\leftarrow$ AC DS $\leftarrow$ AC SS $\leftarrow$ AC SP $\leftarrow$ AC AC $\leftarrow$ variable AC $\leftarrow$ @address
1000	8	JMP label JMP @address	Carrega em PC o endereço de memória em label ou @address.	PC $\leftarrow$ IR[11-0]
1001	9	JN label JN @address	Se a flag 'S' (Signal) estiver setada em 1, carrega em PC o endereço de memória em label ou @address em PC.	If SFR[1] = 1 then PC $\leftarrow$ IR[11-0]
1010	a	JZ label JZ @address	Se a flag 'Z' (Zero) estiver setada em '1', carrega em PC o endereço de memória em label ou @address em PC.	If SFR[0] = 1 then PC $\leftarrow$ IR[11-0]
1011	b	LDI variable LDI @address	Carrega em AC o conteúdo da posição de memória apontada pela posição de memória apontada por variable ou @address.	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] MAR $\leftarrow$ MBR MBR $\leftarrow$ M[MAR] AC $\leftarrow$ MBR
1100	c	STI variable STI @address	Armazena o conteúdo de AC na posição de memória apontada pela posição de memória apontada por variable ou @address.	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] MAR $\leftarrow$ MBR MBR $\leftarrow$ AC M[MAR] $\leftarrow$ MBR
1101	d	NAND variable NAND @address	Realiza uma operação lógica NAND entre os bits do dado em AC e do dado na posição de memória apontada variable ou @address, gravando o resultado em AC.	MAR $\leftarrow$ IR[11-0] MBR $\leftarrow$ M[MAR] AC $\leftarrow$ AC ^ MBR

Apêndice A - Instruções do Processador Cariri

1110	e	SHIFT variable SHIFT @address	Realiza as operações de deslocamento aritmético de bits à direita (Right Shift) e à esquerda (Left Shift) no conteúdo de AC e grava o resultado em AC. O dado na posição de memória apontada por variable ou @address indica o sentido da operação: Se o dado for igual a 0, realiza o deslocamento de bits em AC para direita. Se o dado for igual a 1, realiza o deslocamento de bits em AC para esquerda.	$MAR \leftarrow IR[11-0]$ $MBR \leftarrow M[MAR]$ If $MBR = 0$ then $AC \leftarrow AC \gg 1$ Else $AC \leftarrow AC \ll 1$
1111	f	RET	Desvia o fluxo de execução do programa para a instrução posterior à última chamada da instrução CALL.	$MAR \leftarrow SP$ $MBR \leftarrow M[MAR]$ $PC \leftarrow MBR$ $SP \leftarrow SP + 1$

Fonte: Próprio autor (2021).

O Quadro A.2 apresenta um resumo das pseudo-instruções do processador Cariri, com descrições e exemplos de uso.

Quadro A.2. Pseudo-Instruções do Processador Cariri.

Instrução	Descrição	Exemplo
.<segmento>	Identifica o início de um segmento de memória.	<code>.code</code> <code>.data</code> <code>.bss</code> <code>.stack 10</code>
;	Delimitador de comentário de linha.	<code>;comentário de linha</code>
<label>:	Atribui um rótulo a uma instrução.	<code>main: LDA var1</code>
<name>:	Atribui um nome a uma variável.	<code>Var1: DD 1</code>
DD	Reserva espaço de memória para armazenar um número inteiro de 16 bits e o inicializa com um inteiro positivo ou negativo, nas bases decimal, binário e hexadecimal, em notação complemento a 2. Deve ser utilizada no segmento de definição de variáveis (.data).	<code>Var1: DD 1</code> <code>Var1: DD 0x1</code> <code>Var1: DD 0000000000000001b</code> <code>Var2: DD -1</code> <code>Var2: DD 0xffff</code> <code>Var2: DD 1111111111111111b</code>
DB	Reserva espaço de memória para armazenar um conjunto de bytes e o inicializa com um caractere (char) ou uma cadeia de caracteres (string). Deve ser utilizada no segmento de definição de variáveis (.data).	<code>Var1: DB 'A'</code> <code>Var2: DB "Hello, World!"</code>
RESD	Reserva espaço de memória para armazenar um ou mais números inteiros de 16-bits. Deve ser utilizada no segmento de declaração de variáveis não inicializadas (.bss).	<code>Var1: RESD 1</code> <code>Array1: RESD 10</code>
RESB	Reserva espaço de memória para armazenar um ou	<code>Var1: RESB 1</code>

Apêndice A - Instruções do Processador Cariri

	mais bytes. Deve ser utilizada no segmento de declaração de variáveis não inicializadas (.bss).	<code>Array1: RESB 10</code>
INITD	Reserva espaço de memória para armazenar um ou mais números inteiros de 16 bits e o inicializa. Deve ser utilizada no segmento de definição de variáveis (.data).	<code>Var1: INITD 1</code> <code>Array1: INITD 1,2,3,4,5</code>
INITB	Reserva espaço de memória para armazenar um ou mais bytes e/ou inteiros de 16 bits e o inicializa. Deve ser utilizada no segmento de definição de variáveis inicializadas (.data).	<code>Var1: INITB 'A'</code> <code>Var2: INITB 'A','B','C'</code> <code>Var3: INITB "Hello,World!"</code> <code>Var4: INITB "Hello",World!"</code> <code>Var5: INITB 'A',0</code> <code>Var5: INITB "Hello!",0</code>
ORG	Incrementa o contador de localização do montador. Isso significa que pode ser utilizado para adicionar espaçamentos de memória entre instruções, variáveis e/ou segmentos de memória.	<code>ORG 10</code>

Fonte: Próprio autor (2021).

APÊNDICE B – INTRODUÇÃO À ARITMÉTICA COMPUTACIONAL

No [Capítulo 4 - Introdução à Organização de Computadores](#), viu-se que os processadores incluem o subsistema chamado de Unidade Lógica e Aritmética (ULA), que é responsável pelo processamento de dados. É na ULA onde realizam-se diferentes tipos de operações, tais como as aritméticas e as lógicas, sobre diferentes tipos de dados, como números inteiros, de ponto flutuante e booleanos.

Percebe-se então que compreender como os dados numéricos são representados pelo computador e como são realizadas operações sobre eles é um passo importante no estudo e projeto de sistemas computacionais. Esse estudo constitui-se na Aritmética Computacional.

Neste apêndice, serão tratadas brevemente as representações de números inteiros e de ponto flutuante pelo computador.

B.1 Números Inteiros

Números inteiros compreendem o conjunto de números positivos, negativos e zero. O conjunto dos números inteiros é representado pela letra “ $\mathbb{Z}$ ”, com pode se visto a seguir.

$$\mathbb{Z} = \{\dots, -3, -2, -1, 0, 1, 2, 3, \dots\}$$

B.1.1 Bases Inteiras

Os números inteiros podem ser representados em diferentes bases, como a decimal, binária, hexadecimal e octal. Dentre elas, a base decimal é a que é utilizada frequentemente no dia-a-dia para contagem de objetos, medidas de tempo, finanças, entre outros.

Apêndice B - Introdução à Aritmética Computacional

A base decimal é composta pelos algarismos no intervalo de 0 a 9. A base binária é formada pelos algarismos 0 e 1. A base hexadecimal compreende o intervalo de 0 a 9 e de “a” a “f”. Por fim, a base octal é formada pelos algarismos no intervalo de 0 a 7. No quadro a seguir vê-se a correspondência de representação dos números inteiros entre bases.

Quadro B.1. Correspondência de representação de números inteiros entre bases distintas.

Decimal	Binária	Hexadecimal	Octal	Decimal	Binária	Hexadecimal	Octal
0	0	0	0	10	1010	a	12
1	1	1	1	11	1011	b	13
2	10	2	2	12	1100	c	14
3	11	3	3	13	1101	d	15
4	100	4	4	14	1110	e	16
5	101	5	5	15	1111	f	17
6	110	6	6	16	10000	10	20
7	111	7	7	17	10001	11	21
8	1000	8	10	18	10010	12	22
9	1001	9	11	19	10011	13	23

Fonte: Próprio autor (2021).

B.1.2 Conversão entre Bases Inteiras

A conversão de representação de um número inteiro entre bases pode ser realizada de diferentes formas. As subseções a seguir apresentam algumas técnicas de conversão.

B.1.2.1 Método da Expansão de Base

Um número inteiro pode ser representado em função de potências de sua base. Por exemplo, o número inteiro 163_{10} (o “10” subscrito indica a base decimal) pode ser exposto da seguinte forma:

$$163_{10} = 100_{10} + 60_{10} + 3_{10}$$

$$163_{10} = 1 \times 10^2_{10} + 6 \times 10^1_{10} + 3 \times 10^0_{10}$$

onde, por estar na base decimal, utilizou-se os algarismos no intervalo de 0 a 9, como coeficientes, e potências de base 10.

Essa forma de exposição de um número inteiro pode ser generalizada para um formato polinomial, como pode ser visto a seguir:

$UVXYZ = U \times (B)^4 + V \times (B)^3 + X \times (B)^2 + Y \times (B)^1 + Z \times (B)^0$	(1)
--	-------

onde, “U”, “V”, “X”, “Y” e “Z” são os coeficientes do polinômio, formados pelos os algarismos da base, e “B” é valor numérico da base. No caso das bases decimal, binária, hexadecimal e octal, “B” assume os valores 10, 2, 16 e 8, respectivamente.

O método de conversão por expansão de bases considera a estrutura do polinômio na equação (1) para realizar conversões entre as bases decimal, binária, hexadecimal e octal.

Tomando como exemplo o número 163_{10} , para convertê-lo para as bases binárias, hexadecimal e octal, utiliza-se o quadro de correspondência entre bases (Quadro B.1) e os seguintes passos:

Inicialmente, deve-se expor o número em base decimal em função da sua base: $163_{10} = 100_{10} + 60_{10} + 3_{10}$ $163_{10} = 1 \times 10^2_{10} + 6 \times 10^1_{10} + 3 \times 10^0_{10}$ Em seguida, deve-se converter os coeficientes e as bases das potências para base binária: $163_{10} = (1_2) \times (1010_2)^2 + (110_2) \times (1010_2)^1 + (11_2) \times (1010_2)^0$ $163_{10} = 1100100_2 + 111100_2 + 11_2$ $163_{10} = 10100011_2$

Inicialmente, deve-se expor o número em base decimal em função da sua base: $163_{10} = 100_{10} + 60_{10} + 3_{10}$ $163_{10} = 1 \times 10^2_{10} + 6 \times 10^1_{10} + 3 \times 10^0_{10}$ Em seguida, deve-se converter os coeficientes e as bases das potências para base hexadecimal: $163_{10} = (1_{16}) \times (a_{16})^2 + (6_{16}) \times (a_{16})^1 + (3_{16}) \times (a_{16})^0$ $163_{10} = 64_{16} + 3c_{16} + 3_{16}$ $163_{10} = a3_{16}$

Inicialmente, deve-se expor o número em base decimal em função da sua base:

$$163_{10} = 100_{10} + 60_{10} + 3_{10}$$

$$163_{10} = 1 \times 10^2_{10} + 6 \times 10^1_{10} + 3 \times 10^0_{10}$$

Em seguida, deve-se converter os coeficientes e as bases das potências para base octal:

$$163_{10} = (1_8) \times (12_8)^2 + (6_8) \times (12_8)^1 + (3_8) \times (12_8)^0$$

$$163_{10} = 144_8 + 74_8 + 3_8$$

$$163_{10} = 243_8$$

Os mesmos procedimentos podem ser utilizados a conversão de inteiros em bases octal e hexadecimal para decimal, e de hexadecimal para octal.

B.1.2.2 Método da Divisão

Outra forma de conversão de números inteiros entre bases, é o método da divisão. O método da divisão talvez seja o método mais utilizado por aqueles que estão sendo introduzidos em aritmética computacional, pela sua simplicidade.

Em termos gerais, o método de conversão entre bases por divisão consiste em dividir sucessivamente o número inteiro a ser convertido pelo valor da base alvo da conversão, e reserva-se os restos inteiros das divisões. Com os restos inteiros, forma-se o número convertido.

A seguir, pode-se verificar a conversão do número 163_{10} para as bases binária, hexadecimal e octal, respectivamente, por meio do método da divisão.

Inicialmente, deve-se dividir sucessivamente o número em base decimal por 2:

$$163_{10} / 2_{10} = 81_{10}, \text{ com resto } \mathbf{1}$$

$$81_{10} / 2_{10} = 40_{10}, \text{ com resto } \mathbf{1}$$

$$40_{10} / 2_{10} = 20_{10}, \text{ com resto } \mathbf{0}$$

$$20_{10} / 2_{10} = 10_{10}, \text{ com resto } \mathbf{0}$$

$$10_{10} / 2_{10} = 5_{10}, \text{ com resto } \mathbf{0}$$

$$5_{10} / 2_{10} = 2_{10}, \text{ com resto } \mathbf{1}$$

$$2_{10} / 2_{10} = 1_{10}, \text{ com resto } \mathbf{0}$$

$$1_{10} / 2_{10} = 0_{10}, \text{ com resto } \mathbf{1}$$

Agrupando os restos das divisões inteiras em ordem inversa, tem-se o número convertido:

$$163_{10} = 10100011_2$$

Inicialmente, deve-se dividir sucessivamente o número em base decimal por 16:

$$163_{10} / 16_{10} = 10_{10}, \text{ com resto } 3$$

$$10_{10} / 16_{10} = 0, \text{ com resto } 10 \text{ (ou } a \text{ em hexadecimal)}$$

Agrupando os restos das divisões inteiras em ordem inversa, tem-se o número convertido:

$$163_{10} = a3_{16}$$

Inicialmente, deve-se dividir sucessivamente o número em base decimal por 8:

$$163_{10} / 8_{10} = 20_{10}, \text{ com resto } 3$$

$$20_{10} / 8_{10} = 2_{10}, \text{ com resto } 4$$

$$2_{10} / 8_{10} = 0_{10}, \text{ com resto } 2$$

Agrupando os restos das divisões inteiras em ordem inversa, tem-se o número convertido:

$$163_{10} = 243_8$$

B.1.2.3 Método da Tabela de Conversão

O método da tabela de conversão é utilizado exclusivamente entre bases que possuem valores múltiplos, como são os casos das bases binária (2), octal (8) e hexadecimal (16).

O método é bastante simples e consiste em:

1. Para conversão de binário para octal: dado um número binário, agrupar seus bits de 3 em 3 dígitos, da direita para a esquerda e, com apoio da tabela no Quadro B.2, converter cada grupo de 3 bits para o respectivo dígito octal;
2. Para conversão de binário para hexadecimal: dado um número binário, agrupar seus bits de 4 e 4 dígitos, da direita para a esquerda e, com apoio da tabela no Quadro B.2, converter cada grupo de 4 bits para o respectivo dígito hexadecimal;
3. Para conversão de octal para binário, deve-se converter cada dígito do número octal para a base binária, com apoio da tabela do Quadro B.2;
4. Para conversão de hexadecimal para binário, deve-se converter cada dígito do número hexadecimal para a base binária, com apoio da tabela do Quadro B.2;
5. Para conversão de octal para hexadecimal, deve-se realizar o passo 3 e, em seguida, o passo 2, anteriormente descritos; e

6. Para conversão de hexadecimal para octal, deve-se realizar o passo 4 e, em seguida, o passo 1, anteriormente descritos.

Quadro B.2. Tabela de Conversão de Números Inteiros entre Bases Binária, Octal e Hexadecimal.

Binária	Octal	Hexadecimal	Binária	Octal	Hexadecimal
0000	000 = 0	0000 = 0	1000	-	1000 = 8
0001	001 = 1	0001 = 1	1001	-	1001 = 9
0010	010 = 2	0010 = 2	1010	-	1010 = a
0011	011 = 3	0011 = 3	1011	-	1011 = b
0100	100 = 4	0100 = 4	1100	-	1100 = c
0101	101 = 5	0101 = 5	1101	-	1101 = d
0110	110 = 6	0110 = 6	1110	-	1110 = e
0111	111 = 7	0111 = 7	1111	-	1111 = f

Seguem alguns exemplos de conversão entre bases binária, octal e hexadecimal:

- Para converter o número $a3_{16}$ para binário:

Deve-se converter cada dígito hexadecimal em sua respectiva sequência de bits na base binária:

$$a3_{16} = (a_{16}) (3_{16})$$

$$a3_{16} = (1010_2) (0011_2)$$

Em seguida, agrupar todos os bits:

$$a3_{16} = 10100011_2$$

- Para converter o número 243_8 para binário:

Deve-se converter cada dígito octal em sua respectiva sequência de bits na base binária:

$$243_8 = (2_8) (4_8) (3_8)$$

$$243_8 = (010_2) (100_2) (011_2)$$

Em seguida, agrupar todos os bits:

$$243_8 = 010100011_2$$

Deve-se remover o 0 (zero) excedente à esquerda:

$$243_8 = 10100011_2$$

Apêndice B - Introdução à Aritmética Computacional

- Para converter o número 10100011_2 para hexadecimal:

Inicialmente, deve-se agrupar os bits do inteiro em base binária de 4 em 4 dígitos:

$$10100011_2 = (1010_2) (0011_2)$$

Em seguida, converter cada grupo de bits para a base hexadecimal:

$$a3_{16} = (a_{16}) (3_{16})$$

Por fim, agrupa-se os dígitos hexadecimais:

$$a3_{16} = a3_{16}$$

- Para converter o número 10100011_2 para octal:

Inicialmente, deve-se agrupar os bits do inteiro em base binária de 3 em 3 dígitos:

$$10100011_2 = (10_2) (100_2) (011_2)$$

Caso o grupo da esquerda, contenha menos de 3 dígitos, deve-se acrescentar zeros (0) à esquerda para formar um grupo com 3 dígitos:

$$10100011_2 = (010_2) (100_2) (011_2)$$

Em seguida, converter cada grupo de bits para a base octal:

$$10100011_2 = (2_8) (4_8) (3_8)$$

Por fim, agrupa-se os dígitos octais:

$$10100011_2 = 243_8$$

- Para converter o número $a3_{16}$ para octal:

Inicialmente, deve-se converter o inteiro em base octal em base binária:

$$a3_{16} = (a_{16}) (3_{16})$$

$$a3_{16} = (1010_2) (0011_2)$$

$$a3_{16} = 10100011_2$$

Após a conversão para a base binária, deve-se converter para a base octal:

$$a3_{16} = (010_2) (100_2) (011_2)$$

$$a3_{16} = (2_8) (4_8) (3_8)$$

$$a3_{16} = 243_8$$

- Para converter o número 243_8 para hexadecimal:

Inicialmente, deve-se converter o inteiro em base octal em base binária:

$$243_8 = (2_8) (4_8) (3_8)$$

$$243_8 = (010_2) (100_2) (011_2)$$

$$243_8 = 010100011_2$$

Deve-se remover o 0 (zero) excedente à esquerda:

$$243_8 = 10100011_2$$

Após a conversão para a base binária, deve-se converter para a base hexadecimal:

$$243_8 = (1010_2) (0011_2)$$

$$243_8 = (a_{16}) (3_{16})$$

$$243_8 = a3_{16}$$

B.2 Representação de Números Inteiros pelo Computador

O conjunto de números inteiros é infinito. Ele pode abranger, além do zero, infinitos números negativos e positivos. Ao contrário dos computadores, que possuem memórias com capacidade de armazenamento limitada, a representação de números inteiros deve seguir algumas restrições.

No projeto da arquitetura de um computador, além das instruções, deve-se definir quais serão os tipos de dados tratados, sua precisão e como podem ser representados pelo computador.

A precisão está relacionada ao número de bits disponíveis para representação de um dado. Como os computadores possuem precisão finita, dados numéricos, tais como os números inteiros, terão seu intervalo limitado. Por exemplo, considerando computadores com precisão de 8-bits, somente será possível representar 2^8 (256) números inteiros diferentes. Já em computadores com precisão de 16-bits, o intervalo numérico de inteiros aumenta para 2^{16} (65536). Ao considerar a representação de inteiros não negativos, o intervalo numérico, com uma precisão de 8-bits, seria $[0; 2^8-1]$ ou $[0; 255]$. Já com uma precisão de 16-bits, o intervalo seria de $[0; 2^{16}-1]$ (ou $[0; 65535]$).

Já a forma de representação de dados pode variar de acordo com o tipo de dado. Os números inteiros, por exemplo, podem ser representados de três formas, como será visto nas subseções seguintes.

B.2.1 Notação Sinal Magnitude

A notação Sinal Magnitude reserva um bit, dentre os bits disponíveis para representação de números inteiros, para representação do sinal. Geralmente, utiliza-se o bit mais significativo para representar o sinal, sendo o bit 1 para sinal negativo e 0 para positivo. As Figuras B.1 (a) e (b) ilustram como os números inteiros 10 e -10 podem ser representados na notação Sinal Magnitude, com precisão de 8-bits, respectivamente.

Figura B.1. Representação de números inteiros na notação Sinal Magnitude.

	8-bits							
Valor	0	0	0	0	1	0	1	0
Posição	7	6	5	4	3	2	1	0

(a) Representação do número 10.

	8-bits							
Valor	1	0	0	0	1	0	1	0
Posição	7	6	5	4	3	2	1	0

(a) Representação do número -10.

Fonte: Próprio autor (2021).

Apesar da simplicidade, a notação Sinal Magnitude apresenta dois problemas. O primeiro problema pode surgir no resultado de operações aritméticas, como é mostrado a seguir.

Supondo que deseja-se subtrair o número 1 de 1, pode-se representar essa operação como uma adição:

$$1_{10} - 1_{10} = 1_{10} + (-1_{10})$$

Representando os números inteiros em notação Sinal Magnitude, com precisão de 8-bits, tem-se:

$$1_{10} + (-1_{10}) = 00000001_2 + (10000001_2)$$

$$1_{10} + (-1_{10}) = 10000010_2$$

Analisando, percebe-se o seguinte resultado:

$$1_{10} + (-1_{10}) = -2_{10}$$

Como foi visto no exemplo anterior, o uso de notação signal magnitude pode levar a resultados incorretos em operações aritméticas.

O segundo problema envolve a representação do número 0, como pode ser visto a seguir.

Supondo que:

$$0_{10} = -0_{10}$$

Representando os números inteiros em notação Sinal Magnitude, com precisão de 8-bits, tem-se:

$$0_{10} = 00000000_2$$

$$-0_{10} = 10000000_2$$

Desta forma, se:

$$0_{10} = -0_{10}$$

Então:

$$00000000_2 = 10000000_2$$

O segundo problema da notação Sinal Magnitude consiste em incluir duas representações distintas para o número zero, que além de levar a erros em operações aritméticas, como visto anteriormente, reduz em uma unidade a capacidade de representação de um intervalo numérico.

B.2.2 Notação Complemento a 1

A notação Complemento a 1 apresenta uma abordagem diferente e resolve o problema gerado em operações aritméticas da notação Sinal Magnitude.

Basicamente, a forma de representação de números inteiros positivos é idêntica à da notação Sinal Magnitude. O que muda é a forma de representação de inteiros negativos. Um número negativo é representado pela inversão dos bits do respectivo inteiro positivo. Por exemplo, para representação do inteiro -10 em notação Complemento a 1, realiza-se os seguintes passos:

Inicialmente, representa-se o número inteiro 10 em notação Complemento a 1, com precisão de 8-bits:

$$10_{10} = 00001010_2$$

Em seguida, inverte-se todos os seus bits para obter o número inteiro -10, em notação Complemento a 1:

$$-10_{10} = 11110101_2$$

Para verificar se a notação Complemento a 1 permite realizar operações aritméticas que apresentem resultados corretos, segue o exemplo de subtração utilizado anteriormente.

Supondo que deseja-se subtrair o número 1 de 1, pode-se representar essa operação como uma adição:

$$1_{10} - 1_{10} = 1_{10} + (-1_{10})$$

Representando os números inteiros em notação Complemento a 1, com precisão de 8-bits, tem-se:

$$1_{10} + (-1_{10}) = 00000001_2 + (11111110_2)$$

$$1_{10} + (-1_{10}) = 11111111_2$$

Analisando o resultado, percebe-se que, se:

$$0_{10} = -0_{10}$$

E representando-os em notação Complemento a 1, com precisão de 8-bits, tem-se:

$$0_{10} = 00000000_2$$

$$-0_{10} = 11111111_2$$

Então, o resultado da operação aritmética anterior, apresentou o seguinte resultado:

$$1_{10} + (-1_{10}) = 0_{10}$$

No exemplo anterior, viu-se que a notação Complemento a 1 permite realizar operações aritméticas com sucesso. No entanto, assim como na notação Sinal Magnitude, há duas representações distintas para o número zero.

B.2.3 Notação Complemento a 2

A notação Complemento a 2 utiliza o mesmo modelo de representação de números inteiros positivos das notações anteriores. Já para a representação de números inteiros negativos, utiliza, a princípio, a mesma técnica da notação Complemento a 1, que consiste na inversão dos bits do respectivo número inteiro positivo, e segue adicionando uma unidade ao número invertido. Para exemplificar os passos da notação Complemento a 2, representa-se o número inteiro -10 da seguinte maneira:

Inicialmente, representa-se o número inteiro 10 em notação Complemento a 1, com precisão de 8-bits:

$$10_{10} = 00001010_2$$

Em seguida, inverte-se todos os seus bits para obter o número inteiro -10, em notação Complemento a 1:

$$-10_{10} = 11110101_2$$

Por fim, acrescenta-se 1 para encontrar o número inteiro -10, em notação Complemento a 2:

$$-10_{10} = 11110101_2 + 1_2$$

$$-10_{10} = 11110110_2$$

De forma a verificar se a notação Complemento a 2 continua permitindo a realização de operações aritméticas com resultados corretos, segue o exemplo de subtração utilizado anteriormente.

Supondo que deseja-se subtrair o número 1 de 1, pode-se representar essa operação como uma adição:

$$1_{10} - 1_{10} = 1_{10} + (-1_{10})$$

Representando os números inteiros em notação Complemento a 1, com precisão de 8-bits, tem-se:

$$1_{10} + (-1_{10}) = 00000001_2 + (11111110_2)$$

O próximo passo é passar o número inteiro negativo para notação Complemento a 2, acrescentando 1:

$$1_{10} + (-1_{10}) = 00000001_2 + (11111110_2 + 1_2)$$

$$1_{10} + (-1_{10}) = 00000001_2 + (11111111_2)$$

Enfim, realiza-se a adição:

$$1_{10} + (-1_{10}) = 100000000_2$$

Analisando o resultado, percebe-se que o valor resultante possui 9-bits. Porém, como considerou-se a precisão de 8-bits, o bit mais significativo (à esquerda) é descartado, resultado assim em:

$$1_{10} + (-1_{10}) = 00000000_2$$

$$1_{10} + (-1_{10}) = 0_{10}$$

Com o exemplo anterior, percebe-se que a notação Complemento a 2, além de continuar permitindo operações aritméticas com resultados corretos, introduz uma forma única de representação do número zero, corrigindo assim os problemas apresentados nas notações anteriores.

Atualmente, todos os computadores utilizam a notação Complemento a 2 para representação de número inteiros.

B.3 Representação de Números de Ponto Flutuante pelo Computador

Os números de ponto flutuante, ou números reais, possuem, no computador, um modelo de representação totalmente diferente dos modelos estudados para representação de números inteiros.

No passado, os computadores apresentaram diferentes abordagens para representação de números de ponto flutuante, até que em 1985, criou-se o padrão IEEE 754 (IEEE, 1985).

O IEEE 754 permitiu que os computadores pudessem ter uma representação correta de números reais e que eles pudessem ser trocados entre diferentes computadores.

No padrão IEEE 754, inicialmente, foram definidos quatro tipos de formatos, porém, em 2008, ele foi atualizado para suportar um total de seis formatos (IEEE, 2008), que são:

- “Half”: 16-bits (1985);
- “Single” (conhecido também como “float”): 32-bits (1985);
- “Double”: 64-bits (1985);
- “Double Extended”: 80-bits (1985);
- “Quadruple”: 128-bits (2008); e
- “Octuple”: 256-bits (2008).

Independentemente do formato, a representação de um número de ponto flutuante é dada em notação científica, da seguinte maneira:

$(-1)^S 1, M \times B^E$	(2)
--------------------------	-----

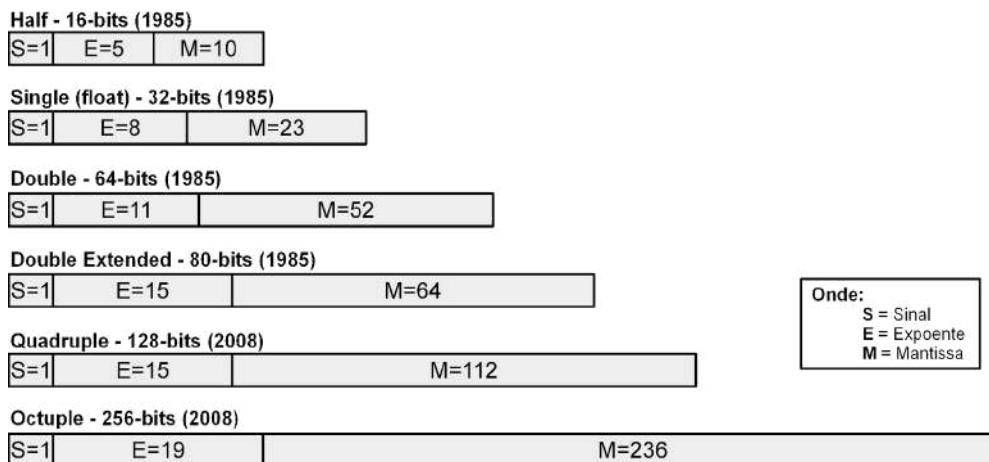
Onde,

- S (“Sinal”): indica se o número de ponto flutuante é positivo ou negativo;
- B (“Base”): consiste da base, que pode ser decimal, binária, octal ou hexadecimal;
- E (“Expoente”): representa o expoente do número de ponto flutuante; e
- M (“Mantissa”): consiste do valor fracionário do número ponto flutuante.

Por exemplo, o número -12,345 poderia ser representado em notação científica da seguinte forma: $-1,2345 \times 10^1$, onde $S=1$, $M=2345$, $B=10$ e $E=1$.

Dependendo do formato para representação de números ponto flutuante do padrão IEEE 754, serão reservados diferentes bits para representação do Expoente e Mantissa. O campo Sinal tem tamanho fixo de 1-bit e a Base utilizada é sempre binária. A Figura B.2 ilustra os bits reservados para campos dos diferentes formatos para representação de números de ponto flutuante do padrão IEEE 754.

Figura B.2. Formatos do Padrão IEEE 754 para Representação de Número de Ponto Flutuante.



Fonte: Próprio autor (2021).

O processo de representação de um número ponto flutuante utilizando o padrão IEEE 754 é um pouco complexo, porém traz um grande potencial de precisão numérica.

A conversão do número -12,345 em ponto flutuante com precisão “Single (float)” envolve os seguintes passos:

1. Converter a parte inteira para a base binária:

$-12_{10} = -1100_2$

2. Converter a parte fracionária para a base binária, através de sucessivas multiplicações das partes fracionárias pelo número 2:

$0,345_{10} * 2_{10} = 0,69_{10}$ $0,69_{10} * 2_{10} = 1,38_{10}$ $0,38_{10} * 2_{10} = 0,76_{10}$ $0,76_{10} * 2_{10} = 1,52_{10}$ $0,52_{10} * 2_{10} = 1,04_{10}$ $\dots$
Obs: Realiza-se essa tarefa até 23 vezes, dado que são reservados 23-bits para a Mantissa no formato <i>Single (float)</i>, ou até que seja obtido 0. Considerando os algarismos destacados em negrito, agrupando-os, tem-se: $0,345_{10} = 01011000010100011111_2$

3. Com as duas conversões tem-se o seguinte número binário:

$$-12,345_{10} = -1100,01011000010100011111_2$$

4. Em seguida, deve-se expor o número resultante em notação científica:

$$-12,345_{10} = -1,10001011000010100011111_2 \times 2^3_{10}$$

5. Os passos seguintes envolvem converter o número resultante para o formato *Single (float)* do padrão IEEE 754. Como se trata de um número negativo, no campo Sinal (S), deve ser adicionado o bit 1:

$$S = 1_2$$

6. No formato *Single (float)* do padrão IEEE 754, o campo Expoente (E) possui 8-bits. Isso quer dizer que pode armazenar expoentes no intervalo $[-126; 127]$. No entanto, a notação científica adotada para representação de números reais, só admite expoentes positivos. Sendo assim, para compor o campo Expoente (E), o expoente 3 (em “ 2^3 ”) deve ser somado à 127 para se ajustar ao intervalo $[1; 254]$.

$$E = 3_{10} + 127_{10}$$

$$E = 130_{10}$$

Convertendo para a base binária:

$$E = 10000010_2$$

7. Por fim, o campo Mantissa (M) recebe a parte fracionária convertida para a base binária:

$$M = 10001011000010100011111_2$$

8. Com isso, o número -12,345 é representado no formato *Single (float)* do padrão IEEE 754 da seguinte maneira:

Considerando os campos:

$$S = 1_2$$

$$E = 10000010_2$$

$$M = 10001011000010100011111_2$$

Ao agrupá-los, forma-se:

$$-12,345_{10} = 11000001010001011000010100011111_2$$

Sobre o resultado da conversão, é importante observar os seguintes aspectos:

- Se o expoente e a mantissa forem iguais a 0, o número é zero;
- Se o expoente for igual a 0 e todos os bits da mantissa forem iguais a 1, o número não está normalizado;
- Se o expoente for igual a 255 e a mantissa for igual a 0, o número é considerado infinito; e
- Se o expoente for igual a 255 e todos os bits da mantissa foram iguais a 1, o número é classificado como “NaN” (*Not a Number*).

Já a conversão de um número em formato *Single (float)* do padrão IEEE 754 para a notação decimal é um processo simples. Considerando o número $11000001010001011000010100011111_2$ para convertê-lo, devem ser realizados os seguintes passos:

1. Inicialmente, identifica-se o sinal, através do bit mais significativo:

$$11000001010001011000010100011111_2$$

$$-1000001010001011000010100011111_2$$

2. Em seguida, deve-se encontrar o expoente, através dos próximos 8-bits:

$$-1000001010001011000010100011111_2$$

Em que:

$$10000010_2 = 130_{10}$$

$$10000010_2 = 127_{10} + 3_{10}$$

3. O passo agora é expor a parte fracionária em função do expoente encontrado:

$$-1,10001011000010100011111_2 * 2^3_{10}$$

ou

$$-1100,01011000010100011111_2$$

4. Converte-se as partes inteira e fracionária para a base decimal:

Convertendo a parte inteira, tem-se:

$$-1100_2 = -12_{10}$$

E convertendo a parte fracionária, tem-se:

$$01011000010100011111_2 = 0 \times 2^{-1}_{10} + 1 \times 2^{-2}_{10} + 0 \times 2^{-3}_{10} + 1 \times 2^{-4}_{10} + \dots + 1 \times 2^{-22}_{10} + 1 \times 2^{-23}_{10}$$

$$01011000010100011111_2 = 0,25_{10} + 0,0625_{10} + \dots + 0,000000238_{10} + 0,000000119_{10}$$

$$01011000010100011111_2 \approx 0,345_{10}$$

5. Por fim, adiciona-se as partes inteiras e fracionárias convertidas para a base decimal:

$$-12_{10} + 0,345_{10} = -12,345_{10}$$

APÊNDICE C – INTRODUÇÃO À LÓGICA BOOLEANA E CIRCUITOS LÓGICOS

Como foi visto no [Capítulo 2 - História do Computador](#), em 1854, George Boole concebeu um sistema em que era possível expressar lógica (raciocínio) através de formalismos e regras matemáticas, a chamada Álgebra de Boole ou Álgebra Booleana.

A princípio, Boole observou que seria possível representar ideias com símbolos, chamados de Variáveis Booleanas. Além disso, a álgebra booleana associa dois valores lógicos para uma variável booleana: 0 (“Verdadeiro”) ou 1 (“Falso”) (PIMENTA, 2017). É importante destacar que uma variável booleana não pode assumir os dois possíveis valores lógicos ao mesmo tempo.

Seja A uma variável booleana. Então:

Se $A = 0$, então $A \neq 1$

Se $A = 1$, então $A \neq 0$

C.1 Operações Booleanas

Além das variáveis booleanas, Boole estabeleceu três operações básicas que podem ser realizadas sobre as variáveis booleanas.

A primeira operação é a de “Inversão”, “Complemento” ou “Negação”, também chamada de “NÃO” (“NOT”), que realiza a inversão do valor lógico associado a uma variável booleana. Neste livro, representaremos o operador de negação NOT com o símbolo “ $\neg$ ”.

Seja A uma variável booleana. Então:

Se $A = 0$, então $\neg A = 1$

Se $A = 1$, então $\neg A = 0$

A segunda operação é a de “Conjunção”, também chamada de “E” (“AND”), e combina os valores lógicos de duas variáveis booleanas de forma a gerar um novo valor lógico. A operação de conjunção AND estabelece um valor lógico resultante verdadeiro se ambas as variáveis tiverem valores lógicos verdadeiros. Neste livro, representaremos o operador de conjunção AND com o símbolo “.” (símbolo aritmético de multiplicação).

Sejam A e B duas variáveis booleanas. Então:

Se $A = 0$ e $B = 0$, então $A . B = 0$

Se $A = 1$ e $B = 0$, então $A . B = 0$

Se $A = 0$ e $B = 1$, então $A . B = 0$

Se $A = 1$ e $B = 1$, então $A . B = 1$

A terceira operação é a de “Disjunção”, também chamada de “OU” (“OR”), e combina os valores lógicos de duas variáveis booleanas de forma a gerar um novo valor lógico. A operação de disjunção OR estabelece um valor lógico resultante verdadeiro se, pelo menos, alguma das variáveis tiver valor lógico verdadeiro. Neste livro, representaremos o operador de disjunção OR com o símbolo “+” (símbolo aritmético de adição).

Sejam A e B duas variáveis booleanas. Então:

Se $A = 0$ e $B = 0$, então $A + B = 0$

Se $A = 1$ e $B = 0$, então $A + B = 1$

Se $A = 0$ e $B = 1$, então $A + B = 1$

Se $A = 1$ e $B = 1$, então $A + B = 1$

C.2 Funções Booleanas

As variáveis e operações booleanas podem ser combinadas para compor operadores lógicos mais complexos e funções booleanas.

Por exemplo, existem as operações booleanas de Negação da Conjunção, também chamada de “NÃO E” (“NOT AND” ou “NAND”) e de Negação da Disjunção, também chamada de “NÃO OU” (“NOT OR” ou “NOR”). Para compor as operações booleanas NAND e NOR, deve-se combinar os operadores NOT-AND e NOT-OR, respectivamente:

Sejam A e B duas variáveis booleanas. Então, a conjunção AND delas é expressa da seguinte forma:

Se $A = 0$ e $B = 0$, então $A \cdot B = 0$

Se $A = 1$ e $B = 0$, então $A \cdot B = 0$

Se $A = 0$ e $B = 1$, então $A \cdot B = 0$

Se $A = 1$ e $B = 1$, então $A \cdot B = 1$

Uma vez calculada a conjunção, o passo seguinte é aplicar o operador de negação NOT:

Se $A \cdot B = 0$, então $\neg(A \cdot B) = 1$

Se $A \cdot B = 1$, então $\neg(A \cdot B) = 0$

Sejam A e B duas variáveis booleanas. Então, a disjunção OR delas é expressa da seguinte forma:

Se $A = 0$ e $B = 0$, então $A + B = 0$

Se $A = 1$ e $B = 0$, então $A + B = 1$

Se $A = 0$ e $B = 1$, então $A + B = 1$

Se $A = 1$ e $B = 1$, então $A + B = 1$

Uma vez calculada a disjunção, o passo seguinte é aplicar o operador de negação NOT:

Se $A + B = 0$, então $\neg(A + B) = 1$

Se $A + B = 1$, então $\neg(A + B) = 0$

Além das operações booleanas NAND e NOR, existem outras, tais como a Disjunção Exclusiva (“EXCLUSIVE OR” ou “XOR”) e Disjunção Não-Exclusiva (“EXCLUSIVE NOT OR” ou “XNOR”).

As Funções Booleanas são expressões lógicas mais complexas, que podem envolver diversas variáveis e operadores lógicos. Assim como ocorreu com os operadores lógicos NAND e NOR, as operações lógicas NOT, AND e OR podem ser utilizadas para representar qualquer função booleana.

Há dois métodos padrões para expressar as funções booleanas:

1. Soma dos Mintermos: Um mintermo é basicamente a conjunção (ou “Produto”) das variáveis envolvidas na função booleana. Portanto, uma função booleana pode ser expressa pela disjunção (“Soma”) de mintermos; e
2. Produto dos Maxtermos: Um maxtermo é basicamente a disjunção (ou “Soma”) das variáveis envolvidas na função booleana. Portanto, uma função booleana pode ser expressa pela disjunção (“Produto”) de maxtermos.

Segue um exemplo de obtenção de uma função booleana pelo método de Soma dos Mintermos:

Sejam A, B e S variáveis booleanas. A partir dos valores lógicos assumidos por A e B, observa-se o comportamento de S a seguir:

Se $A = 0$ e $B = 0$, então $S = 1$

Se $A = 1$ e $B = 0$, então $S = 1$

Se $A = 0$ e $B = 1$, então $S = 0$

Se $A = 1$ e $B = 1$, então $S = 0$

Deseja-se obter uma função booleana que expresse o comportamento de S a partir de A e B, utilizando o método de Soma dos Mintermos. Para tanto, deve-se observar os casos em que S apresenta valor lógico 1, que são:

Se $A = 0$ e $B = 0$, então $S = 1$ (Caso 1)

Se $A = 1$ e $B = 0$, então $S = 1$ (Caso 2)

O passo seguinte consiste em obter os mintermos (produto das variáveis), para cada um dos casos. Os mintermos são obtidos a partir do produto das representações das variáveis de acordo com os valores lógicos assumidos:

1. Se o valor da variável A ou B foi igual a 0, representa-se como $\neg A$ ou $\neg B$, respectivamente
2. Se o valor da variável A ou B for igual a 1, representa-se como A ou B, respectivamente
3. Por fim, realiza-se a soma dos mintermos para cada caso.

Para o (Caso 1):

Como $A = 0$ e $B = 0$, o mintermo é $(\neg A \cdot \neg B)$

Para o Caso 2:

Como $A = 1$ e $B = 0$, o mintermo é $(A \cdot \neg B)$

Então, realizando a soma dos mintermos, tem-se a função booleana que expressa o comportamento de S:

$$S = (\neg A \cdot \neg B) + (A \cdot \neg B)$$

C.3 Propriedades da Álgebra Booleana

Em 1904, Edward Vermilye Huntington estabeleceu os seguintes postulados (axiomas) da álgebra booleana:

1. Comutativa:

Para todo x e $y \in A$:

$$x + y = y + x$$

$$x \cdot y = y \cdot x$$

2. Distributiva:

Para todo x, y e $z \in A$:

$$x \cdot (y + z) = (x \cdot y) + (x \cdot z)$$

$$x + (y \cdot z) = (x + y) \cdot (x + z)$$

3. Identidade:

Para todo $x \in A$: $x + 0 = x$ $x \cdot 1 = x$
--

4. Complemento:

Para todo $x \in A$: $x + \neg x = 1$ $x \cdot \neg x = 0$
--

Com os postulados de Huntington, surgiram as Propriedades da Álgebra Booleana. As propriedades são utilizadas na simplificação de expressões lógicas. Seguem algumas propriedades:

Adição Lógica: $A + 0 = A$ $A + 1 = 1$ $A + A = A$ $A + \neg A = 1$	Multiplicação Lógica: $A \cdot 0 = 0$ $A \cdot 1 = A$ $A \cdot A = A$ $A \cdot \neg A = 0$	De Morgan: $\neg(A \cdot B) = \neg A + \neg B$ $\neg(A + B) = \neg A \cdot \neg B$ $A \cdot B = \neg(\neg A + \neg B)$ $A + B = \neg(\neg A \cdot \neg B)$
Comutatividade: $A + B = B + A$ $A \cdot B = B \cdot A$	Complementação: $\neg\neg A = A$	Absorção: $A + (A \cdot B) = A$ $A \cdot (A + B) = A$

Por exemplo, podemos utilizar os postulados e propriedades para demonstrar as propriedades da Absorção:

$A + (A \cdot B) = (A \cdot 1) + (A \cdot B) \quad (\text{multiplicação})$ $A + (A \cdot B) = A \cdot (1 + B) \quad (\text{distributiva})$ $A + (A \cdot B) = A \cdot 1 \quad (\text{adição})$ $A + (A \cdot B) = A \quad (\text{multiplicação})$

$$A \cdot (A + B) = (A \cdot A) + (A \cdot B) \quad (\text{distributiva})$$

$$A + (A \cdot B) = A + (A \cdot B) \quad (\text{multiplicação})$$

$$A + (A \cdot B) = A \quad (\text{ver prova anterior})$$

C.4 Conceitos de Circuitos Lógicos

No [Capítulo 2 - História do Computador](#), viu-se que, em 1940, Claude Shannon sistematizou o uso da Álgebra de Booleana e da Aritmética Binária na eletrônica para construção de computadores.

O sistema criado por Shannon permitiu a criação de qualquer circuito eletrônico através do emprego de funções lógicas. Ele propôs o uso de chaves, que ao serem fechadas permitem a passagem de corrente elétrica de um ponto para outro, em um circuito eletrônico. Dependendo da configuração das chaves, elas passam a representar os operadores lógicos da Álgebra Booleana, e, com isso, pode-se compor as funções lógicas para montar circuitos eletrônicos.

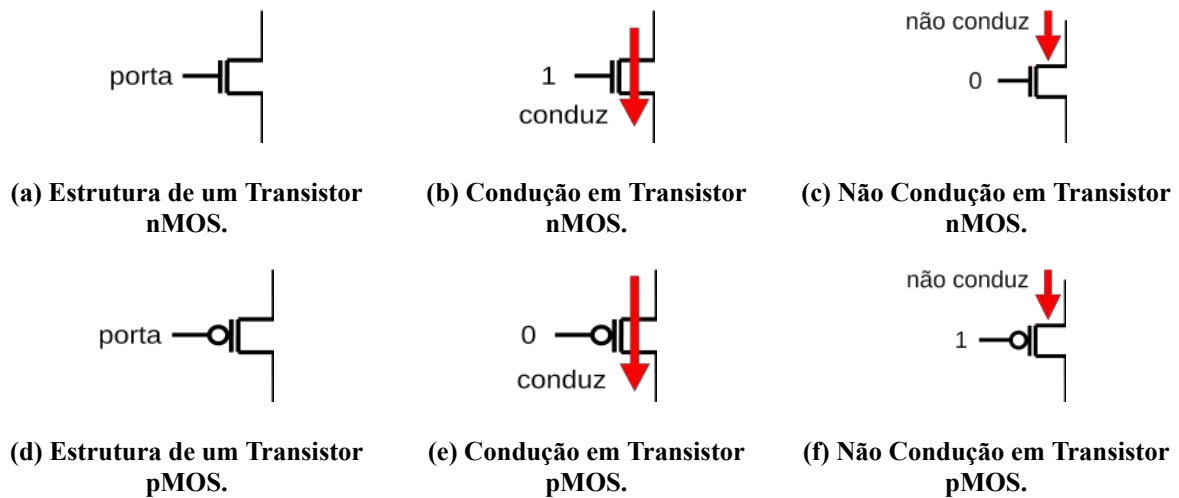
Viu-se ainda no Capítulo 2 que, com o passar das gerações de computadores eletrônicos, as tecnologias utilizadas para a fabricação de circuitos eletrônicos mudaram. Atualmente, utiliza-se os transistores para compor as chaves lógicas de Shannon. O tipo de transistor mais utilizado na fabricação de circuitos integrados é o CMOS (*Complementary Metal Oxide Semiconductor*) (VAHID, 2008). CMOS consiste de um processo de fabricação de circuitos integrados baseados em silício, tais como processadores, microcontroladores, alguns tipos de memória e outros circuitos digitais.

Transistores CMOS possuem um pólo chamado “porta”, que ao aplicar uma determinada tensão a ele, uma corrente elétrica fluirá através do transistor (funcionamento semelhante a uma chave). Transistores do tipo “nMOS” necessitam de uma tensão positiva na porta para que a corrente elétrica flua através do transistor, enquanto que transistores do tipo “pMOS”, necessitam de uma tensão negativa.

As Figuras C.1 (a) e (d) ilustram diagramas esquemáticos de transistores CMOS dos tipos nMOS e pMOS, respectivamente. Nas Figuras C.1 (b) e (c), é possível visualizar o comportamento de um transistor do tipo nMOS, onde ao se aplicar uma tensão positiva na porta, a corrente flui através do transistor; o que já não acontece quando se aplica uma tensão negativa. Nas Figuras C.1 (e) e (f), podem ser vistos os diagramas esquemáticos com o

comportamento de um transistor do tipo pMOS, onde, para que a corrente flua através do transistor, é necessário aplicar uma tensão negativa na porta.

Figura C.1. Transistores CMOS.

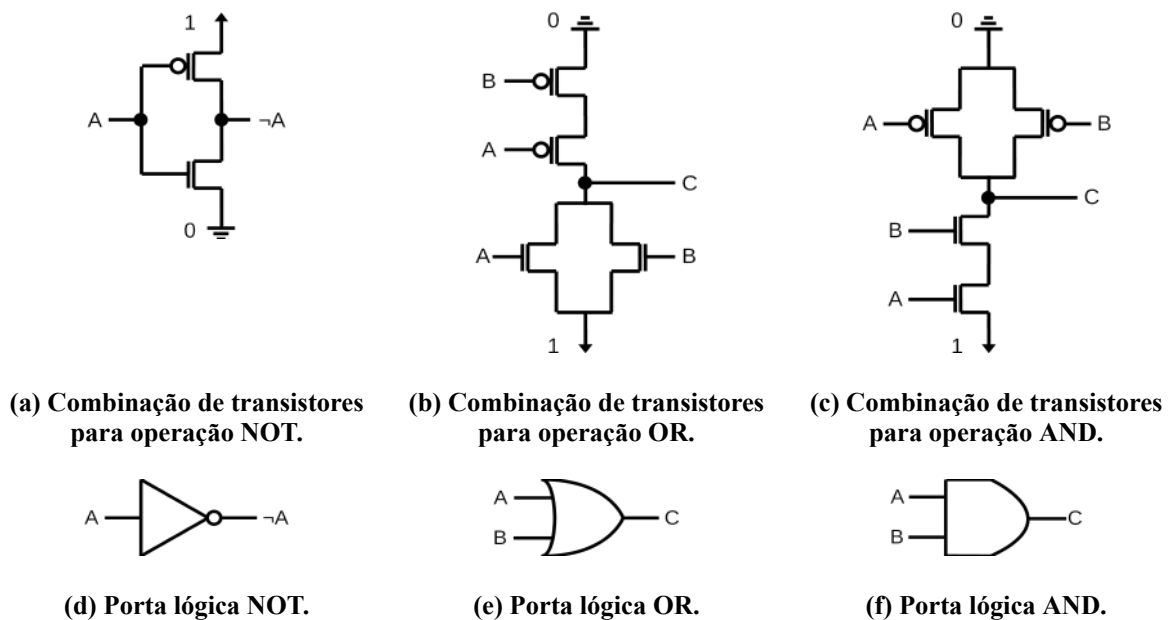


Fonte: Próprio autor (2021).

Com os transistores é possível implementar circuitos integrados em escalas incrivelmente pequenas. Contudo, ao se utilizar os transistores, a criação de circuitos integrados mais elaborados torna-se uma tarefa complexa (VAHID, 2008).

Desta forma, para simplificar o projeto de novos circuitos integrados, criou-se, através de diferentes combinações de ligações de transistores, blocos de circuitos que representam as operações lógicas da Álgebra Booleana, as chamadas Portas Lógicas. Com o suporte das portas lógicas e da Álgebra Booleana, pode-se construir novos circuitos integrados com mais facilidade. Na Figura C.2, pode-se visualizar as portas lógicas que implementam as operações lógicas NOT, OR e AND.

Figura C.2. Composição de operadores lógicos com transistores CMOS.



Fonte: Próprio autor (2021).

Na Figura C.2 (a), a porta lógica NOT é implementada pela combinação de dois transistores, sendo um do tipo pMOS e um do tipo nMOS. A Figura C.2 (d) apresenta o símbolo que representa a porta lógica NOT.

Na Figura C.2 (b), a porta lógica OR é implementada pela combinação de dois transistores do tipo pMOS e dois do tipo nMOS. A Figura C.2 (e) apresenta o símbolo que representa a porta lógica OR.

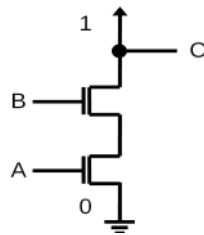
Na Figura C.2 (c), a porta lógica AND é implementada pela combinação de dois transistores do tipo pMOS e dois do tipo nMOS. A Figura C.2 (f) apresenta o símbolo que representa a porta lógica OR.

Analisando as combinações de transistores CMOS utilizadas para compor as portas lógicas NOT, OR e AND, observa-se que, em todas elas, foram utilizados transistores dos tipos nMOS e pMOS. Além disso, observa-se que, dadas as combinações dos transistores, todos os sinais de entrada precisaram ser invertidos, por conta da natureza do comportamento do transistor pMOS. A necessidade de ter que inverter sinais de entrada, pode gerar atrasos de propagação de corrente elétrica no circuito do transistor.

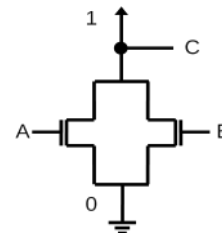
É possível combinar transistores do mesmo tipo para compor operações lógicas e evitar atrasos de propagação de corrente. Porém, dependendo da operação lógica e do tipo de transistor CMOS que será utilizado na implementação da respectiva porta, pode ser necessário utilizar um maior número de transistores.

A Figura C.3 ilustra as combinações de ligações de transistores do tipo nMOS para compor as operações lógicas NAND e NOR.

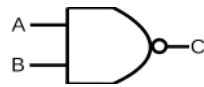
Figura C.3. Composição de operadores lógicos com transistores nMOS.



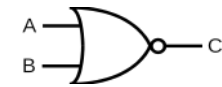
(a) Combinação de transistores para operação NAND.



(b) Combinação de transistores para operação NOR.



(c) Porta lógica NAND.



(d) Porta lógica NOR.

Fonte: Próprio autor (2021).

Na Figura C.3, observa-se que as implementações das portas lógica NAND e NOR são bastante simples, pois utilizou-se apenas dois transistores do tipo nMOS.

As portas lógicas NAND e NOR são denominadas “Portas Lógicas Universais” por possuírem uma propriedade bastante interessante: com elas é possível implementar qualquer operador ou função lógica (PATRICK; FARDO; CHANDRA, 2020).

Por exemplo, demonstra-se a seguir, com as propriedades da Lógica Booleana, que é possível compor as operações lógicas NOT, OR e AND utilizando a operação lógica NAND.

$\neg A = \neg (A \cdot A)$	(multiplicação)	(1)
-----------------------------	-----------------	-----

$A + B = \neg(\neg(A + B))$	(complementação)	(2)
$A + B = \neg(\neg A \cdot \neg B)$	(De Morgan)	
$A + B = \neg(\neg(A \cdot A) \cdot \neg(B \cdot B))$	(multiplicação)	

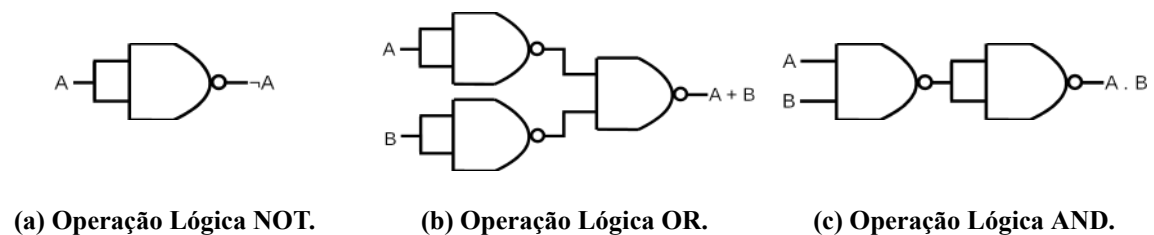
$A \cdot B = \neg(\neg(A \cdot B))$	(complementação)	(3)
$A \cdot B = \neg(\neg(A \cdot B) \cdot \neg(A \cdot B))$	(multiplicação)	

Apêndice C - Introdução à Lógica Booleana e Circuitos Lógicos

Pela simplicidade de implementação das portas lógicas NAND e NOR, que necessitam de apenas dois transistores, e por possuírem a propriedade de portas lógicas universais, muitos circuitos integrados são produzidos utilizando apenas portas lógicas universais.

A partir das funções lógicas nas equações (1), (2) e (3) e da combinação de portas lógicas NAND, é possível construir os circuitos lógicos para as operações lógicas NOT, OR e AND, como pode ser visto na Figura C.4.

Figura C.4. Composição de operadores lógicos com portas NAND.



Fonte: Próprio autor (2021).

APÊNDICE D - TABELA ASCII

O Quadro D.1 apresenta uma tabela com os códigos e respectivos caracteres ASCII. Os caracteres destacados em cor vermelha são do tipo “não imprimíveis” e os em azul são “imprimíveis”.

Quadro D.1. Tabela de códigos ASCII.

hex	dec	char	hex	dec	char	hex	dec	char	hex	dec	char
0	0	Null	20	32	Space	40	64	@	60	96	`
1	1	Start of Header	21	33	!	41	65	A	61	97	a
2	2	Start of Text	22	34	"	42	66	B	62	98	b
3	3	End of Text	23	35	#	43	67	C	63	99	c
4	4	End of Transmission	24	36	\$	44	68	D	64	100	d
5	5	Enquiry	25	37	%	45	69	E	65	101	e
6	6	Acknowledgment	26	38	&	46	70	F	66	102	f
7	7	Bell	27	39	'	47	71	G	67	103	g
8	8	Backspace	28	40	(	48	72	H	68	104	h
9	9	Horizontal Tab	29	41	)	49	73	I	69	105	i
a	10	Line Feed	2a	42	*	4a	74	J	6a	106	j
b	11	Vertical Tab	2b	43	+	4b	75	K	6b	107	k
c	12	Form Feed	2c	44	,	4c	76	L	6c	108	l
d	13	Carriage Return	2d	45	-	4d	77	M	6d	109	m
e	14	Shift Out	2e	46	.	4e	78	N	6e	110	n
f	15	Shift In	2f	47	/	4f	79	O	6f	111	o
10	16	Data Link Escape	30	48	0	50	80	P	70	112	p

Apêndice D - Tabela ASCII

11	17	Data Link Control 1	31	49	1	51	81	Q	71	113	q
12	18	Data Link Control 2	32	50	2	52	82	R	72	114	r
13	19	Data Link Control 3	33	51	3	53	83	S	73	115	s
14	20	Data Link Control 4	34	52	4	54	84	T	74	116	t
15	21	Negative Ack.	35	53	5	55	85	U	75	117	u
16	22	Synchronous Idle	36	54	6	56	86	V	76	118	v
17	23	End of Trans. Block	37	55	7	57	87	W	77	119	w
18	24	Cancel	38	56	8	58	88	X	78	120	x
19	25	End of Medium	39	57	9	59	89	Y	79	121	y
1a	26	Substitute	3a	58	:	5a	90	Z	7a	122	z
1b	27	Escape	3b	59	;	5b	91	[	7b	123	{
1c	28	File Separator	3c	60	<	5c	92	\	7c	124	
1d	29	Group Separator	3d	61	=	5d	93	]	7d	125	}
1e	30	Record Separator	3e	62	>	5e	94	^	7e	126	~
1f	31	Unit Separator	3f	63	?	5f	95	_	7f	127	Del

APÊNDICE E – FERRAMENTAS DO SIMULADOR COMPSIM

O simulador CompSim conta com um conjunto de ferramentas para auxiliar o aprendizado dos conceitos de Arquitetura e Organização de Computadores e o projeto de sistemas computacionais. Este apêndice apresenta detalhes de cada uma dessas ferramentas.

E.1 Code Helper

“Code Helper” é um assistente para apoio à programação em *Assembly*. Programação em Assembly não é uma tarefa fácil, pois cada instrução possui particularidades. Por exemplo, há instruções que necessitam de parâmetros numéricos, ou de endereço de memória, ou de nome de variável, ou de rótulo de instrução, ou de nome de registrador, ou não necessitam de parâmetro.

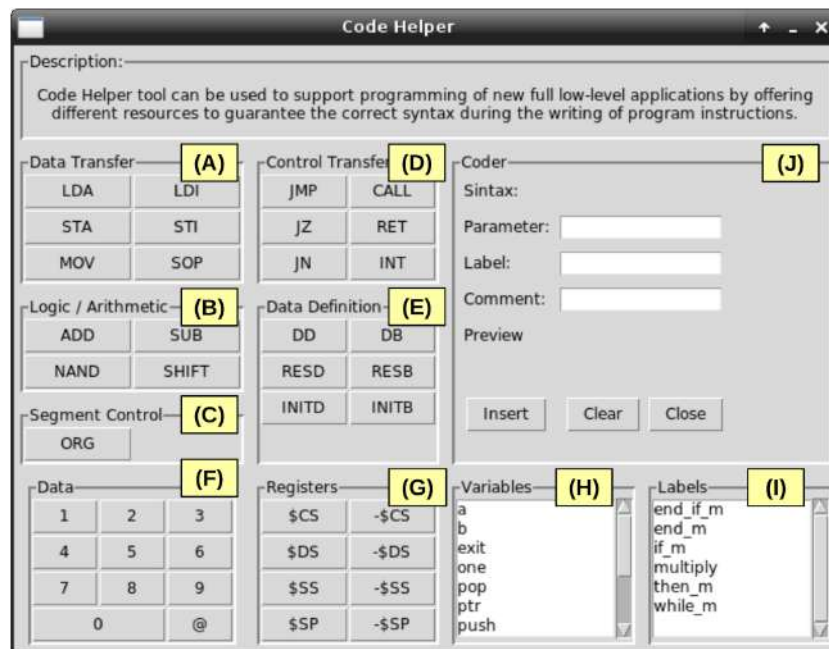
Há instruções, como as de definição de dados, que necessitam de um nome de variável e o respectivo parâmetro, porém dependendo da instrução, os parâmetros podem ser de tipos diferentes, tais como número inteiro, caracter ou string, ou a combinação de todos eles. Às vezes, é desejável adicionar rótulos a determinadas instruções ou até mesmo comentários sobre elas, para simplificação da leitura e interpretação do código.

Code Helper possui uma interface gráfica, que pode ser vista na Figura E.1, com diversos controles gráficos que permitem a construção das instruções de um programa Assembly com a sintaxe correta.

A Interface gráfica do Code Helper contém controles com atalhos para:

- (A) Instruções de Transferência de dados: LDA, STA, LDI, STI, MOV e SOP;
- (B) Instruções para Operações Lógicas e Aritméticas: ADD, SUB, NAND e SHIFT;
- (C) Pseudo-Instrução de controle de segmento: ORG;
- (D) Instruções de Controle de Transferência de Fluxo de Execução: JMP, JZ, JN, CALL, RET e INT;
- (E) Pseudo-Instruções para Definição de Dados: DD, DB, RESD, RESB, INITD e INITB;

Figura E.1. Interface gráfica da ferramenta Code Helper.



Fonte: Próprio autor (2021).

- (F) Dados Numérico e Endereços de Memória;
- (G) Registradores: \$CS,\$DS, \$SS, \$SP, -\$CS,-\$DS, -\$SS e -\$SP;
- (H) Nome de Variáveis Definidas no Programa; e
- (I) Rótulos Definidos para determinadas Instruções.

Na medida em que os controles gráficos são utilizados para compor uma instrução, na Figura E.1 (J), é possível visualizar um *preview* da instrução em construção.

Por exemplo, para construir uma instrução de leitura de dados em memória LDA, o primeiro passo é acionar o assistente Code Helper, cujo procedimento pode ser realizado acessando o menu “Program”, opção “Code Helper”, ou pressionando as teclas de atalho “Ctrl+Espaço”.

Após acionar o assistente de codificação, o passo seguinte consiste em clicar no botão “LDA”. Ao clicar no botão, alguns recursos do Code Helper serão desabilitados, pois não podem ser utilizados com a instrução STA, como são os casos dos parâmetros de rótulos de instruções e registradores, como mostra a Figura E.2.

Figura E.2. Instrução LDA.

Fonte: Próprio autor (2021).

O próximo passo consiste em definir qual será o parâmetro da instrução LDA: se será um nome de variável ou um endereço de memória. Caso seja um nome de variável, basta clicar no nome da variável desejada no controle gráfico “Variables”. Caso seja um endereço de memória, pode-se clicar nos controles em “Data”, para compor o endereço.

Por fim, pode-se adicionar um rótulo e um comentário para a instrução LDA, preenchendo os campos “Label” e “Parameter”, no painel “Coder”.

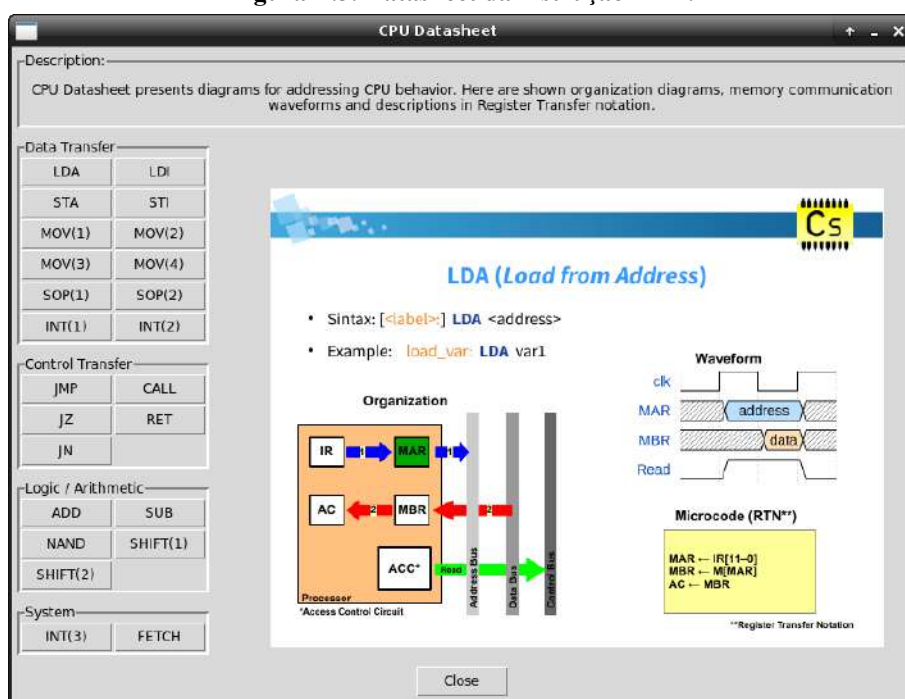
O preview completo da instrução pode ser visto em “Preview”, no painel “Coder”, como mostra a Figura E.2. No *preview*, A instrução LDA recebeu como parâmetro a variável “a”, recebeu o rótulo de instrução “Ler_var_a” e comentário de instrução “Realiza a leitura da variável a”.

Para inserir a instrução no editor de código, na posição em que encontra-se o cursor, basta clicar no botão “Insert”. Pode-se desfazer todas as configurações da instrução, no Code Helper, com o botão “Clear”. E, com o botão “Close”, encerra-se o assistente de codificação Code Helper.

E.2 CPU Datasheet

“CPU Datasheet” é uma ferramenta que traz informações sobre as instruções do processador Cariri. A Figura E.3 ilustra o uso da ferramenta CPU Datasheet com informações sobre a instrução LDA.

Figura E.3. Datasheet da instrução LDA.



Fonte: Próprio autor (2021).

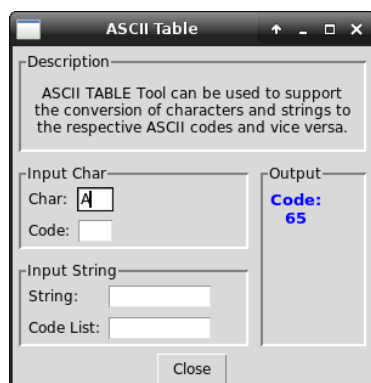
Dentre as informações disponibilizadas pela ferramenta CPU Datasheet estão: a sintaxe de uso da instrução ("Syntax"); um diagrama de organização, que demonstra a sequência de operações internas do processador para a execução de uma instrução ("Organization"); um diagrama de formas de onda, onde pode-se visualizar os sinais do barramento em operações que envolvem transferências de dados ("Waveform"); e uma descrição do comportamento da instrução ("Microcode RTN"), através de uma notação formal chamada *Register Transfer Notation* (RTN), a qual pode ser utilizada para descrever as microoperações de uma instrução (a sequência de transferências de dados entre registradores do processador, barramento, memória e periféricos, que compõem a instrução).

A ferramenta CPU Datasheet pode ser acessada através do menu "Help", opção "CPU Datasheet", ou pela tecla de atalho "F2".

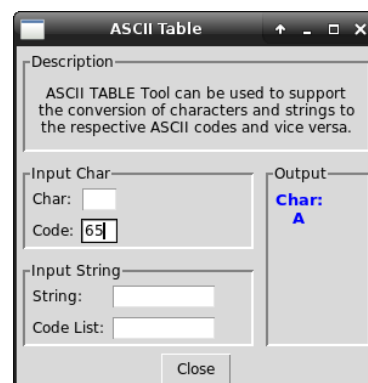
E.3 ASCII Table

A ferramenta “ASCII Table” suporta a conversão de códigos ASCII para os respectivos caracteres, bem como a conversão de códigos ASCII em caracteres. Ela está disponível no menu “Tools”, opção “ASCII Table”. A Figura E.4 ilustra o uso da ferramenta ASCII Table para conversão de um caractere e uma *string* nos respectivos códigos ASCII e vice-versa.

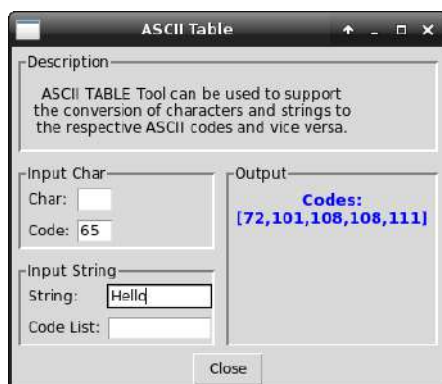
Figura E.4. Conversão de código para caracteres ASCII.



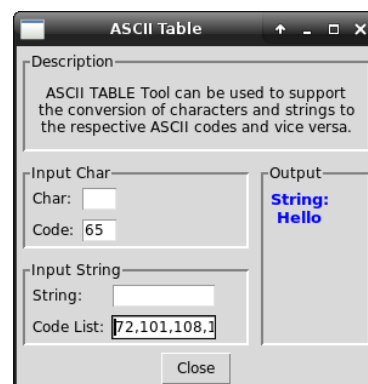
(a) Conversão de caractere para código ASCII.



(b) Conversão de código ASCII para caractere.



(c) Conversão de string para códigos ASCII.



(d) Conversão de códigos ASCII para string.

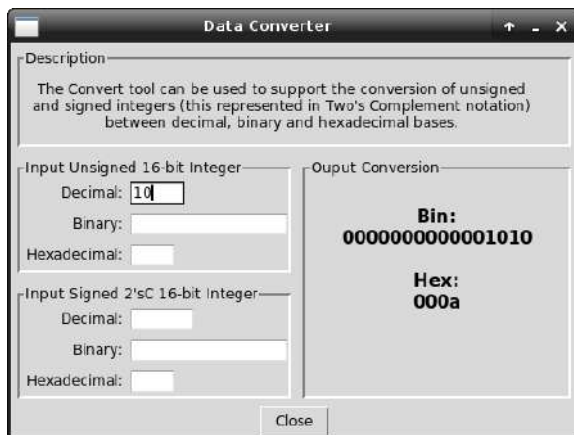
Fonte: Próprio autor (2021).

Na Figura E.4 (a), o caractere “A” está sendo convertido para “65”, que é o seu respectivo código na Tabela ASCII. Na Figura E.4 (b), o código ASCII “65” está sendo convertido para o caractere “A”. Nas Figuras E.4 (c) e (d), a *string* “Hello” está sendo convertida para a sequência de códigos ASCII “72,101,108,108,111” e vice-versa, respectivamente.

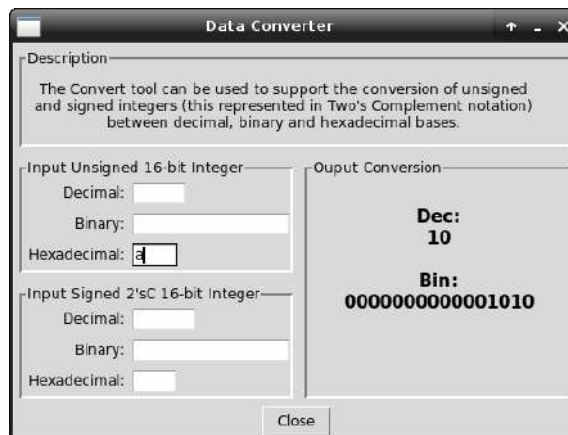
E.4. Converter

A ferramenta “Converter” é utilizada para conversão de números inteiros de 16-bits sem sinal (*unsigned int*) e com sinal (*signed int*) com representação em notação Complemento a 2) entre as bases decimal, binária e hexadecimal. A Figura E.5 ilustra a conversão de números inteiros com e sem sinal entre bases.

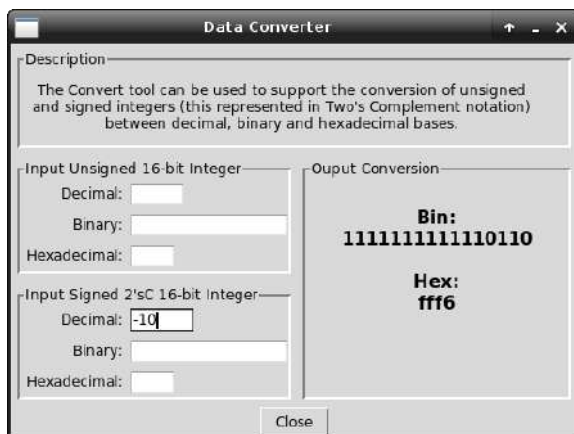
Figura E.5. Conversão de números inteiros entre bases decimal, binária e hexadecimal.



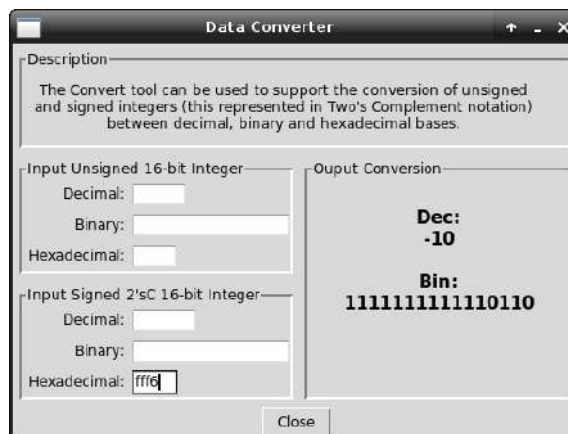
(a) Conversão de inteiro sem sinal na base decimal para as bases binária e hexadecimal.



(a) Conversão de inteiro sem sinal na base hexadecimal para as bases decimal e binária.



(c) Conversão de inteiro com sinal na base decimal para as bases binária e hexadecimal.



(a) Conversão de inteiro com sinal na base hexadecimal para as bases decimal e binária.

Fonte: Próprio autor (2021).

Na Figura E.5, em:

- (a), o inteiro sem sinal “10”, em base decimal, é convertido para as bases binária e hexadecimal;
- (b), o inteiro sem sinal “a”, em base hexadecimal, é convertido para as bases decimal e binária;

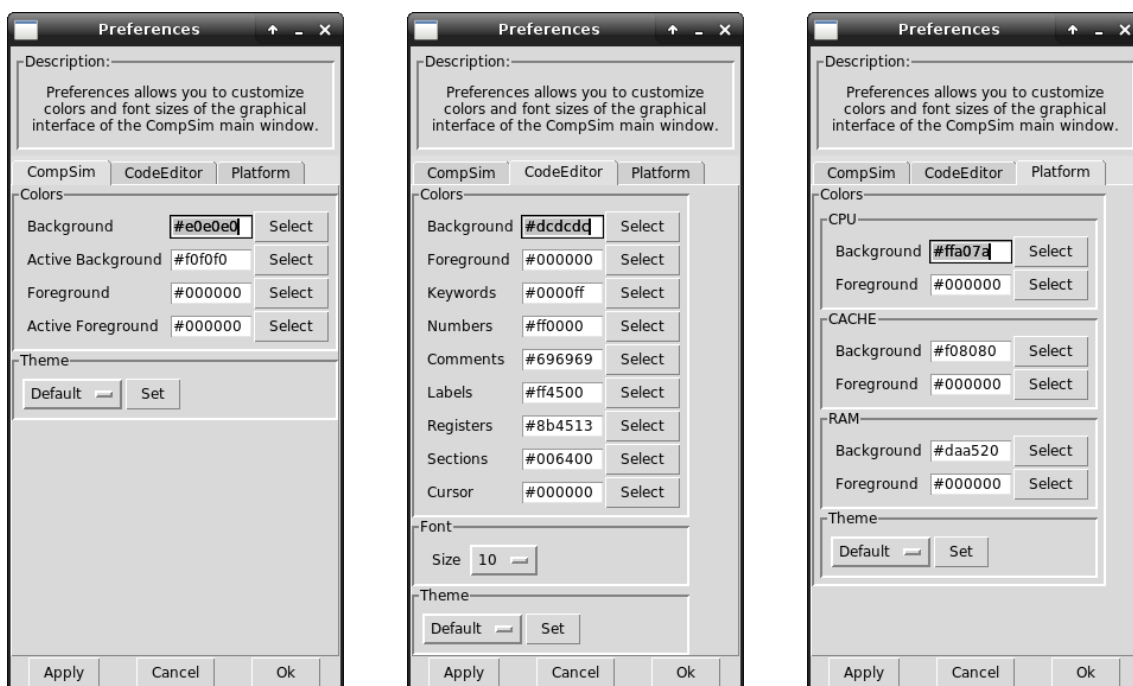
- (c), o inteiro com sinal “-10”, em base decimal é convertido para as bases binária e hexadecimal; e
- (d), o inteiro com sinal “fff6”, em base hexadecimal, é convertido para as bases decimal e binária.

O conversor de inteiros sem e com sinal entre bases está disponível no menu “Tools”, opção “Converter”.

E.5 Editor de Preferências Gráficas

O simulador CompSim traz o suporte de personalização da sua interface gráfica, através do menu “Edit”, opção “Preferences”. O “Editor de Preferências Gráficas” contém abas, que podem ser vistas nas Figuras E.6 (a), (b) e (c), que trazem controles que permitem configurar as cores dos componentes gráficos da janela principal do CompSim, do editor de código e dos componentes da plataforma de hardware virtual, respectivamente. Além disso, nas preferências do editor de código, é possível ainda escolher o tamanho da fonte de exibição dos códigos-fonte em linguagem *Assembly*, como mostra a Figura E.6 (b).

Figura E.6. Editor de Preferências Gráficas.



(a) Aba de configuração do CompSim.

(b) Aba de configuração do Editor de Código.

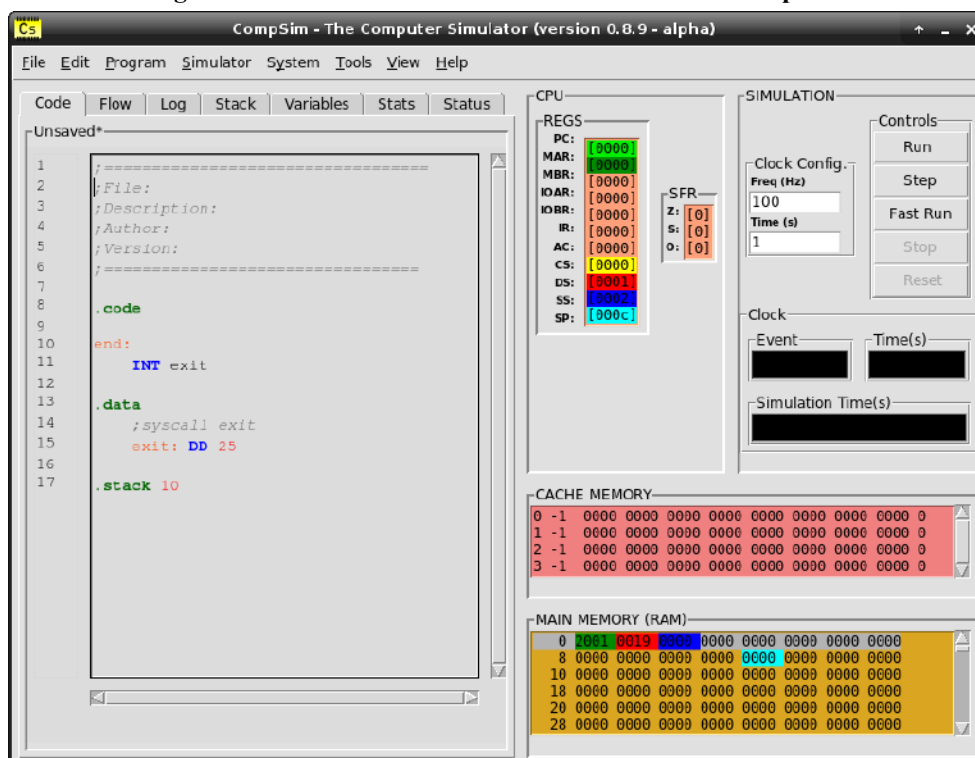
(c) Painel de configuração dos componentes da Plataforma de Hardware Virtual.

Fonte: Próprio autor (2021).

Além de permitir editar livremente os esquemas de cores, o Editor de Preferências Gráficas traz três temas de cores, que são o “Default”, “Dark” e “Matrix”, como pode ser vistos nas Figura E.7, E.8 e E.9, respectivamente.

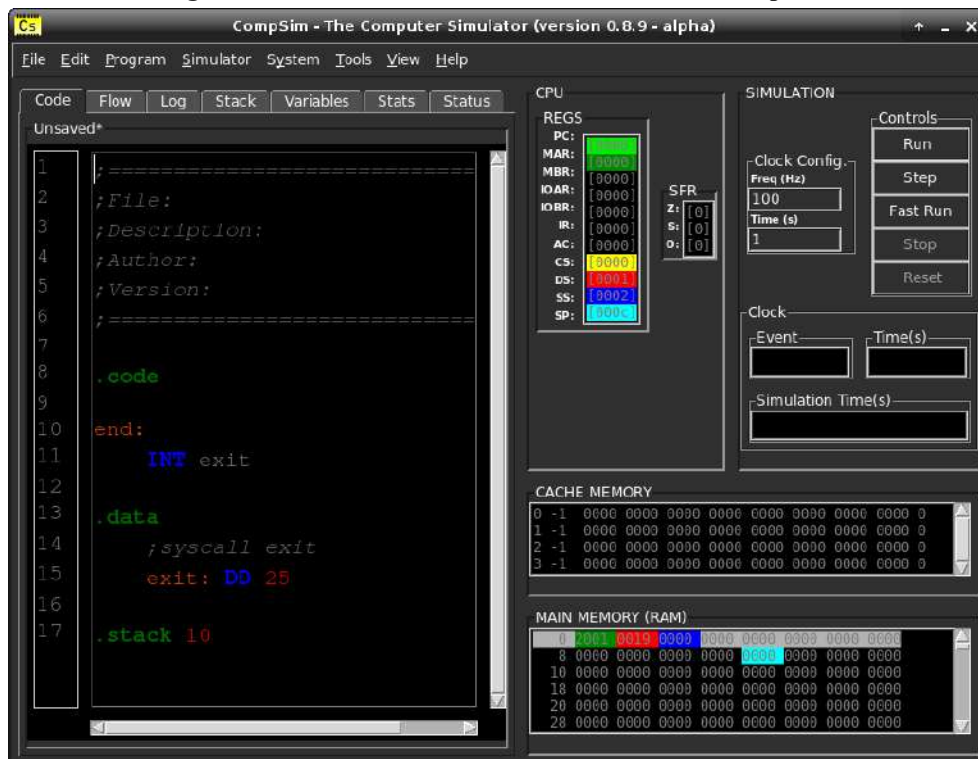
Ao editar e aplicar as preferências de cores, será criado um arquivo no diretório do simulador CompSim, chamado “CompSim.conf”. Este arquivo contém um registro das configurações aplicadas e, enquanto ele estiver no diretório do CompSim, sempre que o simulador for executado, será aplicado automaticamente a ele o esquema de cores salvo. Assim, sempre que iniciar o simulador não será necessário reconfigurar as preferências da interface gráfica. Com este recurso, por exemplo, os temas gráficos padronizados, “Default”, “Dark” e “Matrix”, podem ser modificados para compor novos temas.

Figura E.7. Tema de cores “Default” do simulador CompSim.



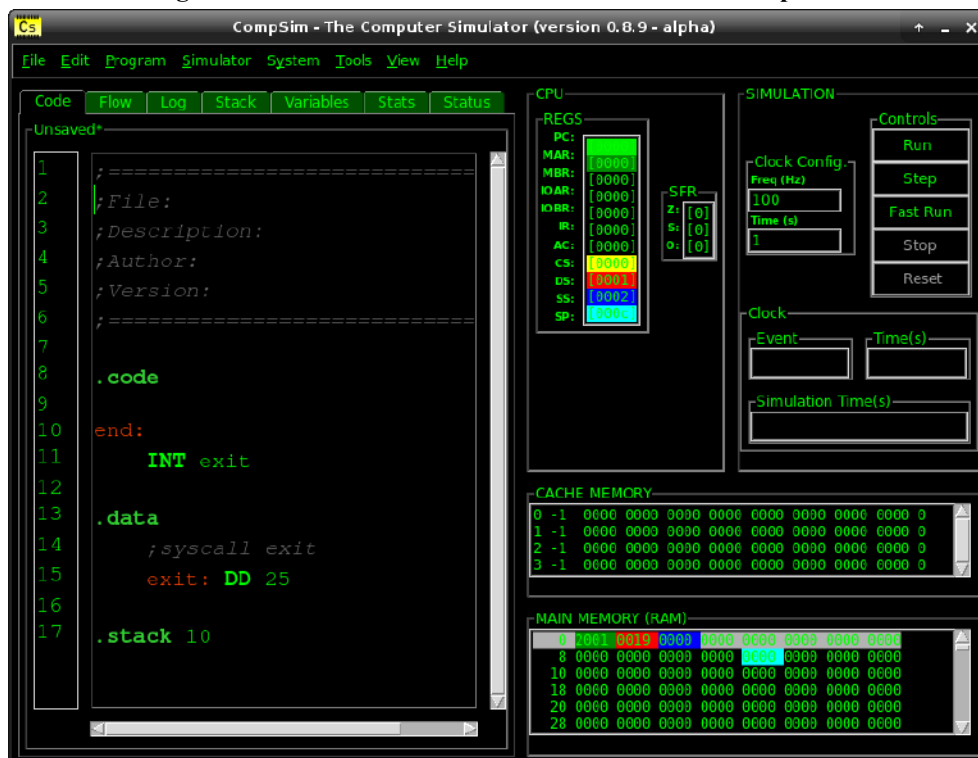
Fonte: Próprio autor (2021).

Figura E.8. Tema de cores “Dark” do simulador CompSim.



Fonte: Próprio autor (2021).

Figura E.8. Tema de cores “Matrix” do simulador CompSim.



Fonte: Próprio autor (2021).

Se você possui seu próprio esquema de cores, o que acha de compartilhá-lo, enviando uma captura de imagem e o seu arquivo “CompSim.conf” no grupo de discussão do projeto CompSim no Google Groups¹⁵ (COMPSIM, 2021b)? Se as suas preferências forem bem aceitas pela comunidade, poderá fazer parte do conjunto padrão de temas do Simulador CompSim. Já pensou que pode estar colaborando com o projeto? Esta é apenas uma das formas... No grupo de discussão, pode-se encontrar outras oportunidades de colaboração. ;-)

E.6 Emulador de Sistema para Arduino UNO

O simulador CompSim conta também com um emulador da plataforma de hardware virtual para execução em uma *board* Arduino UNO. O emulador tem como objetivo permitir a execução do sistema computacional (hardware e software), projetado no CompSim, em modo *standalone* (sem necessidade de uso do simulador) e com um maior desempenho. Com o suporte do emulador, é possível criar sistemas computacionais reais e embarcados numa *board* Arduino UNO.

Para utilizar o emulador do simulador CompSim, inicialmente é necessário criar o sistema computacional e validá-lo no simulador, através dos seguintes passos: 1) criar a plataforma de hardware virtual; 2) criar a aplicação (programa em linguagem *Assembly*); 3) simular o sistema computacional para verificar se a aplicação se comporta corretamente durante a execução na plataforma.

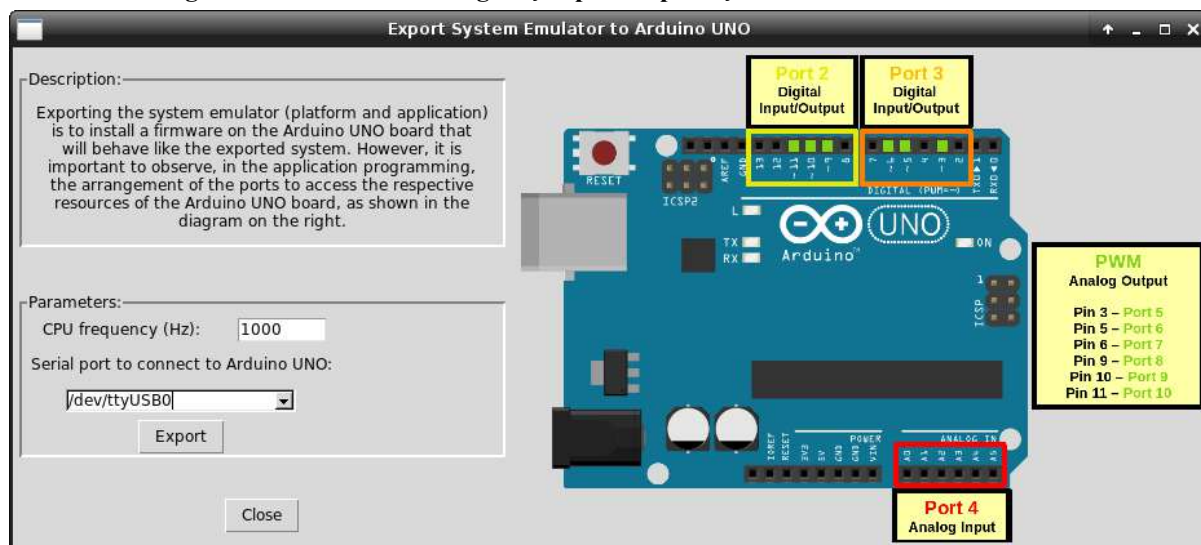
É importante ressaltar que, para o sistema computacional ser emulado numa *board* Arduino UNO, dentre os periféricos disponíveis, somente é possível utilizar o Arduino UNO (ver [Capítulo 13 - Entrada/Saída](#)). Além disso, assim como com o periférico Arduino UNO, deve-se ter a Arduino IDE¹⁶ (ARDUINO, 2021c) instalada.

Para exportar o emulador do sistema computacional simulado para uma *board* Arduino UNO, deve-se acessar o menu “System”, submenu “Export to Arduino UNO”, opção “Emulator”. Em seguida, será aberto um painel para configurações de exportação do emulador de sistema, como mostra a Figura E.10.

¹⁵ Grupo de Discussão do Projeto CompSim. Disponível em: <<https://groups.google.com/g/compsim-the-computer-simulator>>. Acesso em: 01 abr. 2021.

¹⁶ Arduino IDE. Disponível em: <<https://www.arduino.cc/en/software>>. Acesso em: 25 mai. 2021.

Figura E.10. Painel de configuração para exportação do Emulador de Sistema.



Fonte: Próprio autor (2021).

O painel ilustrado na Figura E.10 mostra, à direita, um diagrama com as configurações de portas para acesso às operações de escrita e leitura digital e analógica, caso o sistema computacional utilize o periférico Arduino UNO. Observa-se que, por padrão, o emulador utiliza os mesmos endereços de portas do periférico Arduino UNO, como pode ser visto no Quadro 12.1. À esquerda e abaixo, no painel da Figura E.10, há dois parâmetros, que devem ser configurados para exportação do emulador: 1) “CPU frequency (Hz)”: deve-se informar a frequência de operação do emulador (o número de instruções do sistema computacional emulado que serão executadas por segundo); e 2) “Serial port to connect to Arduino UNO”: deve-se informar a porta serial na qual a *board* Arduino UNO está conectada.

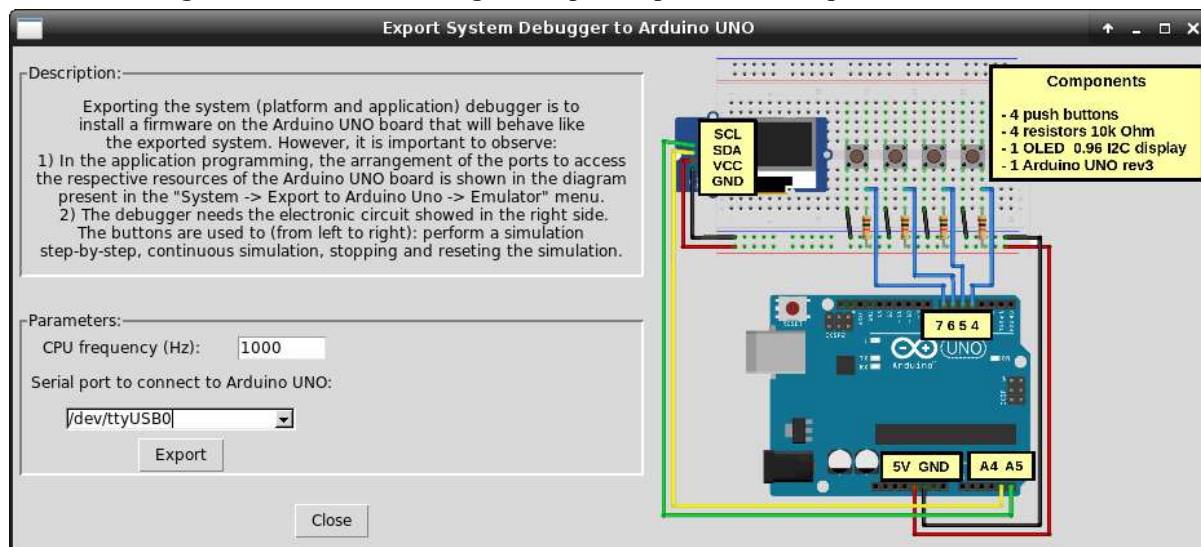
Ao clicar no botão “Export”, o emulador será exportado como um *firmware* que será gravado na memória *flash* da *board* Arduino UNO. Com isso, ao energizar a *board* Arduino UNO - via conexão em uma porta serial USB ou ligação de algum tipo de fonte de alimentação, como uma bateria de 9V, um carregador portátil (*powerbank*) ou até mesmo uma fonte de alimentação AC/DC -, será executado automaticamente o emulador do sistema exportado, sem necessidade de execução do simulador CompSim.

Além do emulador, é possível também exportar o sistema em forma de depurador (*debugger*) para uma *board* Arduino UNO, através do menu “System”, submenu “Export to Arduino UNO”, opção “Debugger”. O depurador necessita de um circuito eletrônico específico, onde é possível executar emulador em modo contínuo e passo-a-passo, parar uma simulação e resetar a simulação. Durante a execução do depurador, é possível também

visualizar o conteúdo dos registradores do processador Cariri no emulador. Para tanto, o circuito eletrônico do depurador inclui uma *board* Arduino UNO, quatro botões de pressão (*push buttons*), quatro resistores de 10K Ohm e um *display* OLED 0.96 I²C.

A Figura E.11 mostra o painel de exportação do depurador, onde pode ser visto, à direita, um diagrama esquemático com a configuração do circuito eletrônico do depurador, e, à esquerda e abaixo, os controles de exportação do depurador.

Figura E.11. Painel de configuração para exportação do Depurador de Sistema.

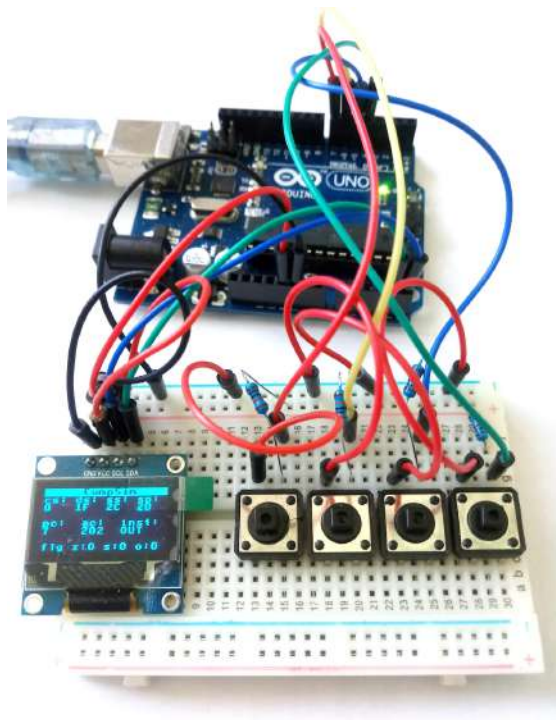


Fonte: Próprio autor (2021).

Uma vez que o sistema é exportado, para depuração em uma *board* Arduino UNO, os botões de pressão podem então ser utilizados para: executar o emulador em modo contínuo; executar o emulador em modo passo-a-passo, onde cada passo é dado à cada vez que o botão é pressionado; parar a execução do emulador; e reiniciar a execução.

Com o *display* OLED, é possível visualizar os estados dos principais registradores do processador Cariri, durante a execução do emulador. A Figura E.12 mostra o circuito eletrônico do depurador, durante a execução do emulador. No *display* OLED, é possível visualizar os conteúdos dos registradores de segmento CS, DS e SS, apontador de topo da pilha SP, contador de programa PC e acumulador AC, além da instrução em execução e das flags *Zero* (Z), *Signal* (S) e *Overflow* (O), do registrador de status SFR.

Figura E.12. Visão do circuito eletrônico do Depurador do Emulador de Sistema.



Fonte: Próprio autor (2021).

APÊNDICE F – PUBLICAÇÕES

Ao longo dos anos, através de pesquisas científicas, o Projeto CompSim vem buscando estar alinhado com as novas tendências tecnológicas em Arquitetura e Organização de Computadores e em como tornar o seu processo de ensino-aprendizado mais espontâneo, atrativo, dinâmico, motivante, desafiador e efetivo.

Assim, este apêndice lista, por categoria, as publicações científicas do Projeto CompSim. Na medida do possível, disponibilizou-se os links para acesso direto à publicação na web.

F.1 Artigos Completos Publicados em Periódicos

ESMERALDO, G. A. R. M.; BARBOSA, E.; CARTAXO, L. F.; MENDES, C. S. R. . Aprendizado Prático de Subsistemas de Entrada/Saída em Projetos de Sistemas Computacionais com Suporte do Simulador CompSim. Comunicações em Informática, v. 4, p. 15-19, 2020. Disponível em: <<https://periodicos.ufpb.br/ojs2/index.php/cei/article/view/51752>>.

ESMERALDO, G. A. R. M.; LISBOA, E. B.; MENDES, C. S. R.; CARTAXO, L. F.; RIBEIRO, C. V.; SANTOS, P. S.; MORATO, L. F. B.; NASCIMENTO, M. S. . Uma Abordagem Integrada de Hardware e Software para o Aprendizado de Subsistemas de Entrada/Saída em Projetos de Sistemas Computacionais. INTERNATIONAL JOURNAL OF COMPUTER ARCHITECTURE EDUCATION, v. 9, p. 1-9, 2020. Disponível em: <http://www2.sbc.org.br/ceacpad/ijcae/v9_n1_dec_2020/IJCAE_v9_n1_dez_2020_paper_1_vf.pdf>.

CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . Utilizando VArduino para Criação de Periféricos Virtuais baseados em Arduino UNO para o Simulador CompSim. REVISTA TECNOLOGIAS NA EDUCAÇÃO, v. 33, p. 1-17, 2020. Disponível em: <<https://tecedu.pro.br/ano12-numero-vol33-edicao-tematica-xiv/>>.

ESMERALDO, G. A. R. M.; CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B. . Um Simulador Educacional para Apoio ao Projeto de Sistemas Computacionais: Hardware, Software e suas Interfaces. Brazilian Journal of Development, v. 5, p. 11123-11133, 2019. Disponível em: <<https://www.brazilianjournals.com/index.php/BRJD/article/view/2656>>.

ESMERALDO, G. A. R. M.; MENDES, C. S. R.; CARTAXO, L. F.; LISBOA, E. B. . Apoio ao Aprendizado em Arquitetura e Organização de Computadores: Um Estudo Comparativo entre Simuladores Computacionais. REVISTA TECNOLOGIAS NA EDUCAÇÃO, v. 31, p. 1-17, 2019. Disponível em: <<https://tecedu.pro.br/ano31-numero-vol31-edicao-tematica-xii/>>.

LISBOA, E. B.; CARTAXO, L. F.; MENDES, C. S. R.; ESMERALDO, G. A. R. M. . Uma Metodologia Educacional para Aprendizado Prático de Organização e Arquitetura de Computadores com Apoio de Simulador Computacional. Brazilian Journal of Development, v. 5, p. 31062-31068, 2019. Disponível em: <<https://www.brazilianjournals.com/index.php/BRJD/article/view/5430>>.

ESMERALDO, G. A. R. M.; LISBOA, E. B. . Uma Ferramenta para Exploração do Ensino de Organização e Arquitetura de Computadores. INTERNATIONAL JOURNAL OF COMPUTER ARCHITECTURE EDUCATION, v. 6, p. 68-75, 2017. Disponível em: <http://www2.sbc.org.br/ceacpad/ijcae/v6_n1_dec_2017/ijcae_v6_n1_dec_2017.html>.

F.2 Capítulos de Livros

ESMERALDO, G. A. R. M.; PROTO, E. C. P. da S.; LISBOA, E. B.; BARROS, E. N. da S. Uma Abordagem Prática para Aprendizagem em Arquitetura e Organização de Computadores com Apoio do Simulador Computacional CompSim. Livro de Minicursos da

IV Escola Regional de Computação do Ceará, Maranhão e Piauí (ERCEMAPI 2021). **(Em fase de publicação)**.

ESMERALDO, G. A. R. M.; MENDES, C. S. R.; CARTAXO, L. F.; LISBOA, E. B. .
SIMULADORES DE ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES:
POTENCIALIDADES E DESAFIOS NO PROCESSO DE ENSINO-APRENDIZAGEM. In:
Patrício Moreira de Araújo Filho, Raimundo Luna Neres, Ernane Rosa Martins e Raimundo
José Barbosa Brandão. (Org.). Educação 4.0: Tecnologias Educacionais. 1ed. São Luís:
Editora Pascal, 2020, v. 1, p. 109-121. Disponível em:
<<https://editorapascal.com.br/2020/07/21/coletanea-educacao-4-0-editora-pascal-vol-1/>>.

ESMERALDO, G. A. R. M.; LISBOA, E. B. UM AMBIENTE VIRTUAL APLICADO AO
ENSINO E PESQUISA EM ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES.
Princípios e aplicações da computação no brasil. 1ed.: Antonella Carvalho de Oliveira, 2019,
v. 1, p. 45-49. Disponível em:
<<https://www.atenaeditora.com.br/wp-content/uploads/2019/01/E-book-Princ%C3%ADpios-e-Aplica%C3%A7%C3%B5es.pdf>>.

BARBOSA, E. L.; CARTAXO, L. F.; MENDES, C. S. R; ESMERALDO, G. A. R. M. .
AMBIENTE DE PROJETO DE HARDWARE E SOFTWARE INTEGRADOS PARA
APRENDIZADO E ENGENHARIA DE SISTEMAS COMPUTACIONAIS. Pesquisa
Científica e Inovação Tecnológica nas Engenharias 2. 1ed.: Atena Editora, 2019, v. , p.
261-272. Disponível em: <<https://www.atenaeditora.com.br/post-artigo/29131>>.

F.3 Artigos Completos Publicados em Anais de Congressos

PROTO, E. C. P. da S.; ESMERALDO, G. A. R. M.; LISBOA, E. B.; BARROS, E. N. da S.
Projetos de Sistemas Computacionais com Suporte de Simulação, de Plataformas Abertas de
Prototipação e de Hardware Reconfigurável com o CompSim. In: Workshop de Iniciação
Científica em Arquitetura de Computadores e Computação de Alto Desempenho - XXII
Simpósio em Sistemas Computacionais de Alto Desempenho (WSCAD 2021) - IEEE
XXXIII International Symposium on Computer Architecture and High Performance
Computing (SBAC-PAD 2021). **(Em fase de publicação)**.

ESMERALDO, G. A.; LISBOA, E. M.; SOUSA, M. S.; MENDES, C. S.; RIBEIRO, C. V.; MORATO, L. F.; CARTAXO, L. F.; DOS SANTOS, P. S.; DO NASCIMENTO, M. S. CompSim: An Integrated Environment for Learning and Designing of Embedded Computational Systems. In: 2020 XIV Congreso de Tecnología, Aprendizaje y Enseñanza de la Electrónica (XIV Technologies Applied to Electronics Teaching Conference) (TAEE), 2020, Porto. 2020. Disponível em: <<https://ieeexplore.ieee.org/document/9163675>>.

ESMERALDO, G. A.; LISBOA, E. B.; SOUSA, M. S.; MENDES, C. S. R.; RIBEIRO, C. V.; MORATO, L. F. B.; CARTAXO, L. F.; SANTOS, P. S.; NASCIMENTO, M. S. CompSim: Ambiente Integrado para Aprendizado e Projeto de Sistemas Computacionais Embarcados. In: Actas de lo 2020 XIV Congreso de Tecnología, Aprendizaje y Enseñanza de la Electrónica (XIV Technologies Applied to Electronics Teaching Conference) (TAEE), 2020. Disponível em: <http://taee.etsist.upm.es/components/com_repositorytaee/uploads/resources/58/2020S5BA04.pdf>.

ESMERALDO, G. A.; CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B. . VArduino: Um Componente Virtual do Arduino UNO como Dispositivo Periférico do Simulador CompSim. In: Anais do V Congresso sobre Tecnologias na Educação (Ctrl+E 2020), 2020. Disponível em: <<https://sol.sbc.org.br/index.php/ctrl/article/view/11381>>.

ESMERALDO, G. A. R. M.; LISBOA, E. B.; MENDES, C. S. R.; CARTAXO, L. F.; RIBEIRO, C. V.; MORATO, L. F. B.; SANTOS, P. S.; NASCIMENTO, M. S. . Uma Abordagem Integrada de Hardware e Software para o Aprendizado de Subsistemas de Entrada/Saída em Projetos de Sistemas Computacionais. In: Workshop sobre Educação em Arquitetura de Computadores (WEAC) - XXI Simpósio em Sistemas Computacionais de Alto Desempenho (WSCAD), 2020. Disponível em: <https://sol.sbc.org.br/index.php/wscad_estendido/article/view/14098>.

ESMERALDO, G. A. R. M.; MENDES, C. S. R.; CARTAXO, L. F.; LISBOA, E. B. . Um Estudo Comparativo entre Simuladores Computacionais para Apoio à Disciplina de Arquitetura e Organização de Computadores. In: Anais do IV Congresso sobre Tecnologias

na Educação (Ctrl+E 2019), 2019. Disponível em:
<<https://sol.sbc.org.br/index.php/ctrlE/article/view/8915>>.

ESMERALDO, G. A. R. M.; CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B. . Um Simulador Educacional para Apoio ao Projeto de Sistemas Computacionais: Hardware, Software e suas Interfaces. In: Anais do XXVI Workshop sobre Educação em Computação (WEI 2018) - XXXVIII Congresso da Sociedade Brasileira de Computação (CSBC 2018), 2018. Disponível em: <<https://sol.sbc.org.br/index.php/wei/article/download/9881/9770/>>.

LISBOA, E. B.; CARTAXO, L. F.; MENDES, C. S. R.; ESMERALDO, G. A. R. M. . Ambiente Integrado de Hardware e Software Aplicado ao Ensino de Projeto de Sistemas Computacionais. In: Anais do III Congresso sobre Tecnologias na Educação (Ctrl+E 2018), 2018. Disponível em: <http://ceur-ws.org/Vol-2185/CtrlE_2018_paper_24.pdf>.

ESMERALDO, G. A. R. M.; LISBOA, E. B. . Uma Ferramenta para Exploração do Ensino de Organização e Arquitetura de Computadores. In: Anais do Workshop sobre Educação em Arquitetura de Computadores (WEAC). XVIII Simpósio em Sistemas Computacionais de Alto Desempenho (WSCAD 2017), 2017.

ESMERALDO, G. A. R. M.; LISBOA, E. B. . Um Ambiente Virtual Aplicado ao Ensino e Pesquisa em Arquitetura e Organização de Computadores. In: IV Encontro Nacional de Computação dos Institutos Federais (EnCompIF 2017). Anais do XXXVII Congresso da Sociedade Brasileira de Computação (CSBC 2017), 2017. Disponível em: <<https://sol.sbc.org.br/index.php/encompif/article/view/9941>>.

ESMERALDO, G. A. R. M.; LISBOA, E. B. . CompSim: Um Ambiente para o Ensino Integrado de Arquitetura e Organização de Computadores. In: Anais do II Congresso sobre Tecnologias na Educação (Ctrl+E 2017), 2017. Disponível em: <http://ceur-ws.org/Vol-1877/CtrlE2017_MOA_07_117.pdf>.

F.4 Resumos Expandidos Publicados em Anais de Congressos

SILVA, E. L. F.; DOS SANTOS, M. A.; RIBEIRO, C. V; ALMEIDA, L. de M.; NETO, O. M. de A.; DA SILVA, J. P. F.; ESMERALDO, G. A. R. M.; LISBOA, E. B. Instrumentando o Simulador CompSim com um Web Help. In: XVIII Semana Nacional de Ciência e Tecnologia (SNCT/IFS 2021) do Instituto Federal de Sergipe (IFS), 2021. **(Em fase de publicação)**.

PROTO, E. C. P. da S.; ESMERALDO, G. A. R. M. Modelagem Estrutural de Plataforma de Hardware Virtual em Hardware Reconfigurável para Apoio ao Aprendizado e Projetos de Sistemas Computacionais. In: Encontro de Iniciação Científica e Tecnológica do IFCE, 2021. **(Em fase de publicação)**.

RIBEIRO, C. V.; MORATO, L. F. B.; JESUS, M. D. C.; NASCIMENTO, M. S.; SANTOS, P. S.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . Integração da Simulação Computacional a Dispositivos Reconfiguráveis para Apoiar o Ensino de Sistemas Complexos. In: Anais da XVII Semana Nacional da Ciência e Tecnologia (SNCT/IFS 2020) do Instituto Federal de Sergipe (IFS), 2020. Disponível em: <http://www.ifs.edu.br/images/propex/SNCT_2020/ANAIS_SNCT_2020-compactado.pdf>.

MENDES, C. S. R.; CARTAXO, L. F.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . Uma Abordagem Integrada de Hardware e Software para o Ensino de Organização e Arquitetura de Computadores. In: XXIX Simpósio Brasileiro de Informática na Educação (SBIE 2018) - VII Congresso Brasileiro de Informática na Educação (CBIE 2018), 2018, Fortaleza. Anais do XXIX Simpósio Brasileiro de Informática na Educação (SBIE 2018), 2018. Disponível em: <<http://br-ie.org/pub/index.php/sbie/article/view/8133>>.

LISBOA, E. B.; ESMERALDO, G. A. R. M. . Um Ambiente Integrado para o Estudo e Projeto de Sistemas Computacionais Baseados em Plataformas. In: III Fórum de Educação em Engenharia da Computação (FEEC 2017). XXX Symposium on Integrated Circuits and Systems Design (SBCCI 2017). Chip on the Sands, 2017, Fortaleza. Anais do XXX Symposium on Integrated Circuits and Systems Design (SBCCI 2017). Chip on the Sands, 2017. Disponível em: <<https://chiponthesands.sbmicro.org.br/feec-2017/technical-program>>.

MENDES, C. S. R.; CARTAXO, L. F.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . COMPSIM: Um Simulador para Ensino de Organização e Arquitetura de Computadores. In: VI Semana de Iniciação Científica do Instituto Federal de Educação, Ciência e Tecnologia do Ceará campus Crato (SEMIC 2017), 2017, Crato. Anais da VI Semana de Iniciação Científica do Instituto Federal de Educação, Ciência e Tecnologia do Ceará campus Crato (SEMIC 2017), 2017. Disponível em: <<http://semic.crato.ifce.edu.br/anais/anais2017.pdf>>.

F.5 Resumos Publicados em Anais de Congressos

CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . LEARNING IN THE DESIGN OF COMPUTATIONAL SYSTEMS WITH SUPPORT OF A SIMULATION ENVIRONMENT INTEGRATED TO CONFIGURABLE HARDWARE PLATFORM. In: I Congresso Internacional Virtual de Pesquisa, Pós-graduação e Inovação do IFCE, 2020, Fortaleza. Anais do I Congresso Internacional Virtual de Pesquisa, Pós-graduação e Inovação do IFCE, 2020.

CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . INTEGRATION OF A RECONFIGURABLE HARDWARE PLATFORM INTO A SIMULATION ENVIRONMENT TO SUPPORT LEARNING IN COMPUTATIONAL SYSTEMS PROJECTS. In: I Congresso Internacional Virtual de Pesquisa, Pós-graduação e Inovação do IFCE, 2020, Fortaleza. Anais do I Congresso Internacional Virtual de Pesquisa, Pós-graduação e Inovação do IFCE, 2020.

CARTAXO, L. F.; MENDES, C. S. R.; BARBOSA, E.; ESMERALDO, G. A. R. M. . Componente Virtual para Simulação do Arduino UNO como Periférico do Simulador CompSim. In: III Workshop de Ciências Exatas Aplicadas, 2019, Crato. Anais do III Workshop de Ciências Exatas Aplicadas (III WCEA). Crato, 2019. v. 1. Disponível em: <https://wcea.crato.ifce.edu.br/3wcea/3wcea_resumo1.pdf>.

SANTOS, G. A. S.; ESMERALDO, G. A. R. M.; LISBOA, E. B. . CompSim e Eletrônica Digital. In: III Workshop de Ciências Exatas Aplicadas (III WCEA), 2019, Crato. Anais do III Workshop de Ciências Exatas Aplicadas (III WCEA). Crato, 2019. v. 1. Disponível em: <https://wcea.crato.ifce.edu.br/3wcea/3wcea_resumo2.pdf>.

Apêndice F - Publicações

MORATO, L. F. B.; SANTOS, P. S.; NASCIMENTO, M. S.; RIBEIRO, C. V.; ESMERALDO, G. A. R. M.; LISBOA, E. B. . Integração de Hardware e Software para Implementação de Funções Booleanas. In: III Workshop de Ciências Exatas Aplicadas (III WCEA), 2019, Crato. Anais do III Workshop de Ciências Exatas Aplicadas (III WCEA). Crato, 2019. v. 1. Disponível em: <https://wcea.crato.ifce.edu.br/3wcea/3wcea_resumo4.pdf>.

CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . Um Emulador Embarcado para Projetos de Sistemas Computacionais Físicos no CompSim. In: III Workshop de Ciências Exatas Aplicadas (III WCEA), 2019, Crato. Anais do III Workshop de Ciências Exatas Aplicadas (III WCEA). Crato, 2019. v. 1. Disponível em: <https://wcea.crato.ifce.edu.br/3wcea/3wcea_resumo8.pdf>.

CARTAXO, L. F.; ESMERALDO, G. A. R. M. . Um Estudo sobre Gamificação Aplicado ao Ensino-Aprendizado de Arquitetura e Organização de Computadores. In: III Workshop de Ciências Exatas Aplicadas (III WCEA), 2019, Crato. Anais do III Workshop de Ciências Exatas Aplicadas (III WCEA). Crato, 2019. v. 1. Disponível em: <https://wcea.crato.ifce.edu.br/3wcea/3wcea_resumo9.pdf>.

CARTAXO, L. F.; MENDES, C. S. R.; LISBOA, E. B.; ESMERALDO, G. A. R. M. . Uma Interface para Conexão de Periféricos Virtuais Modulares no Simulador CompSim. In: III Workshop de Ciências Exatas Aplicadas (III WCEA), 2019, Crato. Anais do III Workshop de Ciências Exatas Aplicadas (III WCEA). Crato, 2019. v. 1. Disponível em: <https://wcea.crato.ifce.edu.br/3wcea/3wcea_resumo13.pdf>.

MENDES, C. S. R.; CARTAXO, L. F.; LISBOA, E. B.; ESMERALDO, G. . Compilador C para Aprendizado em Programação e em Compiladores no CompSim. In: VII Seminário de Iniciação Científica e Tecnológica do IFCE campus Crato (SEMIC 2018), 2018, Crato. Anais do VII Seminário de Iniciação Científica e Tecnológica do IFCE campus Crato (SEMIC 2018), 2018. v. 1. Disponível em: <<http://semic.crato.ifce.edu.br/anais/anais2018.pdf>>.

SANTOS, S. M.; SANTOS FILHO, S. V.; ESMERALDO, G.; LISBOA, E. B. . Integração do Simulador CompSim com Dispositivos Eletrônicos Reconfiguráveis. In: II Workshop de

Apêndice F - Publicações

Ciências Exatas Aplicadas (WCEA 2018), 2018, Crato. Anais do II Workshop de Ciências Exatas Aplicadas (WCEA 2018), 2018. Disponível em: <https://wcea.crato.ifce.edu.br/2wcea/2wcea_resumo6.pdf>.

MENDES, C. S. R.; CARTAXO, L. F.; LISBOA, E. B.; ESMERALDO, G. . Um Compilador C para Projetos Embarcados no CompSim. In: II Workshop de Ciências Exatas Aplicadas (WCEA 2018), 2018, Crato. Anais do II Workshop de Ciências Exatas Aplicadas (WCEA 2018), 2018. Disponível em: <https://wcea.crato.ifce.edu.br/2wcea/2wcea_resumo13.pdf>.

ESMERALDO, G. A. R. M.; LISBOA, E. B. . Um Ambiente Integrado para o Estudo e Projeto de Sistemas Computacionais. In: I Workshop de Ciências Exatas Aplicadas (I WCEA), 2017, Crato. Anais do I Workshop de Ciências Exatas Aplicadas (I WCEA), 2017. Disponível em: <https://wcea.crato.ifce.edu.br/1wcea/1wcea_resumo7.pdf>

TERMOS PARA BUSCA

A

[Ábaco](#)

[Abordagem Metodológica](#)

[Ada Lovelace](#)

[ADC](#)

[AI Accelerator](#)

[Alan Turing](#)

[Álgebra Booleana](#)

[Alocação de Memória](#)

[Alocação Dinâmica de Memória](#)

[APU](#)

[Arduino](#)

[Array/Vetor](#)

[Arquitetura de Computadores](#)

[ASCII](#)

[Autômato](#)

B

[Barramento](#)

[BSS](#)

C

[Calculadora Mecânica](#)

[Caminho de Dados](#)

[Charles Babbage](#)

[Chip](#)

[Ciclo de Instrução](#)

[Circuito Integrado](#)

[CISC](#)

[CMOS](#)

[Código de Operação \(Opcode\)](#)

[Coerência de Cache](#)

[Compatibilidade de Código Binário](#)

[Componentes do Computador](#)

[Computação Quântica](#)

[Computador](#)

[Computador Biológico](#)

[Computador de Propósito Geral](#)

[Computador Eletromecânico](#)

[Computador Óptico](#)

[CompSim](#)

[Conjunto de Instruções da Arquitetura \(ISA\)](#)

[Controle de Fluxo de Execução](#)

D

[DSP](#)

E

[Endianess](#)

[ENIAC](#)

[Entrada/Saída Programada com Espera Ocupada](#)

[Entrada/Saída Mapeada em Memória](#)

[Entrada/Saída Mapeada em Porta](#)

[Entrada/Saída por Acesso Direto à Memória \(DMA\)](#)

[Entrada/Saída por Interrupção](#)

[Estruturas de Controle de Fluxo de Execução](#)

[Expansão de Códigos de Operação](#)

F

[Fetch](#)

[Firmware](#)

[Flip-Flop](#)

[FPGA](#)

[Funções do Computador](#)

G

[GPU](#)

[GPGPU](#)

H

[Hierarquia de Memórias Cache](#)

[Heap](#)

I

[IEEE 754](#)

[Internet das Coisas](#)

[IoT Processor](#)

L

[Lei de Moore](#)

[Linguagem de Montagem \(Assembly\)](#)

[Localidade Espacial e Temporal](#)

M

[Máquina Analítica](#)

[Máquina de Turing](#)

[Mapeamento de Memória Cache](#)

[MDPS](#)

[Memória Principal \(RAM\)](#)

[Memória Cache](#)

[Microprocessador](#)

[Modelo de Memória de um Programa](#)

[Modelo de von Neumann](#)

[Modo de Endereçamento](#)

[Modularização de Programas](#)

[Montador \(Assembler\)](#)

[MPPA](#)

N

[Notação Complemento a 1](#)

[Notação Complemento a 2](#)

[Notação Sinal Magnitude](#)

O

[Organização de Computadores](#)

[Otimização de Desempenho](#)

[Overflow](#)

P

[Paralelismo em Nível de Instrução](#)

[Paralelismo ao Nível de Tarefa](#)

[Pilha do Programa \(Stack\)](#)

[Pipeline](#)

[Plataforma Mandacaru](#)

[Política de Escrita em Memória Cache](#)

[Ponteiro](#)

[Porta](#)

[Porta Lógica](#)

[Porta Lógica Universal](#)

[Processador \(CPU\)](#)

[Processador Cariri](#)

[Produto dos Maxtermos](#)

[Projeto Baseado em Plataformas](#)

[Pseudo-Instrução](#)

[PWM](#)

Q

[QPU](#)

[Qubit](#)

R

[Registrador](#)

[Relé Eletromecânico](#)

[Relocação de Memória](#)

[RISC](#)

[RTN](#)

S

[Segmento de Memória](#)

[Sequência de Fibonacci](#)

[Simulação com CompSim](#)

[Simuladores Computacionais](#)

[Soma dos Mintermos](#)

[Subsistema de Entrada/Saída](#)

[Substituição de Blocos na Memória Cache](#)

[Superescalar](#)

[Supremacia Quântica](#)

T

[Taxas de Acerto e Falta \(Hit/Miss\)](#)

[Taxonomia de Flynn](#)

[Thread](#)

[Tipos de Instrução do Processador](#)

[Tipos de Operandos do Processador](#)

[Transistor](#)

U

[Unidade de Controle \(UC\)](#)

[Unidade Lógica e Aritmética \(ULA\)](#)

V

[Válvula Eletrônica](#)

[Vara de Contagem](#)

[VPU](#)

W

[Waveform](#)

X

[x86](#)

FUNDAMENTOS E PRÁTICAS DE ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Estudos de Caso com o Simulador CompSim

Arquitetura e Organização de Computadores (AOC) é uma disciplina, presente em cursos nas áreas de computação e de eletrônica, que trata dos aspectos de projeto, operação e programação de sistemas computacionais. Ela é considerada uma das mais difíceis, pois, além de incluir conteúdos complexos e extensos, que devem estar alinhados às novas tendências tecnológicas, necessita de laboratórios especializados de hardware, para que os estudantes possam ter os primeiros contatos com os componentes físicos do computador e desenvolvam as habilidades necessárias ao exercício da profissão.

Este livro traz uma abordagem de aprendizado aplicado, na qual os conceitos básicos de AOC são introduzidos através de práticas por simulação. Neste livro, adotou-se o simulador CompSim como estudo de caso, por:

- Incluir um ambiente gráfico integrado e completo que simplifica e dinamiza o projeto, programação, simulação, visualização e análise de desempenho de sistemas computacionais virtuais;
- Incluir componentes virtuais de simulação com as características dos respectivos componentes de computadores reais;
- Permitir a sua integração com hardware físico, provendo assim simulações com interação com componentes eletrônicos reais; e
- Possibilitar o desenvolvimento experiências práticas, de forma assíncrona e com alto desempenho, com cenários virtuais que se assemelham aos reais e presentes no estado da arte.

Apesar de ser um livro introdutório, ele contém uma rica coleção de exemplos práticos que têm por objetivo ilustrar e reforçar a compreensão dos conceitos e das técnicas de projetos de sistemas computacionais. Os exemplos são bastante didáticos e simples de realizar, entretanto não deixam de refletir a aplicação de técnicas emergentes de projeto utilizadas em computadores complexos, de grande porte e, até mesmo, nos embarcados.

Desta forma, Fundamentos e Práticas em Arquitetura e Organização de Computadores - Estudos de Caso com o Simulador CompSim busca oferecer uma fundação sólida de aprendizado para estudantes e novos projetistas de sistemas computacionais, através de uma abordagem metodológica aplicada de aprendizado e projeto mais espontânea, atrativa, dinâmica, motivante, desafiadora e efetiva.

